

台灣自編大學教科書
@prof

大數據分析 與雲端運算

Big Data Analytics and Cloud Computing

全九篇 三十八章

機器人叫獸（李世淵） 編著

關於本書

《大數據分析與雲端運算》為台灣自編大學教科書系列之一，全書共九篇三十八章，涵蓋大數據與雲端運算的完整技術堆疊——自分散式系統原理、批次與串流運算框架、大規模探勘演算法、生成式人工智慧，到雲端平台、資料治理與整合應用。

本書適用對象為大學智慧機器人學程、智慧製造學程之學生，亦可作為高中人工智慧與機器人社團課程之進階讀本，或產業界工程人員的自學參考。

各章體例一致，均含：叫獸開講（情境導引）、學習目標、核心概念表、正文六至七節、公式與流程（含完整計算示範）、案例研討四則、實作活動三項、延伸閱讀、本章小結、以及習題與解答提示。全書所有數值範例均經程式驗算。

編著者

李世淵（機器人叫獸）

教學助理

李天宇、李宇晴

聯絡與教學資源

LINE 官方帳號 @prof

序言

我開始寫這本書，是因為一個在課堂上反覆出現的落差。

每學期我都會問學生一個問題：「如果現在給你十億筆資料，你會怎麼做？」多數人的回答，是把資料結構與演算法課上學到的方法直接放大——用一個更大的陣列、寫一個更快的迴圈、換一台更好的電腦。這個直覺完全正常，卻也正是這本書要處理的核心問題：當規模跨過某個門檻，事情不是「變得更難」，而是「變得不同」。

單機時代的許多前提，在大規模的世界裡會一一失效。機器不再是可靠的，而是「一定會壞」的；資料不再能被完整掃描，而必須以近似換取可行；精確的答案不再總是最好的答案，因為一個要算三天的正確結果，往往不如一個一秒算完、誤差百分之二的估計來得有用。這些觀念的轉換，比任何一項具體技術都更值得學。

因此，本書雖然涵蓋了相當多的技術——從 HDFS、Spark、Kafka 到 Kubernetes 與大語言模型——但我真正希望讀者帶走的，是貫穿全書的三條主線。

其一，規模改變一切。

當資料大到單機處理不了，儲存必須分散、運算必須平行、容錯必須內建、而精確必須讓位給近似。這是全書最根本的前提，也是第 3 章那八個「分散式運算的誤解」之所以重要的原因。

其二，取捨無所不在。

一致性與可用性、批次與串流、精確與即時、雲端與地端、彈性與可靠——本書幾乎每一章都在談某種取捨。而工程判斷的核心，從來不是尋找「最好的技術」，而是辨識「這個情境的代價結構是什麼」。同樣是系統故障，資料系統的正解是「盡量維持服務」，安全系統的正解卻是「立刻停止」——技術原則不是普世的教條。

其三，原理會遷移，工具不會。

讀者會在書中一再看到同一組原理以不同面貌出現：分區、複製、冪等、不可變、以摘要換取可行。第 11 章 RDD 的不可變性、第 25 章 Kafka 的追加式日誌、第 28 章容器映像檔的分層、第 30 章 Lakehouse 的中繼資料清單，骨子裡是同一個設計哲學。技術每幾年就換一輪，但這些原理不會。理解它們，面對任何新技術都能快速上手。

本書的體例也反映了這個立場。每章開頭的「叫獸開講」不是裝飾，而是刻意從一個具體的困境切入，因為技術的意義只有在問題的脈絡中才看得清楚；每章的「公式與流程」都附上完整的計算，因為我希望讀者不只知道「有這個方法」，還能真的算出那個數字；而四則「案例研討」則多半取材自實際發生過的失敗——因為失敗往往比成功更能說明原理。

最後我想說的是，這本書談的許多能力，將來會影響到許多看不見的人。推薦系統會塑造他們看到的世界，模型會影響對他們的判斷，資料會揭露他們未曾同意揭露的事。技術能力帶來的不只是價值，也帶來相應的責任。這是在第 19、33 章特別著墨、也希望讀者始終記得的一件事。

願各位讀完之後，不只多會了幾項工具，更多了一種看問題的方式。

李世淵（機器人叫獸）

目錄

第壹篇 導論與基礎

第 1 章	大數據與雲端運算導論.....	1
第 2 章	資料科學與資料工程生命週期.....	17
第 3 章	分散式系統原理.....	29
第 4 章	資料模型、編碼與儲存基礎.....	41

第貳篇 分散式儲存與資料庫

第 5 章	分散式檔案系統與物件儲存.....	53
第 6 章	NoSQL 資料庫與資料建模.....	64
第 7 章	資料複製、分區與一致性.....	75
第 8 章	分散式交易與共識演算法.....	86

第參篇 批次運算框架

第 9 章	MapReduce 程式設計模型.....	98
第 10 章	叢集資源管理與排程.....	109
第 11 章	Apache Spark 核心.....	122
第 12 章	Spark SQL、DataFrame 與 Dataset.....	134
第 13 章	批次資料管線與 ETL／ELT.....	146

第肆篇 大數據探勘與分析演算法

第 14 章	相似性探勘：MinHash 與 LSH.....	158
第 15 章	頻繁項集與關聯規則探勘.....	170
第 16 章	大規模分群演算法.....	183
第 17 章	降維與矩陣分解.....	196
第 18 章	連結分析與大規模圖處理.....	208
第 19 章	推薦系統.....	221
第 20 章	分散式機器學習與大規模模型訓練.....	235

第伍篇 生成式 AI 與開源生態

第 21 章	大語言模型（LLM）.....	248
第 22 章	AI 代理（AI Agents）.....	261
第 23 章	開源協作與模型生態系：.....	274

第陸篇 串流處理與即時分析

第 24 章	資料串流模型與近似演算法.....	288
第 25 章	訊息佇列與事件驅動架構.....	301
第 26 章	串流處理引擎.....	314

第柒篇 雲端運算平台與架構

第 27 章	雲端運算概論與服務模型.....	327
--------	------------------	-----

第 28 章	虛擬化與容器技術.....	339
第 29 章	容器編排與 Kubernetes.....	353
第 30 章	雲端資料倉儲、資料湖.....	365
第 31 章	無伺服器運算與雲原生架構.....	379

第捌篇 資料治理、安全與維運

第 32 章	資料治理與資料品質.....	392
第 33 章	資料安全、隱私與法規遵循.....	406
第 34 章	DataOps 與 MLOps.....	420

第玖篇 整合實務與應用

第 35 章	邊緣運算與物聯網.....	433
第 36 章	機器人與智慧製造整合.....	447
第 37 章	資料視覺化與決策支援.....	461
第 38 章	專案實作：	474

台灣自編大學教科書
@prof
大數據分析與雲端運算

第壹篇 導論與基礎

Foundations

第 1 章

大數據與雲端運算導論

Introduction to Big Data and Cloud Computing

機器人叫獸（李世淵） 編著

叫獸開講

一九九八年，兩個史丹佛大學的博士班學生想做一件在當時聽起來近乎狂妄的事：把整個網際網路抓回來，放進自己的機器裡，然後幫全世界的網頁排名。他們沒有錢買昂貴的大型主機，只能到處撿拾便宜的個人電腦，堆在宿舍與車庫裡，用網路線把它們串起來。

問題很快就來了。當你手上有一千台廉價電腦，「每天都會有幾台壞掉」不是意外，而是常態。硬碟會壞、電源會燒、網路線會被踢掉。傳統的想法是「想辦法讓機器不要壞」，但他們選了另一條路：假設機器一定會壞，那就讓軟體有辦法在機器壞掉時，自己把工作接下去做完。

這個轉念，後來變成了 Google 的兩篇論文：二〇〇三年的 GFS 分散式檔案系統，與二〇〇四年的 MapReduce 運算模型。這兩篇論文被一位工程師讀到，照著做出了開源版本，取名叫 Hadoop——名字來自他兒子的一隻黃色玩具象。大數據時代，就從這隻玩具象開始了。

同學，我要你記住這段故事的重點，不是誰發明了什麼，而是那個轉念：當規模大到某個程度，「把單一機器變強」這條路會走到盡頭，我們必須改成「把很多台普通機器組織起來」。這一整本書要教你的，就是這件事怎麼做——以及當這些機器不在你的機房、而在雲端時，事情會變成什麼樣子。

學習目標

研讀完本章之後，你應該能夠：

1. 說明大數據的定義，並以 4V／5V 特性描述一組實際資料的性質。
2. 區分資料、資訊、知識與智慧，並說明四種資料分析層次的差異。
3. 說明雲端運算的五大特徵，並比較 IaaS、PaaS 與 SaaS 三種服務模型。
4. 解釋大數據與雲端運算為何相互需要，並說明兩者匯流的技術動因。
5. 描繪大數據技術生態系的分層架構，並指出各層的代表性技術。
6. 說明本書的整體架構，並完成後續章節所需之學習環境準備。

核心概念

核心概念	說明
大數據 (Big Data)	資料的規模、產生速度或型態複雜度，超出單一機器與傳統工具在合理時間內處理的能力。
4V／5V 特性	量 (Volume)、速 (Velocity)、多樣 (Variety)、真實 (Veracity) 與價值 (Value)，描述大數據的五個面向。
水平擴充 (Scale-out)	以增加機器數量取代提升單機規格，是大數據系統的基本擴充策略。
雲端運算 (Cloud Computing)	透過網路隨需取得可組態之共享運算資源，並依使用量計費的運算模式。
服務模型	IaaS、PaaS、SaaS 三種抽象層次，決定使用者與供應商之間的責任分界。
技術生態系	由擷取、儲存、運算、分析到應用等分層技術所構成的整體堆疊。

表 1-1 本章核心概念一覽

1.1 大數據的定義與 4V／5V 特性

1.1.1 什麼樣的資料才算「大」

許多人第一次聽到「大數據」，直覺會問：「到底要多大才算大？一 TB 算不算？」這個問題其實沒有一個固定的數字答案，因為「大」是相對的概念。一份三百 GB 的資料，放在你的筆記型電腦上處理會非常吃力，但放進一座擁有數百個節點的叢集，可能幾分鐘就跑完了。

因此，在學術與工業界較被接受的操作型定義是：

大數據，是指其資料量、產生速度或結構複雜度，已經超出單一機器與傳統資料處理工具（如關聯式資料庫、試算表）在可接受的時間與成本內完成處理的資料集合。

這個定義的巧妙之處在於，它把重點從「絕對數字」轉移到「處理能力的邊界」。換句話說，當你發現手上的工具已經無能為力，必須改用分散式的方法來處理時，你面對的就是大數據問題。這也正是本書存在的理由：教你在跨過那條界線之後該怎麼辦。

1.1.2 資料的量級感

要培養對「大」的直覺，先建立量級感是有幫助的。表 1-2 列出常見的資料單位與生活化的對照。

單位	大小	生活對照
MB（百萬位元組）	10^6 位元組	一張手機拍攝的照片約 3~5 MB。
GB（十億位元組）	10^9 位元組	一部高畫質電影約 4~8 GB。
TB（兆位元組）	10^{12} 位元組	一台個人電腦的硬碟容量，約可存放二十萬張照片。
PB（千兆位元組）	10^{15} 位元組	大型社群平台每日新增的資料量級。
EB（百京位元組）	10^{18} 位元組	全球網際網路每日流量的量級。
ZB（十垓位元組）	10^{21} 位元組	全球資料總量的年度量級。

表 1-2 資料單位與生活化對照

值得注意的是，這些量級每隔幾年就會往上推移一階。一九九〇年代的「海量資料」，以今天的標準來看不過是一個隨身碟的容量。所以，與其記住某個門檻數字，不如記住那個判準：你的工具是否已經到達極限。

1.1.3 4V 與 5V 特性

業界最常用來刻畫大數據性質的架構，是所謂的 4V。近年多數教材再加上第五個 V，成為 5V。

特性	中文	內涵與範例
Volume	資料量	資料的規模龐大。例如一條產線上百個感測器以每秒百次的頻率取樣，一天便產生數十億筆紀錄。
Velocity	產生速度	資料以極高的速率持續湧入，且往往要求即時回應。例如機器手臂的力矩異常必須在毫秒內偵測。
Variety	多樣性	資料型態複雜，涵蓋結構化（資料表）、半結構化（JSON、日誌）與非結構化（影像、聲音、文字）。
Veracity	真實性	資料的品質與可信度參差不齊。感測器可能漂移、紀錄可能遺漏、標註可能錯誤。
Value	價值	資料本身不等於價值；價值密度往往很低，需要透過分析才能萃取出來。

表 1-3 大數據的 5V 特性

這五個 V 之中，前三個（量、速、多樣）描述的是「技術挑戰」，第四個（真實）提醒我們資料品質的問題，而第五個（價值）則點出了整件事的目的。初學者常犯的錯誤，是只顧著追求前三個 V 的技術，卻忘了第五個 V 才是所有努力的終點。

叫獸提醒

價值密度低，是大數據非常重要卻常被忽略的性質。一支監視器連續錄了一整個月，真正有意義的畫面可能只有三十秒。這說明了為什麼我們需要如此龐大的運算能力——不是因為資料本身珍貴，而是因為必須篩過大量的資料，才能找到那少數珍貴的部分。

1.1.4 垂直擴充與水平擴充

面對資料量成長，工程上有兩條路可走。第一條稱為垂直擴充（Scale-up），意思是把單一台機器換得更強——加更多記憶體、換更快的處理器、插更大的硬碟。第二條稱為水平擴充（Scale-out），意思是不換機器，而是增加機器的數量，讓它們合作完成同一件工作。

垂直擴充的優點是簡單：程式完全不用改寫，換台機器繼續跑就是了。但它有三個難以迴避的問題。其一是物理上限——單機的記憶體與處理器核心數終究有極限。其二是成本曲線陡峭——效能提升一倍，價格往往不只提升一倍，高階伺服器的單位效能成本遠高於一般機器。其三是單點故障——所有雞蛋都放在同一個籃子裡，機器一停，服務就中斷。

水平擴充恰恰相反。它的成本曲線接近線性，理論上沒有規模上限，而且天生具備容錯的可能性。代價則是：程式必須重新設計。你得處理資料如何切分、任務如何分派、節點故障了怎麼辦、中間結果如何彙整——這些在單機時代根本不存在的問題。

面向	垂直擴充（Scale-up）	水平擴充（Scale-out）
做法	提升單一機器的規格	增加機器數量並協同運作
成本曲線	陡峭，高階硬體單位成本高	接近線性，可用一般規格機器
規模上限	受單機物理極限限制	理論上無上限
容錯能力	單點故障風險高	可設計為容忍節點故障
軟體複雜度	低，程式不需修改	高，須重新設計為分散式

表 1-4 垂直擴充與水平擴充的比較

整個大數據領域，本質上就是在回答一個問題：既然我們被迫走上水平擴充這條路，那麼隨之而來的軟體複雜度，該如何用系統與框架把它藏起來，讓應用開發者不必每次都從零面對？MapReduce、Spark 乃至各種雲端託管服務，都是這個問題的答案。

1.2 從資料到價值：典型應用場景

1.2.1 資料、資訊、知識與智慧

資料如何變成價值？資訊科學領域有一個經典的層級模型，稱為 DIKW 金字塔，由下而上分別是資料（Data）、資訊（Information）、知識（Knowledge）與智慧（Wisdom）。

層級	回答的問題	智慧製造情境範例
資料 Data	是什麼？	感測器記錄到主軸溫度為 78.4 °C。
資訊 Information	誰、何時、何處？	三號機台的主軸在今日下午三時溫度達 78.4 °C，較平時高出 12 °C。
知識 Knowledge	如何發生？	根據歷史紀錄，主軸溫度異常上升多半肇因於軸承潤滑不足。
智慧 Wisdom	該怎麼做？	在本週停機保養時優先更換三號機台軸承，以避免非計畫性停機。

表 1-5 DIKW 層級與智慧製造情境

這個金字塔提醒我們，資料分析的目的不是產出更多數字，而是支持更好的決策。本書後續所有的技術——從分散式儲存到機器學習——都是在幫助你把金字塔的底層往上推。

1.2.2 四種資料分析層次

與 DIKW 相呼應的，是分析工作的四個層次。這個分類在規劃專題時特別有用，因為它能幫你界定「我到底要做到哪一步」。

- **描述性分析（Descriptive）**：回答「發生了什麼」。以統計摘要與視覺化呈現現況，例如產線本月的稼動率報表。
- **診斷性分析（Diagnostic）**：回答「為什麼發生」。透過關聯分析、下鑽探索找出原因，例如找出良率下降與某批原料的關聯。
- **預測性分析（Predictive）**：回答「將會發生什麼」。以機器學習模型推估未來，例如預測某台設備在未來兩週內故障的機率。
- **指示性分析（Prescriptive）**：回答「該怎麼做」。在預測之上進行最佳化與決策建議，例如自動排定最省成本的保養排程。

四個層次的難度與價值同步遞增。多數組織的資料分析都停留在前兩層，能夠穩定做到第三、第四層的，往往就是產業中的領先者。

1.2.3 典型應用場景

大數據的應用早已滲透到各行各業，以下列舉幾個與本學程關係密切的領域。

- **智慧製造**：產線即時監控、預測性維護、自動光學檢測與數位孿生。本書第 37 章將深入探討。
- **機器人與自駕**：多感測器融合、同步定位與地圖建構（SLAM）、視覺辨識。本書第 36 章將專章討論。
- **電子商務**：個人化推薦、動態定價、需求預測與供應鏈最佳化。

- **智慧醫療：**醫學影像判讀、電子病歷探勘、流行病學監測與藥物開發。
- **智慧城市：**交通流量預測、能源管理、公共安全與環境監測。
- **科學研究：**天文觀測、基因定序、氣候模擬等領域的巨量資料處理。

1.2.4 三個常見的迷思

在進入技術細節之前，有三個初學者常見的迷思值得先澄清，它們會影響你日後規劃專題的方向。

迷思一：資料越多，結論就越可靠

資料量增加確實能降低隨機誤差，但無法修正系統性偏誤。如果你的感測器安裝位置本身就有問題，那麼蒐集一年的資料與蒐集一天的資料，得到的都是同樣偏斜的結論——只是你會對錯誤的結論更有信心。這正是 Veracity 這個 V 存在的理由：資料的品質與代表性，遠比數量重要。

迷思二：相關就是因果

大數據分析極擅長找出相關性，但相關不等於因果。經典的例子是：冰淇淋銷量與溺水人數高度相關，但兩者並無因果關係，真正的原因是氣溫。在製造現場也一樣——某項參數與良率同步變動，可能只是因為兩者都受同一個上游因素影響。要建立因果關係，需要實驗設計或因果推論方法，而非僅憑資料的統計關聯。

迷思三：模型越複雜越好

在許多實務情境中，一個簡單、可解釋、能穩定運作的模型，價值高於一個準確率略高但無人能理解、也難以維護的複雜模型。特別是在製造與醫療領域，決策者必須能說明「為什麼系統這樣建議」，可解釋性往往是能否導入的關鍵。

叫獸提醒

我看過太多專題敗在同一個地方：同學花了八成時間在調模型，只花兩成時間在理解資料。實務上剛好要倒過來。資料工程與資料理解，才是決定專案成敗的關鍵；模型往往是最後、也最容易替換的那一塊。

1.3 雲端運算的興起與服務模型概觀

1.3.1 為什麼會有雲端

讓我們回到那個車庫。假設你今天也想做一件需要一千台電腦的事，你會遇到什麼？你得先花一大筆錢買機器、租機房、拉電源、裝空調、聘人維運，而且必須在還不確定專案會不會成功之前，就把這些錢全部投下去。更糟的是，如果你的運算需求只在每月月底暴增三天，其餘二十七天這些機器就在空轉。

雲端運算解決的正是這個問題。它把運算資源變成像水電一樣的公共服務：需要時打開水龍頭，用多少付多少，不用時關掉。這個轉變在財務上有個專門的說法，叫做「資本支出轉為營運支出」（CapEx to OpEx）——你不再需要預先購置資產，而是把運算當成日常費用來支付。

1.3.2 雲端運算的五大特徵

美國國家標準與技術研究院（NIST）提出的定義，是目前最被廣泛引用的版本。它指出雲端運算具備五項本質特徵：

- (一) 使用者可自行開通運算資源，無須與供應商人員互動。
- (二) 資源可透過標準網路機制，由各種裝置存取。
- (三) 供應商的資源以多租戶模式匯集，動態分配給不同使用者。
- (四) 資源可迅速地向上或向下擴充，對使用者而言近乎無限。
- (五) 資源使用可被量測、監控與報告，據以計費。

這五項特徵之中，「快速彈性」與「可計量服務」對大數據處理特別關鍵。因為大數據的運算需求往往是脈衝式的——平時安靜，跑批次分析時卻需要上百台機器。若非雲端，這種需求幾乎無法在合理成本下滿足。

1.3.3 三種服務模型

雲端服務依照抽象化程度的不同，可分為三種模型。理解它們的差異，本質上是理解「哪些事由你負責、哪些事由供應商負責」。

模型	供應商提供	使用者負責	大數據領域範例
IaaS	運算、儲存、網路等基礎設施	作業系統、中介軟體、應用程式	在雲端虛擬機上自行架設 Hadoop 叢集
PaaS	基礎設施加上執行平台與中介軟體	應用程式與資料	託管式 Spark 服務、雲端資料倉儲
SaaS	完整的應用軟體服務	僅需設定與使用	線上商業智慧與視覺化分析工具

表 1-6 三種雲端服務模型的比較

叫獸提醒

有個好記的比喻：IaaS 像是租一間空廚房，鍋碗瓢盆與食材都要自己準備；PaaS 像是租一間附設備與調味料的廚房，你只要專心煮菜；SaaS 則是直接上餐廳點餐。三者沒有優劣，只有適不適合——研究用途常需要 IaaS 的彈性，而快速交付的專案則往往選擇 PaaS。

部署模型

除了服務模型，雲端還可依部署方式分為公有雲（由第三方供應商營運，開放給大眾）、私有雲（由單一組織專用）、混合雲（兩者併用，敏感資料留在本地而彈性運算放上公有雲）與社群雲（由具共同需求的數個組織共享）。製造業因涉及製程機密，採用混合雲的比例相當高。

1.3.4 雲端的限制與風險

雲端並非萬靈丹。一門負責任的課程，必須同時說明它的代價。以下四項風險，在實務導入時都必須事先評估。

- **成本失控：**「用多少付多少」是優點，也是陷阱。一支寫得不好的查詢，可能在幾小時內掃過數十 TB 的資料而產生高額帳單。因此雲端成本管理（FinOps）已成為專門的課題，本書第 27 章將討論。
- **廠商鎖定：**當你大量使用某家供應商的專屬服務，日後要遷移將極為困難。降低風險的做法是盡量採用開放標準與開源技術，例如以開放表格式儲存資料，而非綁定特定廠商的專有格式。

- **資料主權與法規：**資料實際存放在哪個國家，會牽涉當地法規的管轄。製造業的製程資料、醫療機構的病歷資料，往往受法令限制不得任意跨境。這也是混合雲在台灣製造業普及的主因之一。
- **網路依賴與延遲：**雲端服務仰賴穩定的網路連線。對於必須在毫秒內反應的機器控制而言，把運算放到雲端在物理上就不可行——光是來回的網路延遲就已超出容許範圍。這正是邊緣運算存在的理由。

理解這些限制，你才能做出恰當的架構判斷。實務上最常見的設計原則是：把需要即時反應的運算留在邊緣，把需要大規模儲存與訓練的工作放上雲端，並在兩者之間設計清楚的資料流。這個原則會在本書第玖篇被反覆應用。

1.4 大數據與雲端的匯流趨勢

本書把「大數據分析」與「雲端運算」放在同一門課裡，並不是因為兩個題目都很熱門，而是因為它們在技術上彼此需要。若把兩者拆開來看，都只能看到半張圖。

1.4.1 大數據為什麼需要雲端

- **彈性容量：**大數據的運算需求高度不均勻。雲端讓你在需要時取得上百個節點，用完立刻釋放，不必為尖峰需求長期養機器。
- **儲存成本：**物件儲存的單位成本遠低於自建儲存陣列，且具備極高的耐久性，適合長期保存原始資料。
- **託管服務：**雲端供應商提供託管式的 Spark、Kafka 與資料倉儲，大幅降低叢集維運的人力負擔。
- **全球佈署：**資料往往產生於世界各地，雲端的多區域架構讓資料能就近收集與處理。

1.4.2 雲端為什麼需要大數據

反過來看，大數據也是雲端最主要的工作負載之一。正因為有海量資料需要儲存與分析，雲端供應商才有動機發展物件儲存、分散式查詢引擎與各種資料服務。近年再加上生成式人工智慧的訓練與推論需求，更把雲端推向新一波的成長。

1.4.3 三個關鍵的架構演進

大數據與雲端的匯流，具體表現為以下三個架構層次的轉變。這三個轉變會反覆出現在本書後續章節，值得你現在先建立印象。

演進	從	到
儲存運算分離	資料與運算綁在同一批機器 (Hadoop 早期)	資料存於物件儲存，運算節點可獨立擴縮
批次到串流	每日／每小時的批次處理	事件驅動的即時串流處理
地端到邊雲協同	所有運算集中於資料中心	邊緣裝置先行處理，再與雲端協同

表 1-7 大數據架構的三個關鍵演進

其中「儲存運算分離」尤其重要。早期的 Hadoop 強調「把運算搬到資料旁邊」，因為當時網路頻寬昂貴；但隨著資料中心網路速度大幅提升，以及雲端物件儲存的普及，現代架構反而傾向把兩者分開，讓儲存與運算各自依需求擴充。這正是資料湖與 Lakehouse 架構的基礎，本書第 30 章將詳細說明。

1.5 技術生態系全景

初學者面對大數據領域，常見的困擾是名詞太多：Hadoop、Spark、Kafka、Flink、Kubernetes、Delta Lake……彼此的關係看起來像一團亂麻。其實只要用「分層」的角度去看，整張圖就會清楚起來。

1.5.1 六層架構

層次	任務	代表性技術
應用層	呈現分析結果、支援決策	商業智慧工具、儀表板、應用系統
分析層	探勘、建模與人工智慧	Spark MLlib、機器學習框架、大語言模型
運算層	分散式批次與串流運算	MapReduce、Spark、Flink
儲存層	巨量資料的持久化保存	HDFS、物件儲存、NoSQL、Lakehouse
擷取層	資料的收集與傳輸	Kafka、訊息佇列、ETL 工具
資料來源層	資料的產生	感測器、機台、日誌、交易系統、影像

表 1-8 大數據技術生態系的六層架構

除了這六層之外，還有兩個「貫穿性」的面向橫跨所有層次：一是資源管理與部署（YARN、Kubernetes、雲端平台），二是治理與安全（資料目錄、存取控制、法規遵循）。它們不屬於任何單一層，而是支撐整個堆疊運作的基礎。

讓我們由下而上走一遍這張圖，你會發現它其實就是資料的一生。

資料誕生於最底層的來源層——一顆感測器記錄下溫度、一台相機拍下影像、一筆交易被寫入系統。這些資料必須被送到能處理它們的地方，於是有了擷取層：Kafka 之類的訊息佇列扮演緩衝與轉運的角色，確保上游產得再快，下游也不會被沖垮。

進入儲存層後，資料要面對第一個真正的抉擇：以什麼形式保存？是原封不動存成檔案，還是整理成結構化的資料表？前者彈性高但難以查詢，後者查詢方便但失去細節。這個取舍催生了資料湖與 Lakehouse 的架構之爭，本書第 30 章會詳細討論。

運算層則是把資料轉為結果的引擎。批次處理適合「昨天的整批資料」，串流處理適合「此刻正在發生的事」，兩者的取舍與融合，是第參篇與第陸篇的主題。再往上的分析層，才輪到大家最熟悉的機器學習與人工智慧登場——但你會發現，模型只是整座塔的一層，它的效果高度仰賴底下五層是否穩固。

最上層的應用層，則是資料真正碰觸到人的地方：一張儀表板、一則告警、一個自動排定的保養工單。若沒有這一層，底下所有的努力都不會產生價值。

1.5.2 三個里程碑

若把生態系的發展濃縮成一條時間軸，有三個里程碑最值得記住。

年代	里程碑	意義
二〇〇三～二〇〇六	GFS、MapReduce 與 Hadoop	確立了以廉價機器叢集處理巨量資料的典範。
二〇〇九～二〇	Spark 與記憶體運算	以記憶體運算大幅提升速度，並統一批次、串流與機器學

年代	里程碑	意義
一四		習的介面。
二〇一五 至今	雲原生與 Lakehouse	容器化、儲存運算分離與開放表格式，使大數據平台全面雲端化。

表 1-9 大數據技術發展的三個里程碑

有趣的是，這三個階段並非彼此取代，而是層層累積。今天你在雲端上跑的 Spark 作業，底層仍然承襲了 MapReduce 的分散式思維，而 MapReduce 的容錯設計，又源自那個「假設機器一定會壞」的轉念。技術會更迭，但原理會留下——這也是本書為何要花整個第壹篇與第貳篇打底的原因。

1.6 本書架構、學習地圖與工具準備

1.6.1 本書的九篇架構

本書共分九篇、三十八章。這樣的編排並非把主題平鋪並列，而是有一條明確的推進線：先建立原理，再處理資料的存放，接著談如何運算，然後是如何分析，最後才是如何部署與落地。

篇別	主題	在學習路徑中的角色
第壹篇	導論與基礎	建立系統思維、資料模型與分散式系統的基本觀念。
第貳篇	分散式儲存與資料庫	回答「資料放在哪裡、如何一致地存取」。
第參篇	批次運算框架	以 Hadoop 與 Spark 處理大規模批次運算。
第肆篇	探勘與分析演算法	本書的分析核心，涵蓋相似性、頻繁項集、分群、圖與推薦。
第伍篇	生成式 AI 與開源生態	大語言模型、AI 代理與開源模型協作平台。
第陸篇	串流處理與即時分析	從批次跨入即時，處理不斷湧入的資料流。
第柒篇	雲端運算平台與架構	虛擬化、容器、Kubernetes 與雲端資料架構。
第捌篇	治理、安全與維運	讓系統在真實環境中可信、合規且可持續運作。
第玖篇	整合實務與應用	邊緣到雲端的整合，收束於智慧製造與機器人專題。

表 1-10 本書九篇架構與學習路徑

如果把這條路徑濃縮成一句話，那就是：由單機到分散式、由批次到串流、由地端到雲端、由資料到人工智慧。你可以在每讀完一篇時回頭看這句話，確認自己走到了哪裡。

1.6.2 三條學習路徑

不同背景的讀者，可以採取不同的閱讀策略。

- **完整修習（建議）：**依序研讀第壹篇至第玖篇。適合修習本課程的大學部學生，可建立完整的知識體系。
- **分析導向：**第壹篇 → 第參篇 → 第肆篇 → 第伍篇。適合已具備系統基礎、希望專注於演算法與人工智慧的讀者。
- **工程導向：**第壹篇 → 第貳篇 → 第柒篇 → 第捌篇 → 第玖篇。適合偏重平台建置、雲端部署與維運的讀者。

若你是高中階段的自學讀者，建議先掌握第 1、11、12 章與第玖篇的實作內容，暫時略過較深的數學推導，待日後有需要時再回頭補齊。

1.6.3 每章的固定結構

為便於學習，本書每一章均採用相同的編排結構，你可以依自己的需求選擇閱讀順序。

- **叫獸開講：**以故事或情境破題，建立學習動機。
- **學習目標：**列出研讀後應具備的能力，可作為自我檢核清單。
- **核心概念：**以表格整理本章關鍵術語，適合快速預習與複習。
- **正文：**分節展開的完整課文，含公式、流程與圖表。
- **案例研討：**三個由淺入深的實際案例，銜接理論與應用。
- **實作活動：**動手練習，多數可在個人電腦或雲端筆記本完成。
- **延伸閱讀：**指向經典教材與原始論文，供深入探究。
- **小結與習題：**重點回顧與觀念、計算、討論三類題目。

1.6.4 工具準備

本書所需的工具依使用階段可分為三組。你不需要在此刻全部安裝完成，只要先建立基本的 Python 環境即可，其餘工具將在對應章節中逐步引入。

階段	工具	首次使用章節
基礎	Python 3.10 以上、pandas、NumPy、matplotlib	第 1 章（本章實作活動）
分散式運算	Apache Spark (PySpark)、Hadoop	第 9 章、第 11 章
容器與雲端	Docker、Kubernetes、公有雲免費層帳號	第 28 章、第 29 章
邊緣與機器人	樹莓派或 NVIDIA Jetson、ROS2	第 35 章、第 36 章

表 1-11 本書工具準備一覽

叫獸提醒

很多同學讀技術書會卡在第一關：環境裝不起來，然後就放棄了。我的建議是，如果本機安裝連續失敗超過一小時，就直接改用雲端筆記本服務。學習的重點是理解概念與寫出程式，而不是跟安裝程式搏鬥。等你有餘裕時再回頭處理本機環境即可。

公式與流程：分散式運算的效益極限

既然分散式運算的精神是「用更多機器」，那麼是不是機器越多就一定越快？答案是否定的，而阿姆達爾定律（Amdahl's Law）正是描述這個限制的經典公式。

假設一項工作中，可平行化的比例為 p ($0 \leq p \leq 1$)，不可平行化的比例為 $1 - p$ ，而我們使用 n 個處理單元，則整體加速比 $S(n)$ 為：

$$S(n) = 1 / [(1 - p) + p / n]$$

式 1-1 阿姆達爾定律

這個公式告訴我們一件相當殘酷的事：當 n 趨近於無限大時，加速比的上限是 $1 / (1 - p)$ 。也就是說，只要工作中還有 5% 無法平行化，那麼無論你投入多少台機器，最快也只能加速二十倍。

可平行比例 p	$n = 10$	$n = 100$	$n \rightarrow \infty$ (上限)
0.50	約 1.8 倍	約 2.0 倍	2 倍
0.90	約 5.3 倍	約 9.2 倍	10 倍
0.95	約 6.9 倍	約 16.8 倍	20 倍
0.99	約 9.2 倍	約 50.3 倍	100 倍

表 1-12 不同平行化比例下的加速比

叫獸提醒

阿姆達爾定律的實務啟示是：與其一味增加機器，不如先想辦法降低不可平行化的部分。在後續章節你會看到，Shuffle 階段的資料交換、任務啟動的開銷、以及最後彙整結果的步驟，往往就是那個「不可平行化的 $1 - p$ 」。優秀的分散式程式設計，很大一部分就是在跟這個比例搏鬥。

案例研討**案例一 搜尋引擎：大數據的起點**

前面提到的 Google 故事，正是大數據的原型案例。網頁的數量以十億計，且不斷變動；每一次搜尋都必須在數百毫秒內回應。這同時觸及了 Volume（網頁總量）、Velocity（即時查詢）與 Variety（文字、圖片、影音）三個面向。

其解法的核心思想有三：其一，用大量廉價機器取代少數昂貴主機；其二，把資料切成區塊並複製多份，讓任何一台機器故障都不影響整體；其三，把運算拆解成 Map 與 Reduce 兩個簡單步驟，讓程式設計師不必處理分散式的複雜細節。這三點，構成了本書第貳篇與第參篇的骨幹。

案例二 智慧工廠的產線監測

設想一座工廠，產線上部署了兩百個感測器，每個以每秒一百次的頻率取樣。單純計算，每秒便產生兩萬筆紀錄，一天約十七億筆。若每筆紀錄佔一百位元組，一天的原始資料量約為一百七十 GB，一年超過六十 TB。

這個案例的挑戰不只在於量。真正困難的地方在於：異常必須在毫秒等級被偵測（Velocity），資料同時包含數值訊號、影像與設備日誌（Variety），而感測器本身可能漂移或失效（Veracity）。因此，實務上的架構通常是：邊緣裝置先行過濾與特徵萃取，串流引擎負責即時告警，雲端則保存完整資料供長期分析與模型訓練。這條「邊緣—串流—雲端」的路徑，正是本書第陸篇與第玖篇的主題。

案例三 自主移動機器人的地圖建構

一台在廠區內自主移動的機器人，同時運行光達、相機與慣性量測單元。它必須一邊估計自己的位置，一邊建立環境地圖，這個問題稱為同步定位與地圖建構（SLAM）。

從資料的角度看，這是一個極具代表性的多模態串流問題：不同感測器的取樣頻率不同、時間戳需要對齊、資料量龐大且不可能全部上傳雲端。因此，實務做法是在機器人端（邊緣）完成即時的定位與避障，而把地圖優化、多機資料融合與模型再訓練放到雲端。這個「即時在邊緣、學習在雲端」的分工，是機器人大數據應用的通則，本書第 36 章將深入探討。

案例四 大語言模型的訓練：當資料本身成為競爭力

近年最受矚目的大數據應用，非大語言模型莫屬。訓練一個現代的語言模型，需要從網際網路、書籍、程式碼庫等來源蒐集數以兆計的詞元（token），資料量動輒數十至數百 TB。這個規模本身已是 Volume 的極致展現。

但真正的挑戰不在於量，而在於品質。原始的網頁抓取結果充滿了重複內容、機器生成的垃圾文字、格式錯亂的頁面與大量廣告。研究顯示，資料的清理與去重對模型品質的影響，往往大於單純增加資料量。因此，訓練前的資料處理管線通常包含以下步驟：

- 步驟 1.** 語言辨識與篩選，保留目標語言的文本。
- 步驟 2.** 品質過濾，剔除過短、重複度過高或明顯無意義的內容。
- 步驟 3.** 去重（Deduplication），移除近似重複的文件，避免模型過度記憶特定內容。
- 步驟 4.** 個資與敏感內容的偵測與移除。
- 步驟 5.** 詞元化（Tokenization），將文本轉為模型可讀的數值序列。

請特別注意第三個步驟。要在數十億份文件中找出「內容近似但不完全相同」的重複項，若採用兩兩比對，計算量將是天文數字。實務上使用的正是本書第 14 章要教的 MinHash 與區域敏感雜湊（LSH）——一個誕生於一九九〇年代的演算法，在三十年後成為訓練大語言模型的關鍵基礎設施。

這個案例同時觸及了五個 V：資料量龐大（Volume）、來源多樣涵蓋網頁與程式碼（Variety）、品質參差需要大量清理（Veracity）、訓練過程需要持續更新語料（Velocity），而最終資料品質直接決定了模型的能力（Value）。它也說明了為何本書要把探勘演算法與生成式 AI 編排在相鄰的兩篇——前者正是後者的基礎工程。

叫獸提醒

這個案例最值得記住的一句話是：模型架構大家都看得到，論文都公開了；真正拉開差距的，往往是沒有公開的資料處理管線。當同學問我「該學演算法還是該學調模型」，我的答案通常是——先把資料處理學好，那是別人拿不走的功夫。

實作活動

活動一 估算你的資料足跡

請以自身情境估算一天所產生的資料量。可考慮的來源包括：手機拍攝的照片與影片、通訊軟體訊息、串流影音的觀看時數、瀏覽器的操作紀錄等。請完成下列步驟：

- 步驟 1.** 列出至少五項資料來源，並估計每項每日的資料量。
- 步驟 2.** 計算你一天、一個月與一年的總資料量。
- 步驟 3.** 將你的年資料量乘以全台灣人口，估算全國的年資料量級。
- 步驟 4.** 討論：這些資料之中，「有價值」的比例大約是多少？呼應 5V 的哪一項特性？

活動二 建置本課程的 Python 環境

後續章節將大量使用 Python 與 PySpark。請完成下列環境準備，並截圖記錄結果。

- 步驟 1.** 安裝 Python 3.10 以上版本，並建立一個名為 bigdata 的虛擬環境。
- 步驟 2.** 於虛擬環境中安裝 pandas、numpy、matplotlib 與 pyspark 四個套件。
- 步驟 3.** 撰寫一支程式，讀入任一 CSV 檔並輸出其列數、欄數與前五筆資料。
- 步驟 4.** 在 Python 中執行 import pyspark 並印出版本號，確認安裝成功。

若你的電腦環境安裝不易，也可改用雲端筆記本服務（如 Google Colab）完成本活動，這同時也是你第一次實際使用 SaaS 型態的雲端服務。

活動三 雲端服務模型的辨識練習

請找出三項你日常使用的網路服務，分別判斷它們屬於 IaaS、PaaS 或 SaaS，並說明判斷依據：這項服務中，哪些部分由供應商負責、哪些部分由你負責？完成後，於課堂上與同組同學交換比較。

延伸閱讀

- **《Mining of Massive Datasets》第 1 章：**Leskovec、Rajaraman 與 Ullman 著。本書主教材，第 1 章對資料探勘的基本觀念有精要的鋪陳，且全書可自官方網站免費取得。
- **《Designing Data-Intensive Applications》第 1 章：**Kleppmann 與 Riccomini 著。以可靠性、可擴充性、可維護性三個面向切入資料系統設計，是理解本書第 3 章的最佳預備。
- **《Fundamentals of Data Engineering》第 1、2 章：**Reis 與 Housley 著。完整介紹資料工程生命週期，與本書第 2 章密切相關。
- **GFS 與 MapReduce 原始論文：**Google 於二〇〇三與二〇〇四年發表。篇幅不長且行文清晰，建議在讀完本書第參篇後回頭閱讀，會有更深的體會。
- **NIST 雲端運算定義 (SP 800-145)：**美國國家標準與技術研究院發布，僅數頁，是雲端運算五大特徵與三種服務模型的權威出處。

本章小結

1. 大數據並非由某個固定的數字門檻所定義，而是指資料的規模、速度或複雜度已超出單一機器與傳統工具的處理能力，必須改採分散式方法。
2. 5V 特性——量、速、多樣、真實、價值——分別描述了大數據的技術挑戰、品質問題與最終目的。其中價值密度偏低，正是需要龐大運算能力的根本原因。
3. 資料要經過 DIKW 的層層轉化才能成為決策依據；對應地，分析工作可分為描述性、診斷性、預測性與指示性四個層次，難度與價值同步遞增。
4. 雲端運算具備隨需自助、廣泛存取、資源池化、快速彈性與可計量服務五大特徵，並依抽象程度分為 IaaS、PaaS 與 SaaS 三種服務模型。
5. 大數據與雲端彼此需要：前者需要雲端的彈性容量與低成本儲存，後者則以大數據為主要工作負載。兩者的匯流表現為儲存運算分離、批次轉串流、地端轉邊雲協同三個架構演進。
6. 大數據技術生態系可分為資料來源、擷取、儲存、運算、分析與應用六層，另有資源管理與治理安全兩個貫穿性面向。
7. 阿姆達爾定律指出，加速比的上限取決於不可平行化的比例；因此分散式系統的優化重點，往往在於降低那個無法平行的部分。

習題

一、觀念題

1. 請以自己的話說明大數據的操作型定義，並解釋為何不宜用固定的資料量門檻來界定。
2. 何謂價值密度？請以一個生活中的例子說明價值密度偏低的情形。
3. 請比較 IaaS、PaaS 與 SaaS 三種服務模型在責任分界上的差異。
4. 為什麼現代大數據架構傾向「儲存運算分離」？早期 Hadoop 的設計又為何選擇相反的策略？

二、計算題

1. 某條產線有 150 個感測器，每個以每秒 50 次取樣，每筆紀錄 80 位元組。請計算一日與一年的原始資料量（以 GB 與 TB 表示）。
2. 某項工作可平行化的比例為 0.92。請以阿姆達爾定律計算 $n = 20$ 與 $n = 200$ 時的加速比，並說明兩者差距為何不如機器數量的差距顯著。

三、思考與討論

1. 請就本學程的專業領域，提出一個你認為具備 5V 特性的資料應用構想，並說明它屬於四種分析層次中的哪一層。
2. 有人主張「有了雲端，就不需要學分散式系統的原理了」。請提出至少兩點理由反駁或支持此一說法。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

關鍵在於掌握「相對性」。定義應扣住：資料的量、速度或複雜度已超出單一機器與傳統工具在可接受時間與成本內的處理能力。不宜用固定門檻的理由至少有二：其一，硬體效能與工具能力持續進步，今日的門檻明日即失效；其二，同樣的資料量在不同運算環境下的難易度差異極大，三百 GB 在筆電上是難題，在叢集上則否。可補充：判準應為「是否已被迫改採分散式方法」。

第 2 題

價值密度指的是單位資料量中所含有效資訊的比例。大數據的典型特徵是價值密度低——必須篩過大量資料，才能找出少數關鍵。生活化例子如：行車記錄器連續錄影數十小時，真正需要調閱的可能僅事故當下的數秒；或社群平台上的大量貼文中，與某產品有關的有效意見僅佔極小比例。答題時應點出：正因價值密度低，才需要龐大的運算能力來完成篩選。

第 3 題

應以「責任分界」為軸線回答。IaaS 由供應商負責運算、儲存與網路等基礎設施，使用者須自行管理作業系統、中介軟體與應用程式，彈性最大但維運負擔最重；PaaS 再往上包辦執行平台與中介軟體，使用者只需專注於應用程式與資料；SaaS 則提供完整應用，使用者僅需設定與使用。可用本章的廚房比喻（空廚房／附設備的廚房／餐廳）輔助說明，並補充三者無優劣之分，端視需求而定。

第 4 題

需分別說明兩個時代的限制條件。早期 Hadoop 選擇「把運算搬到資料旁邊」（資料在地性），是因為當時資料中心的網路頻寬相對昂貴且有限，搬移大量資料的成本高於搬移程式。現代則因兩項

變化而轉向：其一是資料中心網路頻寬大幅提升，跨節點傳輸不再是瓶頸；其二是雲端物件儲存普及且成本低廉。儲存運算分離的好處在於兩者可依各自需求獨立擴縮——需要大量運算時擴充運算節點，用完即釋放，不必為此長期保有儲存。這正是資料湖與 Lakehouse 架構的基礎。

二、計算題

第 1 題

依序計算即可。

- (1) 每秒紀錄數： $150 \times 50 = 7,500$ 筆／秒。
- (2) 每秒資料量： $7,500 \times 80 = 600,000$ 位元組／秒 $= 0.6$ MB／秒。
- (3) 每日資料量： $600,000 \times 86,400 = 51,840,000,000$ 位元組 ≈ 51.84 GB／日。
- (4) 每年資料量： $51.84 \times 365 = 18,921.6$ GB ≈ 18.92 TB／年。

延伸思考：這僅是單一產線的原始資料。若工廠有十條產線，年資料量即接近 190 TB；再考慮備援副本（通常三份），實際儲存需求將達約 570 TB。這正是為何製造業導入大數據時，儲存架構的規劃必須及早進行。

第 2 題

代入阿姆達爾定律 $S(n) = 1 / [(1 - p) + p / n]$ ，其中 $p = 0.92$ ，故 $1 - p = 0.08$ 。

- (1) $n = 20$: $S = 1 / (0.08 + 0.92/20) = 1 / (0.08 + 0.046) = 1 / 0.126 \approx 7.94$ 倍。
- (2) $n = 200$: $S = 1 / (0.08 + 0.92/200) = 1 / (0.08 + 0.0046) = 1 / 0.0846 \approx 11.82$ 倍。
- (3) 理論上限： $n \rightarrow \infty$ 時， $S \rightarrow 1 / 0.08 = 12.5$ 倍。

說明差距的關鍵：機器數量增加為十倍（20 $\rightarrow$ 200），加速比卻僅由約 7.94 倍提升至約 11.82 倍，增幅不到 1.5 倍。原因在於分母中的常數項 0.08（不可平行化的部分）並不會隨 n 減少；當 n 夠大時， p/n 已趨近於零，整體加速比便被 $1 / (1 - p)$ 這個上限牢牢卡住。實務啟示是：與其持續增加機器，不如設法降低不可平行化的比例。

三、思考與討論

第 1 題

答題應包含三個部分。其一，明確描述應用構想與資料來源；其二，逐一對照 5V 說明其成立之處，特別注意不要只談 Volume 而忽略 Veracity 與 Value；其三，指出所屬分析層次並說明理由。以「產線刀具磨耗預測」為例：Volume 為多台機台的高頻振動訊號，Velocity 為需即時判斷，Variety 為振動、電流與影像並存，Veracity 為感測器可能漂移，Value 為降低非計畫性停機成本；因其目的在推估未來何時需要更換刀具，屬於預測性分析。若進一步自動排定更換排程，則進入指示性分析。

第 2 題

此題旨在訓練論證能力，正反皆可，但須有具體理由。支持方可主張：託管服務已封裝多數複雜性，一般開發者無須理解底層即可完成工作，學習成本應投入更貼近應用的領域。反對方（本書立場偏向此側）可提出：其一，效能調校與成本控制均需理解底層原理，否則寫出的查詢可能掃過遠超必要的資料量而產生高額費用；其二，故障排除時若不理解分區、複製與一致性等機制，將無從判斷問

題根源；其三，架構選型（如是否採用最終一致性、如何設計分區鍵）本身就是分散式系統的決策，雲端只是把選項呈現出來，並未代為決定。建議答題時舉出至少一個具體情境佐證。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 1 章 大數據與雲端運算導論

台灣自編大學教科書
@prof
大數據分析與雲端運算

第壹篇 導論與基礎

Foundations

第 2 章

資料科學與資料工程生命週期

The Data Science and Data Engineering Lifecycle

機器人叫獸（李世淵） 編著

叫獸開講

二〇一二年，《哈佛商業評論》登出一篇文章，把「資料科學家」封為二十一世紀最性感的職業。一時之間，無數聰明的年輕人湧入這個領域，以為每天的工作就是建構酷炫的模型、訓練人工智慧、在會議上秀出漂亮的預測結果。

然後，現實給了他們一記悶棍。進了公司才發現，真正的工作大概是這樣的：想訓練模型，卻找不到資料在哪裡；找到了資料，欄位名稱是一堆沒人看得懂的縮寫；好不容易看懂了，發現有三成的紀錄是空的、日期格式有五種寫法、同一個客戶出現了四個不同的編號。多數資料科學家後來坦承，他們有大約八成的時間，不是在做分析，而是在清理與整理資料。

這個落差，催生了一個新的專業：資料工程師。如果說資料科學家是餐廳裡光鮮亮麗的主廚，那麼資料工程師就是後廚裡負責採購、清洗、備料、把食材準時送到爐邊的人。沒有這個人，主廚連一道菜都端不出來。近十年來業界逐漸有了共識——決定一個資料專案成敗的，往往不是模型有多先進，而是那條把資料從源頭送到模型面前的「管線」蓋得好不好。

這一章，我們要退後一步，看清楚資料從誕生到產生價值的完整旅程。這趟旅程有個名字，叫做「資料工程生命週期」。你會發現，第 1 章講的那些技術——儲存、運算、雲端——其實都是這條生命週期上的某一站。先看懂整張地圖，後面每一章你才知道自己走到了哪裡。

學習目標

研讀完本章之後，你應該能夠：

1. 說明資料工程生命週期的五個階段，並指出貫穿其中的六個要素。
2. 區分資料科學家、資料工程師與其他資料相關角色的職責分工。
3. 辨識常見的資料來源類型，並說明其結構與產生模式的差異。
4. 比較批次擷取與串流擷取的適用情境，並說明變更資料擷取（CDC）的概念。
5. 說明資料轉換、儲存與服務三個階段的核心任務，並區分 ETL 與 ELT。
6. 說明資料治理的三大支柱，並以生命週期觀點規劃資料的保存與淘汰。

核心概念

核心概念	說明
資料工程生命週期	資料從生成、擷取、轉換、儲存到服務的完整流程，是本章的主軸。
資料科學 vs 資料工程	前者著重從資料萃取洞見與建模，後者著重建置可靠的資料流與基礎設施。
資料擷取 (Ingestion)	將資料從來源系統搬移到處理與儲存環境的過程，分批次與串流兩種模式。
ETL 與 ELT	擷取、轉換、載入的三個步驟；其順序的差異，反映了運算與儲存架構的演進。
變更資料擷取（CDC）	持續捕捉來源資料庫的變動，將其轉為資料流的技術。
資料治理	確保資料可被發現、安全可靠且權責清楚的制度與實務。
貫穿性要素	橫跨生命週期各階段的六個面向：安全、資料管理、DataOps、架構、編排與

核心概念	說明
	軟體工程。

表 2-1 本章核心概念一覽

2.1 資料科學與資料工程的分工

2.1.1 需求層級金字塔

要理解資料科學與資料工程的關係，有一個極為傳神的模型，稱為「資料科學需求層級金字塔」。它借用了心理學馬斯洛的需求層級概念：一個人得先滿足溫飽與安全，才談得上自我實現；同樣地，一個組織得先把資料的收集、儲存與清理做好，才談得上人工智慧。

層級（由下而上）	任務	對應本書章節
收集	資料的產生與蒐集	第 2 章、第 35 章
搬移與儲存	擷取、管線與可靠儲存	第 2、5、13 章
探索與轉換	清理、整合、資料準備	第 2、13 章
聚合與標記	彙整、特徵、標註	第 20 章
學習與最佳化	機器學習、深度學習	第 20、21 章
人工智慧	生成式 AI 與智慧決策	第 21、22 章

表 2-2 資料科學需求層級金字塔

這張金字塔的殘酷之處在於：越往上越吸引人，但越往下越決定成敗。許多組織急著蓋頂端的人工智慧，卻建在鬆軟的地基上，結果可想而知。資料工程師的工作，正是把金字塔的下半部打穩；資料科學家則在穩固的地基上，追求上半部的價值。

2.1.2 兩種角色的日常

為了讓分工更具體，我們對照兩種角色的典型工作內容。

面向	資料工程師	資料科學家
核心目標	建置可靠、可擴充的資料流	從資料中萃取洞見與預測
主要產出	資料管線、資料倉儲、資料集	分析報告、預測模型
關注重點	資料的可用性、品質與時效	模型的準確度與可解釋性
常用工具	SQL、Spark、Kafka、Airflow	Python、統計、機器學習框架
失敗的樣子	管線斷裂、資料延遲或遺漏	模型偏誤、結論不可靠

表 2-3 資料工程師與資料科學家的對照

叫獸提醒

這兩種角色沒有高下之分，而且界線正在模糊。近年出現的「分析工程師」（Analytics Engineer）就站在兩者之間，用工程的紀律去做分析的工作。對同學來說，好消息是：這門課兩邊都會教到。你不必現在就決定要當哪一種人，但你必須兩種語言都聽得懂。

2.2 資料生成與來源

生命週期的第一站，是資料的誕生。一個常被忽略的事實是：資料工程師通常無法控制資料來源——那些系統往往由別的團隊、甚至別的公司維護。你能做的，是理解每一種來源的脾氣，並為它的不完美做好準備。

2.2.1 常見的資料來源類型

- **交易系統 (OLTP)**：支撐日常營運的資料庫，如訂單、庫存、帳務。特性是寫入頻繁、要求即時一致，但不適合直接跑大型分析查詢。
- **應用程式日誌**：系統運行時產生的紀錄，如伺服器日誌、使用者操作軌跡。量大、半結構化，是行為分析與故障診斷的重要來源。
- **物聯網感測資料**：來自機台、車輛、環境感測器的時間序列資料。高頻、連續、且常帶有雜訊與遺漏，是本學程最貼近的資料型態。
- **第三方 API**：由外部服務提供的資料，如天氣、地圖、金融行情。取用方便但受限於對方的格式、額度與穩定性。
- **檔案與文件**：CSV、試算表、影像、PDF 等。格式多樣，常需大量前處理，是非結構化資料的大宗。

2.2.2 有界資料與無界資料

除了來源類型，還有一個更根本的分類會深深影響後續的架構選擇：資料是「有界」還是「無界」的。

有界資料 (Bounded Data) 是有明確起點與終點的一批資料，例如「上個月的所有交易紀錄」。它適合批次處理——把整批資料一次讀進來、算完、輸出結果。

無界資料 (Unbounded Data) 則是沒有終點、持續湧入的資料流，例如「產線感測器此刻仍在產生的訊號」。它必須以串流處理來面對——資料一到就處理，而非等待整批到齊。

這個區分之所以重要，是因為現實中的資料其實幾乎都是無界的——交易一筆接一筆發生、感測器從不停歇。所謂的「有界」，往往只是我們人為地在某個時間點切一刀，把無界的資料流切成一批批來處理。理解這一點，你才會明白為什麼近年的架構逐漸從批次倒向串流，這也是本書第陸篇的核心。

2.2.3 來源的關鍵特性

面對任一個資料來源，資料工程師都會先問幾個問題，以決定該如何對接。

- (一) **結構 (Schema)**：資料的欄位與型別是否固定？來源方會不會無預警地更動結構？
- (二) **產生速率**：資料以多快的速度產生？是穩定的涓流，還是脈衝式的暴衝？
- (三) **資料品質**：是否有遺漏、重複、格式不一或明顯錯誤的值？
- (四) **可存取性**：透過什麼方式取得？資料庫連線、API、檔案傳輸，還是訊息佇列？
- (五) **時效需求**：下游需要的是即時資料，還是每日更新一次即可？

其中「結構會不會無預警更動」是實務上最常見的災難來源。上游工程師改了一個欄位名稱，往往渾然不覺下游有一整條管線因此斷裂。這也是為什麼「結構演進」與「資料契約」近年成為資料工程的重要議題，本書第 4 章與第 32 章會再觸及。

2.3 資料擷取與整合

資料擷取（Ingestion）是把資料從來源搬到我們能處理它的地方。這一步聽起來只是「搬運」，卻是整條生命週期中最常出問題的環節——因為它是我們的系統與那些我們無法控制的來源之間的交界。

2.3.1 批次擷取與串流擷取

擷取的方式主要分為兩種，它們的差別呼應了前一節的有界與無界之分。

面向	批次擷取	串流擷取
搬移時機	定期整批搬移（如每小時、每日）	資料一產生即持續搬移
延遲	較高，取決於批次週期	低，可達秒級或毫秒級
適用情境	報表、歷史分析、模型訓練	即時告警、線上監控、動態決策
實作複雜度	較低，容錯較單純	較高，需處理亂序與故障復原
代表技術	排程工具、ETL 批次作業	Kafka、訊息佇列、串流引擎

表 2-4 批次擷取與串流擷取的比較

實務上，多數組織兩者並存：串流負責即時的部分，批次負責大量的歷史處理。如何在同一個架構中容納兩者，正是第 23 章要討論的 Lambda 與 Kappa 架構所要解決的問題。

2.3.2 推送與拉取

無論批次或串流，資料的移動方向都可分為兩種模式。「推送」（Push）是由來源方主動把資料送出，例如感測器持續發送訊號；「拉取」（Pull）則是由擷取方定時去來源方索取，例如每小時去資料庫查詢一次新增的紀錄。兩者各有適用場合：推送延遲低但來源方須負擔較多責任；拉取控制權在我方但可能造成來源負載。許多系統採用混合模式，例如以拉取為主、在特定事件時輔以推送。

2.3.3 變更資料擷取（CDC）

有一種特別重要的擷取技術值得單獨介紹：變更資料擷取（Change Data Capture, CDC）。它要解決的問題是——我如何持續地知道一個交易資料庫裡「有哪些資料改變了」，而不必每次都把整個資料庫重掃一遍？

CDC 的巧妙做法，是去讀取資料庫本身用來記錄每一筆變動的交易日誌（transaction log）。每當有一筆資料被新增、修改或刪除，這個變動就會被捕捉下來，轉成一則事件，送進資料流。如此一來，一個原本靜態的資料庫，就變成了一條源源不絕的資料流。

叫獸提醒

CDC 是一個「觀念轉換」的絕佳例子。傳統上我們把資料庫看成一張張靜止的表；但只要換個角度，把每一次的異動看成一則事件，資料庫就變成了資料流。這個「表與流本是一體兩面」的洞見，是現代資料架構的重要基石，第 25 章談 Kafka 與事件溯源時你會再遇到它。

2.4 資料轉換、儲存與服務

資料進來之後，還要經過轉換、找到棲身的儲存之所，最後才能被端上桌供人使用。這三個階段緊密相連，我們一併討論。

2.4.1 儲存：分層的思維

大數據的儲存很少是單一的一個地方，而是分層的。原始資料通常先原封不動地落在「著陸區」，保留最完整的細節以備日後回溯；清理與整合後的資料進入「整備區」，供大多數分析使用；高度彙整、可直接消費的資料則放在「服務區」。這種分層，常以「銅、銀、金」三層來比喻，是資料湖與 Lakehouse 的典型設計，本書第 30 章會深入探討。

2.4.2 轉換：從 ETL 到 ELT

轉換是把原始資料整理成可用形式的過程，包含清理、去重、格式統一、關聯合併與彙整。這裡有一個影響深遠的順序之爭：先轉換再載入（ETL），還是先載入再轉換（ELT）？

面向	ETL（擷取—轉換—載入）	ELT（擷取—載入—轉換）
處理順序	在載入前，先於中途轉換好	先把原始資料載入，再於儲存中轉換
時代背景	儲存昂貴、運算受限的早期	雲端儲存低廉、運算可彈性擴充的現代
彈性	較低，轉換邏輯改變須重跑	較高，原始資料仍在，可隨時重新轉換
適合場景	來源穩定、需求明確	探索性分析、需求多變

表 2-5 ETL 與 ELT 的比較

ETL 轉向 ELT，本質上是第 1 章談過的「儲存運算分離」在資料處理層面的具體展現。當儲存變得便宜、運算可以隨需擴充，我們就沒有理由在載入前急著丟棄原始細節了——先全部存下來，需要時再算，成為新的主流。

2.4.3 服務：讓資料產生價值

生命週期的終點，是把資料交付給真正的使用者。服務的形式主要有三種：

- **分析：**以報表、儀表板與商業智慧工具支援決策，是最常見的服務形式。
- **機器學習：**為模型訓練提供乾淨的特徵資料集，並在推論階段供給即時特徵。
- **反向 ETL：**把分析後的結果送回營運系統，例如將客戶分群結果寫回行銷平台，讓洞見真正驅動行動。

值得強調的是：如果資料最終沒有被任何人使用、沒有促成任何決策或行動，那麼前面所有的擷取、轉換與儲存都只是成本，而非價值。這呼應了第 1 章的第五個 V——價值，永遠是整條生命週期的終點。

2.5 資料治理與生命週期管理

2.5.1 資料治理的三大支柱

當資料的規模與參與的人數成長，混亂就會隨之而來：沒人知道某份資料在哪、由誰維護、能不能信任、可不可以給某人看。資料治理，就是為了對抗這種混亂而生的制度與實務。它可歸納為三大支柱。

- **可發現性：**讓對的人能找到對的資料，並理解它的來源與意義。核心工具是資料目錄與中繼資料。
- **安全與隱私：**確保資料只被授權者存取，並符合個資與法規要求。詳見第 33 章。
- **可問責：**每一份資料都有明確的擁有者，資料的來龍去脈（血緣）可被追溯。

2.5.2 中繼資料與資料目錄

支撐可發現性的關鍵，是中繼資料（Metadata）——也就是「描述資料的資料」。它記錄了一份資料的名稱、欄位意義、來源、更新頻率、擁有者與品質狀態。把全組織的中繼資料匯集起來、提供搜尋與瀏覽的系統，就是資料目錄（Data Catalog）。一個好的資料目錄，能讓新進成員在幾分鐘內找到並理解自己需要的資料，而不是花幾天去問遍整個團隊。這個主題在第 32 章會有完整討論。

2.5.3 資料的生命週期管理

資料和商品一樣，也有保鮮期與淘汰的時候。依存取頻率，常把資料分為三種溫度：

溫度	存取特性	儲存策略
熱資料	頻繁存取、要求快速回應	存於高速儲存，成本較高
溫資料	偶爾存取	存於一般儲存，成本適中
冷資料	極少存取、僅供備查或法遵	存於封存儲存，成本極低但取用較慢

表 2-6 資料的三種溫度與儲存策略

妥善的生命週期管理，會依溫度自動把資料在不同層之間搬移，並依法規與價值決定何時封存、何時刪除。這不只是成本問題——保留過期而無用的資料，本身就是一種風險（例如個資外洩的隱患）。因此「該刪就刪」也是資料治理的一部分。

2.6 角色、團隊與端到端工作流程

2.6.1 貫穿生命週期的六個要素

前面談的是生命週期的五個「階段」，它們像是一條輸送帶上的五站。但還有六個要素，並不屬於任何單一站，而是像地基一樣貫穿全程。

貫穿性要素	說明
安全	存取控制與資料保護，必須內建於每一個階段，而非事後補上。
資料管理	治理、品質、中繼資料與主資料的整體管理。
DataOps	借鏡 DevOps，以自動化、監控與快速迭代來營運資料管線（第 34 章）。
資料架構	各階段技術選型與整體結構的設計原則。
編排 (Orchestration)	協調各項任務的執行順序與相依關係，如 Airflow（第 13 章）。
軟體工程	以版本控制、測試與良好設計來對待資料程式碼（第 23 章）。

表 2-7 資料工程生命週期的六個貫穿性要素

2.6.2 一次端到端的走查

讓我們用一個具體例子，把整條生命週期串起來走一遍。假設要為一座工廠打造「設備健康儀表板」：

- 第 1 站。** 生成：機台上的振動與溫度感測器持續產生時間序列訊號（無界資料）。
- 第 2 站。** 擷取：邊緣裝置先做初步過濾，再透過訊息佇列以串流方式送入雲端。
- 第 3 站。** 儲存：原始訊號落入著陸區保存，同時寫入時間序列儲存供查詢。

第 4 站。 轉換：串流引擎即時計算滾動平均與異常指標，批次作業每日彙整趨勢。

第 5 站。 服務：儀表板呈現即時狀態，異常時自動告警，並將預測結果回寫維護系統。

同時，安全要素確保只有維護人員能看到資料，編排要素協調批次與串流作業的先後，DataOps 要素則在管線斷裂時發出警報。你看，第 1 章的六層架構、本章的五個階段與六個要素，講的其實是同一件事的不同切面。

2.6.3 團隊與成熟度

最後談談人。小型組織往往由一兩個人身兼數職，從擷取到分析全包；隨著規模成長，才逐漸分化出資料工程師、資料科學家、分析工程師、機器學習工程師與資料分析師等專職。一個團隊的「資料成熟度」，可粗略分為三階段：起步期（資料散落、以人工處理為主）、擴張期（建立標準化管線與倉儲）、以及最佳化期（高度自動化、以資料驅動決策）。認清自己的組織在哪個階段，比盲目追求最新技術更重要。

公式與流程：資料管線的容量法則

設計資料擷取管線時，一個核心問題是：我的緩衝區要開多大？我的管線來得及消化湧入的資料嗎？回答這個問題，可以借助排隊理論中一條優雅而通用的公式——利特爾法則（Little's Law）。

利特爾法則指出，在一個穩定的系統中，系統內平均的項目數量 L ，等於項目的平均到達速率 λ ，乘以每個項目停留在系統中的平均時間 W ：

$$L = \lambda \times W$$

式 2-1 利特爾法則

把它套到資料管線上： λ 是每秒湧入的事件數， W 是每個事件從進入到離開管線的平均時間， L 則是任一瞬間「還在管線裡」的事件數。這個 L ，正是你的緩衝區至少要能容納的量。

舉例來說，若一條串流管線每秒湧入 8,000 筆事件，每筆事件平均花 0.25 秒通過管線，則同時在管線中的事件數為：

$$L = 8,000 \times 0.25 = 2,000 \text{ 筆}$$

這告訴我們，緩衝區只要能穩定容納約兩千筆事件即可。但關鍵在於「穩定」二字——一旦下游變慢， W 上升， L 便會等比例膨脹。下表顯示同樣的到達速率下，處理時間惡化如何推升緩衝需求。

平均處理時間 W	到達速率 λ	在途事件數 L
0.25 秒（正常）	8,000 筆／秒	2,000 筆
0.50 秒（下游變慢）	8,000 筆／秒	4,000 筆
0.80 秒（尖峰壅塞）	8,000 筆／秒	6,400 筆
1.50 秒（嚴重積壓）	8,000 筆／秒	12,000 筆

表 2-8 處理時間惡化對緩衝需求的影響

叫獸提醒

利特爾法則的實務啟示是：緩衝區的大小不是憑感覺開的，而是算出來的；而且要按「最壞情況的 W 」來算，不能只看正常值。當下游處理不及、 W 持續上升而緩衝區被塞爆時，系統就會產生「背壓」（backpressure），迫使上游放慢或丟棄資料。理解這條法則，你才能在第

25 章設計 Kafka 的分區與消費者時，做出合理的容量規劃。

案例研討

案例一 性感職業背後的水電工

回到本章開頭的故事。一家新創公司高薪聘來了一位資料科學家，期待他用機器學習提升營收。三個月後，這位科學家幾乎沒有產出任何模型——不是他能力不足，而是他發現公司的資料四散在七個系統裡、格式互不相容、且沒有任何一份文件說明欄位的意義。

公司這才醒悟，補進了一位資料工程師。這位工程師花了兩個月，建立起一條把七個系統的資料匯整到單一倉儲的管線，並整理出一份資料目錄。此後，那位資料科學家的產出突然「爆發」了。這個案例的教訓很直接：沒有穩固的資料工程地基，再厲害的資料科學也是空中樓閣。這正是需求層級金字塔的真實寫照。

案例二 智慧工廠的設備健康儀表板

承接本章 2.6.2 的走查，讓我們看它在真實情境中的樣貌。某工廠為兩百台機台建置健康監測，資料以串流方式進入雲端，原始訊號保存於著陸區，串流引擎即時計算異常指標，批次作業每晚彙整趨勢，最後由儀表板呈現。

這個案例的價值，在於它同時用到了批次與串流：即時告警靠串流（低延遲），長期趨勢與模型訓練靠批次（大量資料）。它也完整走過了生命週期的五個階段，是理解本章最好的整合範例。第 35 章與第 37 章將以此為基礎，進一步深入邊緣運算與智慧製造的細節。

案例三 電商的 ELT 轉型

一家電商原本採用傳統 ETL：每晚把當日交易在中途伺服器上轉換好，再載入資料倉儲。問題是，每當行銷團隊想到一個新的分析角度，就得請工程師修改轉換邏輯、並重跑歷史資料，往往要等上數週。

他們改採 ELT 之後，先把原始交易資料原封不動載入雲端資料倉儲，轉換則在倉儲內按需進行。如今行銷團隊能自助地嘗試新的分析，因為原始資料一直都在，隨時可以用新的角度重新切分。這個轉變的背後，正是雲端「儲存便宜、運算彈性」所帶來的架構鬆綁。

案例四 讓資料庫開口說話的 CDC

某銀行的風控團隊需要「即時」偵測可疑交易，但交易資料寫在核心資料庫裡，而風控團隊被嚴格禁止直接查詢這個資料庫——因為任何額外的查詢都可能拖慢攸關營運的交易處理。

解法是導入變更資料擷取（CDC）。系統讀取核心資料庫的交易日誌，把每一筆新交易即時轉為事件，送入串流平台，風控模型再從串流上讀取。核心資料庫的負載完全不受影響，風控卻能在毫秒級收到每一筆交易。這個案例展示了 CDC 如何把一個「碰不得」的資料庫，安全地轉化為一條即時資料流——也預示了第 25 章事件驅動架構的威力。

實作活動

活動一 繪製一條資料生命週期

請選擇一個你熟悉的系統（例如社群平台、線上遊戲、或校園選課系統），描繪它的資料工程生命週期。

- 步驟 1. 找出至少三個資料來源，並判斷各屬有界或無界資料。
- 步驟 2. 畫出從生成、擷取、儲存、轉換到服務的流程圖。
- 步驟 3. 在圖上標註：哪些環節適合批次、哪些適合串流？
- 步驟 4. 指出這條生命週期中，你認為最可能出問題的一站，並說明理由。

活動二 動手做一條小型 ETL/ELT

請以 Python (pandas) 實作一條迷你資料管線，體會轉換的實際內容。

- 步驟 1. 準備一份帶有髒資料的 CSV（含遺漏值、重複列、日期格式不一）。
- 步驟 2. 撰寫程式：讀入資料 → 去除重複 → 統一日期格式 → 填補或剔除遺漏值。
- 步驟 3. 將清理後的資料輸出為 Parquet 格式，並比較它與原 CSV 的檔案大小。
- 步驟 4. 反思：你剛才做的是 ETL 還是 ELT？若把清理步驟改到載入之後，會有何不同？

完成後你會發現，那個「去除重複、統一格式、處理遺漏」的步驟，就是資料科學家口中最耗時的那八成工作。

活動三 設計一份資料目錄的欄位

請為活動二清理後的資料集，設計一份中繼資料紀錄。至少包含：資料集名稱、擁有者、來源、更新頻率、每個欄位的意義與型別、以及已知的品質問題。完成後與同組同學交換，請對方僅憑你的中繼資料，判斷這份資料能否用於某個特定分析——藉此體會可發現性的價值。

延伸閱讀

- 《Fundamentals of Data Engineering》：Reis 與 Housley 著。本章的主要依據，第 1 至 3 章對資料工程生命週期與貫穿性要素有最完整的闡述。
- 《Designing Data-Intensive Applications》第 3、11 章：Kleppmann 與 Riccomini 著。第 3 章談儲存引擎，第 11 章談串流處理與 CDC，與本章 2.3、2.4 節密切相關。
- 資料科學需求層級金字塔 (Monica Rogati)：一篇廣為引用的部落格文章，以馬斯洛需求層級比喻資料科學的基礎工程，是理解 2.1 節的最佳補充。
- 《DMBOK：資料管理知識體系指南》：資料管理協會 (DAMA) 出版，是資料治理與資料管理領域的權威參考，可作為 2.5 節的延伸。

本章小結

1. 資料工程生命週期包含生成、擷取、轉換、儲存與服務五個階段，並有安全、資料管理、DataOps、架構、編排與軟體工程六個貫穿性要素橫跨全程。
2. 資料科學需求層級金字塔說明：越往上（人工智慧）越吸引人，越往下（收集與儲存）越決定成敗；資料工程師打地基，資料科學家追求頂端價值，兩者缺一不可。
3. 常見資料來源包括交易系統、應用日誌、物聯網感測、第三方 API 與檔案；資料又可分為有界（適合批次）與無界（適合串流），而現實中的資料幾乎都是無界的。
4. 擷取分批次與串流兩種模式，並有推送與拉取之別；變更資料擷取 (CDC) 透過讀取交易日誌，把靜態資料庫轉化為即時資料流。

5. 儲存採分層設計（著陸／整備／服務）；轉換的順序由 ETL 演進為 ELT，反映了雲端儲存運算分離的影響；服務則以分析、機器學習與反向 ETL 三種形式交付價值。
6. 資料治理立基於可發現性、安全與可問責三大支柱，中繼資料與資料目錄是可發現性的核心；資料依熱、溫、冷三種溫度進行生命週期管理，該保存則保存、該刪除則刪除。
7. 利特爾法則（ $L = \lambda \times W$ ）指出，管線中的在途資料量等於到達速率乘以停留時間；緩衝區容量應按最壞情況的處理時間來規劃，以避免背壓。

習題

一、觀念題

1. 請以需求層級金字塔說明，為何一個組織不宜在資料基礎薄弱時，就急於導入人工智慧。
2. 何謂有界資料與無界資料？請各舉一例，並說明它們分別適合批次或串流處理的理由。
3. 請說明變更資料擷取（CDC）的運作原理，以及它相較於「定時重掃整個資料庫」的優點。
4. ETL 與 ELT 的差異為何？為什麼雲端時代的架構逐漸倒向 ELT？

二、計算題

1. 某串流管線每秒湧入 12,000 筆事件，每筆事件平均停留 0.3 秒。請以利特爾法則計算在途事件數；若下游變慢使停留時間升為 0.9 秒，在途事件數變為多少？若緩衝區上限為 8,000 筆，此時是否會發生背壓？
2. 某工廠每日產生 51.84 GB 的感測資料，規定熱資料保留 7 天、溫資料保留 90 天、冷資料保留 3 年。假設資料在對應期間內皆須保存，請估算熱、溫、冷三層各需的儲存容量（以 GB 或 TB 表示，冷資料以 365 天／年計）。

三、思考與討論

1. 有人說「資料科學家才是主角，資料工程師只是支援」。請根據本章內容，提出你的看法與至少兩點論據。
2. 請以你熟悉的一個資料來源為例，說明它可能在「結構無預警更動」上帶來什麼風險，並提出至少一種防範做法。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

應以金字塔「由下而上、層層相依」的性質作答。人工智慧位於頂端，倚賴其下的收集、儲存、清理、彙整等層級皆穩固。若基礎薄弱——資料散落、品質低劣、無法穩定供給——則模型即使演算法先進，也會因輸入不可靠而失效（呼應第 1 章的 Veracity）。可補充案例一：補進資料工程師、打穩地基後，資料科學的產出才得以爆發。

第 2 題

有界資料有明確起訖，如「上月所有交易」，因整批到齊，適合批次處理，可一次讀入算完。無界資料持續湧入而無終點，如「感測器此刻的訊號」，需串流處理，資料一到即處理。理由的核心在

於：批次處理需等待整批到齊，對無界資料而言等待永遠不會結束；反之，對已完整的有界資料採串流則徒增複雜度。可補充：現實資料多為無界，「有界」常是人為切分的結果。

第 3 題

CDC 透過讀取資料庫的交易日誌來捕捉每一筆新增、修改、刪除，並轉為事件送入資料流。相較於定時重掃整個資料庫，其優點至少有三：其一，只擷取「變動的部分」，資料量與運算量都大幅降低；其二，幾乎不增加來源資料庫的查詢負載（讀日誌而非查表），對營運系統友善；其三，延遲極低，可達近即時。可補充案例四的銀行風控情境作為佐證。

第 4 題

ETL 在載入前先於中途轉換，適合儲存昂貴、運算受限的早期環境；ELT 先把原始資料載入，再於儲存中按需轉換。倒向 ELT 的原因與第 1 章「儲存運算分離」一脈相承：雲端儲存低廉，故無須在載入前急於丟棄原始細節；雲端運算可彈性擴充，故可負擔在倉儲內大規模轉換。ELT 的額外好處是保留原始資料，需求改變時可用新角度重新轉換（呼應案例三電商情境）。

二、計算題

第 1 題

套用利特爾法則 $L = \lambda \times W$ 。

- (1) 正常情況： $L = 12,000 \times 0.3 = 3,600$ 筆。
- (2) 下游變慢： $L = 12,000 \times 0.9 = 10,800$ 筆。
- (3) 判斷背壓：10,800 筆已超過緩衝區上限 8,000 筆，故會發生背壓，系統將迫使上游放慢或丟棄資料。

延伸思考：到達速率不變，僅因停留時間由 0.3 秒升為 0.9 秒（三倍），在途量即成長為三倍。這說明監控 W （處理延遲）與監控 λ （流量）同等重要——延遲的惡化會直接轉化為緩衝壓力。

第 2 題

以每日 51.84 GB 乘上各層保留天數即可。

- (1) 熱資料（7 天）： $51.84 \times 7 = 362.88 \text{ GB} \approx 0.35 \text{ TB}$ 。
- (2) 溫資料（90 天）： $51.84 \times 90 = 4,665.6 \text{ GB} \approx 4.67 \text{ TB}$ 。
- (3) 冷資料（3 年，1,095 天）： $51.84 \times 1,095 = 56,764.8 \text{ GB} \approx 56.76 \text{ TB}$ 。

延伸思考：冷資料量遠大於熱資料（約 156 倍），卻極少被存取。這正是分層儲存的價值所在——把佔比最大、存取最少的冷資料放在成本最低的封存層，可大幅降低總儲存成本。若三層都用高速儲存，將是巨大的浪費。

三、思考與討論

第 1 題

本題訓練論證能力，重點在於能否以本章概念支撐立場。較周延的回答會指出「主角／配角」這個框架本身有問題：兩種角色是相依關係而非主從關係。可援引需求層級金字塔（沒有下層地基，上層無從發揮）、案例一（資料科學產出取決於資料工程是否到位）、以及「分析工程師」等混合角色的興起（界線正在模糊）。若學生選擇支持原說法，也應誠實面對地基問題，並提出如何在資料科學主導下確保工程品質。

第 2 題

應包含三部分：其一，指出所選來源可能如何無預警更動結構（例如上游改欄位名、改型別、改單位）；其二，說明其下游衝擊（管線斷裂、數值錯誤而未被察覺，後者更危險）；其三，提出防範做法，如：建立資料契約明訂結構、在擷取端加入結構驗證與告警、對關鍵欄位設定監控、或保留原始資料以便回溯修正。可連結 2.2.3 與第 4、32 章。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 2 章 資料科學與資料工程生命週期

台灣自編大學教科書
@prof
大數據分析與雲端運算

第壹篇 導論與基礎

Foundations

第 3 章

分散式系統原理

Principles of Distributed Systems

機器人叫獸（李世淵） 編著

叫獸開講

一九九九年，一位任職於昇陽電腦（Sun Microsystems）的工程師彼得·戴奇（Peter Deutsch）整理出一份清單，叫做「分散式運算的八個誤解」。這份清單列出了工程師在設計分散式系統時，最常誤以為理所當然、但其實完全不成立的假設。第一條就是：「網路是可靠的」。

這句話聽起來像廢話——網路當然會斷啊，誰不知道？但問題在於，我們寫程式時，往往不自覺地假設它不會斷。你呼叫遠端的一個服務，程式碼看起來就跟呼叫本機的一個函式一模一樣；於是你很自然地以為，它一定會回應、一定會準時、而且回應一定正確。分散式系統會咬人的地方，正是這些「一定」。

清單上的其他七條同樣扎心：延遲是零、頻寬無限、網路很安全、拓撲不會變、只有一個管理者、傳輸成本是零、網路是同質的。每一條，都是把單機時代的直覺，錯誤地套用到分散式世界的結果。而過去二十幾年，無數當機事故、資料遺失與詭異的錯誤，追根究柢都能對應到這八條之一。

同學，第 1 章我告訴你「假設機器一定會壞」；這一章我們要把這句話講得更透徹。當你很多台機器用會斷的網路串起來，你面對的不再是「機器會不會壞」的問題，而是「當它壞掉、當網路斷掉、當訊息遲到或重複到達時，我的系統還能不能給出正確的結果」。這就是分散式系統原理的核心，也是後面所有儲存、運算、雲端技術的共同地基。

學習目標

研讀完本章之後，你應該能夠：

1. 說明分散式系統相較於單機系統的根本差異，並列舉分散式運算的常見誤解。
2. 以可靠性、可擴充性與可維護性三個面向，評估一個資料系統的設計品質。
3. 區分延遲與吞吐量，並正確解讀百分位延遲（如 p99）的意義。
4. 說明 CAP 定理的內涵，並在分割發生時判斷系統該偏向一致性或可用性。
5. 辨識常見的故障模型，並說明複製與容錯如何提升系統的可靠性。
6. 解釋水平擴充與資料分區的基本概念，作為後續章節的預備。

核心概念

核心概念	說明
分散式系統	由多台以網路互連的機器協同運作、對外表現如同單一系統的架構。
可靠性 (Reliability)	系統在遭遇故障時仍能正確運作的能力。
可擴充性 (Scalability)	系統面對負載成長時，維持效能的能力。
可維護性 (Maintainability)	系統易於營運、理解與演進的程度。
延遲與吞吐量	延遲是單一請求的回應時間；吞吐量是單位時間可處理的請求數。

核心概念	說明
CAP 定理	在網路分割時，系統無法同時確保一致性與可用性，必須取捨。
容錯 (Fault Tolerance)	透過複製與備援，使部分元件故障不致造成整體失效的設計。

表 3-1 本章核心概念一覽

3.1 為何需要分散式系統：規模與極限

3.1.1 三個逼你走向分散式的理由

沒有人是因為喜歡複雜才選擇分散式系統的——恰恰相反，分散式帶來的麻煩多如牛毛。人們之所以還是走上這條路，是因為被以下三種力量之一（或全部）逼到了牆角。

- **容量超出單機極限**：當資料量或運算量超過任何一台機器所能負荷，就只能把工作分散到多台機器上。這是第 1 章水平擴充的直接後果。
- **容錯與高可用**：若整個服務只跑在一台機器上，那台機器一停，服務就全斷。要讓服務在硬體故障時仍持續運作，就必須有多台機器互為備援。
- **降低延遲**：使用者散布全球，若所有請求都要回到同一個資料中心，遠方的使用者將飽受延遲之苦。把機器部署到靠近使用者的多個地點，能大幅縮短回應時間。

值得注意的是，這三個理由對應到本章接下來要談的三個品質面向：容量對應可擴充性、容錯對應可靠性、而它們共同影響系統的可維護性。

3.1.2 分散式運算的八個誤解

承接叫獸開講的故事，我們把那八個誤解完整列出。它們不是歷史趣談，而是一份你每次設計系統時都該拿出來核對的檢查清單。

#	誤解（人們錯以為……）	真相與後果
1	網路是可靠的	封包會遺失、連線會中斷；必須設計重試與逾時機制。
2	延遲是零	跨節點通訊有實質延遲；頻繁的遠端呼叫會拖垮效能。
3	頻寬是無限的	大量資料傳輸受頻寬限制；須考量資料在地性。
4	網路是安全的	傳輸可能被竊聽或竄改；須加密與驗證。
5	網路拓撲不會變	節點會增減、路由會改變；系統須能動態適應。
6	只有一位管理者	多方共管導致設定不一致；須有明確的協調機制。
7	傳輸成本是零	序列化與網路傳輸都有成本；須權衡通訊頻率。
8	網路是同質的	節點的硬體與效能各異；不能假設處處相同。

表 3-2 分散式運算的八個誤解

叫獸提醒

這八條的共同根源只有一句話：分散式系統裡，「遠端」和「本機」是兩回事，但程式碼常常讓它們長得一模一樣。一個寫成 `result = service.getData()` 的呼叫，本機版本幾乎不會失敗，遠端版本卻可能逾時、可能收到重複回應、可能永遠等不到答案。優秀的分散式工程師，

會在心裡替每一個遠端呼叫都畫上一個問號。

3.2 可靠性、可擴充性與可維護性

如何評斷一個資料系統設計得好不好？《Designing Data-Intensive Applications》提出的三個面向，已成為業界通用的語言：可靠性、可擴充性與可維護性。本章其餘各節，其實都是在深化這三個字。

3.2.1 可靠性：容忍故障

可靠性的白話定義是：即使出了差錯，系統仍能繼續正確運作。這裡的「差錯」有個專門的字，叫做「故障」（fault）——指某個元件偏離了它應有的行為。能夠預期並容忍故障的系統，稱為「容錯」系統。

要特別區分兩個詞：故障（fault）是單一元件出問題，失效（failure）則是整個系統停止對外提供服務。容錯設計的精髓，就是不讓局部的故障演變成整體的失效。故障的來源通常分為三類：

- **硬體故障**：硬碟損壞、記憶體錯誤、電源中斷。對策是備援與複製——讓任一硬體壞掉都有替補。
- **軟體錯誤**：程式臭蟲、資源耗盡、連鎖故障。這類錯誤往往同時擊倒多個節點，比硬體故障更棘手。
- **人為疏失**：設定錯誤、誤刪資料、操作失當。研究顯示，人為疏失是線上服務故障的主因之一。

有趣的是，有些系統會「故意」製造故障來驗證容錯能力，最著名的是 Netflix 的「混沌猴」（Chaos Monkey）——它會在正式環境中隨機關掉伺服器，逼迫團隊確保系統真的禁得起故障。這種「平時就練習受傷」的思維，正是可靠性工程的精髓。

3.2.2 可擴充性：應付成長

可擴充性描述的是：當負載增加時，系統維持效能的能力。要談可擴充性，得先能「描述負載」與「描述效能」。

負載可用「負載參數」來刻畫，例如每秒請求數、讀寫比例、同時上線人數等。效能則有兩種問法：其一，負載增加而資源不變時，效能會如何下降？其二，要維持效能不變，需要增加多少資源？回答這兩問，才算真正掌握了一個系統的可擴充性，而不是空泛地說它「可以擴充」。

3.2.3 可維護性：讓系統活得久

一套系統的成本，多數並非發生在初次開發，而是在漫長的後續維護。可維護性可再細分為三個面向：

- **可營運性**：讓維運團隊容易保持系統順暢運行，例如良好的監控與清楚的告警。
- **簡單性**：控制複雜度，讓新成員容易理解系統，降低誤解與出錯的機會。
- **可演進性**：讓系統容易因應新需求而修改，也稱為可延展性或可修改性。

這三者常被初學者忽略，因為它們在專案初期看不出價值。但一套將運行五年、經手數十位工程師的系統，可維護性往往才是決定成敗的關鍵。這也是為什麼本書第捌篇要專門討論治理與維運。

3.3 延遲、吞吐量與效能度量

3.3.1 兩個容易混淆的詞

延遲 (Latency) 與吞吐量 (Throughput) 是效能討論中最基本、卻也最常被混為一談的兩個詞。

延遲是「單一請求從發出到收到回應所花的時間」，關心的是「快不快」；吞吐量是「單位時間內能處理的請求數量」，關心的是「多不多」。一個常見的比喻是高速公路：延遲是一輛車從起點開到終點要多久，吞吐量是這條路每小時能通過多少輛車。兩者未必同向——拓寬車道能提升吞吐量，卻不會讓單一輛車開得更快。

面向	延遲 (Latency)	吞吐量 (Throughput)
定義	單一請求的回應時間	單位時間處理的請求數
關心	快不快	多不多
典型單位	毫秒 (ms)	每秒請求數 (RPS)、每秒筆數
改善手段	縮短處理路徑、就近部署	增加平行度、擴充節點

表 3-3 延遲與吞吐量的對照

3.3.2 為什麼要看百分位，而非平均

談延遲時，用「平均值」是危險的。因為少數極慢的請求會被大量正常請求稀釋，讓平均值看起來很漂亮，卻掩蓋了部分使用者正在受苦的事實。因此業界改用「百分位延遲」。

所謂 p95 延遲，是指把所有請求的延遲由小到大排序後，第 95 個百分位所在的值——也就是說，有 95% 的請求比它快，有 5% 的請求比它慢。同理，p99、p999（第 99.9 百分位）描述的是更極端的尾端。這些高百分位又稱為「尾端延遲」（tail latency）。

指標	意義	為何重要
p50 (中位數)	半數請求快於此值	反映典型使用者的體驗
p95	95% 請求快於此值	開始顯露少數慢請求
p99	99% 請求快於此值	常用的服務品質保證門檻
p999	99.9% 請求快於此值	揭露最極端的壞體驗

表 3-4 常見的百分位延遲指標

叫獸提醒

有一個違反直覺卻極重要的現象：越是你最重要的客戶，越可能落在尾端延遲裡。因為重度使用者往往資料最多、請求最複雜，處理起來最慢。所以只看平均延遲的系統，等於系統性地忽略了最有價值的那群人。這也是為什麼大型服務的服務等級目標 (SLO) 幾乎都以 p99 而非平均值來訂定。

3.4 一致性、可用性與 CAP 定理

3.4.1 一個無法迴避的取捨

這一節是整章的核心。當資料被複製到多台機器上，一個根本的難題就浮現了：如果此刻網路把這些機器切成兩半，讓它們無法互通，你的系統該怎麼辦？

CAP 定理用三個字母描述這個處境。C 是一致性（Consistency）：每次讀取都能拿到最新的寫入結果。A 是可用性（Availability）：每個請求都能得到回應（不論是否為最新）。P 是分割容忍（Partition Tolerance）：即使節點間的網路斷裂，系統仍能運作。

CAP 定理指出：當網路分割（P）發生時，你無法同時保住 C 與 A，只能二選一。這不是工程能力的問題，而是邏輯上的必然。

3.4.2 為什麼只能二選一

用一個具體情境就能理解。假設一份資料同時存在甲、乙兩台機器上，此刻它們之間的網路斷了。這時有人向甲寫入了新值，接著另一個人向乙讀取這份資料。乙該怎麼回應？

它只有兩條路：第一條，乙照樣回覆它手上的舊值——這保住了可用性（有回應），卻犧牲了一致性（回的不是最新值）。第二條，乙拒絕回應、等到網路恢復、確認自己拿到最新值後才回覆——這保住了一致性，卻犧牲了可用性（在此期間無法服務）。你看，網路一斷，就沒有兩全其美的第三條路。

取捨	分割時的行為	適用場景
偏向一致性（CP）	寧可拒絕服務，也不回舊資料	金融交易、庫存扣減
偏向可用性（AP）	寧可回舊資料，也要保持回應	社群動態、商品瀏覽

表 3-5 CAP 取捨的兩種選擇

3.4.3 對 CAP 的常見誤解

CAP 定理雖然有名，卻也常被誤用。以下澄清三個常見的誤解。

- （一）誤解一：「三選二」。正確的理解是——分割容忍（P）在分散式系統中無可迴避（網路一定會斷），所以真正的選擇只在 C 與 A 之間，是「二選一」而非「三選二」。
- （二）誤解二：「非黑即白」。實務上一致性與可用性都是光譜，系統可在不同操作、不同資料上採取不同策略，而非全域地只做一種選擇。
- （三）誤解三：「平時也要取捨」。CAP 的取捨只在網路分割發生時才被觸發；網路正常時，系統大可同時提供強一致與高可用。

因此，現代較精緻的說法是 PACELC：若發生分割（P），在 A 與 C 間取捨；否則（E, Else），在延遲（L）與一致性（C）間取捨。這更完整地描述了真實系統的處境。本書第 7、8 章談複製與共識時，會把這些取捨落實到具體演算法。

3.5 故障模型與容錯機制

3.5.1 故障的種類

要設計容錯，得先弄清楚「會遇到什麼樣的錯」。分散式系統的故障，依嚴重程度可分為幾種模型。

- **當機故障**：節點直接停止運作，不再回應。這是最單純、最容易處理的一種。
- **遺漏故障**：節點漏收或漏送某些訊息，但本身仍在運行。
- **時序故障**：節點回應了，但太慢或在錯誤的時間點回應，常肇因於時脈不同步。

- **拜占庭故障**：節點以任意、甚至看似惡意的方式行為，可能傳送矛盾或錯誤的訊息。這是最難防範的一種。

其中「拜占庭故障」一詞源自「拜占庭將軍問題」：幾位將軍必須透過信使協調進攻，但其中可能有叛徒故意傳假消息，如何仍能達成一致？這個問題看似抽象，卻正是區塊鏈要解決的核心難題——在互不信任的節點間達成共識，本書第 8 章會深入探討。

3.5.2 複製：容錯的基石

對抗故障最基本的武器是複製（Replication）——把同一份資料保存在多台機器上。如此一來，任何一台機器壞掉，資料都不會遺失，服務也能由其他副本接手。複製看似簡單，卻牽動一連串難題：多份副本之間如何保持同步？寫入該等所有副本都確認，還是只要一部分確認即可？當副本們的資料出現分歧時，該以誰為準？

這些問題沒有標準答案，只有取捨——而取捨的核心，正是前一節的 CAP。要求所有副本同步（強一致）會犧牲可用性與延遲；允許副本暫時分歧（最終一致）則換來高可用與低延遲。第 7 章將把複製的各種策略——主從、多主、無主——一一攤開細談。

3.5.3 逾時與重試的兩難

既然「網路是可靠的」是誤解，程式就必須處理「請求送出去卻遲遲沒有回應」的情況。標準做法是設定逾時（timeout）與重試（retry）：等太久沒回應就重送。但這裡藏著一個經典兩難——你怎麼知道對方是「真的掛了」，還是「只是慢了一點」？

若對方其實只是慢，你卻判定它掛了而重送請求，就可能造成同一個操作被執行兩次（例如重複扣款）。這引出了「冪等性」（idempotency）的重要概念：把操作設計成「執行一次與執行多次的結果相同」，如此即使重試也不會出錯。冪等性是分散式系統設計中反覆出現的救命觀念，值得你現在就牢牢記住。

3.6 水平擴充與資料分區概念

3.6.1 複製與分區：兩種正交的手段

面對成長，分散式系統有兩種基本手段，它們互相獨立、也常常併用。

複製（Replication）是把同一份資料放到多台機器上，主要目的是容錯與提升讀取效能——多個副本可以分攤讀取請求。分區（Partitioning，又稱分片 Sharding）則是把一份大資料切成若干塊，每台機器只負責其中一塊，主要目的是突破單機的容量與寫入極限。

兩者常被同時採用：先把資料分區到多台機器以擴充容量，再為每個分區建立複製以確保容錯。這正是絕大多數大型資料系統的基本骨架。

3.6.2 資料怎麼切

分區的關鍵決策是「依什麼把資料切開」，常見有兩種策略。

- **依範圍分區**：依鍵值的區間切分，例如姓氏 A 至 M 放一台、N 至 Z 放另一台。優點是範圍查詢有效率，缺點是容易造成負載不均（熱點）。
- **依雜湊分區**：先對鍵值計算雜湊值，再依雜湊決定落點。優點是分布均勻，缺點是失去了範圍查詢的效率。

分區最怕的是「熱點」(hotspot)——大量請求集中湧向同一個分區，使它不堪負荷，其餘分區卻閒置。如何選擇好的分區鍵以避免熱點，是一門需要理解資料存取模式的學問。本書第 7 章會完整討論分區策略、再平衡與請求路由。

3.6.3 承先啟後

至此，第壹篇的系統原理已備。你已經理解：為何要分散 (3.1)、如何評斷好壞 (3.2)、如何度量效能 (3.3)、分割時的根本取捨 (3.4)、如何面對故障 (3.5)，以及擴充的兩種手段 (3.6)。這些原理不會憑空消失——接下來第貳篇談分散式儲存與資料庫，會把複製、分區、一致性一一化為具體技術；第參篇談批次運算，會讓你看到容錯的思想如何落實在 MapReduce 與 Spark 之中。原理是根，技術是葉；根扎得深，葉才長得穩。

公式與流程：複製如何提升可用性

我們常說「複製能提升可用性」，但到底能提升多少？這可以用機率精確地算出來，而算完的結果往往令人驚訝。

假設單一節點在任一時刻正常運作的機率（可用度）為 a 。若一項服務需要「至少一個節點存活」才能運作，且部署了 n 個彼此獨立的節點，則因為「全部同時故障」的機率為 $(1 - a)^n$ ，整體可用度為：

$$A = 1 - (1 - a)^n$$

式 3-1 n 個獨立節點的整體可用度（至少一個存活）

這條公式的威力在於指數項。假設單一節點的可用度為 99%（即每一百小時約有一小時故障），我們來看增加節點數如何改變整體可用度。

節點數 n	整體可用度 A	約略的年停機時間
1（單機）	99%	約 3.65 天／年
2	99.99%	約 53 分鐘／年
3	99.9999%	約 32 秒／年
4	99.999999%	約 0.3 秒／年

表 3-6 節點數與整體可用度（單節點可用度 99%）

僅僅從一台增加到兩台，年停機時間就從三天半驟降到不到一小時；三台更是達到俗稱「六個九」的等級。這就是複製的威力——可用度隨節點數指數提升。

叫獸提醒

但請注意公式裡藏著一個關鍵的前提：「彼此獨立」。現實中節點往往並不獨立——它們可能共用同一組電源、同一台網路交換器、同一份有臭蟲的軟體。一旦共用的東西出事，多個節點會「一起」故障， $(1 - a)^n$ 的美好假設就瞬間瓦解。這也是為什麼雲端服務要把副本分散到不同的「可用區」，正是為了降低這種共同故障。算式會給你信心，但工程師要對它的前提保持警覺。

案例研討

案例一 被自己嚇大的 Netflix

串流龍頭 Netflix 把整個服務建在公有雲上，橫跨成千上萬台伺服器。這麼大的規模，硬體故障不是偶發事件，而是每天都在發生的日常。傳統思維會想方設法「避免」故障，但 Netflix 反其道而行，開發了「混沌猴」——一隻會在正式環境中隨機關掉伺服器的程式。

這聽起來像自殘，背後卻是深刻的可靠性哲學：與其祈禱故障不要來，不如主動、頻繁地製造小故障，逼迫每一個服務都必須具備容錯能力，否則平時就會被混沌猴打趴。如此一來，當真正的大規模故障來襲時，系統早已身經百戰。這個案例完美呼應了 3.2.1 節——可靠性不是靠運氣，而是靠平時就練習受傷。

案例二 購物車與庫存的不同抉擇

同一個電商網站，內部不同的功能對 CAP 做出了相反的選擇，這正說明了 3.4 節「取捨並非全域」的道理。

商品瀏覽與購物車偏向可用性（AP）：就算某個資料中心與其他節點失聯，也要讓使用者能繼續瀏覽、能把商品加入購物車——即使看到的庫存數字可能稍微過時，也遠比讓網站無法使用要好。反之，最後結帳扣減庫存的那一刻則偏向一致性（CP）：系統寧可讓使用者稍等，也絕不能因為資料不一致而把同一件僅存的商品賣給兩個人。同一家公司，因功能性質不同而做出不同取捨，這才是 CAP 在實務中的真實樣貌。

案例三 一次逾時重試引發的重複扣款

某支付系統曾發生一起事故：使用者付款後，因網路壅塞，付款服務遲遲未收到銀行的成功回應，於是依設定自動重試，又送出一扣款請求。結果銀行其實兩次都成功了——使用者被扣了兩次款。

問題的根源，正是 3.5.3 節的兩難：付款服務無法分辨「銀行真的沒收到」與「銀行收到了但回應遲到」。事後的修正之道，是為每一筆交易賦予唯一的識別碼，讓銀行端具備冪等性——同一個交易識別碼即使收到多次，也只扣款一次。這個案例是「冪等性」為何是分散式系統救命觀念的最佳註腳。

案例四 把副本放對地方

一家新創把資料庫的三個副本都部署在同一個機房，自認為「有三副本，很安全」。直到某天，那個機房因空調故障而整體斷電，三個副本同時失聯，服務全斷數小時。

這個慘痛教訓，正是式 3-1 那個「彼此獨立」前提的反面教材。三個副本雖然數量足夠，卻共用了同一組電源與空調，因此並不獨立——共同故障一發生，指數級的可靠性瞬間歸零。事後他們把三個副本分散到三個不同的可用區，才真正達到當初以為早就擁有的可靠性。工程的細節，往往就藏在這些「看似等價、實則天差地別」的部署決策裡。

實作活動

活動一 為你的系統做 CAP 抉擇

請選定一個你熟悉的應用（如即時聊天、線上投票、共筆文件），思考它在網路分割時的取捨。

- 步驟 1. 找出應用中至少三種不同的操作（如讀取、寫入、刪除）。
- 步驟 2. 為每一種操作判斷：分割時應偏向一致性還是可用性？

步驟 3. 說明你的理由——若做了相反的選擇，使用者會遭遇什麼？

步驟 4. 討論：是否所有操作都該做相同選擇？為什麼？

活動二 用程式感受尾端延遲

請以 Python 實驗，親眼看見平均值如何掩蓋尾端延遲。

步驟 1. 產生一萬筆模擬延遲數據：其中 95% 落在 10~50 毫秒，另外 5% 落在 500~2000 毫秒。

步驟 2. 計算這組數據的平均值、p50、p95、p99。

步驟 3. 比較平均值與 p99 的差距，說明為何「只看平均」會誤導。

步驟 4. 畫出延遲的分布直方圖，觀察尾端的那一小撮。

完成後你會發現，那 5% 的慢請求幾乎不影響平均值，卻主宰了 p99——而 p99 才是最有價值客戶的真實體驗。

活動三 計算你的可用度目標

請運用式 3-1 做一次可用度規劃。假設你負責一個要求「五個九」（99.999%，年停機約 5 分鐘）可用度的服務，而單一節點的可用度為 99.5%。請計算：至少需要幾個彼此獨立的節點，才能達成目標？並說明在真實部署時，你會採取哪些措施來確保這些節點盡可能獨立。

延伸閱讀

- 《**Designing Data-Intensive Applications**》第 1、5、6、8、9 章：Kleppmann 與 Riccomini 著。本章的主要依據：第 1 章談三大品質面向，第 5、6 章談複製與分區，第 8、9 章談分散式系統的麻煩與一致性。
- 〈分散式運算的八個誤解〉：由 Peter Deutsch 等人提出，網路上有諸多解說版本，是理解 3.1 節的經典起點。
- **CAP 定理原始論述與 PACELC 延伸**：Eric Brewer 提出 CAP，Daniel Abadi 提出 PACELC。兩者搭配閱讀，可完整掌握 3.4 節的取捨。
- **Google 網站可靠性工程 (SRE)**：Google 出版的 SRE 專書，對可靠性、SLO 與尾端延遲有大量實務討論，是 3.2、3.3 節的絕佳補充。

本章小結

1. 人們選擇分散式系統，是被容量超出單機、需要容錯高可用、以及降低延遲三種力量之一所逼；隨之而來的複雜，源於分散式運算的八個誤解，其根本是「遠端不等於本機」。
2. 評斷資料系統可用可靠性、可擴充性與可維護性三個面向；可靠性靠容忍硬體故障、軟體錯誤與人為疏失，並區分局部故障與整體失效。
3. 延遲關心單一請求快不快，吞吐量關心單位時間處理多少；描述延遲應使用百分位（p95、p99）而非平均，因為尾端延遲往往落在最有價值的客戶身上。
4. CAP 定理指出：網路分割時無法同時確保一致性與可用性，只能二選一；此取捨可依操作與資料而異，且僅在分割發生時觸發，PACELC 進一步補上了無分割時的延遲與一致性取捨。
5. 故障依嚴重程度分為當機、遺漏、時序與拜占庭四種模型；複製是容錯的基石，而逾時重試會引發「慢或掛」的兩難，須以冪等性來化解。
6. 複製與分區是兩種正交的擴充手段：複製提升容錯與讀取效能，分區突破容量與寫入極限，兩者常併用；分區須慎選鍵值以避免熱點。

7. 複製對可用度的提升呈指數級 ($A = 1 - (1 - a)^n$)，但其前提是節點彼此獨立；共用電源、網路或軟體會造成共同故障，使此假設失效。

習題

一、觀念題

1. 請說明「故障 (fault)」與「失效 (failure)」的差異，並解釋容錯設計的目標為何。
2. 為什麼描述服務延遲時應使用 p99 而非平均值？請以尾端延遲的概念說明。
3. 請以 CAP 定理說明：為何在網路分割時，一致性與可用性無法兼得？並各舉一個適合偏向一致性與偏向可用性的應用。
4. 何謂冪等性？它為何能化解逾時重試所帶來的兩難？

二、計算題

1. 某服務的單一節點可用度為 99%。請以式 3-1 計算部署 2 個與 3 個獨立節點時的整體可用度，並換算其約略的年停機時間（一年以 365 天計）。
2. 承上題，若單一節點可用度僅為 95%，要達到整體可用度 99.99% 以上，至少需要幾個獨立節點？（提示：逐一代入 $n = 2, 3, 4, \dots$ 計算）

三、思考與討論

1. 案例四中，三個副本部署在同一機房而導致同時失效。請以式 3-1 的前提說明此設計的謬誤，並提出至少兩種提升副本獨立性的做法。
2. 有人主張「既然雲端供應商已保證高可用，我的應用就不必再考慮容錯」。請根據本章內容評論此說法。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

故障 (fault) 指單一元件偏離應有行為，如一顆硬碟損壞；失效 (failure) 指整個系統停止對外提供服務。兩者是「局部」與「整體」之別。容錯設計的目標，正是不讓局部的故障擴大為整體的失效——透過複製、備援與隔離，使部分元件出問題時，系統整體仍能正確運作。可補充案例一：Netflix 以混沌猴主動製造故障，確保每個服務都具備容錯能力。

第 2 題

平均值會被大量正常請求稀釋，使少數極慢請求被掩蓋，讓數字好看卻不反映部分使用者的痛苦。p99 表示 99% 的請求快於此值、1% 慢於此值，能揭露尾端的壞體驗。更關鍵的是：重度使用者往往資料最多、請求最複雜，最容易落在尾端，因此只看平均等於系統性忽略最有價值的客戶。這也是 SLO 多以 p99 訂定的原因。

第 3 題

應以 3.4.2 的情境作答：資料同存兩節點，網路斷裂後一方被寫入新值，另一方收到讀取請求時，只能在「回舊值（保可用、犧牲一致）」與「拒絕回應等同步（保一致、犧牲可用）」之間二選一，

故無法兼得。偏向一致性的例子如結帳扣庫存、金融交易；偏向可用性的例子如商品瀏覽、社群動態。可補充：此取捨僅在分割時觸發，且可依操作而異（案例二）。

第 4 題

冪等性指「執行一次與執行多次結果相同」的性質。逾時重試的兩難在於：無法分辨對方是真的沒收到、還是回應遲到；若對方其實已處理，重試就會造成重複執行（如重複扣款）。若操作具冪等性，則即使因重試而收到多次，也只產生一次效果，兩難即被化解。常見做法是為每筆操作賦予唯一識別碼（如案例三的交易識別碼）。

二、計算題

第 1 題

套用 $A = 1 - (1 - a)^n$ ，其中 $a = 0.99$ ，故 $1 - a = 0.01$ 。年總時數以 $365 \times 24 = 8,760$ 小時計。

- (1) $n = 2$: $A = 1 - 0.01^2 = 1 - 0.0001 = 0.9999$ (99.99%)。年停機 = $8,760 \times 0.0001 = 0.876$ 小時 ≈ 53 分鐘。
- (2) $n = 3$: $A = 1 - 0.01^3 = 1 - 0.000001 = 0.999999$ (99.9999%)。年停機 = $8,760 \times 0.000001 = 0.00876$ 小時 ≈ 32 秒。

延伸思考：僅增加一個節點 ($2 \rightarrow 3$)，年停機就從約 53 分鐘降到約 32 秒。這說明可用度隨節點數呈指數改善，但前提是節點獨立（見第三部分討論題）。

第 2 題

套用同一公式， $a = 0.95$ ， $1 - a = 0.05$ ，逐一代入求使 $A \geq 0.9999$ 的最小 n 。

- (1) $n = 2$: $A = 1 - 0.05^2 = 1 - 0.0025 = 0.9975$ (不足)。
- (2) $n = 3$: $A = 1 - 0.05^3 = 1 - 0.000125 = 0.999875$ (不足，未達 99.99%)。
- (3) $n = 4$: $A = 1 - 0.05^4 = 1 - 0.00000625 = 0.99999375$ (達標，超過 99.99%)。

故至少需要 4 個獨立節點。延伸思考：與第 1 題相比，單節點可用度較低 (95% vs 99%) 時，需要更多節點才能達到同等目標——地基越弱，越需要更多冗餘來補強。

三、思考與討論

第 1 題

關鍵在於式 3-1 的「彼此獨立」前提。三個副本共用同一機房的電源與空調，並不獨立；一旦共用資源故障，三副本會同時失效， $(1 - a)^n$ 的機率相乘不再成立，指數級可靠性瞬間歸零。提升獨立性的做法至少有：其一，將副本分散到不同的可用區或機房，避免共用電源與網路；其二，避免所有副本跑同一版本、同一組態的軟體，以降低共同的軟體臭蟲風險；其三，錯開維護時間，避免人為操作同時影響多個副本。

第 2 題

本題訓練批判性思考。較周延的回答會指出此說法過於樂觀：其一，雲端的高可用保證通常針對「基礎設施層」，你的應用邏輯若無容錯設計，一個程式臭蟲或一次錯誤部署仍可造成整體失效；其二，雲端服務本身也會故障（且合約中的 SLA 往往低於你以為的水準），區域性事故時有所聞；其三，責任共擔模型下，資料的一致性、重試冪等、跨區部署等仍是使用者的責任。可連結案例四與式 3-1 的獨立性前提，說明「把雞蛋放在同一朵雲、同一區」同樣有共同故障風險。

台灣自編大學教科書
@prof
大數據分析與雲端運算

第壹篇 導論與基礎

Foundations

第 4 章

資料模型、編碼與儲存基礎

Data Models, Encoding, and Storage Foundations

機器人叫獸（李世淵） 編著

叫獸開講

我念研究所時，實驗室有一位學長，程式寫得極快。有一次他負責把感測器資料存下來，為了圖方便，他把每一筆量測值直接用逗號串成一長串文字，寫進一個檔案，格式是他自己隨手定的——沒有欄位名稱、沒有型別、沒有說明，只有他自己看得懂。

三個月後，學長畢業了。輪到我接手那批資料，打開檔案，看到的是一片望不到盡頭的數字海：3.14,78.4,1,0,2201,... 每一欄是什麼意思？溫度是攝氏還是華氏？那個 1 和 0 是開關還是良品與不良品？沒有人知道。那批花了半年蒐集的珍貴資料，因為當初沒有好好設計「資料長什麼樣子」，幾乎等於作廢。

這件事給我上了重要的一課：資料怎麼「長」，和資料本身一樣重要。在你動手寫任何一行分析程式之前，你得先回答三個問題——資料用什麼模型來組織（是一張張表，還是巢狀的文件，還是一張關係網）？資料在檔案與網路上用什麼格式來編碼（純文字還是二進位）？資料在硬碟上又用什麼引擎來存取（適合大量寫入，還是快速查詢）？

這一章，是第壹篇的最後一塊拼圖。第 1 章給你全局，第 2 章給你生命週期，第 3 章給你系統原理，而這一章，要帶你走進資料最貼身的三層：模型、編碼、儲存。把這三層搞懂，你才不會像當年的我一樣，對著一片數字海束手無策；也才能在後面第貳篇談分散式資料庫時，真正聽懂它們在吵些什麼。

學習目標

研讀完本章之後，你應該能夠：

1. 區分關聯式、文件與圖三種資料模型，並依應用特性選擇合適者。
2. 辨識結構化、半結構化與非結構化資料，並說明其對處理方式的影響。
3. 比較文字式與二進位編碼格式，說明 JSON、Avro、Parquet 的適用場景。
4. 解釋列式儲存與欄式儲存的差異，並說明欄式儲存為何適合分析查詢。
5. 說明 B-tree 與 LSM-tree 兩種儲存引擎的原理與讀寫取舍。
6. 說明索引與資料壓縮如何影響儲存與查詢效能。

核心概念

核心概念	說明
資料模型	組織資料的抽象方式，主要有關聯式、文件與圖三種。
結構化程度	資料是否具備固定綱要（schema），分結構化、半結構化與非結構化。
編碼格式	資料在檔案與網路傳輸時的表示法，分文字式與二進位。
列式與欄式儲存	資料在磁碟上按「一整列」或「一整欄」連續存放的兩種佈局。
B-tree	以就地更新、平衡樹狀結構為基礎的儲存引擎，讀取效率高。
LSM-tree	以追加寫入、分層合併為基礎的儲存引擎，寫入吞吐高。
索引	為加速查詢而額外建立的資料結構，以空間與寫入成本換取讀取速度。

表 4-1 本章核心概念一覽

4.1 關聯式、文件與圖資料模型

資料模型是「我們如何看待與組織資料」的心智框架。它是整個系統中影響最深遠的一層決策——因為它不只決定資料怎麼存，更決定我們能怎麼想、怎麼問問題。以下介紹三種最主流的模式。

4.1.1 關聯式模型：歷久彌新的表格

關聯式模型把資料組織成一張張「表」（table），每張表由列（記錄）與欄（欄位）構成，表與表之間透過鍵值建立關聯。它誕生於一九七〇年代，至今仍是最廣泛使用的模型，背後有數十年成熟的理論與工具支撐。

它的強項在於「結構嚴謹」與「善於處理多對多關係」。當資料之間有錯綜複雜的關聯——一張訂單對應多項商品、一項商品出現在多張訂單——關聯式模型能用清晰的方式表達，並以強大的查詢語言（SQL）進行任意角度的組合查詢。它的代價則是綱要相對僵固：欄位一旦定好，要更動並不輕鬆。

4.1.2 文件模型：巢狀的彈性

文件模型把相關的資料包成一份「文件」（通常是 JSON 格式），允許巢狀結構——一份文件裡可以再裝著清單與子物件。它常被歸類為 NoSQL 的一種，強項在於彈性：不同文件可以有不同的欄位，綱要不必要事先固定。

文件模型特別適合「一對多」且資料常被「整包一起讀取」的情境。例如一份履歷——姓名、學歷清單、工作經歷清單——很自然地就是一份巢狀文件，且我們幾乎總是整份一起讀。但它也有弱點：處理多對多關係與跨文件的關聯查詢時，會比關聯式模型吃力許多。

4.1.3 圖模型：關係即主角

當資料之間的「關係」本身比資料更重要時，圖模型就登場了。它以「頂點」（vertex）表示實體、以「邊」（edge）表示實體之間的關係，特別擅長回答「多層關聯」的問題——例如社群網路中的朋友的朋友、道路網中的最短路徑、詐騙偵測中的可疑金流環路。

這類問題若用關聯式模型來做，往往需要層層的表格連接（join），既難寫又慢；用圖模型則自然直觀。本書第 18 章談連結分析與大規模圖處理時，會再深入這個模型。

模型	組織方式	強項	典型應用
關聯式	表格與鍵值關聯	嚴謹、善於多對多與彈性查詢	交易系統、財會、庫存
文件	巢狀的 JSON 文件	彈性綱要、整包讀取	內容管理、使用者設定檔
圖	頂點與邊	多層關聯與路徑查詢	社群網路、推薦、詐騙偵測

表 4-2 三種資料模型的比較

叫獸提醒

初學者常問：「哪一種模型最好？」這問題本身就問錯了。模型沒有最好，只有最適合。選錯模型的代價非常高——它像地基，蓋到一半才發現地基不對，幾乎只能打掉重來。所以選模型的關鍵，不是看哪個時髦，而是誠實地問：我的資料之間，最主要的關係長什麼樣子？我最常問的問題是哪一類？

4.2 結構化、半結構化與非結構化資料

除了「用什麼模型」，資料還有另一個貫穿全書的分類軸：它有多「結構化」。這個分類，直接決定了資料在進入分析之前，需要多少前處理的功夫。

4.2.1 三種結構化程度

- **結構化資料**：具備固定綱要、欄位與型別皆明確，天生就能放進表格。例如交易紀錄、感測器數值。最容易分析。
- **半結構化資料**：有結構但不嚴格固定，欄位可增可減，通常自帶標記。例如 JSON、XML、日誌檔。需要一定的解析與整理。
- **非結構化資料**：沒有預先定義的欄位結構，如影像、聲音、自由文字、影片。佔資料總量的大宗，也最難直接分析，往往需先以模型萃取特徵。

要特別澄清一個常見誤解：「非結構化」不代表「沒有資訊」，而是「沒有預先定義的欄位」。一張產品照片裡明明有大量資訊——瑕疵、尺寸、顏色——只是這些資訊沒有以現成的欄位呈現，必須透過影像處理或深度學習把它「萃取」出來，變成結構化的特徵。這正是本書第肆篇與第 36 章的重要工作。

4.2.2 讀時綱要與寫時綱要

結構化程度的背後，藏著一個影響深遠的設計哲學之爭：綱要（schema）該在「寫入時」就強制，還是在「讀取時」才套用？

寫時綱要（schema-on-write）在資料寫入前就檢查它是否符合預定結構，不合格就拒收，這是傳統關聯式資料庫的做法——好處是資料品質有保障，壞處是彈性低。讀時綱要（schema-on-read）則先把原始資料照單全收，直到讀取時才決定如何解讀，這是資料湖的典型做法——好處是彈性極高、能容納各種型態，壞處是資料品質的責任被推遲到讀取端。

你會發現，這其實是第 2 章 ETL 與 ELT 之爭的更底層版本。ELT 之所以可行，正是因為採用了讀時綱要——先原封不動存下來，需要時再賦予結構。這兩種哲學沒有絕對優劣，而是反映了「要嚴謹還是要彈性」的永恆取舍。

4.3 序列化與編碼格式

資料在記憶體中是一種樣子（物件、結構），但要寫進檔案、或透過網路傳到另一台機器時，就必須轉換成一連串的位元組。這個「從記憶體格式轉為位元組序列」的過程，稱為序列化（serialization）或編碼（encoding），反向則稱為反序列化。編碼格式的選擇，深深影響儲存空間、傳輸效率與跨系統的相容性。

4.3.1 文字式格式

文字式格式以人類可讀的文字來表示資料，最常見的有三種。

- **CSV**：以逗號分隔的表格文字，簡單普及，幾乎所有工具都支援。但缺乏型別資訊、不擅表達巢狀結構，且逗號與換行的處理常出狀況。
- **JSON**：以鍵值對與巢狀結構表示資料，是網路 API 的通用語言，可讀性佳、支援巢狀。缺點是冗長、缺乏嚴格型別、且不含綱要。
- **XML**：以標籤描述結構，表達力強、生態成熟，但語法繁瑣、體積龐大，近年逐漸被 JSON 取代。

文字式格式的共同優點是「人看得懂、跨平台無礙」，共同缺點是「體積大、解析慢、缺乏型別保證」。它們適合設定檔、API 交換與小量資料，但一旦資料量進入大數據等級，這些缺點就會被放大成沉重的成本。

4.3.2 二進位格式

為了克服文字式格式在大數據下的效率問題，業界發展出多種二進位格式。它們犧牲了人類可讀性，換來更小的體積與更快的處理。

- **Avro**：以綱要為核心的二進位格式，資料與綱要分離，特別擅長處理「綱要會隨時間演進」的情境，常用於串流與資料交換。
- **Protocol Buffers**：由 Google 發展，以緊湊、快速著稱，廣泛用於跨服務通訊，需事先定義綱要並產生程式碼。
- **Parquet**：欄式儲存的二進位格式，專為分析而生。能只讀取需要的欄、壓縮率高，是資料湖與大數據分析的事實標準。
- **ORC**：與 Parquet 定位相近的欄式格式，在特定生態系中廣泛使用。

其中 Parquet 值得特別記住——它幾乎是現代大數據分析的預設檔案格式，本書後續多章都會用到。它的高效，關鍵就在於採用了「欄式儲存」，這正是下一節的主題。

格式	類型	是否含綱要	主要用途
CSV	文字	否	簡易表格交換
JSON	文字	否	API 與設定檔
Avro	二進位	是（分離）	串流、綱要演進
Parquet	二進位（欄式）	是	大數據分析儲存

表 4-3 常見編碼格式的比較

4.3.3 綱要演進：向前與向後相容

資料的綱要不是一成不變的——今天你可能為紀錄加一個新欄位，明天可能移除一個舊欄位。問題在於，新舊版本的程式與資料往往必須共存。這帶出兩個重要概念：向後相容（新程式讀得懂舊資料）與向前相容（舊程式讀得懂新資料）。

良好的編碼格式（如 Avro 與 Protocol Buffers）在設計上就考慮了綱要演進，讓你能安全地增刪欄位而不破壞既有系統。這在大型組織中至關重要——你不可能要求全公司的系統在同一秒鐘一起升級。這也呼應了第 2 章提到的「結構無預警更動」災難：有了良好的綱要演進機制與資料契約，那類災難就能大幅減少。

4.4 列式儲存與欄式儲存

4.4.1 同一張表，兩種擺法

想像一張有一百萬列、每列二十欄的表。它在磁碟上該怎麼擺？有兩種根本不同的佈局。

列式儲存（row-oriented）把「同一列」的所有欄位連續存放：第一列的二十個欄位排在一起，接著才是第二列。這是傳統交易資料庫的做法，適合「一次讀寫一整列」的操作，例如讀出某位顧客的完整資料、或新增一筆訂單。

欄式儲存 (column-oriented) 則把「同一欄」的所有值連續存放：一百萬列的第一欄全部排在一起，接著才是第二欄。這是分析型資料庫與 Parquet 的做法，適合「只挑幾個欄位、但要掃過大量列」的分析查詢。

4.4.2 為什麼分析愛用欄式

分析查詢有一個典型樣貌：「在一百萬筆交易中，計算某商品的平均銷售額」。這個查詢只需要「商品」與「金額」兩欄，卻要掃過所有列。

在列式儲存下，為了取得這兩欄，磁碟必須把每一列的全部二十欄都讀進來，再丟棄用不到的十八欄——白白搬運了九成的資料。在欄式儲存下，只需讀取「商品」與「金額」這兩欄的連續區塊，其餘完全不碰。光是這一點，就能帶來數倍到數十倍的效率差距。

欄式儲存還有一個額外紅利：壓縮。因為同一欄的資料型別相同、值也常常重複（例如「商品」欄只有幾百種商品名反覆出現），壓縮演算法能發揮得淋漓盡致，進一步縮小體積、加快讀取。這兩個優勢疊加，正是欄式儲存主宰大數據分析的原因。

面向	列式儲存	欄式儲存
連續存放	同一列的各欄位	同一欄的各列值
適合操作	讀寫整列（交易）	掃描少數欄、大量列（分析）
寫入	快，一次寫一列	較慢，一列須拆散到各欄
壓縮效果	普通	極佳（同欄同型別）
代表	傳統 OLTP 資料庫	Parquet、分析型資料庫

表 4-4 列式儲存與欄式儲存的比較

叫獸提醒

這裡藏著第 1 章「量級感」的實際應用。當你的查詢只要兩欄、資料卻有二十欄時，選對儲存佈局就等於少搬九成的資料。在 TB、PB 等級，這九成不是省一點點，而是省下數小時的運算與大筆的雲端費用。所以當你日後看到有人堅持把分析資料存成 Parquet 而非 CSV，別覺得他龜毛——他是在替你省時間也省錢。

4.5 儲存引擎：B-tree 與 LSM-tree

再往下一層，是資料庫「引擎」如何把資料實際寫到磁碟、又如何把它找回來。這一層決定了讀與寫誰快誰慢。當代兩大主流引擎——B-tree 與 LSM-tree——各自代表了一種截然不同的取舍哲學。

4.5.1 B-tree：就地更新的老將

B-tree 是傳統關聯式資料庫的主力引擎，已有數十年歷史。它把資料組織成一棵平衡的樹狀結構並「就地」更新——要修改一筆資料，就直接找到它在磁碟上的位置改寫。

B-tree 的強項是讀取：因為結構平衡，任何一筆資料都能在少數幾步內找到，讀取效率穩定而可預期。它的相對弱點在寫入：每次寫入都可能牽動樹的結構調整，且就地改寫對磁碟而言是隨機存取，速度不如循序寫入。

4.5.2 LSM-tree：追加寫入的新秀

LSM-tree (Log-Structured Merge-tree) 是許多 NoSQL 與大數據儲存的引擎。它的核心思想恰與 B-tree 相反：不就地改寫，而是把所有寫入都「追加」到檔案末端，之後再於背景把這些檔案分層合併、整理。

這個「只追加、不改寫」的設計，把原本零散的隨機寫入變成了高效的循序寫入，因此寫入吞吐極高，特別適合寫入密集的场景（如大量感測器資料湧入）。它的代價是讀取可能較慢——因為一筆資料的最新值可能散落在多個尚未合併的檔案中，須逐一查找。

面向	B-tree	LSM-tree
寫入方式	就地更新（隨機寫）	追加寫入（循序寫）
寫入吞吐	普通	高
讀取效率	穩定、可預期	可能較慢（需查多檔）
空間	可能有碎片	合併前可能有冗餘
代表	傳統關聯式資料庫	許多 NoSQL、時序資料庫

表 4-5 B-tree 與 LSM-tree 的比較

這個取舍的口訣是：讀多寫少偏 B-tree，寫多讀少偏 LSM-tree。回想第 2 章的物聯網感測資料——高頻、大量、持續湧入的寫入——正是 LSM-tree 的主場，這也是多數時序資料庫採用它的原因。

4.6 索引與資料壓縮

4.6.1 索引：用空間換時間

若沒有索引，資料庫要找一筆特定資料，只能從頭到尾把整張表掃一遍——資料一大，這就慢得難以忍受。索引（index）是為加速查詢而額外建立的資料結構，就像書本的目錄：與其一頁頁翻找，不如先查目錄直接翻到那一頁。

索引的本質是一種取舍：它以「額外的儲存空間」與「較慢的寫入」（因為每次寫入都要順便更新索引），換取「大幅加快的讀取」。因此索引不是越多越好——為每一欄都建索引，會讓寫入被拖垮、儲存被撐爆。該為哪些欄位建索引，取決於你最常用什麼條件來查詢，這又回到了「理解資料存取模式」這個反覆出現的主題。

4.6.2 資料壓縮：省下的不只是空間

大數據的資料量龐大，壓縮幾乎是必備手段。但壓縮省下的，不只是儲存費用——在許多情況下，它還能讓查詢變快。這聽起來矛盾（多了解壓縮的步驟怎麼還會更快？），道理其實很直接：磁碟與網路的速度，遠慢於處理器解壓縮的速度。少讀一半的位元組所省下的時間，往往遠超過解壓縮所花的時間。

常見的壓縮取舍是「壓縮率」與「速度」之間的權衡：有些演算法壓得很小但較慢，有些壓得普通但極快。分析場景常選擇「解壓快」的演算法，因為讀取頻繁；封存場景則選擇「壓得小」的演算法，因為極少讀取、重在省空間。這又呼應了第 2 章的冷熱資料分層——不同溫度的資料，連壓縮策略都可以不同。

4.6.3 第壹篇的總結

到這裡，第壹篇「導論與基礎」四章完整收束。讓我們把這條線回顧一遍：第 1 章建立了大數據與雲端的全局視野，第 2 章鋪陳了資料從生成到服務的生命週期，第 3 章講透了分散式系統的原理與取捨，而本章則深入資料最貼身的三層——模型、編碼、儲存。

這四章合起來，回答的是一個問題：在動手處理大數據之前，你需要先具備哪些觀念地基？答案是——你要知道資料如何被看待（模型）、如何流動（生命週期）、如何在會故障的機器間協同（分散式原理），以及如何被實際存放（編碼與儲存）。地基已成，接下來第貳篇，我們就要拿起這些工具，正式走進分散式儲存與資料庫的世界，看看 HDFS、NoSQL、複製與分區這些技術，是如何把本篇的原理化為現實。

公式與流程：欄式儲存省下多少 I/O

4.4 節說欄式儲存「只讀需要的欄」能大幅省下磁碟讀取，但到底省多少？我們可以用一個簡單的模型把它量化，讓你對這個效益有精確的感覺。

假設一張表共有 C 個欄位，各欄位平均大小相近；某查詢只需其中 k 個欄位。在理想情況下，欄式儲存所需讀取的資料量，相對於列式儲存（須讀取全部欄位）的比值為：

$$\text{讀取比} = k / C$$

式 4-1 欄式相對於列式的讀取資料量比

換言之，理論上的 I/O 節省比例為 $1 - k/C$ 。若再考慮欄式儲存因同型別而獲得的壓縮增益（設壓縮比為 c ，即壓縮後為原本的 $1/c$ ），則欄式實際讀取量相對列式的比值約為：

$$\text{實際讀取比} \approx (k / C) \times (1 / c)$$

式 4-2 納入壓縮增益後的讀取比

我們以一張 20 欄的表為例，看不同查詢欄數與壓縮比下的讀取量（以列式未壓縮為基準 100%）。

查詢欄數 k	僅欄式 (k/C)	欄式+壓縮 3 倍	相對列式節省
2	10%	約 3.3%	約 96.7%
4	20%	約 6.7%	約 93.3%
8	40%	約 13.3%	約 86.7%
20 (全欄)	100%	約 33.3%	約 66.7%

表 4-6 20 欄表在不同查詢欄數下的讀取量（列式未壓縮＝100%）

叫獸提醒

看最後一列：就算查詢用到全部二十欄，欄式儲存靠壓縮仍只讀約三分之一的量。而分析查詢極少用到全部欄位——多半只碰幾欄——這時節省高達九成以上。這就是為什麼「把分析資料存成 Parquet」不是龜毛，而是常識。當然，這個模型是理想化的：實務上還要考慮查詢引擎的最佳化、資料的實際分布與欄位大小不均，但它足以讓你對「為何欄式主宰分析」建立正確的直覺。

案例研討

案例一 一片沒有說明的數字海

回到叫獸開講的故事。那批被學長用自訂格式存下的感測資料，之所以幾乎報廢，根源不在資料本身，而在編碼——沒有欄位名稱（缺綱要）、沒有型別（不知是溫度還是開關）、沒有文件（缺中繼資料）。

若當初改用一個自帶綱要的格式（如 Avro 或 Parquet），把欄位名稱、型別與單位一併寫入，那麼即使原作者離開，接手者也能立刻讀懂。這個案例把本章三層都串了起來：模型（該用表格描述這些量測）、編碼（該用自帶綱要的格式）、以及第 2 章的中繼資料（該記錄欄位意義）。一個小小的格式決定，決定了半年心血是資產還是垃圾。

案例二 把 CSV 換成 Parquet 的那一天

某資料團隊長期把每日的交易日誌存成 CSV，分析師每次跑報表——通常只用到十幾個欄位中的三、四欄——都要等上二十分鐘。後來他們把儲存格式改為 Parquet，同一份報表的執行時間驟降到不到兩分鐘。

這個十倍的加速，正是式 4-1 與式 4-2 的真實體現：欄式儲存讓查詢只讀那三、四欄，加上壓縮讀取量剩下不到原本的十分之一。更棒的是，儲存體積也縮小了數倍，雲端費用同步下降。這個案例說明：有時候讓系統變快最有效的方法，不是升級硬體，而是換一個更聰明的資料佈局。

案例三 感測資料庫為何選 LSM-tree

一座工廠要為數千個感測器建立時序資料庫，資料以每秒數十萬筆的速度持續湧入，而查詢多半是「回顧某段時間的趨勢」，相對不頻繁。團隊在選型時，捨棄了傳統的 B-tree 資料庫，改用以 LSM-tree 為引擎的時序資料庫。

理由正是 4.5 節的取捨口訣：這是典型的「寫多讀少」場景。LSM-tree 把海量的隨機寫入轉為高效的循序追加，輕鬆吞下每秒數十萬筆的洪流；而讀取雖略慢，但因查詢不頻繁，這個代價完全可以接受。若當初選了 B-tree，寫入很可能就成為瓶頸。這個案例呼應第 2 章的物聯網感測資料，也預示了第貳篇 NoSQL 的設計取捨。

案例四 選錯模型的社群新創

一家社群新創，初期用關聯式資料庫儲存使用者的交友關係——一張「好友」表，記錄誰和誰是朋友。隨著功能演進，他們想做「推薦你可能認識的人」，也就是找出「朋友的朋友的朋友」。

問題來了：在關聯式模型中，每多一層關係，就要多一次表格自我連接（self-join），查到第三層時，查詢慢得幾乎跑不動。他們最終不得不引入圖資料庫，把交友關係改以頂點與邊表示，同樣的多層關聯查詢瞬間變得又快又直觀。這個案例是 4.1.3 節最好的註腳——當關係本身是主角時，選對模型（圖）能省下日後打掉重來的巨大代價。也預告了第 18 章的圖處理。

實作活動

活動一 為你的資料選一個模型

請針對三種不同性質的資料，判斷最適合的資料模型並說明理由。

- 步驟 1. 一份記錄「學生—選課—課程」多對多關係的校務資料。
- 步驟 2. 一份包含姓名、多段學歷、多段經歷的巢狀履歷資料。

步驟 3. 一份描述「使用者互相追蹤」的社群關係資料。

步驟 4. 為每一種說明：若刻意選用最不適合的模型，會遇到什麼困難？

活動二 親手比較 CSV 與 Parquet

請以 Python (pandas 或 pyarrow) 實測欄式儲存的效益。

步驟 1. 產生或取得一份至少含 15 欄、十萬列以上的資料表。

步驟 2. 分別存成 CSV 與 Parquet，比較兩者的檔案大小。

步驟 3. 撰寫一個「只選其中 3 欄並做彙總」的查詢，分別在兩種格式上執行並計時。

步驟 4. 對照式 4-1 的預測，說明你觀察到的差距是否符合理論。

完成後你會親眼看見：同一份資料、同一個查詢，光是換個儲存格式，速度與體積就能差上好幾倍。

活動三 觀察綱要演進

請設計一個小實驗來體會綱要演進。先定義一個含三個欄位的資料結構並寫入幾筆資料；接著為它「新增一個欄位」，再用新舊兩版的定義分別去讀取先前寫入的舊資料。觀察：舊資料在新綱要下如何呈現（新欄位是空的嗎）？新資料在舊綱要下又如何？藉此體會向前與向後相容的意義，並思考這對「全公司系統無法同時升級」的現實有何幫助。

延伸閱讀

- 《Designing Data-Intensive Applications》第 2、3、4 章：Kleppmann 與 Riccomini 著。本章的主要依據：第 2 章談資料模型與查詢語言，第 3 章談儲存引擎 (B-tree/LSM-tree) 與欄式儲存，第 4 章談編碼與綱要演進。
- **Apache Parquet 官方文件**：說明欄式格式的檔案結構、編碼與壓縮設計，是理解 4.3、4.4 節的第一手資料。
- 《資料庫系統概論》相關章節：任一本資料庫教科書關於關聯式模型、索引與 B-tree 的章節，可補強 4.1 與 4.6 的理論基礎。
- **時序資料庫與 LSM-tree 的實務文章**：多數時序資料庫的技術部落格都會說明其為何採用 LSM-tree，是 4.5 節的實務延伸。

本章小結

1. 資料模型主要有關聯式（善於多對多與彈性查詢）、文件（彈性綱要、整包讀取）與圖（多層關聯與路徑查詢）三種；模型沒有最好，只有最適合，選擇的關鍵在於資料間最主要的關係與最常見的查詢。
2. 資料依結構化程度分為結構化、半結構化與非結構化；「非結構化」不代表沒有資訊，而是沒有預先定義的欄位，須靠模型萃取特徵。綱要可在寫入時強制（重品質）或讀取時套用（重彈性），對應 ETL 與 ELT 之爭。
3. 編碼格式分文字式 (CSV、JSON、XML，可讀但冗長) 與二進位 (Avro、Protocol Buffers、Parquet、ORC，高效但不可讀)；Parquet 是大數據分析的事實標準；良好的格式支援綱要演進的向前與向後相容。
4. 列式儲存把同一列連續存放，適合交易；欄式儲存把同一欄連續存放，適合分析——因為分析常只讀少數欄卻掃大量列，欄式能大幅減少 I/O，且同欄同型別使壓縮效果極佳。

5. B-tree 就地更新、讀取穩定，適合讀多寫少；LSM-tree 追加寫入、寫入吞吐高，適合寫多讀少（如感測時序）；口訣是「讀多寫少偏 B-tree，寫多讀少偏 LSM-tree」。
6. 索引以空間與寫入成本換取讀取速度，不宜過多；壓縮不只省空間，還常因少讀位元組而加快查詢，其策略可隨資料冷熱而不同。
7. 欄式儲存的讀取量約與「查詢欄數／總欄數」成正比（式 4-1），再乘上壓縮增益（式 4-2）；分析查詢多只用少數欄，故節省往往高達九成以上。

習題

一、觀念題

1. 請比較關聯式、文件與圖三種資料模型的強項，並各舉一個最適合的應用情境。
2. 「非結構化資料等於沒有資訊」這句話為何錯誤？請以一個例子說明。
3. 請說明列式儲存與欄式儲存的差異，並解釋為何欄式儲存特別適合分析查詢。
4. 請以「讀多寫少 vs 寫多讀少」的角度，說明 B-tree 與 LSM-tree 各自的適用場景。

二、計算題

1. 某資料表共有 25 欄，某分析查詢只需其中 5 欄。假設各欄大小相近，請以式 4-1 計算欄式儲存相對列式的讀取比與節省比例；若欄式再享有 4 倍壓縮，請以式 4-2 計算實際讀取比（相對列式未壓縮）。
2. 某感測系統每秒寫入 200,000 筆資料，讀取查詢每分鐘僅約 10 次。請說明此場景的「寫讀比」約為多少（以每秒計），並據此判斷應選 B-tree 或 LSM-tree，說明理由。

三、思考與討論

1. 案例四中，社群新創因選錯資料模型而付出重寫的代價。請說明：在專案初期，可以用哪些問題來幫助自己選對模型，以避免日後打掉重來？
2. 讀時綱要提供彈性，卻把資料品質的責任推遲到讀取端。請討論：在什麼情況下這種彈性利大於弊？又在什麼情況下弊大於利？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

關聯式：善於多對多關係與嚴謹的彈性查詢，適合交易、財會、庫存等結構清楚且關係複雜的系統。文件：彈性綱要、擅長整包讀取的一對多資料，適合使用者設定檔、內容管理等。圖：擅長多層關聯與路徑查詢，適合社群網路、推薦、詐騙偵測。作答時應扣住各模型的「強項」與「資料間主要關係的型態」之對應，並可引用表 4-2。

第 2 題

此句錯在把「非結構化」誤解為「無資訊」。非結構化僅指「沒有預先定義的欄位結構」，而非缺乏資訊。例子：一張產品照片含有瑕疵、尺寸、顏色等大量資訊，只是這些資訊未以現成欄位呈現，須透過影像處理或深度學習萃取為結構化特徵。可連結第肆篇與第 36 章。

第 3 題

列式把同一列各欄連續存放，欄式把同一欄各列值連續存放。欄式適合分析的原因有二：其一，分析查詢常只需少數欄卻要掃大量列，欄式只讀所需欄，避免搬運無關欄位（可引式 4-1）；其二，同一欄型別相同、值常重複，壓縮效果極佳，進一步減少 I/O。可引表 4-4 佐證。

第 4 題

B-tree 就地更新、讀取穩定可預期，但寫入涉及隨機存取與結構調整，故適合讀多寫少（如一般交易查詢系統）。LSM-tree 以追加取代改寫，將隨機寫轉為循序寫，寫入吞吐高，但讀取可能需查多個檔案而較慢，故適合寫多讀少（如高頻感測時序資料）。口訣：讀多寫少偏 B-tree，寫多讀少偏 LSM-tree。

二、計算題

第 1 題

套用式 4-1 與式 4-2， $C = 25$ 、 $k = 5$ 、壓縮比 $c = 4$ 。

- (1) 讀取比（僅欄式） $= k/C = 5/25 = 0.2$ （20%）。
- (2) 節省比例 $= 1 - 0.2 = 0.8$ （80%）。
- (3) 納入壓縮：實際讀取比 $\approx (5/25) \times (1/4) = 0.2 \times 0.25 = 0.05$ （5%），即相對列式未壓縮節省約 95%。

延伸思考：僅用五分之一的欄位，加上四倍壓縮，讀取量就降到原本的二十分之一。這解釋了為何實務上把分析資料存成 Parquet 幾乎是預設選項。

第 2 題

先把讀取換算為每秒：每分鐘 10 次 $\div 60 \approx 0.17$ 次/秒。

- (1) 寫入：200,000 筆/秒；讀取：約 0.17 次/秒。
- (2) 寫讀比 $\approx 200,000 \div 0.17 \approx 1,200,000$ 比 1，屬極端的「寫多讀少」。
- (3) 結論：應選 LSM-tree。因為它以追加寫入把海量隨機寫轉為循序寫，可吞下每秒二十萬筆的洪流；讀取雖略慢，但查詢極不頻繁，此代價可接受。

延伸思考：這正是案例三工廠時序資料庫的選型邏輯，也呼應第 2 章物聯網感測資料「高頻、大量、持續湧入」的特性。

三、思考與討論

第 1 題

可提出的自我提問包括：其一，我的資料之間最主要的關係是什麼型態（一對多？多對多？多層關聯？）——多對多偏關聯式、整包一對多偏文件、多層關聯偏圖。其二，我最常問的查詢是哪一類（跨表組合？整份讀取？路徑或鄰居搜尋？）。其三，綱要未來會頻繁變動嗎（是則偏彈性的文件或讀時綱要）。其四，資料規模與一致性要求為何。作答時應強調：模型是地基，選擇成本低但更換成本極高，值得在初期多花時間釐清。可引案例四為戒。

第 2 題

讀時綱要利大於弊的情況：資料來源多樣且結構多變、需求探索性強、希望先保存原始資料以備未來未知的分析（如資料湖、研究性專案）——此時彈性讓你不必預先猜測所有用途。弊大於利的情况：資料將被大量下游穩定消費、對品質與一致性要求高（如財務、法遵報表）——此時把品質責任推遲到每個讀取端，會導致各處重複清理、標準不一、錯誤難追。折衷之道常是分層：著陸區用讀時綱要保留原始彈性，服務區則以寫時綱要保障品質，呼應第 2 章的分層儲存與 2.4 節銅銀金設計。

台灣自編大學教科書
@prof
大數據分析與雲端運算

第貳篇 分散式儲存與資料庫

Distributed Storage & Databases

第 5 章

分散式檔案系統與物件儲存

Distributed File Systems and Object Storage

機器人叫獸（李世淵） 編著

叫獸開講

回到第 1 章那個史丹佛的車庫。當那兩位學生決定用一千台廉價電腦來裝下整個網際網路時，他們立刻撞上一個很基本、卻很要命的問題：一個檔案，如果比任何一台機器的硬碟還大，該怎麼存？

你不能把一個一 TB 的檔案硬塞進一顆只有幾百 GB 的硬碟。傳統作業系統裡「檔案」的概念，天生就活在單一台機器上——它假設檔案就躺在某顆本機硬碟的某個位置。可是當資料大到一台機器裝不下，這個假設就崩潰了。

Google 的答案，是二〇〇三年那篇 GFS 論文。它的想法出奇地直白：既然一個大檔案裝不進一台機器，那就把它「切成很多小塊」，分散存到很多台機器上；再對外假裝這一切都沒發生——讓使用者以為自己面對的，仍然是一個完整的、單一的大檔案。這就是「分散式檔案系統」：底下是成千上萬台機器與無數的資料塊，表面上卻是一個乾淨俐落的檔案系統。

這一章是第貳篇的起點。第壹篇我們談的是原理——複製、容錯、分區；從這一章開始，我們要看這些原理如何長成具體的技術。你會看到第 3 章講的「假設機器會壞、用複製來容錯」是怎麼一磚一瓦地砌進 HDFS 的；也會看到雲端時代，這套地端的檔案系統又如何演化成「物件儲存」，成為今天資料湖的地基。

學習目標

研讀完本章之後，你應該能夠：

1. 說明分散式檔案系統的設計目標，以及它與傳統單機檔案系統的差異。
2. 描述 HDFS 的主從架構，並說明 NameNode 與 DataNode 各自的職責。
3. 解釋資料區塊、副本放置與容錯機制如何保障資料的可靠性。
4. 說明 HDFS 的讀取與寫入流程，以及其一致性模型的特點。
5. 比較分散式檔案系統與物件儲存的差異，並說明物件儲存的適用場景。
6. 說明資料湖的儲存層設計，以及儲存運算分離的意義。

核心概念

核心概念	說明
分散式檔案系統	把大檔案切塊分散存於多台機器，對外呈現為單一檔案系統的技術。
HDFS	Hadoop 分散式檔案系統，GFS 的開源實作，大數據儲存的經典代表。
NameNode	HDFS 的主節點，管理檔案的中繼資料與區塊位置。
DataNode	HDFS 的工作節點，實際儲存資料區塊。
資料區塊 (Block)	檔案被切分的固定大小單位，是複製與分散的基本單元。
副本 (Replica)	同一區塊在不同節點上的多份複製，用於容錯。
物件儲存	以扁平命名空間與 API 存取物件的儲存服務，雲端資料湖的基礎。

表 5-1 本章核心概念一覽

5.1 分散式檔案系統的設計目標

5.1.1 單機檔案系統的天花板

在單一台電腦上，檔案系統的工作很單純：管理硬碟上的檔案，記錄每個檔案存在哪些磁區、有多大、誰能存取。這套機制運作良好——直到資料的規模撞上兩道牆。

第一道牆是容量：單一檔案或整體資料量，超過了任何一顆硬碟所能容納。第二道牆是可靠性與吞吐：把所有雞蛋放在一顆硬碟裡，硬碟一壞，資料全失；而單顆硬碟的讀寫速度，也餵不飽需要同時處理海量資料的運算。分散式檔案系統，就是為了同時翻過這兩道牆而生的。

5.1.2 三個核心設計目標

GFS 與 HDFS 的設計，環繞著幾個在當年頗具顛覆性的假設與目標。理解這些目標，你才會明白它後續的每一個設計為何如此。

- **為大檔案而生**：系統假設檔案很大（動輒 GB 到 TB），數量相對不多。因此它把區塊切得很大，以減少中繼資料的負擔。這與傳統檔案系統「檔案多而小」的假設恰好相反。
- **假設硬體會故障**：在成千上萬台廉價機器上，故障是常態而非例外。因此容錯不是附加功能，而是核心設計——資料必須複製多份，任一機器壞掉都不影響整體。
- **為循序讀寫最佳化**：大數據的典型存取是「一次寫入、多次讀取」與「大規模循序掃描」，而非頻繁的隨機小改寫。系統因此犧牲隨機寫入的效率，全力最佳化循序吞吐。

叫獸提醒

注意第三個目標背後的取舍哲學。HDFS 刻意「不擅長」隨機小量寫入與修改——你幾乎不能在 HDFS 的檔案中間插入一段資料。這在傳統檔案系統看來簡直是缺陷，但它是有意為之的：因為大數據的真實工作負載就是「整批寫、大量讀」，為極少發生的隨機改寫而犧牲循序吞吐，並不划算。這再次印證第 4 章的道理——沒有最好的設計，只有最貼合工作負載的設計。

5.2 HDFS 架構：NameNode 與 DataNode

HDFS 採用「主從式」（master-slave）架構。這是一個貫穿大數據系統的經典模式：由一個（或少數幾個）主節點負責統籌與管理，由眾多工作節點負責實際的粗活。

5.2.1 中繼資料與資料的分離

HDFS 最關鍵的設計，是把「中繼資料」與「資料本身」徹底分開，交給兩種不同的角色管理。

- **NameNode（主節點）**：只管中繼資料——記錄有哪些檔案、每個檔案被切成哪些區塊、每個區塊的副本分別存在哪些 DataNode 上。它是整個系統的「目錄與地圖」，本身不儲存任何實際資料。
- **DataNode（工作節點）**：只管資料本身——實際儲存一個個資料區塊，並定期向 NameNode 回報自己還活著、手上有哪些區塊。它們是實際堆放貨物的倉庫。

這個分離之所以巧妙，在於它讓「管理」與「儲存」能各自獨立擴充。中繼資料量小、需要快速查詢，集中在 NameNode 的記憶體中；而資料量巨大，則水平分散到成百上千個 DataNode。要增加容量，只需加入更多 DataNode，NameNode 不必跟著長大。

5.2.2 NameNode 的關鍵地位與風險

NameNode 掌握了所有檔案的地圖——沒有它，就算 DataNode 上的資料都還在，也沒人知道哪個區塊屬於哪個檔案。這使 NameNode 成為系統中至關重要的角色，也帶來一個明顯的隱憂：它會不會成為第 3 章談過的「單點故障」？

早期的 HDFS 確實有此弱點。後來的版本以「高可用」設計來化解：部署兩個 NameNode（一主一備），主節點的中繼資料變動即時同步給備援節點，並透過第 8 章要談的協調服務來自動偵測故障並切換。這正是第 3 章「複製提升可用度」原理的具體落實——連掌管地圖的主節點，都要有備份。

角色	職責	資源特性
NameNode	管理中繼資料、區塊位置、命名空間	中繼資料存於記憶體，重視低延遲
DataNode	儲存資料區塊、回報狀態、傳輸資料	重視大容量與循序吞吐
備援 NameNode	同步中繼資料，故障時接手	與主節點對等，確保高可用

表 5-2 HDFS 的角色與職責

5.3 資料區塊、副本與容錯

5.3.1 為什麼區塊要切得那麼大

HDFS 把檔案切成固定大小的「區塊」，而且區塊很大——預設常是 128 MB，遠大於傳統檔案系統的數 KB。為什麼要切這麼大？

原因有二。其一是減輕 NameNode 的負擔：NameNode 要為每個區塊記錄中繼資料，區塊越大、數量越少，中繼資料就越少，NameNode 的記憶體才裝得下。其二是最佳化循序讀取：從磁碟讀資料，真正花時間的是「尋找資料起點」的動作；區塊越大，一次尋找後能連續讀取的資料就越多，尋找的相對成本就越低。這與 5.1 節「為大檔案、為循序讀寫」的目標一脈相承。

5.3.2 副本放置：不要把蛋放同一籃

HDFS 對每個區塊預設保存三份副本。但「三份」還不夠——關鍵是這三份放在哪裡。這正是第 3 章「彼此獨立」那個致命前提的實際應用。

HDFS 的預設策略考量了機器的實體位置（機架感知）：第一份副本放在寫入端所在的節點；第二份放在「不同機架」的另一個節點；第三份則放在第二份所在的那個機架裡的另一個節點。這樣安排的用意是——把副本分散到不同機架，即使整個機架因交換器或電源故障而全滅，仍有其他機架上的副本存活。這正是第 3 章案例四「副本同機房共同故障」的正解。

叫獸提醒

副本放置策略是一門在「可靠性」與「效率」之間拿捏的藝術。全部放同一機架，寫入快（同機架網路快）但不耐機架故障；全部放不同機架，最耐故障但寫入慢（跨機架傳輸多）。HDFS 的「一份同機架、兩份跨機架」是個聰明的折衷。你看，第 3 章那些抽象的取捨，到了這裡都變成了一條條具體的放置規則。

5.3.3 故障偵測與自動修復

DataNode 會定期向 NameNode 發送「心跳」訊號，表示自己還活著。若 NameNode 在一段時間內收不到某個 DataNode 的心跳，就判定它故障，並把它上面所有區塊當作「副本不足」。

接著，NameNode 會自動指揮其他仍存有這些區塊的 DataNode，把區塊再複製一份到健康的節點上，使副本數重新回到三份。整個過程自動完成，無須人工介入——這就是第 3 章「容錯」的活生生範例：故障（一個 DataNode 掛了）被系統自我修復，沒有演變成失效（資料遺失或服務中斷）。

5.4 讀寫流程與一致性

5.4.1 寫入流程

理解讀寫流程，就能看清中繼資料與資料如何協作。先看寫入一個檔案的大致步驟：

- 第 1 步。** 客戶端向 NameNode 請求寫入，NameNode 檢查權限後，分配區塊並指定要存放副本的 DataNode 清單。
- 第 2 步。** 客戶端不必自己把資料送給每個 DataNode，而是把資料送給第一個 DataNode，由它「接力」轉送給第二個、再由第二個轉送給第三個，形成一條複製管線。
- 第 3 步。** 三個 DataNode 都確認寫入成功後，逐級回報，客戶端才收到成功的確認。
- 第 4 步。** 所有區塊寫完後，NameNode 更新中繼資料，檔案正式可見。

這條「接力複製管線」的設計很精妙：它讓客戶端只需送出一份資料，卻能高效地產生三份副本，充分利用了各節點的網路頻寬，而非讓客戶端獨自扛起送三份的負擔。

5.4.2 讀取流程

讀取則相對單純：客戶端先問 NameNode「這個檔案由哪些區塊組成、各區塊的副本在哪些 DataNode 上」，取得地圖後，客戶端就直接向最近（或最空閒）的 DataNode 索取資料，不再經過 NameNode。

這裡有個關鍵設計：實際的資料傳輸「不經過 NameNode」。NameNode 只提供地圖，真正的搬運由客戶端與 DataNode 直接進行。這避免了 NameNode 成為資料流的瓶頸——若每一位元組都要經過它，它早就被壓垮了。這也是主從架構的通則：主節點只管調度，不碰粗活。

5.4.3 一致性模型：一次寫入、多次讀取

HDFS 採用「一次寫入、多次讀取」（write-once-read-many）的模型：檔案一旦寫入完成，通常就不再修改，只能追加或整個重寫。這大幅簡化了一致性問題——因為沒有「多方同時修改同一處」的情境，也就避免了第 3 章那些棘手的一致性取舍。

這個選擇再次呼應了設計目標：大數據的典型工作是「產生一批資料、存下來、反覆分析」，很少需要在原地細修。HDFS 用「放棄任意修改」換來了「簡單而可靠的一致性」，是一筆划算的交易。理解這一點，你也就明白了它為何不適合當作需要頻繁小改的線上交易資料庫——那是第貳篇後續 NoSQL 與資料庫的舞台。

5.5 物件儲存與 S3 相容介面

5.5.1 從檔案到物件

HDFS 是「地端」大數據時代的產物——它假設你自己擁有並維護一整座機房的機器。但到了雲端時代，一種更符合雲端精神的儲存形態興起了，那就是「物件儲存」（object storage）。

物件儲存不再模仿傳統檔案系統的「資料夾層層巢狀」結構，而是採用「扁平」的命名空間：每個資料單位是一個「物件」，由一個唯一的鍵（key）來標識，透過網路 API（而非檔案系統呼叫）來存取。你不再「開啟、讀取、關閉」一個檔案，而是「PUT、GET」一個物件。這種介面天生為網路與雲端而生，其中亞馬遜 S3 的 API 已成為業界的事實標準，眾多服務都提供「S3 相容」介面。

5.5.2 物件儲存的特質

物件儲存之所以成為雲端大數據的主力，源於它幾個鮮明的特質。

- **近乎無限的擴充：**由雲端供應商代為管理底層，使用者眼中的容量近乎無限，無須自行規劃機器。
- **極高的耐久性：**供應商在背後自動做多副本與跨區複製，資料耐久性極高（常宣稱多個九）。
- **低廉的成本：**單位儲存成本遠低於自建，且可依存取頻率分層計費（呼應第 2 章的冷熱資料）。
- **儲存與運算分離：**資料存在物件儲存中，運算叢集需要時才啟動、用完即釋放，兩者獨立擴縮。

其中「儲存與運算分離」最為關鍵——它正是第 1 章談過的三大架構演進之一。在 HDFS 的舊模型裡，資料與運算綁在同一批機器上；而在物件儲存的新模型裡，資料靜靜躺在儲存服務中，運算則像用電一樣隨需取用。這個鬆綁，是現代資料湖與 Lakehouse 得以成立的根本。

面向	HDFS（分散式檔案系統）	物件儲存
命名空間	階層式（資料夾）	扁平（鍵值）
存取方式	檔案系統呼叫	網路 API（PUT／GET）
部署	自建自管的地端叢集	雲端託管服務
運算關係	資料與運算常同置	儲存與運算分離
典型代表	HDFS	S3 及各家 S3 相容服務

表 5-3 分散式檔案系統與物件儲存的比較

5.6 資料湖的儲存層與檔案格式

5.6.1 什麼是資料湖

有了物件儲存這個近乎無限、低廉又耐久的地基，一種新的資料架構應運而生：資料湖（Data Lake）。它的核心理念是——先把各種來源、各種型態的原始資料，通通倒進一個中央儲存庫（通常就是物件儲存），暫不強制結構，等到要分析時再賦予它意義。

這正是第 4 章「讀時綱要」的大規模體現。相對於傳統資料倉儲的「先建好結構才放資料」（寫時綱要），資料湖「先放再說」，換來了極高的彈性——它能同時容納結構化的交易表、半結構化的日誌、與非結構化的影像聲音，全部堆在同一個湖裡。

5.6.2 儲存層與運算層的分工

資料湖架構把系統清楚地切成兩層。儲存層由物件儲存擔綱，負責便宜、耐久地存放海量原始資料；運算層則由各種引擎（如第參篇要談的 Spark）擔綱，需要分析時才去讀取儲存層的資料。

這種分工的好處，正是儲存與運算能各自依需求獨立擴縮：資料只增不減時，儲存層默默長大而不必動用運算資源；而遇到大型分析時，運算層可臨時召集大量節點、算完即散。這是雲端經濟性的精髓，也讓資料湖的總成本遠低於「儲存運算綁死」的舊架構。

5.6.3 檔案格式決定成敗

資料湖裡的資料，最終還是以「檔案」的形式躺在物件儲存中——而選什麼檔案格式，直接決定了分析的效率。這裡就接回了第 4 章的結論：分析型資料應以欄式格式（如 Parquet）儲存，才能只讀需要的欄、享受高壓縮，把物件儲存的低成本與欄式的高效率疊加起來。

不過，純粹把 Parquet 檔案堆進物件儲存，仍缺少了資料庫該有的一些能力——例如交易、綱要管理、時間旅行。為了補上這一塊，業界進一步發展出「開放表格式」（如 Delta Lake、Iceberg、Hudi），在檔案之上疊加一層中繼資料管理，讓資料湖也能具備類似資料庫的可靠性。這個把「資料湖的彈性」與「資料倉儲的可靠」合而為一的架構，就是所謂的 Lakehouse——本書第 30 章會專門深入。

5.6.4 承先啟後

本章我們從「一個檔案裝不進一台機器」這個最樸素的問題出發，看它如何催生出分散式檔案系統，並見證第 3 章的複製與容錯原理如何化為 HDFS 的區塊、副本與心跳機制；接著跟隨技術的演進從地端的 HDFS 走到雲端的物件儲存，再到資料湖的架構雛形。這一章解決的是「海量資料存在哪裡」的問題。但把資料存下來只是第一步——當我們需要對這些資料做快速的查詢、更新與多樣的存取時，光有檔案系統就不夠了。這就要進入下一章：NoSQL 資料庫與資料建模。

公式與流程：副本、儲存成本與耐久性

複製能提升可靠性，但天下沒有白吃的午餐——多存幾份，就要多花幾份的空間。這一節我們把「複製的成本」與「複製的效益」都量化，讓你能做出有依據的取舍。

儲存放大

最直接的成本是「儲存放大」。若每個區塊保存 R 份副本，則實際佔用的儲存空間，是原始資料量的 R 倍：

$$\text{實際儲存} = \text{原始資料量} \times R$$

式 5-1 副本機制的儲存放大

以 HDFS 預設的三副本為例，存 100 TB 的原始資料，實際要準備 300 TB 的硬碟——有效儲存率僅約 33%。這個「三倍成本」正是近年「抹除編碼」（erasure coding）技術想改善的目標，它能以較低的空間開銷達到相近的容錯，但代價是修復時的運算較重。

耐久性

那麼，三副本換來的耐久性有多高？沿用第 3 章式 3-1 的獨立性假設：若單一副本在某段期間內遺失的機率為 f ，且 R 份副本彼此獨立，則「三份同時遺失」（即資料真正遺失）的機率為 f^R ，故資料存活的機率為：

$$\text{資料存活機率} = 1 - f^R$$

式 5-2 R 份獨立副本的資料存活機率

假設單一副本在一年內遺失的機率為 1% ($f = 0.01$)，下表顯示副本數如何改變資料的年存活機率與儲存成本。

副本數 R	資料存活機率	儲存放大	遺失機率
1 (不複製)	99%	1 倍	1%
2	99.99%	2 倍	0.01%
3 (HDFS 預設)	99.9999%	3 倍	0.0001%
4	99.999999%	4 倍	0.000001%

表 5-4 副本數對耐久性與成本的影響 (單副本年遺失率 1%)

叫獸提醒

看表 5-4 的取舍：從一份到三份，遺失機率從 1% 驟降到百萬分之一，代價是三倍的儲存空間。為什麼業界普遍選「三」而非更多？因為從三份到四份，可靠性的邊際改善已經很小（多一個九），成本卻又多三分之一——這是典型的邊際效益遞減。三副本，正是可靠性與成本之間那個被實務驗證過的甜蜜點。這也再次呼應式 5-2 的前提：這一切都建立在『副本彼此獨立』之上，這正是機架感知放置存在的理由。

案例研討

案例一 裝不進一台機器的檔案

回到叫獸開講的難題。設想要儲存一個 1 TB 的基因定序檔案，而手上每台機器的硬碟只有 500 GB。傳統檔案系統對此束手無策——它假設一個檔案必須完整躺在一顆硬碟上。

分散式檔案系統的解法優雅而直接：把這 1 TB 切成約八千個 128 MB 的區塊，分散存到數十台機器上，每個區塊再各保存三份。對使用者而言，他看到的仍是一個完整的 1 TB 檔案，渾然不覺底下的切塊與分散。這個案例把本章的核心一次講清：切塊（突破容量牆）、分散（利用多機吞吐）、複製（容錯）——三者合力，翻過了單機的天花板。

案例二 一個機架斷電的午後

某資料中心的一個機架因電源模組故障而整體斷電，該機架上二十台 DataNode 同時離線。若副本策略設計不當——例如某些區塊的三份副本恰好都在這個機架——那麼這些區塊將瞬間全部遺失。

但因為 HDFS 採機架感知放置，任何區塊的三份副本都不會全擠在同一機架，因此這次斷電雖讓大量副本離線，卻沒有任何一個區塊「三份全失」。NameNode 偵測到這些節點失去心跳後，自動從其他機架上倖存的副本重新複製，逐步把副本數補回三份。停電的機架恢復供電後，其上的副本則成為多餘而被清理。整起事件中，沒有任何資料遺失、服務也未中斷——這是 5.3 節容錯機制與 3.4 節可用性原理的完美合演。

案例三 從自建 HDFS 搬上雲端物件儲存

一家公司早年自建了一座數百節點的 HDFS 叢集。隨著資料成長，他們發現痛點越來越多：機房空間見底、硬體汰換昂貴、且叢集的運算與儲存綁死——明明只是想多存資料，卻被迫連運算節點一起買。

他們最終把資料遷移到雲端物件儲存，改採儲存運算分離的架構。此後，儲存容量近乎無限且按量計費，運算叢集則在需要跑分析時才啟動、用完即釋放。總成本顯著下降，維運負擔也大幅減輕。這個案例正是第 1 章「儲存運算分離」演進與本章 5.5 節的真實寫照——技術的演進，往往是為了解開前一代架構綁死的東西。

案例四 資料湖裡的格式之爭

某團隊把所有原始資料以 JSON 檔案倒進雲端物件儲存，建立了資料湖。初期一切順利，但隨著資料量成長到數十 TB，分析查詢越來越慢——每次查詢都得把龐大的 JSON 檔案整個讀出、解析，即使只用到其中幾個欄位。

他們的解法分兩步：先把原始 JSON 轉存為 Parquet 欄式格式，查詢立刻快了數倍（呼應第 4 章欄式儲存的效益）；接著再導入開放表格式，為資料湖補上交易與綱要管理的能力，邁向 Lakehouse。這個案例串起了本章 5.6 節與第 4 章、並預告第 30 章——它說明資料湖的成敗，不只在於「把資料放進去」，更在於「用對格式放進去」。

實作活動

活動一 推演一次區塊切分

請以紙筆推演分散式檔案系統如何處理一個大檔案。

- 步驟 1.** 假設有一個 1 GB 的檔案、區塊大小為 128 MB，計算它會被切成幾個區塊。
- 步驟 2.** 若每個區塊保存 3 份副本，總共會產生幾份區塊、佔用多少實際儲存空間？
- 步驟 3.** 假設有 6 台 DataNode，試著畫出一種「機架感知」的副本放置方式（假設兩個機架各三台）。
- 步驟 4.** 討論：若某一整個機架故障，你畫的放置方式能否保證每個區塊都還有副本存活？

活動二 體驗物件儲存的 API

請使用任一雲端供應商的免費層，或本機的 S3 相容模擬服務，體驗物件儲存的操作。

- 步驟 1.** 建立一個儲存桶（bucket），並以 PUT 上傳一個檔案作為物件。
- 步驟 2.** 以 GET 取回該物件，確認內容一致。
- 步驟 3.** 觀察物件的鍵（key）——注意它是扁平的字串，而非真正的資料夾階層。
- 步驟 4.** 反思：這種「PUT/GET 物件」的介面，與你熟悉的「開啟/讀取檔案」有何不同？

若無法取用雲端服務，也可用開源的 S3 相容工具在本機架設，同樣能體會扁平命名空間與 API 存取的特性。

活動三 為你的資料湖選格式

請設想你要為工廠的感測資料建立一個資料湖。準備一份包含多欄位的模擬資料，分別存成 JSON 與 Parquet 兩種格式放入本機模擬的物件儲存；比較兩者的檔案大小，並撰寫一個「只取其中兩、三欄做彙總」的查詢，比較讀取效率。結合第 4 章的式 4-1，說明你會建議這個資料湖採用哪種格式，理由為何。

延伸閱讀

- **GFS 原始論文**：Google 於二〇〇三年發表的《The Google File System》。篇幅適中、行文清晰，是理解本章所有設計決策的第一手源頭。
- **Apache HDFS 官方架構文件**：說明 NameNode/DataNode、區塊、副本放置與高可用的實作細節，是 5.2、5.3 節的權威補充。
- **《Designing Data-Intensive Applications》第 5 章**：Kleppmann 與 Riccomini 著，關於複製與副本放置的討論，可與本章 5.3 節相互印證。
- **亞馬遜 S3 與物件儲存概念文件**：說明物件儲存的命名空間、耐久性與一致性模型，是 5.5 節的實務參考。

本章小結

1. 分散式檔案系統為突破單機的容量與可靠性天花板而生，其核心手法是把大檔案切成區塊、分散存於多台機器，並對外呈現為單一檔案系統。
2. GFS/HDFS 的三個設計目標是：為大檔案而生、假設硬體必然故障、為循序讀寫最佳化；它刻意犧牲隨機小改寫的能力，以貼合大數據「整批寫、大量讀」的工作負載。
3. HDFS 採主從架構，把中繼資料與資料分離：NameNode 管理檔案地圖與區塊位置，DataNode 實際儲存區塊；此分離讓管理與儲存能各自獨立擴充，NameNode 則以高可用設計避免單點故障。
4. 區塊切得大（預設 128 MB）以減輕中繼資料負擔並最佳化循序讀取；每區塊預設三副本，並以機架感知放置分散於不同機架；DataNode 以心跳回報狀態，NameNode 偵測故障後自動重新複製，實現自我修復。
5. 寫入採接力複製管線、讀取由客戶端直連 DataNode（資料不經 NameNode）；一致性採「一次寫入、多次讀取」模型，以放棄任意修改換取簡單可靠的一致性。
6. 物件儲存以扁平命名空間與網路 API 取代階層式檔案系統，具備近乎無限擴充、極高耐久、低成本與儲存運算分離等特質，S3 API 為事實標準，是雲端資料湖的地基。
7. 副本帶來儲存放大（式 5-1）與耐久性提升（式 5-2）的取舍；三副本是可靠性與成本的實務甜蜜點；資料湖以物件儲存為儲存層、運算引擎為運算層，並須以欄式格式與開放表格式，才能兼顧彈性、效率與可靠。

習題

一、觀念題

1. 請說明 HDFS 為何要把 NameNode 與 DataNode 的職責分開，這樣的分離帶來什麼好處？
2. HDFS 的區塊為何要設計得那麼大（如 128 MB）？請從中繼資料負擔與循序讀取兩方面說明。
3. 何謂機架感知的副本放置？它如何回應第 3 章「副本須彼此獨立」的前提？
4. 請比較分散式檔案系統與物件儲存在命名空間、存取方式與運算關係上的差異。

二、計算題

1. 有一個 5 GB 的檔案，HDFS 區塊大小為 128 MB、副本數為 3。請計算：（一）此檔案被切成幾個區塊（不足一塊者以一塊計）？（二）連同副本，共產生幾份區塊？（三）實際佔用多少儲存空間？
2. 假設單一副本在一年內遺失的機率為 2%。請以式 5-2 計算保存 2 份與 3 份獨立副本時，資料的年存活機率；並說明為何從 2 份增為 3 份，可靠性的邊際改善相對有限。

三、思考與討論

1. HDFS 採「一次寫入、多次讀取」模型，幾乎不支援在檔案中間任意修改。請討論：這個看似「缺陷」的設計，為何反而適合大數據的工作負載？又在什麼情況下它會成為真正的限制？
2. 案例三中，公司從自建 HDFS 遷移到雲端物件儲存。請討論：這樣的遷移帶來哪些好處？又可能引入哪些新的風險或考量（可回顧第 1 章 1.3.4 節）？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

NameNode 管中繼資料（檔案地圖、區塊位置），DataNode 管資料本身（實際區塊）。分離的好處：其一，管理與儲存可各自獨立擴充——中繼資料量小、集中於 NameNode 記憶體以求快速查詢，資料量大則水平分散到眾多 DataNode；要增容量只需加 DataNode，NameNode 不必跟著長大。其二，職責清晰，資料傳輸可繞過 NameNode（讀取時客戶端直連 DataNode），避免主節點成為資料流瓶頸。可引表 5-2。

第 2 題

兩方面：其一，中繼資料負擔——NameNode 須為每個區塊記錄中繼資料，區塊越大、數量越少，中繼資料越少，NameNode 記憶體才裝得下。其二，循序讀取——磁碟讀取的主要開銷在「尋找起點」，區塊越大，一次尋找後可連續讀取的資料越多，尋找的相對成本越低。兩者都呼應 5.1 節「為大檔案、為循序讀寫」的設計目標。

第 3 題

機架感知放置指副本放置時考量節點的實體機架位置：如第一份在寫入端節點、第二份放到不同機架、第三份放在第二份所在機架的另一節點，使三份副本不會全擠在同一機架。它直接回應第 3 章式 3-1 的「彼此獨立」前提——同機架共用電源與交換器，屬共同故障源；把副本跨機架分散，才能在整個機架故障時仍有副本存活（見案例二）。

第 4 題

命名空間：分散式檔案系統為階層式（資料夾巢狀），物件儲存為扁平（鍵值）。存取方式：前者用檔案系統呼叫（開啟／讀取／關閉），後者用網路 API（PUT／GET）。運算關係：前者資料與運算常同置（資料在地性），後者儲存與運算分離。可引表 5-3，並補充物件儲存為雲端託管、近乎無限擴充。

二、計算題

第 1 題

$5\text{ GB} = 5 \times 1024\text{ MB} = 5120\text{ MB}$ ，區塊 128 MB 。

- (1) 區塊數 $= 5120 \div 128 = 40$ 個（整除，恰 40 塊）。
- (2) 連同 3 份副本： $40 \times 3 = 120$ 份區塊。
- (3) 實際儲存 $= 5\text{ GB} \times 3 = 15\text{ GB}$ （式 5-1）。

延伸思考：若檔案為 5.1 GB (5222.4 MB)，則區塊數 = 無條件進位($5222.4 \div 128$) = 41 塊，最後一塊未滿也算一塊——這說明區塊是分配的最小單位。

第 2 題

套用式 5-2，資料存活機率 = $1 - f^R$ ， $f = 0.02$ 。

(1) $R = 2: 1 - 0.02^2 = 1 - 0.0004 = 0.9996$ (99.96%)。

(2) $R = 3: 1 - 0.02^3 = 1 - 0.000008 = 0.999992$ (99.9992%)。

邊際改善有限的原因：從 2 份到 3 份，遺失機率由 0.04% 降到 0.0008%，雖仍是大幅下降，但對多數應用而言 99.96% 與 99.9992% 的差異在實務上已難以察覺；而成本卻實實在在多了 50% (2 倍變 3 倍儲存)。這正是邊際效益遞減——可靠性的每一個額外的「九」，都要付出越來越不成比例的成本。

三、思考與討論

第 1 題

適合的原因：大數據的典型工作是「產生一批資料、存下、反覆分析」，很少在原地細修；放棄任意修改，換來的是大幅簡化的一致性（沒有多方同時改同一處的問題），使系統更簡單可靠、且利於循序高吞吐。成為限制的情況：當應用需要頻繁的隨機小量更新時——如線上交易系統要即時改某筆訂單狀態——HDFS 就不適用，此時該用專為此設計的資料庫（第貳篇後續的 NoSQL 與關聯式資料庫）。核心觀念是「設計貼合工作負載」。

第 2 題

好處：儲存容量近乎無限且按量計費、免除機房與硬體汰換的維運、儲存與運算解綁（可各自擴縮、運算用完即釋放）、總成本與人力負擔下降。新風險與考量（呼應 1.3.4）：成本失控（不當查詢可能掃過大量資料而產生高額帳單）、廠商鎖定（大量使用專屬服務後遷移困難，宜採開放格式）、資料主權與法規（資料落在哪個區域受當地法規管轄）、以及網路依賴與延遲（存取需經網路）。周延的回答會指出遷移是一種取捨，而非全然的升級。

第貳篇 分散式儲存與資料庫

Distributed Storage & Databases

第 6 章

NoSQL 資料庫與資料建模

NoSQL Databases and Data Modeling

叫獸開講

二〇〇〇年代中期，幾家網路巨頭同時撞上了同一堵牆。亞馬遜的購物車、臉書的訊息、Google 的網頁索引——這些系統要服務的資料量與流量，已經大到任何一台再昂貴的資料庫伺服器都扛不住。他們手上的關聯式資料庫，是為單機或少數幾台機器設計的，面對這種規模，就像要一頭牛去拉一列火車。

關聯式資料庫並沒有錯——它嚴謹、可靠、SQL 強大，統治了資料庫世界三十年。但它有兩個在超大規模下變得致命的堅持：第一，它預設所有資料要放在一起才能做關聯查詢，難以水平切分到成百上千台機器；第二，它為了保證交易的絕對正確，寧可犧牲可用性——而對一個購物網站來說，「暫時看到稍舊的庫存」遠比「整個網站當機」要好。

於是這些公司做了一件離經叛道的事：他們放棄了關聯式資料庫的某些堅持，自己打造新的儲存系統。亞馬遜寫出了 Dynamo，Google 寫出了 Bigtable。這些系統被統稱為 NoSQL——這個名字常被誤會成「沒有 SQL」，但它真正的意思其實是「Not Only SQL」（不只是 SQL）。它們不是要取代關聯式資料庫，而是為關聯式模型不擅長的場景，提供另一種選擇。

這一章，我們要認識 NoSQL 這個大家族。你會看到它為何而生、有哪幾種類型、各自適合什麼；也會遇到本章最重要的一組觀念——為了換取極致的可擴充與可用，NoSQL 常放寬了對「一致性」的要求，這就是 BASE 與最終一致性。上一章 HDFS 解決的是「大檔案存哪裡」，這一章要解決的則是「海量的、需要快速讀寫的結構化資料，該放進什麼樣的資料庫」。

學習目標

研讀完本章之後，你應該能夠：

1. 說明 NoSQL 興起的背景，以及它與關聯式資料庫的根本差異。
2. 辨識鍵值、文件、欄族與圖四大類 NoSQL 資料庫，並說明各自的適用場景。
3. 說明欄族資料庫（如 Bigtable、Cassandra）的資料組織方式與強項。
4. 區分 ACID 與 BASE 兩種取舍哲學，並解釋最終一致性的意義。
5. 運用法定人數（quorum）機制，判斷讀寫如何達成可調的一致性。
6. 說明 NoSQL 的資料建模原則，以及反正規化為何在此成為常態。

核心概念

核心概念	說明
NoSQL	「不只是 SQL」，泛指為超大規模與高可用而放寬部分關聯式堅持的資料庫。
四大類型	鍵值、文件、欄族、圖，各以不同方式組織資料、擅長不同查詢。
欄族資料庫	以列鍵與欄族組織稀疏大表的資料庫，如 Bigtable、Cassandra。
ACID 與 BASE	前者強調交易的絕對正確，後者以最終一致換取高可用與可擴充。
最終一致性	允許副本暫時分歧，但保證在沒有新寫入後，終將趨於一致。
法定人數（Quorum）	以讀寫需徵得的最少節點數來調節一致性強度的機制。

核心概念	說明
反正規化	刻意讓資料重複以避免關聯查詢，是 NoSQL 常見的建模手法。

表 6-1 本章核心概念一覽

6.1 NoSQL 的興起與分類

6.1.1 為什麼需要 NoSQL

承接叫獸開講，NoSQL 的興起是被三股力量推出來的，它們與第 3 章談的分散式動機一脈相承。

- **超大規模的資料與流量：**當資料量與請求量超出單機資料庫的極限，就需要能天生水平切分到大量機器的系統。
- **對高可用的極致要求：**對許多網路服務而言，「一直能用」比「永遠絕對正確」更重要；寧可回稍舊的資料，也不要整個服務停擺。
- **彈性的資料型態：**現代應用的資料型態多變，關聯式模型「先定死綱要」的僵固，成為快速迭代的絆腳石。

要特別強調：NoSQL 不是「更好的關聯式資料庫」，而是做了「不同取捨」的資料庫。它用放棄一致性、放棄部分查詢能力，換來了可擴充性與可用性。這正是第 3 章 CAP 定理的實踐——沒有全能的系統，只有針對特定需求的取捨。

6.1.2 四大類型概觀

NoSQL 是一個大家族，依資料組織方式可分為四種主要類型。理解它們的差異，是選型的第一步。

類型	資料組織	強項	典型場景
鍵值	一個鍵對應一個值	極快的讀寫、簡單	快取、工作階段、計數
文件	巢狀的 JSON 文件	彈性綱要、整包存取	使用者設定檔、內容管理
欄族	列鍵下的多欄族稀疏表	海量寫入、可擴充	時序、日誌、大規模紀錄
圖	頂點與邊	多層關聯與路徑查詢	社群、推薦、詐騙偵測

表 6-2 NoSQL 四大類型概觀

叫獸提醒

你會發現，這四類其實對應了第 4 章的資料模型，只是換上了「為分散式而生」的外衣。文件與圖你在第 4 章已見過；鍵值可看成最精簡的資料模型；欄族則是為海量而生的新面孔。所以別把 NoSQL 當成全新的怪獸——它的骨子裡，還是第 4 章那句話：模型沒有最好，只有最適合。選 NoSQL 的哪一類，問的仍是同一個問題：我的資料長什麼樣、我最常怎麼查？

6.2 鍵值與文件資料庫

6.2.1 鍵值資料庫：極簡的力量

鍵值資料庫是最單純的一種：它就像一本超大的字典，你給一個鍵（key），它還你一個值（value），如此而已。它不關心值裡面裝的是什麼——可能是一個數字、一段文字，或一整包序列化的物件。

正因為極簡，它也極快。由於存取模式單純（依鍵取值），它很容易被切分到大量機器上，達到極高的吞吐與極低的延遲。這使它成為快取、使用者工作階段、即時計數、排行榜等場景的首選。它的限制也源於這份極簡：你只能用鍵來查資料，無法像 SQL 那樣依「值的內容」做複雜查詢。

6.2.2 文件資料庫：帶結構的彈性

文件資料庫可以看成鍵值資料庫的進化：值不再是不透明的一團，而是一份「有結構、可查詢」的文件（通常是 JSON）。這讓它比鍵值多了一項能力——你可以依文件內部的欄位來查詢與索引。

它繼承了第 4 章文件模型的特質：彈性綱要（不同文件可有不同欄位）、擅長巢狀的一對多資料、以及「整包讀寫」的效率。它特別適合資料型態多變、需要快速迭代的應用，例如使用者設定檔、產品目錄、內容管理系統。相對地，處理跨文件的多對多關聯查詢，仍是它的弱項。

6.3 欄族資料庫（Bigtable／Cassandra）

6.3.1 一張可以無限寬又極稀疏的表

欄族資料庫是四大類型中最容易被誤解的一種，因為它的名字裡有「欄」，容易讓人聯想到第 4 章的欄式儲存——但兩者是不同的概念。這裡我們把它講清楚。

欄族資料庫（如 Google 的 Bigtable、開源的 Cassandra 與 HBase）把資料組織成一張巨大的表，但這張表有兩個關鍵特性。其一，它可以「無限寬」——每一列可以有數以萬計、甚至百萬計的欄，而且不同列的欄可以完全不同。其二，它是「稀疏」的——某一行沒有的欄，完全不佔空間，不像關聯式表那樣每欄都要留位子（就算是空值）。

這種結構天生適合「維度極多、且大量留白」的資料。經典的例子是 Google 用 Bigtable 儲存網頁：以網址為列鍵，每個網頁可能連到成千上萬個其他網頁，這些連結就成了該列的欄——每個網頁的連結數與對象都不同，且絕大多數的「欄位組合」是空的。用關聯式表來裝這種資料，會是一場災難；用欄族表，卻恰到好處。

6.3.2 為海量寫入而生

欄族資料庫的另一個強項是驚人的寫入吞吐與線性的可擴充性。這並非偶然——它的底層儲存引擎，正是第 4 章談過的 LSM-tree。追加式的寫入讓它能吞下海量的資料湧入，而它的分散式設計（如 Cassandra 的無主架構）讓你只要加機器，吞吐就近乎線性地成長。

這使它成為時序資料、感測紀錄、活動日誌等「寫入密集、且持續成長」場景的理想選擇——正是本學程最貼近的資料型態。回想第 2 章的物聯網感測資料、第 4 章的 LSM-tree 選型案例，你會發現它們都指向同一類技術。

面向	關聯式表	欄族表
欄位	固定、每列相同	動態、每列可不同、可極多
空值	仍佔位子	不存在即不佔空間（稀疏）
底層引擎	多為 B-tree	多為 LSM-tree

面向	關聯式表	欄族表
強項	複雜關聯查詢	海量寫入、線性擴充

表 6-3 關聯式表與欄族表的比較

6.4 圖資料庫與查詢

6.4.1 當關係本身是主角

圖資料庫我們在第 4 章已初步認識：它以頂點表示實體、以邊表示關係，專為「關係錯綜複雜、且查詢常需多層跳躍」的資料而生。這裡我們補充它在 NoSQL 家族中的定位與查詢方式。

圖資料庫與其他三類 NoSQL 有個有趣的反差：鍵值、文件、欄族都在「淡化資料間的關聯」以換取可擴充性，而圖資料庫恰恰相反——它把關聯當成第一等公民，全力最佳化「沿著關係走」的查詢。這使它在其他資料庫最吃力的地方（多層關聯）表現最出色，但也讓它較難像其他 NoSQL 那樣輕易水平切分（因為關係會跨越切分的邊界）。

6.4.2 圖查詢的威力

圖資料庫的查詢語言，讓「沿關係遍歷」變得直觀。以「找出使用者的朋友的朋友」為例：在關聯式資料庫中，這需要一層層的表格自我連接，寫起來繁瑣、跑起來緩慢（回想第 4 章案例四）；在圖資料庫中，這只是「從某頂點出發，沿著『朋友』邊走兩步」，語法簡潔、執行高效。

這種「多層遍歷」的能力，在許多場景中價值連城：社群網路的關係推薦、詐騙偵測中的可疑金流環路、知識圖譜的推理、供應鏈的溯源。本書第 18 章談連結分析與大規模圖處理時，會把圖的演算法（如 PageRank）進一步展開。

6.5 BASE 與最終一致性

這一節是本章的理論核心，也是理解 NoSQL 取捨哲學的關鍵。它直接承接第 3 章的 CAP 定理，把那個抽象的取捨，變成一套具體的設計哲學。

6.5.1 從 ACID 到 BASE

傳統關聯式資料庫奉行 ACID：原子性、一致性、隔離性、持久性。它的精神是「寧可慢、寧可拒絕服務，也要保證每一刻的資料都絕對正確」。這在銀行轉帳等場景中至關重要——你絕不希望帳戶餘額有一瞬間是錯的。

但在超大規模、高可用的場景下，ACID 的堅持會與可用性衝突（正是 CAP 的取捨）。於是 NoSQL 界提出了另一套哲學，戲稱為 BASE：基本可用（Basically Available）、軟狀態（Soft state）、最終一致（Eventually consistent）。它的精神恰與 ACID 相反：「寧可暫時不一致，也要保持隨時可用」。

面向	ACID（關聯式）	BASE（NoSQL）
核心精神	絕對正確優先	持續可用優先
一致性	強一致（隨時正確）	最終一致（終將正確）
可用性	分割時可能拒絕服務	分割時仍盡量回應

面向	ACID (關聯式)	BASE (NoSQL)
適合場景	金融、庫存、帳務	社群、瀏覽、日誌、快取

表 6-4 ACID 與 BASE 的對照

6.5.2 最終一致性到底是什麼

「最終一致性」常被初學者誤解為「資料會不一致」，其實不然。它的正確意思是：允許各副本在短時間內暫時分歧，但保證只要停止新的寫入，經過一段時間後，所有副本終將收斂到相同的值。

用一個生活比喻：你在社群平台改了大頭貼，可能你的朋友過幾秒才看到新照片——這幾秒內，不同人看到的版本暫時不一致，但很快就會一致。對社群這種場景，這種短暫的分歧完全可以接受，換來的卻是系統在任何情況下都能快速回應。這就是最終一致性的價值：用「短暫、可接受的不一致」，換取「持續的高可用與低延遲」。

6.5.3 法定人數：可調的一致性

最巧妙的是，一致性的強弱其實可以「調」。許多 NoSQL 系統採用「法定人數」(quorum) 機制：設資料共有 N 份副本，每次寫入須至少 W 份副本確認才算成功，每次讀取須至少從 R 份副本讀取。透過調整 W 與 R ，就能在一致性與效能之間自由滑動。

關鍵的規則是：只要 $W + R > N$ ，讀取集合與寫入集合就必然重疊，這保證你每次讀取至少會碰到一份「有最新寫入」的副本，因而讀得到最新值——這就達成了強一致。反之，若 $W + R \leq N$ ，則可能讀到全是舊值的副本，得到的就是最終一致。下一節的公式與流程會把這個機制算給你看。

6.6 NoSQL 資料建模與反正規化策略

6.6.1 思維的大翻轉：先想查詢，再設計資料

關聯式資料庫的建模有一套經典方法：正規化 (normalization) ——把資料拆成一張張沒有重複的表，靠關聯把它們串起來。這在關聯式世界是美德，因為它消除了資料冗餘、避免了更新時的不一致。

但在 NoSQL 世界，思維要來個一百八十度大翻轉。關聯式是「先設計好資料結構，再想能怎麼查」；NoSQL 則是「先確定要怎麼查，再據此設計資料結構」。原因很直接：NoSQL 多半不擅長（甚至不支援）跨表的關聯查詢，所以你不能指望查詢時再把資料湊起來——你必須在寫入時，就把「未來會一起被讀取的資料」先組織在一起。

6.6.2 反正規化：刻意的重複

這個思維翻轉，直接導出了 NoSQL 最鮮明的建模手法：反正規化 (denormalization) ——刻意讓資料重複。

舉個例子：一篇部落格文章與它的作者資訊。在關聯式設計中，文章表只存作者 ID，作者的姓名與頭像放在另一張作者表，查詢時再關聯起來。但在 NoSQL 中，你可能會直接把作者的姓名與頭像，複製一份存進每一篇文章的文件裡——這樣讀取文章時，一次就拿到所有需要的資料，不必再去關聯。

這當然有代價：資料重複佔了更多空間，而且作者改名時，得去更新所有文章裡的副本。但這正是 NoSQL 的取舍哲學——它用「較多的儲存空間」與「較複雜的更新」，換取「極快的讀取」。在

讀取遠多於寫入、且儲存便宜的今天，這往往是一筆划算的交易。這也再次呼應了第 4 章「用空間換時間」的反覆主題。

叫獸提醒

初學關聯式資料庫的同學，第一次看到 NoSQL 鼓勵「資料重複」，往往覺得渾身不對勁——正規化的訓練告訴你重複是萬惡之源。但請記住：正規化與反正規化，是為了不同目標而生的不同工具。正規化為了「寫入時的一致與省空間」，反正規化為了「讀取時的快與簡單」。沒有對錯，只有你的應用是讀多還是寫多。學會在兩者間依情境切換，才是真正掌握了資料建模。

6.6.3 承先啟後

本章我們走過了 NoSQL 的四大類型、認識了 BASE 與最終一致性的取捨哲學，也翻轉了資料建模的思維。你會發現，這一章其實是前幾章原理的一場「實戰演練」：第 3 章的 CAP 在這裡化為 ACID 與 BASE 的抉擇，第 4 章的資料模型與 LSM-tree 在這裡化為四大類型與欄族的引擎，第 5 章的分散思維在這裡化為法定人數的讀寫。

但我們刻意留下了一個大問題還沒細講：當資料被複製成多份、又切分到多台機器時，這些副本與分區之間，究竟如何協調、如何保持（最終）一致、衝突了又如何解決？本章只點到為止的這些機制，正是下一章「資料複製、分區與一致性」要正式攤開來細談的主題。

公式與流程：法定人數與可調的一致性

6.5.3 節說一致性可以「調」，關鍵就在法定人數的一條不等式。這一節我們把它講透，讓你能親手為系統設定一致性強度。

設一份資料共有 N 份副本。每次寫入須至少 W 份副本確認成功（寫入法定人數），每次讀取須至少從 R 份副本讀取（讀取法定人數）。要保證「讀得到最新寫入」（強一致），必要條件是讀寫集合必然重疊，即：

$$W + R > N$$

式 6-1 強一致的法定人數條件

道理很直觀：寫入碰了 W 份、讀取碰了 R 份，若 $W + R$ 大於總數 N ，依鴿籠原理，這兩個集合至少有一份副本重疊——而那份重疊的副本，必然帶著最新的寫入。於是讀取一定會看到它，強一致因此成立。

透過調整 W 與 R ，可以在一致性、讀效能與寫效能之間自由取捨。以 $N = 3$ 為例，看幾種常見的設定。

W	R	特性	適合場景
3	1	寫慢讀快、強一致	讀遠多於寫、要求即時正確
1	3	寫快讀慢、強一致	寫遠多於讀、要求即時正確
2	2	讀寫均衡、強一致 ($2+2>3$)	讀寫相當、要求強一致的通用設定
1	1	讀寫皆快、僅最終一致 ($1+1<3$)	極致效能、可容忍短暫不一致

表 6-5 $N = 3$ 時不同法定人數設定的取捨**叫獸提醒**

看最後兩列的對照，你就懂了 CAP 在這裡是怎麼「按鈕化」的。 $W = R = 2$ 時， $2 + 2 = 4 > 3$ ，滿足式 6-1，強一致成立，但每次讀寫都要碰兩份副本，較慢也較不耐故障； $W = R = 1$ 時， $1 + 1 = 2$ ，不大於 3，只能保證最終一致，但讀寫都極快、且只要有一份副本活著就能服務。同一套系統、同一份資料，你竟然可以用兩個數字，就在『強一致』與『高可用』之間滑動——這就是法定人數的優雅之處，也是第 3 章 CAP 取捨最具體的一次落地。

案例研討**案例一 永不當機的購物車**

亞馬遜的購物車是 NoSQL 誕生的經典動因。對電商而言，購物車絕不能因為某台伺服器或某段網路故障就無法使用——哪怕使用者剛加入的商品晚幾秒才同步，也遠比「無法把商品加入購物車」要好。

因此亞馬遜的 Dynamo 選擇了 BASE 而非 ACID：它偏向可用性（AP），採用最終一致，並以法定人數調節。就算網路分割，各節點也照樣接受「加入購物車」的寫入，事後再讓副本收斂一致。這個設計把本章 6.5 節的取捨哲學展現得淋漓盡致——對購物車而言，「一直能用」的價值，遠高於「每一刻都絕對一致」。這也正是第 3 章案例二「購物車 AP、結帳 CP」的資料庫層面實作。

案例二 用欄族表裝下整個網路

Google 的 Bigtable 最初就是為了儲存網頁索引而生。想像要為全球數百億個網頁建索引，每個網頁有標題、內容、以及指向其他網頁的大量連結——每個網頁的連結數與對象都天差地別，且絕大多數「網頁對網頁」的組合是不存在的。

若用關聯式表，光是為這種「維度極多、極度稀疏」的資料留位子就會撐爆儲存。Bigtable 的欄族結構恰好完美契合：以網址為列鍵，每個連結成為一個動態的欄，沒有的連結完全不佔空間。加上 LSM-tree 引擎帶來的海量寫入能力，它輕鬆吞下了整個網路的規模。這個案例把 6.3 節「無限寬又稀疏」的抽象特性，變成了看得見的真實應用。

案例三 一次法定人數的調校

某團隊用 $N = 3$ 的分散式資料庫存放使用者的個人化設定。上線初期他們設 $W = R = 1$ ，追求極致效能，結果偶爾發生「使用者剛改了設定，重新整理卻看到舊值」的抱怨——因為 $1 + 1 = 2$ ，不滿足 $W + R > 3$ ，只是最終一致。

他們的修正很優雅：不必更換資料庫，只需把設定改為 $W = R = 2$ 。此時 $2 + 2 = 4 > 3$ ，滿足式 6-1，強一致成立，「改了立刻看得到」的問題迎刃而解。代價是每次讀寫多碰一份副本、略慢一些，但對設定這種讀寫都不算頻繁的場景完全可以接受。這個案例讓你看見：法定人數不是一次定死的，而是可以隨需求隨時「轉旋鈕」的活工具。

案例四 反正規化救了慢半拍的動態牆

一個社群 App 的「動態牆」需要顯示每則貼文以及發文者的姓名和頭像。工程師起初依關聯式的習慣正規化：貼文只存發文者 ID，顯示時再去使用者表把姓名頭像關聯進來。結果動態牆一次要顯示幾十則貼文，就得做幾十次關聯查詢，載入慢得令人抓狂。

他們改用反正規化：在每則貼文的文件中，直接複製一份發文者當下的姓名與頭像。此後顯示動態牆只需讀出貼文本身，一次到位，速度大幅提升。代價是使用者改頭像時，得在背景更新其歷史貼文中的副本——但這種寫入遠比讀取罕見，取捨划算。這個案例是 6.6 節反正規化的活教材，也再次印證了「讀多寫少就用空間換時間」的通則。

實作活動

活動一 為場景選對 NoSQL 類型

請為下列四個場景，各選出最適合的 NoSQL 類型並說明理由。

- 步驟 1. 一個需要極快讀寫的排行榜與工作階段快取。
- 步驟 2. 一個型態多變、需要快速迭代的產品目錄。
- 步驟 3. 一個每秒湧入數十萬筆、持續成長的感測時序資料。
- 步驟 4. 一個要做「可能認識的人」推薦的社群關係網路。

完成後，回顧第 4 章的資料模型，說明每個選擇背後對應的是什麼樣的資料關係與查詢型態。

活動二 推演你的法定人數

請以紙筆推演不同法定人數設定的效果。

- 步驟 1. 假設 $N = 5$ ，列出所有能滿足 $W + R > N$ 的 (W, R) 組合。
- 步驟 2. 在這些組合中，哪一組讀取最快（ R 最小）？哪一組寫入最快（ W 最小）？
- 步驟 3. 若你的應用是「寫少讀多、且要求強一致」，你會選哪一組？為什麼？
- 步驟 4. 若改為 $W = R = 2$ 、 $N = 5$ ，這是強一致還是最終一致？（提示：比較 $W + R$ 與 N ）

活動三 把正規化改成反正規化

請設計一個「線上課程與學生評論」的資料。先以關聯式的正規化方式，畫出「課程表」「學生表」「評論表」三張表與其關聯；接著假設你要用文件資料庫，且最常見的查詢是「顯示某課程的所有評論，含評論者姓名」。請把資料重新設計為反正規化的文件結構，並說明：這樣做讓什麼查詢變快了？又讓什麼更新變麻煩了？

延伸閱讀

- 《Designing Data-Intensive Applications》第 2、5、6 章：Kleppmann 與 Riccomini 著。第 2 章比較資料模型，第 5 章談複製與法定人數，第 6 章談分區，是本章 6.1、6.5 節的最佳依據。
- **Dynamo 論文**：亞馬遜二〇〇七年發表的《Dynamo: Amazon's Highly Available Key-value Store》，是最終一致與法定人數機制的經典源頭。
- **Bigtable 論文**：Google 二〇〇六年發表，說明欄族資料模型與其大規模設計，是理解 6.3 節的第一手資料。
- **Cassandra 官方資料建模文件**：以實例說明「先想查詢、再設計資料」與反正規化的建模原則，是 6.6 節的實務補充。

本章小結

1. NoSQL (不只是 SQL) 為超大規模、高可用與彈性資料型態而生，它不是更好的關聯式資料庫，而是做了不同取舍——放寬一致性與部分查詢能力，換取可擴充性與可用性，是第 3 章 CAP 的實踐。
2. NoSQL 分為鍵值（極快、依鍵存取）、文件（彈性綱要、整包存取）、欄族（海量寫入、稀疏大表）與圖（多層關聯查詢）四大類型，各對應不同的資料關係與查詢型態。
3. 欄族資料庫（Bigtable、Cassandra）以列鍵下的動態欄族組織稀疏大表，可無限寬且空欄不佔空間，底層多用 LSM-tree，擅長海量寫入與線性擴充，適合時序與日誌。
4. ACID 強調絕對正確優先，BASE 強調持續可用優先；最終一致性允許副本短暫分歧，但保證停止寫入後終將收斂，以可接受的短暫不一致換取高可用與低延遲。
5. 法定人數機制以 N 份副本、 W 份寫入確認、 R 份讀取，透過 $W + R > N$ （式 6-1）保證讀寫集合重疊而達成強一致；調整 W 、 R 即可在一致性、讀效能與寫效能間自由滑動。
6. NoSQL 建模思維與關聯式相反：先確定查詢、再設計資料；並以反正規化（刻意讓資料重複）避免關聯查詢，用較多空間與較複雜的更新，換取極快的讀取。
7. 正規化與反正規化沒有絕對優劣，而是為不同目標（寫入一致省空間 vs 讀取快而簡單）而生的工具；選擇取決於應用是讀多還是寫多。

習題

一、觀念題

1. 「NoSQL 就是沒有 SQL、要取代關聯式資料庫」——請指出這句話的兩處錯誤，並說明 NoSQL 真正的定位。
2. 請說明欄族資料庫「無限寬」與「稀疏」兩個特性，並舉一個適合用它儲存的資料例子。
3. 請比較 ACID 與 BASE 的核心精神，並解釋「最終一致性」為何不等於「資料會不一致」。
4. 為什麼 NoSQL 的資料建模常採用反正規化？這樣做用什麼代價換取了什麼好處？

二、計算題

1. 某分散式資料庫的副本數 $N = 5$ 。（一）若設定 $W = 3$ 、 $R = 3$ ，是否滿足強一致條件？（二）若要「讀取最快」又維持強一致， R 最小可設為多少（此時 W 為何）？（三）若 $W = 2$ 、 $R = 2$ ，屬強一致還是最終一致？
2. 某系統 $N = 3$ ，目前設定 $W = 1$ 、 $R = 1$ 以追求效能，但使用者反映偶有「改了看到舊值」。請說明此現象的原因（以 $W + R$ 與 N 的關係說明），並提出一個仍為 $N = 3$ 、但能達成強一致的最省成本設定。

三、思考與討論

1. 反正規化讓讀取變快，卻讓「作者改名」這類更新變麻煩。請討論：在什麼樣的讀寫比例與應用特性下，這個取舍是划算的？又在什麼情況下反而得不償失？
2. 有人主張「既然有了 NoSQL 的高可用與可擴充，關聯式資料庫可以淘汰了」。請根據本章與第 3 章的內容評論此說法。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

兩處錯誤：其一，NoSQL 意為「Not Only SQL（不只是 SQL）」，而非「沒有 SQL」——許多 NoSQL 也提供類 SQL 的查詢介面。其二，它並非要「取代」關聯式資料庫，而是為關聯式不擅長的場景（超大規模、高可用、彈性型態）提供另一種選擇。真正定位：做了不同取捨（放寬一致性與查詢能力，換取可擴充與可用）的資料庫，與關聯式互補而非替代。

第 2 題

無限寬：每列可有極多（數萬至百萬）欄，且不同列的欄可完全不同。稀疏：某列沒有的欄不佔任何空間，不像關聯式表空值仍留位子。適合的例子如：網頁連結索引（每個網頁連向不同且大量的其他網頁）、每位使用者屬性差異極大的事件紀錄、感測器時序（欄隨時間動態增加）。可引案例二 Bigtable 網頁索引。

第 3 題

ACID 精神是「絕對正確優先」，寧可慢或拒絕服務也要隨時一致；BASE 精神是「持續可用優先」，寧可暫時不一致也要隨時能回應。最終一致性不等於「會不一致」，因為它保證「只要停止新寫入，經過一段時間後所有副本終將收斂到相同值」——分歧是短暫且會自動消除的（如改大頭貼後朋友過幾秒才看到）。可引表 6-4。

第 4 題

因為 NoSQL 多不擅長跨表關聯查詢，無法在查詢時把分散的資料湊起來，故須在寫入時就把「會一起被讀取的資料」組織在一起——反正正規化正是把相關資料（含重複部分）預先放在一起。代價：資料重複佔更多空間、且更新時須同步多份副本；好處：讀取時一次到位、極快且簡單。在讀多寫少、儲存便宜的情境下划算（呼應「用空間換時間」）。

二、計算題

第 1 題

以式 6-1 ($W + R > N$, $N = 5$) 逐項判斷。

- (1) $W = 3$ 、 $R = 3$: $3 + 3 = 6 > 5$ ，滿足，為強一致。
- (2) 讀取最快即 R 最小，須維持 $W + R > 5$ ，故 R 最小為 1，此時 W 須 > 4 ，即 $W = 5$ 。
($W = 5$ 、 $R = 1$)
- (3) $W = 2$ 、 $R = 2$: $2 + 2 = 4$ ，不大於 5，屬最終一致。

延伸思考： $R = 1$ 讀最快但 $W = 5$ 寫最慢且最不耐寫入故障（須全部副本確認）——這說明「讀快」與「寫快」在強一致下不可兼得，須依讀寫比例取捨。

第 2 題

原因： $W = 1$ 、 $R = 1$ 時， $W + R = 2$ ，不大於 $N = 3$ ，不滿足式 6-1，讀寫集合未必重疊，故讀取可能碰到全是舊值的副本，只能保證最終一致，因而出現「改了看到舊值」。

- (1) 最省成本的強一致設定： $W = 2$ 、 $R = 2$ ($2 + 2 = 4 > 3$ ，滿足)。

說明：相較 $W = 3$ 、 $R = 1$ 或 $W = 1$ 、 $R = 3$ （讀或寫其一須碰全部三份副本、較不耐故障）， $W = R = 2$ 讀寫都只碰兩份、較均衡且較耐單一節點故障，是通用的強一致甜蜜點（呼應案例三的調校）。

三、思考與討論

第 1 題

划算的情況：讀遠多於寫（重複資料極少被更新）、被複製的欄位不常變動（如發文當下的姓名）、儲存成本低廉、且對讀取延遲要求高——此時以空間與偶爾的更新麻煩，換取大量讀取的極速，值得。得不償失的情況：寫入或更新頻繁（每次都要同步大量副本）、被複製的資料經常變動、或對即時一致要求極高（副本更新的延遲會造成明顯錯誤）——此時反正規化的更新負擔與不一致風險，可能超過讀取加速的收益。可引案例四並對比。

第 2 題

此說法過於武斷。應指出：其一，關聯式的 ACID 強一致在金融、庫存、帳務等「絕對不容出錯」的場景仍不可取代，最終一致在此會造成嚴重問題。其二，關聯式的 SQL 與複雜關聯查詢，在需要靈活多角度分析時仍有無可比擬的威力。其三，依 CAP，沒有全能系統，NoSQL 的可擴充與可用正是以放寬一致與查詢能力換來的——那些被放棄的能力，在許多場景仍是剛需。周延的結論是：兩者互補，依工作負載選型（甚至同一系統中混用），而非彼此淘汰。可連結第 3 章 CAP 與本章 6.1 節。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 6 章 NoSQL 資料庫與資料建模

台灣自編大學教科書

@prof

大數據分析與雲端運算

第貳篇 分散式儲存與資料庫

Distributed Storage & Databases

第 7 章

資料複製、分區與一致性

Replication, Partitioning, and Consistency

機器人叫獸（李世淵） 編著

叫獸開講

我很喜歡用一間連鎖飲料店來比喻這一章。想像你開了一家手搖飲，生意好到不行，一家店根本做不完。這時你會面臨兩個決定，而這兩個決定，恰好就是本章的兩個主角。

第一個決定：要不要把「同一份菜單與食譜」複製到每一家分店？當然要——這樣無論客人走進哪一家，都點得到一樣的珍珠奶茶。這就是「複製」（replication）：把同一份資料放到多個地方，讓服務更可靠、離客人更近。但麻煩也來了：總部改了配方，怎麼確保三十家分店都同步更新？萬一有兩家分店同時各改了一版配方，該聽誰的？

第二個決定：面對爆量的訂單，一家中央廚房備料根本來不及。於是你把備料工作「切開」——A 廚房負責茶葉、B 廚房負責珍珠、C 廚房負責配料。這就是「分區」（partitioning）：把一份大工作切成好幾塊，讓不同機器各扛一塊，突破單店的產能極限。但麻煩同樣接踵而來：怎麼切才不會讓某個廚房忙到爆、其他廚房卻閒著？又新開一家分廠時，該怎麼重新分配，才不會把整條供應鏈打亂？

複製與分區，是分散式資料系統的任督二脈。第 3 章我把它們當原理點過，第 5、6 章你也一再看到它們的身影。這一章，我們要正式把它們攤開來，一條一條講清楚——複製有哪幾種做法、分區怎麼切、加減機器時如何再平衡、以及當副本們吵起來時，一致性該如何拿捏、衝突又該如何了結。這是第貳篇原理最密集的一章，讀懂了它，後面的資料庫技術你都能看穿底層。

學習目標

研讀完本章之後，你應該能夠：

1. 說明資料複製的三大目的，以及它所帶來的核心挑戰。
2. 比較主從、多主與無主三種複製架構的運作方式與適用場景。
3. 區分依範圍與依雜湊兩種分區策略，並說明各自的優缺點。
4. 說明分區再平衡的目標，並解釋一致性雜湊如何降低再平衡成本。
5. 描述一致性模型的光譜，區分強一致、最終一致與其間的層次。
6. 說明副本衝突發生的原因，以及常見的衝突偵測與解決策略。

核心概念

核心概念	說明
複製 (Replication)	把同一份資料保存在多個節點，用於容錯、擴充讀取與降低延遲。
主從複製	由一個主副本接受寫入，再同步給多個從副本的架構。
多主／無主複製	允許多個節點（或任意節點）接受寫入的架構，可用性高但須處理衝突。
分區 (Partitioning)	把大資料集切成多塊，分散到不同節點，以突破單機容量與吞吐。
再平衡 (Rebalancing)	節點增減時，重新分配分區以維持負載均衡的過程。
一致性雜湊	一種讓節點增減時只需搬移少量資料的分區技術。

核心概念	說明
衝突解決	當多個副本對同一資料產生分歧時，決定最終值的機制。

表 7-1 本章核心概念一覽

7.1 複製的目的與挑戰

7.1.1 為什麼要複製

把同一份資料存好幾份，聽起來很浪費，但它換來三個極有價值的好處——恰好對應第 3 章「走向分散式的三個理由」。

- **容錯**：任一副本所在的節點故障，其他副本仍保有資料，服務不中斷。這是第 5 章 HDFS 三副本的核心動機。
- **擴充讀取**：多個副本可以分攤讀取請求。讀取密集的系統，只要增加副本，讀取吞吐就能提升。
- **降低延遲**：把副本放到靠近使用者的地理位置，讓遠方使用者也能就近讀取，縮短回應時間。

7.1.2 真正的難題：如何同步

如果資料寫進去後就永遠不變，複製會是全世界最簡單的事——複製一次，永遠一致。複製之所以困難，全因為資料會「變」。一旦某個副本被寫入了新值，這個變動就必須設法傳播到所有其他副本；而在傳播完成之前的那段空窗，各副本之間就是不一致的。

這個「變動如何傳播」的問題，衍生出本章幾乎所有的難題：該由誰來接受寫入（複製架構的選擇）？傳播要即時還是可以延遲（同步或非同步）？傳播途中若有副本落後了怎麼辦（複製延遲）？若兩個副本同時被寫了不同的值又怎麼辦（衝突）？接下來各節，就是逐一拆解這些難題。

7.2 主從、多主與無主複製

誰有權力接受「寫入」，是複製架構最根本的分野。依此可分為三種架構，它們在可用性與一致性之間，站在不同的位置。

7.2.1 主從複製：一個說了算

主從複製（又稱單主複製）是最常見的架構：指定一個副本為「主副本」，所有寫入都必須送到它；主副本處理完後，再把變動同步給其他「從副本」。讀取則可以由任一副本服務。

它的優點是簡單而不易衝突——因為所有寫入都經過同一個主副本，寫入的順序是明確的，不會有「兩個地方同時改」的問題。缺點也很明顯：主副本是寫入的單點，它一故障，就必須選出新的主副本（故障切換），這段期間可能無法寫入。此外，同步給從副本若採非同步，從副本可能短暫落後，讀到舊值。

7.2.2 同步與非同步複製

主副本把變動傳給從副本時，可以「等」或「不等」，這帶來一組重要取捨。

同步複製要求主副本等所有（或部分）從副本都確認收到，才回覆寫入成功。好處是從副本保證不落後，壞處是只要有一個從副本慢或故障，寫入就被拖住。非同步複製則主副本自己寫完就先回覆

成功，再於背景慢慢傳給從副本。好處是寫入快、不受從副本拖累，壞處是主副本一旦在傳播完成前故障，尚未同步的資料就可能遺失。這正是第 6 章法定人數想要精細調節的那個取捨。

7.2.3 多主與無主複製

當「單一主副本」成為瓶頸或不夠可用時，就有了另外兩種架構。

- **多主複製**：允許多個副本都能接受寫入（例如每個資料中心各有一個主副本），再互相同步。好處是寫入不再受單點限制、可用性更高，代價是可能發生「多處同時改同一筆」的寫入衝突，必須設法解決。
- **無主複製**：沒有主從之分，客戶端可以把寫入送給任意（多個）副本，讀取也從多個副本讀。這正是第 6 章 Dynamo 式資料庫的做法，搭配法定人數（ $W + R > N$ ）來調節一致性。可用性最高，但衝突處理也最複雜。

架構	誰能寫	衝突風險	取捨
主從	僅主副本	低	簡單、易一致，但主副本為單點
多主	多個主副本	中	可用性高，須解衝突
無主	任意副本	高	可用性最高，衝突處理最複雜

表 7-2 三種複製架構的比較

叫獸提醒

這三種架構其實是一條光譜：從「集權」到「分權」。主從是集權——好管、好一致，但君王一倒就亂。無主是分權——人人可寫、極耐故障，但眾聲喧嘩就得有一套仲裁機制。多主在中間。你會發現，這又是第 3 章 CAP 的影子：越往可用性靠，就越要付出處理衝突與不一致的代價。沒有白吃的午餐，只有想清楚的取捨。

7.3 資料分區（Sharding）策略

複製解決的是「同一份資料放幾份」，分區解決的是「一份大資料怎麼切開」。當資料量大到單一節點裝不下、或寫入量大到單一節點扛不住，就必須把資料切成多塊（稱為分區、分片或 shard），分散到不同節點。

7.3.1 分區的兩種切法

承接第 3 章 3.6.2 的鋪陳，分區的關鍵是「依什麼切」，主要有兩種策略，各有鮮明的優缺點。

- **依範圍分區**：依鍵值的連續區間切分（如日期 1 月放一區、2 月放另一區）。優點是範圍查詢極有效率（查某段時間只需碰少數分區）；缺點是容易產生熱點——若近期資料被密集存取，最新的那個分區就會忙到爆。
- **依雜湊分區**：先對鍵值計算雜湊，再依雜湊值決定落點。優點是資料分布均勻，能有效避免熱點；缺點是失去了範圍查詢的效率（相鄰的鍵被打散到各處）。

7.3.2 熱點：分區最怕的夢魘

分區最大的敵人是「傾斜」(skew)與「熱點」(hotspot)——大量請求集中湧向少數分區，使它們不堪負荷，其餘分區卻閒置。這會讓分區的好處大打折扣：你明明有十台機器，卻只有一台在拼命、其餘九台在納涼。

熱點的成因往往是分區鍵選得不好。經典的反例是「用時間戳當分區鍵並依範圍分區」——因為新資料總是寫到「最新」的那個分區，造成寫入全部擠在一台機器。解法通常是選擇分布更均勻的鍵、或在鍵前加上隨機前綴把熱點打散。慎選分區鍵，是需要理解資料存取模式的功夫，這也是本章反覆強調的主題。

7.4 分區再平衡與請求路由

7.4.1 再平衡：加減機器時的搬遷

叢集不是一成不變的——你會加機器來擴充、也會有機器故障需要移除。每當節點數改變，分區就得重新分配，好讓負載重新均衡，這個過程稱為「再平衡」(rebalancing)。

再平衡有一個核心目標常被忽略：搬得越少越好。因為搬移資料要耗費大量的網路與磁碟資源，搬遷期間還可能影響正常服務。一個設計拙劣的分區方案，加一台機器可能得把幾乎所有資料重新搬一遍，代價高得嚇人。這正是下一節公式要量化的問題。

7.4.2 為什麼「取餘數」是個壞主意

最直覺的分區方法，是「雜湊值對節點數取餘數」——有 N 台機器，就把鍵的雜湊值除以 N ，餘數是幾就放第幾台。這在節點數固定時運作良好，但它有個致命傷：一旦節點數 N 改變（加一台變成 $N + 1$ ），幾乎所有鍵的「餘數」都會改變，導致絕大部分資料都得搬家。

這就像全班依「學號除以 6 的餘數」分成 6 組，某天改成 7 組——幾乎每個人的組別都變了，全班得重新洗牌。對一個動輒數 TB、數 PB 的系統來說，這種「加一台機器就近乎全體搬遷」的代價是無法承受的。

7.4.3 一致性雜湊：只搬該搬的

解決之道是「一致性雜湊」(consistent hashing)。它的巧思是把雜湊空間想像成一個首尾相接的圓環，節點與資料鍵都被雜湊映射到環上的某個位置；每個鍵歸屬於「順時針方向遇到的第一個節點」。

這個設計的美妙之處在於：當你新增一個節點，它只會「接手」環上鄰近的一小段資料——只有這一小段的鍵需要搬家，其餘絕大多數的鍵歸屬完全不變。同理，移除一個節點時，也只有它負責的那一段需要轉交給鄰居。於是「加減一台機器」的搬遷量，從「幾乎全部」驟降到「約 $1/N$ 」。下一節的公式會把這個節省算給你看。

請求路由

分區之後還有一個問題：客戶端想存取某個鍵，怎麼知道它在哪個節點上？這稱為「請求路由」。常見做法有三：讓客戶端直接算出落點、設一個路由層代為轉發、或讓任一節點收到請求後幫忙轉給正確的節點。無論哪種，背後都需要一份「哪個分區在哪個節點」的地圖——這又回到了第 5 章 NameNode 那種「中繼資料集中管理」的思路。

7.5 一致性模型光譜

第 6 章我們把一致性簡化成「強一致 vs 最終一致」的二分，但真實世界其實是一道連續的光譜。這一節，我們把這道光譜攤開，讓你看清楚中間的層次。

7.5.1 從最強到最弱

一致性模型可依「保證的強弱」排成一列。越強的保證越貼近直覺、越好寫程式，但越難達成、代價越高；越弱的保證越好擴充、越高可用，但也越考驗開發者。

一致性層次	保證	代價／特性
線性一致（最強）	全系統彷彿只有單一份最新資料	最貼近直覺，但最難達成、最貴
因果一致	有因果關係的操作，順序被保證	兼顧合理性與可行性的折衷
讀己之寫	至少讀得到自己剛寫的值	體驗友善的常見保證
最終一致（最弱）	停止寫入後終將收斂	最好擴充、最高可用，最考驗開發者

表 7-3 一致性模型的光譜（由強至弱）

7.5.2 一個貼身的例子：讀己之寫

光譜中間有一個特別實用的層次，叫「讀己之寫」（read-your-writes）。它保證的是：你至少能讀到自己剛剛寫的東西——即使別人可能還看不到。

為什麼這個保證重要？想像你在社群上留了一則評論，頁面重新整理後卻不見了（因為你的讀取被導向一個還沒同步的副本）——你會以為評論失敗，於是再留一次，造成重複。「讀己之寫」正是為了避免這種令人困惑的體驗：系統確保你至少看得到自己的動作，即使全系統的強一致還做不到。這說明一致性不是非黑即白，而是可以「只在需要的地方、給需要的保證」。

7.6 衝突偵測與解決

7.6.1 衝突從何而來

在多主與無主複製中，允許多個節點同時接受寫入，就必然面臨一個難題：如果兩個節點在幾乎同一時間，各自把同一筆資料改成了不同的值，該聽誰的？這就是「寫入衝突」。

衝突的根源，是分散式系統中「沒有一個全域統一的時鐘」——你無法可靠地判斷哪個寫入「真的比較晚」（回想第 3 章八個誤解裡的時序問題）。因此，衝突不能靠「看誰比較晚」草率解決，而需要更謹慎的機制。

7.6.2 常見的解決策略

面對衝突，業界發展出幾種常見策略，各有適用場景與代價。

- **最後寫入者勝（LWW）**：為每個寫入附上時間戳，衝突時保留時間戳較晚者。簡單，但因時鐘不可靠，可能默默丟失資料，須謹慎使用。
- **保留多版本，交由應用解決**：系統偵測到衝突時，保留所有分歧的版本，回傳給應用或使用者的去決定如何合併（例如購物車就把兩份清單聯集起來）。
- **無衝突資料型別（CRDT）**：採用特殊設計的資料結構，讓多個副本的修改無論以什麼順序合併，都能自動收斂到一致，從根本上避免衝突。

其中「購物車聯集」是個很有啟發性的例子：與其煩惱兩個版本聽誰的，不如把兩邊加入的商品全都保留下來——寧可多顯示一件商品讓使用者自己刪，也不要弄丟他想買的東西。這說明衝突解決往往沒有通用解答，而要回到「對這個應用而言，怎樣的結果最能接受」。

7.6.3 承先啟後

本章我們把複製與分區這對任督二脈徹底打通：複製有主從、多主、無主三種架構（7.2），分區有範圍與雜湊兩種切法（7.3），節點增減靠一致性雜湊來低成本再平衡（7.4），一致性是一道從線性到最終的光譜（7.5），而衝突則有 LWW、多版本與 CRDT 等解法（7.6）。

但你可能已經注意到，本章有一個問題被刻意迴避了：在多主或無主的架構下，當真的需要「所有節點對某件事達成一致的決定」時——例如「到底誰是新的主副本」「這筆交易到底成不成立」——該怎麼辦？靠時間戳猜測顯然不夠可靠。要在互不信任、可能故障的節點之間，達成一個確定的、大家都同意的決定，需要一套嚴謹的協定。這就是下一章「分散式交易與共識演算法」的主題——從兩階段提交、Paxos、Raft，一路談到區塊鏈如何在互不信任的世界裡達成共識。

公式與流程：一致性雜湊省下多少搬遷

7.4 節說一致性雜湊能把再平衡的搬遷量從「幾乎全部」降到「約 $1/N$ 」，這一節我們把兩種做法都算出來，讓你對這個差距有精確的感覺。

取餘數法的搬遷量

若採「雜湊值對節點數取餘數」，當節點數從 N 增為 $N + 1$ 時，一個鍵的落點是否改變，取決於它對 N 與對 $N + 1$ 取餘數是否相同。可以證明，僅有約 $1/(N+1)$ 的鍵剛好落點不變，其餘幾乎都要搬移。因此需搬遷的鍵之比例約為：

$$\text{取餘數搬遷比} \approx 1 - 1 / (N+1) = N / (N+1)$$

式 7-1 取餘數法在增加一節點時的搬遷比例

一致性雜湊的搬遷量

若採一致性雜湊，新增的節點只接手環上鄰近的一段，其餘鍵歸屬不變。在理想均勻的情況下，新節點約接手全體的 $1/(N+1)$ ，故需搬遷的鍵之比例約為：

$$\text{一致性雜湊搬遷比} \approx 1 / (N+1)$$

式 7-2 一致性雜湊在增加一節點時的搬遷比例

兩者的差距是天壤之別。下表以不同的原節點數 N ，比較「從 N 台加到 $N + 1$ 台」時，兩種方法各需搬動多少比例的資料。

原節點數 N	取餘數搬遷比	一致性雜湊搬遷比	差距倍數
3	約 75%	約 25%	3 倍
9	約 90%	約 10%	9 倍
19	約 95%	約 5%	19 倍
99	約 99%	約 1%	99 倍

表 7-4 兩種分區法在增加一節點時的搬遷比例對照

叫獸提醒

看最後一列：當你已經有 99 台機器、想加到 100 台，取餘數法要搬動約 99% 的資料——幾乎是把整個叢集重寫一遍，這在 PB 等級根本是災難；而一致性雜湊只需搬約 1%。這就是為什麼幾乎所有現代的分散式資料庫，都不用天真的取餘數，而採一致性雜湊或其變體。一個看似小小的分區策略決定，在大規模下就是「能不能加機器」的生死線。

案例研討

案例一 跨海的社群動態

一個全球性的社群平台，使用者遍布美洲、歐洲與亞洲。若所有資料只放在單一地區的資料中心，遠方使用者每次讀取都要跨海往返，延遲高得難以忍受。

它的解法正是 7.1 節「以複製降低延遲」：在各大洲都部署副本，讓使用者就近讀取本地副本。但這也引入了多主的挑戰——不同地區的使用者可能同時修改相關資料，造成衝突。平台因此採用多主複製並搭配衝突解決策略（如對貼文按讚採可自動合併的計數）。這個案例把複製的「好處」（低延遲）與「代價」（衝突）一次呈現，正是 7.2 節多主架構的真實寫照。

案例二 被時間戳出賣的分區

某物聯網平台為感測資料設計分區，圖方便直接「以時間戳為鍵、依範圍分區」。上線後卻發現：明明有二十個節點，卻幾乎只有一個節點在拼命工作、CPU 滿載，其餘十九個幾乎閒置。

問題正是 7.3.2 節的熱點：因為新資料的時間戳永遠是「最新」，寫入全部擠進了「最新時間」那個分區所在的節點。他們的修正是改變分區鍵——改以「感測器 ID 的雜湊」為主鍵分區，讓來自不同感測器的資料均勻分散到二十個節點，寫入負載瞬間攤平。這個案例是「選錯分區鍵造成熱點」最典型的教訓，也呼應了第 4 章 LSM-tree 適合的感測寫入場景。

案例三 加一台機器引發的雪崩

一家公司的分散式快取叢集採用天真的「取餘數」分區。某天為了擴容，他們加了一台機器，把節點數從 8 增為 9。結果災難發生了——依式 7-1，約 89% 的鍵落點改變，幾乎整個快取瞬間失效。

大量請求因為在快取中找不到資料，全部湧向後端資料庫，資料庫不堪負荷而癱瘓，引發連鎖的「快取雪崩」。事後檢討，根源就是分區策略選錯——若當初採用一致性雜湊，加這台機器只需搬約 11% 的鍵（式 7-2），絕不至於引發雪崩。這個案例血淋淋地說明：分區策略不是學術細節，而是攸關系統存亡的工程決策。

案例四 購物車衝突的智慧解法

回到亞馬遜購物車。在無主複製、允許離線操作的設計下，衝突幾乎無可避免：使用者在手機上加了 A 商品，同一時間在平板上加了 B 商品，這兩個寫入送到了不同副本，該聽誰的？

若用「最後寫入者勝」，就會弄丟其中一件商品——使用者明明加了兩件，結帳卻只剩一件，體驗糟糕。亞馬遜的智慧解法是 7.6.2 節的「保留多版本並合併」：把兩個購物車版本「聯集」起來，A 和 B 都保留。寧可讓使用者看到他加過的所有商品（多的自己刪），也絕不弄丟他想買的。這個案例告訴我們：衝突解決的最佳策略，往往不是技術上最巧妙的，而是最貼合「這個應用能接受什麼」的。

實作活動

活動一 為資料選對複製架構

請為下列情境，各選出最適合的複製架構（主從／多主／無主）並說明理由。

- 步驟 1. 一個對一致性要求極高、寫入集中的銀行核心帳務系統。
- 步驟 2. 一個使用者遍布全球、要求各地都能低延遲寫入的協作筆記 App。
- 步驟 3. 一個追求極致可用、可容忍短暫不一致的購物車服務。
- 步驟 4. 討論：你選無主的那個情境，需要搭配什麼機制來處理衝突？

活動二 親手比較兩種分區法

請以 Python 實驗，親眼看見一致性雜湊的威力。

- 步驟 1. 產生一萬個模擬的鍵，先以「雜湊值對 N 取餘數」的方式分配到 $N = 10$ 個節點。
- 步驟 2. 把節點數增為 11，重新計算每個鍵的落點，統計有多少比例的鍵「換了節點」。
- 步驟 3. 改用一致性雜湊（可用簡化版：把節點與鍵都雜湊到 $0 \sim 1$ 的環上，鍵歸屬順時針第一個節點），同樣從 10 增為 11，統計搬遷比例。
- 步驟 4. 對照式 7-1 與式 7-2，說明你的實驗結果是否符合理論預測。

活動三 設計你的衝突解法

請為下列三種資料，各設計一個合理的衝突解決策略，並說明為何這樣選：（一）一個「按讚數」計數器；（二）一份「使用者的暱稱」；（三）一個「共享的待辦清單」。提示：計數器適合能自動合併的作法、暱稱可能適合最後寫入者勝、清單則可能適合聯集——請說明各自的理由與潛在的副作用。

延伸閱讀

- 《Designing Data-Intensive Applications》第 5、6、9 章：Kleppmann 與 Riccomini 著。第 5 章談複製與衝突，第 6 章談分區與再平衡，第 9 章談一致性與共識，是本章的完整依據。
- 一致性雜湊原始論文：Karger 等人於一九九七年提出，說明一致性雜湊如何降低節點增減時的搬遷成本，是理解 7.4 節與式 7-2 的源頭。
- Dynamo 論文：亞馬遜的無主複製與衝突處理（含購物車聯集）之經典，與本章 7.2、7.6 節高度相關。
- CRDT 相關介紹文章：說明無衝突資料型別如何讓副本自動收斂，是 7.6 節「從根本避免衝突」的進階延伸。

本章小結

1. 複製的三大目的是容錯、擴充讀取與降低延遲；其核心難題不在複製本身，而在資料「會變」——變動如何在副本間傳播，衍生出本章幾乎所有的問題。
2. 複製架構依「誰能寫」分為主從（單點寫入、簡單易一致）、多主（多點寫入、須解衝突）與無主（任意寫入、最高可用、衝突最複雜）；同步複製保證不落後但受慢節點拖累，非同步複製快但可能遺失未傳播的資料。
3. 分區把大資料切塊分散，切法有依範圍（範圍查詢快但易熱點）與依雜湊（分布均勻但失去範圍查詢）；分區最怕熱點，慎選分區鍵是避免熱點的關鍵。

4. 節點增減時須再平衡，目標是搬得越少越好；天真的「取餘數」在加減節點時幾乎要搬移全部資料，一致性雜湊則讓搬遷量降到約 $1/N$ ，是現代分散式系統的標準做法。
5. 一致性是一道由強至弱的光譜：線性一致最貼近直覺卻最昂貴，最終一致最好擴充卻最考驗開發者，中間還有因果一致、讀己之寫等實用層次——可只在需要處給予需要的保證。
6. 多主與無主允許並行寫入，必然面臨衝突，其根源是分散式系統缺乏可靠的全域時鐘；解法有最後寫入者勝（簡單但可能丟資料）、保留多版本交由應用合併（如購物車聯集）、與從根本避免衝突的 CRDT。
7. 取餘數搬遷比約為 $N/(N+1)$ （式 7-1），一致性雜湊搬遷比約為 $1/(N+1)$ （式 7-2），兩者差距達 N 倍；在大規模下，這個差距就是「能否安全地加機器」的生死線。

習題

一、觀念題

1. 請說明複製的三大目的，並解釋為何「資料會變」才是複製真正困難的根源。
2. 請比較主從、多主與無主三種複製架構，各舉一個適合的應用情境。
3. 何謂熱點？請以「用時間戳依範圍分區」為例，說明熱點如何產生，以及可行的解法。
4. 請說明為何「雜湊值對節點數取餘數」在擴容時是個壞主意，而一致性雜湊如何改善。

二、計算題

1. 某系統原有 4 個節點，欲擴容為 5 個。請以式 7-1 與式 7-2 分別估算「取餘數法」與「一致性雜湊」各需搬遷的資料比例，並說明兩者差距。
2. 某快取叢集有 24 個節點，採取餘數分區。若加入 1 個節點成為 25 個，約有多少比例的鍵會改變落點？若改用一致性雜湊，又約為多少？請說明這對「快取雪崩」風險的意義。

三、思考與討論

1. 案例四中，購物車採「保留多版本並聯集」而非「最後寫入者勝」。請討論：在什麼樣的應用中，聯集是好策略？又在什麼應用中，聯集反而會造成錯誤？
2. 「讀己之寫」是一種比強一致弱、卻比最終一致強的保證。請說明：為什麼有時候我們寧可只提供這種中間層次的保證，而不追求全系統的強一致？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

三大目的：容錯（副本所在節點故障時，其他副本仍保有資料）、擴充讀取（多副本分攤讀取請求）、降低延遲（就近部署副本讓遠方使用者快速讀取）。困難根源：若資料永不改變，複製一次即永遠一致；正因資料會變，某副本被寫入新值後，變動須傳播到所有副本，傳播完成前各副本即不一致——由此衍生架構選擇、同步或非同步、複製延遲、衝突等一連串難題。

第 2 題

主從：僅主副本接受寫入、再同步給從副本，簡單且不易衝突，但主副本為單點——適合寫入集中、要求強一致的系統（如帳務）。多主：多個主副本皆可寫入再互相同步，可用性高但須解衝突

——適合跨地區、各地都要低延遲寫入者（如全球協作 App）。無主：任意副本可寫、搭配法定人數，可用性最高但衝突處理最複雜——適合追求極致可用、可容忍短暫不一致者（如購物車）。可引表 7-2。

第 3 題

熱點指大量請求集中湧向少數分區，使其過載而其餘閒置。以「時間戳+範圍分區」為例：新資料的時間戳永遠最新，故寫入全部擠進「最新時間」那個分區所在節點，形成寫入熱點（如案例二）。解法：改選分布更均勻的分區鍵（如感測器 ID 的雜湊），或在鍵前加隨機前綴把熱點打散，使負載均勻分散到各節點。

第 4 題

取餘數（雜湊 mod N ）在節點數 N 改變時，幾乎所有鍵的餘數都變，導致近乎全部資料須搬移（式 7-1，約 $N/(N+1)$ ），代價在大規模下無法承受（如案例三雪崩）。一致性雜湊把節點與鍵映射到一個環上，鍵歸屬順時針第一個節點；新增節點只接手環上鄰近一小段，其餘鍵歸屬不變，故搬遷量僅約 $1/(N+1)$ （式 7-2）。

二、計算題

第 1 題

$N = 4$ ，擴為 5（即 $N + 1 = 5$ ）。

- (1) 取餘數（式 7-1）：搬遷比 $\approx N/(N+1) = 4/5 = 0.8$ （約 80%）。
- (2) 一致性雜湊（式 7-2）：搬遷比 $\approx 1/(N+1) = 1/5 = 0.2$ （約 20%）。
- (3) 差距：80% 對 20%，取餘數法的搬遷量是一致性雜湊的 4 倍（即 N 倍）。

延伸思考：差距倍數恰等於原節點數 N ；節點越多，一致性雜湊的優勢越懸殊（見表 7-4）。

第 2 題

$N = 24$ ，加 1 成為 25（ $N + 1 = 25$ ）。

- (1) 取餘數：搬遷比 $\approx 24/25 = 0.96$ （約 96% 的鍵改變落點）。
- (2) 一致性雜湊：搬遷比 $\approx 1/25 = 0.04$ （約 4%）。

對雪崩風險的意義：取餘數法下，加一台機器竟使 96% 的快取鍵失效，大量請求瞬間穿透快取湧向後端資料庫，極易引發快取雪崩、拖垮後端（如案例三）。一致性雜湊僅 4% 失效，衝擊極小。這說明分區策略的選擇，直接決定了擴容是「平順」還是「災難」。

三、思考與討論

第 1 題

聯集是好策略的情況：當「多保留」的成本低、而「弄丟」的成本高時——如購物車（多一件商品使用者可自行刪除，但弄丟想買的商品體驗極差）、協作清單的新增項目。聯集會造成錯誤的情況：當操作本質是「取代」而非「累加」時——如「使用者的暱稱」「帳戶餘額」「開關狀態」，把兩個版本聯集會產生無意義或錯誤的結果（暱稱不能同時是兩個、餘額不能兩份相加）。核心判準：資料的語意是「集合累加」還是「單一取代」。

第 2 題

因為強一致代價高昂（須跨副本協調、犧牲可用性與延遲，見 CAP），而許多場景並不真的需要「全系統隨時一致」，只需要「對使用者而言體驗合理」。「讀己之寫」正是以較低代價提供關鍵的

體驗保證——讓使用者至少看得到自己的動作，避免「留言後刷新卻不見、以為失敗而重複操作」的困惑。這體現了本章的核心思想：一致性不是非黑即白，而應「只在需要的地方、給需要的保證」，用最小的代價換取足夠好的體驗。可連結 7.5 節與第 6 章最終一致性。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 7 章 資料複製、分區與一致性

台灣自編大學教科書
@prof
大數據分析與雲端運算

第貳篇 分散式儲存與資料庫

Distributed Storage & Databases

第 8 章

分散式交易與共識演算法

Distributed Transactions and Consensus Algorithms

機器人叫獸（李世淵） 編著

叫獸開講

一九八二年，三位電腦科學家發表了一篇論文，講了一個聽起來像童話的故事。拜占庭帝國的幾支軍隊包圍了一座城，各由一位將軍統領。他們必須一致決定「全體進攻」或「全體撤退」——若有些進攻、有些撤退，就會全軍覆沒。將軍們分處不同營地，只能派信使傳話。問題來了：信使可能在路上被攔截，而且將軍之中可能有叛徒，會故意對甲說「進攻」、對乙說「撤退」。

這就是著名的「拜占庭將軍問題」。它問的是一個極為深刻的問題：在一群無法完全信任彼此、通訊還不可靠的參與者之間，有沒有辦法達成一個大家都同意的決定？

你可能覺得這是象牙塔裡的思想遊戲，但它其實是分散式系統最根本的難題。上一章末我留了一個問題沒答：當主副本掛了，剩下的節點該怎麼「一致同意」誰是新的主？當一筆交易橫跨三個資料庫，怎麼確保三邊要嘛全部成功、要嘛全部失敗，不會有人成功有人失敗？這些問題，本質上都是同一個問題——共識（consensus）。

這一章是第貳篇的壓軸，也是整本書理論最硬的一章。我們會從交易的 ACID 講起，看兩階段提交如何嘗試（但未能完美地）解決分散式交易；接著進入共識演算法的殿堂，認識 Paxos 與 Raft；再看這些理論如何化為 ZooKeeper、etcd 這些幕後英雄。最後，我們會回到拜占庭將軍——看看比特幣與區塊鏈，如何在一個連「誰是壞人」都不知道的開放世界裡，達成了四十年前那三位科學家認為極其困難的共識。

學習目標

研讀完本章之後，你應該能夠：

1. 說明交易的 ACID 四項特性，並解釋交易為何是資料正確性的基石。
2. 描述兩階段提交的運作流程，並指出它的阻塞問題與限制。
3. 區分線性一致性與因果一致性，並說明兩者的代價差異。
4. 說明共識問題的定義，以及 Paxos 所解決的核心難題。
5. 描述 Raft 的領導者選舉與日誌複製機制，並說明其多數決門檻。
6. 說明分散式協調服務的用途，以及區塊鏈如何在拜占庭環境下達成共識。

核心概念

核心概念	說明
交易（Transaction）	一組被視為單一不可分割單位的操作，具備 ACID 特性。
兩階段提交（2PC）	以協調者統籌、分準備與提交兩階段的分散式交易協定。
線性一致性	最強的一致性保證，全系統彷彿只有單一份最新資料。
共識（Consensus）	在多個節點間就某個值達成一致決定的問題。
Paxos/Raft	兩種經典的共識演算法，後者以易理解著稱。
協調服務	如 ZooKeeper、etcd，為叢集提供領導者選舉與設定管理的基礎設施。
拜占庭容錯	在部分節點可能任意作惡的環境下，仍能達成共識的能力。

表 8-1 本章核心概念一覽

8.1 交易與 ACID 特性

8.1.1 交易：全有或全無的承諾

交易（transaction）是資料庫最重要的抽象之一。它的核心思想只有一句話：把一組操作綁成一個不可分割的整體，要嘛全部成功，要嘛全部當作沒發生過。

最經典的例子是轉帳：從甲帳戶扣一千元、往乙帳戶加一千元。這兩個動作必須綁在一起——若扣款成功但入帳失敗，那一千元就人間蒸發了。交易提供的保證是：這兩步要嘛都成，要嘛都不成，絕不會停在中間那個災難性的狀態。

8.1.2 ACID 四項特性

交易的保證被歸納為四個字母，也就是我們在第 6 章對比 BASE 時提過的 ACID。這裡把它們逐一講清楚。

特性	中文	保證的內容
Atomicity	原子性	交易中的操作是不可分割的整體，全部成功或全部撤銷，不會有中間狀態。
Consistency	一致性	交易前後，資料都必須滿足預先定義的規則（如帳戶餘額不得為負）。
Isolation	隔離性	多個交易並行執行時，彼此不互相干擾，結果如同依序執行。
Durability	持久性	交易一旦提交成功，即使系統當機，結果也不會遺失。

表 8-2 交易的 ACID 四項特性

叫獸提醒

注意 ACID 裡的 C（一致性）與我們談分散式時說的「一致性」不是同一回事，這是初學者最常混淆的地方。ACID 的 C 指的是「資料滿足應用層的規則」（如餘額不得為負），是應用定義的正確性；而分散式的一致性（第 3、7 章）指的是「多個副本之間看到的值是否相同」。同一個詞、兩個世界，讀論文時務必看清楚語境。

8.2 分散式交易與兩階段提交

8.2.1 當交易跨越了機器

在單機資料庫上實現 ACID 已經不容易，但至少所有資料都在同一台機器、由同一個程式掌控。一旦交易橫跨多台機器——例如轉帳的甲帳戶在資料庫 A、乙帳戶在資料庫 B——難度就陡然上升。

問題在於：A 說「我這邊扣款成功了」，B 卻可能因為當機或網路中斷而失敗。此時 A 已經扣了款，卻無法得知 B 到底成不成功。要讓分散在多台機器上的操作「同進同退」，就需要一套協定來統籌，這就是分散式交易協定，其中最經典的是「兩階段提交」。

8.2.2 兩階段提交的流程

兩階段提交（Two-Phase Commit, 2PC）引入一個「協調者」（coordinator）來指揮所有參與者，流程如其名分為兩個階段。

- 第 1 步．** 準備階段：協調者詢問所有參與者「你準備好提交了嗎」。每個參與者做完該做的準備工作（並確保自己之後一定有能力提交），回覆「可以」或「不行」。
- 第 2 步．** 提交階段：若所有參與者都回覆「可以」，協調者就下令「全體提交」；只要有任何一個回覆「不行」，協調者就下令「全體撤銷」。
- 第 3 步．** 參與者收到指令後執行，並回報完成。至此交易結束。

這個設計的關鍵在於：參與者在準備階段回覆「可以」之後，就等於做出了一個承諾——它必須放棄自主權，無論之後如何都得聽協調者的指令。這正是能讓分散各處的操作「同進同退」的機制。

8.2.3 致命傷：協調者一倒就卡住

兩階段提交看似完美，卻有一個嚴重的弱點：如果協調者在「參與者都回覆可以」之後、「下達指令」之前當機了，會發生什麼事？

所有參與者都已經承諾要聽指令、放棄了自主權，卻永遠等不到那道指令。它們不能自作主張提交（萬一協調者原本要下令撤銷呢），也不能自作主張撤銷（萬一協調者已對別人下令提交呢）。於是它們只能傻傻地卡在那裡，同時還握著資料的鎖不放——這稱為「阻塞」（blocking）。

這個致命傷讓兩階段提交在大規模系統中並不受歡迎：它引入了一個單點（協調者），違背了分散式系統追求高可用的初衷。要真正優雅地解決這個問題，就必須跳脫「靠一個指揮官」的思維，改用「大家投票」的方式——這就把我們帶進了共識演算法的世界。

8.3 線性一致性與因果一致性

8.3.1 線性一致性：假裝只有一份資料

第 7 章我們把一致性排成了一道光譜，其中最強的是「線性一致性」（linearizability）。這一節把它講透，因為它是共識演算法所要達成的目標。

線性一致性的白話定義是：讓整個分散式系統，表現得彷彿只有單一份資料、單一台機器。也就是說，一旦某個寫入完成，之後所有的讀取——不論打到哪個節點——都必須讀到這個新值，絕不能讀到舊值。

這個保證極為貼近直覺（就像操作單機一樣），因此程式最好寫。但它的代價也極高：為了確保「任何節點都讀到最新值」，節點之間必須頻繁協調，這既拖慢速度，又在網路分割時被迫犧牲可用性（第 3 章 CAP 的 CP 側）。

8.3.2 因果一致性：只保證該有的順序

線性一致性太貴，但完全放任的最終一致又太弱——有沒有中間的選擇？「因果一致性」（causal consistency）就是一個聰明的折衷。

它的想法是：不強求所有操作都有全域統一的順序，只保證「有因果關係」的操作順序不會顛倒。舉例來說，若甲先發了一則貼文、乙接著回覆了那則貼文，那麼任何人不該「先看到回覆、卻還看不到原貼文」——因為回覆的存在，因果上依賴於原貼文。至於兩個彼此無關的貼文，誰先誰後被看到，就無所謂了。

因為只需維護因果順序、不必全域排序，因果一致的代價遠低於線性一致，卻能避免絕大多數令人困惑的異常。它是許多現代系統選擇的實用甜蜜點。

8.4 共識問題與 Paxos

8.4.1 共識問題的定義

共識問題聽起來抽象，其實可以一句話說完：讓一群節點，就「某個值」達成一致的決定，且這個決定一旦做成就不會反悔。

一個正確的共識演算法必須滿足幾項要求：所有節點最終決定的值必須相同（一致性）、決定的值必須是某個節點真的提議過的（有效性）、而且系統最終必須真的做出決定，不能永遠僵持（終止性）。

為什麼這個抽象問題如此重要？因為分散式系統中大量的實際問題，本質上都是它的變形：選出誰當主副本、決定一筆交易是否提交、決定叢集的成員名單、決定一個分散式鎖歸誰所有——通通都是「大家要一致同意某個值」。解決了共識，這些問題就一併解決了。

8.4.2 Paxos：優雅但難懂

一九九〇年代，Leslie Lamport 提出了 Paxos，這是第一個被嚴格證明正確的共識演算法。它的核心思想是「多數決」：任何決定都必須獲得超過半數節點的同意才能成立。

為什麼「超過半數」是關鍵？因為任兩個「超過半數」的集合必然重疊（這與第 6 章法定人數 $W + R > N$ 的道理如出一轍）。這個重疊保證了：不可能出現兩群節點各自達成了不同的決定——因為重疊的那個節點不會同時同意兩個互相矛盾的值。

Paxos 在理論上優雅完備，但它有一個廣為人知的問題：極難理解、也極難正確實作。原始論文以拜占庭議會的寓言寫成，讓無數工程師讀得一頭霧水；即使讀懂了理論，要把它變成可用的系統仍困難重重。這個「難懂」的痛點，直接催生了下一節的主角。

8.5 Raft 演算法

8.5.1 為「可理解」而生

二〇一四年，兩位史丹佛的研究者提出了 Raft。它的設計目標很特別——不是為了比 Paxos 更快或更強，而是為了「更容易被理解」。他們認為，一個工程師無法理解的演算法，再優雅也難以被正確實作。

Raft 達成可理解性的方法，是把共識問題「拆解」成三個相對獨立、各自單純的子問題：領導者選舉、日誌複製、以及安全性保證。工程師可以逐一攻克，而不必一次面對整團複雜。這個策略非常成功——Raft 如今已是業界最廣泛採用的共識演算法。

8.5.2 領導者選舉

Raft 的第一個巧思，是明確地「選出一個領導者」。所有的寫入都交由領導者處理，再由它複製給其他節點。這其實就是第 7 章的主從複製——差別在於，Raft 用嚴謹的共識機制，來保證「誰是領導者」這件事本身不會出錯。

選舉的機制大致是：每個節點都有一個隨機的倒數計時器；若它在計時結束前都沒收到領導者的心跳，就認為領導者掛了，於是把自己升為候選人並向其他節點拉票。獲得「超過半數」選票的候選人就成為新領導者。隨機化的計時器是個小而美的設計——它讓節點不會同時發起選舉而互相卡住，大幅提高了選舉一次成功的機率。

8.5.3 日誌複製與多數決

選出領導者後，所有的寫入都以「日誌條目」的形式，由領導者依序複製給追隨者。當某條日誌被「超過半數」的節點確認寫入後，領導者就將它標記為已提交，並回覆客戶端成功。

這個「超過半數」的門檻，正是 Raft 容錯能力的來源。它意味著：只要故障的節點數少於半數，系統就還能運作。下一節的公式會把這個門檻精確算出來——你會看到，這也解釋了為什麼叢集節點數幾乎都設成奇數。

8.6 分散式協調服務 (ZooKeeper/etcd)

8.6.1 把共識變成一項服務

共識演算法既然這麼難實作，那麼每個分散式系統都自己寫一套，顯然既浪費又危險。業界因此發展出一個聰明的做法：把共識封裝成一項「基礎服務」，讓其他系統直接使用。這就是分散式協調服務，代表作是 ZooKeeper 與 etcd。

它們的角色，就像是叢集裡「值得信賴的公證人」。其他系統遇到需要達成共識的事情——選主、鎖定、註冊——就交給它來裁決，自己不必碰共識演算法的複雜細節。

典型用途

- **領導者選舉**：幫叢集選出唯一的主節點，並在它故障時協助切換。第 5 章 HDFS 的 NameNode 高可用，正是靠它。
- **設定管理**：集中保管叢集的組態設定，確保所有節點看到同一份設定。
- **服務發現**：記錄有哪些節點正在運作、位址為何，讓服務彼此找得到。
- **分散式鎖**：確保同一時間只有一個節點能執行某項關鍵操作。

你會發現，這些協調服務本身就是這一章理論的產物——ZooKeeper 使用類 Paxos 的協定，etcd 則直接以 Raft 實作。它們是分散式世界裡沉默的幕後英雄：你在第 5 章用的 HDFS、第 29 章要學的 Kubernetes，底層都仰賴它們來維持秩序。

8.7 區塊鏈與加密貨幣（分散式帳本與共識）

前面各節的共識演算法，都建立在一個前提上：節點可能當機、可能失聯，但不會「說謊」。然而回到本章開頭的拜占庭將軍問題——如果參與者之中真的有叛徒呢？這正是區塊鏈要解決的問題。

8.7.1 一個沒有信任的世界

Paxos 與 Raft 適用於「封閉」環境：叢集的節點都由同一個組織掌控，彼此雖可能故障，但不會蓄意作惡。這種故障模型稱為「當機容錯」（Crash Fault Tolerance）。

但比特幣要面對的是完全不同的世界：任何人都能加入網路、節點分散在全球、彼此互不認識、而且其中必然有人想要作弊（例如把同一筆錢花兩次，即「雙重支付」）。在這種開放且互不信任的環境裡達成共識，需要的是「拜占庭容錯」（Byzantine Fault Tolerance）——連叛徒都要能容忍。

8.7.2 區塊鏈的三個支柱

區塊鏈以三項技術的組合，回應了這個挑戰。

- **鏈式雜湊結構**：交易被打包成「區塊」，每個區塊都包含前一個區塊的雜湊值，環環相扣成鏈。只要竄改任何一個舊區塊，其後所有區塊的雜湊都會對不上，竄改立刻曝光——這使帳本具備了「不可竄改」的性質。
- **分散式帳本**：帳本不由任何單一機構保管，而是複製給網路上所有參與者。沒有中心可以被攻陷，也沒有人能片面修改。
- **共識機制**：決定「下一個區塊由誰來記、以哪個版本為準」的規則。比特幣採用工作量證明（PoW），以大量運算競賽取得記帳權；後來的系統多改用權益證明（PoS），以持有的資產作為擔保。

8.7.3 中本聰共識的巧思

比特幣的共識機制常被稱為「中本聰共識」，它的巧思在於：不試圖讓所有節點「投票表決」（在開放網路中，作惡者可以偽造無數身分來灌票，此即「女巫攻擊」），而是改用「誰付出的運算成本最高，就聽誰的」。

具體而言，礦工們競相解一道極耗運算的數學謎題，最先解出者取得記帳權；而當出現分歧的鏈時，規則是「以累積工作量最長的那條鏈為準」。想要竄改歷史，攻擊者就必須重做該區塊之後所有的工作量，並超越全網其餘算力的總和——這在經濟上幾乎不可行。於是，安全性不再依賴「信任某個人」，而是依賴「作惡的成本高於收益」。

代價也很明顯：工作量證明極度耗能、交易吞吐低（每秒僅個位數到數十筆），與 Raft 那種每秒可處理數萬筆的封閉式共識不可同日而語。這再次印證本書反覆的主題——沒有全能的設計，只有針對特定環境的取捨。

面向	Paxos/Raft（封閉）	區塊鏈（開放）
參與者	同一組織掌控、彼此信任	任何人可加入、互不信任
故障模型	當機容錯（節點不作惡）	拜占庭容錯（容忍作惡）
共識方式	多數決投票	工作量／權益證明
效能	高吞吐、低延遲	低吞吐、高延遲、耗能
典型用途	叢集協調、資料庫複製	加密貨幣、去中心化帳本

表 8-3 封閉式共識與區塊鏈共識的比較

叫獸提醒

很多同學一聽到區塊鏈就想到炒幣，反而錯過了它在計算機科學上真正的貢獻。中本聰真正了不起的地方，是他把一個純技術問題（拜占庭共識）轉化成了經濟問題——不去證明「沒有人會作惡」，而是設計出「作惡不划算」的規則。這是四十年來共識研究的一次典範轉移。你可以不投資加密貨幣，但這個「用誘因設計取代信任假設」的思路，值得每一位工程師學起來。

8.7.4 承先啟後

到這裡，第貳篇「分散式儲存與資料庫」四章圓滿收束。我們從第 5 章的分散式檔案系統出發，看資料如何被切塊、複製、分散存放；第 6 章認識了 NoSQL 家族與 BASE 的取捨哲學；第 7 章打通

了複製與分區這對任督二脈；而本章，我們補上了最後也最硬的一塊——當節點必須「達成一致的決定」時，該怎麼辦。

回頭看，這四章其實在回答同一個大問題：海量資料如何在一群會故障、會失聯、甚至可能作惡的機器上，被可靠地存放與存取？答案是複製、分區、法定人數與共識。資料的「安身之所」至此已經解決。接下來第參篇，我們要換一個角度——資料已經好好地躺在那裡了，那麼，我們該如何在這麼多台機器上，把它們高效地「算」出結果來？這就是 MapReduce 與 Spark 的舞台。

公式與流程：容錯門檻與節點數的選擇

共識演算法能容忍幾個節點故障？這不是憑感覺，而是有精確的數學門檻。理解這兩條公式，你就能為叢集選對節點數。

當機容錯：多數決門檻

Paxos 與 Raft 屬於當機容錯——節點會停擺、失聯，但不會說謊。它們要求任何決定須獲得「超過半數」的同意，故所需的法定人數為：

$$\text{法定人數} = \lfloor N / 2 \rfloor + 1$$

式 8-1 當機容錯的多數決門檻

由此可推得，能容忍的故障節點數 f 為：

$$f = \lfloor (N - 1) / 2 \rfloor \quad (\text{即 } N \geq 2f + 1)$$

式 8-2 當機容錯可容忍的故障節點數

拜占庭容錯：更高的門檻

若節點可能任意作惡（傳送矛盾或偽造的訊息），門檻就必須提高。經典結論是，要容忍 f 個拜占庭節點，總節點數必須滿足：

$$N \geq 3f + 1$$

式 8-3 拜占庭容錯所需的節點數

直觀的理由是：作惡節點不只是「缺席」，還會主動散布假訊息，因此誠實的多數必須「大到足以壓過作惡者的干擾」。下表比較兩種模型在不同節點數下的容錯能力。

節點數 N	多數決門檻	可容忍當機 f	可容忍拜占庭 f
3	2	1	0 (需 $N \geq 4$)
4	3	1	1
5	3	2	1
7	4	3	2

表 8-4 不同節點數下的容錯能力

叫獸提醒

看表 8-4 的第 3 與第 5 列，你就明白為什麼叢集節點數幾乎都設奇數。 $N = 3$ 與 $N = 4$ 的多數決門檻分別是 2 與 3，可容忍的當機數都是 1——多加那一台機器，容錯能力完全沒提升，卻多花了一台的錢，還讓每次決策要多徵得一個節點同意。所以 ZooKeeper、etcd 的叢集你

幾乎只會看到 3、5、7 台，這不是迷信，是式 8-1 算出來的結論。

案例研討

案例一 卡在半路的跨行轉帳

設想一筆跨行轉帳：A 銀行扣款、B 銀行入帳，由一個協調者以兩階段提交統籌。準備階段兩邊都回覆「可以」，但就在協調者要下達提交指令的瞬間，協調者的伺服器當機了。

此時 A 與 B 都已承諾聽命、鎖住了相關帳戶，卻永遠等不到指令。它們既不敢自行提交（怕對方其實要撤銷），也不敢自行撤銷（怕對方已經提交），只能持續阻塞、持續持有鎖，連帶讓其他要動這些帳戶的交易全部卡住。這正是 8.2.3 節「協調者單點」的致命傷——它讓一個本該提升可靠性的協定，反而製造了新的單點故障。這也解釋了為何大規模系統寧可改用共識演算法或補償式交易。

案例二 三台還是四台？一次採購的爭論

某團隊要為叢集部署 etcd 協調服務。工程師建議 3 台，主管認為「多一台更保險」，主張買 4 台。誰對？

依式 8-1 與式 8-2：N = 3 時門檻為 2、可容忍 1 台故障；N = 4 時門檻為 3、同樣只能容忍 1 台故障。也就是說，第四台機器不但沒有提升容錯能力，反而讓每次決策要多一個節點同意（更慢、也更容易因網路波動而受影響），還多付了一台的成本。若真要提升容錯，應該一次跳到 5 台（可容忍 2 台故障）。這個案例把式 8-1 從抽象公式變成了實實在在的採購決策——數學在這裡直接省下了錢。

案例三 NameNode 的接班儀式

回到第 5 章。HDFS 的 NameNode 掌管所有檔案的地圖，是不折不扣的關鍵角色。為了高可用它部署了主備兩個 NameNode——但這帶來一個危險：萬一備援節點誤判主節點掛了，就會出現「兩個 NameNode 都認為自己是主」的局面，這稱為「腦裂」（split-brain），可能造成中繼資料損毀。

解法正是 8.6 節的協調服務。HDFS 讓 ZooKeeper 來擔任公證人：想成為主節點者，必須先向 ZooKeeper 取得一個獨占的「鎖」，而 ZooKeeper 以共識演算法保證這個鎖同一時間只會發給一個節點。於是「誰是主」這件事有了唯一且可信的答案，腦裂被根本地杜絕。這個案例把第 5、7、8 章串成一線：主從複製需要選主，選主需要共識，共識則被封裝成一項服務。

案例四 雙重支付與最長鏈

在比特幣網路中，一個作惡者可能嘗試「雙重支付」：先用一筆錢向商家購物，等商家發貨後，再偽造一條不含這筆付款的分支鏈，試圖讓原本那筆交易「從未發生」。

中本聰共識如何阻止？規則是「以累積工作量最長的鏈為準」。作惡者要讓自己的偽造鏈勝出，就必須從那筆交易所在的區塊開始重算，並且追上、超越全網其餘算力持續累積的進度。只要誠實節點掌握了多數算力，作惡者就永遠追不上——而即使他真的擁有超過半數算力，投入的成本也遠超過詐騙所得。這個案例展現了 8.7.3 節的核心巧思：安全性不是靠「信任」，而是靠「讓作惡在經濟上不划算」。

實作活動

活動一 推演兩階段提交的故障情境

請以紙筆推演 2PC 在不同時間點故障的後果。

- 步驟 1. 畫出協調者與三個參與者的兩階段提交流程圖。
- 步驟 2. 推演：若某個參與者在準備階段回覆「不行」，後續會如何？
- 步驟 3. 推演：若協調者在收齊「可以」之後、下達指令之前當機，參與者會陷入什麼狀態？
- 步驟 4. 討論：有哪些做法可以緩解這個阻塞問題？（提示：可考慮讓參與者彼此詢問，或改用共識演算法選出新協調者）

活動二 為你的叢集算容錯

請運用式 8-1 至式 8-3 做一次容量規劃。

- 步驟 1. 計算 $N = 3, 5, 7, 9$ 時的多數決門檻與可容忍當機節點數。
- 步驟 2. 說明為何 $N = 4$ 相較 $N = 3$ 並未提升容錯能力。
- 步驟 3. 若你的系統需要容忍 2 個節點同時當機，最少需要幾個節點？
- 步驟 4. 若改為需要容忍 2 個「作惡」節點，又最少需要幾個節點？（用式 8-3）

活動三 觀察一條鏈式雜湊

請以 Python 實作一個極簡的區塊鏈結構，體會「不可竄改」的原理。建立一個含五個區塊的串列，每個區塊包含「資料」與「前一區塊的雜湊值」（可用 hashlib 的 SHA-256）。完成後，試著竄改第二個區塊的資料，重新計算並檢查後續每個區塊的雜湊是否仍然吻合。請說明：你觀察到什麼現象？這如何解釋區塊鏈的防竄改特性？

延伸閱讀

- 《Designing Data-Intensive Applications》第 7、9 章：Kleppmann 與 Riccomini 著。第 7 章談交易與隔離性，第 9 章談一致性與共識（含 2PC、線性一致與共識演算法），是本章最完整的依據。
- 〈拜占庭將軍問題〉：Lamport、Shostak 與 Pease 於一九八二年發表的經典論文，是理解 8.7 節拜占庭容錯與式 8-3 的源頭。
- 〈In Search of an Understandable Consensus Algorithm〉：Ongaro 與 Ousterhout 提出 Raft 的論文，行文清晰、圖解豐富，是本章 8.5 節的最佳延伸，也常被推薦為共識演算法的入門首選。
- 比特幣白皮書：中本聰於二〇〇八年發表，僅九頁，說明工作量證明與最長鏈規則，是 8.7 節的第一手資料。

本章小結

1. 交易把一組操作綁成不可分割的整體，並以 ACID（原子性、一致性、隔離性、持久性）保證正確；須注意 ACID 的「一致性」指應用規則的滿足，與分散式的副本一致性是不同概念。
2. 兩階段提交以協調者統籌，分準備與提交兩階段，讓跨機器的操作能同進同退；但協調者若在下令前當機，參與者將持鎖阻塞而無法自行決定，這個單點缺陷使它不適用於大規模系統。
3. 線性一致性讓系統彷彿只有單一份資料，最貼近直覺但代價最高；因果一致性只保證有因果關係的操作順序不顛倒，以遠低的代價避免了多數令人困惑的異常，是實用的折衷。

4. 共識問題是「讓一群節點就某個值達成不可反悔的一致決定」，選主、提交、成員管理、分散式鎖皆是其變形；Paxos 以多數決保證任兩個決定集合必然重疊，理論優雅但極難理解與實作。
5. Raft 以「可理解」為設計目標，把共識拆解為領導者選舉、日誌複製與安全性三個子問題；以隨機計時器避免同時選舉，並以超過半數的確認來提交日誌。
6. 分散式協調服務（ZooKeeper、etcd）把共識封裝成基礎服務，提供領導者選舉、設定管理、服務發現與分散式鎖，是 HDFS、Kubernetes 等系統背後沉默的秩序維持者。
7. 區塊鏈面對的是開放且互不信任的環境，需拜占庭容錯；它以鏈式雜湊（防竄改）、分散式帳本（無中心）與共識機制（PoW/PoS）三支柱，把「證明無人作惡」轉化為「使作惡不划算」，代價是低吞吐與高耗能。
8. 當機容錯的門檻為 $\lfloor N/2 \rfloor + 1$ （式 8-1）、可容忍 $f = \lfloor (N-1)/2 \rfloor$ 個故障（式 8-2）；拜占庭容錯則需 $N \geq 3f + 1$ （式 8-3）。因偶數節點不會提升容錯能力，叢集節點數通常設為奇數。

習題

一、觀念題

1. 請說明交易 ACID 中的「一致性」與分散式系統中「一致性」的差異，並各舉一例。
2. 請描述兩階段提交的流程，並說明它的阻塞問題是如何產生的。
3. 請比較線性一致性與因果一致性，並說明為何許多系統選擇後者。
4. Raft 為何要用「隨機化的計時器」來觸發選舉？若改為固定計時器會有什麼問題？

二、計算題

1. 請以式 8-1 與式 8-2 計算： $N = 5$ 、 7 、 9 時的多數決門檻與可容忍的當機節點數各為多少？並說明為何 $N = 6$ 相較 $N = 5$ 並未提升容錯能力。
2. 某系統須容忍 3 個節點發生「當機」故障，請問最少需要幾個節點？若這 3 個節點可能是「作惡」的拜占庭節點，最少又需要幾個節點？（分別用式 8-2 與式 8-3）

三、思考與討論

1. 案例四說明比特幣以「作惡不划算」取代「信任」。請討論：這種以經濟誘因設計取代技術信任假設的思路，還可能應用在哪些場景？它的前提與風險是什麼？
2. 有人主張「既然區塊鏈能在互不信任的環境達成共識，企業內部的資料庫也該全面改用區塊鏈」。請根據本章內容評論此說法。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

ACID 的一致性指「交易前後資料仍滿足應用層定義的規則」，是應用定義的正確性，例如轉帳後兩帳戶總額不變、餘額不得為負。分散式的一致性則指「多個副本之間看到的值是否相同」，例如寫入後其他節點能否讀到最新值（線性一致 vs 最終一致）。兩者同名而異義，前者關乎單一邏輯資料的正確性，後者關乎多份副本的同步程度。

第 2 題

流程：（一）準備階段，協調者詢問所有參與者是否準備好提交，參與者完成準備並回覆可以或不行；（二）提交階段，全部回覆可以則下令全體提交，任一回覆不行則下令全體撤銷。阻塞問題的產生：參與者一旦回覆「可以」即放棄自主權、承諾聽命並持有鎖；若協調者在收齊回覆後、下達指令前當機，參與者既不敢自行提交（怕對方要撤銷）也不敢自行撤銷（怕對方已提交），只能持鎖無限等待，連帶阻塞其他交易（如案例一）。

第 3 題

線性一致性讓系統彷彿只有單一份資料，寫入完成後任何節點的讀取都須讀到新值，最貼近直覺、程式最好寫，但須頻繁跨節點協調，延遲高且在分割時犧牲可用性。因果一致性僅保證有因果關係的操作順序不顛倒（如先有貼文才看得到其回覆），無因果關係者順序不拘。多數系統選後者的原因：代價遠低（不需全域排序），卻已能避免絕大多數令使用者困惑的異常，是效能與體驗的實用甜蜜點。

第 4 題

隨機化計時器可避免多個節點在領導者失聯後「同時」超時並同時發起選舉。若用固定計時器，所有節點會幾乎同時成為候選人並互相拉票，票數容易分散而無人取得超過半數，選舉失敗後又再次同時超時，可能反覆僵持（活鎖），遲遲選不出領導者。隨機化讓某個節點先超時、搶得先機，大幅提高一次選舉即成功的機率。

二、計算題

第 1 題

套用式 8-1（門檻 = $\lfloor N/2 \rfloor + 1$ ）與式 8-2（ $f = \lfloor (N-1)/2 \rfloor$ ）。

- (1) $N = 5$ ：門檻 = $\lfloor 5/2 \rfloor + 1 = 2 + 1 = 3$ ；可容忍 $f = \lfloor 4/2 \rfloor = 2$ 。
- (2) $N = 7$ ：門檻 = $\lfloor 7/2 \rfloor + 1 = 3 + 1 = 4$ ；可容忍 $f = \lfloor 6/2 \rfloor = 3$ 。
- (3) $N = 9$ ：門檻 = $\lfloor 9/2 \rfloor + 1 = 4 + 1 = 5$ ；可容忍 $f = \lfloor 8/2 \rfloor = 4$ 。

$N = 6$ 為何未提升：門檻 = $\lfloor 6/2 \rfloor + 1 = 4$ ，可容忍 $f = \lfloor 5/2 \rfloor = 2$ ，與 $N = 5$ 的可容忍數同為 2；但門檻由 3 升為 4，每次決策須多徵得一個節點同意，反而更慢、更易受網路波動影響，且多付一台成本。故叢集節點數通常取奇數（呼應案例二）。

第 2 題

分別套用兩種容錯模型的公式， $f = 3$ 。

- (1) 當機容錯（式 8-2， $N \geq 2f + 1$ ）： $N \geq 2 \times 3 + 1 = 7$ ，故最少需 7 個節點。
- (2) 拜占庭容錯（式 8-3， $N \geq 3f + 1$ ）： $N \geq 3 \times 3 + 1 = 10$ ，故最少需 10 個節點。

延伸思考：同樣容忍 3 個故障，若故障節點可能作惡，所需節點數由 7 增為 10（約 1.43 倍）。這說明「防範說謊」遠比「防範停擺」昂貴——因為作惡者不只缺席，還會主動散布假訊息，誠實的多數必須大到足以壓過干擾。

三、思考與討論

第 1 題

可能的應用場景：需在互不信任的多方之間協作者，如跨組織的供應鏈溯源、碳權或版權登錄、去中心化的資料市集；亦可延伸到非區塊鏈情境，如以押金或保證金機制抑制惡意行為（垃圾訊息防治、線上競標）。前提：作惡的成本必須可被量化並確實高於預期收益，且誠實參與者須佔據多數資

源。風險：若資源高度集中（如算力或持幣集中於少數人），誘因設計即失效；此外經濟前提會隨市場價格劇烈變動，安全性因而不再是純技術保證。可連結 8.7.3 與案例四。

第 2 題

此說法應予保留。理由：其一，故障模型不同——企業內部叢集由同一組織掌控，屬當機容錯即足，無須為拜占庭容錯付出昂貴代價（式 8-3 需 $N \geq 3f+1$ ，遠高於式 8-2）。其二，效能落差巨大——區塊鏈吞吐低、延遲高且耗能（表 8-3），而企業資料庫多需每秒數萬筆交易，Raft 類共識或傳統資料庫遠更合適。其三，區塊鏈的去中心化特性在企業內部反成負擔（治理、修正錯誤資料困難）。周延的結論：技術選型應依信任模型與效能需求而定；在需要跨組織、互不信任的場景，區塊鏈才展現價值，內部系統則沿用封閉式共識更為務實。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 8 章 分散式交易與共識演算法

台灣自編大學教科書
@prof
大數據分析與雲端運算

第參篇 批次運算框架

Batch Processing Frameworks

第 9 章

MapReduce 程式設計模型

The MapReduce Programming Model

機器人叫獸（李世淵） 編著

叫獸開講

我常問學生一個問題：假設要你數一數學校圖書館裡，所有藏書中「機器人」這三個字總共出現幾次，你會怎麼做？

多數人的第一反應是：一本一本翻，看到一次就在紙上畫一筆。這當然可行，但如果圖書館有五十萬本書，你翻到畢業也翻不完。聰明一點的答案是：找一百個同學來，每人分配五千本，各自數各自的，最後把一百個人的數字加起來。

恭喜你，你剛剛發明了 MapReduce。「每人各自數自己那份」就是 Map，「把大家的數字加總起來」就是 Reduce。這個想法簡單到近乎廢話——但 Google 在二〇〇四年那篇論文的偉大之處，不在於想出這個點子，而在於他們把這個點子背後所有骯髒的雜事，全部包了起來。

你想想，真要找一百個同學來數書，麻煩事可多了：書要怎麼分配？中途有同學生病走了怎麼辦（要不要把他那份重新分給別人）？有人數得特別慢，大家要不要一直等他？最後怎麼把一百張紙條收齊加總？MapReduce 框架的貢獻，就是把這些「分配、容錯、調度、彙整」的雜事全部自動處理掉，只留下兩個最單純的問題交給你：你要對每一份資料做什麼（Map）？你要怎麼把結果合起來（Reduce）？

第貳篇我們解決了「海量資料存在哪裡」，從這一章開始的第參篇，要解決「怎麼把它算出來」。而 MapReduce 就是這一切的起點——它是那隻黃色玩具象的靈魂，也是後來 Spark 的思想源頭。就算今天你在業界不太會直接寫 MapReduce，讀懂它仍然至關重要，因為它定義了整個分散式運算的思考方式。

學習目標

研讀完本章之後，你應該能夠：

1. 說明 MapReduce 的核心思想，以及它為程式設計師隱藏了哪些複雜性。
2. 描述 Map、Shuffle 與 Reduce 三個階段的運作與資料流動。
3. 說明 Combiner 與 Partitioner 的作用，並判斷何時適合使用 Combiner。
4. 運用 MapReduce 的思維，把一個資料處理問題拆解為鍵值對的轉換。
5. 以 MapReduce 實作詞頻統計、Join 與矩陣乘法等經典演算法。
6. 指出 MapReduce 的限制，並說明它如何促成 Spark 等後續框架的出現。

核心概念

核心概念	說明
MapReduce	把運算拆為 Map 與 Reduce 兩階段，由框架自動處理分散、容錯與調度的程式模型。
鍵值對	MapReduce 唯一的資料單位，所有輸入輸出皆為 (key, value) 的形式。
Map 階段	對每筆輸入獨立處理，產生中間鍵值對。
Shuffle 階段	依鍵把中間結果分組並搬移到對應的 Reducer，是效能的關鍵瓶頸。

核心概念	說明
Reduce 階段	對同一個鍵的所有值進行彙整，產生最終結果。
Combiner	在 Map 端先做局部彙整，以減少 Shuffle 傳輸量的最佳化手段。
Partitioner	決定某個鍵應被送往哪一個 Reducer 的規則。

表 9-1 本章核心概念一覽

9.1 MapReduce 的核心思想

9.1.1 把難的事包起來，把簡單的留給你

MapReduce 最深刻的貢獻，其實是一種「關注點分離」。在它出現之前，要寫一個跑在上千台機器上的程式，工程師必須親自處理：資料怎麼切分給各機器、任務怎麼調度、某台機器掛了怎麼重跑、中間結果怎麼在網路上搬移、最後結果怎麼彙整。這些事情極其困難，且與你真正想解決的問題（例如「數一數某個詞出現幾次」）毫無關係。

MapReduce 把這些雜事全部收進框架裡，只要求你回答兩個問題：對單獨一筆資料，你要做什麼？（Map）對同一組結果，你要怎麼合併？（Reduce）剩下的分散、容錯、調度，框架自動代勞。這讓一個不懂分散式系統的工程師，也能寫出跑在一千台機器上的程式——這才是它真正改變世界的地方。

9.1.2 一切皆為鍵值對

MapReduce 的資料模型單純到極致：所有的輸入與輸出，都是「鍵值對」（key-value pair）。整個運算流程可以用一串型別轉換來描述：

Map: $(k_1, v_1) \rightarrow \text{list}(k_2, v_2)$

Reduce: $(k_2, \text{list}(v_2)) \rightarrow \text{list}(k_3, v_3)$

式 9-1 MapReduce 的型別轉換

讀懂這兩行，你就讀懂了 MapReduce 的一半。Map 函式接收一筆輸入，吐出零筆到多筆的中間鍵值對；框架接著把「相同鍵」的所有值收集在一起（這就是 Shuffle）；Reduce 函式則針對每一個鍵，拿到它的整串值，加以彙整後輸出。

這個設計最關鍵的限制，是 Map 必須「獨立」處理每一筆輸入——處理第一筆時，不能依賴第二筆的結果。正是這個限制，讓成千上萬筆資料能被同時、平行地處理；也正是這個限制，讓 MapReduce 有能力自動容錯（任一個 Map 任務失敗，只要重跑那一份即可，不影響其他）。

9.1.3 把運算搬到資料旁邊

MapReduce 還有一個與第 5 章密切相關的設計哲學：資料在地性（data locality）。在動輒 PB 等級的資料面前，搬移資料的成本遠高於搬移程式碼——資料要走網路，程式碼只有幾 KB。

因此 MapReduce 會盡可能把 Map 任務「排到資料所在的那台機器上執行」，讓運算就地進行、避免大量的網路傳輸。這正是它與 HDFS 天生一對的原因：HDFS 知道每個區塊在哪台機器，MapReduce 就依這份地圖來派工。回想第 1 章談過的「儲存運算分離」演進——在網路頻寬昂貴的年代，這種「運算就地」的設計是必要的；直到雲端時代頻寬變便宜，架構才逐漸鬆綁。

9.2 Map、Shuffle 與 Reduce 三階段

現在我們把整條資料流走一遍。以「詞頻統計」為例——計算一大批文件中每個詞各出現幾次——這是 MapReduce 的「Hello World」。

9.2.1 Map 階段：各自為政

輸入的大檔案被切成多個分片，每個分片交給一個 Map 任務處理。在詞頻統計中，Map 函式讀入一行文字，把它切成一個個詞，然後為每個詞吐出一組鍵值對：(詞, 1)。

注意它有多「笨」——它不做任何計數，看到「機器人」就吐出 (機器人, 1)，再看到一次就再吐一次 (機器人, 1)。這種笨拙是刻意的：因為簡單、獨立、無狀態，所以能被大量平行執行，也能在失敗時輕易重跑。

9.2.2 Shuffle 階段：框架的魔法

Map 結束後，各台機器上散落著大量的中間鍵值對。此時框架要做一件關鍵的事：把「同一個鍵」的所有值，從各台機器上收集起來、送到同一個 Reduce 任務。這個依鍵分組並跨網路搬移的過程，稱為 Shuffle。

Shuffle 是唯一不需要你寫程式的階段——它由框架自動完成。但你必須理解它，因為它幾乎總是整個作業的效能瓶頸：這是唯一需要大量跨機器網路傳輸的階段，也是唯一必須「等 Map 幾乎都做完才能完成」的同步點。回想第 1 章的阿姆達爾定律，Shuffle 正是那個「不可平行化的部分」的主要來源。

叫獸提醒

如果你只能從這一章記住一件事，我希望是這句：MapReduce 調校的核心，就是想辦法讓 Shuffle 的資料變少。Map 和 Reduce 是各自機器上的本地運算，快得很；真正拖慢作業的，是那一大堆資料在網路上飛來飛去。這也是為什麼下一節的 Combiner 如此重要，以及為什麼一個分布不均的鍵（熱點）會讓某個 Reducer 忙到天荒地老——這與第 7 章的分區熱點，其實是同一個問題的不同面貌。

9.2.3 Reduce 階段：收攏成果

Reduce 任務拿到的，是一個鍵以及它對應的整串值。在詞頻統計中，Reducer 收到的是 (機器人, [1, 1, 1, ...])，它要做的就是這串 1 加總起來，輸出 (機器人, 856)。

每個 Reducer 通常負責一部分的鍵，最終各自輸出一個結果檔。整個作業結束後，這些檔案合起來就是完整的答案。至此，一次 MapReduce 作業完成——你只寫了兩個簡單的函式，框架卻已經替你在上千台機器上完成了調度、傳輸與容錯。

9.3 Combiner 與 Partitioner

9.3.1 Combiner：在寄出之前先打包

承接上一節的提醒——既然 Shuffle 是瓶頸，最直接的最佳化就是「讓要傳的資料變少」。Combiner 正是為此而生。

Combiner 的做法是：在 Map 任務的機器上，先對該機器產出的中間結果做一次「局部彙整」，再送出去。以詞頻統計為例，某台機器的 Map 產生了一萬個 (機器人, 1)，若直接送出，就是一萬筆資料要走網路；但若先在本機加總成一筆 (機器人, 10000)，網路上就只需傳一筆。這往往能帶來數十甚至數百倍的縮減，本章的公式會把它精確算出來。

但 Combiner 不是隨便都能用。它有一個嚴格的前提：Reduce 的運算必須滿足「結合律與交換律」——也就是說，分批先算再合併，結果必須與一次全部算相同。加總、求最大值、計數都符合；但「求平均」就不符合——先算各機器的平均、再平均那些平均值，得到的答案是錯的（除非各批數量相同）。這是初學者最常踩的坑。

運算	可用 Combiner?	說明
加總 (sum)	可以	分批相加再總加，結果相同。
最大／最小值	可以	分批取極值再取極值，結果相同。
計數 (count)	可以	分批計數再加總即可。
平均 (average)	不可以	平均的平均不等於總平均；須改傳「總和與個數」再於 Reduce 計算。

表 9-2 常見運算是否適用 Combiner

9.3.2 Partitioner：決定誰去哪一桌

當有多個 Reduce 任務時，框架必須決定「某個鍵該送給哪一個 Reducer」。做這個決定的就是 Partitioner，預設做法通常是「鍵的雜湊值對 Reducer 數量取餘數」。

這與第 7 章的分區問題如出一轍，也會遇到同樣的敵人：資料傾斜。若某個鍵的資料量特別龐大（例如中文詞頻中的「的」），負責它的那個 Reducer 就會遠比其他辛苦，導致整個作業被它一個人拖住——因為作業必須等所有 Reducer 都完成才算結束。此時可以自訂 Partitioner，或在鍵上加隨機前綴把熱點打散，這與第 7 章的解法完全相通。

9.4 以 MapReduce 設計演算法

9.4.1 思考的轉換：學會用鍵值對思考

初學 MapReduce 最大的障礙，不是語法，而是思維。你必須把習慣的「迴圈處理」思維，轉換成「鍵值對的分組」思維。

我的建議是，面對任何問題時先問自己三個問題：其一，我最終想「依什麼分組」來彙整結果？那就是 Reduce 的鍵。其二，為了達成這個分組，Map 該從每筆輸入吐出什麼樣的鍵值對？其三，同一個鍵的那串值到手後，我要怎麼把它們合成答案？

以詞頻為例：我想依「詞」分組（鍵是詞），所以 Map 吐出 (詞, 1)，Reduce 把值加總。掌握了這個「先想分組、再設計鍵」的心法，多數問題都能被順利拆解——這其實與第 6 章 NoSQL 建模的「先想查詢、再設計資料」異曲同工。

9.4.2 常見的設計技巧

實務上有幾個反覆出現的設計技巧，值得先認識。

- **複合鍵**：把多個欄位組合成一個鍵（如「年-月」），以達成多層次的分組。

- **以值攜帶標記**：在值中加入來源標記，讓 Reduce 能分辨這筆值來自哪個資料集，是實作 Join 的關鍵。
- **多階段作業**：複雜問題可拆成數個 MapReduce 作業串接，前一個的輸出作為後一個的輸入。
- **善用排序**：框架會把送到 Reducer 的鍵排序，可藉此設計出需要順序的演算法。

9.5 經典範例：詞頻、Join 與矩陣乘法

9.5.1 詞頻統計

詞頻統計前面已經走過一遍，這裡整理成完整的設計，作為對照的基準。

階段	輸入	輸出
Map	(行號, 一行文字)	對每個詞輸出 (詞, 1)
Combiner	(詞, [1,1,...])	(詞, 該機器的局部小計)
Reduce	(詞, [各機器小計])	(詞, 總次數)

表 9-3 詞頻統計的 MapReduce 設計

9.5.2 Join：把兩張表接起來

關聯式資料庫中稀鬆平常的 Join，在 MapReduce 中卻需要一點巧思。假設要把「訂單表」與「顧客表」依顧客編號接起來。

核心技巧是：讓兩個資料集的 Map 都以「顧客編號」為鍵輸出，但在值裡加上來源標記。於是 Reduce 收到某個顧客編號時，手上的那串值裡，就同時有來自顧客表的資料與來自訂單表的資料，它便能在本地把它們配對起來。這種做法稱為「Reduce 端 Join」。

它的缺點是所有資料都必須經過 Shuffle，成本高昂。若其中一張表夠小（如顧客表只有幾 MB），還有一種更快的做法：把小表直接送到每台機器的記憶體中，讓 Map 階段就地完成配對，完全不需要 Shuffle——這稱為「Map 端 Join」，是實務上極重要的最佳化。

9.5.3 矩陣乘法

矩陣乘法是許多機器學習與圖演算法的核心運算（例如第 18 章的 PageRank）。以 MapReduce 計算矩陣 A 與 B 的乘積時，關鍵同樣在於「設計對的鍵」。

基本思路是：結果矩陣中第 i 列第 j 行的元素，等於 A 的第 i 列與 B 的第 j 行做內積。因此我們讓 Map 把 A 與 B 的每個元素，都以「它會貢獻到哪個結果位置」為鍵輸出；Reduce 收到同一個位置 (i, j) 的所有相關元素後，把該乘的乘起來、該加的加起來，就得到該位置的值。

這個例子展示了 MapReduce 思維的威力：即使是看似需要複雜巢狀迴圈的運算，只要能想清楚「以什麼為鍵分組」，就能被拆解成平行的 Map 與 Reduce。

9.6 MapReduce 的限制與演進

9.6.1 三個難以迴避的限制

MapReduce 開創了時代，但它的設計也帶來了明顯的限制，這些限制正是後續框架誕生的動力。

- **中間結果必須落地：**每一輪 Map 與 Reduce 之間，中間資料都要寫入磁碟。對於需要反覆迭代的演算法（如機器學習訓練、PageRank），每一輪都要重讀重寫磁碟，效率極差。
- **表達能力受限：**所有運算都得硬塞進「Map 加 Reduce」兩個階段。稍微複雜的邏輯就得拆成多個作業串接，寫起來繁瑣、跑起來也慢（每個作業之間又要落地一次）。
- **不適合互動與即時：**作業的啟動與調度本身就有可觀的開銷，動輒數十秒起跳。這使它只適合離線批次處理，無法支援互動式查詢或即時分析。

9.6.2 從 MapReduce 到 Spark

這三個限制，幾乎就是 Spark 的產品需求書。二〇〇九年前後誕生的 Spark，正是針對它們逐一提出解方：以記憶體運算取代反覆的磁碟落地、以豐富的運算子取代僵硬的兩階段、以更輕量的調度支援互動查詢。這使 Spark 在迭代型工作負載上，能達到 MapReduce 數十倍的速度。

但請注意，Spark 並沒有推翻 MapReduce 的思想，而是繼承並擴充了它。Spark 的許多運算子（如 map、reduceByKey）名稱就直接沿用；它同樣有 Shuffle，同樣要面對資料傾斜，同樣以「分區的平行處理加上依鍵彙整」為核心。所以這一章絕不是歷史課——你在這裡建立的思維，會原封不動地用在第 11、12 章的 Spark 上。

9.6.3 承先啟後

本章我們從「找一百個同學數書」的比喻出發，理解了 MapReduce 如何把分散式運算的複雜性包裝起來，只留下 Map 與 Reduce 兩個單純的函式給程式設計師；也走過了三個階段的資料流、認識了 Combiner 與 Partitioner 這兩把最佳化的利器，並用詞頻、Join 與矩陣乘法練習了「用鍵值對思考」。

但你可能已經注意到一個被略過的問題：這一千台機器的資源，究竟是誰在分配？當多個作業、甚至多個團隊同時要用這座叢集時，誰決定哪個任務跑在哪台機器上、誰能用多少記憶體？這個「叢集資源該如何管理與調度」的問題，就是下一章的主題。

公式與流程：Combiner 能省下多少 Shuffle

9.3 節說 Combiner 能大幅減少 Shuffle 的傳輸量，這一節我們把它算清楚，讓你看見這個最佳化到底有多值錢。

設共有 M 個 Map 任務，每個 Map 任務產生 K 筆中間鍵值對，而這些鍵值對中「不重複的鍵」約有 D 種。在不使用 Combiner 時，所有中間結果都要送上網路，故 Shuffle 的總傳輸筆數為：

$$\text{無 Combiner: Shuffle 筆數} = M \times K$$

式 9-2 未使用 Combiner 時的 Shuffle 傳輸量

使用 Combiner 後，每個 Map 任務會先在本機把相同鍵合併，因此至多只送出 D 筆（每個不重複的鍵各一筆），故總傳輸筆數降為：

$$\text{有 Combiner: Shuffle 筆數} \approx M \times D$$

式 9-3 使用 Combiner 後的 Shuffle 傳輸量

兩式相除，可得縮減倍數：

$$\text{縮減倍數} = K / D$$

式 9-4 Combiner 的縮減倍數

這個結果很有啟發性：縮減效果只取決於「每個 Map 任務內，平均每個鍵重複出現幾次」。重複度越高，Combiner 越划算。以中文詞頻統計為例，假設有 1,000 個 Map 任務，每個處理一千萬個詞 ($K = 10^7$)，而常用詞彙約十萬種 ($D = 10^5$)。

情境	Shuffle 筆數	說明
無 Combiner	10^{10} 筆 (百億)	$1,000 \times 10^7$ ，海量資料湧上網路
有 Combiner	10^8 筆 (一億)	$1,000 \times 10^5$ ，僅為原本的百分之一
縮減倍數	100 倍	$K/D = 10^7/10^5 = 100$

表 9-4 詞頻統計中 Combiner 的縮減效果

叫獸提醒

一百倍是什麼概念？如果原本 Shuffle 要跑一小時，加了一個十行不到的 Combiner，就變成三十六秒。這大概是整個大數據領域投資報酬率最高的十行程式碼。但也別忘了表 9-2 的警告——用在「平均」這種不滿足結合律的運算上，你不會得到一百倍加速，你會得到一個錯得很安靜的答案。快，但錯，比慢更可怕。

案例研討

案例一 數清整個網路的詞

Google 最初開發 MapReduce，正是為了處理網頁索引這類工作：把數十億個網頁掃過一遍，統計詞彙、建立倒排索引、計算連結關係。這些工作有個共同特徵——邏輯上極其單純（就是數一數、排一排），但資料量大到必須動用上千台機器。

在 MapReduce 出現之前，Google 的工程師每寫一個這類程式，就要重新處理一次分散、容錯與調度，寫出來的程式碼有九成是與「網頁索引」無關的基礎設施。MapReduce 把那九成收進框架後，工程師只需寫剩下那一成的核心邏輯。生產力的躍升，才是這篇論文真正的遺產——這也呼應了本章 9.1 節「把難的包起來、把簡單的留給你」。

案例二 一個 Combiner 換來的下班時間

某團隊每晚要跑一個日誌分析作業，統計各類事件的發生次數。作業原本需要三個多小時，每晚都得有人守著。工程師檢視監控後發現：Map 與 Reduce 的計算時間都很短，將近八成的時間耗在 Shuffle——大量的 (事件類型, 1) 在網路上飛。

他加了一個十餘行的 Combiner，在各機器先把相同事件類型加總。由於事件類型只有數百種，而每台機器處理數百萬筆日誌，依式 9-4，縮減倍數達到數千倍。作業時間從三小時降到二十餘分鐘。這個案例說明：分散式效能調校，往往不是換更強的機器，而是先看清楚「瓶頸在哪一段」——而在 MapReduce 中，答案十之八九是 Shuffle。

案例三 被「的」拖垮的作業

另一個團隊做中文詞頻統計時遇到怪事：作業的進度總是很快衝到 99%，然後就卡在那裡好幾個小時。檢查後發現，絕大多數 Reducer 都在幾分鐘內完成，只有一個 Reducer 還在苦撐——它負責的鍵是「的」。

這正是 9.3.2 節的資料傾斜：中文語料中「的」的出現次數，可能是次常用詞的數十倍，全部湧向同一個 Reducer。因為作業必須等所有 Reducer 完成才算結束，這一個熱點鍵就綁架了整個作業。解法有幾種：加 Combiner 先在 Map 端大幅壓縮、把熱點鍵加上隨機前綴拆成多份再做二次彙整、或自訂 Partitioner。你會發現，這與第 7 章的分區熱點根本是同一個問題——原理相通，解法也相通。

案例四 為什麼機器學習受不了 MapReduce

某研究團隊想用 MapReduce 訓練一個需要迭代一百次的機器學習模型。他們發現，每一輪迭代都要把上一輪的模型參數與訓練資料從磁碟讀出、算完再寫回磁碟——一百輪就是一百次的讀寫循環，光是磁碟 I/O 就佔了絕大部分時間，真正的計算反而微不足道。

這正是 9.6.1 節「中間結果必須落地」的痛點。同樣的工作若改用 Spark，訓練資料可以一次載入記憶體並在多輪迭代間重複使用，速度可提升數十倍。這個案例直接說明了為何 Spark 會取代 MapReduce 成為主流——不是因為 MapReduce 的思想錯了，而是因為它的實作方式在迭代型工作負載上代價太高。這也為第 11 章的 Spark 埋下了伏筆。

實作活動

活動一 用紙筆跑一次 MapReduce

請以人工方式模擬一次完整的 MapReduce 流程，體會資料的流動。

- 步驟 1. 準備三個短句（作為三個輸入分片），例如「機器人 學習 機器人」「學習 資料」「資料 機器人 資料」。
- 步驟 2. 為每個分片寫出 Map 階段的輸出（一連串的鍵值對）。
- 步驟 3. 手動執行 Shuffle：把相同的鍵分組在一起。
- 步驟 4. 寫出 Reduce 階段的輸出，並比較「有無 Combiner」時，Shuffle 階段各需搬移幾筆資料。

活動二 用 Python 實作 MapReduce 思維

不必架設 Hadoop 叢集，也能練習 MapReduce 的思維。請以 Python 完成下列練習。

- 步驟 1. 撰寫一個 map 函式：讀入一行文字，回傳一串 (詞, 1) 的清單。
- 步驟 2. 撰寫一個 shuffle 函式：把多個 map 的輸出，依鍵分組成 (詞, [值清單])。
- 步驟 3. 撰寫一個 reduce 函式：對每個鍵的值清單加總。
- 步驟 4. 加入 combiner：在 shuffle 之前先對每個 map 的輸出做局部加總，並印出「有無 combiner」時進入 shuffle 的資料筆數，驗證式 9-4。

活動三 設計一個 Join

請設計一個 MapReduce 作業，把「學生表（學號, 姓名）」與「成績表（學號, 科目, 分數）」依學號接起來，輸出「姓名, 科目, 分數」。請寫出：兩個資料集的 Map 各應輸出什麼鍵值對（提示：值中需帶來源標記）？Reduce 收到某個學號的值清單後，該如何配對？最後思考：若學生表很小（僅數百筆），能否改用 Map 端 Join？這樣做省下了什麼？

延伸閱讀

- **MapReduce 原始論文**：Dean 與 Ghemawat 於二〇〇四年發表的《MapReduce: Simplified Data Processing on Large Clusters》。篇幅適中、範例清晰，強烈建議在讀完本章後親自閱讀。
- **《Mining of Massive Datasets》第 2 章**：Leskovec 等人著。以演算法角度深入探討 MapReduce 的設計技巧與成本模型，是 9.4、9.5 節的最佳延伸。
- **《Designing Data-Intensive Applications》第 10 章**：Kleppmann 與 Riccomini 著，探討批次處理的設計哲學與 MapReduce 的定位，並延伸至其後繼者。
- **Apache Hadoop MapReduce 官方教學**：提供 Java 版詞頻統計的完整範例與參數說明，適合想動手實作者。

本章小結

1. MapReduce 的核心貢獻是「關注點分離」：把資料切分、任務調度、容錯與網路傳輸等雜事收進框架，只要求程式設計師回答「對單筆資料做什麼 (Map)」與「如何合併結果 (Reduce)」兩個問題。
2. MapReduce 的資料模型全為鍵值對，型別轉換為 Map: $(k_1, v_1) \rightarrow \text{list}(k_2, v_2)$ 、Reduce: $(k_2, \text{list}(v_2)) \rightarrow \text{list}(k_3, v_3)$ ；Map 必須獨立處理每筆輸入，這個限制正是平行化與自動容錯的基礎。
3. 運算分為三階段：Map 各自為政地產生中間鍵值對，Shuffle 由框架依鍵分組並跨網路搬移，Reduce 對每個鍵的值串彙整；Shuffle 是唯一的大量網路傳輸與同步點，也幾乎總是效能瓶頸。
4. Combiner 在 Map 端先做局部彙整以減少 Shuffle 傳輸，但僅適用於滿足結合律與交換律的運算（加總、極值、計數可用，平均不可用，須改傳總和與個數）。
5. Partitioner 決定鍵送往哪個 Reducer，預設為雜湊取餘數；若某鍵資料量特別龐大將造成資料傾斜，使單一 Reducer 拖垮整個作業，解法與第 7 章分區熱點相通。
6. 設計 MapReduce 演算法的心法是「先想依什麼分組、再設計鍵」；常見技巧包括複合鍵、以值攜帶來源標記 (Join)、多階段作業串接與善用框架排序。
7. MapReduce 的三大限制是中間結果必須落地、表達能力受限於兩階段、以及調度開銷不適合互動與即時；這三點正是 Spark 誕生的動因，但 Spark 繼承而非推翻了它的核心思想。
8. Combiner 的縮減倍數為 K/D (式 9-4)，即「每個 Map 任務內平均每個鍵的重複次數」；在詞頻統計等重複度高的場景，可達百倍以上的 Shuffle 縮減。

習題

一、觀念題

1. MapReduce 為程式設計師隱藏了哪些複雜性？請至少列舉三項，並說明這對生產力的意義。
2. 為什麼 Map 函式必須「獨立處理每一筆輸入」？這個限制帶來了哪兩項好處？
3. 為什麼「求平均」不能直接使用 Combiner？請說明原因，並提出正確的做法。
4. 請說明何謂資料傾斜，並列舉至少兩種在 MapReduce 中緩解它的方法。

二、計算題

1. 某 MapReduce 作業有 500 個 Map 任務，每個任務產生 2×10^6 筆中間鍵值對，其中不重複的鍵約有 5×10^3 種。請以式 9-2 至式 9-4 計算：（一）無 Combiner 時的 Shuffle 傳輸筆數；（二）有 Combiner 時的傳輸筆數；（三）縮減倍數。
2. 承上題，若該作業的 Shuffle 階段原本耗時 80 分鐘，且傳輸時間與資料量成正比，請估算加入 Combiner 後 Shuffle 約需多久？若 Map 與 Reduce 合計固定耗時 10 分鐘，總作業時間由多少降為多少？

三、思考與討論

1. 案例四指出機器學習的迭代訓練不適合 MapReduce。請說明原因，並討論 Spark 是以什麼方式解決這個問題的。
2. 有人說「MapReduce 已經被 Spark 淘汰，沒有學習的必要」。請根據本章內容評論此說法。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

至少可列舉：資料的切分與分配、任務的調度與派工、節點故障時的自動重跑（容錯）、中間結果的跨網路搬移與依鍵分組（Shuffle）、以及最終結果的彙整。意義：在此之前，工程師每寫一個分散式程式，都得重複處理這些與業務邏輯無關的基礎設施，程式碼可能九成都花在這上面；框架收攏後，工程師只需撰寫核心的 Map 與 Reduce 邏輯，使不熟悉分散式系統者也能寫出跑在上千台機器的程式（見案例一）。

第 2 題

因為若 Map 處理某筆輸入時需依賴其他輸入的結果，各筆輸入就無法同時計算。兩項好處：其一，平行化——彼此獨立故可將成千上萬筆資料分散到大量機器同時處理；其二，容錯——任一 Map 任務失敗時，只需重跑那一份輸入即可，不影響也不需重算其他任務（無狀態、可重複執行）。

第 3 題

因為平均不滿足結合律：先分別計算各機器的局部平均、再對這些平均值取平均，結果通常不等於全體的總平均（除非各批的資料筆數恰好相同）。例如 [1,2,3] 與 [10] 的總平均為 4，但局部平均分別為 2 與 10，再平均得 6，錯誤。正確做法：讓 Combiner 傳出「局部總和」與「局部筆數」兩個值，Reduce 端再以總和除以總筆數計算平均——即把不可結合的運算，改寫為兩個可結合的運算（加總）。

第 4 題

資料傾斜指某些鍵的資料量遠大於其他鍵，使負責它的 Reducer 工作量遠超其他人；因作業須等所有 Reducer 完成才結束，單一熱點鍵即可拖垮整個作業（如案例三的「的」）。緩解方法：其一，加 Combiner 在 Map 端先大幅壓縮該鍵的資料量；其二，為熱點鍵加上隨機前綴拆成多個子鍵，分散到不同 Reducer，再以第二個作業做二次彙整；其三，自訂 Partitioner 改變鍵的分派規則；其四，若適用，改採 Map 端 Join 等避開 Shuffle 的設計。

二、計算題

第 1 題

$$M = 500, K = 2 \times 10^6, D = 5 \times 10^3.$$

- (1) 無 Combiner (式 9-2) : $M \times K = 500 \times 2 \times 10^6 = 10^9$ 筆 (十億筆)。
- (2) 有 Combiner (式 9-3) : $M \times D = 500 \times 5 \times 10^3 = 2.5 \times 10^6$ 筆 (二百五十萬筆)。
- (3) 縮減倍數 (式 9-4) : $K/D = 2 \times 10^6 / 5 \times 10^3 = 400$ 倍。

延伸思考：也可用 $10^9 \div 2.5 \times 10^6 = 400$ 驗證，兩種算法一致。

第 2 題

傳輸時間與資料量成正比，資料量縮減 400 倍，故時間亦縮為 $1/400$ 。

- (1) 加入 Combiner 後的 Shuffle 時間 = $80 \div 400 = 0.2$ 分鐘（約 12 秒）。
- (2) 原總作業時間 = $80 + 10 = 90$ 分鐘。
- (3) 新總作業時間 $\approx 0.2 + 10 = 10.2$ 分鐘。

延伸思考：總時間由 90 分鐘降至約 10.2 分鐘，約 8.8 倍加速——但請注意，整體加速倍數（8.8）遠小於 Shuffle 的縮減倍數（400），因為 Map 與 Reduce 那 10 分鐘並未被縮短。這正是第 1 章阿姆達爾定律的體現：優化了瓶頸之後，未被優化的部分就成為新的上限。

三、思考與討論

第 1 題

原因：MapReduce 要求每輪的中間結果落地至磁碟，迭代型訓練需反覆讀寫同一份訓練資料與模型參數；迭代一百次即產生一百次的磁碟讀寫循環，I/O 成本遠超實際計算，效率極差。Spark 的解法：以彈性分散式資料集（RDD）將資料快取於記憶體中，讓多輪迭代重複使用同一份記憶體資料，免除反覆落地；同時以 DAG 執行引擎把多個運算串接為一個執行計畫，減少不必要的中間落地與作業啟動開銷。可預告第 11 章。

第 2 題

此說法有部分事實但結論過當。可指出：其一，就「實務撰寫」而言，Spark 確實已大幅取代 MapReduce，這點屬實。其二，但 Spark 繼承而非推翻其思想——運算子名稱（map、reduceByKey）、Shuffle 機制、依鍵彙整的模型、資料傾斜的問題與解法，皆一脈相承；不懂 MapReduce 的人，在 Spark 遇到 Shuffle 效能問題時往往無從下手。其三，MapReduce 建立的「用鍵值對思考」是分散式運算的通用思維，適用於各種框架。其四，仍有大量既有系統以 MapReduce 運行，維護時需要理解。周延的結論：不必以 MapReduce 為主要開發工具，但必須理解其原理，因為它是整個批次運算思維的地基。

第參篇 批次運算框架

Batch Processing Frameworks

第 10 章

叢集資源管理與排程

Cluster Resource Management and Scheduling

機器人叫獸（李世淵） 編著

叫獸開講

我們系上有一間電腦教室，四十台機器。開學初一切太平，但每到期末，事情就變得很有趣。

研究生甲要跑他的深度學習訓練，一個人佔了三十台機器連跑三天。研究生乙的論文口試在後天，急著跑一個十分鐘就好的實驗，卻連一台機器都排不到。大學部的專題生丙寫了一支有臭蟲的程式，把記憶體吃光，連帶把整台機器上其他人的工作一起弄掛。而助教丁想跑一個定期的維護作業，卻永遠搶不到資源。

你看，一旦「多個使用者共用一批機器」，就會冒出一連串問題：資源怎麼分配才算公平？急件能不能插隊？某人的程式失控時，如何不要害到別人？平時閒置的機器，能不能借給別人先用？這些問題，跟四十台電腦是這樣，跟一萬台伺服器也是這樣——只是規模大了，混亂也大了。

上一章結尾我留了一個問題：那一千台機器的資源，究竟是誰在分配？答案就是這一章的主角——叢集資源管理器。它的工作，就是當那個公正、精明又鐵面無私的「資源總管」：誰能用多少、誰先誰後、誰跟誰要隔開，全由它說了算。你會發現，MapReduce 與 Spark 這些運算框架，其實都只是它的「客戶」——它們負責算，而資源總管負責分配算的權利。

學習目標

研讀完本章之後，你應該能夠：

1. 說明叢集資源管理器的角色，以及它與運算框架之間的分工。
2. 描述 YARN 的架構與各元件的職責，並說明一次作業的提交流程。
3. 比較 FIFO、Capacity 與 Fair 三種排程策略的行為與適用場景。
4. 說明資源隔離的必要性，以及容器如何實現隔離。
5. 說明 Mesos 等其他資源管理器的設計差異。
6. 運用資源碎片化的概念，評估容器規格對叢集利用率的影響。

核心概念

核心概念	說明
資源管理器	統籌叢集運算資源、決定分配與調度的系統，如 YARN、Mesos。
YARN	Hadoop 的資源管理框架，把資源管理與運算邏輯分離。
ResourceManager	YARN 的主節點，掌管全叢集資源的分配決策。
NodeManager	YARN 的工作節點代理，管理本機資源並啟動容器。
容器 (Container)	被分配之運算資源（記憶體、CPU）的封裝單位，任務在其中執行。
排程策略	決定「誰先拿到資源、拿多少」的規則，如 FIFO、Capacity、Fair。
多租戶	多個使用者或團隊共用同一叢集，需以配額與隔離避免相互干擾。

表 10-1 本章核心概念一覽

10.1 叢集資源管理的角色

10.1.1 把「算什麼」與「用誰的機器算」分開

要理解資源管理器的價值，得先看看沒有它的世界長什麼樣。早期的 Hadoop（第一代）把資源管理與 MapReduce 的執行邏輯綁在一起：一個叫做 JobTracker 的元件，同時負責「分配資源」與「監督 MapReduce 任務的執行」。

這種綁定造成兩個嚴重問題。其一，JobTracker 要同時做兩件不同性質的事，負擔過重，成為擴充的瓶頸。其二，也更致命的是——這座叢集只能跑 MapReduce。若你想在同一批機器上跑 Spark、跑其他框架，辦不到，因為資源管理是為 MapReduce 客製的。

第二代 Hadoop 的重大改革，就是把這兩件事拆開，把資源管理獨立成一層通用的服務，這就是 YARN（Yet Another Resource Negotiator）。從此，資源管理器只管「誰能用多少機器」，至於拿到資源後要跑 MapReduce、Spark 還是其他什麼，它一概不管。這個「分層解耦」的設計，讓同一座叢集能同時服務多種運算框架，是大數據平台走向通用化的關鍵一步。

10.1.2 資源管理器的四項工作

具體而言，一個資源管理器要處理四件事，它們也是本章各節的主軸。

- **資源盤點：**掌握叢集中每台機器有多少可用的記憶體、CPU 等資源，以及即時的使用狀況。
- **排程決策：**當多個作業同時要資源時，決定誰先拿、拿多少——這是最核心也最有學問的部分。
- **資源隔離：**確保某個作業不會吃掉超出配額的資源，以免影響同一台機器上的其他作業。
- **生命週期管理：**啟動任務、監控其狀態、在失敗時通報或重啟，並在完成後回收資源。

叫獸提醒

這個分層的思想，你在本書後面會一再遇到。第 29 章的 Kubernetes，本質上就是雲原生時代的資源管理器——它管的是容器而非 MapReduce 任務，但要解決的問題完全一樣：資源盤點、排程、隔離、生命週期。所以讀懂這一章的 YARN，你等於預習了 Kubernetes 的半壁江山。技術會換名字，但問題與思路會留下。

10.2 YARN 架構與元件

10.2.1 三個角色

YARN 採用我們已經很熟悉的主從架構（回想第 5 章的 HDFS），由三個主要角色構成。

元件	職責	類比
ResourceManager	全叢集唯一的主節點，掌握所有資源並做出分配決策	資源總管
NodeManager	每台機器上的代理，回報本機資源、啟動與監控容器	各樓層的管理員
ApplicationMaster	每個作業各有一個，負責向總管要資源並調度自己的任務	各專案的工頭

表 10-2 YARN 的三個核心角色

這裡最精妙的設計是 ApplicationMaster。它是「每個作業一個」的臨時角色，隨作業啟動而生、隨作業結束而亡。它負責向 ResourceManager 申請資源，並自行調度該作業內部的任務。

為什麼這個設計重要？因為它把「作業內部怎麼調度」的責任，從中央的 ResourceManager 下放給了各作業自己。如此一來，ResourceManager 的負擔大幅減輕（它只管分配資源這件事），而不同的運算框架也能各自實作自己的 ApplicationMaster，用自己的邏輯調度任務——MapReduce 有 MapReduce 的工頭，Spark 有 Spark 的工頭，彼此不干擾。這正是 YARN 能支援多種框架的技術關鍵。

10.2.2 一次作業的完整流程

讓我們把一次作業提交的流程走一遍，你就能看清這三個角色如何協作。

- 第 1 步。** 客戶端向 ResourceManager 提交作業。
- 第 2 步。** ResourceManager 找一台有空的機器，請它的 NodeManager 啟動一個容器，在其中執行這個作業的 ApplicationMaster（工頭上工）。
- 第 3 步。** ApplicationMaster 啟動後，向 ResourceManager 申請執行實際任務所需的資源（例如「我需要 50 個各含 4 GB 記憶體容器」）。
- 第 4 步。** ResourceManager 依排程策略決定給多少、給哪些機器，回覆一份「資源許可」清單。
- 第 5 步。** ApplicationMaster 拿著許可，直接聯絡相應的 NodeManager，請它們啟動容器並執行任務。
- 第 6 步。** 任務執行期間，各任務向 ApplicationMaster 回報進度；ApplicationMaster 則向 ResourceManager 回報作業狀態。
- 第 7 步。** 作業完成後，ApplicationMaster 歸還資源並自行結束，ResourceManager 回收資源。

請特別注意第 5 步：實際的任務啟動是由 ApplicationMaster 直接聯絡 NodeManager，不經過 ResourceManager。這與第 5 章 HDFS 讀取時「客戶端直連 DataNode、不經 NameNode」的設計哲學完全一致——主節點只管調度決策，不碰資料流與執行細節，這樣才不會成為瓶頸。

10.3 排程策略：FIFO、Capacity 與 Fair

排程策略是資源管理器最核心的學問——它回答的是叫獸開講裡那個問題：當研究生甲、乙、專職生丙同時要機器時，該給誰？三種主流策略提供了不同的答案。

10.3.1 FIFO 排程：先到先做

最單純的策略是先進先出（FIFO）：作業排成一列，先提交的先執行，後面的排隊等待。它的優點是簡單、可預測、實作容易。

但它有一個致命的問題，正是叫獸開講中研究生乙的處境：一個要跑三天的大作業排在前面，後面那個只需十分鐘的小作業，就得傻等三天。這種「大作業堵住整條路」的現象，讓 FIFO 幾乎無法用於多人共用的環境。它只適合單一使用者、作業性質相近的專用叢集。

10.3.2 Capacity 排程：先劃地盤

Capacity（容量）排程的思路是「事先劃分地盤」：把叢集資源依比例切分成多個「佇列」，各佇列有保證的容量。例如研發部門佔 50%、業務部門佔 30%、維運部門佔 20%。每個佇列內部則可再用 FIFO 或其他策略排序。

這個策略的價值在於「可預期的保障」：研發部門知道自己隨時至少有一半的叢集可用，不會被別的部門淹沒。而為了避免資源閒置浪費，它通常還支援「彈性借用」——某佇列若沒用完自己的份額，可暫時借給其他佇列使用；等原主人需要時再要回來。這在企業多部門共用叢集的場景中極為實用。

10.3.3 Fair 排程：人人有份

Fair（公平）排程的思路不同：它不事先劃地盤，而是動態地讓所有執行中的作業「平分」資源。若叢集裡只有一個作業，它可以用滿全部資源；當第二個作業進來，資源就逐漸調整為兩者各半；第三個進來，就各三分之一。

這個策略解決了 FIFO 的堵塞問題——小作業不必等大作業跑完，一進來就能分到資源、很快完成。它特別適合作業大小差異懸殊、且希望互動式查詢能快速回應的環境。代價是實作較複雜，且大作業的完成時間會被拉長（因為它得讓出資源）。

策略	分配方式	優點	缺點
FIFO	先提交先執行	簡單、可預測	大作業堵塞小作業
Capacity	依佇列比例保障，可彈性借用	各團隊有保障、避免閒置	設定較繁瑣
Fair	執行中作業動態平分	小作業回應快、不被堵塞	大作業被拉長、實作複雜

表 10-3 三種排程策略的比較

叫獸提醒

排程策略沒有最好的，只有最符合「你的公平定義」的。仔細想想：讓先來的先做完，公平嗎？讓每個作業分到一樣多，公平嗎？還是該按部門付了多少錢來分？這其實已經不是技術問題，而是組織的政策問題。所以真實世界的叢集，往往是 Capacity 加上 Fair 的混搭——先按部門劃地盤（政策），部門內再公平共享（技術）。技術能執行政策，但不能替你決定政策。

10.4 資源隔離與容器

10.4.1 為什麼一定要隔離

回到叫獸開講裡的專題生丙——他的程式有臭蟲，吃光了機器的記憶體，把同一台機器上其他人的工作一起弄掛。這說明了一件事：光是「分配」資源不夠，還必須「強制」大家不能超用。這就是資源隔離。

沒有隔離的叢集，會出現一種令人抓狂的現象，業界稱為「吵鬧的鄰居」（noisy neighbor）：你的作業跑得又慢又不穩定，不是因為你的程式有問題，而是因為隔壁那個作業把資源吃光了。這種問題極難除錯，因為病因根本不在你的程式裡。

10.4.2 容器：資源的封裝單位

YARN 用「容器」（container）來實現隔離。這裡的容器是一個抽象概念：它代表「某台機器上的一份資源額度」，例如「4 GB 記憶體加 2 個 CPU 核心」。任務必須在容器內執行，而作業系統層的機制會確保它無法用超過額度的資源。

要留意的是，YARN 的容器與第 28 章要學的 Docker 容器並非同一件事，但概念上高度相通——都是「把資源與執行環境封裝起來、彼此隔離」。YARN 的容器偏重資源額度的限制，而 Docker 容器還進一步封裝了檔案系統與相依套件。事實上，現代的 YARN 也支援直接以 Docker 容器來執行任務，兩者正在融合。

隔離的維度主要有三：記憶體（超用即終止該任務）、CPU（限制可用的運算比例）、以及較少被提及但同樣重要的磁碟與網路頻寬。其中記憶體的隔離最為嚴格，因為記憶體超用會直接導致機器不穩定，不像 CPU 只是變慢。

10.5 Mesos 與其他資源管理器

10.5.1 兩種相反的排程哲學

YARN 不是唯一的解法。同時期另一個重要的資源管理器是 Apache Mesos，它採取了一種截然不同、甚至相反的設計思路。

YARN 的模式可稱為「請求—回應」：框架主動說「我需要 50 個容器」，由 ResourceManager 決定給不給、給哪些。決策權在資源管理器手上。

Mesos 的模式則稱為「資源供給」（resource offer）：Mesos 主動把「目前有哪些機器有空、各有多少資源」這份清單提供給各個框架，讓框架自己決定要不要接受、接受哪些。決策權在框架手上。

這個差異看似細微，卻反映了深刻的哲學分歧：究竟該由中央來統籌決定（YARN），還是由各框架依自己的需求自主挑選（Mesos）？前者較易實施全局政策，後者則讓框架能依自身特性做更聰明的選擇（例如某框架特別在意資料在地性，就能從清單中挑出資料所在的機器）。

面向	YARN	Mesos
排程模式	請求—回應（框架提出需求）	資源供給（管理器提供清單）
決策權	在資源管理器	在各運算框架
全局政策	較易實施	較難統一
框架自主性	較低	較高

表 10-4 YARN 與 Mesos 的排程哲學對照

10.5.2 走向 Kubernetes

時序推進到雲原生時代，資源管理的主角逐漸換成了 Kubernetes。它管理的單位不再是「MapReduce 任務」或「Spark 執行器」，而是通用的容器；它面對的不只是批次分析作業，還有長時間運行的線上服務。

但如果你把本章的四項工作（盤點、排程、隔離、生命週期）拿去對照 Kubernetes，會發現它們一一對應。這說明資源管理的問題本質從未改變，改變的只是被管理的對象與所處的環境。本書第 29 章會完整展開 Kubernetes，屆時你會發現自己早已熟悉它要解決的問題。

10.6 多租戶與資源配額

10.6.1 多租戶的三大挑戰

當一座叢集同時服務多個使用者、團隊甚至客戶時，這種架構稱為「多租戶」（multi-tenancy）。它是雲端與企業叢集的常態，也帶來三個必須正面處理的挑戰。

- **資源公平**：如何確保每個租戶都拿到應得的份額，不被強勢租戶擠壓？這靠排程策略與配額。
- **效能隔離**：如何避免某租戶的失控作業影響其他人？這靠容器與資源隔離。
- **安全隔離**：如何確保租戶之間看不到、動不了彼此的資料？這靠權限控管與認證機制，詳見第 33 章。

10.6.2 配額與優先權

實務上的多租戶治理，主要依靠幾項工具的組合。配額（quota）為每個租戶設定資源使用的上限，避免無限制擴張；保證額度（guarantee）則設定下限，確保租戶在需要時至少能拿到基本份額。這一上一下，界定了每個租戶的活動空間。

另一項重要工具是優先權（priority）與搶佔（preemption）。當高優先權的緊急作業進來、而叢集資源已被佔滿時，系統可以「搶佔」——強制中止某些低優先權的任務，把資源騰出來。這聽起來殘酷，但對於「有些作業真的比較重要」的現實而言是必要的。當然，被搶佔的任務必須能夠安全地重跑，這又回到了 MapReduce 那種「任務可重複執行」的容錯設計——你會發現，第 9 章的無狀態設計，在這裡又派上了用場。

10.6.3 承先啟後

本章我們認識了叢集的資源總管：它把「資源管理」從運算框架中解耦出來（10.1），以 ResourceManager、NodeManager 與 ApplicationMaster 三個角色協作（10.2），依 FIFO、Capacity 或 Fair 策略做出分配決策（10.3），用容器強制隔離以杜絕吵鬧的鄰居（10.4），並在多租戶環境中以配額、優先權與搶佔維持秩序（10.6）。

至此，第參篇的舞台已經搭好：第 9 章給了我們分散式運算的思維（MapReduce），本章給了我們執行運算的資源（YARN）。但我們也看到了 MapReduce 的三大痛點——中間結果反覆落地、表達能力受限於兩階段、調度開銷太重。有沒有一個框架，能繼承 MapReduce 的思想，卻擺脫這些包袱？有的，而它徹底改變了大數據運算的格局。下一章，我們就要正式認識這位主角：Apache Spark。

公式與流程：資源碎片化與叢集利用率

排程還有一個常被忽略、卻實實在在浪費錢的問題：資源碎片。這一節我們把它算出來，你會發現「容器規格怎麼設」是一個值得認真對待的決定。

設每台機器可分配的記憶體為 R ，每個容器需要的記憶體為 r 。由於容器不能被切開，一台機器最多只能容納整數個容器，故每台機器可放的容器數為：

$$\text{每節點容器數} = \lfloor R / r \rfloor$$

式 10-1 單節點可容納的容器數

剩下不足一個容器的資源就用不到了，形成「碎片」。其浪費比例為：

$$\text{碎片率} = (R - \lfloor R / r \rfloor \times r) / R$$

式 10-2 單節點的資源碎片率

我們以一台可分配 64 GB 記憶體的機器為例，看不同的容器規格會造成多少浪費。

容器規格 r	可放容器數	已用／浪費	碎片率
4 GB	16 個	64 GB／0 GB	0% (完美整除)
6 GB	10 個	60 GB／4 GB	約 6.3%
10 GB	6 個	60 GB／4 GB	約 6.3%
12 GB	5 個	60 GB／4 GB	約 6.3%
20 GB	3 個	60 GB／4 GB	約 6.3%
40 GB	1 個	40 GB／24 GB	約 37.5%

表 10-5 64 GB 節點在不同容器規格下的碎片率

看最後一系列的警訊：當容器規格大到「一台機器只放得下一個」時，碎片率高達 37.5%——等於你買了一百台機器，卻有三十七台的資源在空轉。若這座叢集有 200 個節點，光是這項浪費就相當於白買了 75 台機器的記憶體。

叫獸提醒

這個計算有一個非常實用的推論：容器規格最好是節點資源的「整除數」。表 10-5 中 4 GB 的碎片率是零，因為 64 除以 4 剛好整除。所以調參數時，別憑感覺填一個 7 GB 或 9 GB，先看看你的節點有多少可分配記憶體，挑一個能整除的數字。這是一個五秒鐘的決定，卻可能替你的叢集省下一成的資源。順帶一提，實務上還要留一部分記憶體給作業系統與 NodeManager 本身，所以 R 通常不等於機器的實體記憶體總量——這點在設定時務必查清楚。

案例研討

案例一 為什麼 Hadoop 要大改版

第一代 Hadoop 的 JobTracker 同時承擔資源管理與 MapReduce 任務監督。當叢集規模擴大到數千個節點時，JobTracker 因為要追蹤成千上萬個任務的狀態，記憶體與運算負擔雙雙爆表，成為整座叢集的擴充天花板。

更麻煩的是，當 Spark 等新框架出現時，企業發現自己被綁死了——想跑 Spark 就得再買一座叢集，因為現有叢集的資源管理是為 MapReduce 客製的。YARN 的誕生一舉解決兩個問題：把資源管理獨立出來、減輕主節點負擔（各作業的調度下放給自己的 ApplicationMaster），同時讓一座叢集能同時服務多種框架。這個案例說明了「分層解耦」在系統設計中的巨大價值——它不只是美學，而是實實在在的擴充能力與投資保障。

案例二 被大作業堵住的口試

回到叫獸開講的場景。某研究室的叢集原本採 FIFO 排程，一位同學提交了要跑三天的訓練作業。隔天，另一位同學為了後天的口試，需要跑一個十分鐘的實驗，卻被告知要等三天。

研究室後來改用 Fair 排程。此後小作業一提交，就能立即分到一部分資源，十分鐘的實驗依然十分鐘完成，而大作業僅是完成時間略微延後。這個案例展示了排程策略對「使用者感受」的巨大影響——同一批硬體、同樣的總吞吐量，只因為排程規則不同，一個環境令人抓狂，另一個則相安無事。技術決策從來不只是技術。

案例三 吵鬧的鄰居

某企業的資料團隊反覆遇到一個詭異現象：他們每日的報表作業，執行時間時而二十分鐘、時而兩小時，毫無規律。他們反覆檢視自己的程式碼與資料量，都找不出原因。

最後查明真相：同一座叢集上，另一個團隊的作業偶爾會失控地佔用大量記憶體與 CPU，而該叢集的資源隔離設定不完整，導致資源被搶走。這正是 10.4.1 節的「吵鬧的鄰居」——病因根本不在受害者的程式裡，難怪怎麼查都查不出來。解法是啟用嚴格的容器資源限制，並以 Capacity 排程為兩個團隊劃分保障額度。這個案例的教訓是：在多租戶環境中，「隔離」不是選配，而是必要的基礎設施。

案例四 整除數省下的七十五台機器

某公司的 200 節點叢集，每節點可分配 64 GB 記憶體。工程師依照過往經驗，把執行器的記憶體設為 40 GB，認為「大一點比較不會出問題」。

監控顯示叢集記憶體使用率長期只有六成多，團隊卻苦於資源不足。分析後發現原因正是式 10-2 的碎片：每節點 64 GB 只放得下一個 40 GB 容器，剩下 24 GB 全數閒置，碎片率 37.5%。全叢集等於有 4,800 GB 的記憶體在空轉——相當於 75 台機器。

他們把容器規格改為 16 GB（64 可整除），每節點可放 4 個容器、碎片率降為零，叢集的有效容量立刻提升了六成。這個案例把一條簡單的除法，變成了看得見的巨大效益——有時候最有價值的優化，不是寫更聰明的程式，而是把一個參數設對。

實作活動

活動一 推演三種排程策略

請以紙筆推演不同排程策略下的作業完成順序與時間。

- 步驟 1.** 假設叢集共有 12 個容器的資源。作業 A 需 12 個容器跑 60 分鐘、作業 B 需 4 個容器跑 5 分鐘，A 先提交、B 在第 1 分鐘提交。
- 步驟 2.** 在 FIFO 策略下，B 何時開始、何時完成？
- 步驟 3.** 在 Fair 策略下（假設資源可平分且立即調整），B 大約何時完成？A 的完成時間又受到多少影響？
- 步驟 4.** 討論：若你是管理者，會選哪一種？若 B 是攸關營運的即時查詢，答案會改變嗎？

活動二 算出你的碎片率

請運用式 10-1 與式 10-2 做一次資源規劃。

- 步驟 1.** 假設節點可分配記憶體為 48 GB，分別計算容器規格為 4、6、8、10、16、32 GB 時的每節點容器數與碎片率。
- 步驟 2.** 找出所有碎片率為零的容器規格（即 48 的整除數）。
- 步驟 3.** 若叢集有 150 個節點，計算容器規格為 10 GB 時，全叢集浪費的記憶體總量。
- 步驟 4.** 討論：除了碎片率，選擇容器規格時還須考慮什麼？（提示：容器太小可能造成單一任務記憶體不足）

活動三 設計一份多租戶政策

請為一座虛擬的校內共用叢集（假設 100 個節點）設計資源治理政策。使用者包含：三個研究室（各有數名研究生跑訓練作業）、大學部專題課程（數十位學生，作業小而多）、以及維運團隊的定期作業。請說明：你會如何劃分佇列與比例？各佇列採用什麼排程策略？是否設定優先權與搶佔？並說明你的「公平」定義是什麼，以及這樣設計可能引發哪些抱怨。

延伸閱讀

- **YARN 原始論文**：Vavilapalli 等人發表的《Apache Hadoop YARN: Yet Another Resource Negotiator》，說明其設計動機與架構，是本章 10.1、10.2 節的第一手資料。
- **Mesos 論文**：Hindman 等人發表，闡述「資源供給」模式的設計哲學，與 YARN 對照閱讀最能體會兩種思路的差異。
- **Apache Hadoop YARN 官方文件**：詳列 Capacity 與 Fair 排程器的參數與設定方式，適合需要實際調校者參考。
- **Google Borg 論文**：說明 Google 內部大規模叢集管理系統的設計，是 Kubernetes 的前身，可作為第 29 章的預習。

本章小結

1. 資源管理器把「資源管理」與「運算邏輯」解耦：第一代 Hadoop 的 JobTracker 兼管兩者而成為瓶頸，且叢集只能跑 MapReduce；YARN 將資源管理獨立為通用服務，使同一座叢集能同時服務多種運算框架。
2. 資源管理器的四項工作是資源盤點、排程決策、資源隔離與生命週期管理；這四項工作在 Kubernetes 等後續系統中依然成立，只是被管理的對象改變。
3. YARN 由 ResourceManager（資源總管）、NodeManager（各機器代理）與 ApplicationMaster（每作業一個的工頭）三角協作；把作業內部調度下放給 ApplicationMaster，是減輕主節點負擔並支援多框架的關鍵。
4. 作業流程中，實際任務由 ApplicationMaster 直接聯絡 NodeManager 啟動，不經 ResourceManager——與第 5 章「客戶端直連 DataNode」同一哲學：主節點只做決策，不碰執行細節。
5. FIFO 簡單但大作業會堵塞小作業；Capacity 依佇列比例保障容量並支援彈性借用，適合多部門共用；Fair 讓執行中作業動態平分，小作業回應快。排程策略的選擇本質上是組織政策問題。
6. 資源隔離以容器強制限制記憶體、CPU 等額度，以杜絕「吵鬧的鄰居」；YARN 容器與 Docker 容器概念相通但側重不同，現代 YARN 亦支援以 Docker 執行任務。
7. YARN 採「請求—回應」模式（決策權在管理器），Mesos 採「資源供給」模式（決策權在框架）；多租戶治理則以配額（上限）、保證額度（下限）、優先權與搶佔來維持秩序。
8. 資源碎片是常被忽略的浪費：每節點可放 $\lfloor R/r \rfloor$ 個容器（式 10-1），碎片率為 $(R - \lfloor R/r \rfloor \times r)/R$ （式 10-2）；容器規格宜取節點可分配資源的整除數，可將碎片率降為零。

習題

一、觀念題

1. 第一代 Hadoop 的 JobTracker 有哪兩個主要問題？YARN 如何分別解決它們？
2. ApplicationMaster 為何要設計成「每個作業一個」？這樣的設計帶來哪兩項好處？
3. 請比較 FIFO、Capacity 與 Fair 三種排程策略，並各舉一個適合的情境。

4. 何謂「吵鬧的鄰居」？為什麼這類問題特別難以除錯？該如何防範？

二、計算題

1. 某叢集每節點可分配記憶體為 96 GB。請以式 10-1 與式 10-2 計算容器規格為 8 GB、14 GB、32 GB 時的每節點容器數與碎片率，並指出哪些規格能達到零碎片。
2. 承上題，若叢集共有 120 個節點、容器規格設為 14 GB，請計算：（一）全叢集可放的容器總數；（二）全叢集浪費的記憶體總量（GB）；（三）若改為 32 GB 規格，浪費量變為多少？

三、思考與討論

1. 本章指出「排程策略的選擇本質上是組織政策問題」。請就一座校內共用叢集討論：「公平」可能有哪幾種不同定義？各自會讓誰受益、讓誰不滿？
2. 搶佔機制會強制中止低優先權的任務。請討論：這項機制成立的前提是什麼？它與第 9 章 MapReduce 「任務可重複執行」的設計有何關聯？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

兩個問題：其一，JobTracker 同時負責資源管理與 MapReduce 任務監督，負擔過重，在數千節點規模下成為擴充瓶頸；其二，資源管理與 MapReduce 綁定，導致叢集只能跑 MapReduce，無法支援 Spark 等其他框架。YARN 的解法：把資源管理獨立為通用服務層（ResourceManager 只管資源分配），並將各作業內部的任務調度下放給該作業自己的 ApplicationMaster，同時減輕主節點負擔並使多框架得以共用同一叢集（見案例一）。

第 2 題

好處一：減輕 ResourceManager 負擔——作業內部的任務調度、進度追蹤由各 ApplicationMaster 自行負責，中央只需處理資源分配，故可支撐更大規模。好處二：支援多框架——不同運算框架可各自實作專屬的 ApplicationMaster，用自己的邏輯調度任務（MapReduce 與 Spark 各有其工頭），彼此互不干擾，故同一叢集能同時服務多種框架。

第 3 題

FIFO：先提交先執行，簡單可預測，但大作業會堵塞後方小作業——適合單一使用者、作業性質相近的專用叢集。Capacity：依佇列比例保障容量並可彈性借用閒置資源，各團隊有可預期的保障——適合企業多部門共用。Fair：執行中作業動態平分資源，小作業能立即取得資源而快速完成——適合作業大小差異懸殊、需互動式查詢快速回應的環境。可引表 10-3。

第 4 題

「吵鬧的鄰居」指同一台機器（或叢集）上某個作業過度佔用資源，導致其他作業效能低落且不穩定。難以除錯的原因：病因不在受害者自己的程式或資料中，受害者無論如何檢視自身程式碼都找不出問題，且症狀時好時壞（取決於鄰居何時失控），缺乏可重現性。防範方式：啟用嚴格的容器資源限制（記憶體、CPU）以強制隔離，並以 Capacity 排程為各團隊劃分保障額度（見案例三）。

二、計算題

第 1 題

$R = 96$ GB，依式 10-1 (LR/r) 與式 10-2 (碎片率) 計算。

- (1) $r = 8$ GB: $\lfloor 96/8 \rfloor = 12$ 個；已用 96 GB，碎片 0 GB，碎片率 0% (零碎片)。
- (2) $r = 14$ GB: $\lfloor 96/14 \rfloor = 6$ 個；已用 84 GB，碎片 12 GB，碎片率 $12 \div 96 = 12.5\%$ 。
- (3) $r = 32$ GB: $\lfloor 96/32 \rfloor = 3$ 個；已用 96 GB，碎片 0 GB，碎片率 0% (零碎片)。

零碎片的規格為 8 GB 與 32 GB，因兩者皆為 96 的整除數 ($96 \div 8 = 12$ 、 $96 \div 32 = 3$)。14 GB 不整除 96，故產生 12.5% 的浪費。

第 2 題

節點數 = 120，承上題結果計算。

- (1) 容器總數 ($r = 14$ GB) $= 6 \times 120 = 720$ 個。
- (2) 浪費總量 ($r = 14$ GB) $= 12$ GB $\times 120 = 1,440$ GB。
- (3) 改為 32 GB 規格：碎片率 0%，故浪費總量 = 0 GB。

延伸思考：僅將容器規格由 14 GB 調整為 32 GB (或 8 GB)，即可回收 1,440 GB 的記憶體——相當於 15 台 96 GB 機器的容量，而這只是改一個參數的成本。這正是案例四的教訓：最有價值的優化有時只是把一個數字設對。但也須注意，容器規格還要配合單一任務的實際記憶體需求，不能只為整除而設得過小。

三、思考與討論

第 1 題

可提出的「公平」定義至少有：其一，先到先得 (FIFO)——受益者是提早提交者，不滿者是後來的小作業使用者 (急件被堵)。其二，人人均分 (Fair)——受益者是提交小作業與互動查詢者，不滿者是大型訓練作業 (完成時間被拉長)。其三，依投入分配 (Capacity，如依各研究室的計畫經費或人數劃分比例)——受益者是資源投入多的單位，不滿者是資源少但需求急迫的單位 (如專題課程)。其四，依學術優先序 (如口試在即者優先)——公平感高但難以自動化且易生爭議。周延的回答會指出：技術可執行任何政策，但無法替組織決定何謂公平，故通常採混搭 (先按單位劃分佇列，佇列內公平共享)。

第 2 題

成立前提：被搶佔的任務必須能夠被安全地中止並重新執行，且不會因中途中止而產生錯誤結果或副作用 (如重複寫入)。與第 9 章的關聯：MapReduce 的 Map 任務被設計為獨立、無狀態、可重複執行——正是這個特性使任務能被隨時中止再重跑，搶佔才成為可行的機制。若任務具有不可重複的副作用 (如已對外部系統寫入而不具冪等性)，搶佔就會造成資料錯誤。可連結第 3 章的冪等性概念，說明「無狀態與冪等」如何在容錯與資源調度兩個層面同時發揮作用。

第參篇 批次運算框架

Batch Processing Frameworks

第 11 章

Apache Spark 核心

The Core of Apache Spark

機器人叫獸（李世淵） 編著

叫獸開講

我教書時常用一個廚房的比喻，來解釋 Spark 為什麼比 MapReduce 快這麼多。

想像你要做一道要熬煮十次的湯。MapReduce 的做法是這樣：熬第一輪，熬完把整鍋湯倒進冰箱、關火、收工；要熬第二輪時，再把湯從冰箱端出來、重新加熱、熬完再倒回冰箱。如此重複十次。你會發現，真正在熬湯的時間可能不多，絕大部分時間都花在「進冰箱、出冰箱、重新加熱」。

Spark 的做法則是：湯就一直放在爐子上，熬完第一輪不熄火、不進冰箱，直接接著熬第二輪。十輪熬完了，才一次端出來。這就是「記憶體運算」的本質——資料在多輪運算之間，一直待在記憶體裡，不必反覆讀寫磁碟。

二〇〇九年前後，加州大學柏克萊分校的一群研究者，正是看準了第 9 章我們談過的那三個痛點——中間結果反覆落地、表達能力被綁在兩階段、調度開銷太重——而做出了 Spark。結果是：在需要反覆迭代的工作上（機器學習訓練、圖演算法、互動式查詢），Spark 能比 MapReduce 快上數十倍。

但我要提醒你一件重要的事：Spark 不是推翻了 MapReduce，而是站在它的肩膀上。你會在這一章看到大量熟悉的影子——依鍵分組、Shuffle、分區平行處理，全都在。差別在於，Spark 把這些思想包裝得更聰明、更有彈性。所以第 9 章你學的東西，一樣也不會白費。

學習目標

研讀完本章之後，你應該能夠：

1. 說明 Spark 的設計理念，並指出它如何回應 MapReduce 的三大限制。
2. 說明 RDD 的特性，並解釋「彈性」與「不可變」的意義。
3. 區分轉換與行動兩類運算，並說明惰性求值帶來的最佳化空間。
4. 描述 DAG 執行引擎如何把程式轉為執行計畫，並依 Shuffle 劃分階段。
5. 說明快取與持久化的時機，並判斷何時值得把資料留在記憶體。
6. 區分寬相依與窄相依，並說明 Lineage 如何在不複製資料的情況下達成容錯。

核心概念

核心概念	說明
Spark	以記憶體運算與 DAG 執行引擎為核心的分散式運算框架。
RDD	彈性分散式資料集，Spark 最基礎的資料抽象，不可變且可分區。
轉換 (Transformation)	由一個 RDD 產生新 RDD 的運算，惰性執行、不立即計算。
行動 (Action)	觸發實際計算並回傳結果或寫出資料的運算。
惰性求值	轉換不立即執行，待行動觸發時才整體規劃並計算。
DAG	有向無環圖，描述 RDD 之間相依關係的執行計畫。
Lineage (血統)	記錄 RDD 如何從父 RDD 推導而來的資訊，用於故障時重算。

表 11-1 本章核心概念一覽

11.1 Spark 的設計理念與優勢

11.1.1 針對三個痛點的三帖藥

第 9 章我們列出了 MapReduce 的三大限制。Spark 的設計，幾乎可以看成是針對它們逐條開出的藥方。

MapReduce 的限制	Spark 的解法	帶來的效益
中間結果必須落地磁碟	資料可快取於記憶體，多輪重複使用	迭代型工作可快數十倍
表達能力受限於兩階段	提供數十種轉換與行動運算子	複雜邏輯不必拆成多個作業
調度開銷重、不適合互動	DAG 引擎統一規劃、輕量調度	可支援互動式查詢

表 11-2 Spark 對 MapReduce 三大限制的回應

除了這三帖藥，Spark 還有一個常被低估的優勢：統一的引擎。在 Spark 出現前，你可能要用 MapReduce 做批次、用另一套系統做串流、再用第三套做機器學習，彼此格式不通、程式碼無法共用。Spark 則以同一套核心，支撐批次（本章）、SQL（第 12 章）、串流（第 26 章）與機器學習（第 20 章）——學一次，處處可用。這種「一站式」的整合，是它能席捲業界的另一個關鍵。

11.1.2 一個常見的誤解

初學者常聽到「Spark 是記憶體運算，所以快」，然後推論出「所以資料必須全部塞進記憶體才能用 Spark」。這是錯的。

Spark 完全可以處理遠大於記憶體的資料——當記憶體不夠時，它會自動把資料溢寫到磁碟，只是速度會下降到接近 MapReduce 的水準。所以正確的理解是：記憶體是 Spark 的加速器，而不是它的門檻。

另外還要澄清：Spark 的快也不全然來自記憶體。即使在完全不使用快取的情況下，Spark 通常仍比 MapReduce 快，因為它的 DAG 引擎能把多個運算串成一個階段執行，省去了 MapReduce 那種「每一輪都要重新啟動作業、中間都要落地」的開銷。這一點在 11.4 節會看得更清楚。

11.2 RDD：彈性分散式資料集

11.2.1 拆解這五個字

RDD（Resilient Distributed Dataset，彈性分散式資料集）是 Spark 最基礎的抽象。它的名字其實已經把特性說完了，我們逐字拆解。

- **彈性（Resilient）**：指的是容錯能力。當某個分區的資料因節點故障而遺失時，Spark 能夠自動把它「重算」回來，而不需要事先複製多份——這是 11.6 節 Lineage 的功勞。
- **分散式（Distributed）**：資料被切分成多個分區（partition），散落在叢集的各台機器上，可被平行處理。分區是 Spark 平行度的基本單位。
- **資料集（Dataset）**：它就是一批資料的集合，可以來自檔案、資料庫，或由其他 RDD 運算而來。

11.2.2 不可變：一個關鍵的設計決定

RDD 還有一個名字裡沒說、卻極其重要的特性：不可變（immutable）。一個 RDD 一旦建立，就永遠不會被修改。所有的「轉換」運算，都不是改動原本的 RDD，而是產生一個「新的」RDD。

初看之下這很浪費——改個資料還要生一份新的？但這個設計帶來了三個關鍵好處。其一，容錯變簡單：因為父 RDD 永遠不變，任何時候都可以拿它重算出子 RDD。其二，平行安全：多個任務同時讀取同一個 RDD 不會有競爭問題，因為沒有人會改它。其三，可快取：一份不會變的資料，才能安心地存在記憶體裡重複使用。

叫獸提醒

「不可變」是分散式系統中反覆出現的智慧，值得你特別留意。回想第 5 章的 HDFS——它也採「一次寫入、多次讀取」，放棄任意修改，換來簡單可靠的一致性。同樣的思路在這裡再次出現：放棄「就地修改」的方便，換來容錯、平行與快取這三項在分散式環境中更珍貴的能力。當你日後設計系統時，不妨也問問自己：這個東西真的需要可修改嗎？

11.3 轉換、行動與惰性求值

11.3.1 兩類運算

Spark 的所有運算，可以清楚地分成兩大類，理解這個區分是寫好 Spark 程式的基礎。

轉換（Transformation）是「由 RDD 產生新 RDD」的運算，例如 map（逐筆轉換）、filter（篩選）、flatMap（展開）、reduceByKey（依鍵彙整）、join（關聯）。它們的共同特徵是：回傳值仍是一個 RDD。

行動（Action）則是「觸發計算並產出結果」的運算，例如 collect（把結果收回驅動程式）、count（計數）、saveAsTextFile（寫出檔案）、first（取第一筆）。它們的共同特徵是：回傳值不再是 RDD，而是一個具體的值或副作用。

面向	轉換（Transformation）	行動（Action）
回傳	新的 RDD	具體的值或寫出結果
執行時機	惰性，不立即計算	立即觸發整條鏈的計算
代表運算	map、filter、reduceByKey、join	collect、count、save、first

表 11-3 轉換與行動的比較

11.3.2 惰性求值：先規劃，再動手

Spark 最關鍵的設計之一，是「惰性求值」（lazy evaluation）：當你寫下一連串轉換時，Spark 什麼也不做，只是默默記下「你打算做什麼」；直到你呼叫一個行動，它才回頭把整條鏈一次規劃、一次執行。

為什麼要這麼設計？因為「先看到全貌再動手」，能做出許多聰明的最佳化。舉一個最直觀的例子：假設你先做了一個 map（逐筆轉換一億筆資料），接著做 filter（只留下一百筆），最後呼叫 first（只要第一筆）。

如果每個步驟都立即執行，Spark 會老老實實轉換一億筆、篩出一百筆、然後只用了其中一筆——白做了九千九百多萬筆的工。但因為惰性求值，Spark 在看到 first 時，已經知道「你其實只要一筆」，於是它可以只處理必要的資料就提前收工。

除此之外，惰性求值還讓 Spark 能把多個窄相依的運算「合併成一個階段」連續執行，避免產生不必要的中間資料——這正是它即使不用快取也比 MapReduce 快的原因。

11.4 DAG 執行引擎與階段劃分

11.4.1 從程式碼到執行計畫

當行動被觸發時，Spark 會把你寫的那串轉換，整理成一張「有向無環圖」（DAG, Directed Acyclic Graph）——圖上的節點是 RDD，邊則是轉換運算，方向代表資料的流向，而「無環」意味著資料只會往前推進、不會繞回來。

這張 DAG 就是執行計畫。Spark 接著會分析它，決定要分成幾個階段、每個階段要開幾個任務、任務要派到哪些機器。這與 MapReduce 「一個作業就是一輪 Map 加一輪 Reduce」的僵硬模式截然不同——DAG 可以任意長、任意複雜，全部在一次規劃中完成。

11.4.2 階段是怎麼切的

Spark 把 DAG 切成若干「階段」（stage），而切割的依據只有一個：是否需要 Shuffle。

凡是不需要跨機器搬移資料的連續運算，都會被合併進「同一個階段」，在各分區上一氣呵成地執行。例如「先 map 再 filter 再 map」，這三步都是各分區自己就能完成的，於是被併成一個階段——資料在記憶體中連續流過三個運算，中間完全不落地。

而一旦遇到需要依鍵重新分組的運算（如 reduceByKey、join），資料就必須跨機器搬移，此時 Spark 就切開一個新階段。換句話說：階段的邊界，就是 Shuffle 的位置。

叫獸提醒

這裡藏著一個立刻能用的效能心法：看你的 Spark 作業有幾個階段，就大致知道它做了幾次 Shuffle。而 Shuffle——回想第 9 章——幾乎總是效能瓶頸。所以調校 Spark 的第一步，往往不是加機器，而是打開執行計畫看看：這裡面有幾次 Shuffle？有沒有哪一次是可以避免的？（例如把 join 換成廣播小表的 Map 端 join）。你會發現，第 9 章那句「調校的核心就是讓 Shuffle 變少」，在 Spark 一樣管用。

11.5 記憶體管理、快取與持久化

11.5.1 什麼時候該快取

Spark 讓你可以呼叫 cache 或 persist，把某個 RDD 留在記憶體中。但這不是免費的——記憶體有限，快取了這份就少了那份。所以關鍵問題是：什麼時候值得快取？

判斷準則其實只有一句話：當這個 RDD 會被「重複使用」時。因為惰性求值的緣故，若一個 RDD 被兩個不同的行動用到，Spark 預設會「從頭重算兩次」。這時把它快取起來，第二次就能直接取用，省下整條鏈的重算。

最典型的場景就是迭代——機器學習訓練要把同一份資料掃過一百輪，若不快取，就等於從磁碟讀一百次；快取後只需讀一次。這正是本章公式要量化的效益，也是叫獸開講那鍋湯的故事。

11.5.2 持久化的層級

`persist` 允許你選擇資料要存放在哪裡、以什麼形式存放，常見的層級有幾種取捨。

層級	存放方式	取捨
僅記憶體	以物件形式存於記憶體	最快，但佔空間，裝不下就重算
記憶體（序列化）	序列化後存於記憶體	省空間，但存取需解序列化
記憶體加磁碟	記憶體放不下的部分溢寫磁碟	較穩，不會重算但磁碟較慢
僅磁碟	全部存於磁碟	最省記憶體，速度最慢

表 11-4 常見的持久化層級與取捨

實務上的常見錯誤是「什麼都快取」——結果記憶體被塞爆，Spark 被迫頻繁地把資料換進換出，反而比不快取還慢。記住：快取是給「會被重複使用」的資料用的，用完了還可以呼叫 `unpersist` 主動釋放。

11.6 寬窄相依與 Lineage 容錯

11.6.1 兩種相依關係

RDD 之間的相依關係分為兩種，這個區分同時解釋了「階段怎麼切」與「容錯怎麼做」，是本章最核心的觀念。

- **窄相依 (Narrow Dependency)**：父 RDD 的每個分區，最多只被一個子分區使用。例如 `map`、`filter`——第 3 個分區的資料只會流向子 RDD 的第 3 個分區，不需跨機器搬移。
- **寬相依 (Wide Dependency)**：父 RDD 的一個分區，其資料會被打散到多個子分區。例如 `reduceByKey`、`join`——因為要依鍵重新分組，資料必須跨機器搬移，也就是 Shuffle。

這正是 11.4 節階段劃分的真正依據：連續的窄相依可以合併成一個階段（因為資料不必移動），而寬相依處必須切開（因為要等所有上游資料到齊才能分組）。

11.6.2 Lineage：用「重算」取代「複製」

現在來看 RDD 名字裡那個「彈性」是怎麼做到的。傳統的容錯做法是複製——像第 5 章的 HDFS 存三份副本。但複製很貴：佔三倍空間，還要花網路傳輸。

Spark 選擇了一條聰明的路：它不複製資料，而是記住「這份資料是怎麼算出來的」。這份「從哪來、經過哪些轉換」的紀錄，稱為 Lineage（血統）。當某個分區因節點故障而遺失時，Spark 只要照著血統，從它的父 RDD 重算一次即可。

這個設計之所以可行，全靠 11.2 節的「不可變」——因為父 RDD 保證不會變，所以任何時候重算都會得到相同的結果。你看，設計決策之間是環環相扣的：不可變讓 Lineage 成立，Lineage 讓容錯不必複製，不必複製又讓記憶體運算變得可行。

寬窄相依對重算成本的影響

不過，重算的代價與相依類型大有關係。若遺失的分區只有窄相依，重算很便宜——只需重算它對應的那一小段父分區即可。但若涉及寬相依，情況就麻煩了：因為子分區的資料來自多個父分區，重算它可能得把上游一大片資料全部重來，成本高昂。

這也給了我們一個實用建議：在長串的運算中，若已經跨過幾次 Shuffle，適時地做一次快取（或檢查點），能避免故障時付出昂貴的全鏈重算代價。

11.6.3 承先啟後

本章我們認識了 Spark 的核心：它以記憶體快取回應了 MapReduce 的落地之苦（11.1、11.5），以 RDD 這個不可變、可分區的抽象作為基石（11.2），以惰性求值換取全局最佳化的空間（11.3），以 DAG 引擎統一規劃並依 Shuffle 劃分階段（11.4），並以寬窄相依與 Lineage，達成了「不複製也能容錯」的巧妙設計（11.6）。

但 RDD 有一個侷限：它對 Spark 而言只是「一堆物件」，Spark 並不知道裡面裝的是什麼結構。這意味著 Spark 無法替你做太多最佳化——它不知道你的資料有哪些欄位、也不知道你的 filter 條件可以提前套用。如果 Spark 能「看懂」資料的結構，它是不是就能像資料庫的查詢最佳化器一樣，幫你將程式改寫得更快？答案是肯定的，這就是下一章的主題：Spark SQL、DataFrame 與 Dataset，以及那個會自動幫你改寫查詢的 Catalyst 最佳化器。

公式與流程：迭代型工作的加速比

叫獸開講說「湯不進冰箱」能快很多，這一節我們把它算出來——你會看到，加速倍數取決於迭代幾輪，且有一個明確的上限。

設一個需要迭代 N 輪的工作，每輪的實際計算時間為 c ，而每輪從磁碟讀取與寫回的 I/O 時間為 d 。在 MapReduce 模式下，每一輪都必須重新讀入、算完再寫出，故總時間為：

$$T(\text{MapReduce}) = N \times (c + d)$$

式 11-1 磁碟式迭代的總時間

在 Spark 模式下，資料在第一輪讀入後即快取於記憶體，後續各輪直接沿用，故僅需付出一次 I/O 成本：

$$T(\text{Spark}) = d + N \times c$$

式 11-2 記憶體式迭代的總時間

兩者相除，可得加速比：

$$\text{加速比} = N(c + d) / (d + Nc)$$

式 11-3 Spark 相對於 MapReduce 的加速比

我們以一個具體情境試算：每輪計算 10 秒（ $c = 10$ ）、每輪磁碟 I/O 100 秒（ $d = 100$ ）。

迭代輪數 N	MapReduce 總時間	Spark 總時間	加速比
1	110 秒	110 秒	1.0 倍（無差異）
10	1,100 秒	200 秒	5.5 倍
50	5,500 秒	600 秒	約 9.2 倍
100	11,000 秒	1,100 秒	10 倍

迭代輪數 N	MapReduce 總時間	Spark 總時間	加速比
$N \rightarrow \infty$	—	—	11 倍 (上限)

表 11-5 不同迭代輪數下的加速比 ($c = 10$ 秒、 $d = 100$ 秒)

這張表有兩個值得記住的結論。第一，只跑一輪時 Spark 毫無優勢——因為那一次 I/O 誰都躲不掉。第二，隨著迭代輪數增加，加速比逐漸逼近一個上限：當 N 趨近無限大時，加速比趨近 $(c + d) / c = 110 / 10 = 11$ 倍。

叫獸提醒

看到「趨近一個上限」有沒有覺得眼熟？這正是第 1 章阿姆達爾定律的同一個道理——你優化掉了 I/O 這部分，剩下那個沒被優化的計算時間 c ，就成了新的天花板。這也給了你一個實用的判斷準則：如果你的工作只跑一兩輪、或者計算本身就遠比 I/O 耗時（ c 遠大於 d ），那麼硬要用 Spark 快取並不會有多大幫助。優化前先想清楚瓶頸在哪，這句話在整本書都適用。

案例研討

案例一 那個跑不動的機器學習訓練

回到第 9 章案例四：某研究團隊用 MapReduce 訓練一個需迭代一百次的模型，每輪都要把訓練資料從磁碟讀出、算完寫回，光是 I/O 就佔了絕大部分時間。

他們改用 Spark 後，把訓練資料以 cache 留在記憶體中，一百輪迭代只需在第一輪讀取一次磁碟。依式 11-3，若原本每輪 I/O 佔九成時間，加速比可達近十倍——原本要跑三小時的訓練，二十分鐘就結束了。這個案例是 Spark 誕生動機最直接的體現，也解釋了為何機器學習與圖演算法（如第 18 章的 PageRank，同樣需要反覆迭代）成為 Spark 的主場。

案例二 被遺忘的一個 cache

某工程師寫了一支 Spark 程式：先經過一連串複雜的清理與轉換得到一個 RDD，接著對它做了三件事——計算筆數、統計平均、以及寫出檔案（三個行動）。他發現程式跑得異常地慢，遠超預期。

原因正是 11.5.1 節的陷阱：因為惰性求值，這三個行動各自觸發了一次完整的計算——那一連串昂貴的清理與轉換，被從頭做了三遍。他只加了一行 cache，把中間那個 RDD 快取起來，執行時間立刻降為約三分之一。

這個案例是新手最常踩的坑：惰性求值很聰明，但它的代價是「你以為算過了，其實沒有」。判斷準則很簡單——只要一個 RDD 會被多個行動用到，就該考慮快取。

案例三 從十個階段減到三個

某團隊的 Spark 作業執行緩慢，他們打開 Spark 的網頁介面查看執行計畫，發現這支作業竟被切成了十個階段——也就是做了九次 Shuffle。

檢視程式碼後，他們做了幾項調整：把幾個各自 groupByKey 再聚合的操作，改為單次的 reduceByKey（讓局部彙整先在 Map 端完成，這其實就是第 9 章 Combiner 的精神）；並把與一張小表的 join 改為廣播該小表，讓配對在各分區本地完成，完全免去 Shuffle。調整後階段數從十個降到三個，執行時間縮短逾七成。

這個案例印證了 11.4.2 節的心法：階段數就是 Shuffle 數的指標。調校 Spark 時，先看執行計畫有幾個階段，往往比盲目加機器有效得多。

案例四 節點掛了，但作業沒死

一支 Spark 作業執行到一半，叢集中一台機器因硬體故障而下線，該機器上正在處理的數個分區資料隨之遺失。

若是傳統做法，這意味著整個作業失敗、必須從頭重跑。但 Spark 依照 Lineage 血統，得知這些遺失的分區是「由某個父 RDD 經過 map 與 filter 而來」，於是它只在其他健康的機器上，把那幾個分區重算一遍，作業便繼續往下跑，其餘完成的部分完全不受影響。

這個案例展現了 11.6.2 節的精髓：Spark 沒有為容錯付出「複製三份」的空間代價，卻依然扛住了節點故障——它付出的只是「記住怎麼算出來的」這點微不足道的中繼資料。這正是第 3 章「容錯不讓局部故障演變成整體失效」在 Spark 上的優雅實踐。

實作活動

活動一 觀察惰性求值

請以 PySpark 親眼驗證惰性求值的行為。

- 步驟 1.** 建立一個 RDD，串接數個轉換（如 map、filter），觀察此時是否有任何計算發生（可留意執行時間）。
- 步驟 2.** 呼叫一個行動（如 count），觀察計算是否此時才被觸發。
- 步驟 3.** 在轉換的函式中加入 print 敘述，觀察它們何時被印出。
- 步驟 4.** 討論：若把 first 換成 collect，Spark 的處理量會有什麼不同？為什麼？

活動二 測量 cache 的效益

請設計一個實驗，量化快取帶來的加速。

- 步驟 1.** 準備一個較大的資料集，經過數個較耗時的轉換後得到一個 RDD。
- 步驟 2.** 在「不快取」的情況下，對它連續執行三個行動，記錄總時間。
- 步驟 3.** 加上 cache 後重複同樣的三個行動，記錄總時間並計算加速倍數。
- 步驟 4.** 對照式 11-3 的思路，說明你觀察到的加速是否符合「重複使用次數越多、效益越大」的預期。

活動三 判讀執行計畫

請撰寫一支包含 map、filter、reduceByKey 與 join 的 PySpark 程式，執行後打開 Spark 的網頁介面（預設為 4040 埠）查看該作業的 DAG 視覺化圖。請回答：這支作業被切成幾個階段？階段的邊界分別出現在哪些運算之後？這些位置是否都對應到寬相依？最後，試著調整程式（例如改用廣播 join），觀察階段數是否減少。

延伸閱讀

- **RDD 原始論文：**Zaharia 等人發表的《Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing》，完整闡述 RDD、Lineage 與寬窄相依的設計，是本章最重要的第一手資料。

- 《Learning Spark》第 2 版第 1 至 4 章：Damji 等人著。以大量範例介紹 Spark 核心概念與 RDD 操作，是本章的實作補充。
- 《Spark: The Definitive Guide》：Chambers 與 Zaharia 著，對執行計畫、階段劃分與效能調校有深入討論，適合 11.4 節的延伸。
- Apache Spark 官程式設計指南：說明各種轉換與行動運算子、持久化層級與參數設定，是實作時的常備參考。

本章小結

1. Spark 針對 MapReduce 的三大限制提出解方：以記憶體快取取代反覆落地、以豐富運算子取代僵硬的兩階段、以 DAG 引擎的輕量調度支援互動查詢；此外它還以同一套核心統一了批次、SQL、串流與機器學習。
2. 「記憶體運算」不代表資料必須全部裝進記憶體——記憶體不足時 Spark 會自動溢寫磁碟；且 Spark 的快也不全然來自記憶體，DAG 引擎將多個運算合併執行同樣貢獻良多。
3. RDD 是彈性（可依血統重算而容錯）、分散式（切為多個分區平行處理）的資料集，且具備「不可變」特性——不可變使容錯、平行安全與快取三者同時成為可能。
4. 運算分為轉換（回傳新 RDD、惰性）與行動（觸發計算、回傳結果）兩類；惰性求值讓 Spark 能在看到全貌後才規劃執行，得以做出跳過無用計算、合併連續運算等最佳化。
5. 行動觸發時，Spark 將轉換鏈整理為 DAG 執行計畫，並以「是否需要 Shuffle」為唯一依據劃分階段：連續的窄相依合併為同一階段，寬相依處則切開。故階段數即為 Shuffle 次數的指標。
6. 快取的判準是「該 RDD 是否會被重複使用」；持久化層級在速度、空間與穩定性之間取捨。過度快取會擠爆記憶體而適得其反，用畢可主動釋放。
7. 窄相依指父分區最多被一個子分區使用（如 map、filter），寬相依指資料被打散到多個子分區（如 reduceByKey、join）；Lineage 記錄推導過程，使 Spark 能以「重算」取代「複製」來容錯，而這正建立在 RDD 不可變的基礎上。
8. 迭代工作的加速比為 $N(c+d)/(d+Nc)$ （式 11-3）：僅跑一輪時毫無優勢，輪數越多效益越大，但終將趨近上限 $(c+d)/c$ ——這是阿姆達爾定律的又一次體現。

習題

一、觀念題

1. 「使用 Spark 時，資料必須全部放得進記憶體」——請指出這句話的錯誤，並說明記憶體對 Spark 的正確定位。
2. RDD 的「不可變」特性帶來哪三項好處？請分別說明其原因。
3. 請說明惰性求值的運作方式，並舉一個例子說明它能帶來什麼最佳化。
4. 請區分寬相依與窄相依，並說明它們如何同時決定「階段劃分」與「重算成本」。

二、計算題

1. 某迭代工作每輪計算時間 $c = 20$ 秒、每輪磁碟 I/O 時間 $d = 80$ 秒。請以式 11-1 至式 11-3 計算 $N = 1、10、40$ 時的 MapReduce 總時間、Spark 總時間與加速比，並求出 N 趨近無限大時的加速比上限。
2. 承上題，若透過程式優化把每輪計算時間 c 由 20 秒降為 10 秒（I/O 不變），請重新計算 $N = 40$ 時的 Spark 總時間與加速比上限，並說明為何「優化計算」反而使加速比上限提高。

三、思考與討論

1. Spark 以 Lineage「重算」取代 HDFS 的「複製」來達成容錯。請比較這兩種容錯策略各自的成本與適用場景，並說明為何 Spark 能夠選擇重算。
2. 案例二中，工程師因未快取而讓昂貴的轉換被重算三次。請討論：既然快取這麼有用，為何 Spark 不乾脆自動把所有中間結果都快取起來？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

錯誤在於把記憶體視為使用門檻。事實上 Spark 可處理遠大於記憶體的資料——當記憶體不足時會自動將資料溢寫至磁碟，僅是速度下降至接近 MapReduce 的水準。正確定位：記憶體是 Spark 的「加速器」而非「門檻」。並可補充：Spark 的效能優勢也不全來自記憶體，其 DAG 引擎能將多個窄相依運算合併為一個階段連續執行，省去 MapReduce 每輪重啟作業與中間落地的開銷，故即使不使用快取通常仍較快。

第 2 題

其一，容錯變簡單——父 RDD 永不改變，故任何時候都能依血統重算出子 RDD，結果保證一致。其二，平行安全——多個任務同時讀取同一 RDD 不會有寫入競爭問題，因為沒有人會修改它。其三，可快取——唯有不會變動的資料，才能安心地長期存放於記憶體中重複使用。可補充：此思路與第 5 章 HDFS「一次寫入、多次讀取」一脈相承，皆是放棄就地修改以換取容錯、平行與快取。

第 3 題

運作方式：轉換運算不立即執行，Spark 僅記錄「打算做什麼」；直到行動被呼叫，才回頭將整條轉換鏈一次規劃並執行。最佳化範例：若先 map 一億筆、再 filter 剩一百筆、最後只呼叫 first（僅要一筆），立即執行模式會白做九千九百多萬筆的轉換；惰性求值下，Spark 已知最終只需一筆，故可只處理必要資料即提前結束。另一例是把多個連續的窄相依運算合併為同一階段執行，避免產生不必要的中間資料。

第 4 題

窄相依：父 RDD 的每個分區最多只被一個子分區使用（如 map、filter），資料不需跨機器搬移。寬相依：父分區的資料被打散到多個子分區（如 reduceByKey、join），須依鍵重新分組而產生 Shuffle。對階段劃分：連續窄相依可合併為同一階段（資料不必移動），寬相依處必須切開（須等上游資料到齊）。對重算成本：窄相依重算便宜（只需重算對應的少數父分區）；寬相依重算昂貴（子分區資料來自多個父分區，可能須重算上游一大片），故長鏈中宜適時快取或設檢查點。

二、計算題

第 1 題

$c = 20$ 、 $d = 80$ ，依式 11-1 ($N(c+d)$) 與式 11-2 ($d+Nc$) 計算。

- (1) $N = 1$: $MR = 1 \times 100 = 100$ 秒；Spark $= 80 + 20 = 100$ 秒；加速比 $= 1.0$ 倍（無差異）。

- (2) $N = 10$: $MR = 10 \times 100 = 1,000$ 秒; $Spark = 80 + 200 = 280$ 秒; 加速比 ≈ 3.57 倍。
- (3) $N = 40$: $MR = 40 \times 100 = 4,000$ 秒; $Spark = 80 + 800 = 880$ 秒; 加速比 ≈ 4.55 倍。
- (4) 上限: $N \rightarrow \infty$ 時, 加速比 $\rightarrow (c+d)/c = 100/20 = 5$ 倍。

延伸思考: $N = 40$ 時的 4.55 倍已相當接近上限 5 倍, 再增加輪數效益有限——這與表 11-5 的趨勢一致。

第 2 題

c 由 20 降為 10, d 仍為 80。

- (1) $N = 40$ 時的 Spark 總時間 $= 80 + 40 \times 10 = 480$ 秒 (原為 880 秒)。
- (2) 加速比上限 $= (c+d)/c = (10+80)/10 = 9$ 倍 (原為 5 倍)。

為何優化計算反而提高加速比上限: 加速比上限的意義是「I/O 佔總工作量的比重」。當計算時間 c 縮短, I/O (d) 在單輪工作中所佔的比例就相對上升, 因此「省下重複 I/O」這件事的價值也隨之放大。換言之, c 越小, Spark 免除重複 I/O 的優勢越顯著。反之若 c 遠大於 d (計算極耗時而 I/O 微不足道), 則快取的效益就很有限——這正是叫獸提醒中「優化前先想清楚瓶頸在哪」的量化說明。

三、思考與討論

第 1 題

複製 (HDFS): 成本是三倍儲存空間與副本間的網路傳輸, 但故障時可立即取用其他副本、恢復極快, 且適用於「原始資料」——因為原始資料無法被重算出來, 必須實際保存。重算 (Spark Lineage): 成本僅是記錄推導過程的少量中繼資料, 極為輕量, 但故障時需花時間重算, 若涉及寬相依則代價高昂; 適用於「由其他資料推導而來的中間結果」。Spark 能選擇重算的關鍵前提有二: 其一, RDD 不可變, 故重算必得相同結果; 其二, 其處理的多為可重新推導的中間資料, 而非唯一的原始資料來源。周延的回答會指出兩者互補——Spark 的原始輸入通常仍存放在有複製機制的 HDFS 或物件儲存上。

第 2 題

因為快取並非免費, 自動全部快取會帶來多重問題: 其一, 記憶體有限——中間結果往往遠大於可用記憶體, 全部快取將導致頻繁換進換出 (thrashing), 反而比不快取更慢。其二, 多數中間 RDD 只被使用一次, 快取它們毫無效益卻佔用空間, 還會排擠真正需要快取的資料。其三, Spark 無法預知使用者後續會呼叫哪些行動, 難以自動判斷哪些 RDD 值得保留。因此設計上把決定權交給開發者, 由最清楚存取模式的人來判斷。可補充: Spark SQL (第 12 章) 因為「看得懂」資料結構與查詢意圖, 得以做出更多自動最佳化——這正是下一章要談的方向。

Batch Processing Frameworks

第 12 章

Spark SQL、DataFrame 與 Dataset

Spark SQL, DataFrame, and Dataset

機器人叫獸（李世淵） 編著

叫獸開講

我先講一個計程車的故事。

假設你搭上一台計程車，跟司機說：「往前開三個路口，左轉，再直走兩百公尺，右轉，然後迴轉，接著……」你一步一步地下指令，司機一步一步地照做。他不知道你要去哪裡，所以就算他知道有一條更近的路，也幫不上忙——因為你只告訴他「怎麼做」，沒告訴他「要去哪」。

換一種說法：「請載我到火車站。」這下司機立刻就能運用他的專業——他知道哪條路在塞車、哪裡在施工、哪條巷子可以抄近路，於是他替你選了一條最快的路線。你只說了「要什麼」，把「怎麼做」交給更懂的人。

上一章的 RDD，就是第一種說法。你告訴 Spark：先 map、再 filter、再 reduceByKey。Spark 老老實實照做，但它其實幫不上什麼忙——因為 RDD 對它而言就是「一堆看不透的物件」，它不知道裡面有哪些欄位、不知道你的 filter 在篩什麼，自然無從最佳化。

而這一章的 DataFrame 與 Spark SQL，是第二種說法。當你寫下「請給我業績大於一萬的訂單的總金額」，Spark 就「看懂」了你的意圖，於是它體內那個叫 Catalyst 的最佳化器就會開始動腦筋：這個篩選條件可以提前到讀檔案時就套用嗎？這些欄位其實根本不用讀吧？這兩個步驟可以合併嗎？結果往往是——你寫的程式比較短，跑起來卻比你自已手工優化的 RDD 還快。

學習目標

研讀完本章之後，你應該能夠：

1. 說明結構化 API 的價值，並比較 RDD 與 DataFrame 的差異。
2. 區分 DataFrame 與 Dataset，並說明型別安全的取捨。
3. 使用 Spark SQL 與 DataFrame API 完成常見的資料查詢與彙整。
4. 描述 Catalyst 最佳化器的運作階段，並說明述詞下推與欄位裁剪。
5. 說明 Tungsten 執行引擎如何從記憶體與程式碼層面提升效能。
6. 判斷 UDF 的使用時機與代價，並掌握常見的效能調校原則。

核心概念

核心概念	說明
結構化 API	讓 Spark 知道資料綱要（欄位與型別）的高階介面，含 DataFrame、Dataset 與 SQL。
DataFrame	具備綱要的分散式資料表，等同於 Dataset[Row]。
Dataset	具備綱要且在編譯時期做型別檢查的資料集（限 Scala/Java）。
Catalyst	Spark SQL 的查詢最佳化器，負責把查詢改寫得更有效率。
述詞下推	把篩選條件提前到資料來源端執行，以減少讀取量。

核心概念	說明
欄位裁剪	只讀取查詢實際用到的欄位，跳過其餘欄位。
Tungsten	Spark 的執行引擎，以堆外記憶體與程式碼生成提升執行效率。

表 12-1 本章核心概念一覽

12.1 結構化 API 概觀

12.1.1 從「怎麼做」到「要什麼」

承接叫獸開講的比喻，這裡我們把它講成技術語言。RDD 是「命令式」（imperative）的介面——你逐步指定每一個運算動作；DataFrame 與 SQL 則是「宣告式」（declarative）的介面——你描述想要的結果，由系統決定如何達成。

宣告式之所以能更快，關鍵在於「資訊」。當你用 RDD 寫下一個 filter，你傳給 Spark 的是一個函式，Spark 只能把它當成黑箱去執行；但當你用 DataFrame 寫下同樣的篩選條件，Spark 拿到的是一個「可以被分析的運算式」——它看得懂這是在比較某個欄位與某個常數，於是它可以把這個條件提前、下推到資料來源，甚至根本不必讀取整個檔案。

這是一個很值得記住的通則：你給系統的資訊越多，系統能替你做的最佳化就越多。這也解釋了為何本書一路走來，從 MapReduce 的兩階段、到 RDD 的 DAG、再到 DataFrame 的查詢計畫，抽象層次越來越高——每一次提高，都是為了讓框架「知道得更多」。

12.1.2 三種結構化 API

Spark 的結構化 API 有三種面貌，但它們底層走的是同一條路。

API	特性	型別檢查	適用語言
SQL	以標準 SQL 語法查詢	執行時期	全部
DataFrame	以程式化 API 操作有綱要的資料表	執行時期	全部（含 Python）
Dataset	具備綱要且編譯期型別安全	編譯時期	Scala、Java

表 12-2 三種結構化 API 的比較

關鍵在於：無論你用哪一種寫，它們最終都會被轉成同一份查詢計畫，交給同一個 Catalyst 最佳化器處理。因此效能幾乎沒有差別——選哪一種，取決於你偏好 SQL 的直觀、程式化 API 的靈活，還是編譯期型別檢查的安全。

叫獸提醒

這對用 Python 的同學是個好消息。在 RDD 時代，PySpark 有個惡名昭彰的效能問題：Python 寫的函式必須在 Python 的行程中執行，資料要在 JVM 與 Python 之間來回搬運與序列化，慢得令人絕望。但改用 DataFrame 後，你寫的運算式會被轉成查詢計畫，實際執行是在 JVM 裡完成的——Python 只是「下訂單」，不「下廚」。所以同樣的邏輯，用 DataFrame 寫的 PySpark 可能比用 RDD 寫的快好幾倍。這是本章最實用的一個結論。

12.2 DataFrame 與 Dataset

12.2.1 DataFrame：帶著綱要的 RDD

DataFrame 可以理解為「一張分散式的資料表」：它像 RDD 一樣被切分成多個分區、散布在叢集上，但額外帶著一份綱要（schema）——明確記載有哪些欄位、各是什麼型別。

這份綱要正是一切最佳化的來源。有了它，Spark 才能在你寫錯欄位名時立刻報錯（而不是跑了半小時才失敗）、才能只讀取用得到的欄位、才能把資料以更緊湊的二進位格式存放（而非笨重的 JVM 物件）。

綱要可以由 Spark 自動推斷（例如讀取 Parquet 時直接取自檔案的中繼資料，回想第 4 章——Parquet 本身就自帶綱要），也可以由你明確指定。實務上，讀取 CSV 或 JSON 這類不含綱要的格式時，明確指定綱要通常更快也更安全，因為自動推斷需要先掃過一部分資料。

12.2.2 Dataset：多一層編譯期的保護

Dataset 在 DataFrame 之上再加一層：型別安全。使用 DataFrame 時，資料的每一列是通用的 Row 物件，欄位以字串名稱存取——若你打錯欄位名或用錯型別，要到執行時期才會發現。Dataset 則讓你把每一列對應到一個具體的類別，編譯器在編譯階段就能替你檢查。

代價是它只在 Scala 與 Java 中可用——因為 Python 是動態型別語言，沒有編譯期型別檢查可言。所以對使用 PySpark 的同學來說，DataFrame 就是你的主力工具。

值得一提的是，在 Spark 的內部實作中，DataFrame 其實就是 Dataset[Row]——兩者本是同源，只是型別參數不同。理解這一點，你就不會被這兩個名詞搞混。

12.3 Spark SQL 與資料來源

12.3.1 用 SQL 查詢分散式資料

Spark SQL 讓你能直接以標準 SQL 語法，查詢分散在上百台機器上的資料。做法是先把 DataFrame 註冊成一張「暫存檢視表」，之後就能像操作一般資料庫一樣下 SQL 查詢。

這件事的意義比表面上更大。它意味著大數據分析的門檻被大幅降低——不會寫 Scala、不懂 RDD 的資料分析師與業務人員，只要會 SQL，就能直接分析 PB 等級的資料。這是 Spark 能在企業中普及的關鍵之一。

而且，SQL 與 DataFrame API 可以自由混用：你可以先用 DataFrame API 做一些程式化的前處理，註冊成檢視表後用 SQL 查詢，再把結果接回 DataFrame 繼續處理。因為兩者都會被轉成同一份查詢計畫，混用完全不會有效能損失。

12.3.2 資料來源與格式的選擇

Spark 能讀寫多種資料來源——檔案系統（HDFS、物件儲存）、資料庫（透過 JDBC）、以及各種檔案格式。而格式的選擇，直接決定了最佳化能發揮多少威力。

格式	自帶綱要	支援欄位裁剪	支援述詞下推
CSV	否	有限	否
JSON	否（需推斷）	有限	否

格式	自帶綱要	支援欄位裁剪	支援述詞下推
Parquet	是	是	是
ORC	是	是	是

表 12-3 常見資料格式對最佳化的支援

這張表把第 4 章與本章串了起來：欄式格式（Parquet、ORC）之所以是大數據分析的標準選擇，不只因為它壓縮率高，更因為它能配合 Catalyst 做欄位裁剪與述詞下推——最佳化器再聰明，若資料格式不配合，也施展不開。這是「軟體最佳化」與「資料佈局」必須相互配合的典型例子。

12.4 Catalyst 查詢最佳化器

Catalyst 是 Spark SQL 的心臟，也是本章最核心的內容。它的工作是：把你寫的查詢，改寫成一個效果相同、但執行起來快得多的版本。

12.4.1 四個階段

一份查詢從你寫下到真正執行，會經過 Catalyst 的四個階段。

第 1 階段 解析： 把 SQL 或 DataFrame 運算轉為「未解析的邏輯計畫」，此時只知道語法結構，還不知道欄位是否真的存在。

第 2 階段 邏輯最佳化： 查詢綱要目錄以確認欄位與型別，接著套用一系列規則改寫計畫——述詞下推、欄位裁剪、常數摺疊等，都在這個階段發生。

第 3 階段 實體計畫： 為邏輯計畫產生數個可能的執行方式（例如 join 該用廣播還是 Shuffle），並依成本模型挑出最好的一個。

第 4 階段 程式碼生成： 把最終計畫編譯成高效的 Java 位元組碼，交由 Tungsten 執行。

這個流程與傳統資料庫的查詢最佳化器如出一轍——Spark 等於把數十年來資料庫領域累積的最佳化智慧，搬到了分散式運算上。

12.4.2 兩個最重要的最佳化

Catalyst 有數十條最佳化規則，但對效能影響最大的通常是這兩個，也是本章公式要量化的對象

述詞下推 (Predicate Pushdown)

你寫的篩選條件，若寫在查詢的後段，Catalyst 會設法把它「往前推」，越靠近資料來源越好。理想情況下，它會被推到讀檔階段——讓 Parquet 檔案憑著內部的統計資訊，直接跳過那些「整個區塊都不可能符合條件」的資料，連讀都不讀。

舉例來說，若你查詢「2026 年的訂單」，而 Parquet 檔案的每個資料區塊都記錄了該區塊中日期的最大最小值，那麼所有最大日期小於 2026 年的區塊，就能被整塊跳過。原本要掃十億筆，可能只需讀一億筆。

欄位裁剪 (Column Pruning)

這一項我們在第 4 章已經算過：若一張表有二十欄，而你的查詢只用到三欄，欄式儲存讓 Spark 只讀那三欄。Catalyst 的工作，就是分析整份查詢計畫、判斷出「到底哪些欄位是真正需要的」，然後只讀它們。

有趣的是，這項最佳化常常能省下你自己都沒意識到的浪費——例如你讀進一張寬表、做了一連串轉換、最後只 select 兩個欄位輸出，Catalyst 會回頭發現「其實從一開始就只需要讀那兩欄」。這正是宣告式介面的威力：它能做全局的分析，而你逐步下指令時往往看不到全局。

12.5 Tungsten 執行引擎

12.5.1 從最佳化計畫到高效執行

Catalyst 負責「決定要怎麼做」，而 Tungsten 負責「把它做得快」。它從三個層面榨出效能，這三項都與硬體的特性密切相關。

- **堆外記憶體與二進位格式：**把資料以緊湊的二進位形式存放在 JVM 堆外，避免龐大的 JVM 物件開銷，也大幅減輕垃圾回收的負擔。這是可能的，正因為 DataFrame 知道綱要——知道型別，才能決定二進位佈局。
- **快取感知的運算：**設計資料結構與演算法時，刻意配合處理器快取的特性，讓常用資料盡量留在快取中，減少存取主記憶體的次數。
- **全階段程式碼生成：**把一個階段中的多個運算，動態編譯成一支專用的程式碼，讓資料在暫存器層級連續流過，而非逐一呼叫各個運算子。

12.5.2 全階段程式碼生成的巧思

第三項值得多說幾句，因為它的思路很有啟發性。傳統的查詢執行方式，是每個運算子（篩選、投影、聚合）各自實作，資料像水流過一道道關卡，每經過一道就要呼叫一次函式、產生一次中間結果。

Tungsten 的做法則是：既然在編譯期就知道這個階段要做哪些運算，何不直接生成一段「把這些運算全部寫在一起」的程式碼？如此一來，資料只需被讀取一次，就在一個緊湊的迴圈中完成所有運算，中間不必產生任何物件、也不必反覆呼叫函式。

這個做法帶來的效果，往往是數倍的提升。它也再次印證本章的主題——因為 Spark「知道」你要做什麼（拜結構化 API 之賜），它才有機會做這種層級的最佳化。若是 RDD 那種黑箱函式，Spark 連生成什麼程式碼都不知道。

12.6 UDF 與效能調校

12.6.1 UDF：便利與代價

當內建函式滿足不了需求時，你可以撰寫自訂函式（UDF, User-Defined Function）。它很方便，但有一個必須認清的代價：對 Catalyst 而言，UDF 是一個黑箱。

這意味著所有的最佳化都在 UDF 前面止步——Catalyst 無法把篩選條件推到 UDF 之後、無法分析它用到哪些欄位、也無法把它納入程式碼生成。你等於在一條最佳化良好的高速公路中間，設了一個要下車步行的路段。

在 PySpark 中代價更大：Python 的 UDF 必須把資料從 JVM 序列化、送到 Python 行程執行、再序列化送回來，這個來回的開銷極為可觀。因此實務上的原則是：能用內建函式解決的，絕不寫 UDF；非寫不可時，優先考慮向量化的 UDF（一次處理一整批資料而非逐筆），可大幅降低往返開銷。

12.6.2 常見的調校原則

結合本章與第 11 章的內容，這裡整理幾條實用的調校原則。

- **優先使用結構化 API：**同樣的邏輯，DataFrame 通常比 RDD 快，且在 PySpark 中差距尤其明顯。
- **用欄式格式儲存分析資料：**Parquet 讓述詞下推與欄位裁剪得以生效，這是最佳化的前提。
- **盡早篩選、只選需要的欄位：**雖然 Catalyst 會自動最佳化，但明確地寫出來仍有助於可讀性與確保效果。
- **善用廣播 join：**與小表 join 時廣播該表，可完全避免 Shuffle（呼應第 9 章的 Map 端 Join）。
- **少寫 UDF：**UDF 是最佳化的黑洞；非用不可時採向量化版本。
- **查看執行計畫：**以 explain 檢視實際的實體計畫，確認述詞下推與廣播是否真的生效。

叫獸提醒

最後這一條「查看執行計畫」是我最想強調的習慣。很多同學調校時全憑猜測，改了半天不知道有沒有效。其實 Spark 早就準備好了答案——呼叫 explain，它會把最終的執行計畫攤開給你看：述詞有沒有被下推、join 用的是廣播還是 Shuffle、讀了哪些欄位，一目瞭然。這就像醫生看 X 光片，別靠望聞問切猜病因。養成這個習慣，你的調校功力會突飛猛進。

12.6.3 承先啟後

本章我們見證了一次抽象層次的躍升：從 RDD 那種「一步步告訴 Spark 怎麼做」的命令式，走向 DataFrame 與 SQL 那種「告訴 Spark 要什麼」的宣告式（12.1）。因為 Spark 得以「看懂」資料的綱要與查詢的意圖（12.2、12.3），Catalyst 才能施展述詞下推與欄位裁剪等最佳化（12.4），Tungsten 才能以二進位格式與程式碼生成榨出硬體效能（12.5）——而 UDF 之所以昂貴，正是因為它把這扇「看懂」的窗戶關上了（12.6）。

至此，我們已經掌握了 Spark 的核心與結構化 API。但在真實的專案中，資料處理很少是「寫一支程式跑一次」那麼單純——它通常是一條每天定時執行、由數十個步驟串接而成的流水線：先從各個來源擷取資料、清理、轉換、彙整、再載入到目的地，而且中途任何一步失敗都要能重試與告警。要如何設計、編排並維護這樣一條資料管線？這就是第參篇最後一章，也就是下一章的主題。

公式與流程：述詞下推與欄位裁剪的加乘效果

Catalyst 的兩大最佳化——述詞下推與欄位裁剪——各自能省下多少？兩者合起來又如何？我們把它算清楚。

設一張表共有 C 個欄位、 N 筆資料。查詢只用到 k 個欄位，且篩選條件的選擇率為 s （即符合條件的資料佔全體的比例， $0 < s \leq 1$ ）。欄位裁剪讓讀取的欄位數由 C 降為 k ，述詞下推讓讀取的筆數由 N 降為約 $s \times N$ 。故實際讀取量相對於「全表掃描」的比值為：

$$\text{讀取比} = (k / C) \times s$$

式 12-1 欄位裁剪與述詞下推的合併讀取比

節省比例則為：

$$\text{節省比例} = 1 - (k / C) \times s$$

式 12-2 合併最佳化的節省比例

關鍵在於兩者是「相乘」而非「相加」的關係——這使得效果呈現加乘放大。我們以一張 40 欄的表為例，看不同情況下的讀取量（以全表掃描為 100%）。

查詢欄數 k	選擇率 s	僅欄位裁剪	僅述詞下推	兩者合併
4	100%（無篩選）	10%	100%	10%
4	10%	10%	10%	1%
4	1%	10%	1%	0.1%
20	10%	50%	10%	5%

表 12-4 40 欄表在不同條件下的讀取量（全表掃描＝100%）

看第三列：只用 4 個欄位、篩選出 1% 的資料，合併後的讀取量僅為全表的千分之一——等於原本要讀 1 TB 的查詢，實際只需讀 1 GB。這解釋了為何同一份資料、同一個問題，寫成 DataFrame 查詢可能比手工的 RDD 程式快上百倍。

叫獸提醒

但請務必記住式 12-1 成立的前提：資料必須以支援這兩項最佳化的格式儲存（回看表 12-3）。若你的資料是 CSV，述詞下推無從發生、欄位裁剪也極有限——最佳化器再聰明，也不可能在不讀取的情況下知道哪些列符合條件。所以這條公式真正的教訓是：Catalyst 的威力有一半掌握在「你當初選了什麼格式存資料」這個決定上。第 4 章講欄式儲存時我說 Parquet 不是龜毛而是常識，這裡就是它的兌現。

案例研討

案例一 一行 SQL 打敗兩百行 RDD

某團隊有一支跑了兩年的 RDD 程式，約兩百行，負責每日的訂單彙整。一位新進工程師把它改寫成約十行的 DataFrame 查詢，結果不只程式碼短了二十倍，執行時間還縮短了逾六成。

資深工程師起初不服氣——他當年可是手工做了不少優化。但檢視執行計畫後真相大白：Catalyst 自動把篩選條件下推到了讀檔階段（原本的 RDD 版本是先讀完整份資料才篩選），並且只讀取查詢用到的六個欄位（原本讀了全部三十多欄）。依式 12-1，光這兩項就帶來近十倍的讀取量縮減。

這個案例的教訓是：在結構化資料的處理上，「相信最佳化器」通常勝過「自己手工優化」。人能想到的優化有限，而 Catalyst 會不厭其煩地套用數十條規則、還會依成本模型挑選最佳方案。

案例二 把 CSV 換成 Parquet 之後

某資料湖裡存放著以 CSV 格式儲存的感測資料，每日新增數十 GB。分析師抱怨即使只查「某一天、某三個欄位」的資料，查詢也要跑二十分鐘。

團隊把資料轉存為 Parquet 並依日期分區後，同樣的查詢只需十餘秒。原因有三：日期分區讓 Spark 直接跳過其他日期的檔案；述詞下推讓它依 Parquet 的區塊統計再跳過大部分資料；欄位裁剪讓它只讀三個欄位。三者相乘，讀取量降到原本的極小一部分。

這個案例呼應了第 4 章案例二與本章的公式前提——最佳化器的威力，必須有正確的資料佈局來配合。這也是為何實務上「把原始資料轉存為分區的 Parquet」幾乎是資料湖的標準作業程序。

案例三 一個 UDF 毀掉的最佳化

某工程師寫了一支 PySpark 查詢，執行卻異常緩慢。他很困惑：明明用的是 DataFrame，也存成了 Parquet，該做的都做了。

問題出在他寫了一個 Python UDF 來做字串處理，且把它用在篩選條件裡。對 Catalyst 而言這是黑箱——它無法把這個條件下推，只能先把整份資料讀出來、逐筆送進 Python 行程執行 UDF、再把結果送回 JVM。原本可以在讀檔階段就跳過的九成資料，現在全部要讀出來並跨行程往返。

解法是改用 Spark 內建的字串函式重寫該邏輯。改寫後述詞得以下推，執行時間從十餘分鐘降到不到一分鐘。這個案例是 12.6.1 節最直接的印證：UDF 不只是「慢一點」，它會讓整條最佳化鏈失效。

案例四 被 explain 揪出的 Shuffle

某作業涉及一張大表與一張小維度表的 join，工程師以為 Spark 會自動採用廣播 join（把小表送到各節點、免除 Shuffle）。但作業始終跑得很慢。

他呼叫 explain 檢視實體計畫，發現 Spark 用的其實是 Shuffle join——原因是那張「小表」是由前面一連串轉換產生的，Spark 無法準確估算它的大小，因而未超過自動廣播的判斷條件。他明確地為該表加上廣播提示後，計畫改為廣播 join，執行時間縮短逾八成。

這個案例展示了 12.6.2 節那個習慣的價值：憑猜測調校可能永遠找不到問題，而 explain 三秒鐘就把答案攤在你面前。它也呼應第 11 章「階段數即 Shuffle 數的指標」——最貴的操作往往就藏在你以為不會發生的地方。

實作活動

活動一 比較 RDD 與 DataFrame 的效能

請以 PySpark 實測兩種 API 的效能差距。

- 步驟 1. 準備一份至少含 15 欄、百萬列以上的資料（可用第 4 章活動的資料）。
- 步驟 2. 以 RDD API 撰寫一個查詢：篩選符合某條件的列，並依某欄位分組計數。
- 步驟 3. 以 DataFrame API 撰寫邏輯完全相同的查詢，記錄兩者的執行時間。
- 步驟 4. 討論：差距有多大？若把資料格式從 CSV 換成 Parquet，差距是否更明顯？為什麼？

活動二 判讀執行計畫

請養成查看執行計畫的習慣，親眼確認最佳化是否生效。

- 步驟 1. 對一份 Parquet 資料撰寫查詢：只選取三個欄位，並加上一個篩選條件。
- 步驟 2. 呼叫 explain 檢視實體計畫，找出計畫中關於「已讀取欄位」與「下推的篩選條件」的描述。
- 步驟 3. 把同一份資料另存為 CSV，執行相同查詢並再次 explain，比較兩者計畫的差異。
- 步驟 4. 說明你的觀察如何印證表 12-3 與式 12-1 的前提。

活動三 量測 UDF 的代價

請設計實驗，量化 UDF 對效能的影響。先以 Spark 內建函式撰寫一段字串處理與篩選的查詢並計時；接著以功能完全相同的 Python UDF 改寫同一段邏輯，再次計時。請比較兩者的執行時間，並各自呼叫 explain 觀察篩選條件是否被下推。最後思考：若這個 UDF 改寫為向量化版本（一次處理一批資料），你預期效能會有什麼變化？為什麼？

延伸閱讀

- **Spark SQL 論文**：Armbrust 等人發表的《Spark SQL: Relational Data Processing in Spark》，完整說明 Catalyst 的設計與四階段流程，是本章 12.4 節的第一手資料。
- **《Learning Spark》第 2 版第 5 至 9 章**：Damji 等人著，以大量範例介紹 DataFrame、Spark SQL 與資料來源操作，是本章最直接的實作補充。
- **《Spark: The Definitive Guide》結構化 API 篇**：Chambers 與 Zaharia 著，對查詢計畫、join 策略與效能調校有深入討論。
- **Apache Spark SQL 效能調校官方文件**：說明廣播門檻、自適應查詢執行等參數，是實務調校時的常備參考。

本章小結

1. RDD 是命令式介面（告訴 Spark 怎麼做），DataFrame 與 SQL 是宣告式介面（告訴 Spark 要什麼）；宣告式之所以更快，是因為你給了系統更多資訊，系統才有最佳化的空間。
2. 結構化 API 有 SQL、DataFrame 與 Dataset 三種面貌，但最終會被轉成同一份查詢計畫、交由同一個 Catalyst 處理，故效能幾乎相同；Dataset 提供編譯期型別安全但僅限 Scala 與 Java，Python 使用者以 DataFrame 為主力。
3. 在 PySpark 中，DataFrame 相較 RDD 的效能優勢尤其顯著——因為運算式會被轉為查詢計畫並在 JVM 中執行，Python 只負責「下訂單」而不「下廚」，避免了跨行程的資料往返。
4. DataFrame 是帶著綱要分散式資料表，綱要正是一切最佳化的來源；讀取 Parquet 時可直接取得綱要，而讀取 CSV、JSON 時明確指定綱要通常更快也更安全。
5. Catalyst 分為解析、邏輯最佳化、實體計畫與程式碼生成四個階段；其中最重要的兩項最佳化是述詞下推（把篩選提前到資料來源，跳過整個資料區塊）與欄位裁剪（只讀查詢實際用到的欄位）。
6. Tungsten 從堆外二進位格式、快取感知運算與全階段程式碼生成三方面提升執行效率；其中程式碼生成把一個階段的多個運算合併成單一緊湊迴圈，可帶來數倍提升。
7. UDF 對 Catalyst 而言是黑箱，會使述詞下推等最佳化失效；PySpark 的 Python UDF 還需跨程序列化往返，代價更高。能用內建函式就別寫 UDF，非寫不可時優先採用向量化版本。
8. 述詞下推與欄位裁剪的效果為相乘關係，合併讀取比為 $(k/C) \times s$ （式 12-1），可帶來加乘放大的節省；但此效果的前提是資料以支援這兩項最佳化的欄式格式（如 Parquet）儲存。

習題

一、觀念題

1. 請以「命令式」與「宣告式」的差異，說明為何 DataFrame 通常比 RDD 快。
2. 為什麼在 PySpark 中，DataFrame 相較 RDD 的效能優勢特別明顯？
3. 請分別說明述詞下推與欄位裁剪的作用，並說明為何它們需要欄式格式的配合。
4. 為什麼撰寫 UDF 會讓 Catalyst 的最佳化失效？實務上應如何因應？

二、計算題

1. 某資料表有 50 個欄位、共 20 億筆資料。某查詢只用到 5 個欄位，篩選條件的選擇率為 4%。請以式 12-1 與式 12-2 計算：（一）僅做欄位裁剪的讀取比；（二）僅做述詞下推的讀取比；（三）兩者合併的讀取比與節省比例。
2. 承上題，若原始資料以 Parquet 儲存共 2 TB。（一）請估算合併最佳化後實際需讀取的資料量。（二）若改以 CSV 儲存（假設述詞下推完全失效、欄位裁剪也無法生效，等同全表掃描），需讀取多少？（三）兩者相差幾倍？

三、思考與討論

1. 案例一中，資深工程師手工優化的 RDD 程式，敗給了新人十行的 DataFrame 查詢。請討論：這是否意味著理解底層原理已不重要？請說明你的看法與理由。
2. 本章指出「你給系統的資訊越多，系統能替你做的最佳化就越多」。請以本書前面章節的例子（如第 4 章的綱要、第 9 章的 Combiner），說明這個原則的其他體現。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

命令式（RDD）：逐步指定每個運算動作，傳給 Spark 的是不透明的函式，Spark 只能當黑箱照做，無從得知你在篩什麼、用到哪些欄位。宣告式（DataFrame／SQL）：描述想要的結果，Spark 拿到的是可被分析的運算式與綱要資訊，因而能施展述詞下推、欄位裁剪、運算合併等最佳化。核心原則：你給系統的資訊越多，系統能替你做的最佳化就越多。可引計程車比喻（只說「怎麼走」vs 說「要去哪」）。

第 2 題

因為在 RDD 模式下，Python 撰寫的函式必須在 Python 行程中執行，資料需在 JVM 與 Python 之間反覆序列化與傳輸，開銷極大。改用 DataFrame 後，你寫的運算式會被轉換為查詢計畫，實際執行完全在 JVM 中完成——Python 只負責「下訂單」（建構查詢計畫）而不「下廚」（執行運算），故避免了跨行程往返，效能可提升數倍。

第 3 題

述詞下推：把篩選條件提前到最靠近資料來源處執行，理想上推到讀檔階段，使不可能符合條件的資料區塊連讀都不讀（如依 Parquet 區塊的最大最小值統計整塊跳過）。欄位裁剪：分析查詢計畫判定真正需要的欄位，只讀取這些欄位。需欄式格式配合的原因：兩項最佳化都要求「能只讀部分資料」——欄式格式（Parquet、ORC）將同欄資料連續存放且內含區塊統計資訊，故可跳過欄與列；而 CSV 為列式純文字、無統計資訊，必須整份讀入解析才知內容，最佳化器再聰明也無從施展（見表 12-3）。

第 4 題

因為 UDF 對 Catalyst 而言是無法分析的黑箱：它看不出 UDF 用到哪些欄位、也無法判斷 UDF 中的條件能否下推，故所有最佳化在 UDF 處止步，且無法納入全階段程式碼生成。在 PySpark 中更需跨程序列化往返，代價加倍。因應方式：優先以 Spark 內建函式改寫（可完整享受最佳化）；若確實無法避免，改用向量化 UDF（一次處理一批資料而非逐筆），可大幅降低跨行程往返的開銷。

二、計算題

第 1 題

$C = 50$ 、 $k = 5$ 、 $s = 4\% = 0.04$ 。

- (1) 僅欄位裁剪： $k/C = 5/50 = 0.1$ (10%)。
- (2) 僅述詞下推： $s = 0.04$ (4%)。
- (3) 合併 (式 12-1)： $(5/50) \times 0.04 = 0.1 \times 0.04 = 0.004$ (0.4%)；節省比例 (式 12-2)： $1 - 0.004 = 0.996$ (99.6%)。

延伸思考：兩項最佳化是相乘而非相加——若誤以為相加，會得到「省 10% 加省 96%」的錯誤直覺。相乘使效果加乘放大，讀取量降至全表的千分之四。

第 2 題

原始資料 2 TB，承上題合併讀取比 0.004。

- (1) Parquet 合併最佳化後： $2 \text{ TB} \times 0.004 = 0.008 \text{ TB} = 8 \text{ GB}$ 。
- (2) CSV (等同全表掃描)：需讀取全部 2 TB = 2,000 GB。
- (3) 相差倍數： $2,000 \div 8 = 250$ 倍。

延伸思考：同一份資料、同一個查詢，僅因儲存格式不同，讀取量就相差 250 倍。這說明「資料佈局」與「查詢最佳化」必須相互配合——Catalyst 的威力有一半掌握在當初選格式的那個決定上 (呼應案例二與第 4 章)。

三、思考與討論

第 1 題

較周延的回答會指出：恰恰相反，理解底層原理更重要了，只是重點轉移。理由包括：其一，要知道「何時該相信最佳化器、何時不能」——如案例三的 UDF 與案例四的廣播 join，都需要理解 Catalyst 的運作機制才能診斷。其二，要能判讀執行計畫 (explain)，而看懂計畫需要理解 Shuffle、寬窄相依、join 策略等第 11 章的概念。其三，資料佈局的決策 (選什麼格式、如何分區) 仍由人做，且如第 2 題所示可造成數百倍差距。其四，最佳化器只能改寫查詢，無法修正錯誤的資料模型或演算法選擇。結論：抽象層提高後，人的價值從「手工優化」轉向「架構決策與問題診斷」，而這兩者都更依賴原理。

第 2 題

可舉的例子包括：其一，第 4 章的綱要與欄式格式——把型別與欄位資訊告訴系統 (Parquet 自帶綱要)，系統才能做欄位裁剪與型別特化的壓縮。其二，第 9 章的 Combiner——你告訴框架「這個聚合滿足結合律」，框架才敢在 Map 端先做局部彙整。其三，第 11 章的 cache——你告訴 Spark「這份資料會被重複使用」，它才知道值得留在記憶體。其四，第 7 章的分區鍵——你告訴系統資料該如何切分，它才能避免熱點。其五，廣播提示——你告訴 Spark 這張表夠小，它才能改用免 Shuffle 的策略。共同模式：系統的最佳化能力受限於它所能得知的資訊，而「表達意圖」往往比「指定步驟」更能釋放這種能力。

@prof

大數據分析與雲端運算

第參篇 批次運算框架

Batch Processing Frameworks

第 13 章

批次資料管線與 ETL／ELT

Batch Data Pipelines and ETL／ELT

機器人叫獸（李世淵） 編著

叫獸開講

我有一位在科技公司做資料工程的學長，他跟我說過一句話，我印象很深：「我們這行最大的謊言，就是『這支程式我在本機跑過了，沒問題』。」

他說，本機跑一次成功，跟每天凌晨三點自動跑一次、連續跑三年都成功，中間隔著一整個宇宙。因為真實世界裡，上游系統會突然改欄位名稱、某天的資料會遲到兩小時、網路會斷、磁碟會滿、有人會手殘把檔案刪了、還會有一天因為閏年而出現你從沒想過的日期格式。

他們公司有一條每晚要跑的資料管線，串接了二十幾個步驟。剛上線那陣子，幾乎每兩三天就會半夜跳出警報，得有人爬起來處理。後來他們做了三件事：讓每一步都可以安全地重跑（不會因為重跑而算錯）、讓失敗的步驟自動重試幾次、以及在資料進來時就先檢查品質——之後，值班電話才終於安靜下來。

這一章，就是要談這件事。前面四章我們學了怎麼「算」——MapReduce 的思維、YARN 的資源、Spark 的引擎與最佳化。但真實的專案不是寫一支程式跑一次，而是要建造一條每天穩定運轉的流水線。這一章要教你的，是如何把那些技術組裝成一條「跑得動、也跑得久」的資料管線。這是第參篇的最後一章，也是最貼近業界日常的一章。

學習目標

研讀完本章之後，你應該能夠：

1. 說明資料管線的組成，並區分 ETL 與 ELT 的適用情境。
2. 描述資料擷取的常見模式，並說明全量與增量擷取的取舍。
3. 列舉資料品質的檢核維度，並設計基本的品質驗證機制。
4. 說明資料清理與轉換的常見任務，以及冪等性為何是管線設計的核心。
5. 說明工作流編排的必要性，並以 DAG 描述任務的相依關係。
6. 設計管線的監控、告警與重試機制，以提升端到端的可靠度。

核心概念

核心概念	說明
資料管線	把資料從來源搬運、轉換到目的地的一連串自動化步驟。
ETL/ELT	擷取、轉換、載入的兩種順序安排，反映儲存與運算成本的演變。
全量與增量擷取	每次搬移全部資料，或只搬移新增與變動的部分。
冪等性	同一步驟執行一次與執行多次，結果相同的性質。
工作流編排	依相依關係安排任務的執行順序、排程與重試，如 Airflow。
資料品質	資料在完整性、正確性、一致性與時效性等面向的健康程度。
回填 (Backfill)	為歷史區間重新執行管線，以補齊或修正過往資料。

表 13-1 本章核心概念一覽

13.1 ETL 與 ELT 的差異

13.1.1 順序背後的成本邏輯

第 2 章我們初步認識了 ETL 與 ELT，這裡把它講得更透徹。兩者的字母完全一樣，只是 T（轉換）與 L（載入）的順序對調——但這個對調的背後，是整個產業硬體成本結構的改變。

ETL（擷取—轉換—載入）誕生於儲存昂貴、運算受限的年代。當時的資料倉儲空間寸土寸金，所以要在載入之前，先把資料清理、彙整成精簡的形式，只把「有用的」放進去。轉換通常在一台中途的伺服器上完成。

ELT（擷取—載入—轉換）則是雲端時代的產物。當物件儲存變得極為低廉、運算又能彈性擴充，「先全部存下來、需要時再算」就成了更划算的選擇。原始資料原封不動載入資料湖，轉換則在目的地內部進行，且可以隨時用新的角度重新轉換。

面向	ETL	ELT
轉換位置	載入前的中途伺服器	載入後的目的地內部
保留原始資料	否（只留轉換後結果）	是（原始資料完整保存）
需求變更時	須修改邏輯並重新擷取	原始資料仍在，可重新轉換
適合場景	來源穩定、需求明確、儲存昂貴	探索性分析、需求多變、雲端環境

表 13-2 ETL 與 ELT 的比較

叫獸提醒

這裡有個觀念要澄清：ELT 不是「比較新所以比較好」，而是「在雲端的成本結構下比較划算」。如果你的場景是地端機房、儲存空間有限、而且需求非常明確固定，ETL 依然是合理的選擇。技術的演進往往不是「後者取代前者」，而是「前提改變了，最佳解也跟著改變」。你在第 5 章看過同樣的故事——儲存運算分離之所以成為主流，也是因為網路頻寬變便宜了。

13.2 資料擷取與整合

13.2.1 全量擷取與增量擷取

管線的第一步是把資料從來源取回來，而最關鍵的決策是：每次都搬全部，還是只搬變動的部分？

全量擷取每次把來源的整份資料重新搬一遍。它的優點是簡單、不會遺漏、而且天生冪等（重跑幾次結果都一樣）。缺點是當資料量大時極為浪費——為了幾千筆新增資料，搬動了幾億筆舊資料。

增量擷取則只搬新增或變動的部分，通常依「更新時間戳」或第 2 章談過的 CDC（變更資料擷取）來判斷。效率高得多，但複雜度也高：時間戳若不可靠（例如來源系統改資料卻沒更新時間戳），就會漏資料；而「上次抓到哪裡」這個狀態也必須被可靠地記錄下來。

面向	全量擷取	增量擷取
搬移量	整份資料，浪費但簡單	僅變動部分，高效
冪等性	天生具備	須額外設計
風險	資料量大時難以負荷	可能漏抓（時間戳不可靠時）

面向	全量擷取	增量擷取
適用	小型維度表、來源無時間戳	大型事實表、有可靠時間戳或 CDC

表 13-3 全量與增量擷取的比較

實務上常見的折衷是「分區覆寫」：以日期為分區，每次只重跑最近幾天的分區並整批覆寫。這既避免了全量的浪費，又保留了「重跑會得到相同結果」的冪等性——它是實務上最常被推薦的模式，下一節會再展開。

13.2.2 來源多樣性的挑戰

真實的管線往往要面對數個甚至數十個來源：交易資料庫、應用日誌、第三方 API、合作夥伴傳來的檔案。每個來源的格式、更新頻率、可靠度都不同，整合它們的困難常被低估。

其中最惡名昭彰的問題，是第 2 章提過的「上游無預警改結構」。上游工程師改了一個欄位名，完全不知道下游有一整條管線因此斷裂。防範之道包括：在擷取端加入網要驗證（發現不符立刻告警，而非默默算出錯誤結果）、與上游建立「資料契約」明訂結構、以及保留原始資料以便回溯修正。這也再次呼應了 ELT 保留原始資料的價值。

13.3 資料清理與品質檢核

13.3.1 資料品質的六個維度

「資料品質不好」是一句太籠統的抱怨。要能有效改善，必須先把它拆解成可以量測的維度。

維度	檢查什麼	檢核範例
完整性	該有的資料是否都在	關鍵欄位是否有空值？筆數是否明顯偏少？
正確性	值是否符合現實	溫度是否落在合理範圍？金額是否為負？
一致性	跨系統或跨欄位是否矛盾	訂單總額是否等於明細加總？
唯一性	是否有不該出現的重複	同一筆交易是否被記錄兩次？
時效性	資料是否夠新	最新一筆資料是否在預期時間內抵達？
有效性	格式是否符合規範	日期格式、代碼是否在允許清單內？

表 13-4 資料品質的六個檢核維度

13.3.2 把檢核變成管線的一部分

業餘的做法是「出事了再去查資料」；專業的做法是「讓管線自己檢查，有問題就攔下來」。也就是把品質檢核寫成管線中的一個步驟，每次執行都自動驗證。

檢核發現問題時，通常有三種處理方式，該選哪一種取決於問題的嚴重程度：其一，讓管線失敗並告警（適用於嚴重問題，如關鍵欄位大量空值——寧可沒有資料，也不要錯的資料）；其二，隔離問題資料並繼續（把有問題的紀錄丟進「隔離區」供人工檢視，其餘正常處理）；其三，記錄警告但照常執行（適用於輕微異常）。

叫獸提醒

這裡有個原則我希望你記住：錯誤的資料，比沒有資料更危險。沒有資料，大家會知道「今天

報表還沒好」；但錯誤的資料會被當成正確的，一路流進儀表板、流進決策、甚至流進機器學習模型的訓練集裡。等到幾個月後有人發現數字不對，往往已經無法追溯是哪一天開始錯的。所以在管線裡設下品質關卡、讓它在源頭就攔住錯誤，是一項高報酬的投資。

13.4 資料轉換與彙整

13.4.1 常見的轉換任務

轉換是把原始資料變成可用形式的過程，常見的任務包括幾類。

- **清理**：處理空值、修正格式、統一單位與編碼（例如把五種日期寫法統一）。
- **去重**：移除重複紀錄，常以業務主鍵加上時間戳判定保留哪一筆。
- **關聯**：把多個來源的資料以共同的鏈接起來，例如訂單接上顧客與商品資訊。
- **彙整**：依維度分組計算指標，例如每日每產品線的銷售額。
- **衍生欄位**：由既有欄位計算出新欄位，例如由出生日期算出年齡級距。

這些任務在 Spark 中多半就是第 12 章的 DataFrame 運算——本章談的是「如何把它們組織成可靠的流程」，而非運算本身的語法。

13.4.2 冪等性：管線設計的第一原則

如果這一章只能記住一個詞，我希望是「冪等性」（idempotency）——同一個步驟執行一次與執行多次，結果完全相同。你在第 3 章談逾時重試時已經遇過它，在這裡它更是管線設計的第一原則。

為什麼如此重要？因為管線一定會失敗，失敗就要重跑。若某個步驟不具冪等性——例如它的做法是「把今天的資料附加到結果表」——那麼重跑一次，今天的資料就被加了兩遍，數字直接錯掉，而且很難察覺。

讓步驟具備冪等性的最常見做法，是「以覆寫取代附加」：不要說「把今天的資料加進去」，而要說「把日期等於今天的那個分區，整個換成這批資料」。如此一來，無論重跑幾次，那個分區的內容都一樣。這正是 13.2.1 節「分區覆寫」模式受推崇的原因。

另一個做法是使用「合併」（upsert）語意：以主鍵為準，存在就更新、不存在就插入。這同樣保證重複執行不會產生重複資料。

13.4.3 回填：面對過去的能力

與冪等性密切相關的，是「回填」（backfill）的能力——為過去某段區間重新執行管線。

你一定會需要它。可能是發現三個月前的邏輯寫錯了、可能是上游補送了一批遲到的資料、也可能是新增了一個欄位想套用到歷史資料。若你的管線設計成冪等且以日期分區，回填就只是「指定日期區間、重跑一次」這麼簡單；若不是，回填就會變成一場需要小心翼翼手動處理的災難。

所以設計管線時，請一開始就假設「這條管線總有一天要重跑過去某一天」。這個假設會自然引導你走向正確的設計：以時間分區、冪等覆寫、參數化的執行日期。

13.5 工作流編排（Airflow）

13.5.1 為什麼需要編排器

當管線只有三個步驟時，寫個排程指令就夠了。但當它長到二十幾個步驟、彼此還有複雜的先後相依時，你會需要一個專門的工具，這就是工作流編排器（orchestrator），業界最廣為使用的是 Apache Airflow。

編排器要解決的問題包括：哪些任務可以平行、哪些必須等待前置完成？某個任務失敗時，該重試幾次、要不要通知人？如何為過去的日子重跑（回填）？如何看出昨天到底是哪一步卡住了？這些若要自己土法煉鋼地實作，會演變成一套沒人維護得動的腳本。

13.5.2 又見 DAG

Airflow 描述工作流的方式，是我們已經很熟悉的 DAG——有向無環圖。每個節點是一個任務，每條邊代表「前者完成後，後者才能開始」的相依關係，而「無環」保證了不會出現彼此互等的死結。

你會發現 DAG 這個概念在本書出現了三次：第 11 章 Spark 用它描述 RDD 的相依與執行計畫，這裡用它描述任務的相依與排程，而第 32 章談資料血緣時還會再遇到它。這不是巧合——凡是「有先後順序、且不能循環」的關係，DAG 都是最自然的表達方式。

編排器依 DAG 決定執行順序：沒有相依關係的任務可以同時跑，有相依的則依序等待。它同時提供排程（每天幾點執行）、重試（失敗自動重試幾次、間隔多久）、告警（失敗時通知誰）以及執行歷史的可視化。

13.5.3 任務設計的原則

使用編排器時，有幾條實務原則值得遵守。

- **任務要小而明確：**一個任務只做一件事。任務太大時，失敗後必須整個重跑，浪費且難以定位問題。
- **任務必須幂等：**編排器會自動重試，若任務不幂等，重試本身就會製造錯誤資料。
- **以執行日期為參數：**任務應接受「要處理哪一天」作為參數，而非在程式中寫死「今天」——這是回填能力的前提。
- **避免任務間隱性相依：**若任務 B 其實依賴 A 的輸出，就要在 DAG 中明確宣告，別依賴「排程時間剛好排在後面」的巧合。

13.6 管線監控與錯誤處理

13.6.1 該監控什麼

一條管線「沒有跳出錯誤」，不等於「一切正常」。有一種最可怕的失敗叫做「安靜的失敗」——程式順利執行完畢、沒有任何錯誤訊息，但產出的資料是錯的或不完整的。因此監控不能只看「有沒有失敗」，還要看幾個面向。

- **執行狀態：**任務是否成功、耗時多久、是否比平常久很多（可能是資料量異常或效能劣化）。
- **資料量：**今天產出的筆數是否落在合理範圍？突然只有平常的一成，往往意味著上游出了問題。
- **品質指標：**空值比例、重複率等 13.3 節的檢核結果，是否惡化。
- **時效性：**資料是否在承諾的時間前備妥？下游的人幾點會需要它？

其中「資料量的異常偵測」特別值得投資：它能抓到大量的安靜失敗。若一條管線平常每天產出約一百萬筆，今天只有八萬筆，即使程式沒報錯，也絕對值得立刻檢查。

13.6.2 錯誤處理與重試策略

面對失敗，設計良好的管線會分類處理。有些錯誤是「暫時性」的——網路抖動、來源系統短暫繁忙、資源暫時不足；這類錯誤自動重試往往就能解決。有些則是「永久性」的——程式邏輯有臭蟲、來源結構改變、權限被撤銷；這類錯誤重試一百次也沒用，該做的是立刻通知人。

重試時的常見做法是「指數退避」：第一次等一分鐘、第二次等兩分鐘、第三次等四分鐘。這避免了在來源系統已經很吃力時，還被重試流量進一步壓垮。而重試的次數要有上限——無限重試會讓真正的問題被掩蓋，也可能造成資源浪費。

最後別忘了告警的品質。告警太少會漏掉問題，但告警太多更糟——當值班的人每天收到五十封警報，其中四十九封是誤報，他很快就會全部忽略，那麼第五十封真正重要的也就跟著被忽略了。這稱為「告警疲勞」，是維運上非常實際的風險。

13.6.3 承先啟後

本章我們把第參篇的技術，組裝成了一條能夠實際運轉的流水線：釐清了 ETL 與 ELT 背後的本邏輯（13.1）、選擇擷取策略（13.2）、把品質檢核內建為管線的關卡（13.3）、以冪等性與回填能力設計轉換步驟（13.4）、以 DAG 編排任務的相依與重試（13.5），並建立起監控與錯誤處理的機制（13.6）。

到這裡，第參篇「批次運算框架」四章圓滿收束。回頭看，第 9 章給了我們分散式運算的思維，第 10 章給了執行的資源，第 11、12 章給了高效的引擎與最佳化，而本章則把它們組織成可靠的日常運作。你已經能夠處理「昨天的、整批的」資料了。

但你有沒有注意到，這整篇都建立在一個假設上：資料是「一批一批」到齊的，我們等它到齊了再開始算。可是回想第 2 章談過的無界資料——產線的感測器此刻仍在產生訊號、交易此刻仍在發生。如果我們需要的不是「昨天的報表」，而是「此刻的異常告警」呢？當資料像水流一樣永不停歇地湧入，批次的思維就不夠用了。這就是第肆篇之後我們要面對的新世界，而在那之前，我們要先進入本書分析的核心——第肆篇的大數據探勘與分析演算法。

公式與流程：管線的端到端可靠度

一條管線由許多步驟串接而成，而「整條管線成功」需要每一步都成功。這件事的數學後果，比多數人直覺想像的嚴重得多。

設管線共有 n 個步驟，每個步驟單次執行的成功率為 q ，且各步驟獨立。則整條管線一次執行成功的機率為：

$$P(\text{成功}) = q^n$$

式 13-1 無重試時的端到端成功率

這是一個乘法，而乘法對長度極為敏感。即使每一步都有 99% 的成功率，二十步串起來也只剩約 81.8%——換句話說，每五天就有一天會失敗。

現在加入重試。若每個步驟允許重試至多 k 次（即最多執行 $k + 1$ 次），且各次嘗試獨立，則單一步驟最終成功的機率提升為 $1 - (1 - q)^{(k+1)}$ ，故整條管線的成功率為：

$$P(\text{成功, 含重試}) = [1 - (1 - q)^{(k+1)}]^n$$

式 13-2 含重試機制的端到端成功率

我們以 20 個步驟、單步成功率 99% 為例，看重試次數帶來的改變。

重試次數 k	單步最終成功率	端到端成功率	約略失敗頻率
0 (不重試)	99%	約 81.8%	每 5.5 天失敗一次
1	99.99%	約 99.8%	約每 500 天一次
2	99.9999%	約 99.998%	約每 50,000 天一次

表 13-5 重試次數對端到端可靠度的影響 ($n = 20$ 、 $q = 99\%$)

叫獸提醒

表 13-5 是我最想讓你記住的一張表。只加「重試一次」，管線的失敗頻率就從「每五天一次」變成「約一年半一次」——這正是叫獸開講裡那位學長，讓值班電話終於安靜下來的關鍵。但請注意式 13-2 有一個沒寫出來的前提：重試要有意義，步驟就必須是冪等的。若不冪等，重試不但救不了你，還會製造出重複或錯誤的資料——你把「失敗」換成了「安靜地算錯」，那更糟。所以 13.4.2 節的冪等性與這裡的重試，是一組必須成對出現的設計。

案例研討

案例一 每兩天響一次的值班電話

回到叫獸開講的故事。那條串接二十幾個步驟的管線，剛上線時幾乎每兩三天就要有人半夜起來處理。工程師們一開始以為是程式寫得不好，拼命檢查邏輯。

但依式 13-1，即使每一步都有 99% 的可靠度，二十步的端到端成功率也只有約 81.8%——失敗根本不是「寫得不好」，而是「串太長又沒有重試」的數學必然。他們後來做的三件事恰好對症下藥：讓每步冪等（重跑安全）、加上自動重試（依式 13-2 把成功率拉到 99.8%）、以及品質前置檢核（把上游的問題擋在源頭）。

這個案例的教訓是：分散式系統的可靠度問題，往往不能只靠「把每個零件做得更好」，而要靠「設計出容忍零件失敗的機制」。這正是第 3 章容錯思想在資料管線上的體現。

案例二 重跑一次，業績多一倍

某公司的每日銷售彙整管線，某天因網路問題在最後一步失敗。工程師手動重跑後，發現當日業績數字竟然變成了兩倍。

原因是那個步驟的實作方式是「把今天計算好的結果附加到彙整表」。第一次執行其實已經成功寫入，只是在回報狀態時斷線；重跑時又附加了一次，於是同一天的資料被記了兩遍。

修正方式是改為分區覆寫：不再說「附加今天的資料」，而是「把日期等於今天的分區整個換掉」。如此無論重跑幾次，結果都相同。這個案例是 13.4.2 節冪等性最直接的血淚教訓——也說明了為何「不冪等的重試」比不重試更危險：它把一次明顯的失敗，變成了一個安靜的錯誤數字。

案例三 安靜的失敗

某團隊的管線連續一週執行成功、沒有任何錯誤告警。直到業務單位反映報表數字怪怪的，他們才發現：上游系統七天前調整了一個篩選條件，導致傳來的資料只剩原本的一成。

管線本身毫無問題——它忠實地處理了收到的資料，只是收到的資料本來就少了九成。程式沒有錯誤，所以沒有告警；但資料是錯的，而且錯了整整一週才被發現。

他們後來加上了 13.6.1 節的資料量異常偵測：每次執行後比對筆數與近期的平均值，落差超過門檻就告警。這類「資料層面」的監控，正是抓出安靜失敗的關鍵——因為安靜的失敗，依定義就不會觸發任何程式層面的錯誤。

案例四 那個沒人看的告警

某公司的資料平台設定了非常完整的告警：任何任務只要執行時間超過平常、任何欄位只要有空值、任何步驟只要重試過，都會發信通知。結果值班工程師每天收到數十封警報，其中絕大多數無關緊要。

三個月後的某一天，一封真正嚴重的告警（核心交易資料完全沒有進來）夾在四十幾封例行警報中，被順手略過。問題直到隔天下午才被發現。

這就是 13.6.2 節的「告警疲勞」。他們後來重新設計告警：依嚴重程度分級，只有真正需要立即處理的才發送即時通知，其餘彙整成每日摘要。告警數量從每天數十封降到每週一兩封，而每一封都會被認真對待。這個案例提醒我們：監控系統的目標不是「偵測到最多問題」，而是「讓真正重要的問題被看見」。

實作活動

活動一 設計一條資料管線

請為「每日彙整校園 Wi-Fi 使用紀錄」設計一條批次資料管線。

- 步驟 1.** 列出至少五個步驟（如擷取、驗證、清理、彙整、載入），並畫出它們的 DAG。
- 步驟 2.** 為每個步驟判斷：它是否冪等？若否，該如何改寫成冪等？
- 步驟 3.** 設計至少三項品質檢核（對照表 13-4 的維度），並說明檢核失敗時該如何處理。
- 步驟 4.** 假設每步成功率為 98%、共 6 步，請以式 13-1 計算端到端成功率；再以式 13-2 計算加上重試 1 次後的成功率。

活動二 把不冪等的步驟改寫

請以 Python 或 SQL 實作並比較兩種寫法的差異。

- 步驟 1.** 準備一份帶有日期欄位的資料，寫一個「把當日資料附加到結果表」的步驟。
- 步驟 2.** 連續執行兩次，觀察結果表的筆數變化（重現案例二的問題）。
- 步驟 3.** 改寫為「先刪除當日分區、再寫入」或使用合併（upsert）語意。
- 步驟 4.** 再次連續執行兩次，確認結果不再重複，並說明這如何保障了重試的安全性。

活動三 實作品質檢核與異常偵測

請為活動一的管線實作自動化的品質關卡。撰寫一支程式，在資料處理完成後自動檢查：關鍵欄位的空值比例是否超過門檻、是否有重複的主鍵、以及本次筆數與最近七次平均的落差是否超過某個百分比。當任一項檢查失敗時，讓程式以非零狀態結束（模擬觸發告警）。完成後思考：這三項檢查分別對應表 13-4 的哪些維度？又分別能抓到案例二與案例三中的哪一種問題？

延伸閱讀

- 《Fundamentals of Data Engineering》：Reis 與 Housley 著。關於資料擷取、轉換與編排的章節，是本章最完整的依據，對 ETL/ELT 的取舍也有深入討論。
- **Apache Airflow 官方文件**：說明 DAG 的撰寫方式、排程與回填機制、重試與告警設定，是 13.5 節的實作參考。
- 《Designing Data-Intensive Applications》第 10、11 章：Kleppmann 與 Riccomini 著，第 10 章談批次處理的容錯與冪等設計，與本章 13.4 節相互印證。
- **資料品質工具的說明文件**：如 Great Expectations 等開源工具，示範如何把 13.3 節的品質檢核系統化地寫成可重複執行的驗證。

本章小結

1. ETL 與 ELT 的順序差異反映硬體成本結構的演變：ETL 生於儲存昂貴的年代（載入前先轉換以節省空間），ELT 生於雲端（儲存低廉、運算彈性，故先全部載入再按需轉換並保留原始資料）。
2. 擷取策略分全量（簡單、天生冪等，但浪費）與增量（高效，但須額外設計且可能漏抓）；實務常用折衷是「以日期分區、整批覆寫最近數個分區」，兼顧效率與冪等性。
3. 資料品質可拆解為完整性、正確性、一致性、唯一性、時效性與有效性六個可量測的維度；應把檢核寫成管線中的關卡，並依嚴重程度決定失敗、隔離或警告。錯誤的資料比沒有資料更危險。
4. 冪等性是管線設計的第一原則——同一步驟執行一次與多次結果相同。常見做法是以「分區覆寫」或「合併（upsert）」取代「附加」；不冪等的重試會把明顯的失敗變成安靜的錯誤。
5. 回填能力（為過去區間重新執行）是管線的必備需求；設計時應假設「總有一天要重跑過去某一天」，這自然導向以時間分區、冪等覆寫、並以執行日期為參數的設計。
6. 工作流編排器以 DAG 描述任務相依，提供排程、重試、告警與回填；任務設計應小而明確、必須冪等、以執行日期為參數，並明確宣告相依而非依賴排程時間的巧合。
7. 監控不能只看「有沒有失敗」，還須監控資料量、品質指標與時效性，以抓出「程式成功但資料錯誤」的安靜失敗；錯誤應區分暫時性（自動重試、指數退避）與永久性（立即通知人），並避免告警疲勞。
8. 端到端成功率為 q^n （式 13-1），對步驟數極為敏感——20 步各 99% 僅得約 81.8%；加入重試後提升為 $[1-(1-q)^{(k+1)}]^n$ （式 13-2），僅重試一次即可將成功率拉至約 99.8%，但其前提是步驟必須冪等。

習題

一、觀念題

1. 請說明 ETL 與 ELT 的差異，並解釋為何雲端環境使 ELT 成為主流。這是否代表 ETL 已無用武之地？
2. 請比較全量擷取與增量擷取的優缺點，並說明「分區覆寫」為何是實務上受推薦的折衷方案。
3. 何謂冪等性？請以案例二為例，說明不冪等的步驟為何會讓「重試」變得比「不重試」更危險。
4. 何謂「安靜的失敗」？為什麼只監控程式的執行狀態無法偵測它？應該如何補強？

二、計算題

1. 某資料管線共有 15 個步驟，每步單次執行的成功率為 98%。請以式 13-1 計算端到端成功率，並估算平均每幾天會失敗一次（假設每日執行一次）。

2. 承上題，若為每個步驟加上重試機制。請以式 13-2 計算：（一）重試 1 次時的單步最終成功率與端到端成功率；（二）重試 2 次時的端到端成功率。並說明為何重試的效益會呈現遞減。

三、思考與討論

1. 本章主張「錯誤的資料比沒有資料更危險」。請就一個你熟悉的應用情境，討論這句話的具體後果，並說明你會在管線的哪些位置設下品質關卡。
2. 案例四的團隊因告警過多而錯過真正重要的警報。請討論：設計告警機制時，該如何在「不漏掉問題」與「不製造疲勞」之間取得平衡？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

差異：ETL 在載入前於中途伺服器轉換，只把處理後的結果放進目的地；ELT 先把原始資料載入目的地，再於其內部按需轉換。雲端使 ELT 成為主流的原因：物件儲存極為低廉（不必為省空間而預先丟棄原始細節）、運算可彈性擴充（負擔得起在目的地大規模轉換），且保留原始資料使需求變更時能重新轉換。ETL 並非無用：在地端機房、儲存有限、需求明確固定的場景仍屬合理選擇。核心觀念是「前提改變，最佳解才跟著改變」，而非後者取代前者。

第 2 題

全量：每次搬移整份資料，簡單、不會遺漏、天生冪等，但資料量大時極浪費。增量：只搬新增或變動部分（依時間戳或 CDC），效率高，但須可靠記錄「上次抓到哪」，且時間戳不可靠時可能漏抓。分區覆寫之所以受推薦：以日期分區、每次只重跑最近數個分區並整批覆寫——既避免全量的浪費（只處理近期資料），又保有冪等性（重跑同一分區結果相同），同時也是回填能力的基礎。

第 3 題

冪等性指同一步驟執行一次與執行多次結果完全相同。案例二中，步驟採「把當日資料附加到彙整表」，第一次其實已寫入成功、僅回報狀態時斷線，重跑後又附加一次，導致當日業績變兩倍。危險之處在於：不重試時，失敗是「明顯的」（有錯誤、有告警、大家知道資料還沒好）；而不冪等的重試會把它變成「安靜的錯誤」——程式顯示成功、無任何告警，但數字已錯，可能流入報表與決策而長期無人察覺。故冪等性與重試必須成對設計。

第 4 題

安靜的失敗指程式順利執行完畢、無任何錯誤訊息，但產出的資料是錯的或不完整的（如案例三：上游改條件致資料只剩一成，管線忠實處理了收到的資料）。只監控執行狀態無法偵測，因為依定義它不會觸發程式層面的錯誤——程式本身沒有問題。補強方式：加入「資料層面」的監控，包括資料量異常偵測（比對筆數與近期平均，落差超過門檻即告警）、品質指標追蹤（空值比例、重複率）、以及時效性檢查（最新資料是否在預期時間內抵達）。

二、計算題

第 1 題

套用式 13-1, $n = 15$ 、 $q = 0.98$ 。

- (1) 端到端成功率 $= 0.98^{15} \approx 0.7386$ (約 73.9%)。
- (2) 失敗機率 $= 1 - 0.7386 = 0.2614$ (約 26.1%)。
- (3) 平均失敗間隔 $= 1 \div 0.2614 \approx 3.8$ 天, 即約每 4 天就會失敗一次。

延伸思考：每一步都有 98% 的高可靠度，串起來卻不到四天就失敗一次——這正說明長管線的可靠度問題是「數學必然」，不能只靠把每個步驟做得更好來解決。

第 2 題

套用式 13-2, 單步最終成功率 $= 1 - (1 - q)^{(k+1)}$, 其中 $1 - q = 0.02$ 。

- (1) $k = 1$: 單步最終成功率 $= 1 - 0.02^2 = 1 - 0.0004 = 0.9996$; 端到端 $= 0.9996^{15} \approx 0.9940$ (約 99.4%)。
- (2) $k = 2$: 單步最終成功率 $= 1 - 0.02^3 = 1 - 0.000008 = 0.999992$; 端到端 $= 0.999992^{15} \approx 0.99988$ (約 99.99%)。

效益遞減的原因：第一次重試把端到端成功率由 73.9% 拉到 99.4%，改善幅度極大（失敗率由 26.1% 降到 0.6%，約 43 倍）；第二次重試僅由 99.4% 提升到 99.99%（失敗率由 0.6% 降到 0.012%）。因為失敗率隨重試次數呈指數下降，當它已經很小時，再降低的絕對效益就很有限，卻仍要付出等待與資源的成本。故實務上重試次數通常設在 2 至 3 次即足，且必須設上限以免掩蓋真正的永久性錯誤。

三、思考與討論

第 1 題

答題應包含三部分。其一，具體後果：以智慧製造為例，錯誤的感測資料可能導致誤判設備健康狀態（該保養的沒保養、或無謂停機），甚至被用於訓練預測模型而使錯誤長期固化；以商業報表為例，錯誤數字會流入決策，且往往數月後才被發現而難以追溯起始點。相對地，「沒有資料」會立即被察覺（報表空白），大家知道不能依賴它。其二，關卡位置：擷取後立即驗證綱要與筆數（攔住上游變更）、清理後檢查空值與重複（唯一性、完整性）、彙整後比對總量與歷史區間（一致性、合理性）、以及載入前的最終驗收。其三，可補充處理策略的分級（失敗、隔離、警告）。

第 2 題

可提出的平衡原則包括：其一，依嚴重程度分級——只有需要「立刻有人起床處理」的才發即時通知，其餘彙整為每日摘要或儀表板。其二，以影響為準而非以事件為準——判準應是「這件事會不會影響下游使用者」，而非「有沒有異常發生」（如重試一次後成功就不需告警）。其三，設定合理門檻並定期校準——誤報率高的規則應調整或移除，而非放任累積。其四，告警內容要可行動——說明發生什麼、影響為何、建議如何處理，避免收到後仍需長時間查證。其五，定期檢討：統計哪些告警從未導致任何行動，這些即是應刪除的候選。核心觀念：監控的目標不是偵測最多問題，而是讓真正重要的問題被看見並被處理。

第肆篇 大數據探勘與分析演算法

Mining & Analytics Algorithms

第 14 章

相似性探勘：MinHash 與 LSH

Similarity Mining: MinHash and LSH

機器人叫獸（李世淵） 編著

叫獸開講

我每學期都要改幾百份報告，而且每年都會遇到抄襲。有些抄得很笨，整段複製貼上；有些聰明一點，換幾個詞、調換句子順序。問題來了：我要怎麼在幾百份報告裡，找出哪兩份「長得很像」？

最笨的辦法，是每兩份都比對一次。三百份報告，要比 $300 \times 299 \div 2 = 44,850$ 次。這還算勉強做得到。但如果是三十萬份呢？比對次數會暴增到大約四百五十億次——就算每次比對只要一毫秒，也要跑上一年半。

這就是「相似性探勘」的核心難題：兩兩比對的次數，會隨資料量的平方成長。資料量增加一百倍，比對次數增加一萬倍。這種成長速度，讓暴力法在大數據面前完全失效。

這一章要教你的，是一套極其漂亮的解法。它分三步：先把文件變成集合（Shingling），再把龐大的集合壓縮成一個短短的指紋（MinHash），最後用一種巧妙的分桶技巧，讓「可能相似」的文件自動掉進同一個桶裡（LSH）——如此一來，我們只需要比對同桶內的少數文件，而不必兩兩硬碰硬。

而且我要提醒你，這絕不是只能用來抓抄襲的小把戲。回想第 1 章案例四——訓練大語言模型時，要從數十億份網頁中找出近似重複的文件並移除，用的正是這套技術。一個一九九〇年代的演算法，三十年後成了訓練 AI 的關鍵基礎設施。這就是演算法的力量：它不會過時，只會找到新的舞台。

學習目標

研讀完本章之後，你應該能夠：

1. 說明相似性探勘的平方成長難題，並指出暴力法為何不可行。
2. 運用 Shingling 把文件轉換為集合，並選擇合適的 shingle 長度。
3. 計算兩個集合的 Jaccard 相似度，並說明其意義與限制。
4. 說明 MinHash 的原理，並解釋簽章相同的機率為何等於 Jaccard 相似度。
5. 運用 LSH 的分帶技巧，依需求設定 b 與 r 以調整相似度門檻。
6. 評估 LSH 的偽陽性與偽陰性，並應用於近似重複偵測的實務。

核心概念

核心概念	說明
相似性探勘	在大量物件中找出彼此相似者，難點在於比對次數隨資料量平方成長。
Shingling	把文件切成連續 k 個字元或詞的片段集合，將文字轉為集合。
Jaccard 相似度	兩集合交集大小除以聯集大小，衡量集合的重疊程度。
MinHash	以雜湊最小值構成的短簽章，其相等機率等於 Jaccard 相似度。
簽章矩陣	把每份文件的長集合壓縮為固定長度簽章後排成的矩陣。
LSH	區域敏感雜湊，讓相似的項目高機率落入同一桶中的分桶技巧。

核心概念	說明
候選對	經 LSH 篩選後值得實際比對的少數配對。

表 14-1 本章核心概念一覽

14.1 相似性問題與應用

14.1.1 平方成長的詛咒

承接叫獸開講，我們把問題的規模講精確。若有 N 個物件，兩兩比對的次數為組合數：

$$\text{比對次數} = N(N - 1) / 2 \approx N^2 / 2$$

式 14-1 暴力兩兩比對的次數

這個平方項是災難的來源。下表讓你直觀感受它的成長速度。

物件數 N	比對次數	每次 1 微秒約需時間
1,000	約 50 萬次	0.5 秒
100,000	約 50 億次	約 1.4 小時
1,000,000	約 5,000 億次	約 5.8 天
100,000,000	約 5×10^{15} 次	約 158 年

表 14-2 暴力比對的規模成長

看最後一列：一億份文件——這在網頁或語料庫中根本不算多——用暴力法要跑一百五十八年。而且這還只是「比對次數」，尚未計入每次比對本身的成本（比較兩份長文件並不便宜）。所以我們需要的不是更快的機器，而是一個能避開平方成長的演算法。

14.1.2 哪些問題是相似性問題

相似性探勘的應用範圍，比初學者想像的廣得多。

- **近似重複偵測**：找出內容幾乎相同的網頁、新聞或文件。搜尋引擎用它避免重複結果，LLM 訓練用它清理語料。
- **抄襲與剽竊偵測**：比對學術報告、程式碼或新聞稿的雷同程度。
- **推薦系統**：找出「買了相似商品的顧客」或「內容相似的影片」，這與第 19 章密切相關。
- **實體解析**：判斷不同來源的兩筆資料（如兩份客戶名單）是否指向同一個人或公司。
- **異常偵測**：在製造領域，比對感測器訊號的波形樣態，找出與正常樣態不相似的異常。

叫獸提醒

最後一項值得你特別留意，因為它最貼近本學程。產線上的振動訊號、電流波形，都可以被轉換成一組特徵集合；如此一來，「這段訊號和已知的故障樣態有多像」就變成了一個相似性問題。同樣一套 MinHash 與 LSH，換個資料型態就能用在設備診斷上。這就是為什麼我一直強調要學原理而非只學工具——原理會遷移，工具不會。

14.2 Shingling：把文件變成集合

14.2.1 為什麼要先變成集合

整套方法的第一步，是把「文件」轉換成「集合」。為什麼？因為集合有一個極為好用的性質——我們有現成、簡潔且數學性質良好的方式來衡量兩個集合有多像（下一節的 Jaccard 相似度），而「兩段文字有多像」則含糊得多。

Shingling 的做法很直接：把文件切成一個個「連續 k 個單位」的片段，這些片段的集合就代表這份文件。單位可以是字元，也可以是詞。

舉個中文的例子。取 $k = 3$ （以字元為單位），文件「機器人學導論」會被切成：{ 機器人, 器人學, 人學導, 學導論 } 這四個 shingle。注意是「連續且重疊」地滑動切割，而非平均分段。

14.2.2 k 該取多少

k 的選擇是一個關鍵的取捨，太小與太大都會出問題。

k 太小時，幾乎任何兩份文件都會共用大量 shingle。以中文 $k = 1$ 為例，那就是在比較「用了哪些字」——但常用字大家都會用，於是所有文件看起來都很像，失去了鑑別力。

k 太大時，則只有幾乎逐字相同的文件才會共用 shingle。稍微改寫、換個詞序，相似度就掉到接近零——這樣就抓不到「改寫過的抄襲」了。

實務上的經驗法則是： k 要大到「隨機兩份文件共用某個 shingle 的機率很低」。英文以字元計常取 $k = 5$ 到 9（短文件取小、長文件取大），中文因單字資訊量較高，字元 k 取 3 到 5 通常就足夠若以詞為單位， k 取 2 到 3 是常見選擇。

14.2.3 雜湊壓縮

實務上還有一個小技巧：shingle 本身是字串，佔空間也不好比對，因此通常會先把每個 shingle 雜湊成一個整數（例如 32 位元）。如此一來，一份文件就變成了一組整數的集合，既省空間也加快後續運算。

雜湊會有碰撞（不同 shingle 得到相同整數）的風險，但只要雜湊空間夠大（如 2^{32} 約四十億）碰撞機率極低，對整體結果的影響可以忽略。這也是本章反覆出現的思想：用可控的少量誤差，換取巨大的效率提升。

14.3 Jaccard 相似度

14.3.1 定義與直覺

有了集合，就能衡量相似度。最常用的度量是 Jaccard 相似度，定義為兩集合交集的大小除以聯集的大小：

$$J(A, B) = |A \cap B| / |A \cup B|$$

式 14-2 Jaccard 相似度

它的直覺很清楚：分子是「兩者共有的部分」，分母是「兩者合起來的全部」。值域在 0 到 1 之間——完全相同為 1，毫無交集為 0。

舉例來說，若 $A = \{1, 2, 3, 4\}$ 、 $B = \{3, 4, 5\}$ ，則交集為 $\{3, 4\}$ （大小 2）、聯集為 $\{1, 2, 3, 4, 5\}$ （大小 5），故 $J(A, B) = 2 / 5 = 0.4$ 。

為什麼用聯集當分母

初學者常問：為什麼不用「交集除以其中一個集合的大小」？因為那樣會不公平。假設 A 是一篇一萬字的長文、B 是從 A 裡抄了一百字的短文。若以 B 的大小為分母，相似度會是 1.0（B 的內容全在 A 裡），但這兩份文件顯然不該被視為「完全相同」。用聯集當分母，就自然地把「兩者規模的差異」納入了考量。

14.3.2 問題還沒解決

到這裡，我們有了明確的相似度定義。但請注意——問題其實一點都沒解決。要算出所有文件兩兩之間的 Jaccard 相似度，仍然需要 $N^2/2$ 次比對，而且每次比對還要處理兩個可能包含數萬個元素的大集合。

所以接下來我們要解決兩個問題：其一，如何讓「每次比對」變便宜——這是 MinHash 的工作，它把大集合壓縮成短簽章；其二，如何讓「比對次數」大幅減少——這是 LSH 的工作，它讓我們只需比對少數候選對。這兩步合起來，才真正破解了平方成長的詛咒。

14.4 MinHash 簽章

14.4.1 核心洞見

MinHash 是本章第一個精妙之處，它的核心是一個令人驚訝的數學事實。

作法是這樣：選定一個雜湊函式 h ，把集合中的每個元素都雜湊一遍，然後取「所有雜湊值中最小的那一個」，作為這個集合的一個 MinHash 值。用符號寫成 $h_{\min}(A) = \min\{h(x) : x \in A\}$ 。

關鍵定理是：對於兩個集合 A 與 B，它們的 MinHash 值相等的機率，恰好等於它們的 Jaccard 相似度。

$$P[h_{\min}(A) = h_{\min}(B)] = J(A, B)$$

式 14-3 MinHash 的核心定理

為什麼會這樣

直覺的證明如下。想像我們把 $A \cup B$ 中所有元素的雜湊值由小到大排列，然後問：排在最前面的那個元素，是誰？

因為雜湊值可視為隨機的，所以聯集中每個元素成為「最小」的機會均等。而 $h_{\min}(A)$ 與 $h_{\min}(B)$ 會相等，唯一的情況就是——這個最小的元素同時屬於 A 和 B，也就是它落在交集裡。

於是問題變成：「聯集中隨機挑一個元素，它落在交集裡的機率是多少？」答案顯然是 $|A \cap B| / |A \cup B|$ ，正是 Jaccard 相似度。定理得證——優雅得令人拍案。

14.4.2 從一個值到一組簽章

單一個 MinHash 值只能告訴我們「相等或不相等」，資訊量太少（相當於只做了一次隨機實驗）。所以實務上會用 n 個不同的雜湊函式，得到 n 個 MinHash 值，構成這份文件的「簽章」（signature）。

有了簽章之後，兩份文件的相似度就可以這樣估計：計算它們的簽章在多少個位置上相同，除以 n 。依大數法則， n 越大，這個估計值就越接近真實的 Jaccard 相似度。

這一步的價值極大。原本每份文件是一個可能有數萬個元素的大集合，現在被壓縮成 n 個整數（實務上 n 常取 100 到 200）。比較兩份文件，從「比對兩個大集合」變成「比對兩組一百個整數」——每次比對的成本大幅下降，而且所有簽章都能輕鬆放進記憶體。

叫獸提醒

MinHash 是「用隨機性換效率」的典範，這個思想在第伍篇的串流演算法（第 24 章的 Bloom Filter、Flajolet-Martin）會再度出現。它們的共同套路是：放棄精確的答案，改求一個「有數學保證的近似值」，換來記憶體與運算的巨幅節省。在大數據的世界裡，一個一秒算完、誤差 1% 的答案，往往遠比一個算三天的精確答案有價值。這是在演算法課上不太會被強調、但在業界極其重要的觀念轉換。

14.5 區域敏感雜湊 (LSH)

MinHash 解決了「每次比對很貴」，但比對次數仍然是 $N^2/2$ 。LSH 就是要解決這後半個問題——這是本章第二個精妙之處，也是整套方法的關鍵。

14.5.1 核心構想：讓相似的自己碰面

LSH 的想法很反直覺：與其去「找」相似的配對，不如設計一種分桶方式，讓相似的項目自己掉進同一個桶裡。之後我們只需檢查「同一個桶裡的配對」，其餘的連看都不用看。

關鍵在於這個雜湊必須是「區域敏感」的——與一般雜湊追求「均勻打散」相反，它要讓相似的輸入有高機率得到相同的輸出。

14.5.2 分帶技巧

具體做法稱為「分帶」（banding）。假設每份文件的簽章有 n 個值，我們把它切成 b 個「帶」（band），每帶包含 r 個值（因此 $n = b \times r$ ）。

接著，對每一個帶分別做雜湊分桶：若兩份文件在「某一個帶」上的 r 個值完全相同，它們就會在該帶的雜湊中掉進同一個桶。只要在「任何一個帶」上同桶，這兩份文件就成為「候選對」，值得進一步實際比對。

這個設計的巧妙之處在於：越相似的文件，越容易在至少一個帶上完全吻合；而不相似的文件，要在連續 r 個位置上都碰巧相同，機率極低。於是相似的自然浮現，不相似的自然被濾掉。下一節的公式會把這個機率精確算出來。

14.5.3 兩種錯誤

LSH 是近似方法，因此必然有兩類錯誤，你必須理解它們的性質。

- **偽陽性**：不相似的文件卻成了候選對。代價是多做了一些無謂的實際比對——浪費一點運算，但不影響正確性，因為後續的實際比對會把它們剔除。
- **偽陰性**：相似的文件卻沒被選為候選對，因而被永遠漏掉。這比偽陽性嚴重得多，因為它是「真正的遺漏」，後續沒有任何機會補救。

因此調整參數時的原則通常是：寧可容忍多一點偽陽性，也要盡量壓低偽陰性。這與許多檢測問題的取舍思維一致——寧可多篩檢幾個，也不要漏掉真正該找到的。

14.6 近似重複偵測實務

14.6.1 完整流程回顧

讓我們把整套流程串起來，你會看到它如何一步步破解平方成長的詛咒。

步驟	做什麼	解決了什麼
Shingling	把文件切成 k-gram 集合並雜湊	把文字轉為可用集合度量的形式
MinHash	以 n 個雜湊函式壓縮為簽章	讓每次比對從大集合變成短簽章
LSH 分帶	分成 b 帶各 r 值，分別雜湊分桶	讓比對次數從 $N^2/2$ 降為少數候選對
候選驗證	對候選對計算實際相似度	剔除偽陽性，確保結果正確

表 14-3 相似性探勘的四步流程

整套方法的總時間複雜度接近線性，而非平方——這正是它能處理數十億份文件的原因。

14.6.2 實務上的注意事項

- **先確認相似度的定義：**Jaccard 適合「集合重疊」型的相似，若你的問題本質是「向量夾角」或「編輯距離」，則需改用對應的 LSH 變體（如針對餘弦相似度的隨機投影）。
- **參數要依需求調整：** b 與 r 決定了門檻位置。先想清楚「相似度多少以上算重複」，再回推參數，而非隨便設一組。
- **候選驗證不可省略：**LSH 只負責篩出候選，實際的相似度仍須計算。省略這步會讓偽陽性直接混入結果。
- **注意資料傾斜：**若有大量近乎相同的文件，某些桶可能極度膨脹，造成第 7 章談過的熱點問題，需特別處理。

14.6.3 承先啟後

本章我們面對了大數據探勘的第一道難題——平方成長的詛咒，並學會了一套三段式的破解方法：Shingling 把文件變成集合（14.2）、Jaccard 給出明確的相似度定義（14.3）、MinHash 把大集合壓縮成短簽章並保有相似度資訊（14.4）、LSH 則以分帶技巧讓相似者自動聚在一起（14.5）。

我希望你從這一章帶走的，不只是這套演算法，還有它背後的思維：當精確解的成本無法承受時，設計一個「有數學保證的近似解」往往是唯一的出路。這個思維會貫穿整個第肆篇與第伍篇。

接下來，我們要面對另一類經典問題。本章問的是「哪兩個物件很像」，而下一章要問的是：「哪些東西經常一起出現？」買了尿布的人是不是也常買啤酒？產線上哪幾個參數異常，總是同時發生？這就是第 15 章的主題——頻繁項集與關聯規則探勘。

公式與流程：LSH 的 S 曲線

這是本章最重要的一條公式，也是全書我個人最喜歡的一條——它把「調參數」這件事，變成了一件可以精確計算的工程。

設兩份文件的真實 Jaccard 相似度為 s 。依式 14-3，它們的簽章在「任一個位置」相同的機率就是 s 。現在我們逐步推導它們成為候選對的機率。

逐步推導

第 1 步。 在某一個帶中， r 個位置「全部」相同的機率為 s^r 。

第 2 步。 反之，該帶「不完全相同」的機率為 $1 - s^r$ 。

第 3 步。 b 個帶「全都不相同」的機率為 $(1 - s^r)^b$ 。

第 4 步。 故「至少有一帶相同」，即成為候選對的機率為 $1 - (1 - s^r)^b$ 。

$$P(\text{候選對}) = 1 - (1 - s^r)^b$$

式 14-4 LSH 的候選對機率 (S 曲線)

這個函數畫出來是一條漂亮的 S 形曲線：相似度低時機率趨近於 0，相似度高時趨近於 1，而在中間某處急遽上升。這個急遽上升的位置，就是實質的「相似度門檻」，可用下式估計：

$$\text{門檻 } t \approx (1/b)^{1/r}$$

式 14-5 LSH 門檻的近似估計

我們以 $n = 100$ 、 $b = 20$ 、 $r = 5$ 為例（門檻約 0.55），計算不同相似度的文件成為候選對的機率。

真實相似度 s	s^r	$P(\text{候選對})$	判讀
0.2	0.00032	約 0.6%	幾乎必被濾掉（很好）
0.4	0.01024	約 18.6%	低機率通過
0.55	約 0.0503	約 64.1%	門檻附近，急遽上升
0.7	0.16807	約 97.5%	高機率被選出
0.9	0.59049	約 99.999%	幾乎必被選出（很好）

表 14-4 LSH 的候選對機率 ($n = 100$ 、 $b = 20$ 、 $r = 5$)

看這張表的頭尾兩列：相似度 0.2 的文件只有 0.6% 的機率被誤選（偽陽性極低），而相似度 0.9 的文件有 99.999% 的機率被選出（偽陰性極低）。中間的過渡則相當陡峭——這正是我們想要的行為。

叫獸提醒

式 14-5 給了你一個實用的操作方法：先想清楚「相似度多少以上算重複」，再回推 b 與 r 。想要門檻低一點（抓得寬鬆、寧可多抓）就增大 b 、減小 r ；想要門檻高一點（抓得嚴格）就反過來。另外還有一個要記住的規律： r 主要控制曲線的陡峭程度， b 主要控制門檻的位置。而在總數 $n = b \times r$ 固定的前提下，這兩者是互相拉扯的——這就是工程上的取捨，沒有兩全其美，只有想清楚你要什麼。

案例研討

案例一 三十萬份報告的抄襲檢查

回到叫獸開講的難題。三十萬份報告若用暴力法，依式 14-1 需比對約四百五十億次，即使每次一毫秒也要跑一年半。

改用本章的方法後：先以 $k = 4$ 的中文字元 Shingling 把每份報告轉為集合，再以 $n = 100$ 的 MinHash 壓縮成簽章（每份報告從數萬個 shingle 縮成 100 個整數，三十萬份的簽章總共也才三千萬個整數，輕鬆放進記憶體），最後以 $b = 20$ 、 $r = 5$ 做 LSH 分帶。

結果是：實際需要比對的候選對只有數萬對，整個流程在幾分鐘內完成，且依表 14-4，相似度 0.7 以上的抄襲有 97.5% 的機率被抓出來。從一年半到幾分鐘——這就是演算法設計的價值，遠勝過買更快的機器。

案例二 語料去重與大語言模型

第 1 章案例四我們提過，訓練大語言模型時，語料的去重比單純增加資料量更能提升模型品質。而網路抓取的語料中，充斥著大量「近似重複」的內容——同一則新聞被多家網站轉載、同一段文字出現在數千個頁面的頁尾。

這些近似重複若不移除，模型會對這些內容過度記憶，浪費模型容量也損害泛化能力。但要在數十億份文件中找出近似重複，正是最典型的平方成長問題——暴力法徹底不可行。

業界的標準做法正是本章的 MinHash 加 LSH。一個一九九七年前後為搜尋引擎去重而發展的演算法，三十年後成了訓練 AI 的關鍵基礎設施。這也印證了本書反覆的主張：技術會換名字，但原理會留下。

案例三 把振動訊號變成集合

某產線要偵測設備異常。工程師的做法是：把每段振動訊號依振幅切分成若干區間並編碼成符號序列，再對這串符號做 Shingling，得到一個集合——如此一來，一段訊號就變成了一份「文件」。

接著，他們把歷史上已知故障樣態的訊號集合建成索引，再以 LSH 讓新進的訊號與之比對。若某段新訊號與某個已知故障樣態高度相似，系統就發出告警。

這個案例展示了本章方法的可遷移性：只要能把物件轉換成「集合」，MinHash 與 LSH 就能派上用場，完全不限於文字。這也呼應了第 36 章要談的機器人與感測資料整合——把不同型態的資料轉換成統一的數學表示，是資料科學最核心的功夫之一。

案例四 參數設錯的代價

某團隊導入 LSH 做商品去重，設定 $n = 100$ 、 $b = 5$ 、 $r = 20$ 。上線後發現效果很差——大量明顯重複的商品沒被抓出來。

問題出在參數。依式 14-5， $b = 5$ 、 $r = 20$ 的門檻約為 $(1/5)^{(1/20)} \approx 0.923$ ——這意味著只有相似度超過約 0.92 的商品才會被有效抓出。而他們實際想抓的是相似度 0.7 左右的近似商品，這些幾乎全被漏掉，成為嚴重的偽陰性。

他們把參數改為 $b = 25$ 、 $r = 4$ （門檻約 $(1/25)^{(1/4)} \approx 0.447$ ）後，召回率大幅提升。這個案例說明式 14-5 不是紙上談兵——參數設定直接決定了系統抓得到什麼、抓不到什麼。而偽陰性的可怕之處在於：你不會收到任何錯誤訊息，只會默默地漏掉一堆東西，正如第 13 章談過的「安靜的失敗」。

實作活動

活動一 手算 Jaccard 與 MinHash

請以紙筆完成，親自體會 MinHash 定理。

步驟 1. 設 $A = \{a, b, c, d\}$ 、 $B = \{b, c, d, e\}$ ，計算 $J(A, B)$ 。

- 步驟 2.** 設計一個簡單的雜湊函式（例如 $h(x) = (3x + 1) \bmod 7$ ，並把 a 到 e 編號為 1 到 5），分別計算 $h_{\min}(A)$ 與 $h_{\min}(B)$ 。
- 步驟 3.** 換三個不同的雜湊函式重複上述步驟，統計「 $h_{\min}$ 相等」的比例。
- 步驟 4.** 比較這個比例與步驟 1 算出的 Jaccard 相似度，說明式 14-3 的意義（註：雜湊函式太少時誤差會很大，這正說明為何實務要用一百個以上）。

活動二 畫出你的 S 曲線

請以 Python 繪圖，直觀理解參數的影響。

- 步驟 1.** 以式 14-4 撰寫函式，輸入 s 、 b 、 r ，回傳候選對機率。
- 步驟 2.** 固定 $n = 100$ ，分別以 $(b, r) = (20, 5)$ 、 $(10, 10)$ 、 $(50, 2)$ 畫出三條 S 曲線（橫軸為 s 從 0 到 1）。
- 步驟 3.** 以式 14-5 計算三組參數的門檻，並標記在圖上，確認曲線的陡升位置是否吻合。
- 步驟 4.** 討論：哪一組參數最適合「抓相似度 0.8 以上的重複」？哪一組會產生最多偽陽性？

活動三 實作一個小型抄襲偵測器

請以 Python 實作完整的四步流程。準備十份短文（其中刻意讓兩三對內容高度相似），依序完成：Shingling（ k 自訂並說明理由）、MinHash 簽章（ $n = 100$ ）、LSH 分帶（自選 b 與 r 並說明依據）、以及候選對的實際 Jaccard 驗證。最後比較「LSH 找出的候選對數量」與「暴力法的 45 對」，並檢查是否有偽陰性（真正相似卻沒被選出的配對）。

延伸閱讀

- **《Mining of Massive Datasets》第 3 章：**Leskovec、Rajaraman 與 Ullman 著。本章的主要依據，對 Shingling、MinHash 與 LSH 有最完整的數學推導與範例，且全書可自官網免費取得。
- **Broder 的 MinHash 原始論文：**一九九七年為 AltaVista 搜尋引擎去重而發表，是式 14-3 的源頭，行文簡潔易讀。
- **LSH 的其他距離度量：**針對餘弦相似度的隨機投影（SimHash）、針對歐氏距離的 p -stable 分布等變體，適合在理解本章後延伸閱讀。
- **大語言模型語料去重的技術文章：**多數開源語料集（如 Common Crawl 的清理流程）都會說明其 MinHash 去重的參數設定，是案例二的實務延伸。

本章小結

1. 相似性探勘的核心難題是平方成長： N 個物件需比對約 $N^2/2$ 次（式 14-1），一億份文件的暴力比對需時逾百年，故必須設計能避開平方成長的演算法。
2. Shingling 把文件切成連續 k 個單位的重疊片段集合，將文字轉為集合； k 太小則所有文件都相似而失去鑑別力， k 太大則抓不到改寫過的相似，中文字元 k 常取 3 到 5。
3. Jaccard 相似度為交集除以聯集（式 14-2），以聯集為分母才能公平地把兩者規模差異納入考量；但僅有定義並未解決成本問題。
4. MinHash 取集合中所有元素雜湊值的最小者；其核心定理為兩集合 MinHash 值相等的機率恰等於 Jaccard 相似度（式 14-3），因為兩者相等的唯一情況是聯集中最小的元素落在交集裡。
5. 以 n 個雜湊函式構成簽章後，相似度可由「簽章相同的位置比例」估計；這把每份文件從數萬元素的集合壓縮為約一百個整數，使比對成本大幅下降且可全部放入記憶體。

6. LSH 以分帶技巧 ($n = b \times r$) 讓相似項目自動落入同桶：只要在任一帶上 r 個值全同即成為候選對；它把比對次數從 $N^2/2$ 降為少數候選對，是破解平方成長的關鍵。
7. LSH 的候選對機率為 $1 - (1 - s^r)^b$ (式 14-4)，呈 S 形曲線，門檻約為 $(1/b)^{1/r}$ (式 14-5)； r 主要控制陡峭度、 b 主要控制門檻位置，在 n 固定時兩者互相拉扯。
8. 偽陽性只是多做無謂比對（可由候選驗證剔除），偽陰性則是永久遺漏且不會有任何錯誤訊息，故調參原則是寧可容忍偽陽性也要壓低偽陰性；候選驗證步驟不可省略。

習題

一、觀念題

1. 為什麼相似性探勘不能用暴力兩兩比對？請以式 14-1 說明，並解釋為何「買更快的機器」無法解決此問題。
2. Shingling 的 k 值若取得太小或太大，各會產生什麼問題？請說明選擇 k 的原則。
3. 請以直覺說明「MinHash 值相等的機率等於 Jaccard 相似度」的原因。
4. 請比較 LSH 的偽陽性與偽陰性，說明為何調參時應優先壓低偽陰性。

二、計算題

1. 設 $A = \{1, 3, 5, 7, 9\}$ 、 $B = \{3, 5, 7, 11\}$ 。(一) 計算 $J(A, B)$ 。(二) 若有 200 個雜湊函式，依式 14-3 預期約有幾個位置的 MinHash 值會相同？
2. 某系統採 $n = 120$ 、 $b = 30$ 、 $r = 4$ 。(一) 以式 14-5 估算門檻 t 。(二) 以式 14-4 計算真實相似度 $s = 0.3$ 與 $s = 0.8$ 時成為候選對的機率。(三) 若想把門檻提高到約 0.7，在 n 不變的前提下， b 與 r 應如何調整？

三、思考與討論

1. 本章的方法建立在「把物件轉換成集合」之上。請就你熟悉的一種非文字資料（如影像、感測訊號、程式碼），設計一種轉換為集合的方式，並討論這樣轉換會保留什麼資訊、又會遺失什麼。
2. 本章指出「用可控的少量誤差換取巨大的效率提升」是大數據演算法的核心思想。請討論：在什麼樣的應用情境下，這種近似做法是不可接受的？此時該如何因應？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

依式 14-1，比對次數約為 $N^2/2$ ，隨資料量呈平方成長——資料量增加 100 倍，比對次數增加 10,000 倍（見表 14-2：一億份文件需約 158 年）。「買更快的機器」無法解決，是因為硬體效能的提升是線性或常數倍的（快 10 倍、100 倍），而問題規模的成長是平方的；當 N 持續增大，任何常數倍的加速都會被平方項迅速吞沒。唯一的出路是改變演算法的複雜度階數，使其接近線性——這正是 MinHash 與 LSH 所做的事。

第 2 題

k 太小：幾乎任何兩份文件都會共用大量 shingle（如中文 $k = 1$ 等於在比較「用了哪些字」，常用字人人都用），導致所有文件看起來都相似，失去鑑別力。k 太大：只有幾乎逐字相同的文件才共用 shingle，稍加改寫或調換詞序，相似度即降至接近零，抓不到改寫過的抄襲。選擇原則：k 要大到「隨機兩份文件共用某個 shingle 的機率很低」，且需考慮語言特性（中文單字資訊量高，字元 k 取 3 到 5；英文常取 5 到 9）與文件長度（長文件可取較大的 k）。

第 3 題

直覺推理如下：把 $A \cup B$ 中所有元素的雜湊值由小到大排列，因雜湊值可視為隨機，聯集中每個元素成為「最小」的機會均等。而 $h_{\min}(A)$ 與 $h_{\min}(B)$ 相等的唯一情況，是這個最小的元素同時屬於 A 與 B（即落在交集中）——若它只屬於其中一方，另一方的最小值必然是別的元素。故問題化為「從聯集中隨機挑一元素，落在交集的機率為何」，答案即 $|A \cap B| / |A \cup B| = J(A, B)$ 。

第 4 題

偽陽性：不相似的文件被選為候選對，代價僅是多做幾次無謂的實際比對，浪費少量運算，且後續的候選驗證會將其剔除，不影響最終結果的正確性。偽陰性：真正相似的文件未被選為候選對，因而被永久遺漏，後續沒有任何機會補救，且不會產生任何錯誤訊息（呼應第 13 章的「安靜的失敗」）。因兩者代價不對稱——偽陽性可補救、偽陰性不可補救——故調參原則是寧可容忍較多偽陽性，也要盡量壓低偽陰性。

二、計算題

第 1 題

$A = \{1, 3, 5, 7, 9\}$ 、 $B = \{3, 5, 7, 11\}$ 。

- (1) 交集 = $\{3, 5, 7\}$ ，大小為 3；聯集 = $\{1, 3, 5, 7, 9, 11\}$ ，大小為 6。故 $J(A, B) = 3 / 6 = 0.5$ 。
- (2) 依式 14-3，每個位置相同的機率為 0.5，故 200 個雜湊函式中預期約有 $200 \times 0.5 = 100$ 個位置相同。

延伸思考：這正是以簽章估計相似度的原理——只要數一數簽章有幾個位置相同再除以 n，就能估計 Jaccard 相似度，而不必回頭比對原始集合。

第 2 題

$n = 120$ 、 $b = 30$ 、 $r = 4$ 。

- (1) 門檻（式 14-5）： $t \approx (1/30)^{(1/4)}$ 。因 $1/30 \approx 0.0333$ ，開四次方約為 0.427，故門檻約 0.43。
- (2) $s = 0.3$ ： $s^r = 0.3^4 = 0.0081$ ； $P = 1 - (1 - 0.0081)^{30} = 1 - 0.9919^{30} \approx 1 - 0.783 \approx 0.217$ （約 21.7%）。
- (3) $s = 0.8$ ： $s^r = 0.8^4 = 0.4096$ ； $P = 1 - (1 - 0.4096)^{30} = 1 - 0.5904^{30} \approx 1 - 0.0000002 \approx 0.9999998$ （幾乎必定被選出）。
- (4) 提高門檻至約 0.7：門檻與 r 正相關、與 b 負相關，故應增大 r、減小 b。在 $n = 120$ 不變下可取 $b = 12$ 、 $r = 10$ ，則 $t \approx (1/12)^{(1/10)} \approx 0.78$ ；或取 $b = 20$ 、 $r = 6$ ，則 $t \approx (1/20)^{(1/6)} \approx 0.61$ 。可依實際需求在兩者間選擇（如取 $b = 15$ 、 $r = 8$ 得 $t \approx 0.71$ ，最接近目標）。

延伸思考：注意 $s = 0.3$ 時仍有約 21.7% 的機率成為候選對——這就是偽陽性。但它們會在候選驗證階段被剔除，代價僅是多做一些比對，屬可接受的成本。

三、思考與討論

第 1 題

答題應包含三部分：轉換方式、保留了什麼、遺失了什麼。範例一（感測訊號）：把振幅離散化為符號序列後做 Shingling（如案例三）——保留了「局部波形樣態的組合」，遺失了絕對數值大小與精確的時間點。範例二（程式碼）：以語法單元（token）序列做 k-gram——保留了程式結構與慣用寫法，遺失了變數命名與註解（這其實有利於抓出改名後的抄襲）。範例三（影像）：切成區塊後對每塊的特徵量化為視覺詞彙，構成集合——保留了局部紋理的組合，遺失了空間位置關係。周延的回答會指出：轉換必然是有損的，關鍵在於「刻意保留與你的相似性定義相關的資訊，並刻意丟棄無關的部分」——例如抓抄襲時丟棄變數名反而是優點。

第 2 題

不可接受的情境包括：其一，法律或稽核用途——如財務對帳、法定報表，任何近似誤差都可能構成違規，必須精確。其二，安全關鍵系統——如醫療診斷的最終判定、飛航或工控的安全連鎖，偽陰性可能造成人身危害。其三，金額或計次的最終結算——如薪資計算、庫存扣減，錯一筆就是錯。因應方式：其一，以近似方法做「初篩」，再對篩選結果進行精確驗證（這正是 LSH 加候選驗證的兩階段設計思想）。其二，調整參數使偽陰性趨近於零，寧可承受大量偽陽性與運算成本。其三，在可負擔的資料規模內直接使用精確演算法。其四，明確記錄並溝通誤差範圍，讓使用者知道結果的性質——絕不可讓近似值被誤當成精確值使用。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 14 章 相似性探勘：MinHash 與 LSH

台灣自編大學教科書

@prof

大數據分析與雲端運算

第肆篇 大數據探勘與分析演算法

Mining & Analytics Algorithms

第 15 章

頻繁項集與關聯規則探勘

Frequent Itemsets and Association Rule Mining

機器人叫獸（李世淵） 編著

叫獸開講

資料探勘領域有一個流傳了三十年的傳奇故事：某家超市分析銷售資料時，發現「尿布」與「啤酒」經常被一起購買。經理百思不解，深入調查後才明白——是年輕爸爸下班後被太太叫去買尿布，順手也給自己帶了幾罐啤酒。於是超市把啤酒擺到尿布旁邊，業績大增。

這個故事我要老實告訴你：它的真實性其實相當可疑，多數考證認為它經過了大幅美化，甚至可能根本沒發生過。但它之所以能流傳三十年，是因為它精準地說出了一件事——資料裡藏著人的行為模式，而這些模式往往連當事人自己都沒意識到。

上一章我們問的是「哪兩個東西很像」，這一章要問的是完全不同的問題：「哪些東西經常一起出現？」而且注意，這裡的「一起」不需要相像——尿布和啤酒一點也不像，但它們有關聯。

這個問題在你的專業領域同樣重要。產線上哪幾個參數異常，總是同時發生？哪些設備故障往往一前一後出現？機器人的哪幾種感測器讀數組合，預示著即將發生的失效？這些都是同一類問題——找出「經常同時出現的項目組合」。

而這一章的核心，是一個簡單到近乎廢話、卻威力驚人的洞見。我先賣個關子，只說它叫「單調性原理」，而正是它讓一個原本會組合爆炸到完全無法計算的問題，變得可以在幾遍掃描內解決。這是演算法設計的經典之作，值得你細細品味。

學習目標

研讀完本章之後，你應該能夠：

1. 說明購物籃模型的抽象結構，並舉出其在不同領域的應用。
2. 計算項集的支持度，以及關聯規則的信賴度與提升度。
3. 說明單調性原理，並解釋它為何能大幅削減候選項集。
4. 描述 A-Priori 演算法的逐層流程，並指出其瓶頸所在。
5. 說明 PCY 演算法如何利用閒置記憶體進一步剪枝。
6. 說明 SON 演算法的兩階段設計，及其為何適合分散式運算。

核心概念

核心概念	說明
購物籃模型	把資料抽象為「許多籃子，每籃裝著若干項目」的通用模型。
項集 (Itemset)	一組項目的集合，如 {尿布, 啤酒}。
支持度 (Support)	某項集出現在多少個籃子中，是判定「頻繁」的依據。
頻繁項集	支持度達到門檻的項集，是本章要找的目標。
信賴度 (Confidence)	在買了 A 的籃子中，也買了 B 的比例。
提升度 (Lift)	信賴度與 B 的整體出現率之比，衡量關聯是否真的有意義。
單調性原理	頻繁項集的所有子集必然也頻繁；反之，任一子集不頻繁則其超集必不頻繁。

表 15-1 本章核心概念一覽

15.1 購物籃分析與頻繁項集

15.1.1 購物籃模型的抽象威力

這個模型的名字來自超市，但它的抽象結構極為通用：有許多個「籃子」（basket），每個籃子裡裝著若干「項目」（item）。我們要找的，是那些「經常一起出現在同一個籃子裡」的項目組合。

關鍵在於，「籃子」與「項目」可以代表非常多不同的東西。

領域	籃子代表	項目代表	想找出什麼
零售	一次購買交易	商品	常一起被購買的商品
智慧製造	一次生產批次	異常參數或事件	常同時發生的異常組合
醫療	一位病患	症狀或診斷	常併發的症狀組合
網頁分析	一次瀏覽歷程	造訪的頁面	常被一起瀏覽的內容
文件探勘	一篇文件	詞彙	常共同出現的概念

表 15-2 購物籃模型在不同領域的對應

這種「換個名詞就能套用」的特性，正是抽象模型的價值。學會了購物籃分析，你等於同時學會了上述所有問題的解法。

15.1.2 支持度與頻繁項集

要定義「經常一起出現」，我們需要一個量化指標，這就是支持度（support）。某個項集 I 的支持度，是包含 I 的籃子數量（或佔全體籃子的比例）。

接著我們設定一個「支持度門檻」（support threshold），達到門檻的項集就稱為「頻繁項集」。門檻的選擇是一門藝術：設得太高，只會找到幾個顯而易見的組合；設得太低，則會冒出成千上萬個瑣碎而無意義的組合，且運算量暴增。

舉例來說，若有 1,000 個交易，門檻設為 1%（即 10 個籃子），那麼任何出現在 10 個以上籃子中的項集，都算是頻繁項集。

15.2 支持度、信賴度與提升度

15.2.1 從頻繁項集到關聯規則

找到頻繁項集只是第一步。我們真正想要的，往往是形如「買了 A 的人，也會買 B」的規則，寫成 $A \rightarrow B$ 。這稱為關聯規則（association rule）。

衡量一條規則好不好，最基本的指標是信賴度（confidence）——在所有包含 A 的籃子中，同時也包含 B 的比例：

$$\text{confidence}(A \rightarrow B) = \text{support}(A \cup B) / \text{support}(A)$$

式 15-1 關聯規則的信賴度

例如若 100 個籃子含有尿布，其中 60 個也含有啤酒，則規則「尿布 $\rightarrow$ 啤酒」的信賴度為 0.6。

15.2.2 信賴度的陷阱

信賴度看起來很合理，但它有一個嚴重的盲點，這是本節最重要的觀念。

假設某超市有 90% 的籃子都含有「塑膠袋」。那麼幾乎任何規則「X → 塑膠袋」的信賴度都會高達 0.9 左右——但這根本不是什麼有價值的發現，因為不管你買什麼，本來就有九成機率會拿塑膠袋。信賴度高，只是因為塑膠袋本身太普遍。

換句話說，信賴度沒有考慮「B 本身有多常出現」。要修正這個盲點，我們需要提升度。

15.2.3 提升度：關聯是否真的有意義

提升度 (lift) 把 B 的整體出現率納入了分母：

$$\text{lift}(A \rightarrow B) = \text{confidence}(A \rightarrow B) / \text{support}(B)$$

式 15-2 關聯規則的提升度

它的意義是：「買了 A 之後買 B 的機率」相對於「隨便一個人買 B 的機率」，提升了幾倍。判讀方式如下：

提升度	意義	解讀
> 1	正相關	A 與 B 的同時出現超出隨機預期，是有價值的關聯。
≈ 1	獨立	兩者無關，同時出現純屬巧合（如前述的塑膠袋）。
< 1	負相關	買了 A 反而較少買 B，也可能是有用的資訊（如互為替代品）。

表 15-3 提升度的判讀

叫獸提醒

這裡有個必須強調的觀念，也是第 1 章 1.2.4 節談過的迷思：關聯不等於因果。就算「尿布 → 啤酒」的提升度高達 3.0，也不代表「買尿布會導致買啤酒」。可能是共同的第三因素（有嬰兒的年輕家庭），也可能純粹是時間巧合。資料探勘能告訴你「這兩件事常一起發生」，但「為什麼」需要領域知識與進一步的驗證。把關聯當成因果，是資料分析最常見也最危險的錯誤之一。

15.3 A-Priori 演算法

15.3.1 組合爆炸的難題

現在來看計算的困難。若商店有 n 種商品，可能的二元組有 $C(n, 2) = n(n-1)/2$ 種。這個數字成長得多快？下一節的公式會完整計算，這裡先給你一個震撼：一萬種商品，二元組就有約五千萬種。

而三元組呢？ $C(10000, 3)$ 約為一千六百億種。若要為每一種可能的組合都計數，光是記憶體就完全放不下——這才是頻繁項集探勘真正的瓶頸：不是掃描資料太慢，而是候選組合太多，計數器裝不下。

15.3.2 單調性原理：一個簡單卻致命的洞見

A-Priori 演算法的全部威力，來自一個簡單到你會覺得「這不是廢話嗎」的觀察：

若一個項集是頻繁的，則它的所有子集也必然是頻繁的。等價地說：若某個項集不頻繁，則包含它的所有超集也必定不頻繁。

為什麼成立？因為若一個籃子包含 $\{A, B, C\}$ ，它必然也包含 $\{A, B\}$ 。所以 $\{A, B\}$ 出現的次數，至少與 $\{A, B, C\}$ 一樣多——子集的支持度永遠大於或等於超集的支持度。

這句廢話般的觀察，威力在於它的「反面用法」：如果我發現 $\{\text{啤酒}\}$ 這個單項不頻繁，那我就完全不必去檢查 $\{\text{啤酒}, \text{尿布}\}$ 、 $\{\text{啤酒}, \text{麵包}\}$ 、 $\{\text{啤酒}, \text{牛奶}, \text{咖啡}\}$ ……所有包含啤酒的組合，全部可以直接跳過，連數都不用數。

在真實的商店中，絕大多數商品都不頻繁（多數商品乏人問津），因此這一刀砍下去，候選組合的數量往往減少幾個數量級。這就是所謂的「剪枝」（pruning）。

15.3.3 逐層搜尋的流程

A-Priori 依據單調性原理，採取「逐層」（level-wise）的策略：先找出所有頻繁的一元項集，再由它們產生二元候選，如此逐層往上。

- 第 1 步。** 第一遍掃描：計算每個單一項目的出現次數，篩出頻繁的一元項集 L_1 。
- 第 2 步。** 產生候選：僅由 L_1 中的項目兩兩組合，產生二元候選集 C_2 （不在 L_1 中的項目直接排除）。
- 第 3 步。** 第二遍掃描：計算 C_2 中各候選的支持度，篩出頻繁二元項集 L_2 。
- 第 4 步。** 產生候選：由 L_2 產生三元候選 C_3 ，且要求其所有二元子集都必須在 L_2 中（進一步剪枝）。
- 第 5 步。** 重複：如此逐層進行，直到某一層沒有頻繁項集為止。

注意第 4 步的額外剪枝：一個三元候選 $\{A, B, C\}$ 若要頻繁，它的三個二元子集 $\{A, B\}$ 、 $\{A, C\}$ 、 $\{B, C\}$ 都必須頻繁。只要有一個不在 L_2 中，這個候選就可以直接淘汰，不必等到掃描才發現。

15.3.4 A-Priori 的瓶頸

A-Priori 雖然巧妙，但仍有明顯的限制。

- **需要多遍掃描：**找到 k 元頻繁項集需要掃描資料 k 遍。若資料在磁碟上且量極大，每一遍都是可觀的 I/O 成本。
- **第二遍最吃緊：**第一遍只需為每個項目留一個計數器（記憶體壓力小），但第二遍要為所有二元候選計數，這通常是記憶體的最大瓶頸。
- **候選仍可能太多：**當支持度門檻設得較低時， L_1 很大，由它產生的 C_2 依然可能爆量。

其中第二點特別關鍵——它直接催生了下一節的 PCY 演算法。

15.4 PCY 演算法與其改良

15.4.1 被浪費的記憶體

PCY 演算法（以三位發明者 Park、Chen、Yu 命名）源自一個敏銳的觀察：在 A-Priori 的第一遍掃描中，我們只需要為每個單一項目維護一個計數器。若商品有一萬種，那就只用了一萬個計數器——而機器的記憶體可能有好幾 GB，絕大部分是閒著的。

PCY 的想法是：既然記憶體閒著，何不在第一遍掃描時「順便」做點別的事，好讓第二遍能輕鬆一些？

15.4.2 用雜湊桶預先剪枝

具體做法是：在第一遍掃描時，除了為每個項目計數，同時也把每個「二元組」雜湊到一個大陣列的桶（bucket）中，並為每個桶累計計數。

注意，我們不是為每個二元組單獨計數（那樣記憶體就爆了），而是讓許多不同的二元組共用同一個桶——桶的計數是「所有雜湊到此桶的二元組出現次數的總和」。

第一遍結束後，關鍵推論登場：如果某個桶的總計數都還沒達到支持度門檻，那麼雜湊到這個桶的「每一個」二元組，其個別計數必然也達不到門檻——因為個別計數不可能超過總和。於是這些二元組全部可以直接淘汰。

於是在第二遍掃描時，只有「兩個項目都頻繁」且「所在的桶是頻繁桶」的二元組才需要計數。這往往能把 C_2 的規模再削減一大截，讓原本裝不下的第二遍變得可行。

叫獸提醒

PCY 的思路值得你細細品味：它沒有發明新的數學，只是注意到「第一遍有閒置資源」這個工程細節，就換來了實質的效能突破。這在系統設計中是很常見的模式——瓶頸不在演算法本身，而在資源的不均衡使用。下次你調校系統時，不妨也問問：有沒有哪個階段的資源在閒置？能不能提前做點事，讓後面的瓶頸鬆一些？順帶一提，PCY 用「一個聚合的計數來否定所有個體」的技巧，與第 24 章的 Bloom Filter 有異曲同工之妙，屆時你會再遇到這個思想。

15.4.3 多階段與多雜湊的改良

PCY 之後還有兩種常見的改良方向。

- **多階段（Multistage）**：在第一遍與第二遍之間，再插入一遍掃描，用「不同的雜湊函式」對通過第一輪篩選的二元組再分桶一次。兩次不同的雜湊都通過，才進入計數階段，剪枝更徹底。代價是多一遍掃描。
- **多雜湊（Multihash）**：在同一遍掃描中，同時使用兩個較小的雜湊表（而非一個大的）。只有在兩個表中都落於頻繁桶的二元組才保留。省下一遍掃描，但每個表較小、碰撞較多，效果視資料而定。

這兩者的取舍很典型：一個用時間換效果，一個用效果換時間。實務上該選哪個，取決於你的瓶頸是 I/O 還是記憶體。

15.5 SON 與平行化演算法

15.5.1 把資料切開來算

前述演算法都假設資料能被完整掃描數遍。但當資料量大到必須分散在數百台機器上時，我們需要一個天生適合平行化的方法——這就是 SON 演算法（同樣以三位發明者 Savasere、Omiecinski、Navathe 命名）。

SON 的核心想法建立在一個關鍵觀察上：如果一個項集在「整體資料」中是頻繁的，那麼當我們把資料切成若干塊時，它至少必須在「某一塊」中也是頻繁的。

為什麼？用反證法：假設它在每一塊中都不頻繁，也就是在每塊中的佔比都低於門檻，那麼把所有塊加總起來，它的整體佔比也必然低於門檻——與「整體頻繁」矛盾。故得證。

15.5.2 兩階段流程

依據這個觀察，SON 設計了漂亮的兩階段流程，而且兩階段都能完美平行化。

第一階段：找出候選

把資料切成 m 塊，每台機器各拿一塊，在自己那塊上用任何演算法（如 A-Priori）找出「局部頻繁項集」，但門檻要按比例縮小（若整體門檻是 s ，每塊的門檻就設為 s/m ）。接著把所有機器找到的局部頻繁項集聯集起來，作為「全域候選集」。

依前述觀察，真正的全域頻繁項集必定包含在這個候選集中——不會有遺漏（沒有偽陰性）。

第二階段：驗證候選

再掃描資料一次，這次只為候選集中的項集計數，把各機器的局部計數加總，篩出真正達到全域門檻的項集。這一步剔除了第一階段可能產生的偽陽性（在某塊中頻繁、但在整體中不頻繁的項集）。

SON 的優點是只需兩遍掃描，且兩階段都能自然對應到第 9 章的 MapReduce：第一階段的 Map 在各分區找局部頻繁項集、Reduce 取聯集；第二階段的 Map 為候選計數、Reduce 加總篩選。這使它成為分散式環境下的標準選擇。

15.6 關聯規則的產生與評估

15.6.1 由頻繁項集產生規則

找到頻繁項集後，產生關聯規則相對簡單。對每個頻繁項集 I ，把它拆成兩個非空的部分 A 與 $I-A$ ，形成規則 $A \rightarrow (I-A)$ ，然後計算信賴度，保留達到信賴度門檻的規則。

例如頻繁項集 {尿布, 啤酒, 麵包} 可產生多條規則：{尿布} $\rightarrow$ {啤酒, 麵包}、{尿布, 啤酒} $\rightarrow$ {麵包}、{尿布, 麵包} $\rightarrow$ {啤酒} 等等。

這裡也有一個剪枝技巧：若規則 $A \rightarrow (I-A)$ 的信賴度不足，那麼把更多項目從左邊移到右邊（例如從 {尿布, 啤酒} $\rightarrow$ {麵包} 變成 {尿布} $\rightarrow$ {啤酒, 麵包}），信賴度只會更低。因此可以及早停止，不必窮舉所有拆法。

15.6.2 規則太多怎麼辦

實務上最常見的困擾，不是找不到規則，而是找到太多規則——動輒數千條，多數瑣碎無用。幾種常見的因應方式：

- **提高門檻：**同時調高支持度與信賴度門檻，但要注意可能因此漏掉罕見卻珍貴的規則。
- **用提升度篩選：**只保留提升度顯著大於 1 的規則，濾掉那些「因為 B 本來就很常見」的假關聯。

- **找出封閉或極大項集**：若 $\{A,B\}$ 與 $\{A,B,C\}$ 支持度相同，則前者的資訊已被後者涵蓋，可只保留後者，大幅減少輸出。
- **納入領域知識**：由專家判斷哪些規則有實務意義。演算法能找出模式，但「哪些模式值得行動」需要人的判斷。

15.6.3 承先啟後

本章我們從購物籃這個通用模型出發（15.1），學會了以支持度、信賴度與提升度來衡量關聯（15.2），並見識了 A-Priori 如何以一個「廢話般」的單調性原理，把組合爆炸的問題變得可解（15.3）；接著看到 PCY 如何利用閒置記憶體進一步剪枝（15.4），以及 SON 如何以兩階段設計完美適配分散式運算（15.5）。

回顧第 14 章與本章，你會發現一個共同的模式：面對大數據，關鍵往往不是「算得更快」，而是「聰明地不算」——LSH 讓我們不必比對所有配對，單調性原理讓我們不必檢查所有組合。這種「大幅削減搜尋空間」的思維，是資料探勘演算法的共同靈魂。

接下來我們要轉向另一類經典問題。前兩章處理的都是「兩兩之間的關係」或「項目的共現」，但如果我們想問的是：這一大堆資料，能不能自然地分成幾群？哪些顧客的行為模式相似到可以歸為一類？哪些感測訊號可以被視為同一種運轉狀態？這就是第 16 章的主題——大規模分群演算法。

公式與流程：組合爆炸與剪枝的威力

A-Priori 的價值，全在於它砍掉了多少候選。這一節我們把「爆炸」與「剪枝」都算成具體數字，你就會明白單調性原理為何是救命的。

候選項集的組合爆炸

若共有 n 種項目，則可能的 k 元項集數量為組合數：

$$\text{候選數} = C(n, k) = n! / [k!(n - k)!]$$

式 15-3 k 元候選項集的數量

其中二元組的情形最為關鍵（因為它是 A-Priori 的記憶體瓶頸），可簡化為：

$$C(n, 2) = n(n - 1) / 2 \approx n^2 / 2$$

式 15-4 二元候選項集的數量

下表顯示這個數字如何隨項目種類數暴增。

項目種類 n	二元組 $C(n, 2)$	三元組 $C(n, 3)$
100	4,950	161,700
1,000	499,500	約 1.66 億
10,000	約 5,000 萬	約 1,666 億
100,000	約 50 億	約 1.67×10^{14}

表 15-4 候選項集的組合爆炸

一萬種商品（一家中型超市的規模）就有五千萬個二元候選。若每個計數器佔 4 位元組，光是二元組的計數器就需要約 200 MB；而三元組的一千六百億種，則需要約 666 GB——完全不可行。

剪枝後的候選數

現在看單調性原理的威力。設 n 種項目中，只有 f 種是頻繁的（ f 通常遠小於 n ）。依單調性原理只有這 f 種項目能組成頻繁的二元組，故剪枝後的候選數為：

$$\text{剪枝後候選數} = C(f, 2) = f(f - 1) / 2$$

式 15-5 剪枝後的二元候選數

削減倍數則約為：

$$\text{削減倍數} \approx (n / f)^2$$

式 15-6 剪枝的削減倍數

以 $n = 10,000$ 、其中只有 $f = 500$ 種商品達到門檻為例：

情況	二元候選數	計數器記憶體（每個 4 位元組）
不剪枝 $C(10000, 2)$	約 5,000 萬	約 200 MB
剪枝後 $C(500, 2)$	124,750	約 0.5 MB
削減倍數	約 400 倍	$(10000 / 500)^2 = 400$

表 15-5 單調性剪枝的效果（ $n = 10,000$ 、 $f = 500$ ）

叫獸提醒

看式 15-6 那個平方——這是本節最該記住的一點。剪枝的效益不是線性的，而是「頻繁比例」的平方倒數。若只有 5% 的項目頻繁，候選數就減為原本的四分之一；若只有 1% 頻繁，就減為一萬分之一。而真實世界的資料幾乎都符合這個特性（少數熱門商品加上長長的冷門尾巴），這正是 A-Priori 能實際運作的根本原因。演算法之所以有效，往往是因為它抓住了資料的某個真實性質——而不是因為它在數學上很花俏。

案例研討

案例一 塑膠袋的假象

某超市的分析團隊興奮地報告：他們找到了數百條信賴度超過 0.85 的高信賴度規則！例如「買牙刷 → 買塑膠袋」信賴度 0.88、「買醬油 → 買塑膠袋」信賴度 0.91。

但這些規則毫無價值。因為該超市有約 90% 的交易都會購買塑膠袋——不管你買什麼，本來就有九成機率會買塑膠袋。依式 15-2，這些規則的提升度約為 $0.88 / 0.90 \approx 0.98$ ，接近 1，代表兩者根本獨立。

這個案例是 15.2.2 節的直接印證：只看信賴度會被「本來就很常見的項目」所誤導，必須用提升度來校正。實務上有一個簡單的檢查習慣——當你發現某個項目出現在大量規則的右側時，先去看看它的整體支持度是不是本來就很高。

案例二 產線異常的共現樣態

某工廠想了解設備異常之間的關聯。他們把「每一次生產批次」當作籃子，把「該批次中發生的各種異常事件代碼」當作項目，直接套用購物籃分析。

結果找到了幾條高提升度的規則，例如「主軸溫度異常 → 表面粗糙度超標」的提升度達 4.2。這意味著當主軸溫度異常時，出現表面粗糙度問題的機率是平時的四倍多。

有了這個發現，維護團隊就能在偵測到溫度異常時，主動加強後續的品質抽檢，而不必等到成品檢驗才發現問題。這個案例展示了表 15-2 的抽象威力——完全相同的演算法，只是把「商品」換成「異常代碼」，就能用於智慧製造。但團隊也謹記叫獸提醒：這是關聯而非因果，實際的機理仍需工程人員驗證。

案例三 記憶體裝不下的第二遍

某電商有約兩萬種商品，資料團隊執行 A-Priori 時，第一遍掃描順利完成，第二遍卻因記憶體不足而失敗。

原因正是 15.3.4 節的瓶頸：第一遍只需兩萬個計數器（微不足道），但第二遍的二元候選依式 15-4 高達約兩億個，記憶體根本裝不下。他們最初的直覺是「加記憶體」，但成本高昂。

改用 PCY 之後問題迎刃而解：第一遍掃描時，利用原本閒置的記憶體額外建立雜湊桶並計數；第二遍時，只有「兩項都頻繁」且「所在桶為頻繁桶」的二元組才需計數，候選量減少了兩個數量級，順利在既有硬體上完成。這個案例說明：面對資源瓶頸，「換個演算法」往往比「買更多硬體」更有效，也更便宜。

案例四 從單機到叢集

某零售集團的交易資料達數 TB，存放在 HDFS 上，單機根本無法完整掃描。他們採用 SON 演算法搭配 Spark 實作。

第一階段：資料被切成數百個分區，每個分區在各自的執行器上以 A-Priori 找出局部頻繁項集（門檻按比例縮小），再由 Reduce 取聯集得到全域候選。第二階段：再掃描一次，各分區為候選計數，最後加總篩選。

整個流程只需兩遍掃描，且完全平行。這個案例把第參篇與本章串了起來——SON 之所以是分散式環境的標準選擇，正是因為它的兩階段結構天生就是 MapReduce 的形狀。也再次印證第 9 章的觀念：好的分散式演算法，往往是先想清楚「怎麼切開才能各自算、再合併」。

實作活動

活動一 手算支持度、信賴度與提升度

給定五個購物籃：{奶, 麵, 蛋}、{奶, 麵}、{奶, 蛋}、{麵, 蛋}、{奶, 麵, 蛋, 果}。請完成：

- 步驟 1. 計算每個單項的支持度（以籃子比例表示）。
- 步驟 2. 計算項集 {奶, 麵} 的支持度。
- 步驟 3. 計算規則「奶 → 麵」的信賴度與提升度。
- 步驟 4. 判讀：這條規則有意義嗎？請依表 15-3 說明理由。

活動二 實作 A-Priori 並觀察剪枝

請以 Python 實作 A-Priori 的前兩層，親眼看見剪枝的效果。

- 步驟 1. 產生或取得一份購物籃資料（可用開放資料集，或以隨機方式產生，讓少數項目較常出現）。
- 步驟 2. 第一遍：計算各項目支持度，篩出頻繁一元項集 L_1 ，記錄項目總數 n 與頻繁數 f 。
- 步驟 3. 分別計算「不剪枝」的候選數 $C(n, 2)$ 與「剪枝後」的 $C(f, 2)$ ，並以式 15-6 驗證削減倍數是否接近 $(n/f)^2$ 。

步驟 4. 完成第二遍掃描，找出頻繁二元項集，並產生信賴度達門檻的關聯規則。

活動三 設計一個非零售的購物籃分析

請參考表 15-2，為你熟悉的一個領域設計購物籃分析。明確定義：什麼是「籃子」？什麼是「項目」？你希望找到什麼樣的關聯？接著說明你會如何設定支持度門檻（太高與太低各會有什麼問題），以及找到規則後，你會用什麼方式判斷它是「有價值的發現」還是「塑膠袋式的假象」。若你的領域是智慧製造或機器人，可參考案例二的做法。

延伸閱讀

- 《Mining of Massive Datasets》第 6 章：Leskovec 等人著。本章的主要依據，對 A-Priori、PCY、多階段與 SON 有完整的推導與範例。
- Agrawal 與 Srikant 的 A-Priori 原始論文：一九九四年發表，是單調性原理與逐層搜尋的源頭，篇幅適中且易讀。
- FP-Growth 演算法：一種以樹狀結構避免產生候選集的替代方法，只需兩遍掃描，適合在理解 A-Priori 後延伸比較。
- 關聯規則評估指標的比較文獻：除信賴度與提升度外，還有 conviction、leverage 等指標，各有適用場景，可作為 15.6 節的深化。

本章小結

1. 購物籃模型把資料抽象為「許多籃子、每籃若干項目」，其威力在於通用性——只要更換籃子與項目的定義，即可套用於零售、製造異常、醫療併發症、網頁瀏覽與文件探勘等領域。
2. 支持度衡量項集出現的頻繁程度，達到門檻者稱頻繁項集；信賴度為 $\text{support}(A \cup B) / \text{support}(A)$ （式 15-1），但它會被「本身就很常見的項目」誤導。
3. 提升度為信賴度除以 $\text{support}(B)$ （式 15-2），修正了信賴度的盲點：大於 1 為正相關、約等於 1 表獨立、小於 1 為負相關。但關聯永遠不等於因果。
4. 單調性原理指出「頻繁項集的所有子集必然頻繁」，其反面用法（子集不頻繁則超集必不頻繁）使 A-Priori 得以大量剪枝；A-Priori 採逐層搜尋，但需多遍掃描，且第二遍的二元候選計數是記憶體瓶頸。
5. PCY 利用第一遍掃描時閒置的記憶體，額外把二元組雜湊分桶計數；因「桶的總計數未達門檻則桶內所有二元組必未達門檻」，可在第二遍前預先剪除大量候選。多階段與多雜湊為其進一步改良。
6. SON 基於「全域頻繁必在某分塊中局部頻繁」的觀察，設計兩階段流程：先各分塊找局部頻繁項集取聯集為候選，再掃描一次驗證全域支持度；只需兩遍掃描且天生契合 MapReduce。
7. 由頻繁項集產生規則時可利用信賴度的單調性及早停止；規則過多時可提高門檻、以提升度篩選、只保留極大或封閉項集，並輔以領域知識判斷實務價值。
8. 二元候選數為 $C(n, 2) \approx n^2 / 2$ （式 15-4），一萬種商品即約五千萬個候選；剪枝後為 $C(f, 2)$ ，削減倍數約 $(n/f)^2$ （式 15-6）——因效益呈平方放大，且真實資料多為「少數熱門加長尾」，A-Priori 才得以實際可行。

習題

一、觀念題

1. 請說明購物籃模型為何具有高度通用性，並就智慧製造領域舉一個具體的對應方式。

2. 為什麼只看信賴度可能得到錯誤結論？請以「塑膠袋」為例說明，並解釋提升度如何修正此問題。
3. 請說明單調性原理的內容，並解釋它的「反面用法」為何能大幅削減候選項集。
4. PCY 演算法的核心洞見是什麼？請說明它為何能在不增加掃描次數的情況下改善 A-Priori 的瓶頸。

二、計算題

1. 某超市共有 5,000 種商品，其中僅 200 種達到支持度門檻。（一）請以式 15-4 計算未剪枝的二元候選數；（二）以式 15-5 計算剪枝後的候選數；（三）以式 15-6 計算削減倍數，並驗證是否與實際比值相符。
2. 某商店共 1,000 筆交易，其中 250 筆含有商品 B、100 筆含有商品 A、40 筆同時含有 A 與 B。請計算：（一）A 與 B 的支持度；（二）規則 $A \rightarrow B$ 的信賴度；（三）提升度，並依表 15-3 判讀這條規則是否有意義。

三、思考與討論

1. 本章與第 14 章都以「大幅削減搜尋空間」為核心思想（LSH 的分桶、A-Priori 的剪枝）。請比較兩者的削減機制有何異同，並說明它們各自付出了什麼代價。
2. 假設你為工廠找出了一條提升度 4.0 的規則「參數 X 異常 $\rightarrow$ 產品不良」。請討論：在把它變成實際的管理措施之前，你還需要確認哪些事？（提示：回顧關聯與因果的區別）

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

通用性來自其抽象結構——只要問題能被表述為「許多籃子，每籃裝著若干項目，想找常一起出現的項目組合」，即可套用同一套演算法。智慧製造的對應：以「一次生產批次」或「一個時間窗口」為籃子，以「該期間發生的異常事件代碼或超標參數」為項目，即可找出「常同時發生的異常組合」（如案例二的主軸溫度異常與表面粗糙度超標）。亦可以「一台設備的一次維修紀錄」為籃子、「更換的零件」為項目，找出常一起損壞的零件組合。

第 2 題

因為信賴度未考慮「B 本身有多常出現」。以塑膠袋為例：若 90% 的交易都含塑膠袋，則幾乎任何規則「 $X \rightarrow$ 塑膠袋」的信賴度都約為 0.9，看似極高卻毫無資訊價值——不管買什麼本來就有九成機率買塑膠袋。提升度以 $\text{support}(B)$ 為分母（式 15-2），把 B 的整體出現率納入校正：上例的提升度約 $0.88 / 0.90 \approx 0.98$ ，接近 1，正確地顯示兩者其實獨立。故提升度衡量的是「相對於隨機預期提升了幾倍」，而非絕對的共現比例。

第 3 題

單調性原理：若一項集頻繁，則其所有子集必然也頻繁。成立原因是包含 $\{A, B, C\}$ 的籃子必然也包含 $\{A, B\}$ ，故子集的支持度必大於或等於超集。反面用法（逆否命題）：若某項集不頻繁，則包含它的所有超集必定也不頻繁。威力在於——只要發現某個單項不頻繁，所有包含它的組合（無論幾

元) 都可直接跳過、連計數都不必，這使候選數從 $C(n,2)$ 降為 $C(f,2)$ 。因真實資料多為「少數熱門項目加上長長的冷門尾巴」， f 遠小於 n ，故削減幅度極大。

第 4 題

核心洞見：A-Priori 第一遍掃描時只需為每個項目維護一個計數器（如一萬種商品僅一萬個計數器），機器的大部分記憶體處於閒置狀態——既然閒著，何不順便做點事讓第二遍輕鬆些。做法：第一遍同時把每個二元組雜湊到桶中並累計桶計數（多個二元組共用一桶，故記憶體可控）。關鍵推論：若某桶的總計數未達門檻，則雜湊到該桶的每個二元組個別計數必然也未達門檻（個別不可能超過總和），故可整批剔除。因此第二遍只需為「兩項皆頻繁且位於頻繁桶」的二元組計數，大幅降低記憶體需求，且未增加掃描次數。

二、計算題

第 1 題

$n = 5,000$ 、 $f = 200$ 。

- (1) 未剪枝（式 15-4）： $C(5000, 2) = 5000 \times 4999 / 2 = 12,497,500$ （約 1,250 萬）。
- (2) 剪枝後（式 15-5）： $C(200, 2) = 200 \times 199 / 2 = 19,900$ 。
- (3) 削減倍數（式 15-6）： $(n/f)^2 = (5000/200)^2 = 25^2 = 625$ ；實際比值 = $12,497,500 \div 19,900 \approx 628$ ，與估計值 625 相當接近（微小差異來自公式中的 -1 項）。

延伸思考：僅 4% 的商品頻繁，候選數就減為約六百分之一。若計數器各佔 4 位元組，記憶體需求從約 50 MB 降至約 0.08 MB。

第 2 題

總交易 1,000 筆；含 B 者 250 筆、含 A 者 100 筆、同時含 A 與 B 者 40 筆。

- (1) 支持度： $\text{support}(A) = 100/1000 = 0.1$ ； $\text{support}(B) = 250/1000 = 0.25$ ； $\text{support}(A \cup B) = 40/1000 = 0.04$ 。
- (2) 信賴度（式 15-1）： $\text{support}(A \cup B) / \text{support}(A) = 0.04 / 0.1 = 0.4$ 。
- (3) 提升度（式 15-2）： $\text{信賴度} / \text{support}(B) = 0.4 / 0.25 = 1.6$ 。

判讀：提升度 1.6 大於 1，屬正相關——買了 A 之後買 B 的機率，是隨機情況的 1.6 倍，超出隨機預期，故這是一條有意義的規則。值得注意的是，其信賴度僅 0.4 看似不高，但因 B 的整體出現率本就只有 0.25，故 0.4 已屬顯著提升。這正說明：不能只看信賴度的絕對值，必須搭配提升度判讀。

三、思考與討論

第 1 題

相同之處：兩者都放棄「窮舉所有可能」，改以某種結構性質快速排除大量不可能的候選，使複雜度從平方或指數降至可接受的範圍；且都體現「聰明地不算」勝過「算得更快」的思想。相異之處：LSH 的削減是「機率性」的——利用相似項高機率同桶的性質分桶，其正確性只有機率保證，代價是可能產生偽陰性（真正相似卻被漏掉），故需謹慎調參。A-Priori 的削減是「確定性」的——單調性原理是嚴格成立的數學事實，剪掉的候選保證不可能頻繁，故不會有偽陰性，代價則是需要多遍掃描資料（I/O 成本）與逐層進行的時間。周延的回答會指出：LSH 以正確性換效率，A-Priori 以掃描次數換效率，取舍的維度不同。

第 2 題

需確認的事項至少包括：其一，因果方向與機理——是 X 異常導致不良，還是兩者同受第三因素（如環境溫度、原料批次）影響？需由工程人員說明物理機理。其二，時序關係——X 異常是否確實發生在不良之前？若同時或之後發生，因果推論不成立。其三，統計顯著性與樣本量——支持度是否足夠？若僅基於少數幾筆批次，提升度 4.0 可能只是雜訊。其四，混淆因素——是否有其他變數同時影響兩者，應以分層分析或控制變因檢驗。其五，實驗驗證——最可靠的方式是設計對照實驗（刻意調整 X 觀察不良率變化），而非僅憑觀察資料。其六，措施的成本效益——即使因果成立，改善 X 的成本是否低於不良的損失。核心觀念：資料探勘負責「提出假說」，而驗證因果需要領域知識與實驗設計（呼應第 1 章 1.2.4 節的迷思二）。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 15 章 頻繁項集與關聯規則探勘

台灣自編大學教科書

@prof

大數據分析與雲端運算

第肆篇 大數據探勘與分析演算法

Mining & Analytics Algorithms

第 16 章

大規模分群演算法

Large-Scale Clustering Algorithms

機器人叫獸（李世淵） 編著

叫獸開講

我第一次真正理解分群，是在整理實驗室的零件櫃時。

那個櫃子累積了十幾年的東西，馬達、軸承、螺絲、感測器全混在一起。我跟兩個學生說：「把它分類整理好。」結果三個人分出了三套完全不同的系統。甲生按「零件種類」分：馬達一區、軸承一區。乙生按「尺寸大小」分：大件放下層、小件放抽屜。丙生按「用得差不多」分：常用的放外面、罕用的塞後面。

誰對？三個人都對。因為分群這件事，本質上沒有標準答案——它取決於你認為「什麼叫做像」。甲生認為功能相同就是像，乙生認為體積相近就是像，丙生認為使用頻率接近就是像。當你選定了「相似」的定義，分群的結果就跟著決定了。

這正是分群與前面兩章最大的不同。第 14 章找相似的配對、第 15 章找共現的組合，都有明確的目標；而分群是所謂的「非監督式學習」——沒有正確答案、沒有標籤，你只是丟給演算法一堆資料，問它：「這裡面有沒有什麼自然的結構？」

這一章我們要學的，除了最經典的 k-means，還有兩個專為「資料大到記憶體裝不下」而設計的演算法：BFR 與 CURE。同時我也要誠實地告訴你一件多數教科書輕描淡寫的事——當資料的維度很高時，「距離」這個我們習以為常的概念會開始崩壞。這個現象叫做維度災難，而它正是下一章要處理的難題。

學習目標

研讀完本章之後，你應該能夠：

1. 說明分群的定義與非監督式學習的特性，並選擇合適的距離度量。
2. 描述 k-means 的迭代流程，並說明其收斂性與初始化的影響。
3. 運用手肘法與輪廓係數，判斷合適的群數 k。
4. 比較階層式分群與 k-means 的差異，並說明其適用情境。
5. 說明 BFR 演算法如何以摘要統計處理超出記憶體的資料。
6. 說明 CURE 演算法如何以多代表點處理非球形的群。

核心概念

核心概念	說明
分群 (Clustering)	在無標籤的資料中，找出彼此相似者所形成的自然群體。
距離度量	量化兩點相似程度的方式，如歐氏距離、餘弦距離。
k-means	以「指派—更新」反覆迭代，把資料分為 k 群的經典演算法。
質心 (Centroid)	一群資料點的平均位置，代表該群的中心。
手肘法	以群內平方和對 k 作圖，取轉折點決定群數的方法。
BFR	以摘要統計（筆數、總和、平方和）處理超大資料的 k-means 變體。
CURE	以多個代表點描述群體，能處理非球形群的分群演算法。

表 16-1 本章核心概念一覽

16.1 分群的基本概念與距離度量

16.1.1 沒有標準答案的問題

分群屬於「非監督式學習」——資料沒有標籤，我們也不知道正確答案是什麼。這與「監督式學習」（給你一堆已標記為貓或狗的照片，學會分辨新照片）截然不同。

正因為沒有標準答案，分群結果的好壞往往難以客觀評判。承接叫獸開講的零件櫃，三種分法都「正確」，差別只在於哪一種對你的目的有用。因此，評估分群結果時，除了數學指標之外，「這個分群結果能不能被解釋、能不能指導行動」往往才是決定性的標準。

16.1.2 距離度量：定義什麼叫做像

既然分群取決於「什麼叫做像」，那麼選擇距離度量就是分群最關鍵的第一步。常見的度量有幾種，各有適用情境。

度量	衡量什麼	適用情境
歐氏距離	空間中的直線距離	數值型特徵、各維度尺度相近時
曼哈頓距離	各維度差值的絕對值總和	對離群值較不敏感、格狀路徑
餘弦距離	兩向量的夾角（不看長度）	文字向量、只在意方向與比例時
Jaccard 距離	1 減去集合的 Jaccard 相似度	集合型資料（呼應第 14 章）

表 16-2 常見的距離度量

尺度的陷阱

使用歐氏距離時有一個極常見的錯誤：忘記標準化。假設你要為顧客分群，特徵包含「年齡」（範圍約 20 到 70）與「年消費金額」（範圍約 1,000 到 500,000）。若直接計算歐氏距離，消費金額的數值大了好幾個數量級，會完全主宰距離的計算——年齡差二十歲的影響，還不如消費金額差二十元。

因此在分群之前，通常必須先把各特徵標準化到相近的尺度（例如轉為平均 0、標準差 1）。這是實務上最常被忽略、卻最容易毀掉整個分析的細節。

16.1.3 維度災難的預告

還有一個更深層的問題，我要在這裡先提出來。當資料的維度（特徵數）很高時，歐氏距離會逐漸失去意義——因為在高維空間中，所有點之間的距離會趨於相近，「最近的鄰居」與「最遠的點」差別越來越小。

這稱為「維度災難」（curse of dimensionality）。它的後果很嚴重：既然任兩點的距離都差不多，「群」這個概念本身就失去了意義。應對之道通常是先降維——這正是第 17 章的主題。本章的公式部分會用具體數字讓你看見這個現象。

16.2 k-means 與 k-means++

16.2.1 目標與流程

k-means 是最廣為使用的分群演算法。它的目標很明確：把資料分成 k 群，使得「每個點到其所屬群質心的距離平方總和」最小。這個要最小化的量稱為群內平方和（WCSS）：

$$WCSS = \sum_i \sum_{(x \in C_i)} \|x - \mu_i\|^2$$

式 16-1 群內平方和（k-means 的目標函數）

其中 C_i 是第 i 群， μ_i 是該群的質心。演算法本身出奇地簡單，就是兩個步驟反覆交替：

- 第 1 步。** 初始化：隨機選定 k 個點作為初始質心。
- 第 2 步。** 指派步驟：把每個資料點指派給「距離最近的質心」所屬的群。
- 第 3 步。** 更新步驟：重新計算每一群的質心（該群所有點的平均位置）。
- 第 4 步。** 重複第 2、3 步，直到指派結果不再改變（收斂）為止。

這個「指派—更新」的交替，每一輪都保證會讓 WCSS 下降或持平，因此必定收斂。但要注意——它保證收斂到「局部最佳解」，卻不保證是「全域最佳解」。

16.2.2 初始化的陷阱

「只保證局部最佳」這件事，在實務上造成的困擾比想像中大。若初始質心選得不好，k-means 可能收斂到一個明顯很差的結果——例如兩個質心擠在同一群裡，而另外兩個真實的群被迫合併。

最簡單的因應是「多跑幾次」：以不同的隨機初始值執行數十次，選出 WCSS 最小的那次結果。但這只是碰運氣，效率也不好。

更聰明的做法是 k-means++。它的初始化策略是：第一個質心隨機選；之後每選一個新質心時，讓「距離現有質心越遠的點」有越高的機率被選中。直覺上，這會讓初始質心盡量分散開來，避免全部擠在一起。這個小改動大幅提升了結果的穩定性與品質，如今幾乎是所有實作的預設選項。

16.2.3 k-means 的限制

k-means 雖然好用，但它有幾個內建的假設，一旦資料不符合，結果就會很糟。

- **假設群是球形的：**因為以「到質心的距離」來指派，k-means 傾向找出球狀（各方向大小相近）的群。若真實的群呈長條形、環形或月牙形，它會分得很難看。
- **假設群的大小相近：**k-means 傾向把大群切開、把小群合併，因為它最小化的是距離平方和。
- **對離群值敏感：**一個極端的離群點會把質心拉偏（因為質心是平均值）。
- **必須事先指定 k ：**而你通常不知道該分幾群，這正是下一節要處理的問題。

叫獸提醒

第一項限制值得你特別警惕。很多同學跑完 k-means 得到結果，就以為那是「資料真實的結構」——但其實你只是得到了「用球形眼鏡去看這份資料所看到的樣子」。如果真實的群是彎曲的長條狀（這在感測訊號中很常見），k-means 會硬把它切成好幾塊球。所以拿到分群結果後，務必想辦法視覺化（例如先降到二維再畫圖）看一眼。演算法不會告訴你它的假設被違反了——它只會安靜地給你一個看似合理的答案。

16.3 如何決定群數 k

16.3.1 手肘法

k 該取多少？這是分群最惱人的問題。最直觀的想法是「讓 WCSS 越小越好」，但這行不通——因為 k 越大 WCSS 一定越小，當 k 等於資料點數時，WCSS 會變成 0（每個點自成一群），卻毫無意義。

手肘法（elbow method）的做法是：把 k 從小到大逐一嘗試，畫出 WCSS 對 k 的曲線。這條曲線會先急遽下降、然後趨於平緩，那個「轉折點」就像手臂的肘部，通常被視為合適的 k。

轉折點的意義是：在此之前增加 k 能大幅改善分群品質（代表確實找到了新的結構），在此之後則改善有限（只是把原本合理的群硬切開）。本章的公式部分會用具體數字示範這條曲線。

16.3.2 輪廓係數

手肘法有時很主觀——曲線平滑時，肘部在哪裡見仁見智。另一個較客觀的指標是輪廓係數（silhouette coefficient）。

對每個資料點，計算兩個量：a 是它到「同群其他點」的平均距離（越小越好，代表群內緊密），b 是它到「最近的其他群中所有點」的平均距離（越大越好，代表群間分離）。該點的輪廓係數為：

$$s = (b - a) / \max(a, b)$$

式 16-2 單一點的輪廓係數

它的值域在 -1 到 1 之間：接近 1 表示這個點被分得很好（離自己群很近、離別群很遠）；接近 0 表示它落在兩群的邊界上；為負數則表示它可能被分錯群了。把所有點的輪廓係數平均，就得到整體的分群品質分數，可用來比較不同 k 的優劣。

16.3.3 別忘了領域知識

我必須強調：這兩個指標都只是參考。實務上決定 k 的最重要依據，往往是「這個分法對業務有沒有意義」。

舉例來說，若行銷團隊的資源只夠設計三種行銷方案，那麼即使數學指標建議分七群，分三群才是有用的答案。反過來，若某個 k 值的數學指標普通，但分出來的每一群都能被業務人員清楚地描述與命名（「這群是價格敏感的年輕客戶」），那它就是好的分群。演算法給建議，人做決定。

16.4 階層式分群

16.4.1 不必事先決定群數

階層式分群提供了另一種思路：它不要求你事先指定 k，而是建立一棵完整的「分群樹」，讓你事後決定要在哪個高度切開。

最常見的是「聚合式」（agglomerative）作法，其流程直觀得像在合併家族：

- 第 1 步。** 起初，把每個資料點各自視為一個群（共 N 群）。
- 第 2 步。** 找出「最接近的兩群」，把它們合併為一群。
- 第 3 步。** 重複第 2 步，每次減少一群，直到所有點合併成單一群為止。

這個過程會產生一棵樹狀圖（稱為樹狀圖 dendrogram），記錄了合併的順序與距離。你只要在樹上選一個高度橫切一刀，就得到對應的分群結果——切得低群數多，切得高群數少。

16.4.2 群間距離怎麼算

第 2 步的「最接近的兩群」需要定義群與群之間的距離，常見有幾種選擇，它們會導致相當不同的結果。

連結方式	群間距離定義	傾向
單一連結	兩群中最近的一對點的距離	能找出長條或不規則形狀，但易產生鏈狀效應
完全連結	兩群中最遠的一對點的距離	傾向產生大小相近的緊密球形群
平均連結	所有跨群配對距離的平均	介於兩者之間，較為穩健
質心連結	兩群質心之間的距離	與 k-means 精神相近

表 16-3 常見的群間距離定義

16.4.3 為何不適合大數據

階層式分群的優點很吸引人：不必事先決定 k 、能產生完整的層次結構、且結果穩定（不像 k -means 依賴隨機初始化）。但它有一個致命的缺點——計算成本。

因為每一輪都要找出「所有群對之間最近的一對」，基本實作的時間複雜度達 $O(N^3)$ ，即使優化後也需 $O(N^2 \log N)$ ，且需要儲存 $N \times N$ 的距離矩陣。這意味著資料量一萬筆時，光距離矩陣就有一億個元素；十萬筆時則是一百億個——記憶體直接爆掉。

所以階層式分群適合資料量在數千筆以內、且重視可解釋層次結構的場景（例如生物分類、文件主題階層）。面對真正的大數據，我們需要下面兩節的方法。

16.5 大規模分群：BFR 演算法

16.5.1 資料裝不進記憶體怎麼辦

k -means 的每一輪都要掃過所有資料點。若資料量遠超過記憶體，這意味著每一輪都要從磁碟重讀一次——迭代十輪就是十次全量磁碟讀取，成本高得無法接受（這正是第 11 章談過的痛點）。

BFR 演算法（以三位發明者 Bradley、Fayyad、Reina 命名）的解法很聰明：它假設資料只需被讀取一次。做法是把資料分批讀入記憶體，每處理完一批，就用「摘要統計」把已知的資訊濃縮保存，然後丟棄原始資料點。

16.5.2 三個數字就能代表一群點

BFR 的核心洞見是：要維護一個群的質心與離散程度，其實不需要保留所有點，只要三樣東西就夠了。

- **N**：這群點的個數。
- **SUM**：各維度上所有點座標的總和（一個向量）。
- **SUMSQ**：各維度上所有點座標平方的總和（一個向量）。

有了這三樣，質心就是 SUM/N ；而各維度的變異數則是 SUMSQ/N 減去 $(\text{SUM}/N)^2$ 。也就是說，無論這群有一百個點還是一億個點，我們都只需要保存這幾個數字——記憶體用量與資料量完全無關。

更妙的是，這三個統計量具有「可加性」：兩群合併時，只要把各自的 N 、 SUM 、 SUMSQ 分別相加即可。這個性質讓 BFR 能夠一批一批地處理資料，也讓它天生適合分散式運算（回想第 9 章的 Combiner——這正是同樣的思想：找出可結合的摘要量）。

16.5.3 三組集合

BFR 在處理過程中，把資料點歸入三種集合。

集合	內容	如何儲存
丟棄集	已確定屬於某群的點	併入該群的 N、SUM、SUMSQ 後丟棄原始點
壓縮集	彼此靠近但不確定屬於哪群的點群	以迷你群的摘要統計保存
保留集	孤立的離群點	原樣保留，等待後續批次或最終處理

表 16-4 BFR 的三組集合

要注意 BFR 有一個重要的前提假設：它假設每一群在各維度上都呈常態分布，且各維度彼此獨立（也就是群呈「軸向對齊的橢圓」形狀）。這個假設讓它能用簡單的統計量判斷「某點是否夠靠近某群」，但也限制了它的適用範圍——這正是下一節 CURE 要突破的。

16.6 高維與非球形分群：CURE 演算法

16.6.1 當群不是球形的

k-means 與 BFR 都以「單一質心」代表一個群，這隱含了「群是球形」的假設。但真實資料中的群，形狀可能千奇百怪——長條形、彎曲的帶狀、甚至同心圓。

經典的失敗例子是「兩個同心圓」：內圈一群、外圈一群。它們的質心幾乎在同一個位置，k-means 完全無法區分。同樣地，兩條平行的長條形群，用質心來看也會混在一起。

16.6.2 用多個代表點描述一群

CURE (Clustering Using REpresentatives) 的解法是：不要只用一個質心，改用「多個代表點」來描述一個群。

它的做法分為兩階段。第一階段先從資料中抽樣一小部分（能放進記憶體的量），對這個樣本執行階層式分群，得到初步的群。接著為每一群挑選若干個「分散得夠開」的點作為代表點，再把每個代表點朝該群的質心「收縮」一定比例（例如移動 20%）。

這個「收縮」的巧思很重要：它讓代表點不會停留在最外緣（避免離群值的影響），但又保留了群的整體形狀輪廓。第二階段則掃描完整資料，把每個點指派給「最近的代表點」所屬的群。

因為一個群由多個代表點共同描述，CURE 能夠捕捉長條形、彎曲等非球形的結構——這是它相對於 k-means 與 BFR 的核心優勢。

演算法	群的表示方式	適合的群形狀	資料規模
k-means	單一質心	球形、大小相近	中等（需多次掃描）
階層式	完整樹狀結構	視連結方式而定	小 ($O(N^2)$ 以上)
BFR	摘要統計 (N, SUM, SUMSQ)	軸向對齊的橢圓	極大（單次掃描）
CURE	多個收縮後的代表點	任意形狀	大（抽樣加一次掃描）

表 16-5 四種分群演算法的比較

16.6.3 承先啟後

本章我們認識了分群這個「沒有標準答案」的問題（16.1），從最經典的 k-means 及其初始化改良出發（16.2），學會以手肘法與輪廓係數決定群數、但也提醒領域知識才是最終依據（16.3）；接著看了不必指定 k 卻難以擴充的階層式分群（16.4），以及兩個專為大規模設計的演算法——以摘要統計單次掃描的 BFR（16.5），與以多代表點捕捉非球形結構的 CURE（16.6）。

但我在 16.1.3 節埋了一個伏筆還沒解決：維度災難。當特徵數達到數百甚至數千維時，距離會趨於一致，所有分群演算法都會失效——因為它們全都建立在「距離有意義」這個前提上。

那該怎麼辦？答案是：在分群之前，先設法把高維資料壓縮到低維，同時盡量保留原有的結構。這種「用更少的維度表達同樣的資訊」的技術，就是下一章的主題——降維與矩陣分解，包括你可能聽過的 PCA 與 SVD。它不只解決維度災難，也是推薦系統與隱含語意分析的基礎。

公式與流程：手肘法與維度災難

這一節我們用具體數字做兩件事：一是示範如何以手肘法決定 k，二是讓你親眼看見「維度災難」如何摧毀距離的意義。

手肘法的判讀

手肘法的關鍵不是看 WCSS 的絕對值，而是看「每增加一群，改善了多少」。定義改善率為：

$$\text{改善率} = [\text{WCSS}(k-1) - \text{WCSS}(k)] / \text{WCSS}(k-1)$$

式 16-3 增加一群所帶來的改善率

下表是一組實際的分群結果，我們逐一計算改善率。

群數 k	WCSS	改善率	判讀
1	1,000	—	全部視為一群
2	420	58.0%	大幅改善
3	180	57.1%	仍大幅改善
4	150	16.7%	改善驟降←肘部
5	138	8.0%	改善有限
6	130	5.8%	改善有限
7	125	3.8%	幾乎無改善

表 16-6 以改善率判讀手肘位置

看第 3 到第 4 列：改善率從 57.1% 驟降到 16.7%，這個「斷崖」就是肘部。它的意義是——分到三群時，每一群都對應到資料中真實存在的結構；分到第四群之後，只是把原本合理的群硬切開，改善有限。故此例應取 $k = 3$ 。

維度災難的實證

現在來看那個更根本的問題。我們在 d 維空間中隨機產生 200 個點，計算它們到中心點的距離，並觀察「最遠距離」與「最近距離」的相對差異：

$$\text{相對差異} = (d_{\max} - d_{\min}) / d_{\min}$$

式 16-4 距離的相對差異

這個值越大，代表「遠近之分」越明顯；越接近 0，代表所有點的距離都差不多。

維度 d	最近距離	最遠距離	相對差異
2	0.016	0.685	約 42.4 (差異極大)
10	0.454	1.266	約 1.79
100	2.546	3.288	約 0.29
1,000	8.734	9.610	約 0.10 (幾乎無差異)

表 16-7 維度增加時距離的相對差異 (隨機 200 點)

叫獸提醒

表 16-7 是我認為整章最該被記住的一張表。在二維空間中，最遠的點比最近的點遠了四十幾倍——「誰離我近」是個有意義的問題。但到了一千維，最遠與最近只差 10%，所有點看起來都一樣遠。這時候你再問「最近的鄰居是誰」，答案幾乎是隨機的。而 k-means、KNN、以及幾乎所有基於距離的方法，全都建立在「距離有意義」這個前提上——前提垮了，方法就跟著垮了，而且它們不會報錯，只會安靜地給你無意義的結果。所以面對高維資料，先降維再分群不是「可選的優化」，而是「必要的前提」。這就是下一章存在的理由。

案例研討

案例一 被消費金額綁架的顧客分群

某電商想為顧客分群，選用的特徵是「年齡」（20 到 70）、「購買次數」（1 到 50）與「年消費金額」（1,000 到 500,000）。他們直接跑 k-means，結果分出來的三群，差異幾乎完全來自消費金額——年齡與購買次數的分布在各群中看不出區別。

原因正是 16.1.2 節的尺度陷阱：消費金額的數值範圍比其他兩個特徵大了三到四個數量級，在歐氏距離的計算中完全主宰了結果。年齡差五十歲所貢獻的距離，還不如消費金額差一百元。

他們把三個特徵都標準化（轉為平均 0、標準差 1）後重跑，得到的分群才真正反映了三個面向的綜合差異，也才能被行銷團隊解讀為「年輕高頻小額」「中年低頻高額」等有意義的族群。這個案例提醒我們：資料前處理的一個小疏忽，就足以讓整個分析毫無價值。

案例二 兩個同心圓

某團隊分析設備的運轉狀態，把感測資料投影到二維後，肉眼可見資料呈現「內圈一群、外圈一環」的同心結構——對應設備的兩種運轉模式。

但他們跑 k-means ($k = 2$) 卻得到完全錯誤的結果：演算法把整個圓形區域從中間切成左右兩半。原因是內圈與外環的質心幾乎重疊在同一點，而 k-means 只會用「到質心的距離」來判斷歸屬——它眼中的世界只有球形。

改用能處理非球形結構的方法（如 CURE 的多代表點，或以密度為基礎的分群法）後，才正確地分出內圈與外環。這個案例是 16.2.3 節限制的直接印證，也再次說明「拿到分群結果一定要視覺化看一眼」的重要性——若他們沒有先畫圖，可能永遠不會發現 k-means 給的是錯的答案。

案例三 讀一次就好

某製造業累積了數 TB 的感測歷史資料，存放在物件儲存上。分析團隊想對這些資料分群，找出設備的典型運轉樣態，但直接跑 k-means 遇到了困難：資料遠超記憶體，而 k-means 需要迭代十幾輪，每輪都要完整讀取一次資料——光是磁碟與網路的傳輸成本就令人卻步。

他們改用 BFR。因為只需維護每群的三個摘要統計量（N、SUM、SUMSQ），記憶體用量與資料量無關；資料分批讀入，處理完即丟棄原始點。整份資料只被讀取一次，完成了原本無法負擔的分群。

這個案例的關鍵，在於 16.5.2 節那個「可加性」的性質——這與第 9 章 Combiner 要求的結合律是同一個數學條件。當你發現一個統計量可以「分批算再合併」時，它幾乎就註定能被大規模平行化。這是資料工程中極其重要的辨識能力。

案例四 一千維的無意義分群

某團隊把文件轉成一千多維的詞頻向量後，直接跑 k-means 分群。演算法順利收斂、沒有任何錯誤，輪廓係數也算得出來（約 0.08）。他們一度以為只是分群效果「普通」。

但檢視結果後發現，各群的內容毫無主題可言，像是隨機分配的。問題正是維度災難：依表 16-7 的規律，在一千多維空間中，任兩份文件的距離都極為接近，「最近的鄰居」幾乎是隨機的，分群自然毫無意義。輪廓係數接近 0，其實正是「所有點都在邊界上」的警訊。

解法是先以第 17 章的 SVD 把維度降到數十維（保留主要的語意結構），再進行分群——此後各群才呈現出清楚的主題。這個案例說明：高維資料的分群失敗，往往不是分群演算法的問題，而是根本不該在原始維度上分群。

實作活動

活動一 手動執行 k-means

請以紙筆執行兩輪 k-means，體會「指派—更新」的交替。

- 步驟 1.** 在方格紙上標出六個點：(1,1)、(1,2)、(2,1)、(8,8)、(8,9)、(9,8)。
- 步驟 2.** 取初始質心為 (1,1) 與 (2,1)（刻意選得很靠近），執行指派步驟。
- 步驟 3.** 計算新質心，再執行一輪指派，觀察結果是否收斂。
- 步驟 4.** 改以 (1,1) 與 (8,8) 為初始質心重做一次，比較兩次結果，說明初始化為何重要、以及 k-means++ 如何改善此問題。

活動二 畫出你的手肘圖

請以 Python (scikit-learn) 實作並判讀手肘法。

- 步驟 1.** 取得或產生一份含三到四個明顯群集的二維資料（可用 make_blobs）。
- 步驟 2.** 對 $k = 1$ 到 10 分別執行 k-means，記錄各自的 WCSS (inertia)。
- 步驟 3.** 畫出 WCSS 對 k 的折線圖，並以式 16-3 計算各 k 的改善率，找出肘部。
- 步驟 4.** 同時計算各 k 的輪廓係數並作圖，比較兩種方法建議的 k 是否一致。

活動三 親眼見證維度災難

請以 Python 重現表 16-7 的實驗。對 $d = 2, 10, 50, 100, 500, 1000$ 各產生 200 個隨機點（各維度取 0 到 1 的均勻分布），計算所有點到中心點 (0.5, 0.5, ...) 的距離，求出最近與最遠距

離，並以式 16-4 計算相對差異。把相對差異對維度作圖（建議 y 軸取對數）。完成後回答：從幾維開始，相對差異就小到讓「遠近」失去實質意義？這對你日後處理高維資料有什麼啟示？

延伸閱讀

- **《Mining of Massive Datasets》第 7 章**：Leskovec 等人著。本章的主要依據，對 BFR 與 CURE 有完整說明，並深入討論高維空間中分群的困難。
- **k-means++ 原始論文**：Arthur 與 Vassilvitskii 於二〇〇七年發表，說明其初始化策略與理論保證，篇幅簡短易讀。
- **DBSCAN 與密度式分群**：一種以密度而非距離為基礎的方法，能自然處理任意形狀的群並識別離群點，是案例二問題的另一種解法。
- **scikit-learn 分群模組文件**：提供各種分群演算法的比較圖表與適用情境說明，是選擇演算法時的實用參考。

本章小結

1. 分群屬非監督式學習，沒有標準答案——結果取決於你如何定義「相似」，故距離度量的選擇是第一關鍵；使用歐氏距離前務必先標準化各特徵，否則數值範圍大的特徵會主宰結果。
2. k-means 以最小化群內平方和（式 16-1）為目標，透過「指派—更新」交替迭代，必定收斂但僅保證局部最佳；k-means++ 讓初始質心盡量分散，大幅提升結果的穩定性。
3. k-means 內建「群為球形、大小相近」的假設，且對離群值敏感、須事先指定 k；當真實的群為長條、環形等非球形時，它會安靜地給出錯誤結果，故分群後務必視覺化檢視。
4. 決定 k 可用手肘法（找 WCSS 改善率驟降的轉折點）與輪廓係數（式 16-2，衡量群內緊密與群間分離）；但最終依據往往是領域知識——分群結果能否被解釋並指導行動。
5. 階層式分群不必事先指定 k，能產生完整的樹狀層次且結果穩定，但時間複雜度達 $O(N^2 \log N)$ 以上且需儲存 $N \times N$ 距離矩陣，僅適合數千筆以內的資料。
6. BFR 以三個摘要統計量（N、SUM、SUMSQ）代表一群，記憶體用量與資料量無關，且統計量具可加性而能分批處理與平行化，只需單次掃描；但假設各群呈軸向對齊的常態分布。
7. CURE 以「多個收縮後的代表點」取代單一質心來描述一群，因而能捕捉長條、彎曲等任意形狀的結構；先對抽樣做階層式分群產生代表點，再掃描完整資料完成指派。
8. 維度災難是所有基於距離之方法的共同威脅：維度升高時，最遠與最近距離的相對差異急遽縮小（表 16-7，一千維時僅約 10%），使「距離」失去鑑別力。故高維資料必須先降維再分群——這正是第 17 章的主題。

習題

一、觀念題

1. 為什麼說分群「沒有標準答案」？請以叫獸開講的零件櫃為例說明，並解釋這對評估分群結果有何啟示。
2. k-means 為何必定收斂，卻不保證得到最佳解？k-means++ 以什麼策略改善此問題？
3. 請列舉 k-means 的兩項內建假設，並各舉一個「假設被違反」時會發生的錯誤結果。
4. BFR 為何只需保存 N、SUM、SUMSQ 三個統計量？這三者的「可加性」為何使它適合分散式運算？

二、計算題

1. 某次分群的 WCSS 依 $k = 1$ 到 6 分別為 800、300、120、105、96、90。請以式 16-3 計算各 k 的改善率，並判斷肘部位置與建議的 k 值。
2. 某資料點到同群其他點的平均距離 $a = 2.0$ ，到最近其他群所有點的平均距離 $b = 5.0$ 。
(一) 請以式 16-2 計算其輪廓係數並判讀。(二) 若另一點的 $a = 4.0$ 、 $b = 4.5$ ，其輪廓係數為何？兩者相比說明了什麼？

三、思考與討論

1. 案例四中，團隊在一千多維上分群，演算法順利收斂卻毫無意義。請討論：這類「不會報錯的失敗」為何特別危險？你會用哪些方法在事前或事後察覺它？（可連結第 13 章的「安靜的失敗」）
2. 本章介紹的四種演算法各有不同的假設與適用規模（表 16-5）。請為下列情境各選一種並說明理由：（一）三千筆基因資料，需要可解釋的層次關係；（二）十億筆感測資料，群大致呈橢圓形；（三）百萬筆資料，群呈彎曲的帶狀。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

因為分群結果取決於「如何定義相似」，而這個定義本身沒有客觀正確答案。零件櫃的例子中，按種類、按尺寸、按使用頻率分類，三者都成立，只是各自採用了不同的相似性定義（功能相同／體積相近／使用頻率接近）。對評估的啟示：不能只靠數學指標（WCSS、輪廓係數）判定好壞，還必須看「這個分群結果對目的是否有用」——能否被解釋、能否命名、能否指導行動。因此領域知識在分群中的地位，往往高於演算法本身。

第 2 題

必定收斂的原因：指派步驟把每點分給最近質心，必使 WCSS 下降或持平；更新步驟取群內平均作為新質心，也必使 WCSS 下降或持平。WCSS 單調遞減且有下界（不小於 0），故必收斂。不保證最佳的原因：它只能保證每一步都不變差，因而停在「局部最佳解」——初始質心不同可能落入不同的局部解（如兩質心擠在同一真實群中）。k-means++ 的策略：第一個質心隨機選，之後每選新質心時，讓距離現有質心越遠的點有越高機率被選中，使初始質心盡量分散，大幅降低落入劣質局部解的機率。

第 3 題

假設一：群為球形（因以「到質心的距離」指派）。違反時的錯誤：如案例二的兩個同心圓，內圈與外環質心幾乎重疊，k-means 會把圓形區域從中切成左右兩半，完全錯誤；長條形或月牙形的群也會被硬切成數塊。假設二：群的大小相近。違反時的錯誤：因最小化距離平方和，演算法傾向把一個大群切開、同時把兩個小群合併，使結果與真實結構不符。（亦可答對離群值敏感——單一極端點會把質心拉偏。）

第 4 題

因為描述一個群所需的資訊只有質心與離散程度，而兩者皆可由這三個量導出：質心 = SUM/N ；各維度變異數 = $\text{SUMSQ}/N - (\text{SUM}/N)^2$ 。故無論該群有一百個或一億個點，記憶體用量都相

同，與資料量無關。可加性的意義：兩群合併時只需把各自的 N 、 SUM 、 $SUMSQ$ 分別相加即可，無需回頭存取原始資料點。這使得資料可以分批（或分散到多台機器）各自計算局部摘要，最後合併得到全域結果——與第 9 章 Combiner 所要求的結合律是同一個數學條件，故天生適合 MapReduce 式的平行化。

二、計算題

第 1 題

依式 16-3 逐一計算改善率。

- (1) $k = 2: (800 - 300) / 800 = 500 / 800 = 62.5\%$ 。
- (2) $k = 3: (300 - 120) / 300 = 180 / 300 = 60.0\%$ 。
- (3) $k = 4: (120 - 105) / 120 = 15 / 120 = 12.5\%$ 。
- (4) $k = 5: (105 - 96) / 105 = 9 / 105 \approx 8.6\%$; $k = 6: (96 - 90) / 96 = 6 / 96 = 6.25\%$ 。

判讀：改善率在 $k = 3$ 至 $k = 4$ 之間出現斷崖（60.0% 驟降至 12.5%），故肘部位於 $k = 3$ ，建議取 $k = 3$ 。意義是：分到三群前，每增一群都對應到資料中真實的結構；之後只是把合理的群硬切開，改善有限。

第 2 題

依式 16-2, $s = (b - a) / \max(a, b)$ 。

- (1) $a = 2.0$ 、 $b = 5.0$: $s = (5.0 - 2.0) / 5.0 = 3.0 / 5.0 = 0.6$ 。判讀：明顯為正且接近 1，表示該點離自己群近、離最近的其他群遠，被分得很好。
- (2) $a = 4.0$ 、 $b = 4.5$: $s = (4.5 - 4.0) / 4.5 = 0.5 / 4.5 \approx 0.111$ 。判讀：接近 0，表示該點到自己群與到鄰近群的距離相差無幾，落在兩群的邊界上，分群品質不佳。

兩者相比說明：輪廓係數同時考量「群內緊密（ a 小）」與「群間分離（ b 大）」，只有兩者兼備才會得到高分。第二點的 a 與 b 都不小且相近，正是分群結構不明顯的徵兆；若整體平均輪廓係數都接近 0（如案例四的 0.08），往往意味著資料在該特徵空間中根本不存在清楚的群結構——此時應懷疑是否遭遇維度災難。

三、思考與討論

第 1 題

危險之處：演算法順利收斂、無任何錯誤訊息、指標也算得出數值，使人誤以為結果可信（呼應第 13 章「安靜的失敗」——程式成功但結果錯誤）。這類失敗會讓無意義的分群被當成真實發現，進而流入決策或後續模型。事前察覺的方法：檢視資料維度與樣本數的比例、先做降維（PCA/SVD）觀察主要成分能解釋多少變異、以式 16-4 檢查距離的相對差異是否已趨近於 0。事後察覺的方法：檢視平均輪廓係數是否接近 0（表示所有點都在邊界）、將資料降到二三維後視覺化觀察是否有群結構、檢查各群的特徵分布是否有可解釋的差異、以及請領域專家判讀各群是否能被命名與描述。核心原則：任何無標準答案的分析，都必須設計「可證偽」的檢驗方式。

第 2 題

（一）三千筆基因資料、需可解釋層次：選階層式分群。理由是資料量在其可負擔範圍內（數千筆），且它能產生完整的樹狀圖呈現層次關係，正符合生物分類的需求； k -means 無法提供層次資訊。（二）十億筆感測資料、群呈橢圓形：選 BFR。理由是資料量遠超記憶體，BFR 只需單次掃描且

記憶體用量與資料量無關；而「橢圓形」恰好符合其「各維度常態分布、軸向對齊」的假設。（三）百萬筆資料、群呈彎曲帶狀：選 CURE。理由是 k-means 與 BFR 皆假設球形或橢圓形，面對彎曲帶狀會失敗；CURE 以多個代表點描述一群，能捕捉任意形狀，且採抽樣加單次掃描，可負擔百萬筆的規模。（亦可提出 DBSCAN 等密度式方法作為替代方案並說明理由。）

台灣自編大學教科書 | 機器人叫獸 @prof | 第 16 章 大規模分群演算法

台灣自編大學教科書
@prof
大數據分析與雲端運算

第肆篇 大數據探勘與分析演算法

Mining & Analytics Algorithms

第 17 章

降維與矩陣分解

Dimensionality Reduction and Matrix Factorization

機器人叫獸（李世淵） 編著

叫獸開講

上一章結尾，我留下了一個很不安的結論：當維度高到一定程度，距離就失去意義，所有基於距離的方法都會崩潰。這一章，我們要來解決它。

先講一個大家都熟悉的東西——影子。一個立體的茶壺，投影到牆上是一個平面的影子。從三維降到二維，你當然損失了資訊（深度沒了），但如果投影的角度選得好，你仍然一眼就認得出「這是茶壺」。反之，若從正上方投影，可能只看到一個圓，什麼都認不出來。

降維的全部學問，就藏在這個「角度選得好不好」裡面。我們要找的，是那個能讓資料的「形狀」保留得最完整的投影方向。而數學已經替我們回答了這個問題——答案就是主成分分析（PCA）與奇異值分解（SVD）。

這一章我要先給你一個心理準備：它是全書數學味最重的一章。但我保證，你不需要會手算 SVD（那是線性代數課的事），你需要的是理解它「在做什麼」以及「為什麼有用」。我會盡量用幾何直覺來講，把矩陣運算的細節留給軟體。

最後我要強調它的重要性。降維不只是解決維度災難的工具，它還是後面幾章的地基：第 19 章的推薦系統靠矩陣分解找出「隱含的品味維度」，第 21 章的大語言模型把每個詞表示成向量——這些全都是同一套思想。學會這一章，你等於拿到了通往現代 AI 的一把鑰匙。

學習目標

研讀完本章之後，你應該能夠：

1. 說明降維的動機，並區分特徵選擇與特徵萃取的差異。
2. 以幾何直覺說明 PCA 如何尋找最大變異的投影方向。
3. 說明 SVD 的三個矩陣各代表什麼意義。
4. 運用能量保留率決定應保留的維度數。
5. 說明 CUR 分解的動機，並比較其與 SVD 的取捨。
6. 說明隱含語意分析如何以降維捕捉詞義的關聯。

核心概念

核心概念	說明
降維	以較少的維度表達資料，同時盡量保留其結構與資訊。
特徵選擇與萃取	前者挑出既有欄位的子集，後者由既有欄位組合出新的維度。
主成分分析（PCA）	尋找變異最大的正交方向，並投影到這些方向上。
奇異值分解（SVD）	把任意矩陣分解為 U 、 Σ 、 V^T 三個矩陣的乘積。
奇異值	Σ 對角線上的數值，代表各潛在維度的重要程度。
能量保留率	前 k 個奇異值平方和佔總和的比例，用以決定保留維度數。
CUR 分解	以原始的實際列與行構成的分解，保有可解釋性與稀疏性。

表 17-1 本章核心概念一覽

17.1 降維的動機

17.1.1 四個理由

除了上一章的維度災難，降維還有其他幾個實際的好處。

- **對抗維度災難：**維度降低後，距離重新具有鑑別力，分群、最近鄰搜尋等方法才能正常運作。
- **去除雜訊：**資料的變異中，有些來自真實結構，有些來自雜訊。降維時保留主要成分、捨棄次要成分，往往能把雜訊一併濾掉。
- **節省儲存與運算：**一份一萬維的資料降到五十維，儲存與後續運算的成本都降低了兩百倍。
- **視覺化：**人類只能理解二維或三維。要「看見」高維資料的結構，唯一的辦法就是先降維。

第二點特別值得玩味：降維竟然能「提升」品質。這聽起來反直覺——丟掉資訊怎麼會變好？關鍵在於，被丟掉的多半是雜訊而非訊號。這也是第 19 章推薦系統為何要用矩陣分解的原因之一。

17.1.2 選擇還是萃取

降維有兩條路，它們的哲學完全不同。

特徵選擇（feature selection）是「挑」——從原有的一千個欄位中，挑出最有用的五十個，其餘丟棄。好處是結果完全可解釋（你知道保留的是「年齡」「消費金額」這些具體欄位），壞處是被丟棄的欄位所含的資訊就真的沒了。

特徵萃取（feature extraction）則是「合成」——不挑選任何原有欄位，而是把一千個欄位「組合」成五十個全新的維度，每個新維度都是原有欄位的加權組合。好處是能保留更多資訊（因為每個新維度都綜合了所有舊欄位），壞處是新維度往往難以解釋——它可能是「 $0.3 \times \text{年齡} + 0.7 \times \text{消費金額} - 0.2 \times \text{購買次數}$ 」，你很難為它取一個有意義的名字。

本章談的 PCA 與 SVD 都屬於特徵萃取。而 17.5 節的 CUR 分解，則是為了挽回「可解釋性」而生的折衷方案。

17.2 主成分分析（PCA）

17.2.1 該往哪個方向投影

回到茶壺影子的比喻。假設資料點在二維平面上呈現一個狹長的橢圓分布，我們想把它降到一維（投影到一條直線上）。該選哪條線？

直覺告訴我們：應該沿著橢圓的「長軸」投影。因為沿長軸方向，資料點分散得最開，投影後仍能保留它們的相對差異；反之若沿短軸投影，所有點會擠成一團，彼此的差異全部消失。

PCA 把這個直覺形式化：它尋找「資料變異最大的方向」，稱為第一主成分。接著在「與第一主成分垂直」的方向中，再找變異最大的，稱為第二主成分。如此下去，得到一組互相正交的方向，且重要性依序遞減。

降維的做法就是：只保留前 k 個主成分，把資料投影到這 k 個方向所張成的空間。因為捨棄的是變異最小的方向（資料在那些方向上本來就沒什麼差異），損失的資訊自然最少。

17.2.2 為什麼「變異最大」等於「資訊最多」

這是初學者常有的疑問。想像一個極端例子：某個特徵是「所有顧客的國籍」，而你的顧客全是台灣人——這個特徵的變異為零，它對區分顧客毫無幫助，因為每個人都一樣。

反之，變異大的方向意味著「資料點在這個方向上彼此差異明顯」，因此保留它就保留了區分不同資料點的能力。這正是我們降維時最想保留的東西——結構與差異。

換個角度也可以這樣理解：PCA 所保留的投影，等價於「使投影後的重建誤差最小」的投影。兩種說法在數學上是等價的，只是一個從「保留變異」出發，一個從「減少損失」出發。

17.2.3 實作要點

- **務必先標準化**：與第 16 章一樣，若各特徵尺度差異大，變異最大的方向會被大尺度特徵主宰。PCA 前的標準化幾乎是必要步驟。
- **主成分是正交的**：各主成分互相垂直，因此新的維度之間彼此獨立、不重複攜帶相同資訊。
- **解釋性是代價**：主成分是原始特徵的線性組合，通常難以賦予直觀意義。若可解釋性至關重要，應考慮特徵選擇或 CUR。
- **對離群值敏感**：因為變異的計算涉及平方，少數極端值可能大幅扭曲主成分的方向。

17.3 奇異值分解 (SVD)

17.3.1 把矩陣拆成三塊

SVD 是本章的核心工具，也是 PCA 背後的數學機制。它說的是：任何一個 $m \times n$ 的矩陣 A ，都可以被分解成三個矩陣的乘積。

$$A = U \Sigma V^T$$

式 17-1 奇異值分解

這三塊各代表什麼？我們用一個具體情境來理解：設 A 是一個「使用者 $\times$ 電影」的評分矩陣，共 m 個使用者、 n 部電影。

矩陣	維度	意義
U	$m \times r$	使用者與「潛在概念」的關聯強度，例如某使用者有多喜歡科幻片。
Σ	$r \times r$ (對角)	各潛在概念的重要程度（奇異值），由大到小排列。
V^T	$r \times n$	潛在概念與電影的關聯強度，例如某部電影有多「科幻」。

表 17-2 SVD 三個矩陣的意義

這裡的「潛在概念」（latent concept）是 SVD 最迷人的地方——它是演算法自己從資料中發現的，沒有人事先告訴它「科幻」或「愛情」這些類型的存在。SVD 只是純粹地從評分模式中，找出能最有效解釋這些資料的隱藏維度。

17.3.2 奇異值就是重要性

Σ 對角線上的奇異值 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r$ ，由大到小排列，代表各個潛在概念的「份量」。第一個奇異值最大，意味著它所對應的概念最能解釋資料中的變異。

降維的做法因此非常自然：只保留前 k 個最大的奇異值（以及對應的 U 、 V 的前 k 行），其餘捨棄。這稱為「截斷 SVD」（truncated SVD）：

$$\mathbf{A} \approx \mathbf{A}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T$$

式 17-2 截斷 SVD 的低秩近似

有一個很重要的數學保證：在所有「秩為 k 的矩陣」中， $\mathbf{A}_k$ 是與原矩陣 $\mathbf{A}$ 最接近的那一個（誤差最小）。換句話說，SVD 給出的不只是「一個不錯的近似」，而是「最好的近似」。這個結論稱為 Eckart-Young 定理，是 SVD 之所以如此重要的根本原因。

17.3.3 k 該取多少

與上一章決定群數 k 一樣，這裡也要決定保留幾個維度。判準是「能量保留率」——前 k 個奇異值的平方和，佔全部奇異值平方和的比例。下一節的公式會完整說明，實務上常見的做法是保留到 90% 或 95% 的能量。

叫獸提醒

我要提醒你一個常見的誤解。很多人以為 PCA 和 SVD 是兩種不同的方法，其實 PCA 就是對「中心化後的資料矩陣」做 SVD——兩者本質相同，只是傳統上 PCA 從統計角度（共變異數矩陣的特徵向量）來描述，SVD 從線性代數角度（矩陣分解）來描述。你會在不同的教科書看到不同的講法，但它們指的是同一件事。理解這一點，你就不會在讀論文時被兩套術語搞混。

17.4 隱含語意分析 (LSA)

17.4.1 同義詞與一詞多義的難題

SVD 最經典的應用之一，是在文字處理上，稱為隱含語意分析（Latent Semantic Analysis, LSA）。它要解決的是傳統關鍵字比對的兩個老毛病。

第一個是同義詞問題：一篇文章寫「汽車」，另一篇寫「轎車」，若只做字面比對，這兩篇文章的相似度會为零——儘管它們談的是同一件事。

第二個是一詞多義問題：「蘋果」可能指水果，也可能指科技公司。字面相同，語意卻南轅北轍。

17.4.2 降維如何解決語意問題

LSA 的做法是：先建立「詞 $\times$ 文件」矩陣（每格記錄某詞在某文件中出現的次數或權重），再對它做截斷 SVD，把維度從數萬個詞降到數百個「潛在語意維度」。

神奇的事情就發生在這裡。因為「汽車」與「轎車」總是出現在相似的上下文中（同樣與引擎、駕駛、道路等詞共現），SVD 會發現它們在資料中扮演著幾乎相同的角色，於是把它們映射到潛在空間中非常接近的位置。

這意味著：即使兩篇文章一個字都沒重疊，只要它們談的是同一個主題，在降維後的空間中就會彼此靠近。降維不只壓縮了資料，還「發現」了詞義的關聯——而這一切都是從共現模式中自動學到的，沒有任何人工建立的辭典。

你可能已經察覺到，這個思想與第 21 章的詞向量（word embedding）和大語言模型高度相通：都是把詞表示成稠密的低維向量，讓語意相近的詞在向量空間中靠近。LSA 是這條路線的早期先驅，而現代方法只是用了更強大的模型來學習這些向量。原理一脈相承。

17.5 CUR 分解

17.5.1 SVD 的兩個實務問題

SVD 在數學上完美，但用在大規模真實資料時有兩個惱人的問題。

- **失去稀疏性：**真實的矩陣往往極度稀疏（例如評分矩陣中，多數使用者只評過極少數電影，99% 以上的格子是空的）。但 SVD 分解出的 U 與 V 幾乎都是稠密矩陣——原本只需儲存少數非零值，分解後反而佔用更多空間。
- **失去可解釋性：** U 與 V 的每一行都是所有原始列（或行）的線性組合，可能包含正負數的複雜加權，難以賦予任何實際意義。

17.5.2 用真實的列與行

CUR 分解的想法很直接：與其用「合成的」抽象維度，不如直接從原矩陣中「挑選」一些真實的列與行來構成分解。

$$A \approx CUR$$

式 17-3 CUR 分解

其中 C 由原矩陣中實際挑選的若干「行」組成， R 由實際挑選的若干「列」組成， U 則是一個較小的連結矩陣。挑選時通常依「重要性」給予機率，越重要的列與行越可能被選中。

這樣做的好處立竿見影：因為 C 與 R 就是原矩陣的實際列行，它們保有原本的稀疏性；而且它們是「真實存在的」使用者或電影，可以直接解釋——例如「這個潛在維度可以用這五部代表性電影來描述」。

代價則是近似的精確度不如 SVD（畢竟 SVD 有 Eckart-Young 的最佳性保證），且因為挑選帶有隨機性，結果不是唯一的。這又是一次典型的工程取舍：以一點精確度，換取稀疏性與可解釋性。

面向	SVD	CUR
組成元素	合成的抽象維度	原矩陣的實際列與行
近似精確度	最佳（Eckart-Young 保證）	略遜
稀疏性	分解後變稠密	保有原本的稀疏性
可解釋性	低（線性組合難以命名）	高（是真實的資料項目）

表 17-3 SVD 與 CUR 的比較

17.6 分散式矩陣運算

17.6.1 大矩陣怎麼算

當矩陣大到一台機器裝不下時（例如一億使用者乘上一百萬部電影），SVD 的計算本身就成了大數據問題。分散式運算在此有幾種常見策略。

- **矩陣分塊：**把大矩陣切成若干子區塊，分散到不同機器上，各自計算後再彙整——與第 9 章 MapReduce 的矩陣乘法思路一致。
- **迭代式方法：**不直接做完整分解，而是以反覆的矩陣向量乘法逐步逼近前 k 個奇異值（如 Lanczos 法）。因為只需要前 k 個，不必求出全部。

- **隨機化演算法**：先以隨機投影把矩陣壓到較小的規模，再對小矩陣做精確分解。以少量誤差換取大幅的速度提升——這與第 14 章 MinHash 的思想如出一轍。
- **交替最小平方法**：在推薦系統中常用：固定 U 求 V、再固定 V 求 U，反覆交替。每一步都能完美平行化，是 Spark MLlib 的標準做法（詳見第 19 章）。

17.6.2 為什麼隨機化有效

第三種策略值得多說幾句，因為它體現了大數據演算法的核心哲學。直覺上，「隨機丟掉一些資訊再算」聽起來很不可靠，但數學上可以證明：只要隨機投影的維度略高於目標維度 k ，得到的近似解在高機率下會非常接近最佳解。

這與第 14 章的 MinHash、第 24 章的串流演算法是同一個思想——在大數據的規模下，一個「有數學保證的近似」通常遠比「精確但算不完」有價值。你會發現，這個觀念在整個第肆篇與第伍篇不斷出現，值得你內化為思考習慣。

17.6.3 承先啟後

本章我們解決了上一章留下的難題。從降維的四個動機出發（17.1），以幾何直覺理解了 PCA 如何尋找變異最大的投影方向（17.2），接著認識了它背後的數學機制 SVD——把矩陣拆成使用者、概念、項目三塊，並以奇異值衡量各概念的重要性（17.3）；再看到 LSA 如何以降維自動發現詞義關聯（17.4）、CUR 如何以精確度換取稀疏性與可解釋性（17.5），以及大規模矩陣運算的分散式策略（17.6）。

我希望你特別記住 17.3 節那個「潛在概念」的觀念。SVD 沒有被告知電影有「科幻」或「愛情」這些類型，它純粹從評分模式中，自己發現了這些隱藏的維度。這種「從資料中自動發現隱含結構」的能力，正是現代人工智慧的核心思想。

而這個觀念，馬上就會在下一章派上用場。既然 SVD 能從評分矩陣中找出「使用者的品味」與「電影的特質」，那我們是不是就能預測「某個使用者會不會喜歡某部他還沒看過的電影」？答案是肯定的——這正是推薦系統的核心，也是第 19 章的主題。不過在那之前，我們要先處理另一類重要的資料結構：當資料點之間存在明確的「連結」關係時（網頁的超連結、社群的好友關係、金流的往來），該如何分析？這就是下一章「連結分析與大規模圖處理」要回答的問題。

公式與流程：能量保留率與維度的選擇

降維最實際的問題是：到底要保留幾個維度？這一節給你一個可以直接套用的判準。

奇異值的平方 σ_i^2 代表第 i 個潛在概念所「攜帶的能量」（即它解釋了多少變異）。因此保留前 k 個維度所留住的資訊比例，可定義為：

$$\text{能量保留率} = \sum_{i=1..k} \sigma_i^2 / \sum_{i=1..r} \sigma_i^2$$

式 17-4 前 k 個維度的能量保留率

實務上的做法是：先算出所有奇異值，再從大到小累加其平方，直到達成目標比例（常取 90% 或 95%）為止。下表以一組實際的奇異值示範。

k	奇異值 σ_k	σ_k^2 (能量)	累積能量率	判讀
1	45.2	2,043.0	63.93%	主導維度
2	28.7	823.7	89.70%	接近 90%

k	奇異值 σ_k	σ_k^2 (能量)	累積能量率	判讀
3	15.3	234.1	97.02%	超過 95%←建議
4	8.1	65.6	99.08%	邊際效益遞減
5	4.2	17.6	99.63%	幾乎是雜訊
6~10	2.6 以下	合計約 12.9	100%	可視為雜訊

表 17-4 能量保留率與維度選擇 (原始 10 維)

解讀這張表：原本十個維度，只要保留前三個就留住了 97% 的能量。也就是說，我們把資料壓縮到原本的 30%，卻幾乎沒有損失任何有意義的資訊——被丟掉的那 3%，很可能主要是雜訊。

壓縮率的計算

降維的儲存效益也可以量化。原本 $m \times n$ 的矩陣需儲存 mn 個數值；截斷 SVD 後需儲存 U_k ($m \times k$)、 Σ_k (k 個) 與 V_k^T ($k \times n$)，故壓縮率為：

$$\text{壓縮率} = k(m + n + 1) / (mn)$$

式 17-5 截斷 SVD 的儲存壓縮率

以 $m = 100,000$ 個使用者、 $n = 10,000$ 部電影、保留 $k = 50$ 為例：原始需 10 億個數值，分解後只需 $50 \times (100,000 + 10,000 + 1) = 5,500,050$ 個，約為原本的 0.55%——壓縮了約 182 倍。

叫獸提醒

這裡有個觀念我希望你想清楚。表 17-4 中被丟掉的那些小奇異值，我說它們「很可能是雜訊」——這不只是安慰的說法，而是降維能「提升」品質的關鍵。真實資料的訊號通常集中在少數幾個主要方向，而測量誤差、隨機波動這些雜訊則均勻散布在所有方向上。因此當你只保留前幾個大奇異值時，你保住了大部分訊號，卻丟掉了大部分雜訊——訊噪比反而提高了。這就是為什麼推薦系統做完矩陣分解後，預測反而更準；也是為什麼影像壓縮能在大幅減少資料量的同時，看起來幾乎沒有差別。少即是多，在這裡是有數學根據的。

案例研討

案例一 讓一千維的文件重新有意義

回到第 16 章案例四：某團隊把文件轉成一千多維的詞頻向量後直接分群，結果毫無意義——各群內容像隨機分配，輪廓係數僅約 0.08。

他們改為先做 LSA：對「詞 $\times$ 文件」矩陣做截斷 SVD，依式 17-4 保留能量達 95% 的前 120 個潛在語意維度，再對降維後的表示進行 k-means 分群。

結果截然不同——各群呈現清楚的主題（技術文件、法務文件、行銷文案等），輪廓係數提升到 0.4 以上。原因有二：其一，維度從一千多降到一百二，距離重新具有鑑別力（解除了維度災難）；其二，LSA 把同義詞映射到相近位置，使「談同一主題但用詞不同」的文件得以聚在一起。這個案例把第 16、17 兩章完整串了起來。

案例二 SVD 發現了沒有人標註的電影類型

某串流平台對其「使用者 × 電影」評分矩陣做 SVD，取前 20 個潛在維度。研究人員檢視 V^T 矩陣時發現了有趣的現象：第一個潛在維度上得分最高的電影，清一色是科幻與動作片；第二個維度得分高的則多為愛情與劇情片。

關鍵在於，資料中從來沒有任何「類型」的標註。SVD 只看到一堆評分數字，卻自己從「哪些使用者傾向給哪些電影高分」的模式中，還原出了類型的概念。

這個案例是 17.3.1 節「潛在概念」最生動的展現，也解釋了為何矩陣分解能用於推薦——若我們知道某使用者在「科幻維度」上得分很高，而某部他沒看過的電影在該維度上也很高，就有理由推薦給他。第 19 章會把這個想法完整展開。

案例三 稀疏矩陣的儲存悲劇

某電商想對「使用者 × 商品」的購買矩陣做降維。這個矩陣有一百萬使用者、十萬商品，但極度稀疏——平均每位使用者只買過約 20 件商品，非零元素僅約兩千萬個，以稀疏格式儲存不過幾百 MB。

他們直接做 SVD 並保留 $k = 200$ ，結果分解出的 U 矩陣是 $1,000,000 \times 200$ 的稠密矩陣、 V 是 $100,000 \times 200$ 的稠密矩陣，合計約兩億兩千萬個數值——反而比原始的稀疏儲存還大，記憶體直接吃緊。

這正是 17.5.1 節的第一個問題：SVD 會摧毀稀疏性。他們後來改用 CUR 分解，因 C 與 R 就是原矩陣的實際行列，保有原本的稀疏性，儲存量大幅下降；且挑出的代表性商品可以直接被行銷團隊解讀。以一點近似精確度，換回了稀疏性與可解釋性，是務實的取捨。

案例四 被離群值扭曲的主成分

某團隊對感測資料做 PCA，卻發現第一主成分的方向很奇怪——它幾乎完全對應到某一個感測器的讀數，而非預期中多個感測器的綜合變化。

檢查後發現，該感測器在某段時間故障，產生了數十筆極端異常的讀數（正常值的數百倍）。因為 PCA 尋找的是「變異最大的方向」，而變異的計算涉及平方，這些極端值貢獻了巨大的變異，於是第一主成分就被它們「綁架」了。

他們在前處理階段移除這些異常值後重跑，主成分才恢復正常。這個案例呼應 17.2.3 節的提醒——PCA 對離群值敏感。它也再次印證第 13 章的教訓：資料品質檢核不是可有可無的步驟，而是所有後續分析能否成立的前提。

實作活動

活動一 用 PCA 看見高維資料

請以 Python (scikit-learn) 實作降維與視覺化。

- 步驟 1.** 取得一份高維資料集（如手寫數字資料集，64 維以上）。
- 步驟 2.** 先標準化各特徵，再執行 PCA 降到二維，並以散佈圖繪出（不同類別以顏色區分）。
- 步驟 3.** 觀察：同類別的點是否聚在一起？若不標準化直接做 PCA，結果有何差異？
- 步驟 4.** 計算並繪出「累積能量保留率對 k 」的曲線，找出保留 90% 與 95% 能量各需幾個維度。

活動二 降維加分群的組合技

請重現案例一，親自驗證降維能否救回失敗的分群。

- 步驟 1. 取得一批文件（可用新聞或論文摘要），轉換為詞頻或 TF-IDF 矩陣（維度應達數千）。
- 步驟 2. 直接在原始高維空間執行 k-means，計算平均輪廓係數。
- 步驟 3. 以截斷 SVD 降到 100 維左右（依式 17-4 確認能量保留率），再執行 k-means 並計算輪廓係數。
- 步驟 4. 比較兩者的輪廓係數與各群的實際內容，說明降維帶來了什麼改變。

活動三 觀察 SVD 的影像壓縮

請以一張灰階影像（可視為一個數值矩陣）實作截斷 SVD。分別保留 $k = 5, 20, 50, 100$ 個奇異值重建影像，並排比較視覺品質。同時以式 17-4 計算各 k 的能量保留率、以式 17-5 計算壓縮率。完成後回答：從哪個 k 開始，肉眼幾乎看不出與原圖的差異？此時的能量保留率與壓縮率各為多少？這個實驗如何印證「被丟掉的多半是雜訊或細節」的說法？

延伸閱讀

- 《Mining of Massive Datasets》第 11 章：Leskovec 等人著。本章的主要依據，對 PCA、SVD、CUR 有完整說明，並詳述 CUR 的抽樣策略與誤差保證。
- 《Designing Data-Intensive Applications》以外的線性代數教材：任一本線性代數教科書關於特徵值、特徵向量與 SVD 的章節，可補強本章刻意略過的數學細節。
- 隱含語意分析的原始論文：Deerwester 等人於一九九〇年發表，是 LSA 的源頭，也是現代詞向量方法的先驅。
- 隨機化數值線性代數綜述：說明如何以隨機投影加速大規模矩陣分解，是 17.6 節的深化，與第 14 章的近似思想相互呼應。

本章小結

1. 降維的四個動機是對抗維度災難、去除雜訊、節省儲存運算與支援視覺化；其中「去除雜訊」使降維得以在丟棄資訊的同時反而提升品質。
2. 降維分為特徵選擇（從既有欄位中挑選，保有可解釋性）與特徵萃取（把既有欄位組合成新維度，保留更多資訊但難以解釋）；PCA 與 SVD 屬後者。
3. PCA 尋找資料變異最大的正交方向並投影其上；因變異大代表資料點在該方向上差異明顯，保留它即保留了區分能力。使用前務必標準化，且須留意其對離群值敏感。
4. SVD 把任意矩陣分解為 $A = U\Sigma V^T$ （式 17-1）： U 連結資料列與潛在概念， V^T 連結潛在概念與資料行， Σ 的奇異值代表各概念的重要性；這些潛在概念是演算法自資料中自動發現的。
5. 截斷 SVD 只保留前 k 個奇異值（式 17-2），依 Eckart-Young 定理，它是所有秩為 k 的矩陣中最接近原矩陣者——不只是好的近似，而是最好的近似。PCA 本質上即為對中心化資料做 SVD。
6. LSA 對「詞 $\times$ 文件」矩陣做截斷 SVD，使同義詞被映射到相近位置，解決字面比對無法處理同義詞的問題；此思想與現代詞向量及大語言模型一脈相承。
7. CUR 以原矩陣的實際列與行構成分解（式 17-3），犧牲少許近似精確度，換回稀疏性與可解釋性；當原矩陣極度稀疏或需要解釋潛在維度時，往往優於 SVD。
8. 保留維度數可依能量保留率決定（式 17-4），實務常取 90% 或 95%；截斷 SVD 的壓縮率為 $k(m+n+1)/(mn)$ （式 17-5），十萬使用者乘一萬電影取 $k = 50$ 時可壓縮約 182 倍。

習題

一、觀念題

1. 請說明特徵選擇與特徵萃取的差異，並各舉一個適合使用它的情境。
2. PCA 為何以「變異最大」作為選擇投影方向的準則？請說明變異大為何等同於資訊多。
3. 請說明 SVD 中 U 、 Σ 、 V^T 三個矩陣的意義，並解釋何謂「潛在概念」。
4. SVD 在數學上有最佳性保證，為何實務上有時仍選用 CUR？請說明 CUR 的兩項優勢。

二、計算題

1. 某矩陣的奇異值由大到小為 30、20、12、6、3、2。（一）請計算各奇異值的能量（平方）與總能量；（二）以式 17-4 計算保留前 1、2、3、4 個維度的累積能量保留率；（三）若要求保留至少 95% 的能量，應取 k 為多少？
2. 某評分矩陣有 $m = 50,000$ 位使用者、 $n = 5,000$ 部電影。（一）原始矩陣需儲存多少個數值？（二）若以截斷 SVD 保留 $k = 100$ ，依式 17-5 需儲存多少個數值？（三）壓縮了幾倍？

三、思考與討論

1. 本章指出「降維能同時丟掉資訊卻提升品質」。請說明其中的道理，並討論在什麼情況下這個說法會不成立（亦即降維反而損害了結果）。
2. 案例二中，SVD 在沒有任何標註的情況下「發現」了電影類型。請討論：這種從資料中自動發現隱含結構的能力，與第 21 章要談的大語言模型有何相通之處？又有何本質差異？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

特徵選擇：從原有欄位中挑出子集，其餘丟棄；保有完全的可解釋性（保留的是「年齡」「消費金額」等具體欄位），但被丟棄欄位的資訊完全損失。適用情境：需要向非技術人員解釋模型依據、或受法規要求需說明決策理由（如信貸審核）。特徵萃取：把所有原有欄位組成成新的維度，每個新維度綜合了全部舊欄位資訊；保留較多資訊但新維度難以命名。適用情境：以預測準確度或後續運算效率為主要目標，且不需解釋個別維度意義（如影像壓縮、文件分群前的降維）。

第 2 題

因為變異衡量的是「資料點在該方向上彼此差異的程度」。變異為零的方向表示所有資料點在該方向上取值相同（如顧客國籍全為台灣），完全無法用於區分任何兩筆資料，保留它毫無價值。反之變異大的方向意味著資料點在該方向上差異明顯，保留它就保留了區分不同資料點的能力——而結構與差異正是降維時最該保住的東西。另可補充等價說法：PCA 所選的投影同時也是「使重建誤差最小」的投影，兩種觀點在數學上等價。

第 3 題

以「使用者 $\times$ 電影」評分矩陣為例： U ($m \times r$) 表示各使用者與各潛在概念的關聯強度（如某使用者有多偏好科幻）； Σ ($r \times r$ 對角) 的奇異值由大到小排列，代表各潛在概念解釋資料變異的重要程度； V^T ($r \times n$) 表示各潛在概念與各電影的關聯強度（如某電影有多「科幻」）。潛在概念是

指這些隱藏的維度——它們並非人工定義或標註，而是 SVD 純粹從資料的共現與評分模式中自動發現的抽象因素（如案例二中未經標註卻浮現的電影類型）。

第 4 題

優勢一：保有稀疏性。真實矩陣常極度稀疏，但 SVD 分解出的 U 與 V 幾乎都是稠密矩陣，儲存量反而可能超過原始稀疏格式（如案例三）；CUR 的 C 與 R 由原矩陣的實際行列構成，故保留原本的稀疏結構。優勢二：可解釋性。SVD 的維度是所有列行的線性組合（含正負加權），難以賦予意義。CUR 挑出的是真實存在的資料項目（具體的某幾位使用者或某幾部電影），可直接被業務人員理解與描述。代價是近似精確度略遜於 SVD，且因抽樣具隨機性，結果不唯一。

二、計算題

第 1 題

奇異值：30、20、12、6、3、2。

- (1) 能量（平方）依序為 900、400、144、36、9、4；總能量 $= 900 + 400 + 144 + 36 + 9 + 4 = 1,493$ 。
- (2) $k = 1$: $900/1493 \approx 60.28\%$; $k = 2$: $1300/1493 \approx 87.07\%$; $k = 3$: $1444/1493 \approx 96.72\%$; $k = 4$: $1480/1493 \approx 99.13\%$ 。
- (3) 要求保留至少 95%: $k = 2$ 僅 87.07% 不足， $k = 3$ 達 96.72% 已滿足，故取 $k = 3$ 。

延伸思考：原本六個維度只需保留三個即留住近 97% 的能量，維度減半而資訊幾乎無損；被丟棄的後三個維度能量合計僅約 3.3%，很可能主要是雜訊。

第 2 題

$m = 50,000$ 、 $n = 5,000$ 、 $k = 100$ 。

- (1) 原始矩陣： $m \times n = 50,000 \times 5,000 = 250,000,000$ （2.5 億個數值）。
- (2) 截斷 SVD（式 17-5 的分子）： $k(m + n + 1) = 100 \times (50,000 + 5,000 + 1) = 100 \times 55,001 = 5,500,100$ 個數值。
- (3) 壓縮倍數 $= 250,000,000 \div 5,500,100 \approx 45.5$ 倍（即壓縮率約 2.2%）。

延伸思考：與課文中 $m = 100,000$ 、 $n = 10,000$ 、 $k = 50$ 的例子（約 182 倍）相比，本題壓縮倍數較低，原因是 k 較大（100 對 50）而矩陣較小。可知壓縮效益取決於 k 相對於 m 、 n 的比例—— k 越小、原矩陣越大，壓縮效益越顯著。

三、思考與討論

第 1 題

道理：真實資料的訊號通常集中在少數幾個主要方向（對應大的奇異值），而測量誤差、隨機波動等雜訊則較均勻地散布在所有方向上（對應許多小的奇異值）。因此只保留前幾個大奇異值時，保住了大部分訊號卻丟掉了大部分雜訊，訊噪比反而提升——這解釋了為何矩陣分解後推薦更準、影像壓縮後仍清晰。不成立的情況：其一，若重要的訊號本身變異很小（例如某個關鍵但取值範圍狹窄的特徵），會被誤當雜訊丟棄。其二，若未經標準化，大尺度特徵會主宰主成分，導致保留的是尺度而非資訊。其三， k 取得過小，連訊號都被切掉。其四，資料本身各方向變異相近（奇異值衰減緩慢），此時本就不存在可壓縮的低維結構，強行降維只會損失資訊。

第 2 題

相通之處：兩者都是從大量未標註資料中，自動學出「隱含的表示」——SVD 從評分模式中發現潛在概念並把電影與使用者表示為低維向量；大語言模型從文字共現中學出詞向量，使語意相近的詞在向量空間中靠近。兩者都體現「分布假說」的精神：一個項目的意義，可由它與其他項目的共現模式來刻畫。LSA（17.4 節）正是這條路線的早期先驅。本質差異：其一，SVD 是線性方法，只能捕捉線性結構；而神經網路模型可學習高度非線性的關係。其二，SVD 的分解是全域且一次性的，詞的表示固定不變；現代大語言模型能依上下文動態產生表示，故可處理一詞多義（「蘋果」在不同句子中得到不同向量），這正是 LSA 做不到的。其三，SVD 有明確的最佳性保證與可解析的數學結構，神經模型則缺乏此類保證但表達力遠強。周延的回答會指出：原理一脈相承，差別在於表達能力與是否具備上下文感知。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 17 章 降維與矩陣分解

台灣自編大學教科書
@prof
大數據分析與雲端運算

第肆篇 大數據探勘與分析演算法

Mining & Analytics Algorithms

第 18 章

連結分析與大規模圖處理

Link Analysis and Large-Scale Graph Processing

機器人叫獸（李世淵） 編著

叫獸開講

我們回到第 1 章那個史丹佛的車庫。當時我說，那兩位學生把整個網際網路搬進了機器裡。但我沒說的是——把網頁抓回來只是第一步，真正的難題是：面對數億個網頁，當使用者搜尋「大學」時，該把哪一頁排在第一名？

早期搜尋引擎的做法是數關鍵字：哪一頁出現「大學」最多次，就排前面。這個方法很快就被玩壞了——有人在網頁底部用白色字寫上一千次「大學」，肉眼看不見，卻能騙過搜尋引擎衝上第一名。

那兩位學生的洞見是：不要問「這一頁自己說了什麼」，要問「別人怎麼說它」。一個網頁如果被很多網頁連結，代表它可能重要；而如果連結它的那些網頁本身也很重要，那它就更重要。這聽起來像循環定義——重要性由重要性決定——但數學上，這個循環竟然是可以解開的。

這個想法就是 PageRank，Google 的起點。它的深刻之處在於：網頁的價值不在網頁本身，而在網頁之間的「連結結構」裡。這正是本章的主題——當資料點之間存在明確的連結關係時，這些連結本身就是最珍貴的資訊。

而且這絕不只是搜尋引擎的事。社群網路裡誰是意見領袖？金流網路中哪個帳戶是洗錢的樞紐？產線的設備依賴圖上，哪台機器一停就會癱瘓全線？論文引用網路中哪篇是奠基之作？這些全都是同一類問題——而且用的是同一套數學。

學習目標

研讀完本章之後，你應該能夠：

1. 說明圖資料的表示方式，並辨識適合以圖建模的問題。
2. 說明 PageRank 的核心思想，並以隨機衝浪者模型解釋其意義。
3. 指出蜘蛛陷阱與死胡同的問題，並說明 teleport 機制如何化解。
4. 以迭代方式計算小型網路的 PageRank 值。
5. 說明 HITS 演算法中樞紐與權威的相互定義。
6. 描述 Pregel 的以頂點為中心運算模型，並說明其為何適合大規模圖處理。

核心概念

核心概念	說明
圖 (Graph)	由頂點與邊構成的資料結構，用以表示實體與其間的關係。
PageRank	以連結結構衡量頂點重要性的演算法，Google 搜尋的起點。
隨機衝浪者	PageRank 的直覺模型：隨機點擊連結的訪客，長期停留在各頁的機率。
Teleport	以一定機率隨機跳到任一頁的機制，用以化解死胡同與蜘蛛陷阱。
HITS	把重要性拆為「樞紐」與「權威」兩個相互定義的分數。
Pregel	以頂點為中心、逐輪超步進行的大規模圖運算模型。

核心概念	說明
社群發現	在圖中找出內部連結緊密、對外連結稀疏的頂點群。

表 18-1 本章核心概念一覽

18.1 網路與圖資料

18.1.1 哪些問題適合用圖來想

圖由「頂點」(vertex, 代表實體)與「邊」(edge, 代表關係)構成。當你的問題中,「關係」本身就是重點時,圖往往是最自然的表示方式。

領域	頂點	邊	想回答什麼
網頁搜尋	網頁	超連結	哪些網頁重要
社群網路	使用者	追蹤或好友	誰是意見領袖、有哪些社群
金融風控	帳戶	轉帳	哪些帳戶是異常樞紐
智慧製造	設備或工站	物料或依賴	哪個環節是瓶頸或單點風險
學術研究	論文	引用	哪些是奠基性研究
知識圖譜	概念或實體	語意關係	如何推理出未明說的關聯

表 18-2 圖模型在不同領域的對應

你會發現這與第 15 章表 15-2 的精神相同——一個好的抽象模型,換個名詞就能跨領域使用。這也是為什麼第 6 章我把圖資料庫列為 NoSQL 四大類型之一:當關係是主角時,它值得一個專屬的表示方式。

18.1.2 圖的表示與稀疏性

圖在電腦中主要有兩種表示法。鄰接矩陣是一個 $N \times N$ 的方陣,第 i 列第 j 行記錄「頂點 i 是否連到頂點 j 」。它概念清晰、便於數學推導,但有個致命問題:真實的圖幾乎都極度稀疏。

以網頁為例,一個網頁平均可能只連出數十個連結,但網頁總數以億計——鄰接矩陣中,每一列的數十億個格子裡只有數十個非零值。若真的用完整矩陣儲存,一億個頂點就需要一億乘一億個格子,完全不可行。

因此實務上一律採用鄰接串列:為每個頂點只記錄「它連到哪些頂點」的清單。儲存量與「邊的數量」成正比,而非與頂點數的平方成正比。本章所有演算法在實作時,都是在鄰接串列上進行的——雖然我們用矩陣來寫公式,因為那樣比較好懂。

18.2 PageRank 演算法

18.2.1 解開循環定義

承接叫獸開講的洞見:一個網頁的重要性,取決於連結它的網頁有多重要。這個循環定義該怎麼解?

關鍵在於再加一條規則:一個網頁會把自己的重要性,「平均分配」給它所連出的每一個網頁。若某頁的重要性是 1.0,而它連出了 4 個連結,那麼每個被連結的網頁各得 0.25。

於是每個網頁的重要性，就等於「所有連進來的網頁分給它的份額總和」。寫成數學式：

$$r(j) = \sum(i \rightarrow j) r(i) / d(i)$$

式 18-1 PageRank 的基本定義 ($d(i)$ 為 i 的連出數)

這是一組聯立方程式。理論上可以直接解，但當網頁有數億個時，直接解方程完全不可行。實務上採用「迭代」：先給所有網頁相同的初始值（例如各 $1/N$ ），然後反覆套用上式，讓數值逐步收斂到穩定狀態。

18.2.2 隨機衝浪者模型

PageRank 有一個非常優美的直覺解釋，稱為「隨機衝浪者」（random surfer）模型。

想像一位訪客，隨機打開一個網頁，然後在該頁的連結中隨機挑一個點下去，接著再隨機挑一個，如此無止盡地點下去。經過很長時間後，問：他此刻停留在網頁 j 的機率是多少？

答案正好就是網頁 j 的 PageRank 值。也就是說，PageRank 衡量的是「一位隨機瀏覽者長期而言會有多常造訪這一頁」。這個解釋不只直觀，還告訴我們一件事——所有網頁的 PageRank 值加起來必然等於 1（因為訪客總得在某一頁）。

這個模型也解釋了為何 PageRank 難以被作弊。你可以在自己的網頁上放一萬個連結指向自己，但那並不會讓隨機訪客更常走到你這裡——因為訪客要先走到你的網頁才會看到那些連結，而重要性是由「別人指向你」決定的。

18.2.3 兩個會壞事的結構

上述模型有兩個致命的問題，它們都與真實網路的結構有關。

- **死胡同 (Dead End)**：某個網頁沒有任何連出的連結。隨機訪客走到這裡就出不去了，它會像黑洞一樣把重要性吸光——迭代到最後，所有網頁的 PageRank 都會趨近於 0。
- **蜘蛛陷阱 (Spider Trap)**：一群網頁只互相連結、不連向外界（例如一個網頁只連向自己）。訪客一旦進入就永遠出不來，最終這群網頁會吸走全部的重要性，其餘網頁歸零。

這兩個問題不是理論上的假想——真實網路中到處都是。若不處理，PageRank 算出來的結果會完全失去意義。

18.2.4 Teleport: 一個優雅的補丁

解法出奇地簡單：讓隨機衝浪者「偶爾會膩」。

修改後的模型是：在每一步，訪客有 β 的機率（通常取 0.85）照常隨機點擊一個連結；但有 $1 - \beta$ 的機率（約 0.15）會失去耐性，直接在網址列輸入一個隨機網址，跳到任意一頁去。這個隨機跳躍稱為 teleport。

加入 teleport 後，兩個問題同時被解決：走進死胡同的訪客會被 teleport 帶走，陷在蜘蛛陷阱裡的訪客也總會跳出來。修正後的公式為：

$$r(j) = \beta \sum(i \rightarrow j) r(i) / d(i) + (1 - \beta) / N$$

式 18-2 含 teleport 的 PageRank (β 常取 0.85)

這個 β 稱為阻尼係數 (damping factor)。0.85 這個數字的由來，大致對應「使用者平均點擊約六、七個連結後就會換個主題重新開始」的行為觀察。

18.3 主題敏感 PageRank 與 TrustRank

18.3.1 同一個網頁，不同的重要性

標準 PageRank 給每個網頁一個全域的分數。但這忽略了一件事：重要性其實是相對於主題的。一個頂尖的機器人學研究網站，對搜尋「SLAM 演算法」的人極為重要，對搜尋「食譜」的人則毫無價值。

主題敏感 PageRank (Topic-Sensitive PageRank) 的做法很巧妙：它只修改 teleport 的目標。標準版本中，teleport 是「跳到任意一頁」（機率均等）；主題敏感版本則改為「跳到某個主題的代表性網頁集中的一頁」。

這個小改動的效果很大：因為訪客總是被拉回該主題的網頁，重要性自然就集中在與該主題相關的網路區域。針對不同主題各算一份 PageRank，就能依使用者的查詢意圖給出不同的排序。

18.3.2 TrustRank：對抗垃圾網頁

PageRank 雖然比數關鍵字難作弊，但作弊者很快發展出新招：建立大量互相連結的「連結農場」，人為製造大量的連入連結，把目標網頁的分數拱上去。

TrustRank 借用了主題敏感 PageRank 的機制來對抗它。做法是：先由人工挑選一小批確定可信的網站（如知名大學、政府機關、主流媒體）作為「種子」，然後把 teleport 的目標限定在這批種子上。

如此一來，重要性會從可信的種子出發向外流動——距離可信網站越近（透過連結路徑）的網頁，分數越高。而垃圾網頁即使彼此連結再密集，因為沒有可信網站願意連向它們，就得不到重要性的挹注。

這個設計的思想很值得記住：信任不是憑空產生的，而是從已知可信的源頭「傳播」出來的。你會在資安、供應鏈驗證等許多領域看到同樣的思路。

18.4 HITS 演算法

18.4.1 兩種不同的「重要」

HITS (Hyperlink-Induced Topic Search) 與 PageRank 大約同時提出，但它有一個不同的洞見：網頁的重要性其實有兩種，不該混為一談。

- **權威 (Authority)**：本身提供優質內容的網頁。例如某篇權威的技術文件、某個官方規格頁面。
- **樞紐 (Hub)**：本身內容未必出色，但整理了大量指向優質內容的連結。例如一份精心編纂的資源導覽頁。

這個區分在真實網路中很有意義。一份好的「延伸閱讀清單」本身沒有原創內容，但它的價值在於幫你找到好東西——這種價值，用單一分數的 PageRank 難以表達。

18.4.2 相互定義與迭代

HITS 的核心是一組相互定義：

一個網頁是好的權威，若它被許多好的樞紐所指向；一個網頁是好的樞紐，若它指向許多好的權威。

$$a(j) = \sum(i \rightarrow j) h(i) \quad h(i) = \sum(i \rightarrow j) a(j)$$

式 18-3 HITS 的權威與樞紐分數

與 PageRank 一樣，這個循環也用迭代來解：先給所有頁面相同的初始值，然後交替更新權威分數與樞紐分數，每輪之後做正規化（避免數值無限膨脹），直到收斂。

HITS 與 PageRank 還有一個重要差異：PageRank 是對整個網路計算一次、與查詢無關（可離線預先算好）；而 HITS 傳統上是先依查詢找出一個相關的子圖，再在該子圖上計算——它是查詢相關的。這使 HITS 更貼合查詢意圖，但也意味著無法預先算好，即時計算的成本較高。這正是 Google 選擇 PageRank 路線的實務理由之一。

叫獸提醒

我要你注意 PageRank 與 HITS 共同的數學本質：兩者都是在解「特徵向量」問題。PageRank 找的是轉移矩陣的主特徵向量，HITS 找的是相關矩陣的主特徵向量，而它們用的迭代方法（反覆相乘直到穩定）就是線性代數中的「冪法」。你看，第 17 章的 SVD 也是在找特徵結構——這兩章看似在談完全不同的事（降維與網頁排序），骨子裡卻是同一套數學。這就是為什麼我一直說：學原理比學工具划算，因為原理會在你意想不到的地方重複出現。

18.5 圖運算模型：Pregel 與 GraphX

18.5.1 為什麼 MapReduce 不適合圖

PageRank 需要迭代數十輪。若用 MapReduce 實作，每一輪都是一個獨立的作業——每輪都要把整張圖從磁碟讀出、算完再寫回（回想第 9 章與第 11 章的痛點）。迭代三十輪，就是三十次全圖的讀寫，成本高得驚人。

更根本的問題是，MapReduce 的鍵值對模型並不貼合圖的結構。圖運算的本質是「頂點與鄰居交換資訊」，但在 MapReduce 中，你必須把圖的結構在每一輪都重新傳遞一次——明明結構從頭到尾沒變，卻要反覆搬運。

18.5.2 以頂點為中心的思考

Google 為此提出了 Pregel 模型，其核心口號是「像頂點一樣思考」（think like a vertex）。

在這個模型中，你不再撰寫「對整張圖做什麼」的程式，而是撰寫「單一個頂點在每一輪該做什麼」。整個運算分為一輪一輪的「超步」（superstep），每個超步中，每個頂點做三件事：

- 第 1 步。** 接收上一個超步中鄰居傳來的訊息。
- 第 2 步。** 依據這些訊息更新自己的狀態（例如更新自己的 PageRank 值）。
- 第 3 步。** 把新的值傳送給自己的鄰居，供下一個超步使用。

當所有頂點都不再改變（或達到指定輪數），運算就結束。以 PageRank 為例，每個頂點的邏輯簡單到只有幾行：把收到的所有值加總、乘上 β 、加上 teleport 項、然後除以自己的連出數再送給每個鄰居。

這個模型的優勢很明顯：圖的結構常駐在記憶體中不必反覆搬運，每輪只傳遞訊息（資料量遠小於整張圖），而且各頂點的運算天然平行。這使它比 MapReduce 版本快上數十倍。

18.5.3 GraphX 與其他實作

Pregel 是 Google 的內部系統，開源世界有多個對應實作，其中與本書關係最密切的是 Spark 的 GraphX。

GraphX 的巧妙之處在於，它把圖建立在第 11 章的 RDD 之上——頂點與邊分別以 RDD 表示，圖運算則轉換為 RDD 的操作。這帶來一個實務上的大優點：你可以在同一個 Spark 程式中，無縫地把圖運算與 SQL 查詢、機器學習串接起來，不必在不同系統之間搬運資料。

除此之外還有 Apache Giraph（Hadoop 生態系的 Pregel 實作）等選擇。它們的核心模型大同小異，差別主要在生態系整合與效能特性。

18.6 社群發現與 SimRank

18.6.1 什麼是社群

社群（community）指的是圖中一群頂點，它們彼此之間連結緊密，但與群外的連結稀疏。直覺上就是「抱團」的一群——社群網路中的朋友圈、學術網路中的研究領域、產品網路中常被一起購買的商品群。

你可能會覺得這聽起來像第 16 章的分群——確實如此，但有個關鍵差異：第 16 章的分群依據是「特徵空間中的距離」，而社群發現的依據是「連結結構」。兩個頂點可能特徵完全不同，卻因為緊密相連而屬於同一社群。

18.6.2 常見的方法

- **模組度最佳化：**定義一個「模組度」（modularity）指標，衡量群內連結相對於隨機情況的密集程度，再尋找使它最大的分群方式。Louvain 演算法是最廣為使用的實作，速度快且適合大圖。
- **邊介數分割：**計算每條邊的「介數」（有多少最短路徑經過它），反覆移除介數最高的邊——因為連接兩個社群的橋樑邊，必然承載大量最短路徑。概念優雅但計算昂貴。
- **標籤傳播：**每個頂點先給一個獨特標籤，然後反覆採用「鄰居中最多數的標籤」，最終標籤相同者即為同一社群。極快且天然適合 Pregel 模型，但結果較不穩定。

18.6.3 SimRank：結構上的相似

最後介紹一個與第 14 章遙相呼應的概念。第 14 章我們用內容（shingle 集合）來判斷相似，而 SimRank 提出了另一種觀點——用「結構位置」來判斷相似。

SimRank 的核心定義同樣是循環的：兩個頂點相似，若它們所連結的鄰居們彼此相似。

這個定義能捕捉到內容比對抓不到的關聯。例如兩篇論文一個字都沒引用彼此、用詞也不同，但它們引用了幾乎相同的一批參考文獻——SimRank 會判定它們高度相似，因為它們在引用網路中佔據了相似的結構位置。同樣地，兩位使用者若追蹤了幾乎相同的一群人，即使他們從未互動，也可能有相近的興趣。

SimRank 的計算同樣以迭代進行，但成本相當高（涉及所有頂點對），實務上多採近似或抽樣的做法——這又回到了本篇反覆出現的主題：以可控的近似，換取可行的計算。

18.6.4 承先啟後

本章我們回到了第 1 章的車庫，理解了 Google 起家的那個洞見——價值藏在連結結構裡。我們認識了圖資料的表示與稀疏性（18.1）、PageRank 如何以隨機衝浪者模型解開循環定義並以 teleport 化解死胡同與蜘蛛陷阱（18.2）、主題敏感版本與 TrustRank 如何處理主題性與作弊（18.3）、HITS 如何把重要性拆為權威與樞紐（18.4）、Pregel「像頂點一樣思考」的運算模型（18.5），以及社群發現與 SimRank（18.6）。

我特別希望你帶走兩個觀念。第一，PageRank 與 HITS 骨子裡都是在找特徵向量，與第 17 章的 SVD 是同一套數學——原理會在意想不到的地方重複出現。第二，Pregel 的「像頂點一樣思考」是一種重要的思維轉換：從「對整體做什麼」轉為「單一個體該做什麼」，而整體行為由個體的局部互動湧現。這個思路在第 22 章的多代理系統中會再度出現。

接下來，我們要把前幾章的工具兜在一起，回答一個非常實際的問題：如何預測「某個使用者會喜歡什麼」？這需要用到第 17 章的矩陣分解、本章的圖結構思維，還有一些新的觀念。這就是第 19 章的主題——推薦系統。

公式與流程：手算一次 PageRank 迭代

這一節我們用一個只有三個頂點的迷你網路，親手把 PageRank 算到收斂——你會看見「迭代」是如何解開那個循環定義的。

設定

設有三個網頁 A、B、C，連結關係為：A 連向 B 與 C；B 連向 C；C 連向 A。取阻尼係數 $\beta = 0.85$ ，頂點數 $N = 3$ ，初始值皆為 $1/3 \approx 0.3333$ 。

依式 18-2，每一輪的更新規則為：

$$r(j) = 0.85 \times \sum(i \rightarrow j) r(i) / d(i) + 0.15 / 3$$

式 18-4 本例的迭代式 ($0.15/3 = 0.05$)

其中 A 的連出數 $d(A) = 2$ （連向 B、C），故 A 分給 B 與 C 的各是 $r(A)/2$ ；B 的連出數 $d(B) = 1$ ，全部給 C；C 的連出數 $d(C) = 1$ ，全部給 A。

以第一輪的 B 為例：只有 A 連向 B，故 $r(B) = 0.85 \times (0.3333/2) + 0.05 = 0.85 \times 0.1667 + 0.05 = 0.1417 + 0.05 = 0.1917$ 。其餘同理。

輪次	r(A)	r(B)	r(C)
初始	0.3333	0.3333	0.3333
1	0.3333	0.1917	0.4750
2	0.4538	0.1917	0.3546
3	0.3514	0.2428	0.4058
4	0.3949	0.1993	0.4058
6	0.3792	0.2178	0.4030
8	0.3869	0.2168	0.3963

表 18-3 PageRank 的迭代收斂過程 ($\beta = 0.85$)

觀察這張表：數值一開始擺盪得厲害，但擺幅逐輪縮小，逐漸穩定下來。第八輪時 A 與 C 相近（約 0.39）、B 明顯較低（約 0.22）——這符合直覺，因為 B 只被 A 連結一次，且 A 還把重要性分了一半給 C。

另外請注意：每一輪的三個值相加都等於 1，這正是隨機衝浪者模型的必然結果（訪客總得停在某一頁）。這個性質也是實作時很好用的檢查點。

收斂的判定

實務上不會固定跑幾輪，而是設定一個收斂門檻。常見做法是計算前後兩輪的差異總和：

$$\text{誤差} = \sum_j |r_{t+1}(j) - r_t(j)|$$

式 18-5 迭代的收斂判定

當誤差小於某個極小值（如 10^{-6} ）時即停止。一般而言，即使是數億個頂點的網路，PageRank 通常在數十輪內就能收斂到足夠的精度——這也是它能實用的關鍵原因。

叫獸提醒

這裡有個常被忽略的實務重點： β 的選擇會直接影響收斂速度。 β 越接近 1，teleport 的比重越小，重要性在網路中流動得越遠，需要的迭代輪數就越多； β 越小則收斂越快，但也越接近「大家都差不多」的均勻分布，失去鑑別力。0.85 是實務上找到的平衡點——既保有足夠的鑑別力，又能在數十輪內收斂。你在調校任何迭代型演算法時，都會遇到這類「精度與速度」的取捨，而多數時候，業界的預設值都是前人踩過坑後留下的智慧，別隨意更動。

案例研討

案例一 白色文字的騙局

回到叫獸開講。早期搜尋引擎以關鍵字出現次數排名，於是出現了「關鍵字堆砌」的作弊法：在網頁底部用與背景相同顏色的文字，重複寫上數百次目標關鍵字。使用者看不到，搜尋引擎卻照單全收。

PageRank 為何能抵擋這一招？因為它的分數不由「網頁自己說了什麼」決定，而由「別人是否連結它」決定。作弊者可以完全控制自己網頁的內容，卻無法控制別人要不要連向他。

這個案例點出了一個重要的設計原則：當一個指標可以被評估對象自己操控時，它遲早會被操弄；好的指標應該建立在「他人的行為」之上。這個原則不只適用於搜尋引擎——你在設計任何評分或排名機制時（包括學術評鑑、信用評等）都該想想這件事。

案例二 連結農場與信任的傳播

作弊者很快找到了 PageRank 的破口：既然分數來自連入連結，那就自己造一大堆網頁互相連結，形成「連結農場」，再全部指向目標網頁。這確實能把分數拱上去。

TrustRank 的對策是 18.3.2 節的思路：先人工指定一小批確定可信的種子網站，把 teleport 的目標限定在這些種子上。於是重要性從可信的源頭出發、沿著連結向外擴散，越靠近可信網站的頁面分數越高。

連結農場即使內部連結再密集，也得不到分數——因為沒有任何可信網站願意連向它們，等於被隔絕在信任的傳播路徑之外。這個案例展示了一個深刻的觀念：在開放且有對抗性的環境中，「信任

必須有源頭」。這與第 8 章區塊鏈的思路異曲同工——都是在互不信任的世界裡，設法建立可靠的判準。

案例三 產線的單點風險

某工廠想找出「哪一台設備一旦停機，對整條產線的衝擊最大」。他們把工站建模為頂點、把物料與依賴關係建模為有向邊，然後對這張圖執行 PageRank。

結果找出的高分節點，並不是最昂貴或最顯眼的設備，而是一台位於多條產線交會處的中介工站——它本身不起眼，卻是大量物流路徑的必經之處。此前的維護排程完全沒有特別重視它。

團隊據此調整了備品庫存與預防保養的優先序。這個案例展示了表 18-2 的實務價值：同一套演算法，只要把「網頁」換成「工站」、「超連結」換成「物料流」，就能用於製造領域的風險分析。這也再次呼應本書反覆的主張——原理會遷移，工具不會。

案例四 三十輪迭代的代價

某團隊最初以 MapReduce 實作 PageRank，處理一張含數億頂點的圖。每一輪迭代都是一個獨立的 MapReduce 作業——把整張圖讀出、計算、再寫回磁碟。跑滿三十輪需要十幾個小時。

他們改用 Spark GraphX (Pregel 模型) 重寫後，執行時間降到不到一小時。關鍵差異有二：其一，圖的結構常駐記憶體，不必每輪重新讀寫（呼應第 11 章 Spark 對 MapReduce 的改進）；其二，每輪只在頂點間傳遞數值訊息，傳輸量遠小於整張圖的結構。

這個案例把第 9、11 與本章串了起來：MapReduce 的「中間結果必須落地」痛點，在迭代型的圖演算法上被放到最大；而 Pregel 的以頂點為中心模型，正是為這類工作量身打造的解方。

實作活動

活動一 手算 PageRank

請以紙筆重現表 18-3 的計算，親自體會迭代如何解開循環定義。

- 步驟 1.** 畫出三個網頁 A、B、C 的連結圖 ($A \rightarrow B$ 、 $A \rightarrow C$ 、 $B \rightarrow C$ 、 $C \rightarrow A$)，並寫出各自的連出數。
- 步驟 2.** 取 $\beta = 0.85$ 、初始值皆為 $1/3$ ，依式 18-4 手算第一輪與第二輪的三個 PageRank 值。
- 步驟 3.** 與表 18-3 對照，確認你的計算正確；並驗證每一輪的三值總和是否為 1。
- 步驟 4.** 試著把 $C \rightarrow A$ 這條邊移除（使 C 成為死胡同），在「不加 teleport」的情況下手算三輪，觀察數值如何逐漸流失。

活動二 以程式實作 PageRank

請以 Python 實作 PageRank，並觀察參數的影響。

- 步驟 1.** 以鄰接串列表示一張中小型的圖（可自建或取用開放的網路資料集）。
- 步驟 2.** 依式 18-2 撰寫迭代函式，並以式 18-5 判定收斂（門檻取 10^{-6} ）。
- 步驟 3.** 分別以 $\beta = 0.5$ 、 0.85 、 0.95 執行，記錄各自需要的迭代輪數，驗證叫獸提醒中「 β 越大收斂越慢」的說法。
- 步驟 4.** 找出分數最高的前十個頂點，並檢視它們是否符合直覺。

活動三 為你的領域建一張圖

請參考表 18-2，為你熟悉的一個領域設計圖模型。明確定義：頂點代表什麼？邊代表什麼？邊是有向還是無向、是否有權重？接著說明：在這張圖上執行 PageRank，高分的頂點會具有什麼實務意義？若改為執行社群發現，找出的社群又代表什麼？最後思考：這張圖大約會有多少頂點與邊？以鄰接矩陣儲存需要多少空間？以鄰接串列又需要多少？

延伸閱讀

- **《Mining of Massive Datasets》第 5、10 章：**Leskovec 等人著。第 5 章完整推導 PageRank、teleport 與主題敏感版本，第 10 章討論社群發現與 SimRank，是本章的主要依據。
- **Brin 與 Page 的原始論文：**一九九八年發表的《The Anatomy of a Large-Scale Hypertextual Web Search Engine》，Google 的起點，兼具技術與工程視野，值得一讀。
- **Pregel 論文：**Google 於二〇一〇年發表，說明「像頂點一樣思考」的運算模型與超步機制，是理解 18.5 節的第一手資料。
- **Apache Spark GraphX 官方文件：**說明如何在 Spark 中以 Pregel API 實作圖演算法，並附有 PageRank 與連通元件的範例程式。

本章小結

1. 圖以頂點與邊表示實體與關係，適用於網頁、社群、金流、設備依賴、引用與知識圖譜等領域；真實的圖極度稀疏，故實作上一律採鄰接串列而非鄰接矩陣。
2. PageRank 的核心思想是「重要性由連入的網頁決定，且每頁把重要性平均分給連出對象」（式 18-1）；此循環定義以迭代求解，其直覺為隨機衝浪者長期停留在各頁的機率，故所有分數總和為 1。
3. 死胡同會把重要性吸乾、蜘蛛陷阱會把重要性壟斷；teleport 機制讓衝浪者以 $1 - \beta$ 的機率隨機跳到任一頁（式 18-2），同時化解兩者， β 通常取 0.85。
4. 主題敏感 PageRank 只修改 teleport 的目標為某主題的代表頁，使重要性集中於該主題區域；TrustRank 則把 teleport 目標限定為人工挑選的可信種子，使信任由源頭傳播，藉此對抗連結農場。
5. HITS 把重要性拆為權威（提供優質內容）與樞紐（整理優質連結）兩個相互定義的分數（式 18-3），同樣以迭代求解；它是查詢相關的，故無法離線預算，即時成本高於 PageRank。
6. PageRank 與 HITS 本質上都是在解特徵向量問題，其迭代方法即線性代數中的冪法——與第 17 章的 SVD 屬同一套數學。
7. MapReduce 不適合迭代型圖運算（每輪需全圖讀寫且需反覆傳遞結構）；Pregel 以「像頂點一樣思考」的超步模型，讓結構常駐記憶體、每輪僅傳遞訊息，效能可提升數十倍，GraphX 為其建立於 RDD 之上的實作。
8. 社群發現以連結結構（而非特徵距離）找出內密外疏的頂點群，常見方法有模組度最佳化、邊介數分割與標籤傳播；SimRank 則以「鄰居相似則彼此相似」的循環定義，捕捉結構位置上的相似性。

習題

一、觀念題

1. PageRank 為何比「計算關鍵字出現次數」更難被作弊？請說明其背後的设计原則。
2. 請說明死胡同與蜘蛛陷阱各會造成什麼問題，以及 teleport 如何同時化解兩者。

3. 請說明 HITS 中「權威」與「樞紐」的差異，並各舉一個實際的網頁例子。
4. 為什麼 MapReduce 不適合實作 PageRank? Pregel 的「以頂點為中心」模型如何改善？

二、計算題

1. 設有三個網頁：X 連向 Y 與 Z、Y 連向 Z、Z 連向 X。取 $\beta = 0.8$ 、初始值皆為 $1/3$ 。請依式 18-2 手算第一輪迭代後三者的 PageRank 值，並驗證總和是否為 1。
2. 承上題，若把 Z→X 的連結移除（Z 成為死胡同）且不使用 teleport（即 $\beta = 1$ ），請計算前三輪的 PageRank 值，並說明數值總和發生了什麼變化、為什麼。

三、思考與討論

1. 案例一指出「當一個指標可以被評估對象自己操控時，它遲早會被操弄」。請就你熟悉的一種評分或排名機制（如學術引用指標、電商評價、社群按讚數）討論這個問題，並提出可能的改善方向。
2. 本章的 PageRank 與第 17 章的 SVD 都在解特徵向量問題。請討論：為什麼「找主特徵向量」這件事，能同時回答「哪個網頁重要」與「資料的主要結構是什麼」這兩個看似無關的問題？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

因為關鍵字次數是「網頁自己說了什麼」，作弊者可完全控制（如白色文字堆砌關鍵字）；而 PageRank 的分數來自「別人是否連結它」，作弊者無法控制他人的行為。設計原則：當一個指標可被評估對象自己操控時，它遲早會被操弄；好的指標應建立在「他人的行為」之上。可補充：此原則並非萬無一失——連結農場即是作弊者設法「製造他人」來規避（見案例二），故仍需 TrustRank 等進一步的防禦。

第 2 題

死胡同：沒有連出邊的頂點，隨機衝浪者走進去就出不來，重要性只進不出而被吸收，迭代後所有頂點的分數會趨近於 0（總和不再為 1）。蜘蛛陷阱：一群只互相連結、不連向外界的頂點，衝浪者進入後永遠出不去，最終這群頂點會壟斷全部重要性，其餘頂點歸零。teleport 的化解方式：讓衝浪者在每一步都有 $1 - \beta$ 的機率隨機跳到任意頁面——如此走進死胡同者會被帶走，陷在蜘蛛陷阱者也總會跳出，兩個問題同時解決，且保證分數總和維持為 1。

第 3 題

權威：本身提供優質原創內容的網頁，價值在於內容本身，例如官方技術規格文件、權威機構發布的統計報告、原創的研究論文頁面。樞紐：本身內容未必出色，但整理了大量指向優質內容的連結，價值在於「幫你找到好東西」，例如精心編纂的資源導覽頁、課程的延伸閱讀清單、領域入門的資源彙整。兩者相互定義：好權威被許多好樞紐指向，好樞紐指向許多好權威。此區分的意義在於——一份好的連結清單本身沒有原創內容，其價值難以用單一分數的 PageRank 表達。

第 4 題

MapReduce 不適合的原因：其一，PageRank 需迭代數十輪，而 MapReduce 每輪都是獨立作業，須將整張圖從磁碟讀出、算完再寫回，三十輪即三十次全圖讀寫（呼應第 9、11 章「中間結果必須落地」的痛點）；其二，圖的結構從頭到尾不變，卻必須在每一輪的鍵值對中反覆傳遞，造成大量無謂的資料搬運。Pregel 的改善：以超步為單位，每個頂點只需接收鄰居訊息、更新自身狀態、再送出新值；圖的結構常駐記憶體不必反覆讀寫，每輪僅傳遞數值訊息（資料量遠小於整張圖），且各頂點運算天然平行，效能可提升數十倍。

二、計算題

第 1 題

連出數： $d(X) = 2$ （連向 Y、Z）、 $d(Y) = 1$ （連向 Z）、 $d(Z) = 1$ （連向 X）。 $\beta = 0.8$ ，故 teleport 項為 $(1 - 0.8)/3 = 0.2/3 \approx 0.0667$ 。初始值皆 $1/3 \approx 0.3333$ 。

- (1) $r(X)$ ：連入者僅 Z（全額）。 $r(X) = 0.8 \times 0.3333 + 0.0667 = 0.2667 + 0.0667 = 0.3333$ 。
- (2) $r(Y)$ ：連入者僅 X（分一半）。 $r(Y) = 0.8 \times (0.3333/2) + 0.0667 = 0.8 \times 0.1667 + 0.0667 = 0.1333 + 0.0667 = 0.2000$ 。
- (3) $r(Z)$ ：連入者為 X（分一半）與 Y（全額）。 $r(Z) = 0.8 \times (0.1667 + 0.3333) + 0.0667 = 0.8 \times 0.5 + 0.0667 = 0.4 + 0.0667 = 0.4667$ 。
- (4) 驗證總和： $0.3333 + 0.2000 + 0.4667 = 1.0000$ ，符合隨機衝浪者模型的必然性質。

第 2 題

移除 $Z \rightarrow X$ 後，Z 成為死胡同（連出數為 0，其重要性無處可去而消失）。 $\beta = 1$ 表示無 teleport，故更新式簡化為 $r(j) = \sum(i \rightarrow j) r(i)/d(i)$ 。初始皆 $1/3 \approx 0.3333$ 。

- (1) 第一輪： $r(X) = 0$ （無人連向 X）； $r(Y) = 0.3333/2 = 0.1667$ ； $r(Z) = 0.3333/2 + 0.3333 = 0.5000$ 。總和 = 0.6667。
- (2) 第二輪： $r(X) = 0$ ； $r(Y) = 0/2 = 0$ ； $r(Z) = 0/2 + 0.1667 = 0.1667$ 。總和 = 0.1667。
- (3) 第三輪： $r(X) = 0$ ； $r(Y) = 0$ ； $r(Z) = 0$ 。總和 = 0。

總和變化與原因：總和由 1 逐輪遞減至 0。因為 Z 沒有連出邊，每一輪流入 Z 的重要性都被「吸收」而無法再流出，等同於從系統中消失；X 又完全依賴 Z 的回流，故一併歸零。這正是死胡同的破壞性——最終所有頂點的分數都趨近於 0，排名完全失去意義。這也說明了 teleport 為何是必要而非可選的設計。

三、思考與討論

第 1 題

以學術引用指標為例：可被操控之處包括自我引用、朋友間互相引用的「引用聯盟」、以及為衝數量而切割成多篇小論文——本質上與連結農場同構。以電商評價為例：可被操控之處包括刷單好評、競爭對手惡意負評、以及以贈品換取五星。改善方向可包括：其一，引入「來源可信度加權」（如 TrustRank 的思路，來自高可信評論者或高影響力期刊的引用權重更高）；其二，偵測異常的結構樣態（互引環路、短時間集中的評價、帳號註冊時間與行為的異常）；其三，採用難以偽造的行為訊號（實際購買紀錄、閱讀時長）而非易於製造的顯性訊號；其四，多指標並用並定期更換演算法，提高作弊成本。核心觀念：任何單一旦公開的指標，在有足夠誘因時都會被最佳化到失去意義（此即 Goodhart 定律）。

第 2 題

關鍵在於「主特徵向量代表系統在反覆作用下的穩定狀態或主導方向」。PageRank 中，轉移矩陣描述重要性如何一輪輪在網路中流動，反覆相乘後收斂到的穩定分布，就是該矩陣的主特徵向量——它回答「長期而言重要性會累積在哪裡」。SVD 中，資料矩陣的主特徵方向代表變異最大的方向，即資料在此方向上被「拉伸」得最厲害，故它回答「資料的主要結構沿哪個方向展開」。兩者相通之處在於：都是在問「當這個線性變換被反覆施加時，什麼會被放大並存留下來」——被放大的即為主導成分。可補充：兩者的迭代解法（冪法）也完全相同，都是反覆相乘直到穩定。這說明了為何線性代數是資料科學的共同語言——許多表面不同的問題，化為矩陣後其實是同一個數學問題。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 18 章 連結分析與大規模圖處理

台灣自編大學教科書

@prof

大數據分析與雲端運算

第肆篇 大數據探勘與分析演算法

Mining & Analytics Algorithms

第 19 章

推薦系統

Recommender Systems

機器人叫獸（李世淵） 編著

叫獸開講

我想先問你一個問題：你上一次「主動」決定要看哪部影片、聽哪首歌、買哪件商品，是什麼時候？

多數人會發現，答案是「想不太起來」。我們的選擇早已被推薦系統包圍——串流平台首頁的那幾排片單、購物網站的「猜你喜欢」、社群平台的動態排序。有統計指出，某些平台上超過七成的觀看時數，來自推薦而非使用者主動搜尋。

這件事有它迷人的一面，也有它令人不安的一面，我兩邊都會講。迷人的是：推薦系統解決了「選擇過載」——面對數十萬部影片，人類根本無從選起。不安的是：當演算法決定你看到什麼，它也就在某種程度上塑造了你的品味、你的世界觀。這一章的最後，我們會認真談這個問題。

技術上，這一章是第肆篇的一次總驗收。你會看到第 17 章的矩陣分解在這裡找出「隱含的品味維度」、第 14 章的相似度在這裡衡量誰跟誰像、第 18 章的圖思維在這裡把使用者與商品連成網路。前面學的東西，會在這一章匯流。

我還要先給你一個心理建設：推薦系統最難的地方，往往不是演算法，而是資料的稀疏。一個有百萬使用者、十萬商品的平台，評分矩陣中可能超過 99.9% 的格子是空的。如何從這麼稀少的訊號中推測人的偏好，才是真正的挑戰。

學習目標

研讀完本章之後，你應該能夠：

1. 說明推薦系統的價值，並區分顯性與隱含回饋。
2. 說明內容式推薦的原理，並指出其優缺點。
3. 區分使用者導向與物品導向的協同過濾，並計算相似度與預測評分。
4. 說明矩陣分解如何以隱含因子預測未知評分。
5. 說明冷啟動問題的三種型態，並提出因應策略。
6. 運用 RMSE 等指標評估推薦品質，並理解離線指標的侷限。

核心概念

核心概念	說明
效用矩陣	以使用者為列、物品為行，記錄偏好程度的矩陣，通常極度稀疏。
顯性與隱含回饋	前者為使用者主動給的評分，後者由行為（點擊、觀看時長）推得。
內容式推薦	依物品本身的特徵，推薦與使用者過去喜好相似的物品。
協同過濾	依「相似使用者」或「相似物品」的偏好來預測，不看物品內容。
矩陣分解	把效用矩陣分解為使用者因子與物品因子，藉以預測未知評分。
冷啟動	新使用者、新物品或新系統缺乏歷史資料而難以推薦的困境。
RMSE	均方根誤差，衡量預測評分與實際評分差距的常用指標。

表 19-1 本章核心概念一覽

19.1 推薦系統概觀

19.1.1 長尾與選擇過載

推薦系統之所以重要，有兩個結構性的原因。

第一是「選擇過載」。實體書店受限於空間，只能陳列數千本暢銷書；但線上平台可以放上數百萬本。看似是好事，卻造成新問題——使用者根本不可能瀏覽完，多數商品永遠不會被看見。

第二是「長尾效應」。在實體通路中，冷門商品因為佔用架位卻賣不動而被淘汰；但在線上，儲存一件冷門商品的成本趨近於零。於是那些各自銷量微小、但種類極多的冷門商品，加總起來可能佔營收的相當比重——這就是長尾。

而要讓長尾發揮價值，關鍵就在於「如何把冷門商品送到會喜歡它的人面前」。這正是推薦系統的核心任務：它不只是提升點擊率的工具，更是讓長尾得以成立的基礎設施。

19.1.2 效用矩陣與稀疏性

推薦問題的標準抽象是「效用矩陣」(utility matrix)：以使用者為列、物品為行，每一格記錄該使用者對該物品的偏好程度。我們的任務，就是預測那些「空格」的值。

這裡有一個必須先認清的現實：這個矩陣極度稀疏。假設平台有一百萬使用者、十萬部影片，矩陣共有一千億格；而平均每位使用者可能只看過一百部影片，故已知的格子僅約一億個——填滿率僅約 0.1%，超過 99.9% 是空白。

這個稀疏性帶來雙重的困難：一方面可用的訊號極少，另一方面矩陣又大到無法直接處理。本章介紹的所有方法，本質上都是在回答同一個問題——如何從這 0.1% 的已知，推測其餘 99.9%。

19.1.3 顯性與隱含回饋

填入效用矩陣的資料，來源有兩種，它們的性質差異很大。

面向	顯性回饋	隱含回饋
來源	使用者主動給的評分、按讚	點擊、觀看時長、購買、停留
優點	意圖明確、可信度高	資料量大、無需使用者額外付出
缺點	極為稀少、且有評分偏誤	意圖模糊、缺乏負面訊號
「沒有紀錄」的意義	尚未評分，態度不明	可能是不喜歡，也可能是沒看過

表 19-2 顯性回饋與隱含回饋的比較

叫獸提醒

表 19-2 最後一列是實務上最容易踩的坑。隱含回饋只有「正例」——你看到某人點了什麼，卻永遠不知道他「沒點」是因為不喜歡，還是根本沒看到。如果你天真地把「沒有紀錄」全部當作「不喜歡」來訓練模型，就會把大量「其實會喜歡但還沒遇到」的商品標成負面——而那些商品，恰恰是推薦系統最該挖掘的。這個問題有專門的處理方法（如加權的隱含回饋模型），但更重要的是：你要意識到它的存在。資料的「缺失」本身也是一種資訊，而如何詮釋

缺失，往往決定了整個系統的成敗。

19.2 內容式推薦

19.2.1 用物品的特徵來推薦

內容式推薦（content-based）的思路最直觀：分析物品本身的特徵，然後推薦「與使用者過去喜歡的物品相似」的東西。

做法分三步。第一，為每個物品建立特徵向量——電影可用類型、導演、演員、關鍵字；商品可用類別、品牌、價格區間；文章可用第 17 章的 TF-IDF 或 LSA 向量。第二，為每位使用者建立「偏好輪廓」，通常是他評過高分的物品向量的加權平均。第三，計算使用者輪廓與候選物品的相似度（常用餘弦相似度），取最相似者推薦。

你會注意到，這裡用到的相似度計算與第 14、17 章一脈相承——都是把物件表示成向量，再衡量向量的接近程度。

19.2.2 優點與致命傷

- **優點：無需他人資料：**只要有物品特徵與該使用者自己的歷史即可運作，不依賴其他使用者。
- **優點：新物品無障礙：**一件剛上架的商品只要有特徵描述，就能被推薦——沒有「沒人評過所以推不出去」的問題。
- **優點：可解釋：**可以明確告訴使用者「因為你喜歡科幻片，所以推薦這部」。
- **缺點：過度專業化：**它只會推薦與你過去喜好相似的東西，永遠不會給你驚喜。喜歡科幻片的人，就一輩子只收到科幻片。
- **缺點：特徵工程的負擔：**需要有品質良好的物品特徵，而許多領域（如音樂的「感覺」）難以用特徵描述。

其中「過度專業化」是內容式推薦的根本限制。它無法發現「喜歡這部科幻片的人，其實也常喜歡某部紀錄片」這種跨類型的關聯——因為那兩部片的特徵毫不相似。要突破這一點，就需要下一節的方法。

19.3 協同過濾

19.3.1 核心思想：借用他人的經驗

協同過濾（collaborative filtering）的想法極為聰明：它完全不看物品的內容，只看「行為模式」。

其核心假設是：如果甲和乙過去的品味相近，那麼甲喜歡的東西，乙很可能也會喜歡。「協同」二字，正是指借用他人的經驗來為你推薦。

這個轉念解決了內容式推薦的致命傷。因為它不看內容，所以能發現「喜歡這部科幻片的人也常喜歡那部紀錄片」——即使兩者的特徵毫不相似，只要行為模式上高度重合，它就能捕捉到。這也是它能帶來「驚喜」的原因。

19.3.2 兩種方向

協同過濾有兩種實作方向，它們互為對偶。

使用者導向 (user-based)：先找出與目標使用者品味最相近的一群人，再把「他們喜歡而目標使用者還沒接觸過」的物品推薦出去。直覺上就是「找到跟你很像的人，看他們在看什麼」。

物品導向 (item-based)：先計算物品之間的相似度（依「被同一群人喜歡」的程度，而非內容），再推薦「與使用者曾喜歡的物品相似」的東西。直覺上是「你喜歡這個，那些常被一起喜歡的東西你可能也喜歡」。

面向	使用者導向	物品導向
找誰的相似度	使用者與使用者	物品與物品
穩定性	低（人的品味變化快）	高（物品間的關係較穩定）
可預先計算	困難（使用者多且常變）	容易（物品相對少且穩定）
實務偏好	較少單獨使用	業界主流

表 19-3 兩種協同過濾方向的比較

實務上物品導向較受青睞，原因主要是「穩定性」與「可預先計算」：物品數通常遠少於使用者數，且物品之間的相似關係變化緩慢，可以離線算好存起來；而使用者的品味與人數都變動頻繁，難以預先計算。

19.3.3 相似度的計算

無論哪個方向，都需要衡量相似度。最常用的是餘弦相似度——把每個使用者（或物品）視為一個向量，計算兩向量夾角的餘弦值：

$$\cos(\mathbf{a}, \mathbf{b}) = (\mathbf{a} \cdot \mathbf{b}) / (\|\mathbf{a}\| \times \|\mathbf{b}\|)$$

式 19-1 餘弦相似度

為什麼用餘弦而非歐氏距離？因為餘弦只看「方向」不看「長度」。這很重要——有些使用者習慣給高分（評分向量長度大），有些習慣給低分，但只要他們喜好的「相對模式」一致，餘弦相似度就會判定他們相似。

實務上還有一個重要的改良：先減去每位使用者的平均評分，再算餘弦（稱為中心化餘弦或皮爾森相關係數）。這能校正「有人習慣打高分、有人習慣打低分」的個人偏誤，讓比較更公平。

19.3.4 預測評分

找到相似的物品（或使用者）之後，如何預測評分？標準做法是「以相似度為權重的加權平均」：

$$\text{pred}(\mathbf{u}, \mathbf{i}) = \sum_j \text{sim}(\mathbf{i}, \mathbf{j}) \times r(\mathbf{u}, \mathbf{j}) / \sum_j \text{sim}(\mathbf{i}, \mathbf{j})$$

式 19-2 物品導向協同過濾的預測評分

其中 j 遍歷「與物品 i 最相似且使用者 u 曾評過分」的若干物品。直覺很清楚：越相似的物品，其評分在預測中的份量越重。下一節的公式會用具體數字示範一次完整計算。

19.4 矩陣分解與隱含回饋

19.4.1 從第 17 章接手

第 17 章我們看到 SVD 能從評分矩陣中發現「潛在概念」——沒有人標註類型，演算法卻自己找出了科幻、愛情等維度。現在我們要用它來做預測。

核心理想是：把效用矩陣 R 分解為兩個較小的矩陣，一個代表「使用者在各隱含維度上的偏好」(P)，一個代表「物品在各隱含維度上的特質」(Q)：

$$R \approx P Q^T \quad \text{pred}(u, i) = p_u \cdot q_i$$

式 19-3 矩陣分解的預測

預測某使用者對某物品的評分，就是把兩個向量做內積。直覺上：若某使用者在「科幻」這個隱含維度上得分高，而某部電影在該維度上也高，兩者的內積就大，代表他可能會喜歡。

這個方法的威力在於它天生處理了稀疏性——即使某使用者從未看過某部電影，只要他們各自的因子向量都能從其他資料學到，預測就能成立。

19.4.2 為什麼不直接用 SVD

這裡有一個實務上的重要轉折。傳統 SVD 要求矩陣是完整的，但效用矩陣有 99.9% 是空的——空格既不是 0（那代表「評 0 分」），也不能隨便填補。

因此推薦系統採用的是另一種思路：不去分解整個矩陣，而是「只針對已知的格子」來學習 P 與 Q，使預測值盡量接近實際評分。這變成一個最佳化問題：

$$\min \sum (\text{已知的 } u, i) [r(u, i) - p_u \cdot q_i]^2 + \lambda (\|p_u\|^2 + \|q_i\|^2)$$

式 19-4 矩陣分解的目標函數（含正則化）

式中的第二項是「正則化」，用來防止過度擬合——若沒有它，模型會為了完美擬合那少數已知的評分而學出極端的因子值，反而在未知資料上表現很差。 λ 控制正則化的強度。

求解方法有兩種主流：隨機梯度下降 (SGD)，與交替最小平方 (ALS)。ALS 的做法是「固定 Q 求 P、再固定 P 求 Q」，反覆交替——因為固定一邊後，另一邊的每個向量都能獨立求解，故能完美平行化。這正是第 17 章 17.6 節提過的方法，也是 Spark MLlib 的標準實作。

19.4.3 加入偏誤項

實務上還有一個效果顯著的改良：加入「偏誤」(bias) 項。

觀察真實資料會發現，評分的變異中有相當大一部分與「品味」無關，而是來自兩種系統性偏誤：某些使用者天生打分較嚴格或寬鬆；某些電影天生評價就比較高。因此預測式改寫為：

$$\text{pred}(u, i) = \mu + b_u + b_i + p_u \cdot q_i$$

式 19-5 含偏誤項的預測

其中 μ 是全域平均分、 b_u 是使用者偏誤、 b_i 是物品偏誤。把這些「與個人品味無關」的部分先扣除，剩下的內積項才能專心捕捉真正的品味匹配。這個看似簡單的改良，在著名的 Netflix 競賽中被證實能顯著提升準確度。

19.5 冷啟動與混合式推薦

19.5.1 三種冷啟動

冷啟動 (cold start) 是推薦系統最經典的難題，它有三種型態。

- **新使用者**：剛註冊的人沒有任何歷史，協同過濾完全無從下手——不知道他跟誰相似。
- **新物品**：剛上架的商品沒有人評過，在協同過濾中形同隱形，永遠不會被推薦，於是永遠得不到評分（惡性循環）。

- **新系統：**平台剛上線時整個矩陣幾乎全空，所有方法都失效。

其中「新物品」的惡性循環特別值得注意：沒有評分就不被推薦、不被推薦就得不到評分。若不刻意介入，新商品將永無出頭之日。

19.5.2 因應策略

- **改用內容式：**新物品雖無評分，但有特徵描述，可用內容式推薦讓它先曝光——這正是 19.2 節優點的價值所在。
- **引導式輸入：**新使用者註冊時，請他選擇幾個感興趣的類別或評幾部作品，快速建立初始輪廓。
- **善用人口統計：**以年齡、地區等基本資料，先推薦該族群的熱門項目。
- **探索與利用的平衡：**刻意保留一部分推薦名額給「不確定但可能不錯」的新項目，以取得回饋資料。這稱為探索（exploration），與只推已知會成功的「利用」（exploitation）相對。

最後一項的思想很深刻：一個只做「利用」的系統會逐漸僵化——它不斷推薦已知受歡迎的項目，資料因此更集中在這些項目上，於是更加確信它們受歡迎，形成自我強化的迴圈。適度的「探索」是打破這個迴圈、讓系統能持續學習的必要成本。

19.5.3 混合式推薦

實務系統幾乎都不會只用單一方法，而是混合多種訊號。常見的混合方式包括：把多個模型的分數加權平均、依情境切換（新使用者用內容式、老使用者用協同過濾）、或以一個模型的輸出作為另一個模型的輸入特徵。

現代的大型推薦系統通常採「兩階段」架構：先以快速的方法從數百萬候選中「召回」數百個候選（重視速度與召回率），再以複雜的模型對這數百個「精排」（重視準確度）。這個設計讓系統能兼顧規模與品質——你會發現，這與第 14 章 LSH「先篩候選、再精確驗證」的兩階段思想完全一致。

19.6 評估與倫理

19.6.1 離線評估指標

最常用的評估方式，是把已知評分切成訓練集與測試集，用訓練集學模型、在測試集上衡量預測誤差。最常見的指標是均方根誤差（RMSE）：

$$RMSE = \sqrt{\left[\frac{1}{n} \sum (\text{實際} - \text{預測})^2 \right]}$$

式 19-6 均方根誤差

另一個常見指標是平均絕對誤差（MAE），即誤差絕對值的平均。兩者的差別在於：RMSE 因為取平方，會對「大誤差」給予更重的懲罰；MAE 則一視同仁。若你特別在意「不要出現離譜的推薦」，RMSE 是較合適的指標。

對於「排序」而非「評分」的推薦，則常用準確率、召回率、以及考慮位置的指標（如 NDCG，越前面的推薦錯了懲罰越重）。

19.6.2 離線指標的侷限

這裡我要給你一個重要的提醒：RMSE 低不等於推薦好。

原因有幾個。其一，離線測試只能評估「使用者已經評過分的物品」，但推薦系統真正的價值在於推薦「他還沒接觸過」的東西——而那些恰恰無從評估。其二，準確預測不等於有用；準確預測某人會給某部他早就看過的熱門電影五星，毫無推薦價值。其三，離線指標無法衡量多樣性、新穎性與驚喜度——一個只推薦熱門商品的系統可能有很低的 RMSE，卻讓使用者覺得無聊。

因此業界的標準做法是：離線指標用於快速篩選模型，最終決策則依賴線上的 A/B 測試，實際觀察使用者行為的變化。

19.6.3 過濾氣泡與演算法的責任

最後回到叫獸開講中那個令人不安的問題。

推薦系統會產生一種自我強化的效應：它推薦你可能喜歡的內容，你點擊了，它更確信你喜歡這類內容，於是推薦更多同類——久而久之，你接觸到的資訊範圍越來越窄。這個現象稱為「過濾氣泡」（filter bubble）或「同溫層」。

在娛樂內容上，這頂多讓人覺得無聊；但在新聞與觀點上，它可能強化偏見、加深社會分歧。更麻煩的是，這往往不是設計者的惡意，而是「最佳化點擊率」這個看似無害的目標所導致的必然結果——因為最能引發點擊的內容，常常是最能激起情緒或最貼合既有立場的內容。

業界的因應包括：在目標函數中加入多樣性項、刻意注入一定比例的探索性內容、以及提供使用者對推薦邏輯的控制權與透明說明。但這些都是不完美的緩解，而非根本解答。

叫獸提醒

我希望你從這一節帶走的，不是某個技術，而是一個工程師的自覺：當你優化一個指標時，你其實是在替一大群人做決定。點擊率、觀看時長這些指標很好量測，但「使用者的長期福祉」「資訊的多元性」很難量測——而工程上的天性，是去優化那個好量測的東西。這正是危險所在。你將來很可能會參與設計某個影響許多人的系統，屆時我希望你會記得問一句：我優化的這個數字，真的是我想要的結果嗎？這個問題，比任何演算法都重要。

19.6.4 承先啟後

本章是第肆篇的一次匯流。我們看到第 17 章的矩陣分解如何從稀疏評分中學出隱含因子（19.4）、第 14 章的相似度思想如何用於協同過濾（19.3）、以及 LSH 的兩階段架構如何在召回與精排中重現（19.5）。

同時我們也認識了這個領域特有的難題：極端稀疏的效用矩陣、隱含回饋中「缺失即資訊」的詮釋問題、冷啟動的惡性循環、以及離線指標與真實價值之間的落差。最後，我們認真面對了過濾氣泡這個技術以外的責任問題。

到這裡，第肆篇的演算法已經走過相似性、關聯、分群、降維、圖分析與推薦。但你可能已經注意到，這些方法多半是「針對特定問題設計的特定演算法」。有沒有一種更通用的做法——不必為每個問題設計專屬演算法，而是讓機器從資料中自己學出規律？這就是下一章的主題：分散式機器學習與大規模模型訓練，它也是通往第伍篇生成式 AI 的橋樑。

公式與流程：一次完整的協同過濾預測

這一節我們用具體數字，把「算相似度」與「做預測」完整走一遍，最後再評估預測的準確度。

第一步：計算相似度

設有兩部電影 X 與 Y，四位使用者對它們的評分如下（0 表示未評分）：X = (5, 3, 0, 1)，Y = (4, 0, 0, 1)。依式 19-1 計算餘弦相似度：

$$\begin{aligned} \mathbf{a} \cdot \mathbf{b} &= 5 \times 4 + 3 \times 0 + 0 \times 0 + 1 \times 1 = 21 \\ \|\mathbf{a}\| &= \sqrt{(25+9+0+1)} = \sqrt{35} \approx 5.916 \\ \|\mathbf{b}\| &= \sqrt{(16+0+0+1)} = \sqrt{17} \approx 4.123 \\ \cos(\mathbf{X}, \mathbf{Y}) &= 21 / (5.916 \times 4.123) \approx 0.861 \end{aligned}$$

式 19-7 相似度計算範例

相似度 0.861 相當高，代表這兩部電影常被同一群人以相近的方式評分。

第二步：預測評分

現在要預測使用者 U 對物品 X 的評分。我們找出與 X 最相似、且 U 曾評過分的三個物品：

相似物品	與 X 的相似度	U 的評分	貢獻（相似度 × 評分）
物品 A	0.9	5	4.5
物品 B	0.7	4	2.8
物品 C	0.4	3	1.2
合計	2.0	—	8.5

表 19-4 加權平均的預測計算

依式 19-2 計算：

$$\text{pred}(\mathbf{U}, \mathbf{X}) = 8.5 / 2.0 = 4.25$$

式 19-8 預測結果

預測值 4.25 分。注意它落在 3 到 5 之間，且比較靠近相似度高的那些物品的評分——這正是加權平均的作用：越相似的物品，說話的份量越重。

第三步：評估準確度

假設我們對五筆測試資料做了預測，實際值與預測值如下表。依式 19-6 計算 RMSE：

樣本	實際評分	預測評分	誤差平方
1	4	3.5	0.25
2	5	4.8	0.04
3	2	2.9	0.81
4	3	3.1	0.01
5	5	4.2	0.64
合計	—	—	1.75

表 19-5 RMSE 的計算過程

$$\text{RMSE} = \sqrt{(1.75 / 5)} = \sqrt{0.35} \approx 0.592$$

式 19-9 RMSE 計算結果

同一組資料的 MAE 則為 $(0.5+0.2+0.9+0.1+0.8)/5 = 0.5$ 。注意 RMSE (0.592) 大於 MAE (0.5)，這是因為 RMSE 對第 3 與第 5 筆的較大誤差施加了更重的懲罰。

叫獸提醒

RMSE 恆大於或等於 MAE，而兩者的差距正好透露了「誤差的分布是否均勻」。若 RMSE 遠大於 MAE，代表有少數幾筆預測錯得離譜，把平方項拉高了——這時你該去找出那些離譜的案例，而不是滿足於平均值。這個「看差距而非只看單一數字」的習慣，在第 3 章談 p99 延遲時我也強調過：平均值會掩蓋尾端的問題。同一個統計直覺，在效能監控與模型評估上都適用。

案例研討

案例一 那些沒被看見的長尾商品

某電商平台上架了約五十萬件商品，但分析發現，銷售額的八成集中在不到兩萬件的熱門商品上，其餘四十八萬件幾乎乏人問津——不是因為它們不好，而是因為使用者根本不知道它們存在。

導入協同過濾後情況改變了。系統能發現「買了這件冷門露營燈的人，也常買另一款冷門的登山杯」這類關聯——這是內容式推薦抓不到的，因為兩者的商品類別完全不同。長尾商品開始被送到會喜歡它們的人面前，其營收占比在一年內顯著提升。

這個案例展示了 19.1.1 節的核心論點：推薦系統不只是提升點擊率的工具，它是讓長尾經濟得以成立的基礎設施。若沒有它，線上平台「能陳列無限商品」的優勢就無法兌現。

案例二 新片的惡性循環

某串流平台發現，新上架的影片有一半以上在首月的播放量趨近於零。追查原因後發現這是典型的冷啟動惡性循環：新片沒有觀看紀錄，協同過濾無從判斷它與什麼相似，因此不會被推薦；不被推薦就沒人看，於是永遠得不到資料。

他們採取了三管齊下的對策：其一，對新片改用內容式推薦，依類型、演員、導演等特徵讓它先接觸到可能感興趣的族群；其二，在每位使用者的推薦名單中，固定保留兩個名額給「探索性」內容；其三，為新片的預測分數加上一個隨時間遞減的加成，讓它在初期有機會出頭。

這個案例把 19.5.2 節的策略具象化了，也說明了「探索」為何不是浪費，而是系統得以持續學習的必要投資。

案例三 RMSE 很漂亮，使用者卻不滿意

某團隊花了數個月優化模型，把測試集上的 RMSE 從 0.95 降到 0.87，團隊內部相當振奮。但上線後的 A/B 測試卻顯示，使用者的實際點擊率與觀看時長幾乎沒有改善，甚至略微下降。

檢討後發現問題出在 19.6.2 節所說的落差。新模型之所以 RMSE 較低，很大程度是因為它更擅長預測「熱門商品的評分」——而熱門商品本來就佔了測試資料的大宗。但使用者對熱門商品的推薦興趣缺缺，因為他們早就知道那些東西了。真正能帶來價值的，是那些「他不知道但會喜歡」的冷門項目，而這些恰恰在離線指標中份量極小。

團隊後來在評估中加入了多樣性與新穎性指標，並以線上 A/B 測試作為最終判準。這個案例是離線指標侷限最直接的教訓：優化一個代理指標，不等於改善你真正在意的結果。

案例四 同溫層是怎麼長出來的

某內容平台以「觀看時長」為唯一的優化目標。演算法忠實地執行了這個任務——它逐漸發現，能引發強烈情緒反應的內容，最能留住使用者。於是這類內容的推薦權重不斷提高。

幾個月後，平台上出現了明顯的極化現象：使用者被逐漸推向立場越來越鮮明的內容，不同立場的使用者所看到的世界差異越來越大。而這一切，沒有任何工程師寫過「請推薦極端內容」這行程式——它只是「最大化觀看時長」這個目標的自然結果。

這正是 19.6.3 節所警示的：危險往往不來自惡意，而來自「優化了一個容易量測、卻不等於真正目標的指標」。這個案例也呼應了第 18 章案例一的教訓——當一個指標成為目標時，它就不再是好指標。工程師的責任，不只是把系統做得更準，還包括問清楚「準是為了什麼」。

實作活動

活動一 手算協同過濾

請以紙筆完成一次完整的物品導向協同過濾。

- 步驟 1.** 建立一個 5 位使用者 $\times$ 4 部電影的評分矩陣（刻意留下幾個空格）。
- 步驟 2.** 依式 19-1 計算電影 1 與其餘三部電影的餘弦相似度。
- 步驟 3.** 選定一個空格（某使用者對某電影未評分），依式 19-2 以加權平均預測其評分。
- 步驟 4.** 改用「中心化餘弦」（先減去各使用者的平均評分再算）重做一次，比較兩種結果的差異並說明原因。

活動二 以矩陣分解實作推薦

請以 Python 實作矩陣分解式的推薦。

- 步驟 1.** 取得一份公開的評分資料集（如電影評分資料集），切分為訓練集與測試集。
- 步驟 2.** 使用 ALS 或 SGD 實作式 19-4 的矩陣分解，隱含維度 k 取 10 到 50。
- 步驟 3.** 在測試集上依式 19-6 計算 RMSE，並與「一律預測全域平均分」的基準線比較。
- 步驟 4.** 加入式 19-5 的偏誤項後重新訓練，觀察 RMSE 是否改善、改善多少。

活動三 檢視你自己的過濾氣泡

請對一個你常用的推薦平台做一次觀察與反思。連續一週記錄它推薦給你的內容，並分類統計：這些內容涵蓋幾種主題或立場？重複性有多高？是否出現過讓你意外但喜歡的內容？接著嘗試刻意點擊幾則平常不會看的內容，觀察一週後推薦是否改變。最後撰寫一段反思：若你是這個平台的工程師，你會如何在「使用者滿意度」與「資訊多元性」之間取捨？你會用什麼指標來衡量後者？

延伸閱讀

- **《Mining of Massive Datasets》第 9 章：**Leskovec 等人著。本章的主要依據，涵蓋內容式推薦、協同過濾與潛在因子模型，並有詳細的計算範例。
- **Netflix 競賽的技術報告：**Koren 等人關於矩陣分解與偏誤項的論文，是式 19-4 與式 19-5 的源頭，也記錄了推薦系統研究的重要里程碑。
- **隱含回饋的協同過濾：**Hu、Koren 與 Volinsky 的論文，說明如何處理「只有正例」的隱含回饋資料，是表 19-2 問題的標準解法。

- **關於過濾氣泡的討論：**Eli Pariser 的《The Filter Bubble》一書，從社會與倫理角度探討推薦演算法的影響，適合作為 19.6.3 節的延伸思考。

本章小結

1. 推薦系統解決選擇過載並讓長尾經濟得以成立；其標準抽象為效用矩陣，但該矩陣極度稀疏（常低於 0.1% 填滿率），所有方法本質上都在回答「如何由極少的已知推測其餘」。
2. 回饋分顯性（主動評分，可信但稀少）與隱含（行為推得，量大但意圖模糊）；隱含回饋只有正例，「沒有紀錄」可能是不喜歡也可能是沒看過，如何詮釋缺失往往決定系統成敗。
3. 內容式推薦依物品特徵推薦相似項目，優點是不依賴他人資料、新物品無障礙且可解釋，缺點是過度專業化（無法帶來驚喜）與特徵工程負擔。
4. 協同過濾只看行為模式而不看內容，故能發現跨類型的關聯；分使用者導向與物品導向兩種，後者因穩定且可預先計算而成為業界主流。相似度常用餘弦（式 19-1），並可先中心化以校正個人評分偏誤。
5. 矩陣分解把效用矩陣拆為使用者因子與物品因子，以內積預測評分（式 19-3）；因效用矩陣有大量空格，實務上不用傳統 SVD，而是只針對已知格子最佳化並加入正則化（式 19-4），常以 ALS 求解以利平行化。
6. 加入全域平均與使用者、物品偏誤項（式 19-5），可先扣除與品味無關的系統性差異，使內積項專心捕捉真正的品味匹配，實證上能顯著提升準確度。
7. 冷啟動有新使用者、新物品與新系統三種型態，其中新物品會陷入「無評分故不被推薦、不被推薦故無評分」的惡性循環；因應策略包括改用內容式、引導式輸入、人口統計，以及在探索與利用之間取得平衡。
8. RMSE（式 19-6）對大誤差懲罰較重，恆大於或等於 MAE，兩者差距透露誤差分布是否均勻；但離線指標低不等於推薦好——它無法評估未接觸過的物品、也無法衡量多樣性與驚喜度，故最終應依線上 A/B 測試判斷。
9. 推薦系統會產生自我強化的過濾氣泡，其根源往往不是惡意，而是「優化了容易量測卻不等於真正目標的指標」；工程師的責任不只是把系統做得更準，還包括追問「準是為了什麼」。

習題

一、觀念題

1. 請說明內容式推薦與協同過濾的核心差異，並各舉一個前者做不到但後者做得到的情境。
2. 隱含回饋中「沒有紀錄」為何難以詮釋？若把它一律當作負面訊號，會造成什麼問題？
3. 為什麼推薦系統的矩陣分解不直接使用傳統 SVD？實務上改採什麼做法？
4. 請說明「新物品冷啟動」的惡性循環，並提出至少兩種打破它的策略。

二、計算題

1. 兩部電影的評分向量為 $P = (4, 0, 5, 3)$ 、 $Q = (5, 2, 4, 0)$ 。請依式 19-1 計算兩者的餘弦相似度（計算過程須列出內積與兩個範數）。
2. 欲預測使用者 U 對物品 Z 的評分。與 Z 相似且 U 評過分的三個物品資料為：相似度 0.8／評分 4、相似度 0.6／評分 5、相似度 0.3／評分 2。（一）依式 19-2 計算預測評分；（二）若另有四筆預測的實際值與預測值為 (5, 4.3)、(3, 3.4)、(4, 4.0)、(2, 2.6)，請依式 19-6 計算 RMSE 與 MAE，並說明兩者差距的意義。

三、思考與討論

1. 案例三中，RMSE 改善了但使用者滿意度沒有提升。請討論：離線指標與真實價值之間為何會有落差？你會設計哪些補充指標來縮小這個落差？
2. 案例四指出，同溫層是「最大化觀看時長」這個目標的自然結果。請討論：若由你重新設計該平台的優化目標，你會如何定義它？並說明你的定義可能帶來哪些新的問題。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

核心差異：內容式推薦分析物品「本身的特徵」，推薦與使用者過去喜好特徵相似者；協同過濾完全不看內容，只依「行為模式」——若甲乙品味相近，則甲喜歡的乙可能也喜歡。前者做不到而後者做得到的情境：發現跨類型的關聯，例如「喜歡某部科幻片的人也常喜歡某部紀錄片」——兩者的特徵毫不相似，內容式推薦永遠不會把它們連在一起，但只要行為模式重合，協同過濾就能捕捉。反之，協同過濾做不到而內容式做得到的是新物品推薦（無人評過但有特徵描述）。

第 2 題

難以詮釋的原因：隱含回饋只觀察得到「發生了什麼行為」，卻無法區分「沒有紀錄」是因為使用者不喜歡，還是因為他根本沒看到該項目。造成的問題：若一律當作負面訊號，會把大量「其實會喜歡但尚未接觸」的項目標記為不喜歡，而這些恰恰是推薦系統最該挖掘的長尾價值；模型因此學到偏向已曝光項目的偏誤，加劇熱門項目的集中，並使新物品更難出頭。標準解法是採用加權的隱含回饋模型——對「有紀錄」的項目給予高信心權重，對「無紀錄」給予低權重而非視為確定的負例。

第 3 題

因為傳統 SVD 要求矩陣完整，而效用矩陣有超過 99% 是空格；這些空格既不能視為 0（那代表「評 0 分」，與「未評分」意義完全不同），也無法合理填補（任意填補會引入大量虛假訊號並主宰結果）。實務做法：不去分解整個矩陣，而是把它轉為最佳化問題——只針對「已知的格子」學習使用者因子 P 與物品因子 Q ，使預測值盡量接近實際評分，並加入正則化項防止過度擬合（式 19-4）。求解方法有 SGD 與 ALS，後者因「固定一邊求另一邊」時各向量可獨立求解而能完美平行化，是 Spark MLlib 的標準實作。

第 4 題

惡性循環：新物品沒有任何評分或互動紀錄，協同過濾無從判斷它與什麼相似，因此不會被推薦；不被推薦就沒有使用者接觸，於是永遠得不到評分資料——形成自我封閉的迴圈。打破策略（任舉兩種）：其一，對新物品改用內容式推薦，依其特徵描述先接觸可能感興趣的族群；其二，保留固定比例的推薦名額給探索性內容，刻意讓新物品有曝光機會；其三，為新物品的預測分數加上隨時間遞減的加成，使其在初期較易勝出；其四，運用人口統計或情境資訊做初步匹配。可引案例二說明三管齊下的實務做法。

二、計算題

第 1 題

$P = (4, 0, 5, 3)$ 、 $Q = (5, 2, 4, 0)$ ，依式 19-1 計算。

- (1) 內積： $4 \times 5 + 0 \times 2 + 5 \times 4 + 3 \times 0 = 20 + 0 + 20 + 0 = 40$ 。
- (2) $\|P\| = \sqrt{16 + 0 + 25 + 9} = \sqrt{50} \approx 7.071$ 。
- (3) $\|Q\| = \sqrt{25 + 4 + 16 + 0} = \sqrt{45} \approx 6.708$ 。
- (4) 餘弦相似度 $= 40 / (7.071 \times 6.708) = 40 / 47.434 \approx 0.843$ 。

判讀：0.843 屬高度相似，表示這兩部電影常被同一群使用者以相近的模式評分。

第 2 題

(一) 依式 19-2 計算加權平均。

- (1) 分子 $= 0.8 \times 4 + 0.6 \times 5 + 0.3 \times 2 = 3.2 + 3.0 + 0.6 = 6.8$ 。
- (2) 分母 $= 0.8 + 0.6 + 0.3 = 1.7$ 。
- (3) 預測評分 $= 6.8 / 1.7 = 4.0$ 。

(二) RMSE 與 MAE 的計算。四筆誤差分別為 0.7、-0.4、0.0、-0.6。

- (4) 誤差平方：0.49、0.16、0.00、0.36，合計 $= 1.01$ 。RMSE $= \sqrt{(1.01 / 4)} = \sqrt{0.2525} \approx 0.502$ 。
- (5) 誤差絕對值：0.7、0.4、0.0、0.6，合計 $= 1.7$ 。MAE $= 1.7 / 4 = 0.425$ 。

差距的意義：RMSE（約 0.502）大於 MAE（0.425），差距約 0.077。因 RMSE 取平方，會對較大的誤差施加更重的懲罰，故兩者的差距反映了「誤差分布是否均勻」——本例中第一筆誤差 0.7 明顯高於其他，拉大了 RMSE。若差距很小，表示各筆誤差相近；若差距很大，則應去檢視那些離譜的個案，而非滿足於平均值（呼應第 3 章 p99 延遲的統計直覺）。

三、思考與討論

第 1 題

落差的成因：其一，離線測試只能評估「使用者已評過分的物品」，但推薦的真正價值在於他「尚未接觸」的項目，而那些無從評估；其二，準確不等於有用——精準預測某人會給早已看過的熱門片五星，毫無推薦價值；其三，測試資料本身偏向熱門項目，故模型在熱門項目上的改善會主宰指標，卻不對應使用者感受；其四，離線指標完全無法衡量多樣性、新穎性與驚喜度。補充指標可包括：涵蓋率（推薦觸及了多少比例的商品目錄）、新穎度（推薦項目的平均熱門程度，越低越新穎）、群內多樣性（同一份推薦清單中項目彼此的相異程度）、意外度（推薦項目與使用者歷史的距離）、以及長期指標（使用者留存率、跨類別探索行為）。最終仍應以線上 A/B 測試驗證真實行為變化。

第 2 題

可提出的目標設計方向：其一，多目標加權——同時納入觀看時長、內容多樣性、以及使用者事後的滿意度回饋，而非單一指標；其二，改用長期指標——以「三十日後的留存率」或「使用者主動回訪頻率」取代當下的觀看時長，因為激起情緒的內容雖能留住短期注意力，長期可能造成疲乏或反感；其三，納入使用者的明示偏好——提供「少推這類」的控制項並確實反映在模型中；其四，設定多樣性的下限約束（如推薦清單中須涵蓋 N 種主題）。可能帶來的新問題：長期指標回饋週期長、難以快速迭代，且與短期商業指標可能衝突；多目標加權的權重本身是價值判斷，由誰決定、如何取捨並無客觀答案；強制多樣性可能降低即時滿意度，使用者未必領情；而「使用者的長期福祉」本質上難以量測，任何代理指標都可能再次被過度優化。周延的回答會承認：這不是一個能被演算法單獨解決的問題，需要產品、政策與使用者控制權三方面配合。

台灣自編大學教科書
@prof
大數據分析與雲端運算

第肆篇 大數據探勘與分析演算法

Mining & Analytics Algorithms

第 20 章

分散式機器學習與大規模模型訓練

Distributed Machine Learning and Large-Scale Model Training

機器人叫獸（李世淵） 編著

叫獸開講

我實驗室有位學生，去年跟我說他要訓練一個模型，資料大概二十 GB。我問他打算怎麼跑，他說：「就在我的筆電上跑啊。」

三天後他來找我，說跑了兩天還沒跑完，而且風扇聲音大到室友抗議。我們一起算了一下：他的模型要迭代一百輪，每輪要掃過二十 GB 的資料——光是硬碟讀取，一百輪就是兩 TB 的 I/O。他的筆電永遠跑不完。

這件事有兩個層次的啟示。第一個層次很直接：資料大到一定程度，單機就是不行，必須分散。但第二個層次更有意思——當你將訓練分散到很多台機器上，一個新的敵人出現了，那就是「溝通成本」。

想像十個人一起解一道數學題。人多確實算得快，但每算一步，十個人都要停下來對答案、統一進度——如果對答案的時間比算的時間還長，那人多反而更慢。分散式訓練面對的正是這件事：每一輪迭代之後，所有機器都必須交換各自算出的梯度，把模型參數同步一致。機器越多，這個同步就越貴。

所以這一章的主軸，不是「如何把訓練分散出去」（那不難），而是「如何在分散之後，讓溝通成本不要吃掉所有的好處」。這也是第肆篇的最後一章——當我們把演算法擴展到極致，就會走到第伍篇那些動輒需要數千張加速卡訓練數月的大型模型。這一章，就是那座橋。

學習目標

研讀完本章之後，你應該能夠：

1. 說明大規模機器學習的三個挑戰，並指出通訊為何常成為瓶頸。
2. 說明特徵工程的常見手法，並解釋特徵選擇的動機。
3. 描述 Spark MLlib 的管線架構，並說明其設計理念。
4. 說明參數伺服器架構，並區分同步與非同步更新的取舍。
5. 區分資料平行與模型平行，並判斷各自的適用時機。
6. 說明分散式決策樹與梯度提升的實作要點。

核心概念

核心概念	說明
特徵工程	把原始資料轉換為模型可用之特徵的過程，常決定模型成敗。
ML 管線	把前處理、訓練、評估串成可重複執行之流程的架構。
參數伺服器	集中保管模型參數、供多個工作節點讀取與更新的架構。
資料平行	各節點持有完整模型、處理不同資料分片，再同步梯度。
模型平行	模型本身太大而被切分到多個節點，各節點只持有一部分。
同步與非同步	各節點是否須等待彼此完成後才進行下一輪更新。
梯度提升	以序列方式逐棵建樹、每棵修正前面殘差的整體學習方法。

表 20-1 本章核心概念一覽

20.1 大規模機器學習的挑戰

20.1.1 三個維度的「大」

機器學習的「大」有三種截然不同的型態，它們需要不同的解法，混為一談是常見的錯誤。

型態	問題	主要解法
資料太多	樣本數以億計，單機記憶體與運算不足	資料平行：切分資料、各節點算局部梯度
特徵太多	維度以百萬計，參數向量本身就很大	特徵選擇、降維、稀疏表示
模型太大	模型參數量超出單一裝置記憶體	模型平行：把模型本身切開

表 20-2 大規模機器學習的三種型態

多數情況下遇到的是第一種（資料太多），這也是本章著墨最多的。但第三種（模型太大）在近年因大型語言模型而變得重要——一個上千億參數的模型，光是參數就佔數百 GB，遠超單張加速卡的記憶體，非切不可。

20.1.2 真正的敵人：通訊

承接叫獸開講的比喻。分散式訓練的每一輪迭代，通常包含兩個階段：各節點在自己的資料上計算梯度（運算），然後所有節點交換梯度並更新模型（通訊）。

運算的部分很好平行化——機器加倍，這部分的時間就減半。但通訊的部分卻恰恰相反：機器越多，要交換的訊息越多、要等待的對象越多，通訊時間反而增加。

這就構成了一個此消彼長的關係：增加節點會降低運算時間，卻提高通訊時間。當節點數超過某個臨界點，通訊的增加會超過運算的減少，整體反而變慢。本章的公式部分會用具體數字讓你看見這個臨界點。

叫獸提醒

這裡有個直接的實務推論，我希望你記住：分散式訓練「不是機器越多越好」。我見過太多人以為租更多機器就能更快，結果帳單暴增而時間沒省多少，甚至更慢。正確的做法是先做小規模的擴充測試——用 2、4、8、16 台各跑一次，畫出加速比曲線，找出你的臨界點在哪裡，再決定要租多少台。這個測試花不了多少時間，卻能替你省下大筆雲端費用。這也再次呼應第 1 章的阿姆達爾定律：任何平行化都有其上限，而那個上限由「不能平行的部分」決定——在這裡，就是通訊。

20.2 特徵工程與特徵選擇

20.2.1 為什麼特徵比模型更重要

業界流傳一句話：「垃圾進，垃圾出。」再先進的模型，若輸入的特徵無法反映問題的本質，也學不出有用的東西。這也呼應了第 1 章 1.2.4 節的迷思三——多數同學把時間花在調模型，但真正決定成敗的往往是資料與特徵。

特徵工程是把原始資料轉換為模型可用形式的過程，常見的手法包括幾類。

- **數值處理**：標準化（使各特徵尺度相近，呼應第 16、17 章的提醒）、離散化（把連續值分箱）、取對數（處理偏態分布）。
- **類別編碼**：把類別型欄位轉為數值，如獨熱編碼（one-hot）；但類別數極多時會產生高維稀疏向量，需搭配雜湊技巧。
- **時間特徵**：從時間戳萃取出星期幾、是否假日、距上次事件多久等——這類衍生特徵往往極具預測力。
- **交互特徵**：把兩個特徵相乘或組合，捕捉「單獨看不出、合起來才有意義」的關係。
- **領域特徵**：由專業知識設計的特徵，例如製造領域的振動訊號頻譜能量、機器人的關節扭矩變化率。

最後一項最難自動化，也最有價值。這是資料科學家無法被工具取代的部分——因為它需要對問題本身的理解。

20.2.2 特徵選擇

特徵不是越多越好。過多的特徵會帶來三個問題：加劇維度災難（第 16 章）、增加過度擬合的風險、以及拖慢訓練與推論。因此常需進行特徵選擇，常見策略有三類：

- **過濾法**：以統計指標（如與目標的相關性、變異數）獨立評估每個特徵，快速但忽略特徵間的交互作用。
- **包裝法**：反覆訓練模型並測試不同特徵子集的效果，準確但計算昂貴。
- **嵌入法**：在訓練過程中自動進行選擇，如 L1 正則化會把不重要特徵的權重壓到零，是實務上最常用的方式。

20.3 Spark MLlib 與機器學習管線

20.3.1 把整個流程當成一個物件

實務上的機器學習從來不是「呼叫一個訓練函式」那麼簡單，而是一連串步驟：清理、編碼、標準化、特徵組合、訓練、評估。這條流程在訓練時要跑一次，在預測新資料時還要「用完全相同的參數」再跑一次。

這裡藏著一個極容易出錯的地方：訓練時你用訓練資料的平均值做了標準化，預測時就必須用「同一組平均值」，而不能用新資料自己的平均值。手工管理這些狀態，錯誤幾乎無可避免。

Spark MLlib 的管線（Pipeline）架構正是為此而生。它把整條流程封裝成一個物件，訓練時一次擬合、預測時一次套用，所有中間狀態自動保存與重用。

元件	角色	範例
Transformer	把一個 DataFrame 轉換為另一個	標準化、獨熱編碼、已訓練好的模型
Estimator	從資料學出一個 Transformer	訓練演算法、標準化器的擬合
Pipeline	把多個階段串成單一流程	清理→編碼→標準化→訓練
Evaluator	以指標衡量預測品質	RMSE、準確率、AUC

表 20-3 MLlib 管線的四個核心元件

這個設計的另一個好處，是與第 12 章的 DataFrame 完全整合——特徵工程可以用 DataFrame API 或 SQL 完成，接著無縫接入訓練，全部在同一個 Spark 作業中完成，不必在系統之間搬運資料。

20.3.2 MLlib 的定位與限制

MLlib 擅長的是「資料量極大、模型相對簡單」的場景——線性模型、決策樹、隨機森林、k-means、ALS 推薦等。它的強項是能直接在 TB 級的資料上訓練，且與整個 Spark 生態系整合。

但它並不適合深度學習。深度神經網路需要 GPU 加速、需要頻繁的小批次更新、且模型結構複雜多變——這些都不是 Spark 的設計目標。實務上的常見分工是：用 Spark 做大規模的資料前處理與特徵工程，把結果輸出後，再以專門的深度學習框架進行訓練。認清工具的邊界，比勉強用單一工具解決所有問題更明智。

20.4 參數伺服器架構

20.4.1 誰來保管模型

分散式訓練有一個根本問題：模型參數只有一份，但有很多節點要同時讀取與更新它。這份參數該放在哪裡、如何協調？

參數伺服器（parameter server）是最經典的答案。它把系統分為兩種角色：伺服器節點負責保管模型參數（若參數量大，可再切分到多台伺服器上）；工作節點負責在自己的資料分片上計算梯度。

每一輪的流程是：工作節點向伺服器拉取最新參數、在本地資料上計算梯度、把梯度推送回伺服器、伺服器彙整所有梯度並更新參數。這個「拉取—計算—推送」的循環反覆進行，直到收斂。

20.4.2 同步還是非同步

這裡有一個關鍵的設計抉擇，它直接對應到第 3 章的一致性取舍。

同步更新要求伺服器等待「所有」工作節點都送回梯度後，才統一更新參數並進入下一輪。優點是每個節點看到的參數都一致，訓練過程等同於單機的大批次訓練，收斂行為穩定可預測。缺點是整體速度被「最慢的那個節點」拖住——這稱為「落後者問題」（straggler problem）。只要有一台機器變慢，所有機器都得等它。

非同步更新則不等待：任何工作節點算完就推送，伺服器收到就更新。優點是完全沒有等待，速度快、也不怕落後者。缺點是節點拿到的參數可能是「過期」的——它拉取參數後、計算梯度的期間，別人可能已經更新了好幾次，於是它送回的梯度是基於舊參數算的，這稱為「陳舊梯度」（stale gradient）。這會使收斂變得不穩定，甚至可能不收斂。

面向	同步更新	非同步更新
等待	須等所有節點	不等待
參數一致性	所有節點一致	可能過期（陳舊梯度）
收斂	穩定可預測	較不穩定，可能需調小學習率
主要弱點	落後者拖累整體	梯度陳舊影響品質

表 20-4 同步與非同步更新的比較

實務上也有折衷方案，例如「有限陳舊」（bounded staleness）——允許節點超前，但不得超過 k 輪，藉此在速度與穩定之間取得平衡。你會發現這與第 7 章一致性光譜的思路完全一致：極端的兩端各有代價，中間地帶往往才是實用解。

20.4.3 AllReduce：另一條路

參數伺服器不是唯一的架構。近年在深度學習領域更常見的是 AllReduce 模式——它沒有中央伺服器，所有節點以環狀或樹狀的通訊模式，彼此交換並彙整梯度，最後每個節點都得到相同的彙整結果。

它的優勢在於避免了參數伺服器可能成為的通訊瓶頸（所有節點都跟同一組伺服器通訊），且通訊量分散得更均勻。代價是它天生是同步的，因此仍受落後者問題影響。多數現代的深度學習分散式訓練框架，採用的都是這條路線。

20.5 資料平行與模型平行

20.5.1 兩種切法

面對「要分散什麼」這個問題，有兩種根本不同的切法。

資料平行（data parallelism）：每個節點都持有「完整的模型副本」，但只處理「一部分的資料」。各節點算完自己那份資料的梯度後，再彙整同步。這是最常見的做法，適用於「資料很多但模型裝得下」的情境。

模型平行（model parallelism）：當模型本身大到單一裝置裝不下時，就必須把「模型切開」——不同節點持有模型的不同部分，資料則依序流過這些節點。這適用於「模型極大」的情境，如大型語言模型。

面向	資料平行	模型平行
切分對象	資料	模型參數
各節點持有	完整模型 + 部分資料	部分模型 + 全部資料流經
通訊內容	梯度（每輪同步一次）	中間激活值（層與層之間）
適用時機	資料大、模型裝得下	模型大到單機裝不下
實作難度	較低（框架多有內建）	較高（須手動切分與排程）

表 20-5 資料平行與模型平行的比較

20.5.2 管線平行與混合策略

模型平行有一個明顯的缺點：若只是把模型的各層分給不同節點，那麼當第一層在計算時，其餘節點全都閒著等待——設備利用率極低。

改良方法稱為「管線平行」（pipeline parallelism）：把一批資料再切成數個小批次，讓它們像工廠生產線一樣依序流過各節點。當第二個小批次進入第一層時，第一個小批次已經前進到第二層——如此各節點就能同時工作，大幅提升利用率。

實務上訓練超大模型時，這幾種策略會混合使用：在節點內用模型平行切分、節點之間用資料平行複製、再加上管線平行提升利用率。這種組合稱為「3D 平行」，是當前訓練大型語言模型的標準做法。第 21 章談到大語言模型的訓練規模時，你會再看到它。

20.6 分散式決策樹與梯度提升

20.6.1 決策樹為何難以平行

決策樹的訓練過程是：在每個節點上，檢視所有特徵的所有可能切分點，找出「切分後最能區分目標」的那一個。這需要對資料做大量的統計掃描。

在分散式環境中，資料分散在各節點上，而「找最佳切分點」需要全域的統計資訊。若每考慮一個切分點就通訊一次，通訊量將大到無法承受。

標準解法是「直方圖法」：各節點先在自己的資料上，把每個特徵的取值分成若干個箱（bin），統計各箱的樣本數與目標值總和；接著只把這些「直方圖」（而非原始資料）彙整起來，就能在彙整後的直方圖上找出全域最佳切分點。

這個做法的精妙之處，在於它把通訊量從「與資料量成正比」降為「與箱數成正比」——資料再多，直方圖的大小都不變。你會發現這與第 16 章 BFR 的摘要統計、第 9 章的 Combiner 是同一個思想：找出一個可以先在本地彙整、再合併的摘要量。

20.6.2 隨機森林與梯度提升的差異

兩種以決策樹為基礎的整體方法，在平行化上有截然不同的性質。

隨機森林是「平行」建樹：每棵樹以隨機抽樣的資料與特徵獨立訓練，彼此無關，最後投票或平均。因為樹與樹之間毫無相依，它天生就能完美平行——你有幾台機器，就能同時建幾棵樹。

梯度提升則是「序列」建樹：每一棵新樹的任務，是修正前面所有樹加起來的殘差（預測錯誤的部分）。這意味著第二棵樹必須等第一棵完成後才能開始——樹與樹之間存在嚴格的先後相依，無法平行建樹。

面向	隨機森林	梯度提升
建樹方式	各樹獨立、可同時建	序列建樹、後修正前
平行化層次	樹層級（完美平行）	僅能在單棵樹內平行
準確度	穩健，較不易過度擬合	通常更高，但需仔細調參
訓練速度	快（可完全平行）	較慢（受序列相依限制）

表 20-6 隨機森林與梯度提升的比較

那麼梯度提升要如何加速？答案是：既然無法在「樹之間」平行，就在「單棵樹之內」平行——也就是用前一節的直方圖法，把找切分點這件事分散出去。這正是現代梯度提升框架的核心優化方向。

20.6.3 承先啟後

本章我們認識了分散式機器學習的三種「大」（20.1）、特徵工程為何比模型更重要（20.2）、MLlib 如何以管線封裝整條流程（20.3）、參數伺服器與同步非同步的取捨（20.4）、資料平行與模型平行兩種切法（20.5），以及決策樹如何以直方圖法克服通訊瓶頸（20.6）。

到這裡，第肆篇「大數據探勘與分析演算法」七章完整收束。回頭看，我們從相似性探勘（第 14 章）出發，走過關聯規則、分群、降維、圖分析、推薦，最後來到機器學習。這一路上有一個貫穿的主線值得你記住：面對大數據，關鍵從來不是「算得更快」，而是「聰明地不算」——LSH 讓我們不比對所有配對、單調性原理讓我們不檢查所有組合、BFR 讓我們不保留所有資料點、直方圖法讓我們不傳輸所有原始值。

而本章最後那個「通訊成為瓶頸」的問題，也預告了下一篇的世界。當模型的參數量從百萬躍升到千億，當訓練需要數千張加速卡連續運轉數月，這些挑戰會被放大到極致。第伍篇「生成式 AI 與開源生態」要談的大語言模型、AI 代理與開源模型平台，正是這條路走到今天的樣貌。我們下一章見。

公式與流程：擴充效率與最佳節點數

這一節我們把「機器越多未必越快」這件事算成具體數字，並找出你的臨界點。

時間模型

設單機完成一輪訓練的運算時間為 c 。使用 n 個節點時，運算時間降為 c/n ，但每輪需額外付出通訊時間 $m(n)$ ——它隨節點數增加而增加。故總時間為：

$$T(n) = c / n + m(n)$$

式 20-1 分散式訓練單輪的時間模型

加速比與擴充效率則定義為：

$$\text{加速比 } S(n) = T(1) / T(n) \quad \text{擴充效率 } E(n) = S(n) / n$$

式 20-2 加速比與擴充效率

擴充效率是更實用的指標：它衡量「每台機器平均發揮了幾成效益」。E = 100% 代表完美擴充（ n 台就快 n 倍），E = 50% 則代表你租的機器只有一半在發揮作用。

具體試算

設單機運算時間 $c = 1000$ 秒，通訊時間隨節點數增加，取 $m(n) = 10\sqrt{n}$ 秒。下表計算各節點數的表現。

節點數 n	運算 c/n	通訊 $m(n)$	總時間	加速比	效率
1	1000.0	10.0	1010.0	1.00	100%
2	500.0	14.1	514.1	1.96	98%
4	250.0	20.0	270.0	3.74	94%
8	125.0	28.3	153.3	6.59	82%
16	62.5	40.0	102.5	9.85	62%
32	31.2	56.6	87.8	11.50	36%
64	15.6	80.0	95.6	10.56	17%
128	7.8	113.1	120.9	8.35	7%

表 20-7 節點數對加速比與擴充效率的影響

這張表有三個關鍵觀察。第一，加速比在 $n = 32$ 時達到峰值（11.5 倍），之後不升反降——64 台竟然比 32 台還慢。第二，即使在峰值處，擴充效率也只剩 36%，意味著三分之二的機器算力

被通訊吃掉了。第三，若以「效率至少 80%」為標準，最佳選擇其實是 8 台（效率 82%、加速 6.59 倍）。

叫獸提醒

表 20-7 揭示了一個成本上的殘酷現實：從 8 台加到 32 台，機器數變成四倍、費用也是四倍，但速度只從 6.59 倍提升到 11.5 倍（約 1.7 倍）。你多付了三倍的錢，只換來 1.7 倍的速度。這值不值得，取決於「時間對你有多值錢」——若是趕研討會截稿，可能值得；若是例行的每週訓練，那就是在燒錢。所以我要求我的學生在租用雲端資源前，一定要先做小規模的擴充測試、畫出這張表。順帶一提，通訊模型 $m(n)$ 的形狀因架構而異（參數伺服器、AllReduce、網路拓撲都會影響），本例只是示意——你必須測出自己系統的真實曲線。

案例研討

案例一 跑不完的筆電

回到叫獸開講。那位學生的二十 GB 資料要迭代一百輪，單機每輪需完整讀取一次磁碟。粗估每輪僅 I/O 就需數分鐘，一百輪即數小時；再加上運算，加上記憶體不足導致的頻繁交換，兩天跑不完毫不意外。

我們的解法分兩步。第一步不是「加機器」，而是先做第 11 章的優化：把資料轉為 Parquet 欄式格式並只讀必要欄位（第 4、12 章），再以 Spark 快取於記憶體，避免一百輪重複讀取磁碟。光這一步，時間就從兩天降到約三小時。

第二步才考慮分散。但依表 20-7 的邏輯，我們先做了 2、4、8 台的擴充測試，發現效率在 8 台後就明顯下滑，於是就用 8 台，最終約二十分鐘完成。這個案例的教訓是：先優化單機，再考慮分散——很多人跳過第一步直接加機器，結果是把一個沒優化過的爛程式，昂貴地平行化了。

案例二 被一台老機器拖住的訓練

某團隊以同步更新進行分散式訓練，叢集有 32 個節點。他們發現訓練速度遠低於預期，而監控顯示大多數節點的 GPU 使用率只有三成——大部分時間都在等待。

追查後發現，叢集中有一台機器的網路卡規格較舊，傳輸速度只有其他機器的一半。因為採同步更新，每一輪都必須等所有節點送回梯度，於是這一台就決定了全體的節奏——這正是 20.4.2 節的「落後者問題」。

他們的處理是雙管齊下：短期改用「有限陳舊」的折衷策略，允許快的節點超前最多兩輪；長期則汰換那台機器。這個案例說明同步架構的一個重要性質：整體效能由最慢的成員決定，因此叢集的「同質性」比「總算力」更重要。這也呼應了第 3 章的八個誤解之一——「網路是同質的」是個危險的假設。

案例三 直方圖救了通訊量

某公司要在數十億筆交易資料上訓練梯度提升模型做風險預測。初版實作在每次尋找切分點時，都需要在節點間交換大量的統計資訊，通訊量隨資料量成長，訓練一輪就要數小時。

改用直方圖法後情況完全不同：各節點先在本地把每個特徵分成 255 個箱、統計各箱的樣本數與梯度總和，接著只交換這些直方圖。無論資料是十億筆還是百億筆，每個特徵要傳的都只是 255 個箱的統計值——通訊量從「與資料量成正比」變成了常數。訓練時間降為原本的數十分之一。

這個案例是 20.6.1 節的直接印證，也再次展現了本書反覆出現的思想：找出一個「可以先在本地彙整、再合併」的摘要量。從第 9 章的 Combiner、第 16 章的 BFR 到這裡的直方圖，都是同一招——而能否辨識出這種摘要量，往往就是分散式演算法設計的成敗關鍵。

案例四 裝不下的模型

某研究團隊想微調一個大型語言模型，但發現模型參數就佔了數百 GB，遠超單張加速卡的記憶體——這已不是「資料太多」，而是表 20-2 的第三種型態：模型太大。

資料平行在此完全無用，因為它要求每個節點都持有完整的模型副本，而問題正是「一個副本都放不下」。他們必須改用模型平行：把模型的不同層分配到不同的加速卡上，資料依序流過。

但初版實作的利用率極低——當第一張卡在計算時，其餘的卡全都閒著。加入管線平行後（把批次再切成小批次，讓它們像生產線一樣連續流過），利用率大幅提升。這個案例預告了第 21 章的世界：當模型規模跨過某個門檻，訓練本身就成了一項系統工程，而非單純的演算法問題。

實作活動

活動一 畫出你的擴充曲線

請實際測量分散式訓練的擴充效率，而非憑猜測。

- 步驟 1.** 選定一個可分散執行的訓練任務（可用 Spark MLlib 的邏輯迴歸或隨機森林）。
- 步驟 2.** 分別以 1、2、4、8 個執行器（executor）各訓練一次，記錄各自的執行時間。
- 步驟 3.** 依式 20-2 計算各設定的加速比與擴充效率，並繪成折線圖。
- 步驟 4.** 找出「效率仍在 80% 以上」的最大節點數，並說明若要再加機器，代價與收益各如何變化。

活動二 建立一條 MLlib 管線

請以 Spark MLlib 實作完整的機器學習管線，體會 20.3.1 節的價值。

- 步驟 1.** 取得一份含類別欄位與數值欄位的資料集，切分為訓練集與測試集。
- 步驟 2.** 建立管線，依序包含：類別編碼（StringIndexer 與 OneHotEncoder）、特徵組合（VectorAssembler）、標準化、以及一個分類或迴歸模型。
- 步驟 3.** 以訓練集擬合管線，並在測試集上預測與評估。
- 步驟 4.** 刻意改用「手動逐步處理」的方式重做一次，並故意在測試集上重新計算標準化參數——比較兩者的結果差異，說明管線為何能避免這類錯誤。

活動三 比較平行與序列的整體方法

請以同一份資料，分別訓練隨機森林與梯度提升模型（樹的數量設為相同，例如各 100 棵），記錄兩者的訓練時間與測試集準確度。接著把可用的 CPU 核心數從 1 增加到 4 或 8，重複測量兩者的訓練時間。完成後回答：哪一種方法從增加核心數中獲益較多？為什麼？這如何印證表 20-6 中「平行化層次」的差異？最後思考：若你的任務對訓練時間極為敏感，你會選哪一種？

延伸閱讀

- **《Mining of Massive Datasets》第 12 章**：Leskovec 等人著，介紹大規模機器學習的基本方法與其在 MapReduce 上的實作考量。
- **參數伺服器論文**：Li Mu 等人發表的《Scaling Distributed Machine Learning with the Parameter Server》，是 20.4 節的第一手資料，詳述同步與非同步的設計權衡。
- **Apache Spark MLlib 官方文件**：說明管線架構、各演算法的分散式實作與參數調校，是 20.3 節的實作參考。
- **大規模模型訓練的平行策略綜述**：關於資料平行、模型平行與管線平行如何組合的技術文章，是 20.5 節的深化，也可作為第 21 章的預習。

本章小結

1. 大規模機器學習有三種型態：資料太多（用資料平行）、特徵太多（用特徵選擇與降維）、模型太大（用模型平行）；混為一談會導致選錯解法。
2. 分散式訓練的真正瓶頸是通訊：增加節點會降低運算時間，卻提高通訊時間，故存在一個臨界點，超過後整體反而變慢——分散式訓練絕非機器越多越好。
3. 特徵工程往往比模型選擇更決定成敗；常見手法包括數值處理、類別編碼、時間特徵、交互特徵與領域特徵，其中領域特徵最難自動化也最有價值。特徵選擇則分過濾法、包裝法與嵌入法。
4. MLlib 管線把整條流程封裝為單一物件，訓練時一次擬合、預測時一次套用，可避免「訓練與預測使用不同前處理參數」這類難以察覺的錯誤；但 MLlib 適合大資料簡單模型，不適合深度學習。
5. 參數伺服器把系統分為保管參數的伺服器節點與計算梯度的工作節點；同步更新一致性佳但受落後者拖累，非同步更新無等待但有陳舊梯度問題，「有限陳舊」是常見折衷。AllReduce 則是無中央伺服器的另一條路線。
6. 資料平行讓各節點持有完整模型、處理部分資料，通訊內容為梯度；模型平行把模型本身切開，通訊內容為中間激活值。管線平行可解決模型平行的設備閒置問題，三者組合即為訓練超大模型的標準做法。
7. 決策樹的分散式訓練以直方圖法克服通訊瓶頸——各節點先在本地統計分箱結果，只交換直方圖而非原始資料，使通訊量與資料量脫鉤；隨機森林可在樹層級完美平行，梯度提升因序列相依只能在單棵樹內平行。
8. 擴充效率 $E(n) = S(n)/n$ （式 20-2）衡量每台機器發揮的效益；實務上應先做小規模擴充測試、畫出加速比曲線並找出臨界點，再決定投入多少資源——這是第 1 章阿姆達爾定律在訓練上的具體展現。

習題

一、觀念題

1. 請說明大規模機器學習的三種「大」，並各指出其主要解法。為什麼把它們混為一談是危險的？
2. 為什麼分散式訓練「不是機器越多越好」？請以運算時間與通訊時間的消長關係說明。
3. 請比較同步更新與非同步更新的優缺點，並說明「有限陳舊」如何折衷兩者。
4. 分散式決策樹為何要使用直方圖法？它把通訊量從什麼變成了什麼？

二、計算題

1. 設某訓練任務的單機運算時間 $c = 800$ 秒，每輪通訊時間為 $m(n) = 20n$ 秒（ n 為節點數）。請計算 $n = 2, 4, 8, 16$ 時的總時間、加速比與擴充效率（以 $T(1) = 800 + 20 = 820$ 秒為基準），並指出加速比在何處達到峰值。
2. 承上題，若通訊模型改善為 $m(n) = 5n$ 秒（例如改用更好的網路或 AllReduce），請重新計算 $n = 8, 16, 32$ 的總時間與加速比，並說明改善通訊為何能提高可用的節點數上限。

三、思考與討論

1. 案例一中，我們先優化單機才考慮分散。請討論：為什麼「先優化再分散」通常比「直接加機器」更明智？在什麼情況下應該直接分散？
2. 本章與整個第肆篇反覆出現「找出可先在本地彙整、再合併的摘要量」這個思想（Combiner、BFR、直方圖）。請說明這類摘要量必須具備什麼數學性質，並就你熟悉的領域再舉一個可用此思想的例子。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

三種型態與解法：其一，資料太多（樣本數以億計）——用資料平行，切分資料、各節點算局部梯度再同步。其二，特徵太多（維度以百萬計）——用特徵選擇、降維與稀疏表示。其三，模型太大（參數超出單一裝置記憶體）——用模型平行，把模型本身切開。混為一談的危險：解法彼此不可替代。例如面對「模型太大」時採用資料平行完全無效，因為資料平行要求每個節點都持有完整的模型副本，而問題正是「一個副本都放不下」（見案例四）；反之，面對「資料太多」卻去做模型切分，只會徒增複雜度而無助於解決問題。

第 2 題

因為總時間由兩部分構成（式 20-1）：運算時間 c/n 隨節點數增加而「下降」，但通訊時間 $m(n)$ 隨節點數增加而「上升」（節點越多，要交換的訊息與等待的對象越多）。兩者方向相反，故存在一個臨界點：在此之前，運算時間的減少大於通訊時間的增加，總時間下降；超過之後，通訊的增加超過運算的減少，總時間反而上升。表 20-7 即顯示加速比在 32 節點達峰值 11.5 倍後，64 節點反而降至 10.56 倍。實務建議是先做小規模擴充測試、畫出加速比曲線再決定節點數。

第 3 題

同步更新：伺服器等所有工作節點送回梯度後才統一更新。優點是各節點參數一致、收斂穩定可預測；缺點是整體速度被最慢節點決定（落後者問題，見案例二）。非同步更新：任何節點算完即推送、伺服器收到即更新。優點是無等待、速度快、不怕落後者；缺點是節點可能基於過期參數計算（陳舊梯度），使收斂不穩定甚至不收斂。「有限陳舊」的折衷：允許快的節點超前，但超前不得超過 k 輪，超過即須等待。如此既避免了嚴格同步的全體等待，又限制了梯度陳舊的程度，在速度與穩定之間取得平衡（思路與第 7 章一致性光譜相同）。

第 4 題

因為決策樹在每個節點都要檢視所有特徵的所有可能切分點，而找出全域最佳切分點需要全域的統計資訊；若每考慮一個切分點就通訊一次，通訊量將大到無法承受。直方圖法：各節點先在本地把

每個特徵的取值分成固定數量的箱（如 255 個），統計各箱的樣本數與梯度總和，接著只彙整這些直方圖，即可在合併後的直方圖上找出全域最佳切分點。它把通訊量從「與資料量成正比」變成「與箱數成正比」——資料再多，每個特徵要傳的都只是固定數量的箱統計值（見案例三）。

二、計算題

第 1 題

$c = 800$ 、 $m(n) = 20n$ 、基準 $T(1) = 800 + 20 = 820$ 秒。依式 20-1 與式 20-2 計算。

- (1) $n = 2$: $T = 400 + 40 = 440$ 秒; $S = 820/440 \approx 1.86$; $E = 1.86/2 \approx 93\%$ 。
- (2) $n = 4$: $T = 200 + 80 = 280$ 秒; $S = 820/280 \approx 2.93$; $E = 2.93/4 \approx 73\%$ 。
- (3) $n = 8$: $T = 100 + 160 = 260$ 秒; $S = 820/260 \approx 3.15$; $E = 3.15/8 \approx 39\%$ 。
- (4) $n = 16$: $T = 50 + 320 = 370$ 秒; $S = 820/370 \approx 2.22$; $E = 2.22/16 \approx 14\%$ 。

峰值位置：加速比在 $n = 8$ 達到峰值（約 3.15 倍）， $n = 16$ 時反而降至 2.22 倍。但注意此時效率僅 39%，若以效率 70% 以上為標準，實務上應選 $n = 4$ 。

第 2 題

通訊改善為 $m(n) = 5n$ ，基準改為 $T(1) = 800 + 5 = 805$ 秒。

- (1) $n = 8$: $T = 100 + 40 = 140$ 秒; $S = 805/140 \approx 5.75$; $E \approx 72\%$ 。
- (2) $n = 16$: $T = 50 + 80 = 130$ 秒; $S = 805/130 \approx 6.19$; $E \approx 39\%$ 。
- (3) $n = 32$: $T = 25 + 160 = 185$ 秒; $S = 805/185 \approx 4.35$; $E \approx 14\%$ 。

說明：通訊係數由 20 降為 5 後，峰值由 $n = 8$ （加速 3.15 倍）右移至 $n = 16$ （加速 6.19 倍）——不僅可用的節點數上限提高，能達到的最大加速比也接近翻倍。原因在於通訊是「不可平行的部分」，其係數越小，運算時間的減少就能在更多節點數下持續勝過通訊的增加。這正是第 1 章阿姆達爾定律的體現：降低不可平行部分的比重，才能提高平行化的上限——這也是為何業界大力投資高速網路與 AllReduce 等通訊優化。

三、思考與討論

第 1 題

「先優化再分散」更明智的理由：其一，單機優化的效益往往是倍數級且免費的（如案例一改用 Parquet 與快取，時間從兩天降至三小時），而分散化的效益受限於擴充效率且需付費。其二，未優化的程式分散後，其低效會被複製到每個節點，等於「昂貴地平行化了一個爛程式」。其三，優化後所需節點數減少，通訊開銷也隨之降低，反而使後續的分散更有效率。其四，分散化引入複雜度（除錯困難、失敗處理、資源調度），不應輕率承擔。應直接分散的情況：資料量根本超出單機儲存或記憶體上限（連載入都不可能）、有嚴格的時間期限而單機無論如何優化都達不到、或問題本身天然分散（資料原本就分布在多處且不宜集中）。

第 2 題

必須具備的性質：可結合性（結合律）與可交換性——亦即「分批計算再合併」的結果，必須與「一次全部計算」相同。形式上，需存在合併運算使得 $f(A \cup B)$ 可由 $f(A)$ 與 $f(B)$ 推得，而不需回頭存取原始資料。第 9 章明確指出加總、極值、計數符合此性質，而平均不符合（須改傳「總和與個數」兩個可結合量）。其他例子（依領域自選）：製造領域可對感測訊號維護「筆數、總和、平方和、最大值、最小值」等摘要，即可分批算出各機台的平均、標準差與極值而不必集中原始資料（同 BFR

思想)；網路監控可維護各分區的封包計數與位元組總和；影像處理可維護各區塊的直方圖。反例則如「中位數」與「相異元素數」——它們不具可結合性，故需改用近似演算法（如第 24 章的串流演算法），這也再次呼應「以可控近似換取可行計算」的主線。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 20 章 分散式機器學習與大規模模型訓練

台灣自編大學教科書
@prof
大數據分析與雲端運算

第伍篇 生成式 AI 與開源生態

Generative AI & Open-Source Ecosystem

第 21 章

大語言模型 (LLM)

Large Language Models

機器人叫獸 (李世淵) 編著

叫獸開講

我要從一個很老的東西講起：手機的注音輸入法。

你打「機器」，它會猜你下一個字可能是「人」；你打「今天天氣」，它會猜「很好」。這就是語言模型——一個會預測「下一個字是什麼」的東西。它其實已經存在幾十年了，一點也不新。

那為什麼近幾年會突然天翻地覆？答案說來令人意外：因為有人發現，只要把這個「猜下一個字」的模型做得夠大、餵得夠多資料，它會開始展現出一些沒有人教過它的能力——會翻譯、會寫程式、會推理、會解釋笑話。這些能力沒有被特別訓練過，它們是在規模跨過某個門檻後「浮現」出來的。

這件事至今仍讓許多研究者感到不安：我們並不完全理解為什麼會這樣。一個只是在拼命猜下一個字的系統，為什麼會表現得像是理解了什麼？這個問題沒有定論，而我會在這一章誠實地把它攤開來講，包括它的能力邊界與它會犯的錯。

這一章對你們特別重要，因為它把整本書串了起來。你會看到第 17 章的向量表示、第 20 章的分散式訓練、第 14 章的語料去重全部在這裡匯流。而更實際的是——當你日後要在專案中導入 LLM 時，你會面對一個真實的選擇：用雲端的大模型，還是在自己的機器上跑地端模型？這個決定牽涉成本、隱私、延遲與可控性，我會用具體的數字教你怎麼判斷。

學習目標

研讀完本章之後，你應該能夠：

1. 說明語言模型的基本任務，以及大型化為何帶來質變。
2. 說明 Transformer 與注意力機制的核心概念。
3. 比較雲端大模型與地端模型的取捨，並依情境做出選型判斷。
4. 區分預訓練、微調與對齊三個階段的目的。
5. 說明提示工程的基本技巧，以及 RAG 如何補足模型的知識缺口。
6. 描述端到端的推論流程，並運用量化技術估算部署所需的資源。

核心概念

核心概念	說明
語言模型	依前文預測下一個詞元機率分布的模型。
Transformer	以注意力機制為核心的網路架構，是當代大模型的共同基礎。
注意力機制	讓模型在處理每個詞時，動態決定該關注前文中的哪些部分。
預訓練與微調	先在海量通用語料上學通用能力，再以少量特定資料調整。
提示工程	透過設計輸入的措辭與範例，引導模型產生期望的輸出。
RAG	檢索增強生成，先檢索相關文件再交由模型作答。
量化	以較低位元精度表示參數，大幅降低記憶體需求。

表 21-1 本章核心概念一覽

21.1 從語言模型到大語言模型

21.1.1 核心任務其實很簡單

承接叫獸開講，語言模型的任務可以一句話說完：給定前面的文字，預測下一個「詞元」（token）的機率分布。

這裡要先澄清「詞元」這個詞。模型處理的單位既不是字也不是詞，而是一種介於兩者之間的切分單位。英文中常見的詞會是一個詞元，罕見的詞則被拆成數個；中文則常以一到二字為一個詞元。這種切分方式是為了在「詞彙表大小」與「序列長度」之間取得平衡。

模型的訓練目標同樣單純：拿一段真實文字，遮住後面，讓模型猜下一個詞元，猜錯就調整參數。如此在海量文本上重複數兆次。整個過程不需要任何人工標註——文本自己就是答案，這稱為「自監督學習」。這也是為什麼它能吃下整個網際網路的資料：標註成本為零。

21.1.2 規模帶來的質變

既然任務這麼單純，為何會產生如此驚人的能力？關鍵在於「規模定律」（scaling law）——研究發現，模型的表現會隨著參數量、資料量與運算量的增加而穩定提升，且這個關係在很大的範圍內都成立。

更耐人尋味的是「湧現能力」（emergent abilities）：某些能力在小模型上完全看不到（表現近乎隨機），但當規模跨過某個門檻後，突然就出現了。多步驟推理、依範例學習新任務、理解隱喻，都屬於這一類。

為什麼會這樣？坦白說，學界尚無定論。一個常見的解釋是：要準確預測下一個字，模型被迫學會文字背後的規律——要猜對「這道數學題的答案是」後面的字，就不得不學會計算；要猜對故事的後續，就不得不建立對人物意圖的某種表徵。換言之，「預測下一個字」是一個看似簡單、實則涵蓋極廣的任務。

叫獸提醒

我要在這裡先打一劑預防針。「湧現」聽起來很神秘，但請不要把它浪漫化為「模型產生了意識」或「它真的理解了」。目前我們能確定的是：它學到了資料中極為複雜的統計規律，且這些規律足以支撐許多看起來像理解的行為。至於這算不算「真的理解」，是個哲學問題，而不是工程問題。作為工程師，你需要的態度是：既不低估它的實用能力，也不高估它的可靠性。這一章後面談到幻覺與能力邊界時，你會明白為什麼這個平衡很重要。

21.2 Transformer 與注意力機制

21.2.1 注意力：動態決定該看哪裡

Transformer 的核心是「注意力機制」（attention）。它要解決的問題是：處理一個句子時，每個詞的意義往往取決於句中的其他詞，但取決於「哪些詞」則因情況而異。

舉個例子：「機器手臂抓取了那個零件，因為它太重了。」——這裡的「它」指的是零件還是手臂？人類靠常識判斷是零件。要正確處理這個「它」，模型必須「注意」到前面的「零件」而非「手臂」。

注意力機制讓模型在處理每個詞時，動態計算「該給前文中每個詞多少權重」。這些權重不是固定的，而是依當下的內容算出來的——這正是它比早期方法強大的原因。早期的循環網路只能依序累積資訊，越遠的詞影響越弱；注意力則讓任兩個詞之間都能直接建立連結，無論相隔多遠。

21.2.2 三個角色：查詢、鍵與值

注意力的實作可以用一個檢索的比喻來理解。每個詞元被投影成三種向量：

- **查詢 (Query)**：代表「我在找什麼樣的資訊」。
- **鍵 (Key)**：代表「我能提供什麼樣的資訊」。
- **值 (Value)**：代表「我實際攜帶的內容」。

計算方式是：拿當前詞的查詢向量，與所有詞的鍵向量做內積，得到相似度分數；分數越高代表越該關注。把這些分數正規化為權重後，對各詞的值向量做加權平均，就是這個詞在本層的新表示。

你會發現，這裡的「以內積衡量相關性」與第 19 章的餘弦相似度、第 17 章的向量表示是同一套思想——把語意變成向量，用幾何運算來處理語意關係。這正是本書反覆強調的：原理會在不同的地方重複出現。

21.2.3 多頭與堆疊

實務上還有兩個關鍵設計。其一是「多頭注意力」：同時進行多組獨立的注意力計算，各自關注不同面向——有的頭可能專注於語法關係，有的關注指代，有的關注主題。最後把各頭的結果合併。

其二是「堆疊」：把注意力層一層層疊起來（現代大模型常達數十層甚至上百層）。低層通常捕捉表層的語法特徵，高層則形成越來越抽象的語意表徵。

這裡也帶出一個重要的成本特性：注意力需要計算「每個詞與每個詞」的關係，故計算量與序列長度的平方成正比。這就是為什麼「上下文長度」是大模型的關鍵指標，也是為什麼處理長文件的成本會急遽上升——這個平方項，與第 14 章的相似性比對是同一個難題。

21.3 預訓練、微調與對齊

21.3.1 三個階段

現代大模型的養成，通常經過三個性質截然不同的階段。

階段	目的	資料	資料量級
預訓練	學習語言與世界的通用規律	海量無標註文本	數兆詞元
監督式微調	學會遵循指令、以對話形式回應	人工撰寫的指令與回應範例	數萬至數十萬筆
對齊	使輸出更符合人類偏好與安全要求	人類對回應好壞的偏好比較	數萬至數十萬筆

表 21-2 大模型養成的三個階段

這個分工很值得理解。預訓練賦予模型「知識與能力」，但它只會續寫文字，不會回答問題；監督式微調教它「以助手的方式應答」；對齊階段則進一步調整它的行為傾向——例如拒絕有害請求、承認自己不知道、給出更有幫助的回答。

成本上，預訓練佔了絕大部分（動輒數千張加速卡運行數月，即第 20 章談的分散式訓練極致），而後兩階段相對便宜。這也是為何多數組織不會自行預訓練，而是取用開源的預訓練模型再做微調。

21.3.2 微調的實務選擇

當你有特定領域的需求時，微調是常見選項。但完整微調一個大模型代價高昂——要更新所有參數，記憶體需求是推論的數倍。

因此業界廣泛採用「參數高效微調」：凍結原模型的絕大部分參數，只訓練新增的少量參數。其中最流行的做法是低秩適配（LoRA），它的想法正是第 17 章的低秩近似——與其更新一個巨大的權重矩陣，不如學習一個「低秩的修正量」，用兩個瘦長的小矩陣相乘來表示。

這樣做的好處極為顯著：需訓練的參數量可降至原本的百分之一以下，單張消費級顯示卡就能微調中型模型；而且不同任務的適配權重可以分別保存、隨時切換，不必為每個任務都存一份完整模型。

21.4 雲端大模型與地端模型的比較

這一節是本章最具實務決策價值的部分。當你要在專案中導入 LLM 時，第一個要回答的問題就是：呼叫雲端的 API，還是在自己的機器上跑？

21.4.1 兩條路線的本質差異

雲端大模型指的是透過 API 呼叫由服務商代管的模型。你不擁有模型，只是把提示送過去、把結果收回來，依用量計費。

地端模型（也稱本地部署或私有部署）則是把模型權重下載到自己的伺服器或工作站上執行。你完全掌控它，資料不出自家網路，但也必須自行負擔硬體與維運。

面向	雲端大模型	地端模型
能力上限	通常最強（可用最大的模型）	受限於自有硬體，多為中小型模型
資料隱私	資料須送出，受合約與法規約束	資料完全不出網路，隱私可控
成本結構	營運支出，依用量計費，起步成本低	資本支出，須先購置硬體，用量大時單位成本低
延遲	受網路影響，且無法保證	可預測，適合即時場景
可用性	依賴外部服務與網路連線	自主可控，可離線運作
維護負擔	低（服務商負責）	高（須自行部署、更新、調校）
客製化	有限（多僅能提示工程）	高（可微調、可改架構）

表 21-3 雲端大模型與地端模型的比較

21.4.2 如何選擇

這個選擇沒有標準答案，但有幾條清楚的判準。

- **資料敏感性優先考量：**若涉及製程機密、個資或受法規限制的資料（回想第 1 章 1.3.4 節的資料主權），地端往往是唯一選項。
- **用量決定成本結構：**偶爾使用時雲端更划算；但當呼叫量大到一定程度，自建的攤提成本可能更低。應以實際用量試算損益平衡點。
- **延遲要求：**需要毫秒級回應或必須離線運作的場景（如產線即時判斷、機器人端），地端是必然選擇——這正是第 1 章「網路依賴與延遲」的限制。
- **能力需求：**若任務需要最頂尖的推理能力，目前雲端大模型仍有明顯優勢；但若任務相對特定（分類、摘要、格式轉換），中型的地端模型往往已足夠。

實務上越來越常見的是「混合策略」：把敏感或高頻的任務交給地端模型處理，只有少數需要最強能力的複雜任務才送上雲端。這與第 1 章談的混合雲思路完全一致——不是二選一，而是依任務性質分流。

叫獸提醒

我要提醒你一個常被忽略的面向：可控性。使用雲端 API 時，服務商可能更新模型版本，而新版的行為可能與舊版不同——你的提示詞在昨天運作良好，今天卻可能給出不同的結果。對於需要穩定行為的生產系統（尤其是製造領域的品質判定），這種不可控性是真實的風險。地端模型雖然能力可能較弱，但「它今天和明天的行為完全一致」，這在某些場景中比多幾分準確度更重要。選型時，別只比較準確率，也要問：我的系統能否承受它明天悄悄改變？

21.5 提示工程與 RAG

21.5.1 提示工程的基本技巧

提示工程（prompt engineering）指的是透過設計輸入的措辭與結構，引導模型產出期望的結果。它之所以有效，是因為模型的行為高度依賴上下文。幾個經實證有效的基本技巧：

- **明確指定角色與任務：**說清楚你要它扮演什麼角色、完成什麼任務、輸出什麼格式，遠勝過模糊的提問。
- **提供範例：**在提示中給出一到數個輸入輸出的範例，模型能依樣模仿——這稱為「少樣本學習」，是湧現能力的一種展現。
- **要求逐步推理：**對於需要多步驟的問題，要求模型「一步一步思考」往往能顯著提升正確率，因為它把推理過程外顯化了。
- **拆解複雜任務：**與其要它一次完成複雜工作，不如拆成數個步驟依序進行，每步驗證後再往下。

21.5.2 模型不知道的事

大模型有三個結構性的知識限制，理解它們才能正確使用。

第一，知識有截止時間。模型的知識來自訓練資料，訓練完成後發生的事它一概不知。第二，它不知道你的私有資料——你公司的內部文件、產線的規格書、客戶的資料，都不在它的訓練語料中。第三，也是最麻煩的：當它不知道時，它可能會「編造」出看似合理的答案，這稱為「幻覺」（hallucination）。

幻覺為何會發生？因為模型的訓練目標是「產生看起來合理的下一個字」，而非「說實話」。當它缺乏正確資訊時，它仍會盡責地產生流暢、格式正確、看起來很有把握的文字——只是內容是錯的。這是它最危險的失效模式，因為錯誤被包裝得極為可信。

21.5.3 RAG：把資料帶給模型

檢索增強生成（Retrieval-Augmented Generation, RAG）是解決上述問題最實用的方法。它的核心想法是：與其期待模型「記得」所有知識，不如在回答時「查給它看」。

流程分為三步。第一，離線階段：把你的文件切成段落，用嵌入模型轉成向量，存入向量資料庫。第二，查詢階段：把使用者的問題也轉成向量，在向量資料庫中找出最相似的若干段落——這正是第 14 章的相似性搜尋，大規模時常用 LSH 或類似的近似最近鄰技術。第三，生成階段：把檢索到的段落連同問題一起送給模型，要求它「依據以下資料回答」。

RAG 的優勢很明顯：知識可即時更新（改文件即可，不必重訓模型）、能使用私有資料、且可以附上來源讓人查證，大幅降低幻覺。這使它成為企業導入 LLM 最主流的架構。

你應該已經注意到，RAG 幾乎完全建立在本書前面的技術上：文件切分與前處理（第 13 章）、向量表示（第 17 章）、相似性搜尋（第 14 章）、向量資料庫（第 6 章的 NoSQL）。這是一個很好的例子，說明為什麼要把地基打穩。

21.6 端到端推論流程與服務化

21.6.1 一次請求走過的路

理解推論的完整流程，才能知道效能瓶頸與成本落在哪裡。一次典型的請求會經過以下階段：

- 第 1 步． 詞元化：**把輸入文字切分為詞元序列並轉為數值編號。
- 第 2 步． 預填充（Prefill）：**模型一次處理完整個輸入提示，建立內部狀態。這一步可高度平行，但計算量與提示長度的平方相關。
- 第 3 步． 解碼（Decode）：**逐一產生輸出詞元。每產生一個詞元，都要跑一次完整的模型前向計算，且必須等前一個詞元產生後才能算下一個——這一步是序列的，無法平行。
- 第 4 步． 快取重用：**把已計算的鍵值向量存起來（稱為 KV 快取），避免每產生一個詞元都重算整個前文。
- 第 5 步． 解詞元化：**把輸出的詞元序列轉回文字，回傳給使用者。

這裡有個關鍵洞見：預填充與解碼的性質完全不同。預填充是「運算受限」的（一次算很多，吃算力），解碼是「記憶體頻寬受限」的（每次只算一個詞元，但要把整個模型的參數讀過一遍）。這解釋了一個常見的現象——輸出越長，回應越慢，而且慢的程度與輸出長度成正比。

21.6.2 服務化的考量

- **批次處理：**把多個使用者的請求合併處理，可大幅提升硬體利用率。但會增加個別請求的延遲，須在吞吐與延遲間取捨（呼應第 3 章）。
- **串流輸出：**逐字回傳而非等全部生成完才回傳。實際總時間不變，但使用者感受到的等待大幅縮短——這是體驗上的重要設計。
- **KV 快取管理：**長對話的 KV 快取會佔用大量記憶體，需有淘汰策略；這是地端部署最常遇到的記憶體瓶頸。
- **監控與成本控制：**追蹤詞元用量、延遲分布（用 p99 而非平均，見第 3 章）與失敗率；雲端 API 尤須設定用量上限以免帳單失控。

21.7 量化、壓縮與部署

21.7.1 量化：用更少的位元表示參數

模型的參數預設以 16 位元浮點數儲存。量化的想法是：改用更低的精度（如 8 位元或 4 位元整數）來表示，藉此大幅降低記憶體需求。

這聽起來像是會嚴重損害品質，但實證顯示：8 位元量化幾乎無損，4 位元量化在多數任務上也只有輕微退化。原因在於神經網路的參數本身具有相當的冗餘與容錯性——這與第 17 章「丟掉小奇異值多半是丟掉雜訊」的直覺有幾分相通。

量化帶來的效益不只是記憶體。因為解碼階段是記憶體頻寬受限的，參數變小意味著每次讀取的資料量變少，推論速度也會跟著提升。下一節的公式會把這個效益算成具體數字。

21.7.2 其他壓縮手法與部署形態

- **蒸餾**：用大模型的輸出來訓練一個小模型，讓小模型模仿大模型的行為，以較小的規模保留大部分能力。
- **剪枝**：移除對輸出影響甚微的參數或結構，減少計算量。
- **邊緣部署**：把量化後的小模型部署到邊緣裝置上（如第 35 章要談的 Jetson），實現離線、低延遲的推論——這對機器人與產線應用尤其關鍵。

21.7.3 承先啟後

本章我們從「猜下一個字」這個古老任務出發（21.1），理解了注意力機制如何讓模型動態決定該關注什麼（21.2）、預訓練微調對齊三階段的分工（21.3）；接著處理了最具實務價值的雲端與地端選型（21.4）、以提示工程與 RAG 補足模型的知識缺口（21.5）、走過端到端的推論流程（21.6），最後以量化與壓縮讓部署成為可能（21.7）。

我希望你記住兩件事。第一，LLM 不是魔法，它是「規模跨過門檻後的統計學習」，既強大又有明確的失效模式（尤其是幻覺）。第二，RAG 這個當前最主流的企業架構，幾乎完全建立在本書前面的技術之上——這說明大數據的地基，正是通往 AI 應用的必經之路。

不過，到目前為止我們談的 LLM 仍然只是「你問它答」的被動角色。如果我們讓它能夠自己決定要做什麼、能夠呼叫工具查資料或執行動作、能夠記住過去並規劃未來的步驟呢？那它就從一個「會說話的模型」，變成了一個「會做事的代理」。這就是下一章的主題：AI 代理。

公式與流程：模型部署的記憶體估算

這一節要教你一件非常實用的事：拿到一個模型時，如何在五秒鐘內判斷「我的硬體跑不跑得動」。

參數所需的記憶體

模型參數所佔的記憶體，等於「參數量 × 每個參數的位元組數」。以 P 表示參數量、 b 表示每個參數的位元數，則：

$$\text{記憶體 (GB)} = P \times b / 8 / 10^9$$

式 21-1 模型參數的記憶體需求

常見的精度包括 16 位元浮點（每參數 2 位元組）、8 位元整數（1 位元組）與 4 位元整數（0.5 位元組）。下表計算幾種常見模型規模的需求。

參數量	FP16 (16 位元)	INT8 (8 位元)	INT4 (4 位元)
70 億 (7B)	14.0 GB	7.0 GB	3.5 GB
130 億 (13B)	26.0 GB	13.0 GB	6.5 GB
700 億 (70B)	140.0 GB	70.0 GB	35.0 GB
1750 億 (175B)	350.0 GB	175.0 GB	87.5 GB

表 21-4 不同規模與精度下的參數記憶體需求

這張表可以直接用來做部署判斷。以一張 24 GB 記憶體的消費級顯示卡為例：7B 模型以 FP16（14 GB）可以跑；13B 以 FP16（26 GB）放不下，但量化到 INT8（13 GB）就可以；70B 即使量化到 INT4（35 GB）仍然放不下，必須用多卡或更高階的硬體。

別忘了推論的額外開銷

實務上還必須加上 KV 快取等額外開銷。粗略的估算原則是：

$$\text{實際需求} \approx \text{參數記憶體} \times 1.2 \sim 1.5$$

式 21-2 含推論開銷的粗估

其中 KV 快取的大小與「同時處理的請求數」及「上下文長度」成正比——這也是為什麼長對話或高並行會突然爆記憶體。若要支援長上下文或多使用者，這個係數還要往上調。

叫獸提醒

表 21-4 有個訊息值得你特別注意：量化的效益是「倍數級」的。從 FP16 到 INT4，記憶體直接降為四分之一——這往往就是「跑不動」與「跑得動」的分野。更關鍵的是，因為解碼階段受記憶體頻寬限制，參數變小同時也讓推論變快，等於一石二鳥。所以當你想在自己的機器上跑地端模型時，第一個該問的不是「要不要換更貴的顯示卡」，而是「量化過了嗎」。這是投資報酬率最高的一步，而且通常只需要下載一個量化版本的權重檔就完成了。

案例研討

案例一 不能外傳的製程文件

某製造企業想建立一個內部問答系統，讓工程師能以自然語言查詢設備手冊、製程規範與歷年的異常處理紀錄。他們最初評估直接呼叫雲端大模型的 API，但法務部門立刻否決了——這些文件涉及製程機密，不得傳出公司網路。

他們改採地端方案：在自有伺服器上部署一個經 INT8 量化的中型開源模型（依式 21-1，約需 13 GB 記憶體，單卡即可），並以 RAG 架構把內部文件建成向量索引。使用者提問時，系統先在本地檢索相關段落，再交由本地模型作答。

結果是：資料完全不出網路、回應延遲穩定可預測、且長期成本低於按次計費的 API。雖然模型的通用能力不如頂尖的雲端模型，但對「依據給定文件回答」這個相對特定的任務而言，已完全足夠。這個案例展示了 21.4.2 節的判準——當資料敏感性是硬約束時，選擇其實沒有懸念。

案例二 一本正經的胡說八道

某團隊用大模型為客服系統產生回覆。上線初期表現良好，直到有客戶依照系統提供的「產品保固條款第七條」提出申訴——而該公司的保固條款根本沒有第七條。

追查後發現，模型並未取得任何實際的條款資料，它只是依據訓練時看過的無數保固條款，「生成」了一段看起來極為合理、格式正確、語氣專業的文字。這正是 21.5.2 節的幻覺：錯誤被包裝得比正確答案還可信。

他們的修正是導入 RAG：所有涉及公司政策的回答，都必須先檢索實際文件，並在提示中明確要求「只依據以下資料回答，若資料中沒有相關內容，請回答不確定」。同時在輸出中附上引用來源，讓客服人員能快速查核。這個案例是幻覺危害最典型的展現——它的危險不在於錯，而在於「錯得很具有說服力」。

案例三 跑不動的 70B

某研究生想在實驗室的工作站（單張 24 GB 顯示卡）上跑一個 700 億參數的模型。他下載了 FP16 版本，執行後立刻因記憶體不足而失敗。

依式 21-1 估算，70B 模型的 FP16 需求為 140 GB，是他硬體的近六倍；即使量化到 INT4（35 GB），仍超過 24 GB。也就是說，這張卡無論如何都跑不動這個模型——這不是調參數能解決的問題。

他最終改用一個經 INT4 量化的 13B 模型（約 6.5 GB，加上推論開銷約 8 至 10 GB），在同一張卡上順利運行，且對他的任務（技術文件摘要）表現已足夠。這個案例的教訓很實際：在下載任何模型之前，先用式 21-1 算一下——五秒鐘的計算，可以省下數小時的嘗試與挫折。

案例四 昨天還好好的提示詞

某系統以雲端 API 進行產品分類，提示詞經過長期調校，準確率穩定在九成以上。某日開始，分類結果突然出現大量異常，但程式碼與提示詞都沒有任何改動。

原因是服務商更新了模型版本。新版模型在多數任務上表現更好，但對該團隊精心調校的提示詞產生了不同的反應——原本被穩定遵循的格式要求，在新版中偶爾被忽略。

這正是 21.4 節叫獸提醒所警告的可控性風險。他們的因應包括：改用有版本鎖定的 API 端點、建立自動化的回歸測試（每日以固定的測試集檢查輸出品質）、以及對關鍵流程改採地端模型以確保行為一致。這個案例提醒我們：在生產系統中，「可預測」有時比「更聰明」更有價值。

實作活動

活動一 估算你的部署需求

請運用式 21-1 與式 21-2，為三種情境做部署規劃。

- 步驟 1.** 查出你可用的硬體（個人電腦、實驗室工作站或雲端執行個體）有多少可用記憶體。
- 步驟 2.** 計算 7B、13B、70B 三種規模的模型，在 FP16、INT8、INT4 三種精度下各需多少記憶體。
- 步驟 3.** 依式 21-2 加上 1.3 倍的推論開銷，判斷哪些組合是你的硬體跑得動的。
- 步驟 4.** 若要支援四個使用者同時對話（KV 快取增加），你的判斷會如何改變？

活動二 實作一個小型 RAG

請建立一個以自有文件為知識來源的問答系統。

- 步驟 1.** 準備十份左右的文件（可用課程講義、設備手冊或公開技術文件），切分為適當長度的段落。
- 步驟 2.** 以嵌入模型把各段落轉為向量，存入向量資料庫或簡單的向量索引。
- 步驟 3.** 撰寫查詢流程：把問題轉為向量、檢索最相似的三個段落、組成提示送給模型。
- 步驟 4.** 刻意提問一個「文件中沒有答案」的問題，觀察模型是否誠實回答不知道；若它編造答案，試著修改提示詞來改善。

活動三 雲端與地端的選型分析

請為一個具體的應用情境（可自選，建議與智慧製造或機器人相關）撰寫一份選型分析。內容應包含：該應用的資料敏感性如何？預估的每月呼叫量與對應的雲端費用？延遲要求為何？所需的模型能力等級？接著依表 21-3 逐項評估，並運用式 21-1 估算地端方案所需的硬體規格與成本。最後給出你的建議，並說明在什麼條件改變時，你的建議會反轉。

延伸閱讀

- **《Attention Is All You Need》**：Vaswani 等人於二〇一七年發表的 Transformer 原始論文，是 21.2 節的第一手資料，篇幅簡短且圖示清晰。
- **規模定律與湧現能力的相關研究**：關於模型規模、資料量與表現之關係的實證研究，可深化 21.1.2 節對「為何大就不同」的理解。
- **RAG 原始論文與實務指南**：Lewis 等人的檢索增強生成論文，以及各向量資料庫的實務文件，是 21.5.3 節的延伸。
- **模型量化的技術文件**：關於 INT8 與 INT4 量化方法及其品質影響的技術報告，可補強 21.7 節刻意略過的數學細節。

本章小結

1. 語言模型的核心任務是預測下一個詞元，訓練以自監督方式進行而無需人工標註；規模跨過門檻後出現湧現能力，但「為何如此」學界尚無定論，應避免將其浪漫化為理解或意識。
2. Transformer 以注意力機制為核心，讓模型在處理每個詞時動態決定該關注前文的哪些部分；其查詢、鍵、值的內積運算與第 17、19 章的向量相似度一脈相承，但計算量與序列長度的平方成正比。
3. 大模型養成分三階段：預訓練賦予知識與能力（成本佔絕大部分）、監督式微調教它以助手方式應答、對齊調整其行為傾向；參數高效微調（如 LoRA）以低秩修正大幅降低微調成本。
4. 雲端大模型能力最強、起步成本低但資料須外傳且行為可能隨版本改變；地端模型隱私可控、延遲可預測、行為一致但受硬體限制。選型判準包括資料敏感性、用量、延遲要求與能力需求，實務上常採混合策略。
5. 大模型有三個知識限制：訓練資料有截止時間、不知道你的私有資料、以及在缺乏資訊時會產生流暢但錯誤的幻覺——其危險在於錯誤被包裝得極為可信。
6. RAG 以「先檢索、再生成」補足知識缺口，使知識可即時更新、可使用私有資料並可附來源查證；它幾乎完全建立在本書前面的技術之上（文件前處理、向量表示、相似性搜尋、向量資料庫）。
7. 推論分預填充（運算受限、可平行）與解碼（記憶體頻寬受限、序列進行）兩階段，故輸出越長越慢；服務化須考量批次處理、串流輸出、KV 快取管理與用量監控。

8. 參數記憶體為 $P \times b / 8 / 10^9$ (式 21-1)，實際需求約再乘 1.2 至 1.5 倍 (式 21-2)；量化從 FP16 到 INT4 可將記憶體降為四分之一，且因解碼受記憶體頻寬限制而同時提升速度，是地端部署投資報酬率最高的一步。

習題

一、觀念題

1. 語言模型的訓練為何不需要人工標註？這個特性為何對大規模訓練至關重要？
2. 請以「機器手臂抓取了那個零件，因為它太重了」為例，說明注意力機制解決了什麼問題。
3. 何謂幻覺？請說明它為何會發生，以及為何它比「模型回答不出來」更危險。
4. 請說明 RAG 的三個階段，並指出它分別用到了本書前面哪些章節的技術。

二、計算題

1. 某模型有 340 億 (34B) 參數。(一) 請以式 21-1 計算它在 FP16、INT8、INT4 三種精度下的參數記憶體需求；(二) 若你的硬體有 48 GB 可用記憶體，並依式 21-2 取 1.3 倍的推論開銷，哪些精度是可行的？
2. 某企業評估導入方案。雲端 API 每百萬詞元收費 300 元，預估每月用量 8 千萬詞元；地端方案需購置硬體 60 萬元 (以三年攤提)，每月電力與維運約 8 千元。(一) 計算雲端方案的每月成本；(二) 計算地端方案的每月攤提成本；(三) 求出兩者的損益平衡點 (每月用量為多少詞元時成本相等)。

三、思考與討論

1. 案例四中，服務商更新模型導致既有系統行為改變。請討論：在把 LLM 導入生產系統時，應建立哪些機制來管理這種不可控性？
2. 本章指出「湧現能力」的成因學界尚無定論。請討論：作為工程師，在不完全理解一項技術運作原理的情況下使用它，應抱持什麼樣的態度與防護措施？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

因為訓練方式是「遮住後面的字、讓模型猜下一個詞元」，而正確答案就是原文中實際出現的下一個字——文本本身即包含答案，無需額外標註，此即自監督學習。重要性：人工標註是大規模訓練最大的成本瓶頸 (需人力、時間且品質難以確保)；自監督使標註成本降為零，模型因而能吃下整個網際網路等級的語料。這也解釋了為何 LLM 的資料量能達到數兆詞元的量級——若需人工標註，這在經濟上完全不可能。

第 2 題

此句中的「它」在語法上可指涉「零件」或「機器手臂」，須依語意判斷 (重的是零件)。要正確處理這個代名詞，模型必須在處理「它」這個詞時，把注意力放到前文的「零件」而非「手臂」上。注意力機制解決的問題正是：每個詞的意義取決於句中其他詞，但取決於「哪些詞」會隨情況而異，

故需要動態計算權重而非固定規則。相較早期的循環網路只能依序累積資訊、距離越遠影響越弱，注意力讓任兩個詞之間都能直接建立連結，不受距離限制。

第 3 題

幻覺指模型產生流暢、格式正確、看似合理但內容錯誤的輸出。發生原因：模型的訓練目標是「產生看起來合理的下一個字」，而非「說實話」；當它缺乏正確資訊時，仍會盡責地依統計規律生成看似可信的文字。比「回答不出來」更危險的原因：若模型坦承不知道，使用者會自行查證；但幻覺的錯誤被包裝得比正確答案還可信（格式專業、語氣篤定、細節具體），使用者難以察覺而直接採信，可能導致實際損害（如案例二中客戶依據不存在的保固條款提出申訴）。核心觀念：危險不在於錯，而在於「錯得很有說服力」。

第 4 題

三階段與對應技術：其一，離線階段——把文件切成段落並轉為向量存入向量資料庫，用到第 13 章的資料前處理與管線、第 17 章的向量表示（嵌入）、第 6 章的 NoSQL 向量資料庫。其二，查詢階段——把問題轉為向量並召索最相似的段落，用到第 14 章的相似性搜尋（大規模時採 LSH 或近似最近鄰）、第 19 章的餘弦相似度。其三，生成階段——把檢索結果連同問題送給模型作答。可補充：RAG 的優勢在於知識可即時更新、能使用私有資料、並可附來源查證以降低幻覺，這使它成為企業導入 LLM 的主流架構。

二、計算題

第 1 題

$P = 340 \text{ 億} = 3.4 \times 10^{10}$ ，依式 21-1 計算（記憶體 $= P \times b \div 8 \div 10^9$ ）。

- (1) FP16 ($b = 16$) : $3.4 \times 10^{10} \times 16 \div 8 \div 10^9 = 68.0 \text{ GB}$ 。
- (2) INT8 ($b = 8$) : $3.4 \times 10^{10} \times 8 \div 8 \div 10^9 = 34.0 \text{ GB}$ 。
- (3) INT4 ($b = 4$) : $3.4 \times 10^{10} \times 4 \div 8 \div 10^9 = 17.0 \text{ GB}$ 。

可行性判斷（依式 21-2 取 1.3 倍）：FP16 需 $68.0 \times 1.3 = 88.4 \text{ GB}$ ，超過 48 GB，不可行。INT8 需 $34.0 \times 1.3 = 44.2 \text{ GB}$ ，小於 48 GB，可行但餘裕有限（長上下文或多使用者時可能不足）。INT4 需 $17.0 \times 1.3 = 22.1 \text{ GB}$ ，遠低於 48 GB，可行且餘裕充足。結論：INT8 勉強可行 INT4 最為穩妥。

第 2 題

雲端每百萬詞元 300 元、月用量 8 千萬詞元；地端硬體 60 萬元三年攤提、月維運 8 千元。

- (1) 雲端每月成本 $= (80,000,000 \div 1,000,000) \times 300 = 80 \times 300 = 24,000 \text{ 元}$ 。
- (2) 地端每月成本 $= \text{硬體攤提} (600,000 \div 36) + \text{維運} 8,000 = 16,667 + 8,000 = 24,667 \text{ 元}$ 。
- (3) 損益平衡點：設月用量為 x 百萬詞元，令 $300x = 24,667$ ，得 $x \approx 82.2$ 百萬詞元，即約每月 8,220 萬詞元。

判讀：目前用量 8 千萬詞元時，雲端（24,000 元）略低於地端（24,667 元），兩者相當接近。若用量持續成長超過約 8,220 萬詞元，地端即開始占優；反之若用量下降，雲端更划算。但實務決策不應只看成本——如案例一所示，若資料敏感性構成硬約束，即使成本較高仍須選地端；反之若用量波動極大，雲端的彈性也有其價值。

三、思考與討論

第 1 題

應建立的機制包括：其一，版本鎖定——使用可指定版本的 API 端點，避免服務商自動升級；並在升級前於測試環境驗證。其二，自動化回歸測試——建立固定的測試集與期望輸出，每日自動執行並比對，使行為改變能被立即發現（呼應第 13 章的資料品質檢核）。其三，輸出驗證層——對模型輸出做格式與合理性檢查（如必須為預定義的分類之一、數值須落在合理範圍），不合格則走人工或備援流程。其四，可觀測性——記錄輸入輸出樣本與品質指標，以便回溯分析。其五，備援方案——對關鍵流程準備地端模型或規則式後備，並可在雲端異常時切換。其六，合約與溝通——確認服務商的版本政策與變更通知機制。核心原則：把外部 LLM 視為「不完全可控的第三方相依」，比照第 3 章對待網路與外部服務的態度來設計容錯。

第 2 題

應抱持的態度：其一，區分「能用」與「理解」——不理解原理不代表不能使用（許多工程實踐早於理論解釋），但必須承認自己的認知邊界，不對其行為做出超出實證的推論。其二，以實證代替信念——既然無法從原理推導其行為，就必須大量測試，尤其是邊界案例與失效模式。其三，避免過度外推——在測試過的情境中表現良好，不保證在未測試的情境中也如此。防護措施包括：限縮應用範圍於可驗證的任務；設置人為審核關卡於高風險決策；建立輸出驗證與異常偵測；保留可回退的備援路徑；明確告知使用者系統的性質與限制；以及持續監控實際表現而非依賴上線前的評估。可連結第 19 章的倫理討論——工程師的責任不只在於讓系統運作，也在於誠實面對它的不確定性並據此設計防護。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 21 章 大語言模型 (LLM)

台灣自編大學教科書

@prof

大數據分析與雲端運算

第伍篇 生成式 AI 與開源生態

Generative AI & Open-Source Ecosystem

第 22 章

AI 代理 (AI Agents)

AI Agents

機器人叫獸 (李世淵) 編著

叫獸開講

我先講一個實驗室裡真實發生的對比。

第一種情況：學生問模型「這個振動訊號的頻譜大概長什麼樣？」模型憑著訓練時看過的知識，給了一段描述。聽起來很專業，但它其實沒看過那個訊號——它只是在猜。

第二種情況：我們給了模型一個工具，讓它能執行 Python 程式。同樣的問題，這次它做了不同的事：它寫了一段程式讀取訊號檔、做傅立葉轉換、把結果印出來，看了實際數字之後，才告訴學生「主頻在 127 赫茲附近，有明顯的二次諧波」。

差別在哪裡？第一種是「回想」，第二種是「去做」。當一個模型能夠呼叫工具、觀察結果、再決定下一步時，它就從一個「會說話的模型」，變成了一個「會做事的代理」。這是本章的核心轉變。

但我要立刻潑一盆冷水。代理最大的問題，不是它不夠聰明，而是「錯誤會累積」。上一章的模型只要答對一次就好；代理卻要連續做對十步、二十步，而每一步都可能出錯。這正是第 13 章那條端到端可靠度公式的翻版——只是這次，每一步的成功率遠低於資料管線。這一章的技術很有趣，但我希望你帶走的是清醒：知道它能做什麼，更要知道它會在這裡壞掉。

學習目標

研讀完本章之後，你應該能夠：

1. 說明代理與單純語言模型的差異，並指出構成代理的四個要素。
2. 描述 ReAct 範式的推理與行動交替循環。
3. 說明工具使用與函式呼叫的運作機制與設計原則。
4. 區分短期與長期記憶，並說明規劃與反思的作用。
5. 說明多代理協作的常見模式與其取舍。
6. 評估代理的可靠度風險，並設計適當的防護機制。

核心概念

核心概念	說明
AI 代理	能自主決定行動、呼叫工具並依結果調整的系統，以語言模型為決策核心。
ReAct	推理（Reasoning）與行動（Acting）交替進行的代理範式。
工具使用	讓模型能呼叫外部程式、API 或資料庫以取得資訊或執行動作。
函式呼叫	模型以結構化格式輸出「要呼叫哪個函式、參數為何」的機制。
記憶	代理保存並取用過往資訊的機制，分短期（上下文）與長期（外部儲存）。
規劃與反思	把任務拆解為步驟，並在執行後檢視結果、修正做法。
多代理協作	由多個各司其職的代理分工合作完成複雜任務。

表 22-1 本章核心概念一覽

22.1 從模型到代理

22.1.1 被動與主動的分界

承接叫獸開講，我們把差異講得更精確。上一章的語言模型是「被動」的：你給輸入、它給輸出，一問一答，中間不與外界互動。它的知識全部來自訓練時的參數，因此有第 21 章談過的三個限制——知識過期、不知私有資料、以及會產生幻覺。

代理則是「主動」的：它會判斷「要完成這個任務，我需要什麼資訊、該做什麼動作」，然後實際去執行——查資料庫、呼叫 API、執行程式、甚至控制設備。取得結果後，它再依據新資訊決定下一步。

這個轉變帶來一個關鍵好處：模型不必再「憑記憶回答」，而能「查證後回答」。叫獸開講的例子中，第二種做法之所以可信，正是因為它真的算過。這在本質上與第 21 章的 RAG 相通——RAG 是「幫模型查資料」，代理則更進一步，是「讓模型自己決定要查什麼、怎麼查」。

22.1.2 代理的四個要素

一個完整的代理通常由四個部分構成，本章各節會逐一展開。

要素	作用	對應章節
決策核心	以語言模型判斷下一步該做什麼	22.2 ReAct
工具	與外界互動的能力：查詢、計算、執行	22.3 工具與函式呼叫
記憶	保存並取用過往的資訊與經驗	22.4 記憶與規劃
規劃與反思	拆解任務、檢視結果、修正做法	22.4 記憶與規劃

表 22-2 代理的四個構成要素

叫獸提醒

我要提醒一個容易被行銷術語混淆的地方。近年「代理」這個詞被用得非常浮濫，有些產品只是「把提示詞包裝成按鈕」，也自稱代理。判斷的關鍵很簡單：它能不能「自己決定要做什麼」？如果流程是寫死的（先做 A、再做 B、然後 C），那只是一條自動化管線，用第 13 章的編排器就能做，而且更可靠、更便宜。真正的代理，是那個「下一步要做什麼」由模型當場判斷的系統。認清這個區別很重要——因為代理的彈性是有代價的，而那個代價就是可靠度。

22.2 ReAct 範式

22.2.1 推理與行動的交替

ReAct 是目前最廣為使用的代理範式，它的名稱來自 Reasoning（推理）與 Acting（行動）的結合。核心想法是讓模型在「想」與「做」之間交替循環，而非一次想完再做。

一次循環包含三個環節：

- **思考 (Thought)**：模型分析當前狀況，判斷「我現在需要什麼、下一步該做什麼」。
- **行動 (Action)**：模型決定呼叫哪個工具、使用什麼參數。
- **觀察 (Observation)**：系統執行該工具，把結果回傳給模型，作為下一輪思考的依據。

這三步不斷重複，直到模型判斷任務已完成，輸出最終答案。

22.2.2 一個具體的例子

假設任務是「查出我們三號機台上個月的平均稼動率，並判斷是否低於年度目標」。一個 ReAct 代理的執行過程可能如下：

- 第 1 輪。** 思考：我需要三號機台上個月的稼動率資料，應該查詢生產資料庫。行動：呼叫資料庫查詢工具。觀察：取得該月每日的稼動率數列。
- 第 2 輪。** 思考：我需要計算平均值。行動：呼叫計算工具。觀察：得到平均稼動率 78.3%。
- 第 3 輪。** 思考：我還需要知道年度目標是多少，這應該在管理文件中。行動：呼叫文件檢索工具。觀察：找到年度目標為 85%。
- 第 4 輪。** 思考：78.3% 低於 85%，我已經有足夠資訊可以回答了。輸出最終答案。

注意這個過程的關鍵特性：每一步該做什麼，都不是事先寫死的，而是模型依當下情況判斷的。若資料庫查詢失敗，它可能改用別的方式；若發現需要額外資訊，它會多查一輪。這正是代理的彈性所在。

22.2.3 為什麼「把推理寫出來」有效

ReAct 有一個看似瑣碎、實則關鍵的設計：它要求模型把「思考」明確寫出來，而不是直接輸出行動。

這為什麼有效？回想第 21 章的提示技巧——要求模型「一步一步思考」能顯著提升正確率。原因在於模型是自迴歸的：它產生的每個詞元都會成為後續生成的上下文。把推理過程寫出來，等於讓模型「先建立起中間結論」，再據此決定行動，而非一步跳到結果。

這個外顯的推理還有兩個工程上的好處：其一，可除錯——當代理做錯事時，你能看到它「當時是怎麼想的」，而不是面對一個黑箱；其二，可審核——在高風險場景中，人可以檢視推理鏈是否合理，再決定是否放行。

22.3 工具使用與函式呼叫

22.3.1 函式呼叫的機制

工具使用的技術基礎是「函式呼叫」（function calling）。它的運作方式是這樣：

首先，你把可用的工具以結構化的方式描述給模型——每個工具的名稱、功能說明、以及所需參數的格式。接著當模型判斷需要使用某個工具時，它不會直接執行（模型本身沒有執行能力），而是輸出一段結構化的資料，表明「我要呼叫工具 X，參數是 Y」。

真正的執行由你的程式碼負責：解析模型的輸出、實際呼叫該函式、把結果格式化後送回給模型。這個「模型決定、程式執行」的分工至關重要——它意味著你完全掌控模型能做什麼、不能做什麼。

22.3.2 常見的工具類型

- **檢索工具：**查詢向量資料庫、搜尋文件或網頁，補足模型的知識缺口（即第 21 章的 RAG，此處成為代理可自主呼叫的工具）。
- **計算工具：**執行程式碼或呼叫計算函式。這解決了語言模型不擅長精確計算的弱點——與其讓它「猜」算術結果，不如讓它寫程式去算。
- **資料查詢工具：**存取結構化資料庫，例如把自然語言問題轉為 SQL 查詢再執行。

- **行動工具：**實際改變外界狀態，如寄送郵件、建立工單、控制設備。這類工具風險最高，必須嚴格控管。

22.3.3 工具設計的原則

工具設計的品質，往往比模型本身更能決定代理的成敗。幾條實務原則：

- **描述要清楚具體：**模型只能依據你提供的說明判斷何時該用哪個工具。含糊的描述會導致誤用或漏用。
- **功能要單一：**每個工具只做一件事。一個「萬能工具」會讓模型難以判斷該傳什麼參數。
- **回傳要精簡：**工具的輸出會佔用上下文空間。回傳一整份文件不如回傳關鍵摘要——上下文是有限且昂貴的資源。
- **失敗要明確：**工具出錯時應回傳清楚的錯誤訊息，讓模型知道發生了什麼、能否重試。回傳空值或含糊訊息會讓代理陷入困惑。
- **權限要最小化：**只給代理完成任務所必需的權限。特別是寫入類的工具，應嚴格限制範圍。

最後一項不只是安全考量，也是可靠度考量。代理會犯錯，而權限越小，犯錯的後果越可控。

22.4 記憶、規劃與反思

22.4.1 兩種記憶

代理需要記憶，才能在多輪互動中保持連貫。記憶分為兩個層次。

短期記憶就是模型的上下文視窗——當前對話與執行歷程都在其中。它的優點是模型能直接看到，缺點是容量有限且昂貴（回想第 21 章，注意力的計算量與長度成平方）。當任務執行了幾十輪之後，上下文很快就會爆滿。

長期記憶則是把資訊存到外部（如向量資料庫），需要時再檢索回來。它的容量近乎無限，但模型不會自動知道其中有什麼——必須主動檢索。這在技術上就是第 21 章的 RAG，只是這裡儲存的是代理自己的經驗與觀察。

實務上的常見做法是「摘要壓縮」：當上下文接近上限時，把較早的內容摘要成精簡版本，保留關鍵結論而捨棄細節，同時把完整紀錄存入長期記憶備查。

22.4.2 規劃：先想清楚再動手

對於複雜任務，讓代理「邊做邊想」往往效率不彰——它可能走了很多冤枉路才發現方向錯了。因此許多代理架構會加入明確的規劃階段。

常見做法是先讓模型把任務拆解為若干子步驟，形成一個計畫，再依序執行。這樣做的好處是能及早發現方向問題，也讓執行過程更可預測、更容易監控。

但計畫不能一成不變。執行過程中往往會發現新資訊（某個資料不存在、某個工具失敗），此時代理需要「重新規劃」的能力。這種「計畫—執行—依結果調整計畫」的循環，與第 13 章的工作流編排有幾分相似，差別在於：編排器的流程是人寫死的，而代理的流程是模型當場產生的。

22.4.3 反思：檢視自己的成果

反思（reflection）是讓代理在完成某個步驟後，回頭檢視「這個結果合理嗎、達成目標了嗎」，若否則修正重做。

實證顯示這能顯著提升品質，原因與 22.2.3 節相通：模型「檢查一份既有結果」通常比「一次寫對」容易。就像人寫文章，初稿加修改往往勝過一次到位。

但反思有兩個限制必須認清。其一，它會增加成本——每次反思都是一次額外的模型呼叫。其二，也更根本的是：模型不見得能可靠地判斷自己錯了。若它一開始就誤解了任務，反思時很可能用同樣的誤解來檢查，於是「檢查通過」卻依然是錯的。因此對於高風險任務，外部的驗證（用工具實際檢查、或由人審核）遠比自我反思可靠。

22.5 多代理協作

22.5.1 為何要分工

當任務複雜到單一代理難以勝任時，一種思路是讓多個代理分工協作，各自負責不同的角色或專長。

這樣做的理由有幾個：其一，專業化——每個代理可以配置專屬的提示詞、工具與知識，專注做好一件事；其二，上下文分離——各代理維護自己的上下文，避免單一上下文被塞爆；其三，可平行——彼此獨立的子任務能同時進行。

你應該會覺得這個思路很眼熟——這正是第 18 章 Pregel「像頂點一樣思考」的精神：不去設計整體的複雜行為，而是定義單一個體該做什麼，讓整體行為由個體的互動產生。

22.5.2 常見的協作模式

模式	運作方式	適用情境
管線式	各代理依序處理，前者輸出為後者輸入	有明確階段的任務，如研究→撰寫→校對
主從式	一個主代理拆解任務、分派給子代理並彙整結果	任務可分解為獨立子項
辯論式	多個代理提出不同觀點並互相檢視	需要多角度評估、降低單一視角偏誤
審查式	一個代理執行、另一個獨立審查其成果	品質要求高、需外部驗證

表 22-3 多代理協作的常見模式

其中「審查式」值得特別注意，它正是 22.4.3 節反思限制的解方——由「另一個」代理來檢查，避免了自己檢查自己的盲點。這與軟體工程的程式碼審查是同一個道理。

22.5.3 代價：複雜度與可靠度

多代理不是免費的午餐。它帶來三個實際的代價：成本倍增（每個代理都要呼叫模型）、除錯困難（出錯時要追查是哪個環節、以及代理之間的訊息傳遞是否正確）、以及最關鍵的——可靠度進一步下降。

因為每增加一個代理、每多一次交接，就多一個可能出錯的環節。下一節的公式會把這個問題量化，你會發現它比直覺想像的嚴重得多。

因此實務上的原則是：能用單一代理解決的，不要用多代理；能用固定流程解決的，不要用代理。複雜度應該是被需求逼出來的，而不是被技術的新穎性吸引過去的。

22.6 機器人任務代理與具身應用

22.6.1 從螢幕走進實體世界

前面談的代理都在資訊世界中活動——查資料、寫程式、產生文件。而「具身代理」(embodied agent)則是讓代理控制實體的機器人或設備，在真實世界中行動。

這個方向對本學程特別有意義。想像一個場景：操作員以自然語言說「把三號料架上的瑕疵品挑出來放到待檢區」。一個具身代理需要：理解這句話的意圖、規劃出動作序列、呼叫視覺模型判斷哪些是瑕疵品、控制機械手臂執行、並在過程中處理意外（抓取失敗、物件位置偏移）。

技術上，這通常是「語言模型負責高階規劃、既有的控制系統負責低階執行」的分層架構。語言模型擅長理解意圖與拆解任務，但不擅長精確控制；而傳統的運動規劃與控制演算法在後者上成熟可靠。讓兩者各司其職，比要求語言模型直接輸出關節角度務實得多。

22.6.2 實體世界的三個嚴峻挑戰

- **錯誤不可逆：**在資訊世界中，錯誤的輸出可以刪除重來；但機械手臂撞壞了工件、或把零件放錯位置，代價是真實的。這使可靠度的要求遠高於一般應用。
- **感知不完美：**代理對世界的認知來自感測器，而感測器有雜訊、有遮擋、有失效。它必須在資訊不完整的情況下行動——這正是第 3 章「網路是可靠的」那類誤解在實體世界的翻版。
- **即時性要求：**許多控制迴路需要毫秒級的反應，而大模型的推論動輒數百毫秒到數秒。這使得「模型直接控制」在多數情況下不可行，必須採用分層架構，把即時控制留給傳統系統。

第三點也解釋了為何第 21 章的地端部署與量化如此重要：若要讓代理在機器人端運作，模型必須小到能在邊緣裝置上跑、快到能滿足時序要求。這條線索會在第 35、36 章繼續展開。

22.6.3 人在迴路中

面對上述挑戰，實務上最重要的設計原則是「人在迴路中」(human-in-the-loop)——在關鍵決策點保留人的確認。

具體做法包括：對不可逆的動作要求人工確認；設定代理可自主行動的範圍，超出範圍則暫停待命；提供清楚的「當前在做什麼、為什麼這樣做」的顯示，讓人能即時介入；以及最基本的——保留隨時中止的能力。

這不是對技術的不信任，而是對後果的負責。你會發現這個思路與第 19 章談推薦系統倫理時的立場一致：技術的能力越強，設計者對其後果的責任就越大。

22.6.4 承先啟後

本章我們看到了從模型到代理的轉變——從「憑記憶回答」到「查證後回答」、從被動應答到主動行動 (22.1)；認識了 ReAct 的思考、行動、觀察循環 (22.2)、工具與函式呼叫的機制與設計原則 (22.3)、記憶規劃與反思 (22.4)、多代理協作的模式與代價 (22.5)，以及具身代理在實體世界面對的嚴峻挑戰 (22.6)。

我最想讓你帶走的，是那個貫穿全章的清醒：代理的彈性與其可靠度是互相拉扯的。步驟越多、代理越多、自主性越高，出錯的機會就越大——而這是數學上的必然，不是工程上的疏失。因此真正的專業，不在於能不能做出代理，而在於知道何時該用代理、何時該用第 13 章那種可靠的固定流程。

到目前為止，我們談的模型與代理，多數是既有的、別人做好的。但這些模型從哪裡來？權重放在哪裡？別人的成果如何取用、自己的成果如何分享？當一個團隊要協作開發時，程式碼與模型如何版本管理、如何確保可重現？這些「生態系」的問題，就是下一章的主題——開源協作與模型生態系。

公式與流程：代理的可靠度陷阱

這一節要把本章反覆強調的那個警告，變成一條你能親手計算的公式。

代理完成任務需要連續執行 n 個步驟，且每一步都必須正確。設單步正確的機率為 q ，各步驟獨立，則整個任務成功的機率為：

$$P(\text{成功}) = q^n$$

式 22-1 代理多步驟任務的成功率

這條式子與第 13 章的式 13-1 完全相同——但關鍵差異在於，資料管線的單步成功率可以做到 99% 以上，而代理的單步正確率往往只有 90% 到 95%（模型可能誤判該用哪個工具、參數傳錯、或誤讀了工具回傳的結果）。下表顯示這個差異的後果。

單步正確率 q	3 步	5 步	10 步	20 步
99%	97.0%	95.1%	90.4%	81.8%
95%	85.7%	77.4%	59.9%	35.9%
90%	72.9%	59.1%	34.9%	12.2%

表 22-4 單步正確率與步驟數對任務成功率的影响

看中間那一列：即使每一步都有 95% 的正確率（以語言模型而言已算相當可靠），跑十步後成功率只剩約六成，跑二十步則不到四成。這就是為什麼「讓代理自主完成長任務」在實務上如此困難——不是模型不夠聰明，而是機率的乘法太無情。

驗證與重試的效果

因應之道與第 13 章相同：為每一步加上驗證與重試。若每步允許重試至多 k 次，則單步的最終成功率提升為 $1 - (1 - q)^{(k+1)}$ ，整體成功率變為：

$$P(\text{成功, 含重試}) = [1 - (1 - q)^{(k+1)}]^n$$

式 22-2 含驗證重試的成功率

以 $q = 90\%$ 、 $n = 10$ 步為例：

重試次數 k	單步最終正確率	10 步任務成功率	改善
0 (不重試)	90.0%	34.9%	—
1	99.0%	90.4%	約 2.6 倍
2	99.9%	99.0%	約 2.8 倍

表 22-5 驗證重試對代理可靠度的提升 ($q = 90\%$ 、 $n = 10$)

叫獸提醒

表 22-5 給了你設計代理系統最重要的一條原則：與其追求「更聰明的模型」，不如投資在「每一步都能被驗證」。從 34.9% 到 90.4%，靠的不是換更大的模型，而是加上一層檢查與

重試。但這裡有個前提必須成立——你要能「判斷這一步是否正確」。若某個步驟的正確與否無法被程式自動驗證（例如「這段摘要寫得好不好」），重試就失去了依據。所以設計代理時，第一個該問的問題是：我的每一個步驟，都能被檢查嗎？若不能，就該考慮縮短步驟數、或把那個無法驗證的環節交給人。這也再次呼應第 13 章：可重試的前提是冪等與可驗證，兩者缺一不可。

案例研討

案例一 會算的代理與會猜的模型

回到叫獸開講。同樣被問到訊號的頻譜特性，單純的語言模型只能依訓練時看過的類似敘述來「生成」一段聽起來專業的描述——它從未接觸過那個檔案，本質上是在猜。

而具備程式執行工具的代理則走了完全不同的路徑：讀檔、做傅立葉轉換、檢視實際數值、依據數值作答。它的答案有具體依據，而且過程可被檢視與重現。

這個對比揭示了代理最核心的價值：把「語言模型不擅長的事」外包給擅長的工具。模型不擅長精確計算，那就讓它寫程式去算；模型不知道最新資料，那就讓它去查。模型真正該做的，是「判斷該用什麼工具、如何解讀結果」——這才是它的強項。

案例二 二十步之後的崩壞

某團隊開發了一個資料分析代理，希望它能自主完成「讀取資料、清理、分析、產生圖表、撰寫報告」的完整流程。在示範時表現亮眼，但實際部署後，成功率低得令人沮喪——約只有三成的任務能順利跑完。

追查後發現，這個流程實際展開後有近二十個步驟，而每一步的正確率約在九成上下。依式 22-1， 0.9 的二十次方僅約 12%——考慮到部分步驟較簡單，實測的三成成功率其實完全符合數學預期。問題不在任何單一環節，而在步驟太多。

他們的修正有兩個方向：其一，把可以固定的部分（讀取、清理）改成寫死的管線，只讓真正需要判斷的環節交給代理，步驟數從二十降到五；其二，為每個步驟加上自動驗證與重試。兩者合併後，成功率提升到九成以上。這個案例是式 22-1 與式 22-2 最直接的印證。

案例三 權限開太大的代價

某團隊為維運代理配置了資料庫工具，為求方便，給了它完整的讀寫權限。某次代理誤解了使用者的指令，執行了一個範圍過大的更新操作，影響了大量正式資料，最終須從備份還原。

問題的根源不是模型太笨，而是 22.3.3 節的「權限最小化」原則被違反了。代理一定會犯錯——這是機率問題而非能力問題——因此設計時就該假設它會犯錯，並限制犯錯的後果。

他們的修正包括：把工具拆成唯讀查詢與寫入兩類，寫入類的工具需經人工確認；限制單次操作可影響的資料筆數上限；以及所有寫入操作先在測試環境執行、確認影響範圍後才套用。這個案例的教訓與第 13 章相通——設計系統時，要問的不是「它會不會出錯」，而是「它出錯時會怎樣」。

案例四 自己檢查自己的盲點

某團隊為代理加入了反思機制，要求它在產出報告後自我檢查。但他們發現，有一類錯誤反思完全抓不到：當代理一開始就誤解了任務需求時，它的自我檢查也是基於同樣的誤解進行，因此總是「檢查通過」。

這正是 22.4.3 節指出的限制：自我反思無法跳脫自己的認知框架。他們改採 22.5.2 節的「審查式」多代理架構——由一個獨立的審查代理，僅根據「原始任務需求」與「產出結果」進行比對，不接觸執行過程。這樣的獨立視角成功攔截了多數的需求誤解。

這個案例的思路，其實與軟體工程的程式碼審查、學術界的同儕評審完全一致：檢查者必須獨立於執行者。這是一個跨領域通用的原則，值得你記在心裡。

實作活動

活動一 手工模擬一次 ReAct

不必寫程式，先用紙筆理解代理的運作邏輯。

- 步驟 1.** 設定一個需要多步驟的任務（例如「查出某產品的庫存量，若低於安全水位則計算應補貨數量」）。
- 步驟 2.** 列出你會提供哪些工具（名稱、功能、參數），並依 22.3.3 節的原則撰寫工具描述。
- 步驟 3.** 手寫出完整的思考—行動—觀察循環，直到任務完成。
- 步驟 4.** 為每一步標註「這一步可能怎麼出錯」，並以式 22-1 估算整體成功率（假設單步正確率 92%）。

活動二 實作一個雙工具代理

請以任一支援函式呼叫的模型，實作一個具備兩個工具的簡易代理。

- 步驟 1.** 實作工具一：一個計算工具（可執行數學運算或簡單的程式碼）。
- 步驟 2.** 實作工具二：一個查詢工具（可查詢你自建的小型資料表或文件）。
- 步驟 3.** 撰寫代理主迴圈：把工具描述傳給模型、解析其函式呼叫請求、執行並回傳結果，重複直到模型輸出最終答案。
- 步驟 4.** 設計三個需要同時使用兩個工具的問題進行測試，記錄成功率與失敗的原因分類。

活動三 可靠度分析與改善設計

請針對活動二的代理進行可靠度分析與改善。首先，把任務展開為明確的步驟序列並計算步驟數 n ；接著由實測估計單步正確率 q ，以式 22-1 計算理論成功率並與實測比較。然後為每個步驟設計「自動驗證方式」——若某步驟無法被自動驗證，請說明原因並提出替代方案（縮短流程、改為固定管線、或引入人工確認）。最後依式 22-2 估算加上重試後的成功率，並實際實作驗證與重試機制，比較改善前後的差異。

延伸閱讀

- **ReAct 原始論文：**Yao 等人發表的《ReAct: Synergizing Reasoning and Acting in Language Models》，是 22.2 節的第一手資料。
- **反思機制的相關研究：**關於自我反思與外部驗證如何影響代理表現的實證研究，可深化 22.4.3 節對其限制的理解。

- **多代理框架的官方文件：**各主流代理框架關於角色定義、訊息傳遞與協作模式的說明，是 22.5 節的實作參考。
- **具身智慧與機器人語言指令的研究：**關於語言模型如何與機器人控制系統分層整合的技術文獻，是 22.6 節的延伸，也可作為第 36 章的預習。

本章小結

1. 代理與單純語言模型的分界在於「主動」——它能判斷需要什麼、呼叫工具實際執行、再依結果決定下一步，使模型從「憑記憶回答」轉為「查證後回答」；若流程是寫死的，那只是自動化管線而非代理。
2. 代理由四要素構成：以語言模型為決策核心、以工具與外界互動、以記憶保存資訊、以規劃與反思調整做法。
3. ReAct 以思考、行動、觀察三環節交替循環，每一步做什麼皆由模型當場判斷；要求模型把推理外顯，不僅提升正確率，也讓代理可除錯、可審核。
4. 函式呼叫採「模型決定、程式執行」的分工，故開發者完全掌控代理的能力邊界；工具設計應描述清楚、功能單一、回傳精簡、失敗訊息明確，並遵守權限最小化。
5. 短期記憶即上下文視窗（容量有限且成本隨長度平方成長），長期記憶則存於外部並須主動檢索；規劃使執行更可預測，反思能提升品質，但模型無法可靠地跳脫自身的認知框架來檢查自己。
6. 多代理協作有管線式、主從式、辯論式與審查式等模式，其中審查式可解決自我反思的盲點；但多代理會使成本倍增、除錯困難且可靠度進一步下降，應在確有需要時才採用。
7. 具身代理在實體世界面對三個嚴峻挑戰：錯誤不可逆、感知不完美、即時性要求；實務上採「語言模型高階規劃、傳統系統低階控制」的分層架構，並以人在迴路中的設計負責後果。
8. 代理的成功率為 q^n （式 22-1），因單步正確率遠低於資料管線，即使 $q = 95\%$ ，十步後也僅剩約六成；加上驗證與重試可大幅改善（式 22-2），但前提是每一步都能被自動驗證——這應是設計代理時的第一個檢核問題。

習題

一、觀念題

1. 請說明代理與單純語言模型的核心差異，並解釋為何「流程寫死的自動化」不算代理。
2. ReAct 為何要求模型把「思考」明確寫出來？請說明其對正確率與工程實務的兩方面好處。
3. 請說明「權限最小化」原則，並解釋為何它同時是安全考量與可靠度考量。
4. 自我反思有什麼根本限制？「審查式」多代理如何解決這個問題？

二、計算題

1. 某代理完成任務需 12 個步驟，實測單步正確率為 93%。（一）請以式 22-1 計算整體成功率；（二）若把流程重構、將步驟數降為 6 步，成功率變為多少？（三）比較兩者，說明「減少步驟」與「提升單步正確率」何者較有效。
2. 承上題（12 步、 $q = 93\%$ ），若為每一步加上驗證與重試機制。請以式 22-2 計算重試 1 次與 2 次時的整體成功率，並說明為何重試的邊際效益遞減。

三、思考與討論

1. 本章主張「能用固定流程解決的，不要用代理」。請討論：在什麼情況下，代理的彈性確實值得付出可靠度的代價？請以一個具體情境說明你的判準。

2. 案例三中，代理因權限過大而造成實際損害。請就一個你熟悉的應用情境，設計一套代理的權限分級與人工確認機制，並說明你如何決定哪些動作需要人工介入。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

核心差異在於「主動性」：單純語言模型是被動的一問一答，不與外界互動，僅能依訓練時的參數作答（故有知識過期、不知私有資料、會產生幻覺三大限制）；代理則能自行判斷需要什麼資訊、呼叫工具實際執行、並依觀察到的結果決定下一步，使模型從「憑記憶回答」轉為「查證後回答」。流程寫死的自動化不算代理，是因為判斷「下一步做什麼」的主體是撰寫流程的人，而非模型；此類需求用第 13 章的工作流編排器實作即可，且更可靠、更便宜。真正的代理，其行動序列是模型當場依情況產生的。

第 2 題

對正確率的好處：語言模型是自迴歸的，產生的每個詞元都成為後續生成的上下文；把推理過程寫出來，等於讓模型先建立中間結論再據此決定行動，而非一步跳到結果，故能顯著提升正確率（呼應第 21 章「一步一步思考」的提示技巧）。對工程實務的好處有二：其一，可除錯——代理出錯時能看到它當時的推理過程，而非面對黑箱；其二，可審核——高風險場景中，人可檢視推理鏈是否合理再決定是否放行。

第 3 題

權限最小化原則指：只授予代理完成任務所必需的最小權限，特別是具寫入或改變外界狀態能力的工具，應嚴格限制其範圍。作為安全考量：可防止代理被誘導（如惡意提示）而執行超出預期的破壞性操作。作為可靠度考量：代理犯錯是機率問題而非能力問題——依式 22-1，步驟一多就必然有出錯的機會；既然無法保證它不犯錯，就應限制它犯錯的後果。故設計時要問的不是「它會不會出錯」，而是「它出錯時會怎樣」（見案例三）。

第 4 題

根本限制：自我反思是用「同一個模型、同一套認知框架」來檢查自己的產出。若代理在一開始就誤解了任務需求，它的自我檢查也會基於同樣的誤解進行，因而總是「檢查通過」——它無法跳脫自己的盲點。審查式多代理的解法：由一個獨立的審查代理，僅根據「原始任務需求」與「最終產出結果」進行比對，不接觸執行過程與原代理的推理鏈，因而不會繼承其誤解。此思路與軟體工程的程式碼審查、學術界的同儕評審完全一致——檢查者必須獨立於執行者。

二、計算題

第 1 題

依式 22-1, $P = q^n$, $q = 0.93$ 。

- (1) $n = 12$: $P = 0.93^{12} \approx 0.4186$ (約 41.9%)。
- (2) $n = 6$: $P = 0.93^6 \approx 0.6470$ (約 64.7%)。

- (3) 比較：步驟數減半使成功率由 41.9% 提升至 64.7%（提升約 22.8 個百分點）。若改為提升單步正確率而維持 12 步，要達到 64.7% 需 $q = 0.647^{(1/12)} \approx 0.9646$ ，即單步正確率須從 93% 提升到約 96.5%。

結論：兩者皆有效，但「減少步驟」通常更容易達成——把可固定的環節改為寫死的管線是工程上可控的重構，而把模型的單步正確率從 93% 提升到 96.5% 往往極為困難（涉及模型能力本身）。這正是案例二採取的策略。

第 2 題

依式 22-2，單步最終正確率 $= 1 - (1 - q)^{(k+1)}$ ，其中 $1 - q = 0.07$ ， $n = 12$ 。

- (1) $k = 1$ ：單步 $= 1 - 0.07^2 = 1 - 0.0049 = 0.9951$ ；整體 $= 0.9951^{12} \approx 0.9428$ （約 94.3%）。
- (2) $k = 2$ ：單步 $= 1 - 0.07^3 = 1 - 0.000343 = 0.999657$ ；整體 $= 0.999657^{12} \approx 0.9959$ （約 99.6%）。

邊際效益遞減的原因：第一次重試把整體成功率由 41.9% 拉升到 94.3%（提升 52.4 個百分點，失敗率由 58.1% 降至 5.7%，約降十倍）；第二次重試僅由 94.3% 提升到 99.6%（提升 5.3 個百分點）。因為失敗率隨重試次數呈指數下降，當它已經很小時，再降低的絕對效益就很有限，卻仍要付出額外的模型呼叫成本與延遲。故實務上重試次數通常設 1 至 2 次即足（呼應第 13 章的相同結論）。

三、思考與討論

第 1 題

值得付出代價的判準包括：其一，任務的路徑無法事先窮舉——若使用者的需求千變萬化、無法預先寫成有限的流程分支（如開放式的資料探索、故障診斷），代理的彈性才有價值。其二，錯誤成本低且可回復——輸出是草稿、建議或唯讀查詢，人可審核後才採用。其三，有可靠的驗證手段——每步結果能被程式或人快速檢查，使式 22-2 的重試機制得以生效。其四，人力成本高於出錯成本——若原本需要專家花大量時間處理，即使代理成功率僅七成，仍能顯著節省人力。反之，若任務流程固定、錯誤不可逆、或無法驗證，就應選擇固定管線。具體情境舉例（如「工程師以自然語言探索歷史故障紀錄」）並套用上述判準說明即可。

第 2 題

答題應包含分級設計與判準。權限分級可設計為三層：第一層唯讀查詢（查資料、讀文件、執行計算）——代理可完全自主，因錯誤不改變外界狀態且可重做；第二層有限寫入（建立草稿、寫入測試環境、產生待審工單）——代理可自主但須留下紀錄且可回復；第三層不可逆或高影響動作（修改正式資料、寄送對外通知、控制實體設備、核准金額）——必須人工確認。決定何者需人工介入的判準：其一，可逆性（能否輕易還原）；其二，影響範圍（影響一筆或一萬筆）；其三，對外可見性（是否已送達客戶或改變實體世界）；其四，可驗證性（程式能否自動判斷結果正確）。此外應補充配套機制：單次操作的影響筆數上限、寫入前先於測試環境試跑、完整的操作稽核紀錄、以及隨時可中止的機制。可連結 22.6.3 節的人在迴路中原則。

大數據分析與雲端運算

第伍篇 生成式 AI 與開源生態

Generative AI & Open-Source Ecosystem

第 23 章

開源協作與模型生態系：
GitHub 與 Hugging Face
Open-Source Collaboration and Model Ecosystems

機器人叫獸（李世淵） 編著

叫獸開講

每年畢業季，我都會收到同一種求救訊息：「教授，學長的程式我跑不起來。」

我問怎麼跑不起來，答案永遠是那幾種：資料夾裡有 `final.py`、`final2.py`、`final_真的最後版.py`，不知道該跑哪一個；跑起來後說找不到某個套件，裝了之後又說版本不對；好不容易跑起來了，結果數字跟論文裡寫的對不上。而學長已經畢業，聯絡不上。

這不是學生不用功，而是他們從來沒有人教過「如何讓別人（包括半年後的自己）能重現你的工作」。而這件事，在真實的專案裡比寫程式本身更重要——一個跑不起來的成果，等於不存在。

這一章要談的，就是解決這個問題的整套基礎設施。GitHub 管理程式碼的版本與協作，Hugging Face 管理模型與資料集的分享與取用，而它們共同支撐的核心價值只有三個字：可重現。

我也想讓你看到一件事：現代 AI 之所以進展如此快速，很大程度不是因為某個天才的靈光一閃，而是因為這套開源基礎設施讓「站在別人肩膀上」變得極其容易。你今天可以在十分鐘內下載一個別人花了數百萬美元訓練的模型，這在十幾年前是不可想像的。理解並善用這個生態系，是這個時代工程師最划算的一項投資。

學習目標

研讀完本章之後，你應該能夠：

1. 說明開源與可重現性的價值，並列出可重現性的必要條件。
2. 運用 Git 進行版本控制，並說明分支與合併的協作流程。
3. 說明 GitHub 的專案管理功能，並以 Actions 建立自動化流程。
4. 在 Hugging Face Hub 上尋找、評估並取用模型與資料集。
5. 使用 Transformers 與 Datasets 函式庫載入模型並進行推論。
6. 說明模型卡與資料集卡的用途，並理解授權條款的重要性。

核心概念

核心概念	說明
版本控制	記錄檔案變更歷程、可回溯任一時點狀態的機制，代表工具為 Git。
分支與合併	在不影響主線的情況下平行開發，完成後再併回的協作方式。
持續整合	每次提交程式碼即自動執行測試與檢查，及早發現問題。
Hugging Face Hub	以社群為核心的模型、資料集與應用分享平台。
模型卡	記載模型用途、訓練資料、限制與風險的說明文件。
授權條款	規範他人能否使用、修改、商用該成果的法律條件。
可重現性	他人依照紀錄能得到相同結果的性質，是科學與工程的基礎。

表 23-1 本章核心概念一覽

23.1 開源與可重現性文化

23.1.1 可重現性的四個條件

承接叫獸開講，要讓一份工作能被重現，需要同時滿足四個條件。缺一個，別人就跑不起來。

條件	內容	常見的失敗
程式碼	明確的版本，而非「最後一版」	多個檔案不知該跑哪個
環境	套件與其版本、執行環境規格	版本不合而報錯
資料	確切的資料版本與取得方式	資料被覆蓋或找不到
參數與流程	執行的指令、隨機種子、超參數	數字對不上、無法重現

表 23-2 可重現性的四個必要條件

多數人只做到第一項（把程式碼交出去），因而註定失敗。本章介紹的工具，正是分別針對這四項而生：Git 管程式碼版本、容器與相依清單管環境（第 28 章會深入）、資料集平台與版本標記管資料、而設定檔與自動化流程管參數。

23.1.2 為何開源加速了 AI 的發展

開源的價值不只是「免費」，更在於它讓知識的傳遞成本趨近於零。

在封閉的模式下，一個突破要傳播出去，需要透過論文描述、他人重新實作、反覆試錯才能複製——往往耗時數月且結果未必一致。而在開源模式下，作者直接把程式碼與模型權重公開，別人幾小時內就能跑起來、並在此基礎上改進。

這造就了一種累積效應：每個人的起點都是前人的終點。你今天要做一個影像分類系統，不必從頭訓練——下載一個預訓練模型、用自己的資料微調，一個下午就能有不錯的成果。這在十幾年前需要一個團隊工作數月。

叫獸提醒

我要提醒一個容易被忽略的責任面。當你使用別人的開源成果時，你同時繼承了它的優點與缺點——包括你可能不知道的偏誤、限制與授權義務。我看過學生直接拿一個網路上的模型放進專題，卻不知道它的訓練資料有什麼問題、也不知道它的授權其實禁止商用。開源不是「免費隨便拿」，而是「公開且有條件地分享」。使用前務必看清楚模型卡與授權——這既是專業，也是對原作者的尊重。

23.2 Git 版本控制與協作流程

23.2.1 Git 在做什麼

Git 的核心功能是記錄專案在每個時間點的完整狀態，讓你能隨時回到任一版本、比較差異、並追溯每一行是誰在何時為何而改。

幾個基本觀念：

- **提交 (Commit)**：把當前的變更記錄為一個版本，附上說明訊息。一個好的提交應該是「一件完整的小事」，而非把三天的工作全部塞在一起。

- **儲存庫 (Repository)**：整個專案及其完整歷程。本地端有一份，遠端（如 GitHub）也有一份。
- **分支 (Branch)**：從主線分出來的平行開發路徑。你可以在分支上大膽嘗試，不影響主線的穩定。
- **合併 (Merge)**：把分支的變更併回主線。若同一處被兩邊各自修改，會產生「衝突」需人工處理。

你會發現「分支與合併」的概念，與第 7 章的多主複製有幾分神似——都是允許多處平行修改，最後再處理衝突。差別在於 Git 的衝突由人來裁決，而分散式資料庫必須自動解決。

23.2.2 基本的協作流程

團隊協作最常見的模式大致如下：

- 第 1 步。** 從主線建立一個新分支，命名應反映其目的（如 fix-sensor-parser）。
- 第 2 步。** 在該分支上進行開發，過程中做數次有意義的提交。
- 第 3 步。** 推送到遠端，並開啟「拉取請求」（Pull Request），請團隊成員檢視。
- 第 4 步。** 同儕審查：其他成員檢視變更、提出意見，作者依回饋修正。
- 第 5 步。** 通過審查與自動化測試後，合併回主線。

第 4 步的同儕審查值得特別強調。它的價值不只在於找出錯誤，更在於知識的傳遞——審查者會學到別人的做法，作者則被迫把程式碼寫得讓人看得懂。這與第 22 章「檢查者必須獨立於執行者」是同一個原則。

23.2.3 大檔案的問題

Git 的設計是為文字檔（程式碼）而生的——它能高效地記錄每一行的差異。但機器學習專案往往包含大型的模型權重與資料集，這對 Git 是個災難。

原因在於二進位檔案無法做差異比較，每改一次就要完整存一份。一個 500 MB 的模型檔改十次儲存庫就膨脹到 5 GB，且這些歷史永遠無法真正刪除。

解法是「大檔案儲存」（Git LFS）：儲存庫中只保存一個指向實際檔案的指標，真正的檔案存在專門的儲存空間。Hugging Face Hub 的模型儲存庫正是採用這個機制——這也是為何模型與程式碼會被分開管理。

23.3 GitHub 專案管理與 Actions

23.3.1 不只是放程式碼的地方

GitHub 在 Git 之上提供了一整套協作與專案管理的功能，其中對團隊最有價值的幾項：

- **議題 (Issues)**：追蹤待辦事項、臭蟲與功能需求。每個議題有討論串，能連結到解決它的程式碼變更。
- **拉取請求**：如前節所述的變更提案與審查機制，是團隊品質把關的核心。
- **專案看板**：把議題以看板方式視覺化管理進度。
- **版本發布**：為重要的里程碑打上標籤並附上說明，讓使用者知道該用哪一版。
- **說明文件**：README 是專案的門面——它應該讓陌生人在三分鐘內知道「這是什麼、怎麼跑起來」。

最後一項最常被忽略卻最重要。一個沒有 README 的專案，即使程式碼寫得再好，別人也用不起來——這正是叫獸開講中那些求救訊息的根源。

23.3.2 Actions：自動化的品質關卡

GitHub Actions 讓你能定義「當某事發生時，自動執行某些工作」。最常見的用途是持續整合 (CI) —— 每次有人推送程式碼或開啟拉取請求時，自動執行一系列檢查。

典型的檢查包括：安裝相依套件並確認能成功建置、執行自動化測試、檢查程式碼風格、以及在機器學習專案中，用固定的測試集跑一次模型評估，確認品質沒有退步。

這件事的價值在於「及早發現」。一個問題在提交當下被發現，作者記憶猶新、修正只要幾分鐘；但若拖到三個月後才被發現，追查與修正的成本可能是數十倍。這與第 13 章「把品質檢核內建為管線的關卡」是完全相同的思想。

延伸的用途稱為持續部署 (CD)：通過所有檢查後，自動把新版本部署到測試或正式環境。這是第 34 章 DataOps 與 MLOps 的核心機制之一。

23.4 Hugging Face Hub：模型與資料集

23.4.1 AI 領域的共享平台

如果說 GitHub 是程式碼的共享平台，Hugging Face Hub 就是模型與資料集的共享平台。它上面有數十萬個開源模型，涵蓋語言、視覺、語音等各種任務，多數可以直接下載使用。

平台的核心內容有三類：模型儲存庫（權重檔、設定與說明）、資料集儲存庫（訓練與評估用的資料）、以及 Spaces（可直接執行的示範應用，讓人不必安裝就能試用模型）。

對學習者而言，這個平台最大的價值是「降低門檻」——你不需要有訓練大模型的資源，就能站在別人的成果上做事。這正是 23.1.2 節談的累積效應在實務上的體現。

23.4.2 如何評估一個模型

面對數十萬個模型，如何挑選？以下是我建議的評估順序，這也是實務上的專業判斷。

- **任務是否相符**：先確認它是為你的任務訓練的。用文字分類模型去做問答，效果不會好。
- **語言是否支援**：許多模型以英文為主，中文表現可能極差。務必確認其訓練資料涵蓋你需要的語言。
- **規模是否跑得動**：依第 21 章的式 21-1 估算記憶體需求，先確認你的硬體負擔得起。
- **授權是否允許**：有些模型禁止商業使用，有些要求衍生作品也必須開源。這是法律問題，不能忽略。
- **模型卡是否完整**：說明文件的品質，往往反映了作者的嚴謹程度。沒有說明的模型，風險難以評估。
- **社群訊號**：下載量、按讚數與討論區的問題回覆，可作為可靠度的參考，但不應是唯一依據。

23.4.3 資料集也一樣重要

平台上的資料集同樣值得重視。它們讓你能快速取得標準的訓練或評估資料，尤其在建立基準線時特別有用——用公認的資料集評估，別人才知道你的成果好在哪裡。

但使用資料集時要格外注意兩件事：其一是授權與隱私，某些資料集可能含有個資或受版權保護的內容；其二是資料的偏誤，訓練資料中隱含的偏見會直接傳遞到模型的行為上。這也呼應了第 33 章要談的資料倫理。

23.5 Transformers 與 Datasets 函式庫

23.5.1 統一的介面

Transformers 函式庫的最大貢獻，是為數百種不同架構的模型提供了統一的使用介面。在此之前，每個模型的載入方式、輸入格式、輸出解讀都各不相同，換一個模型往往要重寫大量程式碼。

現在的做法是：指定模型名稱、載入對應的詞元化器與模型、輸入資料、取得輸出——不論底層是哪種架構，流程都相同。這使得「試三個不同的模型比較效果」從數天的工作變成幾行程式碼的事。

更高階的封裝稱為「管線」（pipeline），把詞元化、推論、後處理全部包起來，讓你以一行程式碼完成一個任務。這對快速驗證想法極為方便，但正式專案中通常仍會使用較底層的介面，以取得更多控制權。

23.5.2 Datasets 與資料處理

Datasets 函式庫則負責資料的載入與處理。它有兩個值得注意的設計。

其一是記憶體映射：資料不必全部載入記憶體，而是以磁碟映射的方式按需讀取。這讓你能在一般電腦上處理遠大於記憶體的資料集——這與第 5 章談的儲存策略、第 11 章的溢寫機制思路相通。

其二是快取機制：對資料的處理結果會被自動快取，重複執行時直接取用而不必重算。這在反覆調整實驗時能省下大量時間，也呼應了第 11 章 Spark 快取的相同動機。

23.5.3 微調的標準流程

結合第 21 章的知識，一次典型的微調流程如下：

- 第 1 步。** 載入預訓練模型與其對應的詞元化器。
- 第 2 步。** 準備資料集，並套用詞元化與格式轉換。
- 第 3 步。** 設定訓練參數（學習率、批次大小、訓練輪數等）。
- 第 4 步。** 選擇微調策略——完整微調或參數高效微調（如第 21 章談的 LoRA）。
- 第 5 步。** 執行訓練，並在驗證集上監控指標以避免過度擬合。
- 第 6 步。** 評估、保存模型，並撰寫模型卡記錄這一切。

注意最後一步——記錄與模型本身同樣重要。你在第 6 步做的紀錄，決定了三個月後的你能不能重現這次的結果。

23.6 模型卡、資料集卡與 Spaces

23.6.1 模型卡：誠實的說明書

模型卡（Model Card）是一份隨模型附上的說明文件，其理念是：發布一個模型時，不只要說它能做什麼，也要說它「不能做什麼、在什麼情況下會失敗」。

一份完整的模型卡通常包含：

項目	內容	為何重要
預期用途	設計來解決什麼任務、適用什麼場景	避免被用在不適合的地方
訓練資料	來源、規模、涵蓋範圍與時間	了解知識邊界與可能的偏誤
評估結果	在哪些基準上、表現如何	提供客觀的能力參考
限制與偏誤	已知的失效情境與偏見	這是最重要也最常被省略的一節
授權條款	使用、修改、商用的條件	法律責任所在

表 23-3 模型卡的核心內容

「限制與偏誤」這一節之所以最重要，是因為它是唯一能防止誤用的東西。一個模型若在訓練資料中很少見到某個族群、某種產品或某種語言，它在那些情況下的表現就不可靠——但若沒有人寫出來，使用者無從得知。

23.6.2 授權：不能忽略的法律面

開源不等於無條件免費。常見的授權類型有幾種取向：有些極為寬鬆，幾乎允許任何用途；有些要求衍生作品也必須以相同條款開源；有些則明確限制商業使用或特定用途。

實務上必須注意兩件事。其一，模型與資料集可能有不同的授權——用了 A 授權的模型在 B 授權的資料上微調，衍生模型的授權需同時滿足兩者。其二，授權條款會影響你的產品能否商用，這在專題或創業前務必確認。

我的建議很直接：把「確認授權」當成使用任何開源成果的第一個步驟，而不是最後一個。等到系統做完才發現不能用，代價是慘痛的。

23.6.3 Spaces：讓成果被看見

Spaces 讓你能把模型包裝成一個網頁應用並公開分享，讓別人不必安裝任何東西就能試用。

對學習者而言，這有幾個實際的價值：專題成果可以做成一個可互動的展示，遠比一份報告有說服力；求職或申請研究所時，一個能實際操作的作品集比履歷上的文字更具體；而在課堂上，它也讓同學能快速互相試用彼此的成果。

這也體現了開源文化的一個面向：分享不只是給予，也是讓自己的工作被看見、被檢驗、被改進的方式。

23.7 雲科大 AI Console

23.7.1 校內的實作平台

前面幾節談的都是全球性的公開平台。但在教學現場，我們還需要一個更貼近課程需求的環境——這就是雲科大 AI Console 的定位。

它要解決的問題很實際。第一，運算資源：多數同學的個人電腦跑不動較大的模型（回想第 21 章的記憶體估算），需要集中的 GPU 資源共享。第二，環境統一：課程實作若每個人的環境都不同，除錯會耗掉大部分的上課時間。第三，資料治理：課程或產學專案的資料往往不宜上傳到公開平台。

換言之，它是把本章與第 10 章（資源管理）、第 28 章（容器）的概念，落實到校內教學場景的一套整合環境。

23.7.2 與公開平台的關係

要特別說明的是，校內平台與公開平台不是取代關係，而是分工關係——這與第 21 章談的雲端與地端取捨完全一致。

面向	公開平台	校內平台
模型與資料來源	取用全球社群的開源成果	存放課程與專案的私有資料
運算資源	自備或另行租用	校內集中共享的 GPU
環境一致性	各自安裝，容易分歧	統一映像，減少環境問題
適合用途	取材、研究、成果公開展示	課程實作、敏感資料處理

表 23-4 公開平台與校內平台的分工

典型的使用方式是兩者搭配：從 Hugging Face 取得預訓練模型與公開資料集，在校內平台上以自有資料進行微調與實驗，最後把不涉及機密的成果整理後，回饋到公開平台分享。

23.7.3 使用時的專業習慣

無論在哪個平台上工作，有幾項習慣我希望你們養成，它們會跟著你一輩子。

- **一律使用版本控制：**即使是個人的小專案。半年後的你會感謝現在的你。
- **記錄環境與參數：**每次實驗都記下套件版本、超參數與隨機種子——這是可重現性的四個條件之一。
- **寫好 README：**假設讀者是完全不認識你的人，讓他能在三分鐘內跑起來。
- **共享資源要自律：**GPU 是共用的，用完釋放、不要長期佔用閒置的資源（這正是第 10 章多租戶治理的日常實踐）。
- **注意資料分際：**清楚知道哪些資料能上傳到哪裡。一時方便可能造成無法挽回的外洩。

叫獸提醒

我要特別強調「共享資源要自律」這一條，因為它不只是技術問題，也是專業素養。一個人佔著八張 GPU 卻只在跑一個小實驗，其他二十位同學就得等——這在第 10 章我們用「吵鬧的鄰居」來描述它的技術面，但在人的層面，這叫做缺乏公德。你將來進入業界，共用叢集的情況只會更多，而一個懂得自律的工程師，會比技術更強但難以共事的人走得更遠。這句話我每學期都要講一次。

23.7.4 承先啟後

本章我們談了讓工作能被重現與分享的整套基礎設施：可重現性的四個條件（23.1）、Git 的版本控制與協作流程（23.2）、GitHub 的專案管理與自動化品質關卡（23.3）、Hugging Face 的模型與資料集生態（23.4）、Transformers 與 Datasets 的統一介面（23.5）、模型卡與授權的責任面（23.6），以及校內平台的定位與專業習慣（23.7）。

到這裡，第伍篇「生成式 AI 與開源生態」三章完整收束。回顧這一篇：第 21 章讓你理解大語言模型是什麼、能做什麼、又會在這裡失敗；第 22 章讓模型從「會說話」變成「會做事」，同時也看清了可靠度的代價；而本章則說明了這一切之所以能快速發展的基礎——一個讓知識得以累積的開源生態。

接下來我們要轉一個大彎。前面幾篇處理的資料，本質上都是「一批一批」的——我們等資料到齊了再開始分析。但真實世界的資料其實從不停歇：產線的感測器此刻仍在產生訊號、交易此刻仍在發生。如果我們需要的不是「昨天的報表」，而是「此刻的異常告警」呢？當資料像水流一樣永不停止，批次的思維就不夠用了。這就是第陸篇的主題——串流處理與即時分析。

公式與流程：可重現性的機率與模型儲存成本

這一節我們把兩件常被當成「軟性議題」的事，變成可以計算的數字。

可重現性是連乘，不是相加

表 23-2 列出了可重現性的四個條件。關鍵在於，它們是「同時成立」才有效——只要一項失敗，整份工作就無法重現。設第 i 項條件被妥善記錄的機率為 p_i ，則整份工作可被重現的機率為：

$$P(\text{可重現}) = p_1 \times p_2 \times p_3 \times p_4$$

式 23-1 可重現性的連乘關係

這條式子與第 13 章的管線可靠度、第 22 章的代理成功率是同一個數學結構——連乘對任一環節的疏漏都極為敏感。下表以四種情境試算。

情境	程式碼	環境	資料	可重現機率
完全不管理	0.5	0.3	0.4	6.0%
只用版本控制	0.95	0.3	0.4	11.4%
加上環境紀錄	0.95	0.9	0.4	34.2%
四項皆落實	0.95	0.9	0.9	77.0%

表 23-5 各項條件對可重現機率的影響（第四項參數紀錄固定為 1.0）

看第二列：只把程式碼放上版本控制（多數人做到的程度），可重現機率仍只有約 11%。這解釋了叫獸開講中那個現象——學生確實把程式碼交出來了，但仍然跑不起來。因為程式碼只是四個條件之一。

模型儲存的成本估算

另一個實務問題是：反覆微調產生的模型檔該不該全部保留？設每個模型檔大小為 S （GB）、保留 k 個版本，則儲存需求為：

$$\text{儲存需求} = S \times k$$

式 23-2 多版本模型的儲存需求

以第 21 章的估算為基礎，一個 7B 的 FP16 模型約 14 GB。若一次實驗保留 20 個檢查點，就需 280 GB；若三位學生各做五組實驗，總量將達 4.2 TB。

因應之道有三：其一，只保留關鍵檢查點（最佳與最終），其餘刪除；其二，若採 LoRA 等參數高效微調（第 21 章），只需保存數十 MB 的適配權重而非完整模型，儲存量可降至千分之一以下；其三，使用 Git LFS 或專門的模型儲存庫，而非把大檔案塞進一般的版本控制系統。

叫獸提醒

表 23-5 是我每年開學都會秀給學生看的一張表。多數人以為「我有把程式碼備份」就等於做

好了管理，但數字告訴你：那只是把可重現機率從 6% 拉到 11%，仍然是十次有九次跑不起來。真正的關鍵在於「環境」與「資料」這兩項——而它們恰恰是最容易被忽略的，因為在你自己的電腦上，環境是現成的、資料就在那裡，你根本感覺不到它們的存在。可重現性的困難就在這裡：你必須為「一個沒有你的環境的人」著想。這需要刻意的練習，而現在正是開始練習的最好時機。

案例研討

案例一 跑不起來的學長專題

回到叫獸開講。那些求救訊息的共通點是：程式碼有交接，但其他三項條件全部缺失——沒有相依套件的版本清單、沒有記錄用的是哪一版資料、沒有寫下執行的參數與隨機種子。

依式 23-1 估算，這種情況的可重現機率約在 6% 到 11% 之間，也就是十次有九次會失敗——而我們的實際經驗完全吻合這個數字。

我後來要求所有專題必須交出四樣東西：一個 Git 儲存庫（含完整提交歷程）、一份相依套件清單或容器定義檔、資料的取得方式與版本標記、以及一份能讓陌生人跑起來的 README。導入之後，學弟妹接手的成功率大幅提升。這個案例說明：可重現性不是靠交接時的口頭說明，而是靠制度化的紀錄。

案例二 半夜被擋下的錯誤

某團隊在專案中導入了 GitHub Actions，設定每次拉取請求都自動執行測試與模型評估。某位成員在深夜提交了一項修改，自認只是小調整而未仔細測試。

自動化流程在數分鐘內回報失敗——模型在固定測試集上的準確率從 0.91 掉到 0.62。追查後發現，該修改意外改動了資料前處理的順序，導致特徵標準化的參數計算錯誤（正是第 20 章 20.3.1 節警告過的那類錯誤）。

若沒有這道自動化關卡，這個問題可能要到數週後模型上線表現異常時才會被發現，屆時追查的成本將是數十倍。這個案例印證了 23.3.2 節的核心價值：自動化檢查的意義不在於它多聰明，而在於它「每次都會執行、且立刻回報」。

案例三 不能商用的模型

某組同學的專題做得相當出色，並打算參加創業競賽。在最後階段，指導老師詢問他們所用模型的授權條款——他們才發現，該模型的授權明確禁止商業用途。

此時系統已經完成，若要更換模型，等於重做大部分的微調與驗證工作。他們最終改用另一個授權寬鬆的替代模型，並額外花了兩週重新訓練與調校。

這正是 23.6.2 節那句建議的反面教材——把「確認授權」放在第一步而非最後一步。這個成本不是技術問題，而是流程問題：只要在選型時多花五分鐘查看模型卡，就能完全避免。

案例四 佔著 GPU 的那個人

某學期期末，實驗室的共用 GPU 資源持續滿載，多位同學無法執行實驗。查看後發現，其中一位同學在兩週前啟動了一個工作階段做測試，之後就一直沒有釋放——實際上該工作階段大部分時間都處於閒置狀態。

這與第 10 章案例三的「吵鬧的鄰居」在技術上同源，但性質不同：那是資源隔離的技術問題，這是使用習慣的素養問題。技術上可以用配額與逾時自動回收來緩解，但根本的解方仍在於使用者的自覺。

實驗室後來訂立了明確規範：閒置超過一定時間自動釋放、長時間作業須事先登記、並公開顯示各人的資源使用狀況。制度加上透明度，情況隨即改善。這個案例提醒我們：多租戶環境的治理，技術手段與行為規範必須並行。

實作活動

活動一 讓你的專案可被重現

請針對你手上任一個既有的程式專案，依表 23-2 進行改造。

- 步驟 1.** 建立 Git 儲存庫，把專案納入版本控制，並做出至少五個有意義的提交。
- 步驟 2.** 產生相依套件清單（記錄套件名稱與確切版本），並說明執行所需的環境規格。
- 步驟 3.** 記錄資料的來源、版本與取得方式；若資料不便公開，說明如何取得替代資料。
- 步驟 4.** 撰寫 README，包含專案目的、環境安裝步驟、執行指令與預期輸出。完成後請一位同學依你的 README 實際跑一次，記錄他遇到的每一個障礙。

活動二 取用並評估一個開源模型

請從 Hugging Face Hub 挑選並實際使用一個模型。

- 步驟 1.** 選定一個與你興趣相關的任務，搜尋至少三個候選模型。
- 步驟 2.** 依 23.4.2 節的六項判準逐一評估這三個模型，製作一張比較表（含以式 21-1 估算的記憶體需求與授權條款）。
- 步驟 3.** 選定其中一個，以 Transformers 函式庫載入並在自己準備的十筆資料上進行推論。
- 步驟 4.** 記錄實際表現，並檢視模型卡中的「限制與偏誤」一節，說明你觀察到的結果是否與其描述相符。

活動三 建立自動化的品質關卡

請為活動一的專案加上 GitHub Actions。設定一個工作流程，使其在每次推送時自動執行：安裝相依套件、執行至少一項測試（可以是簡單的「程式能否成功執行」檢查）、以及檢查關鍵輸出是否落在合理範圍。完成後，刻意提交一個會導致失敗的變更，確認自動化流程確實攔截了它。最後撰寫一段反思：這道關卡能攔截哪些類型的錯誤？又有哪些類型的錯誤是它攔不住的？

延伸閱讀

- **Git 官方文件與 Pro Git 一書：**系統性介紹版本控制的觀念與操作，是 23.2 節的完整參考，且有免費的中文版本。
- **模型卡的原始論文：**Mitchell 等人發表的《Model Cards for Model Reporting》，闡述為何模型應附上誠實的限制說明，是 23.6.1 節的思想源頭。

- **Hugging Face 官方課程：**免費的線上課程，涵蓋 Transformers 與 Datasets 函式庫的實作，是 23.5 節的最佳延伸。
- **開源授權條款的比較說明：**各主流授權條款的權利義務對照，是 23.6.2 節的實務參考，選型前務必查閱。

本章小結

1. 可重現性需同時滿足程式碼、環境、資料、參數與流程四個條件；多數人只做到第一項，故註定失敗。這四項是連乘關係（式 23-1），任一疏漏都會使整份工作無法重現。
2. 開源的價值不只是免費，更在於使知識傳遞成本趨近於零，造就「每個人的起點都是前人的終點」的累積效應——這是現代 AI 快速發展的重要基礎。
3. Git 以提交記錄版本、以分支支援平行開發、以合併整合成果；團隊協作的標準流程為分支開發、拉取請求、同儕審查、通過測試後合併，其中審查的價值不只在找錯，更在知識傳遞。
4. Git 不適合大型二進位檔案，因無法做差異比較而使儲存庫膨脹；解法為 Git LFS，這也是模型與程式碼分開管理的原因。
5. GitHub Actions 提供自動化的品質關卡，其價值在於「每次都會執行、且立刻回報」——問題在提交當下被發現的修正成本，遠低於數週後才發現。
6. Hugging Face Hub 是模型與資料集的共享平台；評估模型應依序檢視任務相符、語言支援、規模可行、授權允許、模型卡完整與社群訊號六項判準。
7. 模型卡應誠實記載預期用途、訓練資料、評估結果、限制與偏誤及授權；其中「限制與偏誤」最重要卻最常被省略，因為它是唯一能防止誤用的資訊。授權應在選型的第一步確認，而非最後一步。
8. 校內平台與公開平台是分工而非取代關係：從公開平台取材，在校內平台以私有資料實驗，成果整理後再回饋分享。使用共享資源時，自律與資料分際是必備的專業素養。

習題

一、觀念題

1. 請列出可重現性的四個必要條件，並說明為何「只把程式碼交出去」通常不足以讓他人重現你的工作。
2. 為什麼 Git 不適合直接管理模型權重等大型二進位檔案？業界如何解決這個問題？
3. 請說明 GitHub Actions 這類自動化檢查的核心價值，並以「發現問題的時間點」說明其效益。
4. 模型卡中的「限制與偏誤」一節為何最重要？若一個模型沒有提供這節資訊，會有什麼風險？

二、計算題

1. 某研究團隊的四項可重現條件被妥善記錄的機率分別為：程式碼 0.9、環境 0.5、資料 0.6、參數 0.7。（一）請以式 23-1 計算整體可重現機率；（二）若只改善環境一項至 0.95，整體機率變為多少？（三）若四項皆提升至 0.9，又是多少？
2. 某課程有 25 位學生，每人需微調一個 7B 模型（FP16 約 14 GB）。（一）若每人保留 10 個檢查點，依式 23-2 計算總儲存需求；（二）若改用 LoRA，每個適配權重約 40 MB，總需求變為多少？（三）計算節省的倍數。

三、思考與討論

1. 案例三中，團隊因未及早確認授權而付出重做的代價。請討論：在專案的哪些階段應檢視哪些法律與倫理事項？請提出一份你會實際使用的檢核清單。

2. 本章指出「共享資源要自律」既是技術問題也是素養問題。請討論：在多人共用運算資源的環境中，技術手段（配額、逾時回收）與行為規範各能解決什麼、又各有什麼侷限？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

四個條件：程式碼（明確的版本而非「最後一版」）、環境（套件與其版本、執行環境規格）、資料（確切的資料版本與取得方式）、參數與流程（執行指令、隨機種子、超參數）。只交程式碼不足的原因：這四項是連乘關係（式 23-1），任一項缺失都會導致無法重現；且環境與資料恰恰最容易被忽略——因為在作者自己的電腦上，環境是現成的、資料就在那裡，作者根本感覺不到它們的存在。依表 23-5，只做好程式碼管理時，整體可重現機率仍僅約 11%，與實務觀察到的「十次有九次跑不起來」完全吻合。

第 2 題

因為 Git 的設計是為文字檔而生——它能高效記錄每一行的差異；但二進位檔案無法做差異比較，每修改一次就必須完整儲存一份新的副本。故一個 500 MB 的模型檔改十次，儲存庫就膨脹至 5 GB，且這些歷史記錄永久保存、無法真正刪除，導致複製（clone）該儲存庫變得極為緩慢。解法是 Git LFS（大檔案儲存）：儲存庫中只保存一個指向實際檔案的指標，真正的檔案存放於專門的儲存空間。Hugging Face Hub 的模型儲存庫即採用此機制，這也是模型與程式碼分開管理的原因。

第 3 題

核心價值不在於檢查有多聰明，而在於它「每次都會執行、且立刻回報」——人會遺忘、會偷懶、會自認為只是小修改而略過測試，但自動化流程不會。以發現問題的時間點說明效益：問題在提交當下被發現時，作者對這段修改記憶猶新，修正往往只需數分鐘；若拖到數週或數月後（例如模型上線表現異常時）才被發現，則須先追查是哪次變更造成、重建當時的情境、再理解當初的意圖，成本可能是數十倍（見案例二）。此與第 13 章「把品質檢核內建為管線關卡」是相同思想。

第 4 題

因為它是唯一能防止誤用的資訊。模型在訓練資料中很少見到的族群、產品類型或語言，其表現必然不可靠；但使用者無法從模型本身看出這些邊界，只能仰賴作者誠實揭露。缺少此節的風險包括：其一，被用在不適合的場景而產生錯誤結果，且因輸出流暢可信而不易察覺（呼應第 21 章的幻覺）；其二，隱含的偏誤直接傳遞到下游應用，可能造成對特定族群的不公平；其三，使用者無從評估風險，也無法設計適當的防護與驗證機制。故評估模型時，「模型卡是否完整」本身就是可靠度的重要訊號。

二、計算題

第 1 題

依式 23-1, $P = p_1 \times p_2 \times p_3 \times p_4$ 。

- (1) 原始情況： $0.9 \times 0.5 \times 0.6 \times 0.7 = 0.189$ （約 18.9%）。
- (2) 僅改善環境至 0.95： $0.9 \times 0.95 \times 0.6 \times 0.7 = 0.3591$ （約 35.9%）。

(3) 四項皆為 0.9: $0.9^4 = 0.6561$ (約 65.6%)。

延伸思考：只改善單一項（環境由 0.5 提升至 0.95），整體機率就從 18.9% 幾乎倍增至 35.9%——這說明應優先改善「最弱的一環」。連乘結構下，最低的那一項對整體的拖累最大，故投資報酬率最高的做法是找出並補強短板，而非在已經很好的項目上追求完美。

第 2 題

25 位學生、每人 10 個檢查點。

(1) 完整模型： $14 \text{ GB} \times 10 \times 25 = 3,500 \text{ GB} = 3.5 \text{ TB}$ 。

(2) 改用 LoRA： $40 \text{ MB} \times 10 \times 25 = 10,000 \text{ MB} = 10 \text{ GB}$ 。

(3) 節省倍數： $3,500 \text{ GB} \div 10 \text{ GB} = 350$ 倍。

延伸思考：3.5 TB 對一門課程而言是難以負擔的儲存量，而 10 GB 則完全可行。這說明第 21 章的參數高效微調（LoRA）不只降低了「訓練時的記憶體需求」，也大幅降低了「保存實驗成果的儲存成本」——這使得「每次實驗都完整保存」從奢侈變成常態，反過來又提升了可重現性。技術選擇與工程實務往往如此相互影響。

三、思考與討論

第 1 題

答題應依專案階段組織。選型階段（最關鍵）：確認模型與資料集的授權是否允許你的用途（研究、教學、商用）、是否要求衍生作品開源、是否有使用範圍限制；檢視模型卡的訓練資料來源是否合法、是否含個資。開發階段：確認自有資料的蒐集是否取得同意、是否涉及個資（第 33 章）、是否可上傳至所使用的平台；記錄所有第三方元件及其授權。發布階段：確認衍生成果的授權需同時滿足所有上游元件的要求；撰寫自己的模型卡誠實揭露限制與偏誤；若對外提供服務，評估錯誤輸出可能造成的責任。檢核清單應至少包含：上游元件清單與各自授權、資料來源與同意狀態、預期用途是否在授權範圍內、衍生授權的相容性、以及個資與機密的分際。核心原則是「把確認授權放在第一步」——如案例三所示，最後才發現的代價極高。

第 2 題

技術手段能解決的：以配額限制單一使用者可佔用的資源上限，防止極端獨佔；以逾時自動回收釋放閒置資源，處理「忘記關閉」這類無心之過；以優先權與排程策略（第 10 章）確保緊急或重要工作能取得資源；以透明的使用狀況顯示，讓佔用行為可被看見。技術手段的侷限：無法判斷「這個佔用是否合理」——一個真正需要長時間訓練的作業與一個閒置的工作階段，在系統看來可能相似；過嚴的配額會妨礙正當的大型實驗；規則總有漏洞可鑽（如定期送出無意義指令避免被判定閒置）。行為規範能解決的：處理技術難以判定的灰色地帶、建立同儕之間的相互尊重與問責、以及培養長期的專業素養。其侷限：仰賴自覺，對不自律者無強制力；且在人數眾多或匿名的環境中效果遞減。結論：兩者必須並行——技術提供底線與透明度，規範處理灰色地帶並塑造文化。這也適用於任何多租戶或共享資源的環境。

第陸篇 串流處理與即時分析

Stream Processing & Real-time Analytics

第 24 章

資料串流模型與近似演算法

Data Stream Models and Approximation Algorithms

機器人叫獸（李世淵） 編著

叫獸開講

我先給你出一道題，看起來很簡單，但你會發現它出乎意料地難。

假設有一條資料流，每秒鐘湧入十萬筆訪客紀錄，永不停止。現在請你回答：「到目前為止，總共有多少個『不重複』的訪客？」

直覺的做法是拿一個集合，把看過的訪客編號都記下來，最後數一數集合的大小。這完全正確——但如果訪客有十億個不重複的編號呢？這個集合會吃掉幾十 GB 的記憶體，而且它只會越來越大，永遠不會停。你的機器遲早會倒下。

這就是串流處理的根本困境：資料是無限的，記憶體是有限的。你不能把資料全部存下來，甚至不能掃第二遍——每一筆資料流過去，你就只有那一瞬間可以處理它，然後它就消失了。

那怎麼辦？這一章要給你的答案，是本書我個人覺得最漂亮的一組演算法。它們的共同精神是：放棄精確，換取可行。用幾 KB 的記憶體，估算出十億個不重複元素的數量，誤差只有百分之幾——這聽起來像魔術，但它背後是紮實的機率論。

而且我要提醒你，這個「以可控的近似換取可行的計算」的思想，你在第 14 章的 MinHash、第 16 章的 BFR、第 20 章的直方圖法都已經遇過了。這一章，是這個思想的集大成。

學習目標

研讀完本章之後，你應該能夠：

1. 說明串流計算模型的限制，並指出它與批次處理的根本差異。
2. 說明水塘抽樣如何在不知總量的情況下維持均勻樣本。
3. 運用 Bloom Filter 進行成員查詢，並計算其偽陽性率與所需空間。
4. 說明 Flajolet-Martin 演算法如何以極小空間估計相異元素數。
5. 說明 DGIM 演算法如何在滑動視窗上計數。
6. 說明 AMS 草圖的用途，並理解近似演算法的共同設計思想。

核心概念

核心概念	說明
串流計算模型	資料連續無限地到達，只能單次通過且記憶體有限的運算模型。
水塘抽樣	在總數未知的串流中，維持固定大小且均勻分布之樣本的方法。
Bloom Filter	以位元陣列進行成員查詢的機率式結構，只有偽陽性、無偽陰性。
Flajolet-Martin	以雜湊值尾端零的個數，估計相異元素數量的演算法。
滑動視窗	只關注最近 N 筆資料的查詢範圍。
DGIM	以分段桶結構在滑動視窗上近似計數的演算法。
AMS 草圖	估計串流動差（如頻率平方和）的隨機化方法。

表 24-1 本章核心概念一覽

24.1 串流計算模型

24.1.1 三個殘酷的限制

串流處理與前面各篇的批次處理，有著根本的差異。批次處理面對的是「有界資料」（第 2 章）——資料已經完整躺在那裡，你可以反覆掃描、可以排序、可以隨機存取。串流則不同，它有三個限制。

- **單次通過：**每筆資料只會經過你一次。錯過就沒了，不能回頭重看——因為它根本沒有被儲存。
- **記憶體受限：**資料是無限的，但記憶體是有限的。任何「隨資料量成長」的儲存策略最終都會失敗。
- **即時回應：**查詢可能在任何時刻到來，且要求立即回答。你不能說「等我先把資料整理一下」。

這三個限制合起來，就宣告了「精確答案」在多數情況下的死刑。因為許多看似簡單的問題——「有幾個不重複的元素」「某個元素出現過嗎」——若要精確回答，必然需要與資料量成正比的記憶體。

24.1.2 換個問法：近似就好

既然精確不可行，就改變問題。串流演算法的核心思想是：與其花無限的記憶體得到精確答案，不如用固定的記憶體得到「誤差有數學保證」的近似答案。

注意「有數學保證」這幾個字，這是關鍵。這些演算法不是隨便猜，而是能明確告訴你：「在 95% 的情況下，我的答案與真值的誤差不超過 2%」。這種可量化的不確定性，才能讓工程師據以做決策。

實務上，這個交換往往極為划算。以叫獸開講的例子而言，若你想知道網站的不重複訪客數，「一億兩千萬，誤差 $\pm 2\%$ 」與「精確的 121,384,772」在決策上幾乎沒有差別——但前者只需幾 KB 記憶體，後者需要數十 GB。

24.1.3 串流查詢的兩種型態

串流上的查詢大致分為兩類，它們的難度差異很大。

第一類是「整個串流」的查詢：從開始到現在的所有資料。例如「至今有多少不重複訪客」。這類查詢的好處是資料只增不減，狀態可以持續累積。

第二類是「滑動視窗」的查詢：只看最近 N 筆或最近一小時。例如「過去十分鐘的異常次數」。這類困難得多——因為舊資料會「過期」，你必須知道該扣掉什麼，但你又沒有把舊資料存下來。24.5 節的 DGIM 演算法就是為此而生。

叫獸提醒

我要在這裡先建立一個心理準備。這一章的演算法看起來會像魔術——用幾 KB 記憶體估算十億個元素？聽起來不可能。但請你留意它們共同的套路：先用雜湊把資料變成「隨機」的形式，然後觀察這些隨機值的某種統計性質，再從這個性質反推原始資料的規模。這個「觀察隨機性的痕跡」的思路，你在第 14 章的 MinHash 已經見過一次了。當你看懂這個共同骨架，這些演算法就不再是魔術，而是同一個思想的不同變奏。

24.2 串流抽樣

24.2.1 一個看似簡單的難題

最直接的近似方法是抽樣——既然不能全部處理，那就取一部分來代表整體。但在串流上，這件事比想像中困難。

難處在於：你不知道串流總共會有多少筆資料。若要「隨機抽 1000 筆」，傳統做法是先知道總數 N ，再決定每筆的抽中機率。但串流沒有總數——它可能是一百萬筆，也可能是一兆筆，而且此刻仍在增加。

更麻煩的是，你必須隨時都能給出一個「當下的均勻樣本」。也就是說，不管在哪個時間點被問，你手上的樣本都必須是「從至今所有資料中均勻抽出的」。

24.2.2 水塘抽樣

水塘抽樣（reservoir sampling）優雅地解決了這個問題。假設我們要維持大小為 k 的樣本，做法如下：

- 第 1 步。** 前 k 筆資料直接全部放入樣本（水塘）。
- 第 2 步。** 對於第 i 筆資料（ $i > k$ ），以 k/i 的機率決定是否納入樣本。
- 第 3 步。** 若決定納入，則從現有的 k 個樣本中隨機挑一個替換掉。

這個做法保證了一個漂亮的性質：在任何時刻，每一筆已到達的資料被保留在樣本中的機率都恰好是 k/i （其中 i 為目前總筆數）——換言之，樣本永遠是均勻的。

直覺上為什麼成立？因為新資料被納入的機率隨 i 增大而遞減（越後面的資料越不容易被選中），而舊資料被替換掉的機率也隨之遞減——這兩個效應恰好抵消，使每一筆的存活機率始終相等。

水塘抽樣的價值在於：它只需 k 個儲存空間，與串流長度完全無關。無論串流有一百萬筆還是一兆筆，記憶體用量都不變——這正是串流演算法追求的性質。

24.3 Bloom Filter

24.3.1 問題：這個元素出現過嗎

Bloom Filter 解決的問題是「成員查詢」：判斷某個元素是否曾在串流中出現過。這在實務上極為常見——這個網址是否已經爬過？這封郵件的寄件人是否在黑名單上？這個交易編號是否重複？

精確的做法是用一個集合把所有看過的元素存起來。但若元素有十億個、每個 20 位元組，就需要 20 GB 記憶體。Bloom Filter 能把這件事壓縮到幾百 MB 甚至更少。

24.3.2 運作方式

Bloom Filter 由一個長度為 m 的位元陣列（初始全為 0）與 k 個獨立的雜湊函式構成。

插入元素時：用 k 個雜湊函式分別計算該元素的雜湊值，把對應的 k 個位置設為 1。

查詢元素時：同樣計算 k 個位置，檢查它們是否「全部」為 1。若有任何一個是 0，則該元素「必定沒出現過」；若全部都是 1，則該元素「可能出現過」。

注意這個不對稱性——這是 Bloom Filter 最重要的性質。它「絕不會漏報」（沒有偽陰性），但「可能誤報」（有偽陽性）。誤報發生的原因是：那 k 個位置可能是被其他不同的元素分別設為 1 的，湊巧全部撞在一起。

24.3.3 為何這個不對稱性如此有用

初學者可能覺得「會誤報」是個缺陷，但實務上這個不對稱恰好是它的價值所在。

關鍵在於：許多應用中，偽陽性的代價遠低於偽陰性。以網頁爬蟲為例，若 Bloom Filter 誤報「這個網址爬過了」，代價只是漏爬一個網頁；但如果會漏報（把爬過的說成沒爬過），就會造成無限迴圈。以資料庫查詢為例，Bloom Filter 常被用來「快速排除不存在的鍵」——若它說「沒有」，就能直接跳過昂貴的磁碟查詢；若它說「可能有」，再去實際查一次即可，誤報只是多查一次。

這個「以低成本的快速篩選，配合精確的驗證」的兩階段設計，你應該覺得眼熟——第 14 章的 LSH（先篩候選再驗證）、第 15 章的 PCY（以桶計數預先剪枝）都是同樣的模式。事實上，PCY 那個「若桶的總計數未達門檻，則桶內所有元素必未達門檻」的推論，與 Bloom Filter 的「若有位置為 0，則必定沒出現過」在邏輯結構上完全一致。

24.4 相異元素計數：Flajolet-Martin

24.4.1 回到叫獸開講的難題

現在來解決本章開頭那個問題：如何在有限記憶體下，估計串流中有多少個不重複的元素？

Flajolet-Martin 演算法的想法極為巧妙，它建立在一個簡單的觀察上：把元素雜湊成二進位數字後，這些數字的「尾端有幾個連續的零」是有規律的。

具體而言，若雜湊值是均勻隨機的，那麼：約有一半的數字尾端至少有 1 個零、約四分之一至少有 2 個零、約八分之一至少有 3 個零……以此類推，「尾端至少有 r 個零」的機率是 2 的負 r 次方。

24.4.2 從稀有事件反推規模

關鍵的反推來了。如果你在串流中觀察到「尾端有 10 個零」的雜湊值，這代表什麼？

因為出現這種值的機率只有 $1/1024$ ，所以要「碰巧」看到它，你大概需要看過約 1024 個不同的元素。換言之——觀察到的最大尾零數 R ，可以用來估計相異元素數：

$$\text{估計的相異元素數} \approx 2^R$$

式 24-1 Flajolet-Martin 的基本估計式

這就是全部的核心思想。而它需要的記憶體是多少？只需要記住「目前為止看到的最大尾零數 R 」——一個小小的整數。用幾個位元，估計十億個元素的數量。

特別注意：重複出現的元素不會影響結果。因為同一個元素的雜湊值永遠相同，它的尾零數也相同，不會改變 R 。這正是它能「自動去重」的原因——它根本不需要記住看過誰。

24.4.3 降低變異：多組取中位數

式 24-1 的單一估計相當不穩定——因為 R 只要多一，估計值就翻倍。若運氣不好碰到一個尾零特別多的值，估計就會嚴重高估。

標準的改良做法是：使用多組獨立的雜湊函式，各自得到一個估計值，再取這些估計的中位數（或先分組取平均、再取各組的中位數）。

為什麼用中位數而非平均？因為這些估計值是 2 的次方，分布極度偏斜——一個異常大的估計值會嚴重拉高平均，但對中位數幾乎沒有影響。這個「以中位數對抗偏斜分布」的技巧，在統計上很常見，值得記住。

實務上使用的是這個思想的進階版本（如 HyperLogLog），它以更精細的分組與修正，能用約 1.5 KB 的記憶體，達到約 2% 的誤差來估計數十億個相異元素。這是串流演算法最著名的成功案例之一。

24.5 滑動視窗計數：DGIM

24.5.1 過期的難題

現在來看更困難的問題。假設有一條位元串流（每筆是 0 或 1），我們想知道「最近 N 筆之中，有幾個 1」。

困難在於「過期」。精確做法是把最近 N 筆全部存下來，每來一筆新的就丟掉最舊的，同時調整計數。但若 N 是十億，這需要十億位元的儲存。

能不能只存一個計數器？不行——因為你不知道「即將滑出視窗的那一筆」是 0 還是 1，也就無從決定該不該把計數減一。這正是滑動視窗比「整個串流」困難的根源。

24.5.2 DGIM 的分段桶

DGIM 演算法（以三位發明者 Datar、Gionis、Indyk、Motwani 的姓氏縮寫命名）的解法是：不記錄每一筆資料，而是把串流分成一段段的「桶」，每個桶記錄兩件事——它包含多少個 1（桶的大小），以及它最後一個 1 出現的時間戳。

關鍵的設計有兩點。其一，桶的大小必須是 2 的次方（1、2、4、8……）。其二，每種大小的桶最多只能有一或兩個；若出現第三個，就把最舊的兩個合併成一個更大的桶。

這個規則產生了一個重要的效果：越舊的資料，被壓縮得越粗糙。最近的資料以大小為 1 的桶精確記錄，較舊的則被合併成大桶，只知道總數而不知道分布。這符合直覺——我們對近期資料的精確度要求，通常高於久遠的資料。

24.5.3 估計與誤差保證

查詢時，把「完全落在視窗內的桶」的大小全部加起來，再加上「部分落在視窗內的那個最舊的桶」的一半。誤差就來自這最後一個桶——我們不知道它有多少個 1 還在視窗內，只能猜一半。

可以證明，這個做法的誤差不超過 50%；而若把「每種大小的桶最多兩個」改為「最多 r 個」，誤差可降至約 $1/r$ 。也就是說，你能用增加少量記憶體來換取更高的精度——這是一個可調的取舍。

記憶體用量方面：因為桶的大小呈指數成長，覆蓋 N 筆資料只需要約 $\log N$ 個量級的桶，每個桶存兩個約 $\log N$ 位元的數字。故總空間是 $\log$ 的平方級——對於 N 等於十億，這只需要幾百個位元組而非十億位元。

24.6 動差估計：AMS 草圖

24.6.1 什麼是動差

最後一個問題更抽象一些，但實務價值很高。假設串流中有多種不同的元素，各自出現了不同的次數。我們想知道的不是「有幾種元素」，而是「這些次數的分布有多不均勻」。

衡量這件事的標準工具是「動差」（moment）。設第 i 種元素出現了 m_i 次，則第 k 階動差定義為所有 m_i 的 k 次方之和。其中：

- **零階動差**：即相異元素的種數（每個 m_i 的零次方都是 1），正是 24.4 節的問題。
- **一階動差**：即串流的總長度（所有次數之和）。這個很容易，一個計數器即可。
- **二階動差**：所有次數的平方和，又稱「驚奇數」（surprise number），衡量分布的不均勻程度。

二階動差為何重要？因為平方會放大大值的影響。若所有元素出現次數相近，二階動差就小；若少數元素出現極多次（高度傾斜），二階動差就大。這正是偵測「熱點」的指標——回想第 7 章的分區熱點與第 9 章被「的」拖垮的作業，二階動差就是在量測這種傾斜。

24.6.2 AMS 的隨機化估計

精確計算二階動差需要記錄每一種元素的出現次數——這又回到了記憶體隨元素種數成長的困境。AMS 演算法（以 Alon、Matias、Szegedy 三位發明者命名）提供了一個隨機化的估計。

其核心做法是：在串流中隨機挑選若干個位置，記住該位置的元素是什麼，並持續計數「從該位置之後，這個元素又出現了幾次」。接著以一個特定的公式（涉及串流長度與該計數）算出一個估計值。

關鍵性質是：這個估計值的「期望值」恰好等於真實的二階動差——也就是說它是無偏的。單一個估計的變異很大，但取多個獨立估計的平均，就能把變異壓下來。

這個「單一估計無偏但高變異、多個估計取平均降變異」的模式，與 24.4.3 節的 Flajolet-Martin、第 14 章的 MinHash 完全一致。到這裡你應該已經看清楚了：整章的演算法雖然解決不同問題，卻共用同一套設計骨架。

24.6.3 承先啟後

讓我們把本章的共同骨架明確地寫下來，這是最希望你帶走的東西。

演算法	解決的問題	核心觀察	空間
水塘抽樣	維持均勻樣本	遞減的納入機率恰好抵消替換機率	固定 k
Bloom Filter	成員查詢	有位置為 0 則必定未出現	固定位元陣列
Flajolet-Martin	相異元素數	雜湊值尾零數反映元素規模	極小（幾個整數）
DGIM	滑動視窗計數	越舊的資料壓縮得越粗糙	$\log$ 的平方級
AMS	動差估計	隨機取樣位置的無偏估計	固定組數

表 24-2 本章串流演算法的共同骨架

五個演算法，一個共同精神：用固定的記憶體，換取有數學保證的近似答案。而它們的手法也高度相似——以雜湊或隨機取樣把資料變成隨機形式，觀察某種統計痕跡，再反推原始規模，最後以多組獨立估計降低變異。

本章我們建立了串流處理的思維基礎。但你可能已經注意到，我們談的都是「單機上如何用有限記憶體處理串流」，還沒回答一個更前面的問題：這些資料是怎麼可靠地送到我們面前的？當每秒有數十萬筆事件從上千個感測器湧來，誰負責接收、緩衝、並確保它們不會遺失？這就是下一章的主題——訊息佇列與事件驅動架構，我們將認識這個領域的主角：Kafka。

公式與流程：Bloom Filter 的空間與誤差

這一節我們把 Bloom Filter 的設計變成可計算的工程問題——這是本書最實用的公式之一，你日後很可能會真的用到。

偽陽性率

設位元陣列長度為 m 、預期插入元素數為 n 、雜湊函式個數為 k 。插入 n 個元素後，某個特定位元仍為 0 的機率約為 e 的負 kn/m 次方，故偽陽性率（ k 個位置恰好全為 1）為：

$$f \approx [1 - e^{(-kn/m)}]^k$$

式 24-2 Bloom Filter 的偽陽性率

在 m 與 n 給定的情況下，使偽陽性率最小的雜湊函式個數為：

$$k_{\text{opt}} = (m/n) \times \ln 2 \approx 0.693 \times (m/n)$$

式 24-3 最佳的雜湊函式個數

這條式子的意義很直觀： k 太少則位元設得太稀疏、鑑別力不足； k 太多則位元很快被填滿、幾乎全是 1。兩者之間存在一個最佳點。

實際試算

以插入 $n = 100$ 萬個元素為例，看不同的「每元素配置位元數」（ m/n ）會得到什麼結果。

每元素位元 m/n	陣列大小 m	最佳 k	偽陽性率	記憶體
4	400 萬位元	3	約 14.7%	0.5 MB
8	800 萬位元	6	約 2.16%	1.0 MB
10	1000 萬位元	7	約 0.82%	1.25 MB
16	1600 萬位元	11	約 0.046%	2.0 MB

表 24-3 Bloom Filter 的空間與誤差取捨（ $n = 100$ 萬）

這張表值得細看。用 1.25 MB 的記憶體，就能對一百萬個元素做成員查詢，偽陽性率不到 1%。而若用精確的集合來存，假設每個元素 20 位元組，需要 20 MB——Bloom Filter 只用了十六分之一的空間。

更關鍵的是「每元素位元數」與偽陽性率的關係：每增加約 4.8 個位元，偽陽性率就降低約十倍。這是一個非常實用的經驗法則——想要誤差降十倍，記憶體只需多加五個位元。

叫獸提醒

實務上使用 Bloom Filter 有一個必須先想清楚的前提：你要能「事先估計 n 」。因為 m 與 k 是一開始就固定的，若實際插入的元素遠超過預期，位元陣列會被填滿，偽陽性率會急遽惡化，最終退化成「什麼都說有」的無用結構。所以設計時務必留餘裕，並在運行時監控實際的插入量與位元填充率——當填充率接近一半時就該警覺了。另外還有一個限制：標準的 Bloom Filter 不支援刪除（把位元設回 0 可能誤刪別的元素共用的位置），若需要刪除功能，得改用計數式的變體。工程上沒有免費的午餐，這些限制都是換取那十六分之一空間的代價。

案例研討

案例一 數不完的訪客

回到叫獸開講。某內容平台想即時顯示「今日不重複訪客數」，初版實作用一個集合記錄所有看過的訪客識別碼。上線初期運作正常，但隨著流量成長，該服務的記憶體用量持續攀升，每天午後就會因記憶體不足而崩潰重啟——而重啟後計數歸零，資料也就丟了。

他們改用 HyperLogLog (Flajolet-Martin 的進階版本) 後，記憶體用量固定在約 1.5 KB，無論訪客有一百萬還是一億都不變，誤差約 2%。

這個案例的關鍵不只是省記憶體，更是「可預測性」——固定的記憶體用量意味著服務不會再因流量成長而崩潰。對維運而言，一個「永遠只用 1.5 KB」的元件，遠比一個「用量隨流量成長」的元件容易管理。這也呼應了第 3 章對可靠性的討論。

案例二 爬蟲的無限迴圈

某團隊開發網頁爬蟲，需要記錄「哪些網址已經爬過」以避免重複。他們最初用資料庫查詢來判斷，但每爬一個網址就要查一次資料庫，成為嚴重的效能瓶頸。

改用 Bloom Filter 後，判斷在記憶體中瞬間完成。因為它「絕不漏報」，所以不會發生「爬過的網址被誤判為沒爬過」而導致無限迴圈的災難；偶爾的誤報則只會讓某個網頁被跳過，代價可以接受。

這正是 24.3.3 節那個不對稱性的完美應用場景——重點不在於「它會不會錯」，而在於「它錯的方向是安全的」。設計系統時，能夠選擇「錯誤的方向」，往往比追求「完全不錯」更務實。

案例三 產線的即時異常率

某工廠想在儀表板上顯示「最近一小時的異常事件比例」。若把一小時內的所有事件都存在記憶體中，在高頻感測的情境下會佔用大量空間；而且每個事件過期時都要逐一移除，處理成本高。

他們改用 DGIM 式的分段桶結構：近期的事件精確記錄，較久遠的則合併為桶只保留總數。記憶體用量從與事件數成正比，降為與時間跨度的對數成正比。對於「一小時內的比例」這種指標，數個百分點的誤差完全在可接受範圍。

這個案例展示了 24.5.2 節的核心洞見在實務上的合理性——我們對「五分鐘前」的資料精確度要求，本來就高於「五十五分鐘前」。演算法的設計恰好吻合了人的認知需求。

案例四 被熱點鍵拖垮之前

某串流系統的下游採用依鍵分區的處理方式（第 7 章）。團隊擔心某些鍵的流量突然暴增而造成熱點，但又不可能為每個鍵維護精確的計數器——鍵的種類太多了。

他們的做法是以 AMS 草圖持續估計串流的二階動差。當這個值明顯上升時，代表分布正在變得傾斜——某些鍵的佔比異常增加。系統據此發出預警，讓維運人員能在熱點真正癱瘓某個分區之前介入處理。

這個案例把第 7 章的熱點問題、第 9 章被「的」拖垮的作業、與本章的動差估計串了起來。它也說明了近似演算法的一個重要價值：它讓「原本不可能持續監控的指標」變得可監控——因為只有夠便宜的東西，才能一直開著。

實作活動

活動一 實作水塘抽樣

請以程式驗證水塘抽樣的均勻性，親眼看見機率的抵消效應。

- 步驟 1.** 實作水塘抽樣，樣本大小 $k = 10$ ，對一個長度 1000 的序列（元素為 1 到 1000）執行。
- 步驟 2.** 重複執行一萬次，統計每個數字被選入樣本的次數。
- 步驟 3.** 檢查各數字的入選次數是否接近均勻（理論上每個應約為 $10000 \times 10 / 1000 = 100$ 次）。
- 步驟 4.** 刻意把第 2 步的機率改為固定值（如一律 0.01），重複實驗並比較——說明為何「遞減的機率」才是正確做法。

活動二 實作並測試 Bloom Filter

請實作一個 Bloom Filter 並驗證式 24-2 的預測。

- 步驟 1.** 以位元陣列與 k 個雜湊函式實作插入與查詢功能（可用不同的種子產生多個雜湊）。
- 步驟 2.** 設 $n = 10$ 萬，分別以每元素 4、8、10、16 位元建立四個 Bloom Filter，並依式 24-3 設定最佳 k 。
- 步驟 3.** 插入 10 萬個元素後，以另外 10 萬個「確定沒插入過」的元素進行查詢，統計實測的偽陽性率。
- 步驟 4.** 把實測結果與式 24-2 的理論值比較，並與表 24-3 對照，說明是否吻合。

活動三 實作 Flajolet-Martin

請實作相異元素計數並觀察其誤差特性。以雜湊函式把元素轉為二進位字串，記錄「最大尾零數 R 」，並依式 24-1 估計相異元素數。首先以單一雜湊函式，對一個含 10,000 個相異元素（但總筆數 100,000，含大量重複）的串流進行估計，重複二十次並記錄估計值的分布。接著改用二十組獨立雜湊函式取中位數，再重複二十次。比較兩者的誤差分布，說明為何「多組取中位數」能大幅降低變異，以及為何取中位數優於取平均。

延伸閱讀

- **《Mining of Massive Datasets》第 4 章：**Leskovec 等人著。本章的主要依據，對抽樣、Bloom Filter、Flajolet-Martin、DGIM 與 AMS 皆有完整推導。
- **HyperLogLog 論文：**Flajolet 等人發表，說明如何以分組與偏誤修正把相異計數的誤差降至約 2%，是 24.4.3 節的進階延伸。

- **Bloom Filter 的變體綜述**：介紹計數式 Bloom Filter、Cuckoo Filter 等支援刪除或更省空間的改良結構。
- **串流演算法的理論教材**：關於串流模型的空間下界與誤差保證之理論分析，適合有興趣深入數學基礎的讀者。

本章小結

1. 串流計算模型有三個限制：單次通過、記憶體受限、即時回應；這使得許多問題無法求得精確解，必須改以「固定記憶體、誤差有數學保證」的近似演算法應對。
2. 水塘抽樣以 k/i 的遞減機率納入新資料並隨機替換，使任一時刻的樣本皆為均勻分布，且記憶體用量與串流長度無關。
3. Bloom Filter 以位元陣列與多個雜湊函式進行成員查詢，其關鍵性質是不對稱——絕無偽陰性但有偽陽性；因許多應用中偽陽性的代價遠低於偽陰性，這個不對稱恰是它的價值所在。
4. Flajolet-Martin 觀察雜湊值尾端零的個數：出現 r 個尾零的機率為 2 的負 r 次方，故由觀察到的最大尾零數 R 可估計相異元素數約為 2^R ；重複元素不影響結果，因其雜湊值相同。
5. 單一估計變異極大，故實務上以多組獨立雜湊取中位數降低變異；取中位數而非平均，是因為估計值呈 2 的次方而分布極度偏斜，異常大值會嚴重拉高平均。
6. 滑動視窗因舊資料會過期而比整串流查詢困難；DGIM 以大小為 2 的次方的分段桶記錄，並限制每種大小最多兩個桶，使越舊的資料壓縮得越粗糙，誤差不超過 50% 且可透過增加桶數降低。
7. 動差衡量出現次數分布的不均勻程度：零階為相異元素數、一階為總長度、二階（驚奇數）反映傾斜程度，可用於偵測熱點；AMS 以隨機取樣位置得到無偏估計，再以多組平均降低變異。
8. Bloom Filter 的偽陽性率為 $[1 - e^{(-kn/m)}]^k$ （式 24-2），最佳雜湊數為 $0.693 \times (m/n)$ （式 24-3）；每元素多配置約 4.8 個位元，偽陽性率即降低約十倍。使用前須能估計 n ，且標準版本不支援刪除。

習題

一、觀念題

1. 請說明串流計算模型的三個限制，並解釋為何「計算相異元素數」在此模型下無法求得精確解。
2. 為什麼水塘抽樣要使用「 k/i 」這個隨 i 遞減的機率？若改用固定機率會有什麼問題？
3. Bloom Filter 為何「絕無偽陰性但有偽陽性」？請說明這個不對稱性為何在實務上是優點，並舉一個例子。
4. Flajolet-Martin 為何要用「多組估計取中位數」而非「取平均」？請說明其統計上的理由。

二、計算題

1. 某系統需對 $n = 50$ 萬個元素建立 Bloom Filter，配置的位元陣列大小 $m = 500$ 萬位元。
（一）請以式 24-3 計算最佳的雜湊函式個數 k （取最接近的整數）；（二）以式 24-2 計算此設定下的偽陽性率；（三）計算所需記憶體（以 MB 表示），並與「以每元素 24 位元組的精確集合儲存」相比，節省了幾倍。
2. 某 Flajolet-Martin 實作觀察到最大尾零數 $R = 17$ 。（一）請以式 24-1 估計相異元素數；（二）若另外三組獨立雜湊分別觀察到 $R = 15$ 、 18 、 16 ，請列出四個估計值並取中位數；（三）說明為何本例中「中位數」比「平均」更能反映合理的估計。

三、思考與討論

1. 本章五個演算法共用同一套設計骨架（表 24-2）。請以自己的話歸納這個骨架，並說明它與第 14 章的 MinHash、第 16 章的 BFR 有何相通之處。
2. 案例四指出「只有夠便宜的東西，才能一直開著」。請就一個你熟悉的監控情境討論：有哪些指標因為計算成本太高而無法持續監控？近似演算法是否可能改變這個情況？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

三個限制：單次通過（每筆資料只經過一次，錯過即消失，無法回頭重看）、記憶體受限（資料無限而記憶體有限，任何隨資料量成長的儲存策略終將失敗）、即時回應（查詢可能隨時到來且須立即回答）。相異元素數無法精確求解的原因：要精確判斷某個元素「是否曾經出現過」，就必須記住所有看過的元素——否則無從分辨眼前這筆是新的還是重複的。而記住所有相異元素所需的記憶體，必然與相異元素數成正比，在無限串流上終將超出任何有限的記憶體。故只能改求近似。

第 2 題

因為要保證「任一時刻的樣本都是從至今所有資料中均勻抽出的」。當已看過 i 筆資料時，每筆的入選機率都應為 k/i ；新資料以 k/i 的機率被納入，恰好符合此要求。同時，舊資料被替換掉的機率也隨 i 增大而遞減，兩個效應相互抵消，使所有已到達資料的存活機率始終相等。若改用固定機率會有兩個問題：其一，若機率設得太高，樣本會被後來的資料不斷替換，早期資料幾乎不可能存活，樣本嚴重偏向近期；其二，若設得太低，早期資料佔比過高。無論如何都無法維持均勻，且因串流總長未知，根本無從決定該設多少。

第 3 題

無偽陰性的原因：插入元素時會把 k 個對應位置全設為 1，這些位置一旦被設為 1 就不會變回 0，故該元素日後查詢時 k 個位置必定全為 1，絕不會被誤判為「沒出現過」。有偽陽性的原因：某個未插入的元素，其 k 個位置可能剛好被其他不同元素分別設為 1 而湊巧全中，因而被誤判為「可能出現過」。實務上為優點的理由：許多應用中偽陽性的代價遠低於偽陰性。例子如網頁爬蟲——誤報「已爬過」只是漏爬一頁，但若會漏報（把爬過的說成沒爬過）就會造成無限迴圈；或資料庫查詢——說「沒有」可直接跳過昂貴的磁碟存取，說「可能有」再實際查一次即可，誤報僅是多查一次。

第 4 題

因為 Flajolet-Martin 的估計值是 2 的次方 (2^R)，分布極度偏斜且呈指數尺度。若某一組雜湊碰巧觀察到一個尾零數特別大的值（例如 R 比其他組多 5），其估計值會比其他組的 32 倍——這樣一個異常值會嚴重拉高平均，使平均值遠高於真實值。而中位數只看「排在中間的那一個」，完全不受極端值大小的影響，故對偏斜分布穩健得多。實務上常採折衷：先把多組分成數群、群內取平均（降低隨機變異），再對各群的平均取中位數（抵抗極端值），兼得兩者之長。

二、計算題

第 1 題

$n = 500,000$ 、 $m = 5,000,000$ ，故 $m/n = 10$ 。

- (1) 最佳 k (式 24-3) $= 0.693 \times 10 = 6.93 \approx 7$ 。
- (2) 偽陽性率 (式 24-2)： $kn/m = 7 \times 500,000 / 5,000,000 = 0.7$ ； $e^{(-0.7)} \approx 0.4966$ ； $1 - 0.4966 = 0.5034$ ； $f = 0.5034^7 \approx 0.0082$ ，即約 0.82%。
- (3) 記憶體： $5,000,000$ 位元 $\div 8 = 625,000$ 位元組 $= 0.625$ MB。精確集合需 $500,000 \times 24 = 12,000,000$ 位元組 $= 12$ MB。節省倍數 $= 12 \div 0.625 = 19.2$ 倍。

延伸思考：此結果與表 24-3 的「每元素 10 位元」那一列完全一致（偽陽性率約 0.82%），因為偽陽性率只取決於 m/n 的比值與 k ，而與 n 的絕對大小無關。這是一個很實用的性質——你可以先決定「每元素配置幾個位元」，偽陽性率就確定了。

第 2 題

依式 24-1，估計值 $= 2^R$ 。

- (1) $R = 17$ ：估計相異元素數 $= 2^{17} = 131,072$ 。
- (2) 四組的估計值： $R = 15 \rightarrow 32,768$ ； $R = 16 \rightarrow 65,536$ ； $R = 17 \rightarrow 131,072$ ； $R = 18 \rightarrow 262,144$ 。排序後取中間兩者的平均（偶數個），中位數 $= (65,536 + 131,072) / 2 = 98,304$ 。
- (3) 四者的算術平均 $= (32,768 + 65,536 + 131,072 + 262,144) / 4 = 491,520 / 4 = 122,880$ 。

為何中位數較合理：四個 R 值僅相差 3，但對應的估計值卻相差 8 倍（32,768 對 262,144）——這正是指數尺度造成的偏斜。平均值 122,880 明顯被最大的那個估計（262,144）向上拉抬；而中位數 98,304 較不受單一極端值影響，更能代表這組觀察的中心趨勢。若增加更多組獨立雜湊，中位數會更穩定地收斂到真值附近。

三、思考與討論

第 1 題

共同骨架可歸納為四步：其一，以雜湊或隨機取樣，把原始資料轉換為「隨機化」的形式（MinHash 取雜湊最小值、FM 看雜湊值尾零、AMS 隨機挑位置、Bloom Filter 雜湊到位元位置）。其二，觀察這些隨機值的某種統計痕跡（最小值、最大尾零數、位元填充狀態、抽樣計數）。其三，由該痕跡反推原始資料的規模或性質，且此反推有明確的機率保證。其四，以多組獨立實驗取平均或中位數，降低單一估計的高變異。與第 14 章 MinHash 的相通：MinHash 正是「以雜湊最小值的碰撞機率反推 Jaccard 相似度」，骨架完全相同。與第 16 章 BFR 的相通之處略有不同——BFR 用的是「可結合的摘要統計」而非隨機化，但兩者共享更上位的原則：不保留原始資料，只保留一個固定大小、足以回答目標問題的摘要。周延的回答會指出：整個第肆、陸篇的核心思想都是「以可控的近似換取可行的計算」。

第 2 題

答題應包含具體情境、成本障礙與近似解法三部分。可能的情境舉例：其一，網路流量的「每個來源 IP 的不重複目的地數」——精確計算需為每個來源維護一個集合，來源數以百萬計時完全不可行改用 HyperLogLog，每個來源僅需約 1.5 KB，即可持續監控並偵測掃描行為。其二，產線的「每個機台每分鐘的相異錯誤碼種數」——可用同樣方法。其三，「使用者行為的分布傾斜度」——精確計算需記錄每個使用者的次數，改用 AMS 估計二階動差即可持續監控（如案例四）。其四，「最近一小時的近似重複內容比例」——可結合第 14 章的 MinHash 與本章的滑動視窗。核心論點：監控指標的選擇往往不是「哪個最有意義」，而是「哪個負擔得起持續計算」；近似演算法把後者的門檻大

幅降低，因而擴大了可監控指標的範圍——這正是案例四那句「只有夠便宜的東西，才能一直開著」的意涵。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 24 章 資料串流模型與近似演算法

台灣自編大學教科書
@prof
大數據分析與雲端運算

第陸篇 串流處理與即時分析

Stream Processing & Real-time Analytics

第 25 章

訊息佇列與事件驅動架構

Message Queues and Event-Driven Architecture

機器人叫獸（李世淵） 編著

叫獸開講

我想從一間餐廳的廚房講起。

小餐館的做法很單純：服務生點完餐，直接走到廚房跟廚師說「三號桌要一份炒飯」。這叫做「同步呼叫」——服務生站在那裡等，廚師聽到就做。生意清淡時完全沒問題。

但一到用餐尖峰，這個模式就崩潰了。服務生擠在廚房門口排隊喊單，廚師被此起彼落的聲音搞得手忙腳亂；更慘的是，如果廚師手上有事沒空聽，服務生就得一直站在那裡等，前場沒人服務客人。整間店的效率被最慢的那個環節卡死。

稍具規模的餐廳怎麼做？他們在中間放了一個東西——出單機。服務生把單子打上去就走，繼續服務客人；廚師有空就從單子上依序取單來做。前場與後場「解耦」了：服務生不必等廚師，廚師也不必被打斷。單子還天然形成了一個緩衝——尖峰時單子會堆積，但不會有人被卡住，等離峰再慢慢消化。

這台出單機，就是這一章的主角：訊息佇列。上一章我們談了「如何用有限記憶體處理無限的資料流」，但我留了一個問題沒答——這些資料是怎麼可靠地送到我們面前的？當每秒有數十萬筆事件從上千個感測器湧來，誰負責接收、緩衝、確保不遺失、還要讓多個下游各自取用？這就是 Kafka 這類系統存在的理由。

學習目標

研讀完本章之後，你應該能夠：

1. 說明訊息佇列的解耦與緩衝價值，並區分點對點與發布訂閱模式。
2. 描述 Kafka 的主題、分區、代理與副本等核心元件。
3. 說明分區、消費者群組與位移的運作機制。
4. 區分三種傳遞語意，並說明其實作代價。
5. 說明事件驅動架構與事件溯源的設計理念。
6. 運用消費者落後量估算追趕時間，並據以規劃消費者數量。

核心概念

核心概念	說明
訊息佇列	在生產者與消費者之間傳遞訊息的中介，提供解耦與緩衝。
發布訂閱	一則訊息可被多個訂閱者各自取用的傳遞模式。
主題與分區	Kafka 中訊息的分類單位，以及主題內平行處理的切分單位。
位移 (Offset)	訊息在分區中的序號，消費者以此記錄讀到哪裡。
消費者群組	共同分擔一個主題的一組消費者，分區在群組內不重複分配。
傳遞語意	至多一次、至少一次、恰好一次三種可靠度保證。
事件溯源	以事件序列作為系統的真實來源，狀態由事件重播推導。

表 25-1 本章核心概念一覽

25.1 訊息佇列與發布訂閱

25.1.1 解耦與緩衝：兩個核心價值

承接叫獸開講，訊息佇列提供了兩個難以取代的價值。

第一是「解耦」。生產者不需要知道消費者是誰、在哪裡、甚至是否還活著；它只負責把訊息送進佇列就完成任務。這帶來極大的彈性——你可以新增一個消費者來做新的分析，而完全不必修改生產端的程式。這正是第 2 章談的「資料生產者與消費者的分離」在架構上的落實。

第二是「緩衝」。當上游的產生速率短暫超過下游的處理能力時，訊息會在佇列中堆積，而不會壓垮下游或造成資料遺失。這在感測資料的場景中尤其重要——產線不會因為分析系統忙碌就停止產生訊號。

這裡有個重要的觀念：緩衝只能吸收「短暫的」尖峰。若上游的長期平均速率超過下游的處理能力，佇列只會無限成長，最終仍會崩潰——這正是第 2 章利特爾法則的意涵。緩衝爭取的是時間，不是產能。

25.1.2 兩種傳遞模式

模式	運作方式	適用情境
點對點	一則訊息只會被一個消費者取走並處理	工作分派，如訂單處理、任務佇列
發布訂閱	一則訊息可被多個訂閱者各自完整取用	事件廣播，如同一批感測資料供多個分析使用

表 25-2 訊息傳遞的兩種模式

大數據場景中，發布訂閱是更常見的需求。同一批產線資料，可能同時需要即時告警、寫入資料湖、更新儀表板、以及餵給機器學習模型——四個下游各自需要完整的資料，而非瓜分。

Kafka 的巧妙之處在於，它有一套機制同時支援兩種模式：不同的消費者群組各自取得完整資料（發布訂閱），而同一群組內的消費者則分攤資料（點對點）。25.3 節會說明這是如何做到的。

25.2 Kafka 架構與核心元件

25.2.1 把佇列當成日誌

Kafka 最關鍵的設計決策，是把訊息佇列實作為一個「持久化的、只能追加的日誌」（append-only log），而非傳統佇列那種「取走就消失」的結構。

這個看似簡單的轉變帶來了深遠的影響。在傳統佇列中，訊息被消費後就被刪除，因此只能被消費一次；而在 Kafka 中，訊息被寫入後會保留一段設定的時間（可能是數天或數週），消費者只是「讀取」它而不會使它消失。

這帶來三個好處：其一，多個消費者可以各自獨立地讀取同一份資料；其二，消費者可以「回退」重讀（例如程式有臭蟲修正後要重跑）；其三，新加入的消費者可以從頭讀取歷史資料。

你應該覺得這個「只追加、不修改」的設計很眼熟——第 4 章的 LSM-tree、第 5 章 HDFS 的「一次寫入多次讀取」、第 11 章 RDD 的不可變性，全都是同一個思想。追加式的設計簡單、高效、且天然適合分散式。

25.2.2 核心元件

元件	角色	說明
主題 (Topic)	訊息的分類	如「感測資料」「交易事件」，類似資料表的概念
分區 (Partition)	主題內的平行切分	每個分區是一個獨立的有序日誌
代理 (Broker)	叢集中的伺服器節點	負責儲存分區並處理讀寫請求
生產者 (Producer)	寫入訊息的一方	決定訊息要送往哪個分區
消費者 (Consumer)	讀取訊息的一方	以位移記錄自己讀到哪裡

表 25-3 Kafka 的核心元件

其中「分區」是理解 Kafka 的關鍵。一個主題被切分成多個分區，分散在不同的代理上——這正是第 7 章的分區思想。分區的數量直接決定了平行處理的上限，下一節會詳談。

25.2.3 副本與容錯

每個分區可以有多个副本，分散在不同的代理上。其中一個是「領導者」，負責處理所有讀寫；其餘是「追隨者」，持續同步領導者的資料。

當領導者所在的代理故障時，系統會從同步狀態良好的追隨者中選出新的領導者，服務繼續。這正是第 7 章的主從複製與第 8 章的領導者選舉在此的具體應用——而選舉本身，早期版本正是依賴第 8 章談過的協調服務來完成。

這裡有一個重要的可靠度參數：生產者可以設定「寫入需要幾個副本確認才算成功」。設為 1（只需領導者確認）最快，但領導者若在同步前故障，資料就遺失；設為全部副本確認最安全，但延遲最高。這又是第 3 章一致性與可用性取舍的又一次體現。

25.3 分區、消費者群組與位移

25.3.1 分區決定平行度

Kafka 有一條鐵律：在同一個消費者群組中，每個分區只能被一個消費者讀取。

這條規則的用意是保證「順序」——同一個分區內的訊息會被依序處理，不會出現兩個消費者同時處理同一分區而導致亂序。但它也帶來一個直接的後果：消費者的數量若超過分區數，多出來的消費者將完全閒置。

換言之，分區數就是平行度的上限。若你有 4 個分區，那麼最多只有 4 個消費者能同時工作——增加第 5 個消費者不會提升任何吞吐量，它只會閒著。這是實務上極常見的效能陷阱。

叫獸提醒

我要特別提醒分區數的設定，因為它是個「一開始就要想清楚」的決定。分區數可以增加，但不能減少；而且增加分區會改變鍵的分配（回想第 7 章的取餘數問題——加分區時，原本落在某分區的鍵可能改落到別處），破壞原有的順序保證。所以實務上的建議是：預留成長空間，一開始就設定比目前所需更多的分區數。但也別設得太誇張——分區太多會增加代理的檔案控

制代碼與中繼資料負擔，領導者選舉時的恢復時間也會拉長。這是一個需要依「預期的尖峰吞吐量」與「未來兩三年的成長」來估算的工程判斷，而非隨手填的數字。

25.3.2 消費者群組：一套機制兩種模式

現在可以回答 25.1.2 節留下的問題了：Kafka 如何以一套機制同時支援點對點與發布訂閱？

答案在於「消費者群組」。群組內的消費者會分攤該主題的所有分區——每個分區指派給一個消費者，形成點對點的工作分派效果。而不同的群組之間互不影響，各自從頭讀取完整的資料——形成發布訂閱的廣播效果。

以產線資料為例：「即時告警」是一個群組、「寫入資料湖」是另一個群組、「模型推論」是第三個群組。三個群組各自讀取完整的資料流（發布訂閱）；而每個群組內部若有多個實例，則分攤分區以提升吞吐（點對點）。一套機制，兩種需求同時滿足。

25.3.3 位移：讀到哪裡由自己管

位移（offset）是訊息在分區中的序號。每個消費者群組會記錄自己在每個分區上「讀到哪個位移」，這就是它的進度。

這個設計把「進度管理」的責任交給了消費者，帶來很大的彈性：想重讀歷史資料，就把位移調回去；想跳過積壓的舊資料只處理最新的，就把位移直接調到最新。這在維運上極為實用。

但它也帶來一個關鍵問題：位移該在「處理前」還是「處理後」提交？這個看似細微的選擇，直接決定了下一節的傳遞語意——這也是本章最需要小心的地方。

25.4 傳遞語意與可靠性

25.4.1 三種語意

訊息系統的可靠度保證分為三個等級，理解它們的差異至關重要。

語意	保證	風險與代價
至多一次	訊息不會被重複處理	可能遺失（處理失敗就沒了）
至少一次	訊息不會遺失	可能重複處理，須配合冪等性
恰好一次	既不遺失也不重複	實作複雜、效能代價高

表 25-4 三種傳遞語意的比較

這三種語意如何產生？關鍵正是 25.3.3 節提到的位移提交時機。

若在「處理之前」就提交位移，那麼處理失敗時該訊息已被標記為讀過，不會重試——這是至多一次，可能遺失。若在「處理之後」才提交，那麼若處理成功但提交前當機，重啟後會重讀該訊息——這是至少一次，可能重複。

你會發現這與第 3 章談過的逾時重試兩難、第 13 章的管線設計是同一個問題：你無法同時避免遺失與重複，除非付出額外的代價。

25.4.2 實務上的選擇

業界的主流選擇是「至少一次，配合冪等的消費端」。

理由很務實：至少一次的實作簡單、效能好、且遺失資料通常比重複處理嚴重得多。而重複的問題，可以在消費端以冪等性化解——回想第 13 章 13.4.2 節，把「附加」改為「以主鍵覆寫」，重複執行就不會產生錯誤結果。

「恰好一次」雖然聽起來最理想，但它的實作需要生產端的冪等寫入、交易性的位移提交、以及端到端的協調，代價相當高。更重要的是——即使 Kafka 保證了恰好一次，若你的消費端把結果寫到外部系統（資料庫、檔案），那個寫入動作仍可能重複。所以「端到端的恰好一次」往往還是得靠消費端的冪等設計來達成。

我的建議是：與其追求恰好一次的複雜機制，不如把力氣花在「讓你的處理邏輯冪等」。這不僅更簡單可靠，而且能同時應付重試、回填、故障恢復等各種情境——這正是第 13 章反覆強調的第一原則。

25.5 事件驅動架構與事件溯源

25.5.1 從「呼叫」到「發生了什麼」

事件驅動架構（event-driven architecture）是一種以「事件」為中心的系統設計方式。它的思維轉變在於：不再由服務 A 直接呼叫服務 B 要它做事，而是由服務 A 宣告「某件事發生了」，讓所有關心這件事的服務各自反應。

以訂單為例。傳統做法是：訂單服務在建立訂單後，依序呼叫庫存服務、通知服務、物流服務、分析服務。這使訂單服務必須知道所有下游、必須處理各種失敗、而且新增一個下游就要改一次程式碼。

事件驅動的做法是：訂單服務只發布一則「訂單已建立」事件，然後就完成了。庫存、通知、物流、分析各自訂閱這則事件並做出反應。新增下游時，訂單服務完全不必修改。

這個轉變的本質，是把「命令式的協調」換成了「宣告式的通知」——與第 12 章從 RDD 到 DataFrame 的轉變、第 18 章 Pregel 的「像頂點一樣思考」，有著相通的精神：不去集中指揮，而是讓各方依規則自行反應。

25.5.2 事件溯源：把事件當成真相

事件溯源（event sourcing）更進一步：它主張系統的「真實來源」不應是當前狀態，而應是「導致這個狀態的所有事件序列」。

用銀行帳戶來理解最清楚。傳統做法是資料庫裡存一個「餘額：15,000」的欄位，每次交易就更新它。事件溯源則是只記錄「存入 10,000」「提領 3,000」「存入 8,000」這些事件，餘額則由這些事件推導而來。

這樣做有幾個顯著的好處：完整的稽核軌跡（每一分錢的來龍去脈都在）、能回到任意時間點的狀態（重播到那一刻即可）、以及能事後用新的角度重新解讀歷史（例如加入新的統計指標，可以重播全部歷史來計算）。

代價則是：讀取當前狀態需要重播事件（實務上會定期做「快照」來加速）、儲存量較大、且系統設計的複雜度提高。因此它通常用在稽核要求高、或需要時光回溯能力的領域。

25.5.3 與 CDC 的呼應

這裡值得回顧第 2 章談過的變更資料擷取（CDC）。CDC 的做法是讀取資料庫的交易日誌，把每一筆變動轉為事件——這其實就是「從既有的狀態導向系統中，反推出事件流」。

而事件溯源則是反過來：一開始就以事件為主，狀態才是衍生品。兩者殊途同歸，都體現了同一個洞見——「表與流本是一體兩面」。一連串的事件可以推导出狀態，而狀態的每次變化也可以被視為事件。

這個「表流二元性」是理解現代資料架構的關鍵觀念之一，它也是下一章串流處理引擎能夠統一批次與串流的理論基礎。

25.6 資料重播與串流整合

25.6.1 重播的價值

因為 Kafka 保留訊息而非取走即刪，「重播」成為一項極有價值的能力。實務上有幾種常見的應用場景：

- **修正臭蟲後重跑：**發現處理邏輯有誤時，修正後把位移調回去重新處理，即可修復下游的錯誤結果。
- **新增下游系統：**新的分析系統上線時，可以從歷史資料開始讀取，快速建立完整的狀態。
- **災難恢復：**下游系統的資料損毀時，可從訊息佇列重播來重建。
- **測試與驗證：**以真實的歷史資料流測試新版本的處理邏輯，比人造測試資料更貼近實況。

注意，重播能成立的前提仍是「處理邏輯必須冪等」——否則重播只會製造重複的資料。這再次印證了第 13 章那個第一原則的重要性。

25.6.2 串流與批次的橋樑

訊息佇列在現代資料架構中，往往扮演「中央神經系統」的角色。所有的資料先進入佇列，再由不同的下游各取所需——即時分析系統以低延遲消費、批次系統則定期批量讀取寫入資料湖。

這個架構解決了一個長期存在的問題：過去，即時系統與批次系統往往各自建立自己的資料擷取管線，導致同一份資料被重複擷取、且兩邊的口徑可能不一致。以佇列為單一入口後，兩者讀的是同一份資料，一致性自然得到保障。

這也是下一章要談的 Lambda 與 Kappa 架構的基礎。特別是 Kappa 架構，它的核心主張正是「既然佇列能保留並重播歷史，那我們何不用同一套串流引擎處理即時與歷史，徹底廢除批次系統」——這個大膽的想法，完全建立在本章的重播能力之上。

25.6.3 承先啟後

本章我們認識了串流世界的基礎設施。從餐廳的出單機出發，理解了訊息佇列的解耦與緩衝價值（25.1）；看到 Kafka 把佇列實作為持久化日誌的關鍵設計（25.2）、分區與消費者群組如何一套機制支援兩種模式（25.3）、三種傳遞語意的取捨與「至少一次加冪等」的實務選擇（25.4）、事件驅動與事件溯源的思維轉變（25.5），以及重播能力如何成為串流與批次的橋樑（25.6）。

我希望你注意到一件事：本章幾乎每個設計，都能在前面的章節找到源頭。分區來自第 7 章、副本與領導者選舉來自第 7、8 章、追加式日誌的思想貫穿第 4、5、11 章、而冪等性則是第 3、13 章反覆強調的原則。Kafka 不是憑空出現的新發明，它是把這些成熟的分散式系統原理，組裝成一個專為資料流而生的系統。

到這裡，資料已經能夠可靠地流動了。但流動不是目的——我們真正要做的是在資料流過的當下就完成分析：計算最近五分鐘的移動平均、偵測異常樣態、更新即時儀表板。這需要專門的串流處理引擎，以及一套處理「時間」的全新思維（資料遲到了怎麼辦？視窗何時該關閉？）。這就是下一章的主題。

公式與流程：消費者落後量與追趕時間

這一節要教你一個維運上每天都會用到的計算：當訊息開始積壓時，該加幾個消費者？多久能追上？

落後量與淨消化速率

消費者落後量（consumer lag）指的是「已寫入但尚未被消費的訊息數」，也就是生產者的最新位移與消費者當前位移之差。它是串流系統最重要的健康指標。

設生產速率為 P （筆／秒）、單一消費者的處理速率為 C （筆／秒）、消費者數量為 n （且不過分區數）。則系統的淨消化速率為：

$$\text{淨消化速率} = n \times C - P$$

式 25-1 系統的淨消化速率

這個值必須為正，落後量才會下降。若為負，落後量將持續累積，系統終將崩潰——這正是第 2 章利特爾法則的另一種表述。

若當前落後量為 L ，則追趕所需的時間為：

$$\text{追趕時間} = L / (n \times C - P)$$

式 25-2 消化積壓所需的時間

實際試算

設生產速率 $P = 50,000$ 筆／秒，單一消費者可處理 $C = 15,000$ 筆／秒，當前積壓 $L = 3,000$ 萬筆。下表計算不同消費者數的表現。

消費者數 n	總處理能力	淨消化速率	追趕 3000 萬筆
3	45,000 筆／秒	-5,000（負值）	永遠追不上
4	60,000 筆／秒	10,000 筆／秒	約 50 分鐘
5	75,000 筆／秒	25,000 筆／秒	約 20 分鐘
6	90,000 筆／秒	40,000 筆／秒	約 12.5 分鐘

表 25-5 消費者數量對追趕時間的影響

這張表有兩個重要的觀察。第一，3 個消費者的總處理能力（45,000）低於生產速率（50,000），因此落後量只會持續增加——無論等多久都追不上。這是必須立刻警覺的狀態。

第二，注意追趕時間的非線性。從 4 個增為 5 個消費者（增加 25% 的資源），追趕時間卻從 50 分鐘降為 20 分鐘（縮短 60%）。原因是分母是「淨」消化速率——當你的處理能力只略高於生產速率時，淨值很小，稍微增加一點就能帶來巨大的改善。

叫獸提醒

表 25-5 揭示了一個維運上的關鍵直覺：系統在「勉強跟得上」的狀態下最危險。當 $n = 4$ 時，淨消化速率只有 10,000，看起來是正的、系統也確實在追趕——但只要生產速率上升 20%（尖峰時很常見），淨值就變成負的，系統立刻開始積壓。所以規劃容量時，絕不能只算「平均生產速率」，而要以「尖峰速率」為準，並保留至少三到五成的餘裕。另外請務必記得 25.3.1 節那條鐵律——消費者數不能超過分區數，否則加了也是白加。我看過太多人在系統積壓時瘋狂增加消費者，卻忘了分區數只有四個，多出來的全在閒置。監控落後量、算清楚淨消化速率、確認分區數足夠，這三件事是串流維運的基本功。

案例研討

案例一 被下游拖垮的感測系統

某工廠的感測資料原本直接寫入分析資料庫。某日資料庫因為維護作業而變慢，結果整條資料採集鏈路開始阻塞——感測器的資料送不出去，緩衝區塞滿後開始丟棄資料，那段時間的產線紀錄永久遺失。

問題的根源是「緊耦合」：上游的採集直接依賴下游的資料庫，下游一慢，上游就跟著垮。他們改為在中間放入 Kafka：感測器只負責把資料寫入佇列，寫入極快且不受下游影響；資料庫則以自己的節奏從佇列消費。

此後同樣的資料庫維護再度發生時，訊息只是在佇列中暫時積壓，資料庫恢復後便自動追上，沒有任何資料遺失。這個案例展示了 25.1.1 節的兩個核心價值——解耦讓故障不會傳染，緩衝讓短暫的落後不會造成損失。

案例二 加了消費者卻沒變快

某團隊的串流處理出現嚴重積壓，他們立刻把消費者從 4 個增加到 12 個，卻發現吞吐量完全沒有改善，積壓依舊持續惡化。

原因是該主題只有 4 個分區。依 25.3.1 節的鐵律，同一群組中每個分區只能被一個消費者讀取，因此只有 4 個消費者在工作，另外 8 個完全閒置——他們白白付了三倍的運算資源。

正確的做法是先增加分區數。但這裡又遇到一個麻煩：增加分區會改變鍵的分配，破壞原有的順序保證（回想第 7 章的取餘數問題）。他們最終選擇建立一個分區數更多的新主題，讓生產者改寫入新主題，並在過渡期間同時消費兩個主題。這個案例說明：分區數是需要前瞻規劃的設定，事後補救的代價相當高。

案例三 重複扣款，又一次

某支付系統以 Kafka 傳遞交易事件，消費端負責更新帳務。為了確保不遺失，他們採用「至少一次」語意——處理完成後才提交位移。

某次消費者在處理成功、但提交位移之前發生當機。重啟後，該筆訊息被重新讀取並再次處理，導致同一筆交易被扣款兩次。

這與第 13 章案例二的「重跑一次，業績多一倍」是完全相同的問題，只是換了場景。解法也相同：讓處理邏輯幂等。他們為每筆交易加上唯一識別碼，並在寫入前檢查該識別碼是否已處理過——

如此無論訊息被重送幾次，效果都只有一次。這個案例再次印證：至少一次的語意必須配合冪等的消費端，兩者缺一不可。

案例四 從佇列重建的儀表板

某團隊的即時儀表板服務因為程式錯誤，連續三天寫入了錯誤的統計數字。發現時，錯誤資料已經污染了整個報表系統。

因為所有原始事件都保留在 Kafka 中（保留期設為七天），他們得以修正程式後，把消費者的位移調回三天前，重新處理這段期間的所有事件——儀表板在數十分鐘內完全重建，錯誤資料被正確的結果覆蓋。

若當初採用的是傳統的「取走即刪」佇列，這三天的原始資料早已不存在，唯一的選擇是接受錯誤或從其他來源艱難地補救。這個案例展現了 25.6.1 節重播能力的實務價值——而它能成立，關鍵在於 25.2.1 節那個「把佇列當成日誌」的設計決策，以及消費端的冪等性（重播時覆寫而非累加）。

實作活動

活動一 規劃你的容量

請以式 25-1 與式 25-2 完成一次容量規劃。

- 步驟 1.** 設某產線有 200 個感測器，各以每秒 100 筆的頻率產生資料，計算總生產速率 P 。
- 步驟 2.** 假設單一消費者可處理 8,000 筆／秒，計算「剛好跟得上」所需的最少消費者數。
- 步驟 3.** 若要保留 50% 的餘裕以應付尖峰，應配置幾個消費者？分區數至少應設為多少？
- 步驟 4.** 假設某次故障造成 5,000 萬筆積壓，以你規劃的消費者數計算追趕所需時間。

活動二 實作生產者與消費者

請以本機的 Kafka（或相容的訊息佇列）實作一組收發程式。

- 步驟 1.** 建立一個含 3 個分區的主題，撰寫生產者持續送出模擬的感測資料（含機台編號作為鍵）。
- 步驟 2.** 撰寫消費者讀取並印出訊息，觀察同一機台的資料是否都落在同一分區。
- 步驟 3.** 同時啟動 2 個屬於同一群組的消費者，觀察分區如何被分配；再啟動第 4 個，觀察它是否閒置。
- 步驟 4.** 另外啟動一個屬於「不同群組」的消費者，確認它能收到完整的資料流（驗證發布訂閱行為）。

活動三 驗證傳遞語意與冪等性

請設計實驗觀察三種傳遞語意的差異。首先實作一個消費者，在「處理前」提交位移，並在處理過程中刻意讓程式當機，重啟後觀察該訊息是否遺失（至多一次）。接著改為在「處理後」提交，同樣製造當機，觀察該訊息是否被重複處理（至少一次）。最後，把處理邏輯改為冪等（例如以訊息中的唯一識別碼為主鍵寫入，存在則覆寫），重複同樣的實驗，確認即使訊息被處理兩次，最終結果仍然正確。請記錄三次實驗的結果並說明其對應的語意。

延伸閱讀

- 《Designing Data-Intensive Applications》第 11 章：Kleppmann 與 Riccomini 著。深入討論訊息傳遞、日誌式佇列與流批二元性，是本章最重要的理論依據。
- Apache Kafka 官方文件：說明主題、分區、消費者群組、傳遞語意與參數設定，是實作時的必備參考。
- 《Kafka: The Definitive Guide》：以完整篇幅介紹 Kafka 的架構、維運與效能調校，適合需要實際部署者。
- 事件溯源與 CQRS 的架構文獻：說明以事件為真實來源的系統設計模式及其取舍，是 25.5.2 節的延伸。

本章小結

1. 訊息佇列提供解耦（生產者無須知道消費者，故障不會傳染）與緩衝（吸收短暫尖峰）兩大價值；但緩衝只能爭取時間而非增加產能，若長期速率超出處理能力，佇列仍會無限成長。
2. Kafka 的關鍵設計是把佇列實作為持久化的追加式日誌，訊息被讀取而非取走；這使多個消費者能各自獨立讀取、可回退重讀、新消費者亦可從頭讀取歷史。
3. 主題被切分為多個分區分散於各代理，分區是平行處理的單位；每個分區有多個副本並以領導者處理讀寫，故障時從追隨者選出新領導者——此即第 7、8 章原理的具體應用。
4. 同一消費者群組中每個分區只能被一個消費者讀取，故分區數即平行度上限，消費者超過分區數只會閒置；不同群組各自讀取完整資料，因而一套機制同時支援點對點與發布訂閱。
5. 位移由消費者群組自行記錄，帶來重讀與跳過的彈性；而位移的提交時機（處理前或處理後）直接決定了傳遞語意。
6. 三種語意各有取舍：至多一次可能遺失、至少一次可能重複、恰好一次代價高昂；實務主流是「至少一次配合冪等的消費端」，因為端到端的正確性終究仍須靠冪等設計來保障。
7. 事件驅動架構以「宣告事件」取代「直接呼叫」，使新增下游無須修改上游；事件溯源更進一步以事件序列為真實來源、狀態為衍生品，提供完整稽核與時光回溯能力。CDC 與事件溯源殊途同歸，共同體現「表與流本是一體兩面」。
8. 淨消化速率為 $n \times C - P$ （式 25-1），必須為正落後量才會下降；追趕時間為 L 除以淨消化速率（式 25-2）。容量規劃應以尖峰速率為準並保留餘裕，且務必確認分區數足夠，否則增加消費者無效。

習題

一、觀念題

1. 請說明訊息佇列的解耦與緩衝價值，並解釋為何「緩衝只能爭取時間，不能增加產能」。
2. Kafka 把佇列實作為「持久化日誌」而非「取走即刪」，這個設計帶來哪三個好處？
3. 為什麼「消費者數量不能超過分區數」？這條規則的用意何在？違反時會發生什麼？
4. 請說明位移的提交時機如何決定傳遞語意，並解釋為何業界主流選擇「至少一次配合冪等」。

二、計算題

1. 某系統的生產速率 $P = 36,000$ 筆／秒，單一消費者處理速率 $C = 9,000$ 筆／秒。（一）計算「剛好跟得上」所需的最少消費者數；（二）若配置 6 個消費者，以式 25-1 計算淨消化速率；（三）若目前積壓 $L = 2,160$ 萬筆，以式 25-2 計算追趕時間（以分鐘表示）。
2. 承上題，若因尖峰使生產速率上升至 $45,000$ 筆／秒。（一）6 個消費者的淨消化速率變為多少？（二）此時系統能否消化積壓？（三）若要維持至少 $20,000$ 筆／秒的淨消化速率，需要幾個消費者？此時分區數至少應為多少？

三、思考與討論

1. 本章指出 Kafka 的每個設計幾乎都能在前面章節找到源頭。請至少舉出三項並說明其對應的章節與原理。
2. 案例二中，團隊因分區數不足而白白付了三倍資源。請討論：在設計串流系統時，還有哪些「一開始就要想清楚、事後難以更改」的決策？你會如何在專案初期就妥善規劃它們？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

解耦：生產者無須知道消費者是誰、在哪裡、是否存活，只負責寫入佇列；故下游故障不會傳染至上游（如案例一），且新增消費者無須修改生產端。緩衝：當上游產生速率短暫超過下游處理能力時，訊息在佇列中堆積而不會壓垮下游或造成遺失。「只能爭取時間」的原因：緩衝區的作用是吸收「短暫的」波動，讓下游有時間追上；但若上游的長期平均速率持續超過下游的處理能力，落後量將單調遞增，佇列只會無限成長而終將耗盡儲存空間。換言之，緩衝改變的是「何時處理」而非「能處理多少」——這正是第 2 章利特爾法則的意涵。

第 2 題

三個好處：其一，多個消費者可各自獨立讀取同一份資料（因訊息不會因某人讀取而消失），這使發布訂閱模式得以自然實現。其二，消費者可回退重讀——例如處理邏輯有臭蟲修正後，把位移調回去即可重新處理（如案例四）。其三，新加入的消費者可從頭讀取歷史資料，快速建立完整狀態，無須另行匯入。可補充：此「只追加、不修改」的設計思想與第 4 章 LSM-tree、第 5 章 HDFS 一次寫入多次讀取、第 11 章 RDD 不可變性一脈相承。

第 3 題

規則內容：在同一消費者群組中，每個分區只能被一個消費者讀取。用意是保證「順序」——同一分區內的訊息會被依序處理，若允許兩個消費者同時讀取同一分區，將無法確保處理順序，可能導致狀態錯亂（例如「先扣款後退款」被處理成相反順序）。違反時的後果：消費者數量超過分區數時，多出來的消費者不會分配到任何分區，將完全閒置——它們仍消耗運算資源與費用，卻對吞吐量毫無貢獻（如案例二的 8 個閒置消費者）。故分區數即平行度的上限。

第 4 題

提交時機與語意的關係：若在「處理之前」提交位移，則處理失敗時該訊息已被標記為讀過而不會重試，可能遺失——即至多一次。若在「處理之後」才提交，則處理成功但提交前當機時，重啟後會重讀該訊息，可能重複——即至少一次。業界選擇至少一次的理由：其一，實作簡單、效能好；其二，遺失資料通常比重複處理嚴重得多；其三，重複問題可在消費端以冪等性化解（如以唯一識別碼覆寫而非累加）。至於恰好一次，即使佇列層保證了，消費端寫入外部系統的動作仍可能重複，故端到端的正確性終究仍須靠冪等設計——與其追求複雜機制，不如把力氣花在讓處理邏輯冪等。

二、計算題

第 1 題

$P = 36,000$ 、 $C = 9,000$ 。

- (1) 剛好跟得上需 $n \times C \geq P$ ，即 $n \geq 36,000 / 9,000 = 4$ 。故最少需 4 個消費者（此時淨消化速率為 0，僅能持平而無法消化積壓）。
- (2) $n = 6$ ：淨消化速率（式 25-1） $= 6 \times 9,000 - 36,000 = 54,000 - 36,000 = 18,000$ 筆／秒。
- (3) 追趕時間（式 25-2） $= 21,600,000 / 18,000 = 1,200$ 秒 $= 20$ 分鐘。

第 2 題

生產速率上升至 $P = 45,000$ ， C 仍為 9,000。

- (1) $n = 6$ 的淨消化速率 $= 6 \times 9,000 - 45,000 = 54,000 - 45,000 = 9,000$ 筆／秒。
- (2) 能否消化積壓：可以，因淨值仍為正（9,000 筆／秒）。但餘裕已大幅縮小——原本 18,000 降為 9,000，僅剩一半；若生產速率再上升 20% 至 54,000，淨值即歸零，系統將無法追趕。
- (3) 要維持淨消化速率至少 20,000：需 $n \times 9,000 - 45,000 \geq 20,000$ ，即 $n \times 9,000 \geq 65,000$ ， $n \geq 7.22$ ，故至少需 8 個消費者。分區數至少應為 8（否則多出的消費者將閒置）。

延伸思考：本題呈現了叫獸提醒的核心警告—— $n = 6$ 在平時（ $P = 36,000$ ）有 18,000 的充裕餘裕，看似安全；但尖峰時（ $P = 45,000$ ）餘裕腰斬。故容量規劃必須以尖峰速率為基準，而非平均值。

三、思考與討論

第 1 題

可舉的項目包括：其一，分區（partition）——源自第 7 章的資料分區，同樣以鍵決定落點、同樣面臨熱點與再平衡問題，且增加分區會改變鍵的分配（第 7 章的取餘數難題）。其二，副本與領導者選舉——源自第 7 章的主從複製與第 8 章的共識演算法，以領導者處理讀寫、追隨者同步，故障時選出新領導者。其三，追加式日誌——與第 4 章的 LSM-tree、第 5 章 HDFS 的「一次寫入多次讀取」、第 11 章 RDD 的不可變性同源，皆放棄就地修改以換取簡單、高效與容錯。其四，冪等性——第 3 章的逾時重試兩難與第 13 章的管線第一原則，在此以「至少一次配合冪等消費端」再度出現。其五，寫入需幾個副本確認的設定——即第 6 章法定人數與第 3 章一致性可用性取舍的具體化。周延的回答會指出：Kafka 並非全新發明，而是把成熟的分散式系統原理，組裝成專為資料流設計的系統。

第 2 題

其他「事後難改」的決策包括：其一，分區鍵的選擇——決定了資料如何分布與順序保證的範圍，更改將破壞既有的順序語意與狀態分割（呼應第 7 章的熱點問題）。其二，訊息格式與綱要——下游眾多時，格式變更需所有消費者配合，故應自始採用支援綱要演進的格式（第 4 章的 Avro）並建立資料契約。其三，保留期限設定——決定了可重播的時間範圍，過短則失去重播能力，過長則儲存成本高昂。其四，傳遞語意與冪等設計——若消費端一開始未設計為冪等，日後要加上往往需重構整個處理邏輯。其五，主題的切分粒度——過細則管理複雜，過粗則消費者被迫接收無關資料。初期規劃的做法：以「預期尖峰吞吐量」與「未來兩至三年成長」估算分區數並預留餘裕；自始採用具綱要的格式並建立版本管理；把冪等性列為消費端的設計準則而非事後補救；並以文件明確記錄這些決策的理由，供後續維護者參考。

台灣自編大學教科書
@prof
大數據分析與雲端運算

第陸篇 串流處理與即時分析

Stream Processing & Real-time Analytics

第 26 章

串流處理引擎

Stream Processing Engines

機器人叫獸（李世淵） 編著

叫獸開講

我先問你一個問題，看起來很簡單：「上午十點到十一點之間，產線發生了幾次異常？」

你可能覺得這還不容易，把那一小時的資料數一數就好。但問題來了——你要在什麼時候數？

十一點整就數嗎？可是產線邊緣的某台機器網路不穩，它在十點五十九分產生的那筆異常紀錄，可能要到十一點零三分才傳到你手上。你十一點數的話，就漏掉了它。

那等到十一點零五分再數？那如果有一筆資料卡在網路上，十一點十分才到呢？等到十一點半？那如果有台機器離線了兩小時，重連後才把積壓的資料全部送上來呢？

你會發現這是一個永遠等不完的問題。而且更麻煩的是：你等得越久，答案越完整，但也越沒有用——一個要等到中午才知道的「十點到十一點的異常數」，還算是即時分析嗎？

這一章的核心，就是這個「時間」的難題。批次處理沒有這個問題，因為資料早就到齊了；但在串流世界裡，「資料何時到達」與「事件何時發生」是兩件不同的事，而處理這個落差，是串流引擎最困難也最精妙的部分。

學習目標

研讀完本章之後，你應該能夠：

1. 說明批次與串流統一的思想，並解釋無界表的概念。
2. 描述 Spark Structured Streaming 的微批次模型與其取捨。
3. 說明 Flink 的逐筆處理模型與狀態管理機制。
4. 區分事件時間與處理時間，並說明三種視窗的差異。
5. 說明水位線如何在延遲與完整性之間取捨。
6. 比較 Lambda 與 Kappa 兩種架構的設計理念。

核心概念

核心概念	說明
無界表	把持續到達的串流視為一張「不斷追加列」的資料表。
微批次	把串流切成極小的批次依序處理，以批次引擎實現串流。
事件時間	事件實際發生的時間，記錄在資料本身。
處理時間	資料被系統處理的時間，受網路與負載影響。
視窗	把無界的串流切成有界區間以進行彙總的機制。
水位線	系統對「事件時間已推進到何處」的估計，用以判定視窗何時可關閉。
Lambda 與 Kappa	兩種串流架構：前者批次與串流並行，後者僅用串流。

表 26-1 本章核心概念一覽

26.1 批次與串流的統一

26.1.1 無界表的洞見

早期的串流處理與批次處理是兩套完全不同的程式模型——同一個分析邏輯要寫兩次，一次給批次、一次給串流，而且兩邊的口徑常常對不上。

突破來自一個優雅的抽象：把串流視為「一張無界的表」。這張表與一般資料表的差別只有一個——它會不斷有新的列被追加進來，永遠不會結束。

這個抽象的威力在於：一旦串流被視為表，那麼所有你熟悉的表操作（篩選、彙總、關聯）就都能直接套用。你寫的查詢與批次查詢在語法上完全相同，差別只在於——批次查詢執行一次就結束，串流查詢則會隨著新資料到達而持續更新結果。

這正是第 25 章末提到的「表流二元性」的另一面：那裡我們說「一連串事件可以推導出狀態」，這裡則是「持續增長的表就是一條串流」。同一個洞見的兩種表述。

26.1.2 兩種執行模型

有了統一的程式模型，底層的執行方式仍有兩種截然不同的路線。

面向	微批次	逐筆處理
處理單位	把串流切成極小的批次	每筆資料獨立處理
延遲	受批次間隔限制，通常數百毫秒起	可達毫秒級
吞吐量	高（批次處理有規模效益）	高，但單筆開銷較大
容錯	沿用批次的重算機制，較簡單	需維護狀態快照，較複雜
代表	Spark Structured Streaming	Apache Flink

表 26-2 微批次與逐筆處理的比較

這個選擇沒有絕對優劣，關鍵在於你的延遲需求。若能接受數百毫秒到數秒的延遲（多數分析與監控場景都可以），微批次的簡單與高吞吐相當有吸引力；若需要毫秒級的反應（如高頻交易、即時控制），則必須採用逐筆處理。

26.2 Spark Structured Streaming

26.2.1 把串流當成不斷成長的表

Structured Streaming 是 Spark 對串流處理的答案，它完整體現了 26.1.1 節的無界表思想。

在這個模型中，你用第 12 章學過的 DataFrame API 撰寫查詢，而引擎會把它轉換為一系列的增量計算——每當有新資料到達，就以微批次的方式更新結果。

這帶來一個實務上的巨大好處：你在第 12 章學的所有東西都能直接用上。同樣的 DataFrame 操作、同樣的 SQL 語法、同樣的 Catalyst 最佳化器——這使得從批次轉向串流的學習成本極低，也讓同一套邏輯能同時用於歷史資料回填與即時處理。

26.2.2 三種輸出模式

串流查詢的結果會持續更新，因此需要決定「每次更新要輸出什麼」。這有三種模式：

- **完整模式**：每次都輸出完整的結果表。適合結果集小的彙總（如「各機台的累計異常數」），但結果集大時成本高昂。

- **追加模式**：只輸出「新增的列」。適合不會回頭修改舊結果的查詢，例如已關閉的視窗彙總。
- **更新模式**：只輸出「有變動的列」。介於兩者之間，是最常用的模式。

選擇哪一種，取決於你的查詢性質與下游系統的能力。若下游只能追加（如寫入檔案），就只能用追加模式；若下游支援依鍵更新（如資料庫），則更新模式更有效率。

26.2.3 微批次的取舍

微批次的核心取舍在於批次間隔的設定。間隔越短延遲越低，但每個批次的固定開銷（任務排程、狀態存取）佔比就越高，吞吐量下降；間隔越長則相反。

這裡有個常被誤解的地方：微批次的延遲下限，並不只是批次間隔，還要加上該批次本身的處理時間。若你設定 1 秒的間隔，但每批要處理 3 秒，那批次會不斷積壓，延遲將持續累積——這與第 25 章的消費者落後量是同一個道理。

因此設定批次間隔的原則是：必須大於「該間隔內的資料量所需的處理時間」，並保留餘裕。這個關係可以用第 25 章的淨消化速率來檢查——若處理速率跟不上生產速率，再短的間隔也沒有意義。

26.3 Apache Flink 與狀態管理

26.3.1 為串流而生

與 Spark「以批次引擎實現串流」的路線不同，Flink 從一開始就是為串流設計的——它把批次視為「有界的串流」，也就是串流的特例。這個立場恰好與 Spark 相反，卻同樣達成了批流統一。

逐筆處理的最大優勢是延遲：資料一到就處理，不必等待批次邊界，因此能達到毫秒級的反應。這使它在需要極低延遲的場景中佔有優勢。

26.3.2 狀態：串流處理的核心難題

串流處理與批次最根本的差異之一，是「狀態」。

許多串流計算需要記住過去的資訊：計算移動平均需要記住視窗內的資料、偵測「同一機台連續三次異常」需要記住前兩次的狀態、關聯兩條串流需要暫存等待配對的記錄。這些都是狀態。

狀態帶來兩個難題。其一是規模：狀態可能非常大（想像為數百萬個裝置各維護一組狀態），必須有效管理。其二是容錯：若處理節點故障，狀態必須能夠恢復，否則計算結果就錯了。

Flink 的解法是「檢查點」（checkpoint）機制——定期把所有運算子的狀態一致地快照下來並持久化。故障時，系統回到最近的檢查點，重設狀態並從對應的位置重新消費資料。

這裡的關鍵字是「一致地」。因為串流是分散處理的，各節點的進度不同，若隨意各自快照，恢復時會出現有的節點多算、有的少算的情況。Flink 採用一種在資料流中插入「屏障」（barrier）標記的演算法，確保所有節點快照的是「同一個邏輯時間點」的狀態——這個思想與第 8 章的分散式一致性問題一脈相承。

26.3.3 狀態的保存位置

狀態可以保存在記憶體（最快但受容量限制且節點故障即失）、或以嵌入式的鍵值儲存保存在本機磁碟（容量大且可支撐超出記憶體的狀態，代價是存取較慢）。後者常搭配第 4 章談過的 LSM-tree 引擎，因為狀態更新是寫入密集的操作。

無論保存在哪裡，檢查點都會被寫入可靠的分散式儲存（如第 5 章的 HDFS 或物件儲存），以確保節點完全失效時仍能恢復。

26.4 視窗、事件時間與水位線

這一節是本章的核心，也是叫獸開講那個難題的正面回答。

26.4.1 兩種時間

串流處理必須嚴格區分兩種時間，混淆它們是初學者最常見的錯誤。

事件時間（event time）是「事件實際發生的時間」，通常由產生資料的裝置寫在資料裡。例如感測器在 10:59:58 讀到異常，這個時間戳就記錄在該筆資料中。

處理時間（processing time）是「系統處理到這筆資料的時間」。同一筆資料可能因為網路延遲、系統負載、裝置離線等因素，在 11:03:20 才被處理。

兩者的差距造成了兩個現象：「亂序」（後發生的事件可能先到達）與「遲到」（事件在其所屬視窗已經關閉後才到達）。而正確的分析幾乎總是應該基於事件時間——因為「十點到十一點發生了幾次異常」問的是事件何時發生，而非系統何時處理。

26.4.2 三種視窗

視窗是把無界串流切成有界區間以便彙總的機制，常見有三種。

視窗類型	定義	適用情境
滾動視窗	固定長度、彼此不重疊，如每 5 分鐘一個	定期報表，如「每小時的產量」
滑動視窗	固定長度但重疊，如每分鐘計算過去 5 分鐘	移動平均、趨勢監控
工作階段視窗	依活動間隔動態切分，靜止超過閾值即結束	使用者行為分析、設備運轉週期

表 26-3 三種視窗類型

其中工作階段視窗最為特別——它的長度不固定，而是由資料本身的間隔決定。例如把「連續操作、間隔不超過 30 分鐘」視為同一個工作階段。這在分析設備的連續運轉週期時很實用。

26.4.3 水位線：何時該關閉視窗

現在回答叫獸開講的問題：視窗到底什麼時候該關閉、輸出結果？

水位線（watermark）是系統對「事件時間已經推進到哪裡」的估計。它的意義是一個承諾：「我認為事件時間早於 T 的資料，應該都已經到齊了」。當水位線超過某個視窗的結束時間時，該視窗就被關閉並輸出結果。

水位線通常由「目前看過的最大事件時間，減去一個容許延遲」來計算。例如設定容許延遲為 10 秒，當系統看到事件時間 11:00:30 的資料時，水位線就推進到 11:00:20——意味著它認為 11:00:20 之前的資料都到齊了，可以關閉結束時間在此之前的視窗。

這個「容許延遲」的設定，就是延遲與完整性之間的取舍旋鈕：設得越大，等待越久、遲到資料越少被漏掉，但結果越晚產出；設得越小則相反。下一節的公式會把這個取舍量化。

遲到資料的處理

即使設了容許延遲，仍可能有資料在視窗關閉後才到達。處理方式有三種：直接丟棄（最簡單，但會低估）、送入「側輸出」另行處理（保留但不影響主結果）、或允許視窗更新（重新計算並修正已輸出的結果，代價是下游必須能處理修正）。

叫獸提醒

我要強調一個實務上極重要的觀念：水位線是一個「估計」，不是「保證」。它只是說「我猜這之前的都到了」，而不是「這之前的一定都到了」。所以任何基於視窗的串流結果，本質上都帶有不確定性——你必須接受它可能不完整。如果你的應用絕對不能有誤差（例如財務結算），那正確的做法不是把容許延遲設到很長（那會讓即時性完全喪失），而是採用「串流給即時的近似值、批次給最終的準確值」的雙軌設計——這正是 26.6 節 Lambda 架構的理由。認清「即時」與「精確」在串流世界中無法兼得，是這一章最重要的成熟認知。

26.5 Exactly-once 語意

26.5.1 串流的一致性挑戰

第 25 章我們談過訊息傳遞的三種語意。在串流處理引擎中，這個問題更複雜——因為除了訊息本身，還牽涉到「狀態」與「輸出」。

設想一個場景：引擎讀取訊息、更新內部狀態（例如計數器加一）、寫出結果。若在寫出後、記錄進度前當機，重啟後會重讀該訊息，於是狀態被加了兩次、結果也被寫了兩次。

因此串流的 exactly-once 必須同時保證三件事的一致：讀取的位置、內部狀態、以及輸出的結果。這三者必須「同進同退」——恢復時要嘛全部回到某個一致的時點，要嘛全部前進。

26.5.2 實作機制

主流的做法是把「檢查點」與「位移」綁在一起。檢查點中同時記錄了狀態快照與對應的消費位移，故障恢復時兩者一起回退，確保狀態與已讀取的資料完全對應。

但這只解決了「引擎內部」的問題。輸出到外部系統仍可能重複——因為外部系統不受引擎的檢查點管轄。要達成真正的端到端 exactly-once，需要下列條件之一：

- **交易式寫入：**輸出與檢查點納入同一個交易，一起提交或一起回滾。這需要外部系統支援交易。
- **冪等寫入：**以確定性的鍵覆寫而非累加，使重複寫入不改變結果——這正是第 13、25 章反覆強調的原則。

實務上，冪等寫入是更常見也更務實的選擇。它不需要外部系統的特殊支援，且能同時應付重播、回填等各種情境。我要再次強調：與其追求引擎層面複雜的 exactly-once 機制，不如把力氣花在讓輸出邏輯冪等——這是本書從第 3 章一路強調到現在的原則。

26.6 Lambda 與 Kappa 架構

26.6.1 Lambda：雙軌並行

Lambda 架構的設計理念，正是回應 26.4.3 節那個「即時與精確不可兼得」的困境。它的做法是：不要選，兩個都做。

具體而言，系統維護兩條並行的路徑。「速度層」以串流引擎處理即時資料，快速產出近似的結果；「批次層」則定期以批次處理完整的歷史資料，產出精確的結果。查詢時，把兩者合併——用批次層的精確結果作為主體，再補上速度層對「批次尚未涵蓋的最近期間」的估計。

這個設計解決了正確性問題：即使串流因遲到資料而有誤差，批次層最終會修正它。但它有一個廣受詬病的代價——同一套業務邏輯必須實作兩次（一次串流版、一次批次版），不僅開發成本加倍，兩邊的邏輯還可能因為維護不同步而產生不一致。

26.6.2 Kappa：只要串流

Kappa 架構提出了一個大膽的主張：既然訊息佇列能夠保留並重播歷史資料（第 25 章），我們何必還要批次層？

它的做法是：只維護一條串流處理路徑。平時處理即時資料；當需要重新計算歷史（因為邏輯修正或發現錯誤）時，就把消費位移調回起點，用同一套串流程式重跑一遍歷史資料。

這徹底消除了「同一邏輯寫兩次」的問題——只有一套程式碼、一套邏輯、一個口徑。代價則是：需要佇列保留足夠長的歷史（儲存成本），且重播大量歷史資料的運算成本可能很高。

面向	Lambda	Kappa
路徑	批次層與速度層並行	僅串流一條路徑
程式碼	同一邏輯須實作兩次	只有一套
一致性風險	兩套邏輯可能不同步	無此問題
歷史重算	由批次層定期執行	調整位移重播串流
前提條件	無特殊要求	佇列須保留足夠長的歷史

表 26-4 Lambda 與 Kappa 架構的比較

實務上該選哪個？我的建議是：若你的串流引擎已足夠成熟可靠、且佇列能保留所需的歷史，Kappa 的簡潔性通常勝出。但若你的場景對精確性有絕對要求（如財務結算、法規報表），Lambda 那條獨立的批次驗證路徑仍有其價值——它提供了一個「最終真相」的來源。

26.6.3 承先啟後

本章我們正面處理了串流最困難的部分——時間。從無界表的統一抽象出發（26.1），認識了 Spark 微批次與 Flink 逐筆處理兩條路線（26.2、26.3）；接著深入事件時間與處理時間的區分、三種視窗、以及水位線如何在延遲與完整性之間取捨（26.4）；再看串流的 exactly-once 需同時保證讀取位置、狀態與輸出的一致（26.5），最後比較了 Lambda 與 Kappa 兩種架構哲學（26.6）。

到這裡，第陸篇「串流處理與即時分析」三章圓滿收束。回顧這一篇：第 24 章教你在有限記憶體下處理無限資料流、第 25 章讓資料能可靠地流動、本章則讓你能在資料流過的當下完成分析。批次與串流這兩種思維，你現在都具備了。

不過，前面六篇我們談的都是「怎麼做」——怎麼存、怎麼算、怎麼分析、怎麼即時處理。但這些系統要跑在哪裡？誰來提供機器、誰來擴充、誰來確保它們不會因為一台機器故障就全垮？第 1 章

我們談過雲端運算的概念，但一直沒有深入。接下來的第柒篇，我們要正式進入雲端的世界——虛擬化、容器、Kubernetes，以及現代資料架構的核心：資料湖與 Lakehouse。

公式與流程：水位線延遲與完整性的取舍

這一節要把叫獸開講那個「該等多久」的問題，變成一個可以計算的工程決策。

完整性的定義

設某視窗的容許延遲（水位線延遲）為 D ，而資料的延遲分布為 F （即 $F(D)$ 表示「延遲不超過 D 的資料所佔比例」）。則該視窗在關閉時的完整性為：

$$\text{完整性} = F(D)$$

式 26-1 視窗關閉時的資料完整性

相對地，被漏掉的遲到資料比例為 $1 - F(D)$ 。而視窗結果產出的總延遲則為：

$$\text{結果延遲} = \text{視窗長度} + D + \text{處理時間}$$

式 26-2 結果產出的總延遲

這兩條式子清楚地呈現了取舍： D 增加會提升完整性（式 26-1），但也直接增加結果的延遲（式 26-2）。

實際試算

假設某系統的資料延遲呈長尾分布（多數資料很快到達，少數因網路或裝置問題延遲甚久），實測的延遲分布如下表。

容許延遲 D	完整性 $F(D)$	漏失比例	每百萬筆漏失
1 秒	62.0%	38.0%	380,000 筆
2 秒	85.0%	15.0%	150,000 筆
5 秒	96.8%	3.2%	32,000 筆
10 秒	99.2%	0.8%	8,000 筆
30 秒	99.93%	0.07%	700 筆
60 秒	99.995%	0.005%	50 筆

表 26-5 容許延遲與資料完整性的關係

這張表揭示了長尾分布的典型特徵：報酬遞減。從 1 秒增為 5 秒（多等 4 秒），完整性從 62% 大幅躍升至 96.8%——這 4 秒非常值得。但從 30 秒增為 60 秒（多等 30 秒），完整性只從 99.93% 提升到 99.995%，每百萬筆僅多救回 650 筆——這 30 秒的代價通常不值得。

因此實務上的做法是找出那個「膝點」：在本例中，5 到 10 秒是相當合理的選擇——完整性已達 97% 至 99%，而延遲仍在可接受範圍。

叫獸提醒

表 26-5 給你的最重要啟示是：不要憑感覺設水位線延遲，要用實測的延遲分布來決定。我看過太多人隨手填一個「10 秒」或「1 分鐘」，卻從沒量過自己系統的資料到底延遲多久。正確的做法是：先收集一段時間的資料，計算每筆的「處理時間減事件時間」，畫出這個延遲的

分布，再依你能接受的完整性（例如 99%）反推出所需的 D。而且要注意——這個分布會隨網路狀況、裝置數量與系統負載而改變，應該定期重新量測。另外提醒一點：那 0.8% 的遲到資料不該直接丟棄，而應送入側輸出記錄下來。因為當這個比例突然上升時，往往是某台裝置或某段網路出了問題——遲到率本身就是一個有價值的健康指標。

案例研討

案例一 少算的那一小時

回到叫獸開講。某工廠的即時儀表板顯示「每小時異常次數」，但操作員發現一個怪現象：儀表板顯示的數字，總是比隔天的批次報表少了約 3%。

追查後發現，儀表板的串流查詢是以「處理時間」分組的——資料何時被處理，就算在哪個小時。因此那些在 11:03 才到達、但實際發生在 10:59 的異常，被錯誤地計入了 11 點那個小時。

他們改為以「事件時間」分組並設定 15 秒的水位線延遲後，數字與批次報表的差距降到 0.1% 以下。這個案例是 26.4.1 節最直接的印證：串流分析幾乎總是應該基於事件時間，因為問題問的是「事件何時發生」，而非「系統何時處理」。

案例二 等了一分鐘的告警

某團隊為求資料完整，把水位線的容許延遲設為 60 秒，視窗長度則為 1 分鐘。上線後維護人員抱怨：異常告警總是在事發後兩分多鐘才跳出來，等他們趕到現場，不良品已經生產了一大批。

依式 26-2 計算：結果延遲 = 視窗長度 60 秒 + 容許延遲 60 秒 + 處理時間，總計超過兩分鐘——與抱怨完全吻合。而依表 26-5，60 秒的完整性是 99.995%，相較 5 秒的 96.8% 其實只多救回 3% 的資料。

他們把容許延遲改為 5 秒、視窗縮短為 30 秒，告警延遲降至約 40 秒，及時性大幅改善，而完整性的損失在告警場景中完全可以接受（因為告警的目的是「及早發現」而非「精確統計」）。這個案例說明：水位線的設定必須依「這個結果要拿來做什麼」而定，沒有一個放諸四海的最佳值。

案例三 當機後重算的計數器

某串流作業維護著各機台的累計異常計數。某次節點故障後重啟，團隊發現部分機台的計數明顯偏高——重複計算了。

原因是他們的實作把「內部狀態」與「消費位移」分開管理：故障時狀態回到了檢查點，但位移的記錄較新，導致部分資料被重複處理而狀態被重複累加。

修正方式是採用引擎內建的檢查點機制，把狀態快照與位移綁在同一個檢查點中一起提交——這正是 26.5.2 節所說的「三者必須同進同退」。同時他們也把輸出改為冪等寫入（以「機台加時間視窗」為主鍵覆寫，而非累加），使即使發生重複處理，最終結果仍然正確。這個案例把第 13、25 章的冪等原則，在串流狀態管理的場景中又演練了一次。

案例四 兩套邏輯的分歧

某企業採用 Lambda 架構，串流層與批次層各自實作了同一套「有效交易」的判定邏輯。半年後，業務規則調整，開發人員修改了批次層的程式，卻遺漏了串流層。

此後即時儀表板與隔日的正式報表開始出現差異，且因為兩者本來就允許有微小落差（串流是近似值），這個系統性的偏差有整整三週沒被發現，直到財務單位對帳時才揭露。

這正是 26.6.1 節所指出的 Lambda 代價——「同一邏輯寫兩次」不只是開發成本加倍，更是長期的一致性風險。他們後來評估轉向 Kappa 架構，把批次層廢除，改以重播串流來處理歷史重算，徹底消除了兩套邏輯分歧的可能。這個案例也呼應了第 13 章的教訓：系統性的錯誤若不會觸發任何告警，往往要很久才會被發現。

實作活動

活動一 測量你的延遲分布

請實際量測資料延遲，而非憑感覺設定水位線。

- 步驟 1.** 設計一個資料產生器，在每筆資料中寫入事件時間戳，並刻意讓少數資料延遲送出（模擬長尾分布）。
- 步驟 2.** 在消費端記錄每筆資料的「處理時間減事件時間」，收集至少一萬筆。
- 步驟 3.** 計算延遲的分布，並找出對應於 90%、95%、99%、99.9% 完整性的延遲值。
- 步驟 4.** 依式 26-2 計算，若視窗長度為 1 分鐘，這四種設定各會造成多少結果延遲？你會選哪一個？為什麼？

活動二 實作事件時間視窗

請以 Spark Structured Streaming 或 Flink 實作一個含水位線的串流查詢。

- 步驟 1.** 建立一個串流來源（可從第 25 章的 Kafka 讀取，或使用引擎內建的測試來源）。
- 步驟 2.** 以「處理時間」分組計算每分鐘的事件數，觀察遲到資料被歸入哪個視窗。
- 步驟 3.** 改為以「事件時間」分組並設定 10 秒的水位線延遲，比較兩者的結果差異。
- 步驟 4.** 刻意送出一筆延遲 30 秒的資料，觀察它是否被納入正確的視窗，或被判定為遲到而丟棄。

活動三 驗證狀態的容錯

請驗證串流引擎的檢查點機制。實作一個有狀態的串流查詢（例如維護各鍵的累計計數），啟用檢查點並設定合理的間隔。執行一段時間後，刻意中止程式（模擬當機），然後重新啟動。觀察並記錄：重啟後計數是從零開始、從檢查點恢復、還是出現重複累加？接著把輸出改為冪等寫入（以鍵覆寫而非累加），重複同樣的實驗，確認最終結果的正確性。最後撰寫一段說明：檢查點解決了什麼問題？又有什麼是它解決不了、必須靠冪等設計來處理的？

延伸閱讀

- **《Streaming Systems》**：Akidau 等人著。對事件時間、水位線與視窗有最完整深入的闡述，是本章 26.4 節的權威參考。
- **Google Dataflow 模型論文**：提出以「What、Where、When、How」四個問題統一描述串流計算的框架，是理解水位線設計思想的源頭。
- **Apache Flink 官方文件**：說明狀態管理、檢查點機制與 exactly-once 的實作細節，是 26.3、26.5 節的實務參考。
- **Spark Structured Streaming 程式設計指南**：涵蓋輸出模式、水位線設定與與 Kafka 整合的完整範例。

本章小結

1. 把串流視為「不斷追加列的無界表」，使批次與串流得以共用同一套程式模型；此即第 25 章「表流二元性」的另一面表述。
2. 執行模型分微批次（Spark，延遲數百毫秒起、容錯簡單）與逐筆處理（Flink，可達毫秒級、狀態管理較複雜）兩條路線；選擇取決於延遲需求，多數分析與監控場景微批次已足夠。
3. 微批次的批次間隔必須大於該間隔內資料所需的處理時間，否則批次會積壓、延遲持續累積——這與第 25 章的消費者落後量是同一個道理。
4. 狀態是串流處理的核心難題（規模大且需容錯）；Flink 以在資料流中插入屏障的方式，確保所有節點快照的是同一邏輯時間點的狀態，其思想與第 8 章的分散式一致性一脈相承。
5. 必須嚴格區分事件時間（事件實際發生）與處理時間（系統處理到）；兩者的落差造成亂序與遲到，而正確的分析幾乎總應基於事件時間。視窗分滾動、滑動與工作階段三種。
6. 水位線是系統對事件時間推進程度的「估計」而非「保證」，其容許延遲即延遲與完整性的取舍旋鈕；遲到資料可丟棄、送側輸出或觸發視窗更新。
7. 串流的 exactly-once 須同時保證讀取位置、內部狀態與輸出三者一致；檢查點綁定位移可解決引擎內部問題，但端到端仍須靠交易式寫入或（更務實的）冪等寫入。
8. 完整性為 $F(D)$ （式 26-1）、結果延遲為視窗長度加容許延遲加處理時間（式 26-2）；長尾分布下報酬遞減明顯，應以實測延遲分布找出膝點，而非憑感覺設定。Lambda 以雙軌換取精確但同一邏輯須寫兩次，Kappa 僅用串流而以重播處理歷史，前提是佇列保留足夠歷史。

習題

一、觀念題

1. 請說明「無界表」的抽象為何能統一批次與串流的程式模型，並說明它與第 25 章「表流二元性」的關係。
2. 請區分事件時間與處理時間，並說明為何串流分析幾乎總應基於事件時間。
3. 何謂水位線？為什麼說它是「估計」而非「保證」？這對串流結果的正確性有什麼意涵？
4. 串流的 exactly-once 為何比訊息傳遞的 exactly-once 更複雜？端到端的正確性最終該如何保障？

二、計算題

1. 某系統的資料延遲分布為：1 秒內到達 70%、3 秒內 90%、8 秒內 98%、20 秒內 99.5%、60 秒內 99.9%。（一）若要求完整性至少 98%，容許延遲 D 應設為多少？（二）視窗長度為 2 分鐘、處理時間 3 秒時，依式 26-2 計算此設定的結果延遲；（三）若把 D 從 8 秒增為 20 秒，完整性提升多少個百分點？結果延遲增加多少？這樣的交換是否值得？
2. 承上題，某告警場景要求「結果延遲不得超過 45 秒」。（一）若視窗長度為 30 秒、處理時間 3 秒，容許延遲 D 最多可設為多少？（二）此設定下的完整性約為多少？（三）每百萬筆資料約會漏失幾筆？

三、思考與討論

1. 本章指出「即時與精確在串流世界中無法兼得」。請就一個你熟悉的應用情境討論：你會如何在兩者間取舍？是否需要 Lambda 那樣的雙軌設計？
2. 案例四中，兩套邏輯的分歧經過三週才被發現。請討論：若必須採用 Lambda 架構，你會設計哪些機制來及早發現兩層之間的不一致？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

無界表把串流視為「一張會不斷追加列、永不結束的表」。一旦串流被視為表，所有既有的表操作（篩選、彙總、關聯、SQL）就都能直接套用，故同一套查詢語法可同時用於批次與串流——差別僅在批次查詢執行一次即結束，串流查詢則隨新資料到達持續更新結果。與第 25 章的關係：那裡的表述是「一連串事件可推導出狀態（表）」，這裡是「持續增長的表就是一條串流」——兩者是同一個「表流二元性」洞見的兩面。此二元性也是 Kappa 架構得以成立的理論基礎。

第 2 題

事件時間是「事件實際發生的時間」，通常由產生資料的裝置寫在資料中；處理時間是「系統處理到該筆資料的時間」，受網路延遲、系統負載、裝置離線等因素影響。應基於事件時間的原因：分析問題問的幾乎總是「事件何時發生」而非「系統何時處理」——例如「十點到十一點發生幾次異常」，若以處理時間分組，10:59 發生但 11:03 才到達的異常會被錯誤計入 11 點那一小時（如案例一）。此外，以處理時間分組會使同一份資料在不同時間重跑時得到不同結果，喪失可重現性。

第 3 題

水位線是系統對「事件時間已推進到何處」的估計，其意義是一個承諾：「我認為事件時間早於 T 的資料應該都已到齊」，當水位線超過視窗結束時間時該視窗即關閉輸出。之所以是估計而非保證，是因為它通常由「已看過的最大事件時間減去一個容許延遲」推算，而系統無法真正得知是否還有更早的資料尚未抵達（例如某裝置離線兩小時後才回傳）。意涵：任何基於視窗的串流結果本質上都帶有不確定性，可能不完整。故若應用絕對不能有誤差，不應把容許延遲設到極長（那將喪失即時性），而應採「串流給即時近似、批次給最終精確」的雙軌設計（Lambda）。

第 4 題

更複雜的原因：訊息傳遞只需保證「訊息被處理一次」，而串流處理還牽涉「內部狀態」與「輸出結果」——三者必須同進同退。例如引擎在寫出結果後、記錄進度前當機，重啟後會重讀該訊息，導致狀態被重複更新且結果被重複寫出。實作上把狀態快照與消費位移綁在同一個檢查點中一起提交，可解決引擎內部的一致性；但輸出到外部系統不受檢查點管轄，仍可能重複。端到端的保障方式有二：交易式寫入（需外部系統支援交易），或冪等寫入（以確定性的鍵覆寫而非累加）。後者更務實，因不需外部系統特殊支援，且能同時應付重播與回填——這是本書自第 3 章一路強調的原則。

二、計算題

第 1 題

依給定的延遲分布查表與計算。

- (1) 要求完整性至少 98%：查分布可知 8 秒內到達 98%，故 D 應設為 8 秒。
- (2) 結果延遲（式 26-2）= 視窗長度 120 秒 + D 8 秒 + 處理時間 3 秒 = 131 秒（約 2 分 11 秒）。
- (3) D 由 8 秒增為 20 秒：完整性由 98% 提升至 99.5%，即提升 1.5 個百分點；結果延遲由 131 秒增為 143 秒，增加 12 秒。

是否值得的判斷：需視用途而定。若此結果用於「趨勢監控或日常報表」，多等 12 秒換取 1.5 個百分點的完整性通常值得；但若用於「即時告警」，多等 12 秒可能延誤處置，而 98% 的完整性對告警目的已足夠（告警要的是及早發現，而非精確統計）。此即案例二的教訓——水位線設定必須依「這個結果要拿來做什麼」而定。

第 2 題

限制條件為結果延遲 ≤ 45 秒，視窗長度 30 秒、處理時間 3 秒。

- (1) 依式 26-2: $30 + D + 3 \leq 45$ ，故 $D \leq 12$ 秒。考量分布的刻度，實務上會設 $D = 8$ 秒（保留餘裕）或至多 12 秒。
- (2) 完整性：若取 $D = 8$ 秒，完整性為 98%；若取 $D = 12$ 秒，介於 98%（8 秒）與 99.5%（20 秒）之間，約可估為 98.5% 至 99%。
- (3) 漏失比例：以 $D = 8$ 秒的 98% 計算，漏失 2%，即每百萬筆約漏失 20,000 筆；若取 $D = 12$ 秒（約 98.7%），則約漏失 13,000 筆。

延伸思考：本題呈現了實務上的常見情境——延遲上限是由業務需求（告警必須在 45 秒內）反過來決定水位線設定，而非先設水位線再看延遲。此時應把漏失的那 2% 資料送入側輸出記錄，一方面保留稽核軌跡，另一方面遲到率的變化本身就是系統健康的指標。

三、思考與討論

第 1 題

答題應包含情境描述、取舍判準與架構選擇。判準可包括：其一，結果的用途——告警與監控重及時性（可接受 95% 至 99% 的完整性），財務結算與法規報表重精確性（須接近 100%）。其二，錯誤的代價——漏報一次異常的代價，相對於延遲處置的代價孰輕孰重。其三，是否可事後修正——若下游能接受結果更新，則可先出近似值再修正，兩者兼得。是否需要雙軌設計的判斷：若同一份資料同時服務「即時告警」與「正式報表」兩種用途，且後者不容誤差，則雙軌（或至少「串流即時 + 批次每日校正」）有其必要；若僅供監控參考，單一串流路徑即足。周延的回答會指出：更現代的做法是採 Kappa 架構加上「可重播修正」——即時輸出近似值，並定期重播歷史以修正，兼得單一邏輯與最終精確。

第 2 題

可設計的機制包括：其一，自動化對帳——每日（或每小時）自動比對批次層與速度層在重疊期間的結果，差異超過門檻即告警。這是最關鍵的一項，因為案例四的問題正是「沒有任何機制會主動發現差異」。其二，共用邏輯核心——把業務規則抽出為共用的函式庫或設定，讓兩層引用同一份定義，而非各自實作；規則變更時只改一處。其三，共用測試集——以同一組測試資料驗證兩層的輸出必須一致，並納入第 23 章的持續整合流程，任一層修改都會觸發比對。其四，變更流程管控——業務規則的修改須同時提出兩層的變更，程式碼審查時明確檢查（呼應第 23 章的同儕審查）。其五，可觀測性——把兩層的差異率做成常態監控指標並繪成趨勢圖，使系統性偏差在累積前就被看見。核心觀念與第 13 章一致：不會觸發告警的系統性錯誤最危險，故必須主動設計「能發現它」的機制。

大數據分析與雲端運算

第柒篇 雲端運算平台與架構

Cloud Platforms & Architecture

第 27 章

雲端運算概論與服務模型

Cloud Computing and Service Models

機器人叫獸（李世淵） 編著

叫獸開講

我想用一個學生常有的困惑開場：「教授，我們實驗室不是有伺服器嗎？為什麼還要用雲端？」

這是個好問題，而我通常用一個反問來回答：你家有車嗎？那你為什麼還會搭計程車？

答案很清楚——因為需求不同。天天通勤，自己有車最划算；但如果一年只有三天要去很遠的地方，買一台車來放著就荒謬了。更何況，買車還要停車位、要保養、要保險、要驗車，而這些成本你在決定買車那一刻往往沒算進去。

雲端與自建的關係，本質上就是這樣。第 1 章我們談過雲端的基本概念，也提過它把「資本支出轉為營運支出」。但那時我沒說清楚一件事：這個轉換划不划算，完全取決於一個數字——你的機器有多少時間真的在工作。

這一章我們要把這件事算清楚。你會看到一個可能違反直覺的結論：如果你的機器真的能持續滿載，自建其實比雲端便宜；但真實世界中，能持續滿載的機器少之又少。而這個「使用率」的落差，正是雲端商業模式成立的根本原因。理解這個數字，你日後做架構決策時就不會人云亦云。

學習目標

研讀完本章之後，你應該能夠：

1. 說明雲端運算的五大特徵，並解釋規模經濟與資源池化的關係。
2. 比較 IaaS、PaaS、SaaS 三種服務模型的責任分界。
3. 區分公有雲、私有雲與混合雲，並判斷其適用情境。
4. 說明雲端的計費模型，並比較隨需、預留與競價三種購買方式。
5. 運用總持有成本模型，計算自建與雲端的損益平衡點。
6. 說明常見的雲端遷移策略，並指出遷移的風險與注意事項。

核心概念

核心概念	說明
資源池化	供應商把資源匯集為共享池，動態分配給多個租戶。
彈性擴縮	資源可依需求快速增減，使用者無須預先購置。
服務模型	IaaS、PaaS、SaaS 三種抽象層次，決定使用者與供應商的責任分界。
部署模型	公有、私有、混合與社群雲四種部署方式。
總持有成本	涵蓋購置、維運、人力與機會成本的完整成本評估。
FinOps	以財務紀律管理雲端支出、使成本可見可控的實務。
遷移策略	把既有系統搬上雲端的各種做法，如重新託管、重構等。

表 27-1 本章核心概念一覽

27.1 雲端運算的本質

27.1.1 五大特徵的再檢視

第 1 章我們列過 NIST 定義的五大特徵：隨需自助服務、廣泛網路存取、資源池化、快速彈性、可計量服務。這裡我要指出其中兩項的深層意義。

「資源池化」不只是技術上的共享，它是整個雲端商業模式的根基。供應商把數萬台機器匯集成一個資源池，租給成千上萬個客戶——而這些客戶的用量尖峰通常不會同時發生。甲公司的月底結帳尖峰、乙公司的週末促銷尖峰、丙研究室的期末運算尖峰，彼此錯開。

這種「錯峰」使得供應商只需準備遠少於「所有客戶尖峰總和」的機器，就能滿足所有人的需求。這正是規模經濟的來源——也是為什麼雲端能以低於你自建的價格，提供你隨叫隨到的彈性。

「可計量服務」則帶來了另一個深遠的影響：成本變得可見。在自建機房中，一支寫得很爛的程式多跑三小時，你感覺不到成本；但在雲端，那三小時會直接出現在帳單上。這種可見性既是壓力，也是改善的動力——它讓效能優化第一次有了明確的財務意義。

27.1.2 彈性的真正價值

很多人以為雲端的價值是「便宜」，這其實是個誤解。雲端的單位運算成本往往不比自建便宜——它的價值在於「彈性」。

彈性的價值體現在兩個方向。向上的彈性讓你能應付突發的需求：第 20 章那個訓練任務需要 32 台機器跑一天，你不必為此買 32 台機器。向下的彈性同樣重要：任務結束後立刻釋放，不再付費——而自建的機器閒著也是你的。

還有一種常被忽略的彈性價值：試錯成本降低。想試一個新架構？開幾台機器試一週，不合適就關掉，損失有限。若是自建，這個決定要綁住你三到五年。在技術快速變動的今天，這種「不必把賭注押死」的能力，價值可能遠超過帳面的成本差異。

27.2 三種服務模型

27.2.1 責任分界的階梯

第 1 章我們用廚房比喻介紹過三種模型。這裡我們把責任分界看得更精確——關鍵在於「哪些層次由你管、哪些由供應商管」。

管理層次	IaaS	PaaS	SaaS
應用程式	你	你	供應商
資料	你	你	你（內容）
執行環境	你	供應商	供應商
作業系統	你	供應商	供應商
虛擬化	供應商	供應商	供應商
實體硬體	供應商	供應商	供應商

表 27-2 三種服務模型的責任分界

請注意「資料」那一列——即使在 SaaS 中，資料的內容與其合規責任仍在你身上。這是實務上極重要的一點：把系統交給供應商，不等於把責任也交出去。第 33 章談法規遵循時會再深入這個「責任共擔」的觀念。

27.2.2 大數據領域的對應

把三種模型對應到本書的技術，脈絡就清楚了。

模型	大數據領域的形態	取捨
IaaS	租用虛擬機自行架設 Hadoop 或 Spark 叢集	彈性最大、可完全客製，但維運負擔最重
PaaS	託管式 Spark、託管式 Kafka、雲端資料倉儲	免除叢集維運，但受限於供應商提供的版本與設定
SaaS	商業智慧與視覺化分析工具	開箱即用，但客製空間最小

表 27-3 三種服務模型在大數據領域的對應

實務上的趨勢是往 PaaS 移動。原因很直接：自行維運一個 Kafka 或 Spark 叢集需要專門的知識與人力（第 25 章的分區規劃、第 10 章的資源調校），而多數團隊的核心價值不在於「把叢集顧好」，而在於「用資料創造價值」。把不擅長的事外包出去，是理性的分工。

叫獸提醒

但我要提醒一個常被忽略的代價：抽象層次越高，你能掌控的就越少。使用託管服務時，你可能無法選擇特定版本、無法調整某些關鍵參數、也無法在出問題時深入底層排查——你只能開支援單等待。我看過團隊因為託管服務的某個限制而卡住整個專案，卻無能為力。所以選擇服務模型時，除了問「這樣比較省事嗎」，還要問「當它出問題時，我有辦法自己解決嗎」。這個問題的答案，往往決定了你在關鍵時刻是能夠應變，還是只能等待。

27.3 部署模型與混合雲

27.3.1 四種部署方式

- **公有雲**：由第三方供應商營運、開放給大眾使用。優點是彈性最大、無須前期投資；缺點是資料須交由外部保管。
- **私有雲**：由單一組織專用，可自建或委外代管。優點是完全掌控與符合法規；缺點是仍需前期投資與維運人力。
- **混合雲**：公有與私有併用，依資料敏感性與運算需求分流。這是製造業最常見的選擇。
- **社群雲**：由具共同需求的數個組織共享，如產業聯盟或學術聯盟共建的平台。

27.3.2 混合雲為何在製造業普及

台灣的製造業採用混合雲的比例相當高，原因值得深究。

第一是資料主權。製程參數、良率數據、設備配方，往往是企業最核心的機密，且客戶合約可能明訂不得離開特定範圍。這些資料必須留在自有機房。

第二是延遲。第 22 章我們談過，即時控制需要毫秒級反應，而網路往返無法保證。產線的即時判斷必須在本地完成。

第三是網路可靠性。產線不能因為對外網路斷線就停止運作——這正是第 3 章「網路是可靠的」那個誤解的實際後果。

但企業仍需要雲端的彈性，用於模型訓練、跨廠分析、以及與供應鏈夥伴的協作。於是形成了典型的分工：敏感與即時的部分留在地端，需要彈性與跨域的部分放上雲端。這與第 21 章談的雲端與地端模型取捨，是同一個思路在基礎設施層面的體現。

27.3.3 多雲與廠商鎖定

另一個實務議題是「廠商鎖定」。當你大量使用某家供應商的專屬服務，日後要遷移將極為困難——資料格式、API、維運工具全都綁在一起。

有些企業因此採用「多雲」策略，同時使用多家供應商以保留議價空間與轉換彈性。但這也有代價：維運複雜度加倍、團隊要熟悉多套工具、且跨雲的資料傳輸費用可能相當可觀。

較務實的做法是「架構上保留退路」：盡量採用開放標準與開源技術（如以 Parquet 而非專有格式儲存、以 Kubernetes 而非專屬編排服務），使核心資產具備可攜性。這樣即使不真的實施多雲，也保留了必要時搬遷的能力。這與第 23 章談開源價值時的思路相通——開放標準是對抗鎖定的最佳保險。

27.4 計費模型與購買方式

27.4.1 三種購買方式

雲端運算資源通常有三種購買方式，價差可達數倍，選對能省下大量成本。

方式	特性	價格	適用工作負載
隨需	隨時開關、按用量計費	基準價	不可預測、短期、開發測試
預留／承諾	承諾使用一至三年換取折扣	約基準價的 4 至 7 折	長期穩定運行的基礎服務
競價／可中斷	使用閒置資源，可能被隨時回收	約基準價的 1 至 3 折	可容忍中斷的批次運算

表 27-4 三種購買方式的比較

競價型資源特別值得注意——它的折扣極大，代價是「可能被隨時收回」。這聽起來很可怕，但對某些工作負載完全可以接受：例如第 9、11 章談的批次分析作業，本來就具備容錯與重跑能力（任務失敗可重新分派），因此節點被收回只是稍微變慢，不會導致失敗。

這裡有個很好的觀念連結：正因為 MapReduce 與 Spark 從設計之初就假設「機器會壞」（第 3、9 章），它們才能安心地跑在隨時可能消失的廉價資源上。容錯設計不只帶來可靠性，還直接轉化為成本優勢。

27.4.2 容易失控的成本項目

雲端帳單最常出人意料的，往往不是運算費用，而是幾個容易被忽略的項目。

- **資料傳出費用**：把資料從雲端傳出通常要收費，且跨區域、跨可用區的傳輸也可能計費。大量資料的移動成本可能超乎預期。
- **閒置資源**：忘記關閉的測試環境、沒有掛載的儲存磁碟、保留但未使用的固定 IP，這些都在持續計費。

- **低效查詢：**第 12 章談過，一支未經最佳化的查詢可能掃過遠超必要的資料量；在按掃描量計費的服務中，這會直接反映在帳單上。
- **過度佈建：**為求保險而配置遠超實際需求的規格，是自建習慣帶到雲端的常見錯誤。

27.4.3 FinOps：讓成本可見

FinOps 是近年興起的實務領域，其核心主張是：雲端成本管理不該是財務部門月底才做的事，而應成為工程團隊日常的一部分。

它的基本做法有三個環節。首先是「可見」——為所有資源加上標籤，使成本能歸屬到具體的專案、團隊或功能，讓每個人知道自己花了多少。其次是「優化」——依據可見的資料調整規格、清理閒置資源、選擇合適的購買方式。最後是「治理」——設定預算告警與用量上限，避免異常支出無限擴大。

你會發現這與第 13 章談的監控、第 34 章要談的 DataOps 是同一個思想：把原本事後才發現的問題，變成即時可見、可持續改善的指標。成本，其實也是一種需要被監控的系統指標。

27.5 總持有成本與損益平衡

27.5.1 被低估的自建成本

比較自建與雲端時，最常見的錯誤是「只算硬體價格」。真正的成本應以「總持有成本」(TCO) 來評估，它包含幾個常被遺漏的項目。

- **硬體攤提：**設備購置成本除以使用年限。這是唯一大家都會算的項目。
- **機房成本：**空間、電力、空調、不斷電系統。伺服器的耗電與散熱成本，長期而言相當可觀。
- **維運人力：**系統管理、備份、監控、故障處理。這往往是最大也最容易被低估的一項。
- **備援與冗餘：**為確保可用性所需的備用設備與異地備援。
- **機會成本：**資金被綁在設備上，以及採購與建置期間無法使用的等待時間。

其中「維運人力」尤其關鍵。一位工程師的年薪可能就超過數台伺服器的成本，而自建叢集所需的專業維運，往往需要至少一位專職人力。這在學術實驗室特別明顯——那個「維運人力」通常是某位研究生，而他花在顧機器的時間，本該用於研究。

27.5.2 關鍵變數：使用率

承接叫獸開講的結論：自建與雲端的比較，最終取決於「使用率」。

自建的成本是固定的——無論機器跑滿還是閒置，攤提、電力、人力都照付。而雲端的成本與使用量成正比——用多少付多少。

因此當使用率很高時，自建的固定成本被充分利用，單位成本較低；當使用率很低時，自建等於在為閒置付費，雲端則自然省下。下一節的公式會把這個損益平衡點精確算出來，而結果可能會讓你意外。

27.6 雲端遷移策略

27.6.1 五種遷移方式

把既有系統搬上雲端，有從簡單到徹底的幾種做法，投入與收益各不相同。

策略	做法	取捨
重新託管	原封不動搬到雲端虛擬機（俗稱直接搬遷）	最快最簡單，但無法享受雲端特性
調整平台	小幅修改以使用託管服務，如改用託管資料庫	投入適中，可免除部分維運
重構	重新設計為雲原生架構，如改用容器與無伺服器	投入最大，但效益也最高
汰換	改用現成的 SaaS 取代自建系統	最省事，但客製空間受限
保留	維持現狀不遷移	適用於即將汰除或不宜遷移的系統

表 27-5 常見的雲端遷移策略

實務上多數組織會混用多種策略：核心且穩定的系統先重新託管求快，關鍵的資料平台則投入重構以取得雲原生的效益，而週邊的通用功能（如郵件、協作工具）則直接汰換為 SaaS。

27.6.2 遷移的常見陷阱

- **把地端習慣帶上雲：**在雲端開一台永遠開機的大型虛擬機，本質上只是「別人機房裡的自建」——付了雲端的價格，卻沒享受到彈性的好處。
- **低估資料搬遷成本：**把數十 TB 的歷史資料上傳，可能需要數週且產生可觀的傳輸費用；大量資料的遷移往往是專案中最耗時的環節。
- **忽略網路架構：**地端系統之間的低延遲通訊，上雲後可能變成跨區的高延遲呼叫，導致效能大幅劣化。
- **未先優化就搬遷：**一個效率低落的系統搬上雲端，只會讓低效直接轉化為帳單——這與第 20 章「先優化再分散」是同一個道理。

27.6.3 承先啟後

本章我們深入了雲端的本質。從資源池化與錯峰帶來的規模經濟出發（27.1），釐清了三種服務模型的責任分界與其取捨（27.2）、混合雲為何在製造業普及（27.3）、三種購買方式與容易失控的成本項目（27.4）、總持有成本的完整組成與使用率這個關鍵變數（27.5），以及遷移策略與常見陷阱（27.6）。

我最希望你帶走的觀念是：雲端的價值不在便宜，而在彈性；而彈性划不划算，取決於你的使用率。這個判斷需要實際計算，而非人云亦云。

但我們一直還沒回答一個更底層的問題：雲端供應商是怎麼做到「把一台實體機器分給多個客戶、彼此還互不干擾」的？又是什麼技術讓資源能在幾秒內開通、幾秒內釋放？這背後有一項關鍵技術，它同時也是現代軟體交付方式的基礎。這就是下一章的主題——虛擬化與容器技術。

公式與流程：自建與雲端的損益平衡

這一節要把叫獸開講那個「該自建還是上雲」的問題，變成一個可以計算的決策。

兩種成本模型

自建的月成本是固定的，包含硬體攤提與維運支出。設硬體總價為 H 、攤提年限為 Y 年、每月維運成本為 M （含電力、機房、人力），則：

$$\text{自建月成本} = H / (Y \times 12) + M$$

式 27-1 自建的月成本

雲端的月成本則與使用率成正比。設等價運算資源的每小時費率為 R、每月時數以 730 小時計、平均使用率為 U（0 至 1），則：

$$\text{雲端月成本} = R \times 730 \times U$$

式 27-2 雲端的月成本

令兩者相等，即可求出損益平衡的使用率：

$$\text{平衡使用率 } U^* = [H / (Y \times 12) + M] / (R \times 730)$$

式 27-3 損益平衡的使用率

這個 U^* 的意義是：當你的實際使用率高於 U^* 時，自建較便宜；低於 U^* 時，雲端較便宜。

實際試算

設硬體總價 $H = 300$ 萬元、攤提 3 年、每月維運 $M = 4$ 萬元（含電力、空調與分攤的人力）；雲端等價資源每小時 $R = 180$ 元。

先算自建月成本： $3,000,000 \div 36 + 40,000 = 83,333 + 40,000 = 123,333$ 元。

再算平衡使用率： $123,333 \div (180 \times 730) = 123,333 \div 131,400 \approx 93.9\%$ 。

使用率 U	雲端月成本	自建月成本	何者較省
10%	13,140 元	123,333 元	雲端（省 89%）
25%	32,850 元	123,333 元	雲端（省 73%）
50%	65,700 元	123,333 元	雲端（省 47%）
75%	98,550 元	123,333 元	雲端（省 20%）
93.9%	123,333 元	123,333 元	損益平衡點
100%	131,400 元	123,333 元	自建（省 6%）

表 27-6 不同使用率下的成本比較

這張表的結論相當清楚：只有當機器幾乎持續滿載（超過 94%）時，自建才划算。而在真實世界中，這種使用率極為罕見——研究顯示，多數企業自建機房的平均使用率往往落在 10% 到 30% 之間。

叫獸提醒

表 27-6 有兩個地方要提醒你。第一，即使在 100% 使用率下，自建也只比雲端省 6%——而這個微薄的優勢，還沒有把「彈性的價值」算進去。若你某個月需要五倍的資源，自建就完全做不到了。第二，這個計算刻意用了偏低的維運成本（每月 4 萬元）。若你把一位工程師的完整人力成本算進去，自建的成本會顯著上升，平衡點也會跟著往上跑——很可能超過 100%，意味著在任何使用率下自建都不划算。所以做這個計算時，最重要的不是套公式，而是誠實地把所有成本項目列出來。多數人算出「自建比較便宜」的結論，都是因為漏算了維運人力。

案例研討

案例一 閒置了九成的實驗室伺服器

某實驗室三年前購置了一台高階伺服器，用於學生的模型訓練。監控顯示，該機器的平均使用率僅約 12%——多數時間閒置，只在論文截稿前的兩三週滿載。

依式 27-3 的邏輯計算，這個使用模式下雲端的成本僅約自建的八分之一。更關鍵的是尖峰時的體驗：截稿前所有學生搶用同一台機器，排隊等待反而延誤了研究進度——而雲端在那兩三週可以臨時開出五倍的資源。

這個案例呈現了「平均使用率低、但尖峰需求高」這個最適合雲端的典型模式。它也解釋了叫獸開講那個買車的比喻：一年只用三天的車，買它是不划算的。

案例二 被競價資源省下的六成成本

某團隊每晚要跑一批大規模的 Spark 分析作業，原本使用隨需型的虛擬機，每月費用相當可觀。

他們評估後改用競價型資源。因為 Spark 本身具備容錯能力（第 11 章的 Lineage 機制），當節點被回收時，該節點上的任務會自動在其他節點重算，作業仍能完成——只是偶爾需要多花一些時間。

改用後，運算成本下降約六成。這個案例展示了 27.4.1 節的觀念連結：容錯設計不只是可靠性的保障，它讓系統得以安心使用廉價但不穩定的資源，直接轉化為成本優勢。反過來說，若你的系統無法容忍節點消失，就註定得付全價。

案例三 一次查詢的天價帳單

某分析師在雲端資料倉儲上執行了一支探索性查詢。他沒有加上日期篩選，也沒有限定欄位，於是查詢掃過了整個歷史資料表——數十 TB。因為該服務按掃描量計費，這一支查詢的費用超過了團隊該月的預算。

問題的根源有二：技術上，這正是第 12 章談的述詞下推與欄位裁剪未能生效（若加上日期篩選並只選必要欄位，掃描量可能只有千分之一）；管理上，則是缺乏 27.4.3 節的治理機制——沒有預算告警、沒有單次查詢的掃描量上限。

他們後來設定了三道防線：查詢前顯示預估掃描量、單次查詢設定上限、以及每日預算告警。這個案例說明：在雲端，效能優化不再只是技術美學，它有直接的財務後果——而這也是 27.1.1 節所說「可計量服務讓成本變得可見」的雙面刃。

案例四 只是搬家的上雲專案

某企業執行了為期一年的上雲專案，把所有系統以「重新託管」的方式搬到雲端虛擬機上。專案順利完成，但財務部門發現：雲端費用比原本的機房支出還高。

原因是他們把地端的習慣完全帶了過去——所有機器 24 小時開機、規格依照過去的尖峰需求配置、且完全沒有使用彈性擴縮。他們付了雲端的價格，卻只得到「別人機房裡的自建」。

後續的改善包括：把開發測試環境改為下班自動關閉、依實際用量調整規格、對穩定的基礎服務改用預留型購買、並把批次作業改為彈性擴縮。調整後費用降低了四成以上。這正是 27.6.2 節第一個陷阱的寫照——上雲不是搬家，而是必須改變使用資源的方式，否則彈性的價值完全無法兌現。

實作活動

活動一 計算你的損益平衡點

請為一個具體情境完成完整的成本分析。

- 步驟 1. 設定情境：查詢一台適合你需求的伺服器價格，並列出所有自建成本項目（硬體、電力、空間、人力、備援）。
- 步驟 2. 查詢雲端等價規格的每小時費率（可用各供應商的公開價目表）。
- 步驟 3. 依式 27-1 至式 27-3 計算自建月成本與損益平衡使用率。
- 步驟 4. 誠實估計你的實際使用率，據以判斷該選哪一種；接著把維運人力成本提高一倍，觀察平衡點如何變化。

活動二 實測三種購買方式

請在雲端免費層或以公開價目表進行比較分析。

- 步驟 1. 選定一種運算執行個體規格，查詢其隨需、預留（一年期）與競價型的價格。
- 步驟 2. 計算三者的價格比例，並整理成表格。
- 步驟 3. 為下列三種工作負載各選一種購買方式並說明理由：全年運行的網站服務、每晚執行的批次分析、為期兩週的實驗性專案。
- 步驟 4. 若可實際操作，啟動一個競價型執行個體並執行一個可容錯的作業，觀察其行為。

活動三 規劃一個混合雲架構

請為一座智慧工廠設計混合雲架構。首先列出該工廠可能產生的資料類型（如即時感測訊號、品質檢測影像、生產排程、設備維護紀錄），並為每一類判斷：它適合放在地端還是雲端？判斷依據為何（敏感性、延遲要求、資料量、運算需求）？接著畫出整體架構圖，標明資料流向與跨界的傳輸點。最後說明：若對外網路中斷，哪些功能仍能運作、哪些會失效？你會如何設計降級策略？

延伸閱讀

- **NIST 雲端運算定義 (SP 800-145)**：美國國家標準與技術研究院發布，僅數頁，是五大特徵與三種服務模型的權威出處。
- **《Cloud FinOps》**：說明雲端成本管理的組織實務與工程手法，是 27.4.3 節的完整延伸。
- **各雲端供應商的架構最佳實務框架**：涵蓋成本最佳化、可靠性、效能效率等面向的設計原則，是實務規劃時的重要參考。
- **雲端遷移的案例研究文獻**：各產業的遷移經驗與教訓，可作為 27.6 節的實務補充。

本章小結

1. 資源池化的深層意義在於「錯峰」——不同客戶的尖峰不會同時發生，故供應商只需準備遠少於尖峰總和的資源，這是雲端規模經濟的根源；可計量服務則使成本首次變得可見，讓效能優化有了財務意義。
2. 雲端的核心價值不是便宜而是彈性：向上應付突發需求、向下釋放閒置資源、以及大幅降低試錯成本——在技術快速變動的環境中，「不必把賭注押死」的價值可能超過帳面成本差異。
3. 三種服務模型的差異在責任分界（表 27-2），但無論哪一種，資料的內容與合規責任始終在使用者身上；抽象層次越高越省事，但可掌控與可排查的空間也越小。
4. 混合雲在製造業普及的三個原因是資料主權、延遲要求與網路可靠性；而對抗廠商鎖定最務實的做法不是多雲，而是在架構上採用開放標準與開源技術以保留可攜性。

5. 三種購買方式價差可達數倍：隨需最貴但最靈活、預留適合長期穩定負載、競價最便宜但可能被回收。容錯設計使系統能安心使用競價資源，故可靠性直接轉化為成本優勢。
6. 雲端成本最常失控的項目是資料傳出費用、閒置資源、低效查詢與過度佈建；FinOps 以「可見、優化、治理」三環節把成本管理變成工程日常，與監控的思想一脈相承。
7. 自建的總持有成本應包含硬體攤提、機房、維運人力、備援與機會成本，其中維運人力最常被低估；自建成本固定而雲端成本與使用率成正比，故兩者的比較最終取決於使用率。
8. 損益平衡使用率為 $U^* = [H/(Y \times 12) + M] / (R \times 730)$ （式 27-3）；在合理的參數下 U^* 往往高達九成以上，而多數自建機房的實際使用率僅一至三成——這正是雲端商業模式成立的根本原因。

習題

一、觀念題

1. 請說明「資源池化」如何透過錯峰效應造就雲端的規模經濟，並解釋為何這使雲端能同時提供低價與高彈性。
2. 為什麼說「把系統交給供應商，不等於把責任也交出去」？請以三種服務模型的責任分界說明。
3. 請說明為何具備容錯能力的系統，能夠使用競價型資源而取得成本優勢。請以本書談過的技術舉例。
4. 上雲後費用反而增加，可能有哪些原因？請至少列舉三項並說明如何改善。

二、計算題

1. 某單位評估自建叢集：硬體 480 萬元、攤提 4 年、每月維運（電力、空間、人力分攤）9 萬元；雲端等價資源每小時 250 元。（一）依式 27-1 計算自建月成本；（二）依式 27-3 計算損益平衡使用率；（三）若實際平均使用率為 35%，兩者的月成本各為多少？該選哪一種？
2. 承上題，若把維運人力成本從每月 9 萬提高至 18 萬（納入一位專職工程師）。（一）自建月成本變為多少？（二）新的損益平衡使用率為何？（三）此結果說明了什麼？

三、思考與討論

1. 本章指出「抽象層次越高，可掌控的就越少」。請討論：在選擇 IaaS、PaaS 或 SaaS 時，你會用哪些問題來評估這個取舍？請提出一份實際可用的檢核清單。
2. 案例四中，上雲專案只是「搬家」而未改變使用方式。請討論：一個組織要真正發揮雲端的價值，除了技術改造外，還需要哪些流程或文化上的改變？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

錯峰效應：供應商把數萬台機器匯集為資源池，租給成千上萬個客戶，而這些客戶的用量尖峰通常不會同時發生（甲公司月底結帳、乙公司週末促銷、丙研究室期末運算，彼此錯開）。因此供應商只需準備遠少於「所有客戶尖峰總和」的機器，即可滿足所有人的需求，設備的平均使用率遠高於個別客戶自建的水準。這使雲端能同時提供低價與高彈性：低價來自高使用率攤薄了固定成本，高彈性

則來自資源池的緩衝——當某客戶需要暴增的資源時，可從其他客戶當下未使用的餘裕中調度。個別企業自建無法達成此效果，因為它只有自己一個客戶，尖峰與離峰的落差必須自行承擔。

第 2 題

因為責任是「共擔」而非「移轉」。依表 27-2，IaaS 中供應商僅負責實體硬體與虛擬化，作業系統以上皆由使用者管理；PaaS 中供應商多承擔執行環境與作業系統；SaaS 中供應商負責應用程式。但無論哪一種模型，「資料」這一系列的責任始終在使用者身上——資料的內容是否正確、是否含個資、是否符合法規（第 33 章）、存取權限如何設定，這些都不會因為使用了託管服務而轉移給供應商。故實務上必須清楚界定：供應商保證的是「服務可用」，而非「你的資料使用方式合法且正確」。

第 3 題

競價型資源價格僅約基準價的一至三成，代價是可能被隨時回收。具備容錯能力的系統能承受節點消失，故可安心使用。以本書談過的技術為例：MapReduce（第 9 章）的任務設計為獨立、無狀態且可重複執行，節點失敗時只需在其他節點重跑該任務；Spark（第 11 章）以 Lineage 血統記錄推導過程，分區遺失時可依血統重算，無須複製資料；兩者皆因「假設機器會壞」的設計前提（第 3 章）而天生耐受節點消失。故容錯設計不只帶來可靠性，更直接轉化為使用廉價資源的能力與成本優勢（見案例二）。反之，無法容忍節點消失的系統就註定得付全價。

第 4 題

可能原因與改善方式（任舉三項）：其一，把地端習慣帶上雲——所有機器 24 小時開機、依尖峰配置規格、未使用彈性擴縮；改善方式是開發測試環境下班自動關閉、依實際用量調整規格、批次作業改為彈性擴縮（見案例四）。其二，未使用合適的購買方式——全部採隨需型；改善方式是穩定負載改用預留、可容錯的批次改用競價。其三，資料傳出與跨區傳輸費用未納入規劃；改善方式是重新設計資料放置位置，減少跨區流動。其四，閒置資源未清理（忘記關閉的環境、未掛載的磁碟）；改善方式是加上標籤與定期盤點。其五，低效查詢掃描大量資料（見案例三）；改善方式是採欄式格式、加上篩選條件並設定掃描量上限。可補充：根本的改善需搭配 FinOps 的可見、優化、治理三環節。

二、計算題

第 1 題

$H = 4,800,000$ 、 $Y = 4$ 、 $M = 90,000$ 、 $R = 250$ 。

- (1) 自建月成本（式 27-1） $= 4,800,000 / (4 \times 12) + 90,000 = 100,000 + 90,000 = 190,000$ 元。
- (2) 平衡使用率（式 27-3） $= 190,000 / (250 \times 730) = 190,000 / 182,500 \approx 1.041$ ，即約 104.1%。
- (3) 使用率 35% 時：雲端月成本 $= 250 \times 730 \times 0.35 = 63,875$ 元；自建仍為 190,000 元。應選雲端，可省約 66%。

判讀：平衡使用率超過 100%，意味著即使機器全年無休地滿載運行，自建仍比雲端貴——在此參數下，自建在任何情況下都不划算。

第 2 題

維運成本 M 由 90,000 提高至 180,000。

- (1) 自建月成本 = $100,000 + 180,000 = 280,000$ 元。
- (2) 平衡使用率 = $280,000 / 182,500 \approx 1.534$ ，即約 153.4%。
- (3) 說明：平衡使用率遠超過 100%，代表自建在任何使用率下都不可能比雲端便宜。

延伸思考：本題呈現了叫獸提醒的核心警告——維運人力是最常被低估、卻往往最龐大的成本項目。許多人算出「自建比較便宜」的結論，正是因為把這一項當成「反正人已經在了」而未計入。但那位工程師（或研究生）花在顧機器的時間，本可用於更有價值的工作，這就是實實在在的機會成本。做 TCO 分析時，誠實列出所有成本項目，比套用公式本身更重要。

三、思考與討論

第 1 題

檢核清單可包含：其一，可控性——「當它出問題時，我能自己排查嗎？」能否取得日誌、能否調整關鍵參數、能否選擇版本。其二，鎖定風險——「若要離開，搬遷的代價有多大？」資料格式是否開放、API 是否標準、是否有等價的替代方案。其三，能力匹配——「它提供的功能是否涵蓋我的需求？」特別注意那些「目前用不到但未來可能需要」的功能。其四，團隊能力——「我們有能力自行維運嗎？」若團隊缺乏該領域專長，PaaS 的價值就高；反之若已有成熟的維運能力，IaaS 的彈性可能更划算。其五，合規要求——資料放置位置、稽核軌跡、加密控制權是否滿足法規。其六，成本結構——高抽象層次通常單位成本較高，須評估用量成長後的費用曲線。周延的回答會指出：這個取捨沒有標準答案，但應「有意識地選擇」而非預設。

第 2 題

流程與文化上的改變包括：其一，成本意識的下放——把雲端帳單依專案與團隊拆分並公開，使工程師能看見自己的決策造成的支出（FinOps 的「可見」環節）；成本從財務部門的事變成工程日常。其二，資源生命週期的紀律——建立「誰開的誰負責關」的規範、為所有資源加上標籤、定期盤點閒置資源。其三，以彈性為預設的設計思維——新系統設計時就假設資源可隨時增減，而非固定配置；把「這台機器要開多大」的思考換成「負載變化時如何自動調整」。其四，容錯設計的要求——使系統能使用競價資源並容忍節點消失。其五，架構決策的可逆性——鼓勵小規模試驗而非一次到位的大型規劃，善用雲端降低試錯成本的優勢。其六，跨部門協作——財務、採購與工程須共同參與，因為雲端把「一次性採購」變成了「持續的營運決策」。核心觀念：上雲的效益不會自動實現，它需要組織改變使用資源的方式（如案例四）。

機器人叫獸（李世淵） 編著

叫獸開講

第 23 章我提過每年畢業季的求救訊息：「學長的程式我跑不起來。」我當時說可重現性有四個條件，其中「環境」那一項最容易被忽略。這一章，我要告訴你這個問題最終是怎麼被解決的。

先講一個貨櫃的故事。十九世紀的港口是這樣運作的：船靠岸後，工人把成千上萬件形狀各異的貨物一件件搬下來——木桶、麻袋、木箱，每一種都要不同的搬法。裝卸一艘船要花上好幾天，而且貨物在轉運過程中經常損壞或遺失。

一九五〇年代，有人提出一個想法：不要管貨物是什麼，一律先裝進標準尺寸的鋼製箱子。從此港口只需要處理一種東西——貨櫃。船、卡車、火車、起重機全部依這個標準設計，裝卸時間從幾天縮短到幾小時。這個看似平凡的鐵箱子，徹底改變了全球貿易。

軟體業在二〇一〇年代發生了完全相同的事。過去每個應用都有自己的相依套件、版本要求、設定檔，部署時得在目標機器上一項項安裝、一項項對版本——這就是「跑不起來」的根源。而容器技術的想法，正是那個鋼製箱子：把應用程式連同它需要的一切環境全部打包，變成一個標準的、可以在任何地方原樣執行的單位。

這一章我們要看清楚這兩層技術：虛擬化解決了「一台實體機器如何分給多個客戶」，而容器解決了「一個應用如何帶著環境到處跑」。前者是第 27 章雲端得以成立的技術基礎，後者則是下一章 Kubernetes 的前提。

學習目標

研讀完本章之後，你應該能夠：

1. 說明虛擬化的原理，並區分第一型與第二型 Hypervisor。
2. 說明容器與虛擬機的本質差異，並比較兩者的隔離強度與資源開銷。
3. 說明容器映像檔的分層結構，並解釋其為何能大幅節省空間與傳輸。
4. 描述容器的生命週期，並說明狀態與資料應如何處理。
5. 說明容器網路的基本模型，以及容器間如何通訊。
6. 依應用特性判斷應使用虛擬機、容器或兩者併用。

核心概念

核心概念	說明
虛擬化	在一台實體機器上模擬出多台獨立虛擬機器的技術。
Hypervisor	管理與調度虛擬機、分配實體資源的軟體層。
容器	共用主機核心、以作業系統機制隔離的輕量執行環境。
映像檔	包含應用程式與其完整執行環境的唯讀範本。
分層與寫時複製	映像檔以多層堆疊構成，各層可被共用，修改時才複製。
容器編排	自動化管理大量容器的部署、擴縮與復原（見第 29 章）。

核心概念	說明
卷 (Volume)	容器外部的持久化儲存，使資料不隨容器消失。

表 28-1 本章核心概念一覽

28.1 虛擬化的原理與類型

28.1.1 一台變多台

虛擬化要解決的問題，正是第 27 章末留下的伏筆：雲端供應商如何把一台實體伺服器，分給多個彼此不認識、也不該互相干擾的客戶？

答案是在硬體與作業系統之間插入一層軟體——Hypervisor。它把實體資源（處理器、記憶體、儲存、網路）抽象化，並向上呈現為多組「看起來像真實硬體」的虛擬資源。每個虛擬機都以為自己獨佔一台完整的電腦，安裝自己的作業系統，完全不知道旁邊還有其他虛擬機。

這個抽象的價值有三：其一是整併——把原本各自佔用一台機器的多個應用集中到一台實體機器上，提升使用率（回想第 27 章那個 10% 到 30% 的使用率困境）。其二是隔離——一個虛擬機當機或被入侵，不影響其他虛擬機。其三是可移動——虛擬機可以被暫停、打包、搬到另一台實體機器上繼續執行。

28.1.2 兩種 Hypervisor

類型	架構	特性
第一型（原生）	直接安裝在硬體上，本身即是精簡的作業系統	效能高、隔離強，用於資料中心與雲端
第二型（託管）	安裝在既有作業系統之上，作為一般應用程式執行	安裝方便，用於個人電腦的開發測試

表 28-2 兩種 Hypervisor 的比較

差別的關鍵在於「中間隔了幾層」。第一型直接管理硬體，虛擬機的指令幾乎可直接執行；第二型則必須透過底層的作業系統轉手，多了一層開銷。因此雲端資料中心一律採用第一型，而你在自己筆電上跑虛擬機的軟體則多屬第二型。

28.1.3 虛擬化的代價

虛擬化雖然解決了整併與隔離，但它有一個難以迴避的代價：每個虛擬機都必須執行一套完整的作業系統。

這意味著什麼？假設你要在一台機器上跑二十個應用，若各用一個虛擬機，就是二十套作業系統同時運行——二十份核心、二十份系統服務、二十份記憶體佔用。而這些作業系統做的事幾乎一模一樣，卻各佔一份資源。

此外，開機也要時間。一個虛擬機從啟動到可用，往往需要數十秒到數分鐘——因為它真的在「開機」，要載入核心、初始化裝置、啟動系統服務。這在需要快速擴縮的場景中是嚴重的限制。

這兩個代價——資源開銷與啟動時間——正是容器技術要解決的問題。

28.2 容器：共用核心的輕量隔離

28.2.1 關鍵的洞見

容器的核心洞見是一個提問：如果這些應用本來就要跑在同一種作業系統上，我們為什麼要為每一個都準備一套完整的核心？

容器的做法是：所有容器共用主機的作業系統核心，但透過作業系統本身提供的隔離機制，讓每個容器看起來擁有獨立的環境——獨立的檔案系統視野、獨立的行程列表、獨立的網路介面、獨立的資源配額。

這個轉變帶來了根本的差異。因為不需要各自的核心，容器的資源開銷從「一整套作業系統」降到「幾乎只有應用程式本身」；因為不需要開機，容器的啟動時間從數十秒降到不到一秒——它本質上只是「啟動一個被隔離的行程」。

面向	虛擬機	容器
隔離層次	硬體層（各自的核心）	作業系統層（共用核心）
資源開銷	每個實例需完整的作業系統	近乎只有應用程式本身
啟動時間	數十秒至數分鐘	通常不到一秒
隔離強度	強（核心分離）	較弱（共用核心）
作業系統選擇	可執行與主機不同的系統	須與主機核心相容

表 28-3 虛擬機與容器的比較

28.2.2 隔離強度的取捨

表 28-3 中「隔離強度」那一列必須特別說明，因為它是選型時最關鍵的考量。

虛擬機的隔離是在硬體層完成的——各虛擬機有自己的核心，即使某個虛擬機的核心被攻破，攻擊者仍被 Hypervisor 擋在外面。而容器共用主機核心，若核心本身存在漏洞，理論上可能被用來突破隔離，影響到同一主機上的其他容器。

因此在多租戶的場景中（尤其是執行不受信任的程式碼），業界的做法通常是「兩者併用」——以虛擬機作為租戶之間的強隔離邊界，在虛擬機內再以容器承載該租戶的多個應用。你在第 10 章看過的 YARN 容器與這裡的容器概念相通，而雲端上的託管 Kubernetes 服務，底層正是這種「虛擬機中跑容器」的架構。

叫獸提醒

我要澄清一個學生常有的誤解：容器不是「輕量的虛擬機」。這個說法雖然直觀，卻會誤導你的判斷。虛擬機模擬的是「一台電腦」，容器隔離的是「一個行程」——兩者的本質完全不同。這個區別有實際後果：容器裡沒有自己的核心，所以你不能在 Linux 主機上跑一個 Windows 容器；容器裡通常也不該跑一整套系統服務，而應該只跑一個主要的行程。我看過學生把容器當虛擬機用，在裡面裝了一堆服務、還用系統管理工具去啟動它們——這在技術上勉強做得到，但完全違背了容器的設計哲學，也失去了它輕量與可替換的優勢。記住那個貨櫃的比喻：貨櫃裡裝的是貨物，不是另一艘船。

28.3 映像檔的分層與 Registry

28.3.1 分層結構

容器映像檔 (image) 是一個唯讀的範本，包含了執行該應用所需的一切——程式碼、執行環境、函式庫、設定。而它最精妙的設計是「分層」。

一個映像檔不是單一的大檔案，而是由多個唯讀層堆疊而成。典型的結構可能是：最底層是精簡的作業系統檔案、上面一層是執行環境（如 Python）、再上一層是安裝的相依套件、最上層才是你的應用程式碼。

這樣分層的價值在於「共用」。若你有十個應用都以同一個 Python 基礎映像檔為底，那麼那些共用的層在磁碟上只需儲存一份，十個映像檔各自只需存放自己特有的部分。這在儲存與網路傳輸上都帶來巨大的節省——下一節的公式會把這個效益算出來。

28.3.2 寫時複製

既然映像檔的層是唯讀的，那容器執行時要寫入檔案怎麼辦？

答案是在所有唯讀層之上，再疊一個「可寫層」。容器對檔案的任何修改都發生在這一層——若要修改某個來自唯讀層的檔案，系統會先把它複製到可寫層再修改，這稱為「寫時複製」（copy-on-write）。

這個機制有兩個重要的後果。其一是效率：多個容器可以共用同一份唯讀層，只有各自修改的部分才佔用額外空間。其二，也更重要的是——容器刪除時，那個可寫層也隨之消失。這意味著容器內的修改預設是「用完即丟」的，這正是下一節要處理的持久化問題。

你應該覺得這個「共用不變的部分、修改時才複製」的思想很眼熟——第 11 章的 RDD 不可變性、第 25 章的追加式日誌，本質上都是同一個設計哲學：讓「不變」成為預設，把「變化」侷限在最小的範圍。

28.3.3 Registry：映像檔的倉庫

Registry 是儲存與分發映像檔的服務，其角色相當於第 23 章的 GitHub 之於程式碼、Hugging Face 之於模型。你可以推送自己建置的映像檔，也可以拉取他人發布的。

分層結構在這裡發揮了另一個效益：推送或拉取時，只需傳輸「本地沒有的層」。若你更新了應用程式碼並重新建置，通常只有最上面那一層改變，傳輸量可能只有幾 MB，而非整個數百 MB 的映像檔。

這也給了一個實務建議：撰寫映像檔定義時，應把「不常變動的部分」放在下層（如安裝相依套件），「經常變動的部分」放在上層（如複製程式碼）。如此每次修改程式碼時，下層的快取仍可重用，建置與傳輸都會快得多。這是一個小技巧，但在頻繁部署的專案中能省下大量時間。

28.4 容器的生命週期與資料

28.4.1 容器應該是無狀態的

容器的設計哲學中，有一條核心原則：容器應該是「可拋棄的」（disposable）。它可以隨時被停止、刪除、在別台機器上重新啟動，而不影響系統的正確性。

這個原則之所以重要，是因為它是第 29 章自動化編排的前提。編排系統要能自由地移動、重啟、擴充容器，就必須假設「殺掉一個容器再開一個新的」是安全的。若容器內累積了重要的狀態，這個假設就不成立了。

因此實務上的原則是：容器內不保存任何需要留存的資料。所有狀態應該外部化——存到資料庫、物件儲存、或掛載的持久化卷。

這與第 9 章 MapReduce 的無狀態任務設計、第 13 章的冪等性原則，是同一個思想的不同層面讓元件本身不持有不可重建的狀態，系統才能自由地重試、遷移與擴縮。

28.4.2 持久化的做法

既然容器內的可寫層會隨容器消失，需要留存的資料就必須放在外面。常見的做法有幾種：

- **卷 (Volume)**：由容器執行環境管理的持久化儲存區，掛載到容器內的某個路徑。容器刪除後卷仍存在，可被新容器重新掛載。
- **綁定掛載**：直接把主機上的某個目錄掛進容器。簡單直接，但使容器與特定主機綁定，不利於遷移。
- **外部服務**：把資料寫到資料庫、物件儲存或訊息佇列等外部系統。這是雲原生架構最推薦的做法。

其中「外部服務」最符合雲原生的精神——它讓容器本身徹底無狀態，可以在任何節點上啟動而不受限制。這也解釋了為什麼第 5 章的物件儲存、第 25 章的訊息佇列在現代架構中如此重要：它們正是承載狀態的外部服務。

28.4.3 設定與機密

還有一類東西不該寫進映像檔：設定與機密。

原因是同一個映像檔應該能在不同環境（開發、測試、正式）中執行，而各環境的設定不同。若把設定寫死在映像檔中，就得為每個環境建置不同的映像檔——這破壞了「一次建置、處處執行」的價值，也讓測試過的映像檔與正式上線的不是同一個。

正確的做法是透過環境變數或設定檔在「執行時」注入。而密碼、金鑰等機密資訊更是絕對不能寫進映像檔——因為映像檔可能被推送到 Registry、被他人拉取，且分層結構會保留歷史（即使你在後面的層刪除它，前面的層仍然存在）。這類資訊應使用專門的機密管理機制，第 33 章會再深入討論。

28.5 容器網路與通訊

28.5.1 基本的網路模型

每個容器預設會擁有自己的網路命名空間——包含獨立的網路介面與 IP 位址。這使容器之間的通訊看起來就像不同主機之間的通訊。

常見的網路模式有幾種：橋接模式讓容器連接到主機上的一個虛擬網路，彼此可互通並透過主機對外；主機模式讓容器直接使用主機的網路堆疊，效能最好但失去隔離；而在多主機的環境中，則需要覆疊網路（overlay network）讓不同機器上的容器也能直接互通。

跨主機通訊是容器網路最複雜的部分，也是為何需要專門的網路方案。在第 29 章的 Kubernetes 中，這個問題被抽象為統一的網路模型——所有容器都能以 IP 直接互通，不必關心它們實際在哪台機器上。

28.5.2 服務發現的必要性

這裡浮現一個實務問題：容器是動態的，它可能被重啟、被移到另一台機器，IP 位址隨之改變。那麼服務 A 要如何找到服務 B？

答案是「不要用 IP，要用名稱」。容器環境會提供服務發現機制——你以服務名稱來呼叫，由系統負責解析出當下實際的位址。當目標容器被重建或搬移時，呼叫方完全不必修改。

你應該注意到，這正是第 8 章分散式協調服務的功能之一。在 Kubernetes 中，服務發現是內建的核心能力——這也說明了為何理解第 8 章的原理，能讓你更快看懂現代的雲端平台。

28.6 選型：虛擬機、容器，還是兩者

28.6.1 判斷的四個問題

面對一個應用，該用虛擬機還是容器？我建議依序問四個問題。

- **需要多強的隔離？**：若要執行不受信任的程式碼、或多租戶之間須有法規等級的隔離，虛擬機的核心分離較為安全。
- **需要什麼作業系統？**：容器必須與主機核心相容。若需要與主機不同的作業系統，只能用虛擬機。
- **擴縮的頻率如何？**：若需要頻繁、快速地擴縮（如應付流量尖峰），容器的秒級啟動有壓倒性優勢。
- **應用能否無狀態化？**：若應用深度依賴本機狀態且難以外部化，容器的優勢會大打折扣。

28.6.2 大數據場景的實務

回到本書的主軸。容器化對大數據系統帶來的最大價值，其實不是效能，而是「環境一致性」。

回想第 23 章那個可重現性的四個條件——「環境」是最難處理的一項。容器把整個執行環境打包成一個不可變的映像檔，徹底解決了「在我的機器上跑得起來」這個問題。你的 Spark 作業、你的模型訓練程式、你的資料管線，都能以完全相同的環境在開發機、測試環境與正式叢集上執行。

這也是為何第 23 章談可重現性時，我把「環境」的解方指向了容器。而現代的資料平台（包括第 10 章的 YARN 與下一章的 Kubernetes）都已支援以容器執行任務，正是因為這個一致性的價值。

28.6.3 承先啟後

本章我們釐清了兩層技術。虛擬化在硬體層做隔離，讓一台實體機器能安全地分給多個客戶——這是第 27 章雲端商業模式的技術基礎（28.1）。容器則在作業系統層做隔離，共用核心以換取極低的開銷與極快的啟動（28.2），並以分層映像檔實現高效的共用與傳輸（28.3）；其設計哲學要求容器無狀態、可拋棄，狀態則外部化（28.4），而動態的網路則需服務發現來支撐（28.5）。

但你可能已經注意到一個問題：容器很輕、啟動很快，這意味著一個系統可能會有成百上千個容器同時運行。誰來決定每個容器該跑在哪台機器上？某個容器當掉了誰負責重啟？流量增加時誰負責擴充？某台機器故障時，上面的容器該搬到哪裡？

這些問題，本質上與第 10 章的叢集資源管理完全相同——只是被管理的對象從 MapReduce 任務換成了容器。而回答這些問題的系統，已經成為雲原生時代的作業系統。這就是下一章的主題：Kubernetes。

公式與流程：部署密度與映像檔的層共用

這一節用兩組計算，量化容器相對於虛擬機的兩個核心優勢：更高的部署密度，與更省的儲存傳輸。

部署密度

設主機可用記憶體為 H 、單一應用本身需要的記憶體為 A 、而每個執行實例的額外開銷為 V （虛擬機需一整套作業系統，容器則幾乎沒有）。則單台主機可承載的實例數為：

$$\text{實例數} = \lfloor H / (A + V) \rfloor$$

式 28-1 單台主機可承載的實例數

以主機記憶體 $H = 128 \text{ GB}$ 、應用本身需 $A = 2 \text{ GB}$ 為例，虛擬機的作業系統開銷約 $V = 1.5 \text{ GB}$ ，容器則約 $V = 0.05 \text{ GB}$ 。

形式	單實例佔用	可承載實例數	相對密度
虛擬機	$2 + 1.5 = 3.5 \text{ GB}$	36 個	1.0 倍
容器	$2 + 0.05 = 2.05 \text{ GB}$	62 個	約 1.7 倍

表 28-4 部署密度的比較（ $H = 128 \text{ GB}$ 、 $A = 2 \text{ GB}$ ）

同一台機器多跑七成的實例——換算成第 27 章的成本語言，這相當於直接省下約四成的機器數量。而應用越輕量（ A 越小），這個差距越懸殊：若 A 只有 0.5 GB ，虛擬機能跑 64 個而容器能跑 232 個，差距達 3.6 倍。

映像檔的層共用

設有 n 個應用共用同一個基礎映像檔，基礎層大小為 B 、各應用特有的部分為 S 。則實際儲存需求為：

$$\text{總儲存} = B + n \times S$$

式 28-2 分層共用下的總儲存需求

若不共用（每個映像檔各自完整），則需要 $n \times (B + S)$ 。故節省的比例為：

$$\text{節省比例} = (n - 1) \times B / [n \times (B + S)]$$

式 28-3 層共用的節省比例

以基礎映像檔 $B = 800 \text{ MB}$ 、各應用特有部分 $S = 50 \text{ MB}$ 為例：

應用數 n	不共用	共用基礎層	節省比例
1	850 MB	850 MB	0%
5	4,250 MB	1,050 MB	約 75%
10	8,500 MB	1,300 MB	約 85%
20	17,000 MB	1,800 MB	約 89%

表 28-5 映像檔層共用的儲存節省 (B = 800 MB、S = 50 MB)

叫獸提醒

表 28-5 有一個非常實用的推論：應用數越多，共用基礎層的效益越大——因此團隊應該「刻意統一基礎映像檔」。我看過一個團隊，十幾個服務各自用了不同的基礎映像檔（有的 Ubuntu、有的 Debian、有的不同版本），結果不只浪費儲存與傳輸，連安全性修補都要重複做十幾次。統一基礎映像檔是一個成本極低、效益極高的工程紀律。另外提醒一點：這個節省不只在儲存，更在「傳輸」——當你部署到一百個節點時，若基礎層已在各節點快取，實際只需傳輸那 50 MB 的差異層。部署速度的差別會非常明顯。

案例研討

案例一 終於跑得起來的學長專題

回到第 23 章那個永恆的難題。某研究室要求所有專題必須附上容器定義檔，把執行環境完整描述出來——基礎映像檔、相依套件與其版本、環境變數、啟動指令。

此後接手者不必再逐一安裝套件、對版本、猜設定，只需建置映像檔並執行即可。原本要耗費數天的環境重建，縮短到十幾分鐘。

這個案例展示了容器對第 23 章「可重現性四條件」的直接貢獻——它把最難處理的「環境」那一項，從一份可能過時的文件，變成了一個可執行、可驗證的定義檔。依第 23 章式 23-1 的連乘關係，補強這個最弱的一環，對整體可重現性的提升最為顯著。

案例二 被刪掉的訓練成果

某學生在容器中執行模型訓練，訓練了兩天後把檢查點存在容器內的某個目錄。訓練完成後，他停止並刪除了容器準備釋放資源——結果所有訓練成果隨著容器的可寫層一起消失。

這正是 28.3.2 節寫時複製機制的後果：容器內的所有修改都在可寫層，容器刪除時該層即被丟棄。這不是錯誤，而是設計本意——容器就是被設計成可拋棄的。

正確的做法是把輸出目錄掛載為卷，或直接寫入外部的物件儲存。這個案例提醒我們 28.4.1 節那條原則的重要性：容器內不保存任何需要留存的資料。所有狀態必須外部化——這不只是最佳實務，更是使用容器的前提認知。

案例三 密碼藏在第三層

某團隊建置映像檔時，在中間某個步驟寫入了資料庫密碼，後來意識到不妥，在後續的步驟中把該檔案刪除，並認為問題已解決。

但映像檔是分層的，「刪除」只是在上層做了一個標記，那個含有密碼的下層仍然完整存在於映像檔中。任何取得該映像檔的人，都能從歷史層中把密碼取出。而這個映像檔已經被推送到共用的 Registry。

這正是 28.4.3 節警告的情況。正確的做法是機密資訊絕不寫進映像檔，而應在執行時透過機密管理機制注入。這個案例也呼應第 23 章的教訓——版本控制與分層儲存都會保留歷史，「事後刪除」往往無法真正抹除。

案例四 十幾個基礎映像檔

某團隊的十五個微服務，因為由不同成員在不同時期建立，各自使用了不同的基礎映像檔——有的是完整的作業系統映像檔（近 1 GB），有的是精簡版，版本也各不相同。

問題在一次安全性漏洞公告後浮現：他們必須為十五種不同的基礎映像檔分別確認影響、分別更新、分別測試，耗費了大量人力。同時，因為幾乎沒有共用的層，Registry 的儲存量與部署時的傳輸量都遠高於必要。

他們後來統一為兩個標準基礎映像檔（一個給 Python 服務、一個給 Java 服務），依式 28-3 估算，儲存需求降低了約八成，而更重要的是——安全性修補從十五次變成兩次。這個案例說明：技術紀律的價值，往往不在平時，而在出事的時候。

實作活動

活動一 打包你的第一個容器

請把一個既有的程式打包成容器，親身體會環境一致性的價值。

- 步驟 1.** 選擇一個你寫過的 Python 或其他語言的程式（最好有數個相依套件）。
- 步驟 2.** 撰寫容器定義檔：選擇基礎映像檔、安裝相依套件、複製程式碼、設定啟動指令。
- 步驟 3.** 建置映像檔並執行，確認程式在容器中能正確運作。
- 步驟 4.** 把定義檔與程式碼交給一位同學，請他在自己的電腦上建置並執行，記錄他是否遇到任何環境問題。

活動二 觀察分層與快取

請驗證 28.3 節的分層機制與其效益。

- 步驟 1.** 在活動一的定義檔中，把「安裝相依套件」放在「複製程式碼」之前，建置並記錄所需時間。
- 步驟 2.** 修改一行程式碼後重新建置，觀察哪些層被重用（快取命中）、實際只重建了哪幾層、耗時多久。
- 步驟 3.** 把兩個步驟的順序對調（先複製程式碼、再安裝套件），重複上述實驗，比較重建時間的差異。
- 步驟 4.** 說明你的觀察，並解釋為何「不常變動的放下層」是重要的實務原則。

活動三 驗證容器的無狀態性

請設計實驗驗證 28.4 節的持久化議題。首先在容器中執行一個會產生輸出檔案的程式（例如寫入一個結果檔），完成後停止並刪除容器，再啟動一個新容器並檢查該檔案是否還在——重現案例二的情況。接著把輸出目錄改為掛載卷，重複同樣的流程，確認資料在容器刪除後仍然存在。最後撰寫一段說明：哪些資料應該放在容器內、哪些必須外部化？你會如何為一個模型訓練的容器規劃資料的存放位置？

延伸閱讀

- **Docker 官方文件：**涵蓋映像檔建置、分層機制、卷與網路的完整說明，是本章最直接的實作參考。

- 《Docker Deep Dive》：深入說明容器背後的作業系統機制（命名空間與資源控制群組），適合想理解原理者。
- 容器安全的最佳實務指南：說明映像檔掃描、最小權限、機密管理等實務，是 28.4.3 節的延伸。
- 虛擬化技術的教科書章節：任一本作業系統或雲端運算教科書關於 Hypervisor 與虛擬化的章節，可補強 28.1 節的理論基礎。

本章小結

1. 虛擬化在硬體與作業系統之間插入 Hypervisor，把實體資源抽象為多組虛擬硬體，帶來整併、隔離與可移動三項價值；第一型直接管理硬體用於資料中心，第二型則安裝於既有作業系統之上。
2. 虛擬化的代價是每個虛擬機都需執行一套完整的作業系統，造成可觀的資源開銷與數十秒以上的啟動時間；容器正是為解決這兩個代價而生。
3. 容器共用主機核心，以作業系統層的機制達成隔離，故開銷近乎只有應用本身、啟動可在一秒內完成；代價是隔離強度較弱且必須與主機核心相容。容器不是「輕量的虛擬機」——虛擬機模擬一台電腦，容器隔離一個行程。
4. 映像檔由多個唯讀層堆疊而成，各層可被不同映像檔共用；執行時在其上疊加可寫層並採寫時複製，故容器內的修改隨容器刪除而消失。建置時應把不常變動的部分放下層以善用快取。
5. 容器應設計為無狀態、可拋棄，這是自動化編排的前提；需留存的資料應以卷或外部服務（資料庫、物件儲存）持久化，設定則於執行時注入，而機密資訊絕不可寫入映像檔——因分層會保留歷史，事後刪除無法真正抹除。
6. 容器擁有獨立的網路命名空間，跨主機通訊需覆蓋網路；因容器的位址是動態的，必須以服務發現機制透過名稱而非 IP 來呼叫。
7. 選型時應依序評估隔離強度需求、作業系統相容性、擴縮頻率與能否無狀態化；多租戶場景常採「虛擬機提供強隔離邊界、其內以容器承載應用」的併用架構。
8. 部署密度為 $\lfloor H/(A+V) \rfloor$ （式 28-1），容器因開銷極小而密度可達虛擬機的 1.7 倍以上；映像檔層共用的節省比例為 $(n-1)B/[n(B+S)]$ （式 28-3），應用數越多效益越大，故統一基礎映像檔是高報酬的工程紀律。

習題

一、觀念題

1. 請說明虛擬化如何使雲端的多租戶架構成為可能，並指出它的兩個主要代價。
2. 為什麼說「容器不是輕量的虛擬機」？請從隔離層次說明兩者的本質差異，並各舉一個因此產生的實際限制。
3. 請說明映像檔的分層與寫時複製機制，並解釋為何「容器內的資料會消失」是設計本意而非缺陷。
4. 為什麼機密資訊絕不可寫入映像檔？即使在後續步驟中刪除也不行，原因為何？

二、計算題

1. 某主機可用記憶體 $H = 256$ GB，某應用本身需 $A = 3$ GB。虛擬機的作業系統開銷 $V = 1.5$ GB，容器開銷 $V = 0.05$ GB。（一）依式 28-1 計算兩者各可承載幾個實例；（二）計算相對密度；（三）若改為輕量應用 $A = 0.5$ GB，重新計算並說明差距為何擴大。
2. 某團隊有 12 個服務，基礎映像檔 $B = 600$ MB、各服務特有部分 $S = 80$ MB。（一）依式 28-2 計算共用基礎層時的總儲存；（二）計算不共用時的總儲存；（三）依式 28-3 計算節省比例。

三、思考與討論

1. 本章指出容器化對大數據系統的最大價值是「環境一致性」而非效能。請結合第 23 章的可重現性四條件討論這個主張，並說明容器解決了什麼、又沒有解決什麼。
2. 案例四中，統一基礎映像檔的價值在安全性漏洞爆發時才充分顯現。請討論：還有哪些工程紀律具有這種「平時看不出價值、出事時才關鍵」的特性？組織應如何說服團隊在平時就遵守？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

虛擬化在硬體與作業系統之間插入 Hypervisor，把實體資源抽象為多組看似真實硬體的虛擬資源，使每個虛擬機都以為自己獨佔一台完整電腦，彼此完全不知道對方存在。這使供應商能把一台實體伺服器安全地分給多個互不信任的客戶——隔離由 Hypervisor 在硬體層保障，一個虛擬機當機或被入侵不會影響其他虛擬機。兩個主要代價：其一，資源開銷——每個虛擬機都須執行一套完整的作業系統，二十個應用就是二十份核心與系統服務各佔資源；其二，啟動時間——虛擬機真的要「開機」（載入核心、初始化裝置、啟動服務），需數十秒至數分鐘，在需要快速擴縮的場景中是嚴重限制。

第 2 題

本質差異在隔離層次：虛擬機在「硬體層」隔離，各自擁有完整的核心，模擬的是一台電腦；容器在「作業系統層」隔離，共用主機核心，隔離的是一個行程。因此產生的實際限制：其一，作業系統相容性——容器必須與主機核心相容，故無法在 Linux 主機上執行 Windows 容器，而虛擬機則可執行與主機完全不同的作業系統。其二，隔離強度——容器共用核心，若核心存在漏洞，理論上可能被用來突破隔離影響同主機的其他容器；虛擬機的核心分離則提供更強的安全邊界，故多租戶執行不受信任程式碼時仍需虛擬機。可補充：把容器當虛擬機用（在裡面塞一堆系統服務）雖技術上可行，卻違背其設計哲學並失去輕量與可替換的優勢。

第 3 題

分層：映像檔由多個唯讀層堆疊而成（如作業系統基礎層、執行環境層、相依套件層、應用程式層），不同映像檔可共用相同的層，故磁碟上只需儲存一份。寫時複製：容器執行時在所有唯讀層之上疊加一個可寫層，所有修改都發生在此層；若要修改來自唯讀層的檔案，系統會先複製到可寫層再修改。「資料會消失」是設計本意的原因：容器被設計為「可拋棄的」——可隨時停止、刪除、在他處重啟而不影響系統正確性，這正是自動化編排（第 29 章）能自由移動與重啟容器的前提。若容器內累積了不可重建的狀態，此假設即不成立。故正確做法是把狀態外部化到卷或外部服務，而非期待容器保存資料。

第 4 題

因為映像檔是分層的，每個建置步驟都會產生一個新的層，而所有層都完整保留在映像檔中。在後續步驟中「刪除」該檔案，只是在上層做了一個「此檔案已不存在」的標記，含有機密的那個下層仍原封不動地存在——任何取得該映像檔的人，都能從歷史層中把機密取出。而映像檔往往會被推送到共用的 Registry 供他人拉取，等於公開了機密。正確做法：機密資訊應於執行時透過環境變數或

專門的機密管理機制注入，而非寫入映像檔。可補充：此性質與第 23 章版本控制保留完整歷程的道理相同——凡是會保留歷史的系統，「事後刪除」都無法真正抹除。

二、計算題

第 1 題

$H = 256 \text{ GB}$ ，依式 28-1 計算。

- (1) $A = 3 \text{ GB}$ 時：虛擬機每實例 $3 + 1.5 = 4.5 \text{ GB}$ ，可承載 $\lfloor 256 / 4.5 \rfloor = 56$ 個；容器每實例 $3 + 0.05 = 3.05 \text{ GB}$ ，可承載 $\lfloor 256 / 3.05 \rfloor = 83$ 個。
- (2) 相對密度 $= 83 / 56 \approx 1.48$ 倍。
- (3) $A = 0.5 \text{ GB}$ 時：虛擬機每實例 2.0 GB ，可承載 128 個；容器每實例 0.55 GB ，可承載 $\lfloor 256 / 0.55 \rfloor = 465$ 個；相對密度 $= 465 / 128 \approx 3.63$ 倍。

差距擴大的原因：虛擬機的固定開銷（1.5 GB）與應用本身的大小無關。當應用很大（3 GB）時，這 1.5 GB 只佔單實例的三分之一，影響有限；但當應用很輕（0.5 GB）時，這 1.5 GB 竟是應用本身的三倍，開銷完全主宰了資源消耗。故應用越輕量、實例數越多的場景（如微服務），容器的密度優勢越懸殊。

第 2 題

$n = 12$ 、 $B = 600 \text{ MB}$ 、 $S = 80 \text{ MB}$ 。

- (1) 共用基礎層（式 28-2）： $600 + 12 \times 80 = 600 + 960 = 1,560 \text{ MB}$ 。
- (2) 不共用： $12 \times (600 + 80) = 12 \times 680 = 8,160 \text{ MB}$ 。
- (3) 節省比例（式 28-3） $= (12 - 1) \times 600 / (12 \times 680) = 6,600 / 8,160 \approx 0.809$ ，即約 80.9%。也可由 $1 - 1,560 / 8,160 \approx 80.9\%$ 驗證。

延伸思考：節省的不只是儲存，更包括部署時的傳輸量——若基礎層已在各節點快取，部署新版本時實際只需傳輸那 80 MB 的差異層。這也是為何統一基礎映像檔能同時改善儲存成本、部署速度與安全性維護（如案例四）。

三、思考與討論

第 1 題

結合第 23 章的四條件（程式碼、環境、資料、參數與流程）分析：容器直接解決了「環境」這一項——它把作業系統基礎、執行環境、相依套件與其版本全部打包為不可變的映像檔，使環境從「一份可能過時的安裝文件」變成「一個可執行、可驗證的定義」。依式 23-1 的連乘關係，環境往往是最弱的一環（表 23-5 顯示只做版本控制時可重現機率僅約 11%），故補強它對整體提升最為顯著。容器沒有解決的部分：其一，「資料」——容器不管理資料版本，仍需另行記錄資料來源與版本標記；其二，「參數與流程」——超參數、隨機種子與執行指令仍須另行記錄（部分可寫入啟動指令或設定檔）；其三，容器本身若使用浮動的版本標籤（如 latest），仍可能導致不同時間建置出不同的環境，故應鎖定明確的版本；其四，硬體差異（如 GPU 型號、驅動版本）可能仍造成結果不一致。周延的回答會指出：容器是必要而非充分條件。

第 2 題

具有此特性的工程紀律包括：其一，完整的日誌與可觀測性——平時無人查看，但故障時決定了能在十分鐘還是十小時內找到原因。其二，自動化測試與持續整合（第 23 章）——平時像是額外負擔，但在重構或緊急修補時提供了安全網。其三，備份與還原演練——備份人人都做，但「定期演練

還原」常被省略，直到真正需要時才發現備份不可用。其四，可重現的環境紀錄（本章主題）——平時自己的機器跑得動，接手或災難重建時才顯價值。其五，權限最小化與機密管理（第 22、33 章）——平時只是麻煩，外洩時決定了損害範圍。其六，資料血緣與文件（第 32 章）——平時大家都記得，人員異動後才發現無人知曉。說服團隊的做法：其一，把紀律內建於流程而非仰賴自覺（如持續整合自動檢查、範本預設包含必要設定），降低遵守的成本；其二，以真實案例（含組織內部或業界的事故）具體呈現代價，而非抽象呼籲；其三，量化呈現（如案例四的十五次修補變兩次）使效益可見；其四，管理層以身作則並在時程規劃中預留這些工作的時間，而非把它們當成「有空再做」。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 28 章 虛擬化與容器技術

台灣自編大學教科書

@prof

大數據分析與雲端運算

第柒篇 雲端運算平台與架構

Cloud Platforms & Architecture

第 29 章

容器編排與 Kubernetes

Container Orchestration and Kubernetes

機器人叫獸（李世淵） 編著

叫獸開講

上一章結尾我問了一連串問題：成百上千個容器，誰決定它們跑在哪台機器上？當掉了誰重啟？流量暴增時誰擴充？機器故障時上面的容器搬到哪裡去？

如果你覺得這些問題很眼熟，那是因為我們在第 10 章談 YARN 時，問過幾乎一模一樣的問題——只是那時被管理的是 MapReduce 任務，現在是容器。這正是我一直強調的：技術會換名字，但問題會留下。

不過 Kubernetes 有一個設計思想，我認為是它最值得學習的地方，也是它與 YARN 最大的不同。我用一個比喻來說明。

傳統的運維是「命令式」的：你下指令說「在三號機開一個容器」「把五號機的容器關掉」。就像你對助理說「去買咖啡、然後影印、然後訂便當」——你負責想清楚每一步，助理只負責執行。問題是，如果中途出了狀況（咖啡店關門了），助理不會自己處理，你得重新下指令。

Kubernetes 則是「宣告式」的：你不說要做什麼動作，只描述「我要的最終狀態」——例如「這個服務要有三個副本在運行」。然後系統會持續比對「現況」與「你要的狀態」，只要有落差就自動修正。某個副本掛了？它自己補一個。某台機器故障？它把上面的容器搬到別處。你不必下任何指令，因為你要的狀態從來沒變過。

這個從「怎麼做」到「要什麼」的轉變，你在第 12 章的 DataFrame、第 25 章的事件驅動架構都遇過。而在 Kubernetes 上，它成為了整個系統的核心哲學。

學習目標

研讀完本章之後，你應該能夠：

1. 說明容器編排要解決的問題，並指出其與第 10 章資源管理的關係。
2. 說明宣告式設計與調和迴圈的運作原理。
3. 描述 Kubernetes 的控制平面與工作節點的架構與職責。
4. 區分 Pod、Deployment、Service 等核心物件的角色。
5. 說明資源請求與上限的意義，及其對排程的影響。
6. 說明自動擴縮與自我修復的機制與其反應時間的組成。

核心概念

核心概念	說明
容器編排	自動化管理大量容器的部署、調度、擴縮與復原。
宣告式設計	描述期望的最終狀態，由系統自行決定如何達成。
調和迴圈	持續比對現況與期望狀態，並自動修正落差的機制。
Pod	Kubernetes 的最小部署單位，可含一或多個共生的容器。
Deployment	管理 Pod 的副本數與更新策略的控制器。
Service	為一組動態變動的 Pod 提供穩定存取端點的抽象。

核心概念	說明
資源請求與上限	宣告容器所需與可用資源的上下界，供排程與隔離之用。

表 29-1 本章核心概念一覽

29.1 為什麼需要編排

29.1.1 規模帶來的質變

管理三個容器，用手動就夠了。但當容器數量來到數百、數千個，且分散在數十台機器上時，事情就完全不同了。

需要被自動化處理的事情包括：決定每個容器該放在哪台機器（考量資源、親和性、分散性）、監控容器是否健康並在失敗時重啟、在機器故障時把容器遷移到其他節點、依負載自動增減副本數、更新版本時逐步替換而不中斷服務、以及讓服務彼此找得到對方（第 28 章的服務發現）。

這些事情若靠人力，不只做不完，更關鍵的是「反應太慢」——半夜三點某個容器掛了，等值班人員被叫醒處理，服務已經中斷了半小時。編排系統的價值就在於：它以秒為單位持續運作，永不疲倦。

29.1.2 與第 10 章的呼應

第 10 章我列出了資源管理器的四項工作：資源盤點、排程決策、資源隔離、生命週期管理。當時我說「這四項工作在 Kubernetes 中依然成立，只是被管理的對象改變」。現在我們來驗證這句話。

工作	YARN (第 10 章)	Kubernetes
資源盤點	NodeManager 回報本機資源	kubelet 回報節點狀態與可用資源
排程決策	ResourceManager 分配容器	Scheduler 決定 Pod 落在哪個節點
資源隔離	以 YARN 容器限制配額	以資源請求與上限限制配額
生命週期	ApplicationMaster 監控任務	控制器持續調和期望狀態

表 29-2 YARN 與 Kubernetes 的對應

差異主要有二。其一是「對象」：YARN 管理的是有明確終點的批次任務，Kubernetes 則主要面向長時間運行的服務（當然它也能跑批次）。其二，也更本質的是「控制方式」：YARN 是命令式的（ApplicationMaster 主動申請與調度），Kubernetes 則是宣告式的——這正是下一節的主題。

29.2 宣告式設計與調和迴圈

29.2.1 只描述你要什麼

承接叫獸開講。在 Kubernetes 中，你提交的不是「動作指令」，而是一份「期望狀態」的描述——通常是一個設定檔，寫著「這個應用要有 3 個副本、使用某個映像檔、每個需要多少資源、對外開放哪個連接埠」。

提交之後，你的工作就結束了。系統不會問你「那我現在該做什麼」，而是自己開始運作：發現目前有 0 個副本、期望有 3 個，於是建立 3 個。若其中一個掛了，它發現現況變成 2 個、期望仍是 3 個，於是再建一個。

這個「持續比對現況與期望、並自動修正落差」的機制，稱為「調和迴圈」（reconciliation loop）。它是 Kubernetes 所有功能的共同基礎——擴縮、修復、更新、遷移，全都是同一個迴圈在不同情境下的展現。

29.2.2 調和迴圈的三個步驟

- **觀察：**取得系統的當前實際狀態（有幾個副本在跑、各在哪個節點、健康嗎）。
- **比對：**與使用者宣告的期望狀態相比，找出落差。
- **行動：**採取必要的動作以縮小落差（建立、刪除、遷移）。

這三步不斷循環，永不停止。這帶來一個重要的性質：系統具有「自我修復」的能力，而且這個能力不是額外加上的功能，而是設計的必然結果——因為只要現況偏離期望，迴圈就會自動把它拉回來。

叫獸提醒

宣告式設計有一個實務上的重要後果，我要你特別注意：不要用手動指令去修改由控制器管理的資源。我看過學生發現某個 Pod 有問題，就手動把它刪掉——結果幾秒後它又出現了，學生以為系統壞了。其實系統運作得非常正常：你的期望狀態說要 3 個，你刪掉一個變成 2 個，調和迴圈當然會補回來。正確的做法是「修改期望狀態」（例如把副本數改為 0），而非「直接操作現況」。這個觀念的轉換，是初學 Kubernetes 最需要跨過的門檻。你要學會的不是「下指令改變系統」，而是「修改你的宣告，讓系統自己去達成」。

29.3 架構：控制平面與工作節點

29.3.1 兩層架構

Kubernetes 叢集分為「控制平面」與「工作節點」兩部分，這與第 5 章 HDFS、第 10 章 YARN 的主從架構一脈相承。

元件	位置	職責
API 伺服器	控制平面	所有操作的統一入口，負責驗證與寫入狀態
etcd	控制平面	以鍵值儲存保存整個叢集的狀態，是唯一的真實來源
排程器	控制平面	決定新的 Pod 應該放在哪個節點
控制器管理員	控制平面	執行各種調和迴圈，維持期望狀態
kubelet	每個工作節點	在本節點上實際建立與監控容器，並回報狀態
kube-proxy	每個工作節點	維護網路規則，實現服務的流量轉發

表 29-3 Kubernetes 的核心元件

其中 etcd 特別值得注意——它正是第 8 章談過的分散式協調服務，以 Raft 演算法保證多個副本間的一致性。整個叢集的狀態都保存在它裡面，因此它的可靠性至關重要，通常會部署 3 或 5 個副本（回想第 8 章式 8-1 的多數決門檻，這也是為何節點數取奇數）。

29.3.2 一切都經過 API 伺服器

Kubernetes 有一個重要的設計原則：所有元件都不直接互相溝通，一律透過 API 伺服器讀寫狀態。

例如排程器決定某個 Pod 該放在三號節點時，它不會去通知三號節點，而是把這個決定寫回 API 伺服器；三號節點上的 kubelet 則持續向 API 伺服器查詢「有沒有我該執行的 Pod」，發現後才動手建立。

這種「以共享狀態為中心」的設計，使各元件高度解耦——任何元件都可以獨立重啟、升級或替換，只要它遵守 API 的約定。這與第 25 章事件驅動架構的解耦精神相通，也是 Kubernetes 能有龐大擴充生態的原因。

29.4 核心物件：Pod、Deployment 與 Service

29.4.1 Pod：最小的部署單位

Pod 是 Kubernetes 中最小的部署單位。這裡有個常見的誤解需要澄清：Pod 不等於容器——一個 Pod 可以包含一個或多個容器。

同一個 Pod 內的容器共享網路命名空間（可用本機位址互相溝通）與儲存卷，並且總是被排程到同一個節點、一起啟動、一起終止。它們的關係是「共生」的。

多容器 Pod 的典型用途是「輔助容器」模式：主容器跑應用程式，旁邊放一個輔助容器負責收集日誌、或代理網路流量。但要注意——多數情況下，一個 Pod 一個容器才是常態，不要為了方便就把不相關的服務塞進同一個 Pod。

29.4.2 Deployment：管理副本與更新

你通常不會直接建立 Pod，而是透過 Deployment。它是一個控制器，負責維持指定數量的 Pod 副本，並管理版本更新。

Deployment 最有價值的功能之一是「滾動更新」：當你更新映像檔版本時，它不會一次替換所有副本（那會造成服務中斷），而是逐步進行——先啟動一個新版本的 Pod、確認健康後才終止一個舊版本的、如此逐一替換，直到全部更新完成。

若新版本有問題（例如啟動失敗或健康檢查不通過），更新會停止，且可以快速回退到前一版。這個能力使得「部署」從一件需要停機、需要提心吊膽的大事，變成了日常的例行操作——這也是第 23 章談的持續部署得以實現的基礎。

29.4.3 Service：穩定的存取端點

Pod 是短暫的——它可能因為更新、故障、遷移而隨時被替換，每次替換 IP 位址都會改變。那麼其他服務要如何穩定地存取它？

Service 就是為此而生的抽象。它為一組 Pod 提供一個固定的名稱與位址，並自動把流量分配到當下健康的 Pod 上。當 Pod 被替換時，Service 會自動更新其後端清單，呼叫方完全無感。

這正是第 28 章末提到的服務發現，在 Kubernetes 中被抽象為一個一級的物件。你只需以服務名稱呼叫，不必知道對方在哪台機器、有幾個副本、IP 是多少——這是宣告式思維在網路層的體現。

其他常用物件

除了三大核心物件，還有幾個實務上常用的：ConfigMap 與 Secret 分別保存設定與機密（呼應第 28 章「設定不應寫進映像檔」的原則）、PersistentVolume 提供持久化儲存、Job 與 CronJob 則用於執行一次性或定期的批次任務——這使 Kubernetes 也能承擔第參篇那類批次分析工作。

29.5 排程與資源管理

29.5.1 請求與上限

每個容器可以宣告兩個資源數值，它們的意義完全不同，混淆兩者是實務上最常見的錯誤之一。

- **請求 (Request)：**「我至少需要這麼多資源才能正常運作」。排程器依此決定該 Pod 能放在哪個節點——節點必須有足夠的剩餘量才會被選中。
- **上限 (Limit)：**「我最多只能用到這麼多」。這是強制的天花板，超過就會被限制（CPU）或終止（記憶體）。

兩者的關係決定了容器的服務品質。若請求等於上限，代表資源被保證且不會超用，最為穩定；若請求低於上限，則容器平時佔用較少但可在節點有餘裕時多用一些，密度較高但穩定性較低。

這裡有一個實務上的陷阱：記憶體的上限一旦超過，容器會被直接終止（因為記憶體不像 CPU 可以「慢一點」）。因此記憶體的上限必須留有足夠餘裕，否則容器會在流量尖峰時反覆被殺——而這往往在測試環境中不會發生，直到正式上線才爆發。

29.5.2 排程的考量

排程器決定 Pod 該放在哪個節點時，會考慮多項因素。

- **資源是否足夠：**節點的剩餘資源必須滿足 Pod 的請求量，這是最基本的過濾條件。
- **親和性與反親和性：**可指定「要與某些 Pod 放在一起」（如需低延遲通訊）或「不要放在一起」（如同一服務的多個副本應分散）。
- **節點選擇：**可指定 Pod 只能放在具備特定標籤的節點上，例如有 GPU 的節點。
- **汙點與容忍：**節點可標記「汙點」以排斥一般 Pod，只有明確宣告能容忍的 Pod 才會被排上去，常用於保留專用節點。

其中「反親和性」對可用性至關重要：若一個服務的三個副本全被排到同一台機器上，那台機器故障時服務就完全中斷——三個副本等於白做。這正是第 3 章式 3-1「彼此獨立」那個前提的實際應用，也是第 5 章 HDFS 機架感知的同一個思想。

29.6 自動擴縮與自我修復

29.6.1 健康檢查與自我修復

Kubernetes 提供兩種健康檢查，它們的用途不同，實務上常被混用。

- **存活探測：**檢查容器是否還「活著」。失敗時會重啟容器。用於偵測死鎖或無回應的情況。
- **就緒探測：**檢查容器是否「準備好接收流量」。失敗時不會重啟，只會把它從 Service 的後端移除。用於啟動期間或暫時無法服務時。

區分兩者的關鍵在於：一個應用可能活著但還沒準備好（例如正在載入大型模型），此時應該是「不要送流量過來」而非「把它殺掉重啟」。若誤用存活探測，容器會在啟動途中被反覆殺死，永遠無法就緒——這是實務上很常見且難以察覺的設定錯誤。

29.6.2 三個層次的自動擴縮

類型	調整什麼	適用情境
水平擴縮	增減 Pod 的副本數	最常用，適合無狀態服務應付流量變化
垂直擴縮	調整單一 Pod 的資源配置	適合難以水平擴充的應用，但調整需重啟
叢集擴縮	增減叢集的節點數	當現有節點資源不足以容納新 Pod 時

表 29-4 三個層次的自動擴縮

三者常需搭配運作：流量上升時，水平擴縮先增加 Pod 副本；當節點資源用罄而 Pod 無處可放時，叢集擴縮再向雲端申請新的節點。這個組合正是第 27 章「彈性」價值的具體實現——你不必預先為尖峰購置機器，系統會自己伸縮。

但要注意，水平擴縮的前提是應用「無狀態」——這正是第 28 章 28.4.1 節那條原則的原因。若容器內累積了不可重建的狀態，就無法隨意增減副本。

29.6.3 擴縮不是瞬間的

初學者常有的誤解是：設定了自動擴縮，系統就能「立刻」應付流量暴增。實際上，從偵測到新副本真正開始服務，中間有一連串的步骤，每一步都需要時間。

下一節的公式會把這個反應時間拆解開來——你會發現它可能長達數十秒，而這個延遲在面對突發流量時可能是致命的。理解它的組成，才能知道該優化哪一段。

29.6.4 承先啟後

本章我們認識了雲原生時代的作業系統。從編排要解決的問題出發（29.1），釐清了宣告式設計與調和迴圈這個核心哲學（29.2）、控制平面與工作節點的架構（29.3）、Pod、Deployment 與 Service 三大核心物件（29.4）、資源請求與上限及排程考量（29.5），以及健康檢查與三層自動擴縮（29.6）。

我最希望你帶走的，是那個從「命令式」到「宣告式」的思維轉變——你描述期望，系統負責達成。這個思想不只在 Kubernetes，你在第 12 章的 DataFrame、第 25 章的事件驅動架構都見過它而它幾乎總是帶來更好的可維護性與自癒能力。

到這裡，我們已經有了完整的運算基礎設施——虛擬化提供隔離、容器提供一致的封裝、Kubernetes 提供自動化的管理。但還有一個問題沒有回答：這些系統要分析的「資料」，最終該用什麼架構來組織與儲存？第 5 章我們談過資料湖的雛形，但它有明顯的缺陷；而傳統的資料倉儲又太過僵固。有沒有辦法兼得兩者的優點？這就是下一章的主題——雲端資料倉儲、資料湖與 Lakehouse。

公式與流程：副本可用度與擴縮反應時間

這一節用兩組計算，回答兩個實務問題：該部署幾個副本？以及自動擴縮到底多快？

副本數與可用度

設單一 Pod 的可用度為 a ，部署 n 個彼此獨立的副本，且只要有一個存活服務即可運作，則整體可用度為：

$$A = 1 - (1 - a)^n$$

式 29-1 n 個副本的整體可用度

這條式子與第 3 章的式 3-1 完全相同——因為它們是同一個問題。下表以兩種單體可用度計算。

副本數 n	單體 95%	單體 99%	年停機 (99% 時)
1	95%	99%	約 3.65 天
2	99.75%	99.99%	約 53 分鐘
3	99.9875%	99.9999%	約 32 秒
5	> 99.9999%	> 99.99999%	小於 1 秒

表 29-5 副本數對整體可用度的影響

從 1 個增為 3 個副本，年停機時間從三天半降到三十餘秒。但請務必記得式 29-1 的前提——「彼此獨立」。若三個副本被排程到同一台機器上，那台機器故障時三個一起死，這個計算完全不成立。這正是 29.5.2 節反親和性設定的價值所在。

擴縮的反應時間

自動擴縮從「偵測到負載上升」到「新副本開始服務」，需經過一連串步驟。總反應時間為各段之和：

$$\text{反應時間} = T_{\text{偵測}} + T_{\text{排程}} + T_{\text{拉取映像}} + T_{\text{啟動}} + T_{\text{就緒}}$$

式 29-2 自動擴縮的總反應時間

階段	典型耗時	可優化的方向
指標偵測週期	15 秒	縮短採集間隔，但會增加監控負載
決策與排程	5 秒	多為固定開銷，優化空間有限
拉取映像檔	20 秒	縮小映像檔、預先在節點快取（見第 28 章）
容器啟動	5 秒	減少啟動時的初始化工作
就緒探測通過	10 秒	調整探測參數，但不可過於寬鬆
合計	約 55 秒	—

表 29-6 擴縮反應時間的組成

五十五秒意味著什麼？如果你的流量在十秒內暴增三倍，那麼在新副本就緒之前的這近一分鐘裡，既有的副本必須獨自承受全部壓力——很可能已經超載崩潰了。

叫獸提醒

表 29-6 給了兩個實務上極重要的推論。第一，自動擴縮「不能用來應付突發尖峰」，它只能應付「逐漸上升的負載」。若你預期會有瞬間的流量暴衝（如促銷開賣、產線同時啟動），正確的做法是「預先擴充」而非依賴自動擴縮——這需要對業務節奏的理解，而非純技術手段。第二，注意「拉取映像檔」佔了最大的一段（20 秒）。這正是第 28 章那些工程紀律的價值兌現處：縮小映像檔、統一基礎層讓節點能快取共用層、把不常變動的放下層——這些看似瑣碎的習慣，直接決定了你的系統能多快回應負載變化。技術決策的效益，往往要到好幾層之後才顯現。

案例研討

案例一 殺不死的 Pod

某學生發現一個 Pod 行為異常，於是手動把它刪除。幾秒後，一個新的 Pod 出現了。他再刪一次，又出現一個。他以為系統壞了，跑來問我是不是叢集有問題。

系統其實運作得完全正常。該 Pod 由一個 Deployment 管理，而 Deployment 的期望狀態宣告了「要有 3 個副本」。他刪掉一個，現況變成 2 個，調和迴圈偵測到落差，立刻補上一個——這正是 29.2.2 節那個迴圈在忠實執行。

正確的做法是修改期望狀態（把副本數改為 0，或直接刪除 Deployment），而非操作現況。這個案例是初學者最典型的困惑，也是宣告式思維最需要跨過的門檻——你不再「命令系統做事」，而是「修改宣告，讓系統自己去達成」。

案例二 三個副本一起死

某服務部署了 3 個副本，團隊認為依式 29-1 計算，可用度應達 99.99% 以上。但某次一台實體機器故障，服務卻完全中斷了。

檢查後發現，排程器把 3 個副本全都放在了同一個節點上——因為當時只有那台節點有足夠的剩餘資源。於是「3 個副本」在實體上等同於「1 個」，機器一掛全部陣亡。

這正是式 29-1「彼此獨立」前提被違反的後果，與第 3 章案例四「副本同機房共同故障」、第 5 章 HDFS 的機架感知是完全相同的問題。他們後來為該服務設定了反親和性規則，強制副本分散到不同節點。這個案例再次證明：可靠度的計算，前提永遠比公式本身更重要。

案例三 啟動途中被殺的模型服務

某團隊部署一個模型推論服務，該服務啟動時需要載入一個數 GB 的模型，約需 90 秒才能就緒。他們設定了存活探測，逾時 30 秒。

結果服務永遠起不來：容器啟動後 30 秒，模型還在載入、無法回應探測，於是被判定為「不健康」而遭重啟；重啟後又是同樣的循環，無限循環。

問題在於誤用了探測類型。載入中的容器是「活著但還沒準備好」，應該用就緒探測（把它排除在流量之外、耐心等待），而非存活探測（判定失敗就殺掉）。他們改為就緒探測並設定合理的初始延遲後，服務順利上線。這正是 29.6.1 節所警告的情況——兩種探測的用途不同，混用會導致難以察覺的故障。

案例四 來不及擴充的促銷

某電商在促銷開賣的瞬間，流量在數秒內暴增五倍。他們設定了自動擴縮，認為系統會自動應付，但實際上服務仍中斷了約一分鐘。

依式 29-2 分析，他們的反應時間約 55 秒——而流量是在數秒內到位的。在新副本就緒之前的那將近一分鐘，原有的副本承受了五倍的壓力而全數超載。

他們的改善分兩方面：技術上，縮小映像檔並在各節點預先快取基礎層，把拉取時間從 20 秒降到 3 秒；流程上，更重要的是改為「促銷前三十分鐘預先擴充」，不再依賴自動擴縮應付可預期的尖峰。這個案例印證了叫獸提醒的那句話——自動擴縮應付的是逐漸上升的負載，而非瞬間的暴衝。

實作活動

活動一 體驗調和迴圈

請在本機的 Kubernetes 環境（如 minikube 或 kind）中，親身體驗宣告式設計。

- 步驟 1.** 撰寫一個 Deployment 設定檔，指定副本數為 3，並部署你在第 28 章打包的映像檔。
- 步驟 2.** 確認 3 個 Pod 都在運行後，手動刪除其中一個，觀察系統的反應與所需時間。
- 步驟 3.** 把副本數改為 5 並重新套用設定，觀察系統如何達成新的期望狀態。
- 步驟 4.** 最後把副本數改為 0，說明這與「手動刪除 Pod」的結果有何本質差異。

活動二 測量你的擴縮反應時間

請實測式 29-2 的各個組成，找出你系統的瓶頸段落。

- 步驟 1.** 為活動一的 Deployment 設定水平自動擴縮，依 CPU 使用率觸發。
- 步驟 2.** 以壓力測試工具產生負載，記錄「負載上升」到「新 Pod 開始服務」的總時間。
- 步驟 3.** 分別記錄各階段的耗時：指標更新、Pod 建立、映像檔拉取、容器啟動、就緒探測通過。
- 步驟 4.** 找出耗時最長的階段並嘗試優化（例如預先拉取映像檔），再次測量並比較改善幅度。

活動三 設計高可用的部署

請為一個對可用性要求高的服務設計部署方案。首先依式 29-1，計算若單一 Pod 的可用度為 98%，要達到 99.99% 以上的整體可用度需要幾個副本。接著撰寫 Deployment 設定，包含：適當的副本數、資源請求與上限（並說明兩者的差異與設定理由）、反親和性規則（確保副本分散於不同節點）、以及存活與就緒兩種探測（說明各自的參數設定依據）。最後說明：若你的服務啟動需要 60 秒的初始化，探測參數該如何調整以避免案例三的問題？

延伸閱讀

- **Kubernetes 官方文件與概念指南：**涵蓋所有核心物件、排程機制與最佳實務，是本章最直接的參考。
- **Google Borg 論文：**說明 Kubernetes 前身的設計思想與十餘年的營運經驗，是理解其設計取舍的第一手資料。
- **《Kubernetes in Action》：**以循序漸進的方式介紹各項功能與其背後原理，適合系統性學習。
- **雲原生運算基金會的成熟度模型：**說明組織導入雲原生技術的階段與常見陷阱，可作為 29.1 節的實務延伸。

本章小結

1. 容器編排要自動化處理排程、健康監控、故障遷移、擴縮、滾動更新與服務發現；其價值不只在於自動化，更在於以秒為單位持續運作的反應速度。
2. Kubernetes 與第 10 章 YARN 的四項工作——對應，差異在於管理對象（長期服務 vs 批次任務）與控制方式（宣告式 vs 命令式）。
3. 宣告式設計讓使用者只描述期望狀態，由調和迴圈持續「觀察、比對、行動」以自動修正落差；自我修復不是額外功能，而是此設計的必然結果。實務上不應以手動指令操作由控制器管理的資源，而應修改期望狀態。

4. 架構分控制平面（API 伺服器、etcd、排程器、控制器管理員）與工作節點（kubelet、kube-proxy）；etcd 即第 8 章的協調服務，以 Raft 保證一致性，故副本數取奇數。所有元件皆透過 API 伺服器讀寫狀態而不直接溝通，達成高度解耦。
5. Pod 是最小部署單位（可含多個共生容器）、Deployment 管理副本數與滾動更新、Service 為動態變動的 Pod 提供穩定的存取端點——即服務發現在 Kubernetes 中的一級抽象。
6. 資源請求決定排程落點、上限則是強制的天花板；記憶體超過上限會直接終止容器，故須留足餘裕。排程另考量親和性、節點選擇與汙點容忍，其中反親和性對可用性至關重要。
7. 存活探測失敗會重啟容器、就緒探測失敗僅移出流量；誤把需要長時間初始化的服務設為存活探測，會導致啟動途中被反覆殺死。自動擴縮分水平、垂直與叢集三個層次，水平擴縮的前提是應用無狀態。
8. 整體可用度為 $1 - (1 - a)^n$ （式 29-1），但前提是副本彼此獨立，故須以反親和性強制分散；擴縮反應時間為各階段之和（式 29-2），典型約 55 秒，故自動擴縮只能應付逐漸上升的負載，突發尖峰應預先擴充。

習題

一、觀念題

1. 請說明宣告式設計與命令式設計的差異，並解釋為何宣告式能帶來自我修復的能力。
2. 為什麼手動刪除由 Deployment 管理的 Pod 之後，它會自動重新出現？正確的做法應該是什麼？
3. 請說明資源「請求」與「上限」的差異，並解釋為何記憶體的上限特別需要留餘裕。
4. 存活探測與就緒探測有何不同？誤用會造成什麼後果？請以一個需要長時間初始化的服務說明。

二、計算題

1. 某服務的單一 Pod 可用度為 96%。（一）依式 29-1 計算部署 2、3、4 個副本時的整體可用度；（二）若要求整體可用度達 99.99% 以上，至少需要幾個副本？（三）若這些副本被排到同一節點上，實際可用度為多少？說明其意涵。
2. 某系統的擴縮各階段耗時為：指標偵測 30 秒、決策排程 5 秒、拉取映像檔 40 秒、容器啟動 8 秒、就緒探測 12 秒。（一）依式 29-2 計算總反應時間；（二）若把指標偵測週期縮短為 15 秒、並預先快取映像檔使拉取降為 5 秒，新的反應時間為何？（三）改善了多少百分比？

三、思考與討論

1. 本章指出「自動擴縮只能應付逐漸上升的負載」。請就一個你熟悉的情境討論：哪些負載變化是可預期的？你會如何結合預先擴充與自動擴縮來因應？
2. 案例四中，改善映像檔的大小與快取直接縮短了擴縮反應時間。請討論：本書前面章節中，還有哪些「當時看似無關」的技術決策，最終影響了系統層級的表現？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

命令式：使用者下達具體的動作指令（在某節點開一個容器、關掉某個容器），系統被動執行；若中途出現狀況，系統不會自行處理，須由使用者重新下指令。宣告式：使用者只描述期望的最終狀態（這個服務要有 3 個副本），由系統自行決定如何達成。自我修復的原因：宣告式配合調和迴圈——系統持續「觀察現況、與期望比對、採取行動縮小落差」；只要現況偏離期望（副本掛了、節點故障），迴圈就會自動把它拉回來。故自我修復不是額外加上的功能，而是此設計的必然結果——因為期望狀態從未改變，系統會不斷努力達成它。

第 2 題

因為該 Pod 由 Deployment 管理，而 Deployment 宣告的期望狀態是「要有 N 個副本」。手動刪除一個後，現況變為 $N-1$ ，調和迴圈偵測到與期望的落差，於是立刻建立一個新的 Pod 來補足——系統運作完全正常，正是在忠實執行使用者的宣告。正確做法：修改期望狀態，而非操作現況。例如把副本數改為 0（暫停服務）或刪除整個 Deployment（移除服務）。此即宣告式思維的核心轉換——不再「命令系統做事」，而是「修改宣告，讓系統自己去達成」（見案例一）。

第 3 題

請求：宣告「至少需要這麼多資源才能正常運作」，排程器據此決定 Pod 能放在哪個節點——節點須有足夠剩餘量才會被選中。上限：宣告「最多只能用到這麼多」，是強制執行的天花板。記憶體上限需留足餘裕的原因：CPU 超過上限時只是被限制速度（容器變慢但仍存活），而記憶體超過上限時容器會被直接終止——因為記憶體無法「慢一點用」，超出即無法配置。故若記憶體上限設得太緊，容器會在流量尖峰或處理大筆資料時被反覆殺死；更麻煩的是，這往往在測試環境（負載低）中不會發生，直到正式上線才爆發。

第 4 題

存活探測：檢查容器是否還「活著」，失敗時重啟容器，用於偵測死鎖或無回應。就緒探測：檢查容器是否「準備好接收流量」，失敗時不重啟，僅把它從 Service 的後端清單移除。誤用的後果：以需要 90 秒載入模型的推論服務為例，若設為存活探測且逾時 30 秒，則容器啟動 30 秒後模型仍在載入、無法回應，會被判定不健康而遭重啟；重啟後又是同樣循環，服務永遠無法就緒（見案例三）。正確做法是使用就緒探測（讓它不接收流量但耐心等待），並為存活探測設定足夠的初始延遲，使其在初始化完成後才開始檢查。

二、計算題

第 1 題

依式 29-1, $A = 1 - (1 - a)^n$, $a = 0.96$, 故 $1 - a = 0.04$ 。

- (1) $n = 2$: $1 - 0.04^2 = 1 - 0.0016 = 0.9984$ (99.84%) ; $n = 3$: $1 - 0.04^3 = 1 - 0.00064 = 0.99936$ (99.936%) ; $n = 4$: $1 - 0.04^4 = 1 - 0.0000256 \approx 0.999974$ (99.9974%) 。
- (2) 要求 99.99% 以上: $n = 3$ 已達 99.936%, 略低於 99.99%; $n = 4$ 達 99.9974%, 滿足要求。故至少需 4 個副本。
- (3) 若全排在同一節點: 該節點故障時所有副本一起失效, 實際可用度退化為單一節點的可用度 (約 96%, 甚至更低, 因還須計入節點本身的故障率) 。

意涵：式 29-1 的「彼此獨立」是前提而非細節。副本數再多，若共用同一個故障域（同一節點、同一機架、同一可用區），指數級的可靠度提升就完全不成立——這與第 3 章案例四、第 5 章 HDFS 機架感知是同一個教訓。故必須以反親和性強制分散（見案例二）。

第 2 題

依式 29-2，總反應時間為各階段之和。

- (1) 原始： $30 + 5 + 40 + 8 + 12 = 95$ 秒。
- (2) 改善後： $15 + 5 + 5 + 8 + 12 = 45$ 秒。
- (3) 改善幅度 = $(95 - 45) / 95 = 50 / 95 \approx 52.6\%$ 。

延伸思考：改善的兩項合計貢獻了 50 秒，其中「映像檔拉取」一項就佔 35 秒——這正是第 28 章那些工程紀律（縮小映像檔、統一基礎層以利快取、不常變動的放下層）的價值兌現處。即使如此，45 秒的反應時間仍不足以應付瞬間暴增的流量，故對可預期的尖峰仍應採預先擴充。

三、思考與討論

第 1 題

可預期的負載變化包括：其一，時間規律性的——上下班尖峰、午餐時段、月底結帳、每日的批次作業時段。其二，事件驅動的——促銷開賣、新品發表、報名截止、考試選課開放。其三，週期性的——週末與平日差異、季節性需求、學期行事曆（如選課系統）。其四，業務流程觸發的——產線換線、班次交接、每月報表產出。因應策略：對可預期的尖峰採「排程式預先擴充」，在事件前依歷史資料預先把副本數拉高（可用 CronJob 或排程觸發），事件結束後再縮回；對不可預期的變化則保留自動擴縮作為安全網。此外可設定「最小副本數」高於平時需求，以保留緩衝吸收前 55 秒的衝擊。周延的回答會指出：這需要業務知識與技術手段結合，純技術方案無法解決。

第 2 題

可舉的例子包括：其一，第 4 章選擇欄式格式（Parquet）——當時只覺得是儲存細節，卻決定了第 12 章述詞下推與欄位裁剪能否生效，進而影響查詢成本（第 27 章案例三的天價帳單）。其二，第 9 章 MapReduce 任務設計為無狀態可重跑——當時是為了容錯，卻使系統能使用第 27 章的競價型資源而大幅降低成本。其三，第 13 章的冪等性——為了重試而設計，卻在第 25 章的至少一次語意、第 26 章的串流恢復中反覆成為關鍵前提。其四，第 28 章統一基礎映像檔——為了節省儲存，卻在本章直接縮短了擴縮反應時間，並在安全性修補時省下大量人力。其五，第 7 章的分區鍵選擇——影響了熱點、順序保證與日後能否增加分區。共同模式：基礎層的設計決策會沿著整個技術堆疊向上傳播，其影響往往在數層之後、數月之後才顯現——這正是「原理比工具重要」的實際理由，因為只有理解原理，才能預見這些跨層的連鎖效應。

第 30 章

雲端資料倉儲、資料湖

與 Lakehouse

Cloud Data Warehouse, Data Lake, and Lakehouse

機器人叫獸（李世淵） 編著

叫獸開講

我想用兩種圖書館來開場。

第一種是傳統的大學圖書館。每一本書進館前，館員要先分類、編目、貼標籤、決定放在哪一層架上。整理得極為井然有序，你要找什麼，查目錄就能精準定位。代價是——進館的流程很慢，而且如果有一份東西不符合現有的分類體系（比如一箱手稿、一捲錄音帶），館員可能就拒收了。

第二種是某個研究團隊的資料室。他們的原則是「先收下再說」——任何東西進來就往架上放，暫不分類。好處是什麼都不會被拒絕、也不會遺漏；壞處是幾年後，那個房間變成了沒人敢進去的儲藏室，東西都在裡面，但沒人找得到。

這兩種圖書館，就是資料倉儲與資料湖。前者井然有序但僵固，後者彈性極大卻容易淪為「資料沼澤」。過去十幾年，企業被迫在兩者間二選一，甚至同時維護兩套系統——資料先進湖、再挑一部分整理進倉儲，一份資料存兩次、對不上時還要花大量時間查為什麼。

這一章要談的 Lakehouse，就是要打破這個二選一。它的問題意識很直接：既然湖的問題是「缺少秩序」，那我們能不能在湖上「加一層秩序」，而不必把資料搬到另一個系統？答案是可以，而做法出乎意料地優雅。

學習目標

研讀完本章之後，你應該能夠：

1. 區分 OLTP 與 OLAP 兩種工作負載，並說明其對系統設計的影響。
2. 說明資料倉儲的架構與維度建模的基本概念。
3. 說明資料湖的價值與其淪為資料沼澤的原因。
4. 說明 Lakehouse 如何以中繼資料層兼得兩者的優點。
5. 說明開放表格式提供的交易、時間旅行與綱要演進能力。
6. 運用分區與檔案大小的原則，規劃高效的資料佈局。

核心概念

核心概念	說明
OLTP 與 OLAP	前者為高頻小量的交易處理，後者為大量掃描的分析查詢。
資料倉儲	以預先定義的結構整理後儲存，供分析查詢的集中式系統。
維度建模	以事實表與維度表組織資料的分析導向設計方法。
資料湖	以原始形式集中保存各類資料、讀取時才賦予結構的儲存庫。
資料沼澤	資料湖因缺乏治理而淪為無人能用的堆積場。
Lakehouse	在資料湖之上疊加中繼資料層，兼具湖的彈性與倉儲的可靠。
開放表格式	如 Delta Lake、Iceberg、Hudi，提供交易與綱要管理的表抽象。

表 30-1 本章核心概念一覽

30.1 OLTP 與 OLAP

30.1.1 兩種截然不同的工作負載

在談架構之前，必須先釐清一個根本的區分——因為資料倉儲之所以存在，正是為了回應這個區分。

面向	OLTP（交易處理）	OLAP（分析處理）
典型操作	讀寫少數幾筆特定紀錄	掃描大量紀錄後彙總
查詢頻率	極高（每秒數千至數萬次）	較低（每分鐘數次至數十次）
回應要求	毫秒級	秒級至分鐘級可接受
涉及欄位	通常需要整筆紀錄的所有欄位	只需少數幾個欄位
資料量	當前狀態，相對小	歷史累積，極大
典型系統	訂單系統、庫存管理	報表、儀表板、資料探勘

表 30-2 OLTP 與 OLAP 的比較

你應該注意到「涉及欄位」那一列——OLTP 要整筆紀錄、OLAP 只要少數欄位。這正是第 4 章列式與欄式儲存的分野：前者適合列式，後者適合欄式。同一個道理，在這裡以工作負載的語言被重述了一次。

30.1.2 為何不能混用同一個系統

既然資料都在交易系統裡，為什麼不直接在上面跑分析？這是初學者常有的疑問，而答案有三個層次。

第一是效能干擾。一支掃過數億筆紀錄的分析查詢，會大量佔用資料庫的資源，導致同時進行的交易處理變慢甚至逾時。讓報表拖垮訂單系統，是絕對不能接受的。

第二是儲存佈局不同。交易系統以列式儲存為主（要整筆讀寫），而分析需要欄式（只讀少數欄）；同一份資料無法同時對兩者最佳化。

第三是資料範圍不同。交易系統通常只保留當前狀態與近期資料（舊資料會被封存），但分析往往需要數年的歷史。

這三個理由合起來，就是「把分析資料另外集中管理」的必要性——而這正是資料倉儲的起源。

30.2 資料倉儲與維度建模

30.2.1 為分析而設計的結構

資料倉儲的核心思想是：從各個來源系統把資料抽取出來，經過清理與轉換後，以「適合分析」的結構集中儲存。

這裡的關鍵是「適合分析的結構」。交易系統為了避免資料重複與更新異常，會採用高度正規化的設計——一筆訂單的資訊可能散布在七、八張表中。這對交易處理很好，但對分析卻是災難：每次查詢都要連接大量表格，既難寫又慢。

維度建模因此採取相反的策略：適度地反正規化，把資料組織成少數幾張大表，讓分析查詢只需簡單的連接就能完成。這與第 6 章 NoSQL 的反正規化思路相通——都是為了讀取效率，接受一定程度的資料重複。

30.2.2 事實表與維度表

維度建模的核心是把資料分為兩類。

- **事實表**：記錄可量測的「事件」與其數值，如每一筆銷售的金額與數量、每一次生產的良率與耗時。它通常很大（數億至數十億列），但欄位不多。
- **維度表**：記錄描述性的「情境」，如產品資訊、時間、地點、機台。它們相對小，但欄位豐富，用於篩選與分組。

典型的分析查詢就是：從事實表取出數值，依維度表的屬性分組彙總——例如「依產品類別與月份，統計銷售總額」。這種以一張事實表為中心、周圍連接多張維度表的結構，因形狀而被稱為「星狀綱要」。

以智慧製造為例：事實表可能是「每一次生產紀錄」（含良率、耗時、能耗），維度表則包括產品、機台、班別、原料批次。有了這個結構，「比較不同班別在各機台上的良率差異」這類分析就變得直觀。

30.2.3 資料倉儲的限制

資料倉儲運作良好數十年，但在大數據時代遇到了三個難以克服的限制。

- **只能容納結構化資料**：影像、聲音、自由文字、感測波形這些非結構化資料，無法塞進事實表與維度表的結構中。
- **寫時綱要的僵固**：資料必須先符合預定結構才能載入（第 4 章的寫時綱要）。新的資料型態要先改綱要，流程冗長；而不確定未來用途的資料，往往就被排除在外。
- **成本高昂**：傳統資料倉儲的儲存與運算緊密耦合，擴充成本高；且為了「先整理好」，大量資料在進入前就被丟棄，日後想回頭分析已無從取得。

第三點特別值得深思。資料倉儲的哲學是「只保留你確定有用的」，但大數據時代的洞見往往來自「當初以為沒用的資料」。這個哲學上的衝突，正是資料湖興起的原因。

30.3 資料湖與資料沼澤

30.3.1 先收下再說

資料湖的哲學與資料倉儲恰恰相反：把所有資料以原始形式先存下來，不預設結構，等到要分析時才決定如何解讀（第 4 章的讀時綱要）。

這個轉變之所以可行，關鍵在於第 5 章談過的物件儲存——它的成本低到讓「先全部存下來」變得經濟合理。當儲存一 TB 的年成本只有數千元時，為了節省空間而丟棄資料，反而是不划算的。

資料湖的價值很明確：能容納任何型態的資料、保留完整的原始細節、且不必事先決定用途。這使它特別適合探索性分析與機器學習——這些場景往往需要原始資料，而非已被彙總過的結果。

30.3.2 為什麼湖會變成沼澤

但「先收下再說」有一個致命的副作用：如果只收不管，幾年後就沒人知道裡面有什麼了。這就是叫獸開講中那個「沒人敢進去的儲藏室」——業界稱為「資料沼澤」。

資料湖淪為沼澤，通常源於幾個缺失。

- **缺乏中繼資料：**沒有紀錄「這批資料是什麼、從哪來、欄位什麼意思」，後人看到一堆檔案卻無從理解。
- **缺乏綱要管理：**上游改了欄位卻無人知曉（第 2、13 章的老問題），同一個目錄下的檔案格式不一致。
- **缺乏交易保證：**多個作業同時寫入時互相覆蓋；作業中途失敗留下半成品檔案，讀取者無從分辨。
- **缺乏品質控管：**壞資料與好資料混在一起，且沒有機制阻擋。
- **小檔案氾濫：**串流作業每分鐘寫一個小檔，一年下來數十萬個檔案，查詢效能急遽劣化（本章公式會量化這個問題）。

叫獸提醒

我要指出一個很重要的觀察：這五項缺失，沒有一項是「儲存技術」的問題——物件儲存本身運作得非常好。它們全都是「管理能力」的缺失。這也解釋了為什麼資料湖不是一個「產品」，而是一種「架構」：你買不到一個叫做資料湖的東西，你只能建立一套包含儲存、中繼資料、治理與品質控管的完整實踐。多數失敗的資料湖專案，都是把它當成「一個很便宜的大硬碟」——把資料倒進去就以為完事了。這也是為什麼第 32 章的資料治理不是可有可無的配套，而是資料湖能否成立的前提。

30.4 Lakehouse：在湖上加一層秩序

30.4.1 問題意識與解法

面對「倉儲太僵固、湖太混亂」的困境，過去常見的做法是兩者並存：資料先進湖保留原始細節，再挑一部分整理進倉儲供正式報表使用。

但這個雙軌架構的代價不小：同一份資料存兩份（成本加倍）、兩邊的口徑可能不一致（第 26 章 Lambda 架構同樣的問題）、資料從湖搬到倉儲有延遲、且維護兩套系統需要兩套技能。

Lakehouse 的問題意識是：湖缺的其實不是儲存能力，而是「秩序」——交易保證、綱要管理、中繼資料。那麼，我們能不能不搬動資料，而是在湖上「疊加一層」提供這些能力？

答案是可以。做法是在物件儲存的檔案之上，增加一層「中繼資料層」——它記錄了「哪些檔案構成當前這張表」「這張表的綱要是什麼」「每次變更的歷史」。有了這一層，一堆散落的 Parquet 檔案就搖身一變，成為一張具有交易保證的「表」。

30.4.2 中繼資料層如何運作

這個機制的核心其實很直觀：既然檔案本身不可修改（物件儲存的特性），那就用一份「清單」來定義「當前這張表包含哪些檔案」。

新增資料時，寫入新的檔案，然後更新清單把它納入。刪除或修改時，寫入新版本的檔案，並更新清單把舊檔案排除。而關鍵在於——清單的更新是原子性的：要嘛完全生效，要嘛完全沒發生。

這個「以不可變的檔案加上可切換的清單指標」的設計，帶來了幾個原本只有資料庫才有的能力：

- **交易保證：**寫入作業要嘛完整生效、要嘛完全不生效，讀取者永遠不會看到半成品。

- **並行控制**：多個作業同時寫入時，透過清單的版本檢查來偵測衝突（呼應第 7 章的並行控制）。
- **時間旅行**：因為舊版本的清單與檔案都還在，可以查詢「三天前這張表長什麼樣」。
- **綱要演進**：清單記錄了綱要，可安全地增刪欄位而不破壞既有的讀取者（呼應第 4 章的綱要演進）。

你應該覺得這個設計很眼熟——不可變的資料加上可切換的指標，這正是第 11 章 RDD、第 25 章追加式日誌、第 28 章映像檔分層的同一個思想。整本書走到這裡，你會發現真正的核心觀念其實不多，只是不斷以不同形式出現。

30.4.3 三種開放表格式

實作這個中繼資料層的主流方案有三種：Delta Lake、Apache Iceberg 與 Apache Hudi。它們的核心思想相同，在細節上各有側重——有的更強調更新與刪除的效率、有的更強調與多種查詢引擎的相容性、有的更強調大規模表的中繼資料效能。

對學習者而言，重點不在於記憶各家的差異（它們也持續在演進），而在於理解那個共通的核心：以中繼資料層為不可變的檔案賦予表的語意。掌握這個原理，你面對任何一種格式都能快速上手。

特別要強調「開放」二字的價值。這些格式的規格是公開的，多種查詢引擎都能讀取——這意味著你的資料不會被綁定在單一廠商的系統中。這正是第 27 章談廠商鎖定時所說的「以開放標準保留可攜性」，在資料層的具體實踐。

30.5 分層架構與資料佈局

30.5.1 銅銀金三層

第 2 章我們提過資料的分層儲存，這裡把它講完整。實務上常以三層來組織 Lakehouse 中的資料。

層級	內容	用途
銅（原始層）	原封不動的原始資料，僅加上擷取時間等中繼資訊	保留完整細節、可回溯重算、稽核依據
銀（整備層）	經清理、去重、格式統一、綱要標準化後的資料	供多數分析與機器學習使用
金（服務層）	依特定用途彙總的資料，如各主題的維度模型	供報表、儀表板與業務系統直接取用

表 30-3 Lakehouse 的三層架構

這個分層的價值在於「職責分離」。銅層負責「不遺失任何東西」，銀層負責「品質可信」，金層負責「使用方便」。當下游發現問題時，可以回到上一層重新處理，而不必回頭向來源系統重新索取資料——這正是第 13 章談回填能力時的基礎。

你也會注意到，金層的內容其實就是 30.2 節的維度模型。也就是說，Lakehouse 並沒有拋棄資料倉儲的智慧——它只是把倉儲變成了湖中的一層，而非另一個獨立的系統。

30.5.2 分區：讓查詢少讀資料

資料佈局中最重要的決策是「分區」——依某個欄位的值，把資料分別放到不同的目錄中。最常見的是依日期分區。

它的價值在於「分區裁剪」：當查詢帶有該欄位的條件時（如「查上週的資料」），引擎能直接跳過不相關的目錄，連讀都不讀。這與第 12 章的述詞下推是同一個思想，但作用在更粗的粒度上，效果也更顯著。

分區欄位的選擇原則有三：應是「查詢最常用來篩選」的欄位（否則裁剪無從發揮）、基數不宜過高（否則產生大量小分區）、且分布不宜過度傾斜（否則某些分區異常龐大）。日期通常是最佳選擇，因為它同時滿足這三點。

30.5.3 小檔案問題

最後一個關鍵議題是檔案大小。串流寫入的場景中，若每分鐘產生一個檔案，一年就是五十多萬個小檔——而這會嚴重拖垮查詢效能。

原因在於每讀一個檔案都有固定開銷：建立連線、讀取檔頭、解析中繼資料。這個開銷與檔案大小無關，因此檔案越小，開銷佔比越高。下一節的公式會把這個成本算出來，你會發現它的影響遠超預期。

解法是定期執行「壓實」（compaction）——把大量小檔案合併成少數大檔案。多數開放表格式都提供了這個功能，而在 Lakehouse 架構下，因為有交易保證，壓實可以在不影響讀取者的情況下線上進行。這也是相較於「純資料湖」的一項實質優勢。

30.6 查詢引擎與架構選擇

30.6.1 儲存與運算分離的成熟

Lakehouse 架構的一個重要特性，是它徹底實踐了第 1 章談過的「儲存運算分離」。資料以開放格式躺在物件儲存中，而運算則由各種引擎按需啟動——批次用 Spark（第 11、12 章）、串流用串流引擎（第 26 章）、互動查詢用專門的查詢引擎。

這帶來三個實質好處：儲存與運算可各自獨立擴充（不必為了多存資料而買運算資源）、不同用途可選用最適合的引擎、以及運算資源用完即釋放（第 27 章的彈性價值）。

這也解答了一個常見的疑問：為什麼要用開放格式而非某個資料庫的專有格式？因為專有格式只有該資料庫能讀，你就被迫用它的引擎做所有事；而開放格式讓同一份資料能被多種引擎共用，這才是「儲存運算分離」真正的意義。

30.6.2 該選哪一種架構

釐清了三種架構後，實務上該如何選擇？

架構	適合的情境	主要考量
資料倉儲	資料型態單純、需求穩定、以固定報表為主	成熟穩定、工具生態完整
資料湖	型態多樣、探索性強、以機器學習為主	彈性最大，但須有治理配套
Lakehouse	兼有結構化報表與探索性分析需求	架構統一、避免資料重複，是當前主流方向

表 30-4 三種資料架構的適用情境

我的建議是：新建系統時，Lakehouse 通常是合理的預設選擇——它保留了未來的彈性，又不犧牲可靠性。但若你的需求極為單純（只有結構化資料、只做固定報表），傳統資料倉儲的成熟度與易用性仍有其價值，不必為了新技術而增加複雜度。

這也呼應本書反覆的立場：技術選擇應由需求驅動，而非由新穎程度驅動。理解每種架構要解決什麼問題，比記住哪個最新更重要。

30.6.3 承先啟後

本章我們完成了資料架構的全貌。從 OLTP 與 OLAP 的根本區分出發（30.1），理解了資料倉儲的維度建模與其三個限制（30.2）、資料湖的彈性與淪為沼澤的原因（30.3）、Lakehouse 如何以中繼資料層兼得兩者（30.4）、銅銀金分層與分區、小檔案等佈局實務（30.5），以及儲存運算分離的成熟與架構選擇（30.6）。

到這裡，第柒篇「雲端運算平台與架構」五章圓滿收束。回顧這一篇：第 27 章釐清了雲端的本質與成本、第 28 章說明了虛擬化與容器如何提供隔離與一致性、第 29 章展示了 Kubernetes 如何自動化管理、第 30 章則完成了資料架構的最後一塊拼圖。運算與儲存的基礎設施，至此完備。

但基礎設施完備，不等於系統可信。資料湖之所以會變成沼澤，關鍵不在技術而在治理——誰知道這批資料從哪來？誰保證它的品質？誰能存取它？出了問題如何追溯？這些問題若沒有答案，再先進的架構也只是把混亂搬到了新平台上。這就是第捌篇要處理的主題，我們從資料治理與資料品質開始。

公式與流程：小檔案成本與分區裁剪效益

這一節用兩組計算，量化資料佈局的兩個關鍵決策：檔案該多大、以及分區能省多少。

小檔案的固定開銷

設總資料量為 D 、每個檔案大小為 s ，則檔案數為 D/s 。若每開啟一個檔案的固定開銷為 t ，則總開銷為：

$$\text{開檔總開銷} = (D / s) \times t$$

式 30-1 小檔案的固定開銷

關鍵在於這個開銷「與檔案大小無關」——無論檔案是 1 MB 還是 128 MB，開啟它的成本都差不多。因此檔案越小，開銷佔比越高。以總量 125 GB、每次開檔 15 毫秒為例：

單檔大小	檔案數	開檔總開銷	判讀
128 MB	1,000 個	15 秒	合理
16 MB	8,000 個	120 秒	開始明顯
1.28 MB	100,000 個	1,500 秒 (25 分鐘)	嚴重劣化
0.128 MB	1,000,000 個	15,000 秒 (逾 4 小時)	完全不可用

表 30-5 小檔案對查詢的影響（總量 125 GB、開檔 15 毫秒）

看最後兩列：同樣的 125 GB 資料，只因為切成了小檔案，光是「開啟檔案」這件事就要花上數小時——而這還沒開始讀取任何實際內容。這就是為什麼串流寫入必須搭配定期壓實。

實務上的經驗法則是：單檔大小以 128 MB 至 1 GB 為宜。太小則開檔開銷過高，太大則不利於平行處理（回想第 5 章 HDFS 區塊大小的類似取舍）。

分區裁剪的效益

設資料依日期分區、共涵蓋 N 天，而查詢只需其中 n 天。則需掃描的資料比例為：

$$\text{掃描比例} = n / N$$

式 30-2 分區裁剪後的掃描比例

若再結合第 12 章的欄位裁剪（只讀 k 個欄位中的 C 欄），則實際讀取量相對於全表掃描的比值為：

$$\text{實際讀取比} = (n / N) \times (k / C)$$

式 30-3 分區裁剪與欄位裁剪的合併效益

以三年（1,095 天）的資料、共 40 個欄位為例，計算幾種常見查詢的讀取比例。

查詢範圍	分區裁剪	取 5 欄	合併讀取比
1 天	0.09%	12.5%	約 0.011%
7 天	0.64%	12.5%	約 0.080%
30 天	2.74%	12.5%	約 0.342%
365 天	33.33%	12.5%	約 4.17%

表 30-6 分區與欄位裁剪的合併效益（1,095 天、40 欄）

叫獸提醒

表 30-6 第一列那個數字值得你記住：查一天、取五欄，實際只需讀取全表的萬分之一。若全表有 10 TB，這支查詢只需讀約 1 GB。而同樣的查詢若沒有分區、資料又存成 CSV，就得掃描整整 10 TB——在按掃描量計費的雲端服務上，這就是第 27 章案例三那張天價帳單的由來。我要你注意的是：這個一萬倍的差距，不是來自更快的機器或更聰明的演算法，而是來自兩個在「當初存資料時」就做好的決定——依日期分區、以欄式格式儲存。資料佈局是那種「決定的當下毫無感覺、但影響持續數年」的選擇，值得你在專案初期就認真對待。

案例研討

案例一 沒人敢碰的資料湖

某企業三年前建置了資料湖，把所有系統的資料都倒了進去。初期進展順利，但兩年後新進的分析師發現：湖裡有數萬個目錄，卻沒有任何說明——不知道哪些是正式資料、哪些是某人的暫存；不知道欄位的意義；也不知道某批資料是否還在更新。

最後大家的做法變成：需要什麼資料就直接去找原始的來源系統要，資料湖形同虛設。這正是叫獸開講中那個「沒人敢進去的儲藏室」。

追究原因，五項缺失他們全中：沒有中繼資料目錄、沒有綱要管理、沒有交易保證（半成品檔案到處都是）、沒有品質控管、小檔案氾濫。這印證了 30.3.2 節那個叫獸提醒——資料湖不是一個產品，而是一套包含治理的完整實踐。他們後來的重建，正是從第 32 章的資料目錄開始。

案例二 讀到一半的報表

某團隊的日報表在某天出現了異常數字。追查後發現，報表查詢執行的時間點，恰好落在資料寫入作業的中途——當時新資料只寫了一部分，舊資料又已被部分刪除，於是報表讀到了一個「半成品」的狀態。

這是純資料湖架構的典型問題：檔案系統本身沒有交易概念，寫入者與讀取者之間沒有任何協調機制。

導入開放表格式後問題消失。因為寫入是原子性的——在新版本的清單提交之前，讀取者看到的一直是完整的舊版本；提交之後，看到的則是完整的新版本，永遠不會有中間狀態。這正是 30.4.2 節「交易保證」的實務價值，也是 Lakehouse 相較純資料湖最直接的差異。

案例三 被小檔案拖垮的串流管線

某工廠的感測資料以串流方式寫入資料湖，設定為每分鐘產出一個檔案。運行一年後，分析人員抱怨查詢越來越慢——原本三分鐘的月報查詢，變成了將近一小時。

原因正是小檔案累積：一年下來超過五十萬個檔案，每個平均不到 1 MB。依式 30-1 估算，光是開啟這些檔案的固定開銷就相當可觀，而實際的資料讀取反而是次要的。

他們的解法是加入每日的壓實作業，把當日的小檔案合併為數個 128 MB 的大檔。查詢時間隨即回到數分鐘。這個案例說明：串流寫入的便利與查詢效率之間存在張力，而壓實正是化解這個張力的必要配套——它不是可有可無的優化，而是串流入湖架構的必要組成。

案例四 回到三天前的那張表

某團隊在更新資料處理邏輯後，發現產出的結果有誤，但錯誤的資料已經覆蓋了原本正確的版本，且下游系統已經取用。

若是傳統的資料湖，這幾乎無解——檔案已被覆寫，除非有另外的備份，否則無法還原。但他們使用的是具備時間旅行能力的開放表格式，因此能直接查詢「三天前這張表的狀態」，確認正確的內容，並把表回復到該版本。

整個過程只花了不到一小時。這個能力來自 30.4.2 節那個「不可變檔案加上版本化清單」的設計——舊版本的檔案本來就還在，只是不再被當前的清單引用。這也呼應了第 25 章的重播能力：能夠回到過去，是現代資料架構極有價值的一項性質。

實作活動

活動一 實測小檔案的影響

請以實驗驗證式 30-1 的預測。

步驟 1. 準備一份約 1 GB 的資料，分別以「1 個大檔」「100 個中檔」「10,000 個小檔」三種方式寫成 Parquet。

步驟 2. 對三者執行相同的彙總查詢，記錄各自的執行時間。

步驟 3. 計算三者的時間比例，並與式 30-1 的預測比較。

步驟 4. 對小檔案版本執行壓實（合併為少數大檔），再次查詢並記錄改善幅度。

活動二 驗證分區裁剪

請實作分區並觀察其對查詢的影響。

步驟 1. 準備一份含日期欄位、涵蓋至少一年的資料，先以「不分區」的方式寫入。

步驟 2. 執行一支「只查某一週」的查詢，記錄執行時間與掃描的資料量。

步驟 3. 改為依日期分區重新寫入，執行相同的查詢並比較。

步驟 4. 依式 30-3 計算理論上的讀取比例，與實測結果對照，並說明差異可能的來源。

活動三 體驗開放表格式的交易與時間旅行

請以任一開放表格式（如 Delta Lake）建立一張表，體驗其核心能力。首先寫入一批初始資料並記錄版本。接著執行數次更新與刪除操作，每次記錄版本號。然後嘗試查詢「第一個版本」的內容，確認時間旅行功能。最後設計一個實驗驗證交易保證：在一個較長的寫入作業執行到一半時，從另一個工作階段查詢該表，確認讀到的是完整的舊版本而非半成品。完成後撰寫說明：這些能力若沒有中繼資料層，為何在純資料湖上無法達成？

延伸閱讀

- **Lakehouse 架構論文：**Databricks 團隊發表的《Lakehouse: A New Generation of Open Platforms》，闡述其設計動機與技術路徑，是本章 30.4 節的第一手資料。
- **《The Data Warehouse Toolkit》：**Kimball 的經典著作，維度建模的權威參考，是 30.2 節的完整延伸。
- **Delta Lake、Iceberg 與 Hudi 的官方文件：**說明各自的中繼資料設計、交易機制與最佳實務，可對照比較三者的取舍。
- **《Fundamentals of Data Engineering》關於儲存與服務的章節：**Reis 與 Housley 著，從資料工程的角度討論架構選擇，與本章 30.6 節互補。

本章小結

1. OLTP 讀寫少數紀錄、要求毫秒回應，OLAP 掃描大量紀錄只取少數欄位；兩者在效能干擾、儲存佈局與資料範圍上皆有衝突，故必須分開管理——這正是資料倉儲的起源。
2. 資料倉儲以維度建模組織資料（事實表記錄可量測的事件、維度表提供描述性情境），適度反正規化以利分析；但它有只能容納結構化資料、寫時綱要僵固、以及成本高昂三項限制。
3. 資料湖以讀時綱要「先收下再說」，能容納任何型態並保留原始細節，其可行性建立在物件儲存的低成本上；但若缺乏中繼資料、綱要管理、交易保證、品質控管與檔案治理，就會淪為資料沼澤。
4. 資料湖的五項缺失都不是儲存技術的問題，而是管理能力的缺失；故資料湖不是一個產品而是一套架構，治理是它能否成立的前提。
5. Lakehouse 在物件儲存的檔案之上疊加中繼資料層，以「不可變的檔案加上原子性切換的清單」提供交易保證、並行控制、時間旅行與綱要演進——此設計與第 11、25、28 章的思想一脈相承。
6. 實務上以銅（原始）、銀（整備）、金（服務）三層組織資料，職責分離使下游能回到上一層重新處理；金層的內容其實就是維度模型，故 Lakehouse 並未拋棄倉儲的智慧，而是把它變成湖中的一層。

7. 分區依常用的篩選欄位切分（日期最常見）以達成分區裁剪；檔案大小以 128 MB 至 1 GB 為宜，串流寫入須搭配定期壓實，否則小檔案累積將嚴重劣化查詢效能。
8. 開檔總開銷為 $(D/s) \times t$ （式 30-1），與檔案大小無關故小檔越多開銷佔比越高；分區與欄位裁剪的合併讀取比為 $(n/N) \times (k/C)$ （式 30-3），查一天取五欄可能只需讀全表的萬分之一——這個差距來自存資料當下的佈局決定，而非更快的機器。

習題

一、觀念題

1. 請說明為何不能直接在交易系統上執行分析查詢，並列出至少三個理由。
2. 資料倉儲有哪三項限制？請說明其中「只保留確定有用的資料」這個哲學，為何與大數據時代的需求衝突。
3. 資料湖為何會淪為資料沼澤？請指出這些缺失的共同性質，並說明其對「資料湖是產品還是架構」這個問題的意涵。
4. 請說明 Lakehouse 的中繼資料層如何運作，以及它為何能同時提供交易保證與時間旅行。

二、計算題

1. 某資料湖總量 200 GB，開啟一個檔案的固定開銷為 12 毫秒。（一）依式 30-1 計算單檔為 200 MB、2 MB、0.2 MB 三種情況下的檔案數與開檔總開銷；（二）比較 0.2 MB 與 200 MB 兩種情況的開銷相差幾倍；（三）說明為何壓實作業是必要的。
2. 某表依日期分區，涵蓋 2 年（730 天）、共 50 個欄位。某查詢只需最近 14 天、6 個欄位。（一）依式 30-2 計算分區裁剪後的掃描比例；（二）依式 30-3 計算合併欄位裁剪後的實際讀取比；（三）若全表為 8 TB，此查詢實際需讀取多少資料？

三、思考與討論

1. 本章指出 Lakehouse 的核心設計（不可變檔案加上版本化指標）與第 11、25、28 章的思想相同。請說明這個共同思想是什麼，並討論它為何在分散式系統中如此常見。
2. 案例一中，企業的資料湖淪為無人敢用的沼澤。請討論：若要重建這個資料湖，你會依什麼順序處理那五項缺失？為什麼？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

三個理由：其一，效能干擾——分析查詢掃描數億筆紀錄會大量佔用資源，使同時進行的交易處理變慢甚至逾時，讓報表拖垮訂單系統是不可接受的。其二，儲存佈局衝突——交易系統需讀寫整筆紀錄故適合列式儲存，分析只需少數欄位故適合欄式（第 4 章），同一份資料無法同時對兩者最佳化。其三，資料範圍不同——交易系統通常只保留當前狀態與近期資料，舊資料會被封存，但分析往往需要數年的歷史。這三點合起來，即是「把分析資料另外集中管理」的必要性，也是資料倉儲的起源。

第 2 題

三項限制：只能容納結構化資料（影像、聲音、自由文字、感測波形無法塞入事實表與維度表）、寫時綱要的僵固（資料須先符合預定結構才能載入，新型態需先改綱要且流程冗長）、成本高昂（儲存與運算緊密耦合，擴充成本高）。哲學衝突：資料倉儲主張「只保留你確定有用的」，因此在載入前就大量丟棄資料；但大數據時代的洞見往往來自「當初以為沒用的資料」——例如某個未被保留的感測欄位，日後可能正是預測故障的關鍵特徵。一旦丟棄即無法回溯取得，這種不可逆性與探索性分析的本質相衝突，正是資料湖興起的原因。

第 3 題

五項缺失：缺乏中繼資料（不知資料是什麼、從哪來、欄位意義）、缺乏綱要管理（上游變更無人知曉、格式不一致）、缺乏交易保證（並行寫入互相覆蓋、失敗留下半成品）、缺乏品質控管（好壞資料混雜）、小檔案氾濫（查詢效能劣化）。共同性質：這五項都不是「儲存技術」的問題——物件儲存本身運作良好——而全都是「管理能力」的缺失。意涵：資料湖不是一個可以購買的產品，而是一套包含儲存、中繼資料、治理與品質控管的完整架構實踐。多數失敗的資料湖專案，正是把它當成「一個很便宜的大硬碟」，倒進資料就以為完事。故治理（第 32 章）不是配套，而是資料湖能否成立的前提。

第 4 題

運作方式：在物件儲存的檔案之上增加一層中繼資料，以一份「清單」定義「當前這張表包含哪些檔案」以及表的綱要。因物件儲存的檔案不可修改，新增資料時寫入新檔案並更新清單將其納入；刪改時寫入新版本檔案並更新清單排除舊檔案。關鍵在於清單的更新是原子性的。交易保證的來源：在新版本的清單提交之前，讀取者看到的一直是完整的舊版本；提交之後看到的是完整的新版本，永遠不會有中間狀態（見案例二）。時間旅行的來源：舊版本的清單與其引用的檔案都還在（只是不再被當前清單引用），故可指定過去的版本進行查詢或還原（見案例四）。

二、計算題

第 1 題

$D = 200 \text{ GB} = 204,800 \text{ MB}$, $t = 12 \text{ 毫秒} = 0.012 \text{ 秒}$ 。

- (1) 單檔 200 MB：檔案數 $= 204,800 / 200 = 1,024$ 個；開銷 $= 1,024 \times 0.012 \approx 12.3$ 秒。
- (2) 單檔 2 MB：檔案數 $= 102,400$ 個；開銷 $= 102,400 \times 0.012 = 1,228.8$ 秒（約 20.5 分鐘）。
- (3) 單檔 0.2 MB：檔案數 $= 1,024,000$ 個；開銷 $= 1,024,000 \times 0.012 = 12,288$ 秒（約 3.4 小時）。
- (4) 相差倍數： $12,288 \div 12.3 = 1,000$ 倍（即檔案大小相差 1,000 倍，開銷亦相差 1,000 倍）。

壓實必要性：同樣的 200 GB 資料，僅因檔案切得太細，光是「開啟檔案」就從 12 秒暴增至 3.4 小時——而這還沒讀取任何實際內容。串流寫入天然會產生大量小檔（每分鐘一個檔，一年逾五十萬個），故定期壓實不是可有可無的優化，而是串流入湖架構的必要組成（見案例三）。

第 2 題

$N = 730 \text{ 天}$ 、 $n = 14 \text{ 天}$ 、 $C = 50 \text{ 欄}$ 、 $k = 6 \text{ 欄}$ 。

- (1) 分區裁剪（式 30-2）： $14 / 730 \approx 0.01918$ ，即約 1.92%。

(2) 合併欄位裁剪 (式 30-3) : $(14/730) \times (6/50) = 0.01918 \times 0.12 \approx 0.0023$, 即約 0.23%。

(3) 實際讀取量 = $8 \text{ TB} \times 0.0023 \approx 0.0184 \text{ TB} \approx 18.4 \text{ GB}$ 。

延伸思考：8 TB 的表，這支查詢只需讀約 18.4 GB——約為全表的四百三十分之一。若沒有分區、且資料存成 CSV（無法欄位裁剪），則須掃描完整的 8 TB，在按掃描量計費的服務上費用相差四百餘倍。這個差距完全來自「當初存資料時」的佈局決定，而非更快的機器或更好的演算法。

三、思考與討論

第 1 題

共同思想：「不可變的資料 + 可原子切換的指標」——資料本身寫入後永不修改，變更則以「產生新版本並切換指標」來達成。對應各章：第 11 章 RDD 不可變，轉換產生新的 RDD，並以 Lineage 記錄推導關係；第 25 章 Kafka 以追加式日誌儲存訊息，消費者以位移指標決定讀到哪裡；第 28 章容器映像檔的層是唯讀的，修改則在可寫層以寫時複製處理；本章的中繼資料層則以清單指標定義「當前的表」。為何常見於分散式系統：其一，容錯——因舊版本仍在，故障時可回退或重算，無需複雜的復原機制。其二，並行安全——讀取者不會看到寫入中的半成品，讀寫可並行而無須鎖定。其三，簡化一致性——只需保證「指標切換」這一個小動作的原子性，而非整批資料的原子性，大幅降低分散式協調的難度。其四，可回溯——歷史版本自然保留，使時間旅行與稽核成為附帶效果。這正是「讓不變成為預設、把變化侷限在最小範圍」的設計哲學。

第 2 題

建議的順序與理由：第一步應是「中繼資料與資料目錄」——因為在不知道湖裡有什麼之前，其他工作都無從下手；先盤點現有資料、標註來源、擁有者與欄位意義，讓資料至少「可被發現」。第二步是「綱要管理與交易保證」——導入開放表格式，使新進資料具備結構定義與寫入的原子性，避免問題繼續惡化（止血優先於治療）。第三步是「品質控管」——在擷取端加入驗證關卡（第 13 章），阻擋壞資料進入。第四步是「檔案治理」——建立壓實與分區的規範，改善既有的效能問題。最後是「歷史資料的整理」——依重要性分批處理，不重要或無人使用的可考慮封存或刪除。核心原則有二：其一，先「止血」（阻止情況惡化）再「治療」（處理既有問題）；其二，先讓資料「可被發現」，因為找不到的資料，改善它也沒有意義。亦可補充：技術措施必須搭配組織的責任歸屬（每份資料須有明確的擁有者），否則治理無法持續。

第 31 章

無伺服器運算與雲原生架構

機器人叫獸（李世淵） 編著

叫獸開講

「無伺服器」（Serverless）這個名字，是我認為整個資訊領域取得最糟的名字之一。

因為它當然有伺服器。你的程式碼還是跑在某台實體機器上，只是那台機器不歸你管——你不必挑規格、不必安裝作業系統、不必修補漏洞、不必監控它是否還活著。「無伺服器」真正的意思是「無伺服器管理」，可惜這個較精確的名字沒有流行起來。

我用一個比喻來說明它與前面幾章的差別。第 27 章的虛擬機像是租一整層辦公室——你要付整層的租金，不管有沒有人裡面工作。第 29 章的容器像是租共享辦公室的固定座位——比整層便宜，但座位空著時你還是在付錢。而無伺服器則像是租會議室，以分鐘計費——沒開會的時候，一毛錢都不用付。

這個「不用就不付費」的特性極為誘人，尤其對那些「大部分時間沒事、偶爾要處理一下」的工作。但它也有代價，而且是相當實質的代價——最著名的就是「冷啟動」：因為平常沒有機器在等你，第一個請求來的時候，系統得臨時把環境準備好，這可能要花上幾秒鐘。

這一章是第柒篇的最後一章。我們會把無伺服器的價值與代價都算清楚，也會把整個雲原生的架構圖收攏——你會看到前面四章談的虛擬化、容器、編排、資料架構，如何與這一章的內容組合成一套完整的現代系統。

學習目標

研讀完本章之後，你應該能夠：

1. 說明無伺服器運算的定義與其與虛擬機、容器的差異。
2. 描述函式即服務的執行模型，並說明冷啟動的成因與影響。
3. 說明事件驅動的無伺服器管線如何組合各項雲端服務。
4. 說明微服務架構的價值與其代價。
5. 指出雲原生的核心原則，並說明各項技術如何協同運作。
6. 運用成本模型判斷無伺服器與常駐服務的適用界線。

核心概念

核心概念	說明
無伺服器	使用者無須管理伺服器、依實際使用量計費的運算模式。
函式即服務	以單一函式為部署單位、由事件觸發執行的服務形態。
冷啟動	首次請求時需臨時準備執行環境所造成的額外延遲。
事件驅動管線	以事件串接多個託管服務所構成的自動化流程。
微服務	把系統拆分為多個獨立部署的小型服務的架構風格。
雲原生	為雲端環境而設計、充分利用其彈性與託管服務的架構取向。
託管服務	由雲端供應商營運、使用者只需使用而無須維護的服務。

表 31-1 本章核心概念一覽

31.1 無伺服器運算的概念

31.1.1 三個定義性特徵

承接叫獸開講，「無伺服器」並非沒有伺服器，而是指具備以下三個特徵的運算模式。

- **無須管理伺服器：**使用者不接觸底層機器——不選規格、不裝系統、不做修補、不管監控。供應商負責一切基礎設施。
- **依實際使用量計費：**以請求次數與執行時間計費，沒有請求時費用為零。這與虛擬機、容器「不管用不用都要付」有本質差異。
- **自動彈性至零：**流量來了自動擴充，沒有流量時自動縮到零個實例。這個「縮到零」的能力，正是計費為零的前提。

其中第三點最為關鍵。第 29 章的 Kubernetes 雖然能自動擴縮，但通常會維持至少一個副本在運行（否則沒有東西能接收第一個請求）；而無伺服器能真正縮到零——代價則是，那個「重新從零長出來」的過程，就是冷啟動。

31.1.2 三種運算形態的定位

面向	虛擬機	容器	無伺服器
部署單位	一台機器	一個容器	一個函式
啟動時間	數十秒至數分鐘	不到一秒	冷啟動數百毫秒至數秒
閒置時計費	全額計費	全額計費	不計費
管理負擔	最重	中等	最輕
執行時間限制	無限制	無限制	通常有上限
適合的工作	長期運行、需完整控制	長期服務、需彈性擴縮	間歇性、事件觸發

表 31-2 三種運算形態的比較

表中「執行時間限制」那一列常被忽略，卻是實務上的關鍵限制。多數無伺服器平台對單次執行設有時間上限（常見為數分鐘到十幾分鐘），超過就會被強制中止。這意味著它不適合長時間的批次運算——你不能用它來跑一個要三小時的 Spark 作業。

31.2 函式即服務與冷啟動

31.2.1 執行模型

函式即服務（FaaS）是無伺服器最典型的形態。你上傳的不是一個完整的應用，而是一個「函式」——它接收事件作為輸入、執行邏輯、回傳結果，然後結束。

整個生命週期由平台管理：事件到達時，平台找一個可用的執行環境（或建立一個新的）、載入你的程式碼、執行、回傳結果。若短時間內沒有新的事件，該環境會被回收。

這個模型有一個重要的推論：函式必須是「無狀態」的。因為你無法預期下一個請求會被送到哪個環境，也無法保證環境會留存——所有需要跨請求保存的狀態，都必須外部化到資料庫或物件儲存。

這個要求你應該覺得很熟悉——第 28 章的容器無狀態原則、第 29 章水平擴縮的前提、第 9 章 MapReduce 的無狀態任務，全都是同一個要求。而在無伺服器中，它被推到了極致。

31.2.2 冷啟動的成因

冷啟動是無伺服器最常被詬病的問題。它的成因可拆解為幾個階段：

- **配置執行環境：**平台需要在某台機器上準備一個隔離的執行環境（通常是輕量的容器或微型虛擬機）。
- **載入執行時期：**啟動語言的執行環境，如虛擬機器或直譯器。
- **載入程式碼與相依套件：**把你的函式與其相依項目載入記憶體。套件越多、越大，這一步越慢。
- **執行初始化程式碼：**建立資料庫連線、載入設定或模型等一次性的準備工作。

這幾個階段加起來，可能從數百毫秒到數秒不等。而當環境已存在（「熱啟動」）時，這些全部可以跳過，回應時間可能只有數十毫秒——兩者可能相差數十倍。

叫獸提醒

冷啟動有一個特別惱人的性質：它是「間歇性」的。你在開發測試時可能感覺不到（因為你不斷在呼叫，環境一直是熱的），上線後在流量穩定時也還好，卻會在「流量稀疏」或「突然暴增」時大量出現——而這兩個時刻恰恰是使用者感受最敏感的時候。更麻煩的是，它會讓延遲的分布變成雙峰：多數請求數十毫秒，少數請求數秒。這時你若只看平均延遲，會覺得一切正常；必須看第 3 章談過的 p99 才會發現問題。這也再次印證第 3 章那句話——平均值會掩蓋尾端的真相。

31.2.3 緩解冷啟動

- **精簡相依套件：**只引入真正需要的函式庫。這與第 28 章「縮小映像檔」是同一個道理——載入的東西越少越快。
- **預留熱實例：**多數平台提供「預先保留若干個熱環境」的選項，代價是這些實例即使閒置也要計費（等於放棄了部分無伺服器的成本優勢）。
- **定期喚醒：**以排程定期呼叫函式使其保持熱狀態。這是常見但略顯粗糙的做法。
- **選擇啟動較快的執行時期：**不同語言的執行環境啟動速度差異顯著，對延遲敏感的场景可納入考量。
- **接受它：**對於非同步的背景工作（如處理上傳的檔案），多幾秒的冷啟動根本無關緊要——不是所有場景都需要優化。

最後一項值得強調：判斷「這個延遲重不重要」，比盲目優化更有價值。無伺服器最適合的場景，往往正是那些對延遲不敏感的非同步工作。

31.3 事件驅動的無伺服器管線

31.3.1 以事件串接託管服務

無伺服器真正的威力，不在於單一函式，而在於「把多個託管服務以事件串接起來」——這正是第 25 章事件驅動架構在雲端的具體實現。

典型的模式是：某個事件發生（檔案被上傳到物件儲存、訊息進入佇列、資料庫紀錄被更新、排程時間到達），自動觸發一個函式；該函式處理後產生新的事件，再觸發下一個環節。整條流程沒有任何一台需要你管理的伺服器。

以一個實際的資料處理場景為例：感測資料檔案上傳到物件儲存 → 觸發函式進行格式驗證與清理 → 清理後的資料寫入 Lakehouse 的銅層（第 30 章）→ 觸發另一個函式更新統計指標 → 若偵測到異常則發送告警。整條管線可能只需寫幾個小函式，其餘全由託管服務承擔。

31.3.2 與第 13 章管線的對照

你可能會問：這與第 13 章的批次資料管線有什麼不同？

面向	編排式管線（第 13 章）	事件驅動管線
觸發方式	由排程器依時間觸發	由事件發生時觸發
流程定義	以 DAG 明確定義相依順序	隱含於各服務的事件訂閱關係
延遲	受排程週期限制	事件到達即處理
可觀測性	編排器提供集中的執行檢視	流程分散，較難掌握全貌
適合	有明確順序的批次處理	反應式、間歇性的處理

表 31-3 編排式與事件驅動管線的比較

兩者不是取代關係，而是各有適用。需要嚴格順序與集中管控的每日批次，用編排器仍然較好；而「檔案一上傳就處理」這類反應式的工作，事件驅動則自然得多。

要特別注意表中「可觀測性」那一列的代價：事件驅動管線的流程是「隱含」的——它散布在各個服務的觸發設定中，沒有一個地方能看到全貌。當出問題時，追查「這個事件到底流到哪裡去了」可能相當困難。這是採用此架構前必須認清的代價。

31.4 微服務架構

31.4.1 從單體到微服務

傳統的「單體」應用把所有功能打包成一個部署單位。它的優點是簡單——開發、測試、部署都只有一個東西要管。但當系統與團隊都變大時，問題會浮現：任何微小的修改都要重新部署整個系統、一個模組的錯誤可能拖垮全部、不同模組無法依各自的需求獨立擴充、且所有人都在同一份程式碼上工作而互相干擾。

微服務架構的做法是把系統拆分為多個獨立的小服務，各自負責一項業務能力、各自獨立開發部署、彼此透過網路介面溝通。

它帶來的價值包括：各服務可獨立部署（改一個不影響其他）、可依各自的負載獨立擴充、團隊能自主決策與發布、以及故障可被隔離在單一服務內。

31.4.2 代價：分散式系統的所有麻煩

但我要非常明確地指出：微服務不是免費的。當你將單體拆成微服務，你等於是「把函式呼叫變成了網路呼叫」——而這意味著，第 3 章談過的所有分散式系統難題全部登場。

- **網路不可靠**：原本必定成功的函式呼叫，現在可能逾時、失敗、或重複送達（第 3 章的八個誤解）。
- **延遲累積**：一個請求可能經過五、六個服務，每一跳都增加延遲，且尾端延遲會被放大（第 3 章的 p99）。

- **一致性困難**：跨服務的資料一致性無法用單一交易保證，須改用第 8 章談的補償或最終一致設計。
- **可觀測性複雜**：一個請求的完整路徑跨越多個服務，除錯需要分散式追蹤才能拼湊全貌。
- **維運成本上升**：從管理一個部署單位，變成管理數十個——這正是第 29 章 Kubernetes 存在的理由。

因此我的建議與業界的成熟共識一致：不要一開始就採用微服務。從單體開始，等到「規模真的造成了問題」時，再依業務邊界逐步拆分。過早拆分的代價，往往遠大於它帶來的好處——這與第 22 章「能用單一代理解決的就不要用多代理」是同一個判斷原則。

31.5 託管資料服務的整合

31.5.1 整條資料管線的無伺服器化

本書前面各章談的技術，如今幾乎都有對應的託管版本——你不必自建，只需使用。

環節	自建方案（前面章節）	託管方案
訊息佇列	自架 Kafka 叢集（第 25 章）	託管的串流服務，按吞吐量計費
批次運算	自建 Spark 叢集（第 11 章）	託管 Spark，作業結束即釋放
串流處理	自建串流引擎（第 26 章）	託管的串流分析服務
資料儲存	自建 HDFS（第 5 章）	物件儲存加開放表格式（第 30 章）
查詢引擎	自建查詢叢集	無伺服器查詢，按掃描量計費

表 31-4 自建與託管方案的對照

這個轉變的意義，回到第 27 章談的服務模型階梯：從 IaaS 往 PaaS 移動。你放棄了部分控制權換取了大幅減輕的維運負擔。

對多數團隊而言，這是理性的選擇——因為維護一個 Kafka 或 Spark 叢集需要專門的知識與人力，而團隊的核心價值通常不在於此。但也要記得第 27 章的叫獸提醒：抽象層次越高，出問題時你能自己排查的空間就越小。

31.5.2 按掃描量計費的雙面刃

無伺服器查詢服務是個特別值得討論的例子。它讓你不必維護任何叢集，直接對物件儲存中的資料下 SQL，並依「掃描的資料量」計費。

這個計費模式的好處是：不查詢時完全不花錢，非常適合間歇性的分析需求。但它也把「查詢效率」直接轉化成了「錢」——一支未經最佳化的查詢，可能掃過遠超必要的資料而產生高額費用（第 27 章案例三的天價帳單）。

因此在這類服務上，第 12 章的述詞下推與欄位裁剪、第 30 章的分區與檔案佈局，不再只是效能議題，而是直接的成本控制手段。依第 30 章式 30-3 的計算，良好的佈局可能讓同一支查詢的費用相差數百倍——這是本書各章知識最直接轉化為金錢的一個例子。

31.6 雲原生的整體圖像

31.6.1 五項核心原則

走到第柒篇的最後，我們可以把「雲原生」這個常被濫用的詞，收攏成幾項具體的原則。

- **容器化封裝**：以容器提供一致的執行環境與可攜性（第 28 章）。
- **動態編排**：由編排系統自動處理排程、擴縮與修復（第 29 章）。
- **無狀態設計**：運算單元不持有不可重建的狀態，狀態外部化到託管服務。
- **宣告式管理**：描述期望狀態而非執行步驟，讓系統自行調和（第 29 章）。
- **善用託管服務**：把非核心能力交給供應商，團隊專注於自身的價值所在。

你會注意到，這五項之中有三項在前面章節已詳細討論過。這正說明了「雲原生」不是一項新技術，而是這些技術組合起來所形成的一套取向。

31.6.2 選擇運算形態的判斷

面對一項工作，該選虛擬機、容器還是無伺服器？我建議依序問三個問題。

第一，執行時間有多長？若超過無伺服器的上限（通常是十幾分鐘），就只能選容器或虛擬機。第二，流量的形態如何？若是間歇性、稀疏、或難以預測的，無伺服器的「縮到零」極有價值；若是持續穩定的高流量，常駐服務通常更划算（本章公式會算出交叉點）。第三，對延遲有多敏感？若無法容忍偶發的數秒延遲，就必須考慮冷啟動的影響。

實務上最常見的是「混合使用」：核心的線上服務以容器常駐運行、間歇性的背景工作用無伺服器、而需要特殊硬體或完整控制的工作（如 GPU 訓練）則用虛擬機。三者不是互斥的選擇，而是工具箱裡的不同工具。

31.6.3 承先啟後

本章我們釐清了無伺服器的真義（不是沒有伺服器，而是不必管理伺服器）與其三個定義性特徵（31.1）、函式即服務的執行模型與冷啟動的成因與緩解（31.2）、事件驅動管線與編排式管線的取捨（31.3）、微服務的價值與其所引入的全部分散式難題（31.4）、託管資料服務的整合與按掃描量計費的雙面刃（31.5），最後把雲原生收攏為五項核心原則（31.6）。

至此，第柒篇「雲端運算平台與架構」五章完整收束。從第 27 章的雲端本質與成本、第 28 章的虛擬化與容器、第 29 章的 Kubernetes 編排、第 30 章的資料架構，到本章的無伺服器與雲原生整體圖像——現代系統的運算與儲存基礎，你已經有了完整的視野。

但我在第 30 章末說過一句話，這裡要再說一次：基礎設施完備，不等於系統可信。資料湖之所以變成沼澤，關鍵不在技術而在治理。誰知道這批資料從哪來？誰保證它的品質？誰能存取它？出了問題如何追溯？當系統越來越自動化、資料流動越來越快，這些問題只會更加尖銳。這就是第捌篇「資料治理、安全與維運」要處理的主題，我們從資料治理與資料品質開始。

公式與流程：成本交叉點與冷啟動的影響

這一節用兩組計算回答兩個實務問題：什麼時候該從無伺服器換成常駐服務？以及冷啟動到底影響多大？

成本交叉點

設無伺服器每次呼叫的費用為 f （含運算時間），每日呼叫次數為 n ，則每月費用為：

$$\text{無伺服器月費} = n \times 30 \times f$$

式 31-1 無伺服器的月費用

而常駐服務的月費為固定值 S （不論呼叫多少次）。令兩者相等，可得交叉點的每日呼叫次數：

$$\text{交叉點 } n^* = S / (30 \times f)$$

式 31-2 成本交叉點的每日呼叫次數

以每次呼叫 0.0002 元、常駐服務月費 3,000 元為例，交叉點為 $3,000 \div (30 \times 0.0002) = 500,000$ 次／日。

每日呼叫次數	無伺服器月費	常駐月費	何者較省
1,000	6 元	3,000 元	無伺服器（省 99.8%）
10,000	60 元	3,000 元	無伺服器（省 98%）
100,000	600 元	3,000 元	無伺服器（省 80%）
500,000	3,000 元	3,000 元	交叉點
1,000,000	6,000 元	3,000 元	常駐（省 50%）

表 31-5 無伺服器與常駐服務的成本比較

這張表呈現了無伺服器的適用範圍：在低到中等的呼叫量下，它的成本優勢是壓倒性的（呼叫量越低越明顯）；但當流量持續且龐大時，常駐服務反而更划算。

你會發現這與第 27 章表 27-6 的邏輯完全相同——那裡比較的是「使用率」，這裡比較的是「呼叫量」，但本質都是「固定成本 vs 變動成本」的取捨。

冷啟動對平均延遲的影響

設熱啟動的回應時間為 T_w 、冷啟動為 T_c ，而冷啟動發生的比例為 p ，則平均回應時間為：

$$\text{平均延遲} = T_w \times (1 - p) + T_c \times p$$

式 31-3 含冷啟動的平均延遲

以熱啟動 50 毫秒、冷啟動 2,000 毫秒為例，計算不同冷啟動比例下的表現。

冷啟動比例 p	平均延遲	相對熱啟動	判讀
1%	69.5 毫秒	1.4 倍	幾乎無感
5%	147.5 毫秒	3.0 倍	開始明顯
20%	440 毫秒	8.8 倍	體驗劣化
50%	1,025 毫秒	20.5 倍	難以接受

表 31-6 冷啟動比例對平均延遲的影響

叫獸提醒

表 31-6 有一個陷阱我要特別提醒：即使在 1% 的冷啟動比例下，平均延遲看起來只有 69.5 毫秒、相當健康——但那 1% 的使用者，實際感受到的是 2,000 毫秒，整整慢了四十倍。若這 1% 恰好落在你最重要的客戶身上（回想第 3 章：重度使用者往往落在尾端），問題就嚴重了。所以評估無伺服器時，絕不能只看平均延遲，必須看 p99 甚至 p999。這也解釋了為何有些團隊寧可付錢預留熱實例——他們放棄的是部分成本優勢，換取的是延遲分布的可預測性。而「可預測」這件事的價值，往往在服務品質協議中才會被真正認識到。

案例研討

案例一 一年只跑三次的報表

某單位有幾支季度與年度報表程式，原本部署在一台常駐的虛擬機上。這台機器一年當中真正在工作的時間不到十小時，其餘時間都在閒置——但費用照付。

他們把這些程式改為無伺服器函式，由排程事件觸發。依式 31-1，因為呼叫次數極低，每月費用趨近於零，而原本的虛擬機費用則完全省下。

這是無伺服器最典型的適用場景：極度間歇性的工作。這裡也呼應了第 27 章的使用率議題——那台虛擬機的使用率不到 0.2%，遠低於任何合理的損益平衡點。無伺服器的「縮到零」，正是為這類工作而生的。

案例二 被冷啟動嚇跑的使用者

某團隊把面向使用者的查詢 API 改為無伺服器實作，測試階段表現良好（開發時不斷呼叫，環境一直是熱的）。但上線後客服開始接到抱怨：「有時候按下去要等好幾秒才有反應」。

監控顯示平均回應時間僅 120 毫秒，看起來完全正常。但檢視 p99 才發現真相——第 99 百分位的延遲高達 2.4 秒。原因是這個 API 的使用相當稀疏，環境經常被回收，導致頻繁的冷啟動。

這正是 31.2.2 節叫獸提醒所警告的雙峰分布，也再次印證第 3 章「平均值會掩蓋尾端真相」。他們的解法是預留少量熱實例——雖然放棄了部分成本優勢，但把延遲的可預測性換了回來。這個案例說明：面向使用者的同步請求，是無伺服器最需要謹慎評估的場景。

案例三 流量成長後的成本反轉

某新創的核心 API 一開始採用無伺服器，因為流量很小、成本幾乎可以忽略。兩年後業務成長，該 API 的呼叫量達到每日兩百多萬次，而財務發現雲端帳單中這一項的費用異常突出。

依式 31-2 計算，他們的交叉點約在每日五十萬次——也就是說，早在流量成長到四分之一的時候，就已經該考慮轉換了，但沒有人回頭檢視這個決策。

他們最終把這支高頻 API 改為容器常駐服務，費用降低約六成，順帶也消除了冷啟動的問題。這個案例的教訓不在於「無伺服器不好」——它在初期確實是正確的選擇——而在於「架構決策應隨規模定期重新檢視」。第一天的最佳解，未必是第一千天的最佳解。

案例四 拆得太早的微服務

某團隊只有五個人，卻在專案初期就把系統拆成了十二個微服務，理由是「這樣比較先進、也方便日後擴充」。

結果如 31.4.2 節所預言：他們開始面對所有分散式系統的難題——服務間呼叫失敗要處理重試、跨服務的資料一致性難以保證、一個請求要追查五個服務的日誌、本地開發環境要同時啟動十二個服務。原本一週能完成的功能，現在要兩三週。

更諷刺的是，這十二個服務全都部署在同樣的機器上、由同一組人維護、且從未有任何一個需要獨立擴充——微服務的所有好處他們一項也沒得到，代價卻全數承擔了。

他們後來合併回三個服務，開發速度隨即恢復。這個案例是「複雜度應該被需求逼出來」最清楚的反面教材，也與第 22 章「能用單一代理就不要用多代理」是同一個判斷原則。

實作活動

活動一 部署你的第一個函式

請以任一雲端平台的免費層，實際部署並測量一個無伺服器函式。

- 步驟 1. 撰寫一個簡單的函式（例如接收一段文字並回傳字數統計），並部署為 HTTP 觸發。
- 步驟 2. 連續快速呼叫二十次，記錄各次的回應時間。
- 步驟 3. 等待十五分鐘後再呼叫一次，記錄這次的回應時間。
- 步驟 4. 比較兩者的差異，並依式 31-3 估算：若冷啟動比例為 10%，平均延遲會是多少？

活動二 計算你的成本交叉點

請為一個具體情境做成本分析與決策。

- 步驟 1. 查詢某雲端平台的無伺服器計費方式（含請求數與執行時間兩部分），並估算單次呼叫的成本。
- 步驟 2. 查詢一個規格相當的常駐容器或虛擬機的月費。
- 步驟 3. 依式 31-2 計算成本交叉點的每日呼叫次數。
- 步驟 4. 設想三種流量情境（每日一千次、十萬次、五百萬次），分別判斷該用哪一種，並說明若流量持續成長，你會在什麼時機重新評估。

活動三 建構一條事件驅動管線

請組合多個託管服務，建構一條完全無伺服器的資料管線。設計一個情境：當檔案被上傳到物件儲存時，自動觸發函式進行處理（例如驗證格式、萃取摘要），把結果寫入資料庫，並在偵測到異常時發送通知。請畫出架構圖，標明各個觸發點與資料流向。若環境允許則實際實作；若否，則詳細寫出各服務的設定要點。最後回答：這條管線若改用第 13 章的編排器實作，會有什麼不同？各自的優缺點為何？

延伸閱讀

- 《Serverless Architectures on AWS》或各平台的無伺服器指南：說明函式設計、事件整合與最佳實務，是本章的實作參考。
- 雲原生運算基金會的技術地景：整理雲原生生態中各類工具的定位與關係，可作為 31.6 節的全景補充。
- 《Building Microservices》：Sam Newman 著，深入討論微服務的拆分原則、通訊模式與其代價，是 31.4 節的完整延伸。
- 關於冷啟動最佳化的技術文章：各平台針對啟動延遲的優化建議與實測比較，是 31.2.3 節的深化。

本章小結

1. 「無伺服器」是指無須管理伺服器，而非沒有伺服器；其三個定義性特徵為無須管理、依實際使用量計費、以及能自動縮到零——而「縮到零」正是零計費的前提，也是冷啟動的根源。
2. 虛擬機、容器與無伺服器在部署單位、啟動時間、閒置計費與執行時間限制上各有取舍；無伺服器通常有單次執行的時間上限，故不適合長時間的批次運算。
3. 函式即服務要求函式必須無狀態，因為執行環境不保證留存——這與第 9、28、29 章的無狀態原則一脈相承，並在此被推到極致。

4. 冷啟動源於配置環境、載入執行時期、載入程式碼與初始化四個階段；它是間歇性的，會使延遲分布呈雙峰，故評估時必須看 p99 而非平均。緩解方式包括精簡相依、預留熱實例、定期喚醒，但有時「接受它」才是正確答案。
5. 事件驅動管線以事件串接多個託管服務，適合反應式與間歇性的處理；相較第 13 章的編排式管線，它延遲更低但流程隱含、可觀測性較差。
6. 微服務帶來獨立部署、獨立擴充與故障隔離的價值，但也把函式呼叫變成網路呼叫，引入第 3 章談過的所有分散式難題；故應從單體開始，等規模真的造成問題時再逐步拆分。
7. 本書前面各章的技術如今多有託管版本，使用它們是從 IaaS 往 PaaS 移動——減輕維運負擔但降低可控性。按掃描量計費的查詢服務，使第 12、30 章的最佳化直接轉化為成本控制。
8. 成本交叉點為 $S/(30 \times f)$ （式 31-2），呼叫量低時無伺服器壓倒性划算、高時常駐服務更省；含冷啟動的平均延遲為 $Tw(1-p)+Tc \cdot p$ （式 31-3），但平均值會掩蓋那少數使用者所感受到的數十倍延遲。

習題

一、觀念題

1. 為什麼說「無伺服器」是個取得不好的名字？請說明它真正的三個定義性特徵。
2. 冷啟動的成因為何？為什麼說它會使延遲分布呈「雙峰」，且只看平均延遲會誤導？
3. 請比較事件驅動管線與第 13 章的編排式管線，並各舉一個適合的情境。
4. 微服務會引入哪些分散式系統的難題？為什麼建議「從單體開始」？

二、計算題

1. 某無伺服器函式每次呼叫成本為 0.0005 元，而規格相當的常駐服務月費為 4,500 元。
（一）依式 31-2 計算成本交叉點的每日呼叫次數；（二）計算每日 5 萬次與每日 50 萬次時的無伺服器月費；（三）在這兩種流量下各該選哪一種？
2. 某函式的熱啟動回應時間為 80 毫秒、冷啟動為 1,500 毫秒。（一）依式 31-3 計算冷啟動比例為 2%、10%、30% 時的平均延遲；（二）在 2% 的情況下，平均延遲看似健康，但那 2% 的使用者實際等待多久？相當於熱啟動的幾倍？（三）這說明了什麼？

三、思考與討論

1. 案例三中，團隊因未定期檢視而讓架構決策過時。請討論：一個組織應該在什麼時機、以什麼指標來重新評估既有的架構決策？
2. 本章與第 22 章都提出了「不要過早增加複雜度」的建議。請討論：在技術選型時，如何區分「必要的複雜度」與「為了新穎而增加的複雜度」？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

因為它當然有伺服器——程式碼仍在實體機器上執行，只是那台機器不歸使用者管理。較精確的說法應是「無伺服器管理」。三個定義性特徵：其一，無須管理伺服器——不選規格、不裝系統、不做修補、不管監控。其二，依實際使用量計費——以請求次數與執行時間計費，沒有請求時費用為零，

這與虛擬機、容器「不管用不用都要付」有本質差異。其三，能自動縮到零——流量來時自動擴充，無流量時縮至零個實例；此為零計費的前提，但也正是冷啟動的根源。

第 2 題

成因分四階段：配置隔離的執行環境、載入語言的執行時期、載入程式碼與相依套件、執行初始化程式碼（如建立連線、載入模型）。雙峰分布的原因：當環境已存在（熱啟動）時上述步驟全可跳過，回應可能僅數十毫秒；而環境被回收後的首次請求（冷啟動）則須全部重來，可能耗時數秒。故延遲不是集中在某個值附近，而是分裂為「很快」與「很慢」兩群。只看平均會誤導，是因為少數的冷啟動被大量熱啟動稀釋——例如平均 120 毫秒看似健康，但 p99 可能高達 2.4 秒（見案例二）。故必須檢視 p99 甚至 p999（呼應第 3 章）。

第 3 題

差異：觸發方式（排程器依時間 vs 事件發生時）、流程定義（DAG 明確定義 vs 隱含於各服務的訂閱關係）、延遲（受排程週期限制 vs 事件到達即處理）、可觀測性（編排器提供集中檢視 vs 流程分散難掌握全貌）。適合情境：編排式適合「每晚的資料倉儲更新」這類有明確順序、需集中管控與回填能力的批次處理；事件驅動適合「檔案一上傳就處理」「訊息到達即轉換」這類反應式、間歇性的工作。兩者非取代關係而是各有適用，實務上常混用。須特別注意事件驅動在可觀測性上的代價——出問題時追查事件流向可能相當困難。

第 4 題

引入的難題：其一，網路不可靠——原本必定成功的函式呼叫變成可能逾時、失敗或重複的網路呼叫（第 3 章八個誤解）。其二，延遲累積——請求經過多個服務，每跳增加延遲且尾端延遲被放大。其三，一致性困難——跨服務無法用單一交易保證，須改用補償或最終一致設計（第 8 章）。其四，可觀測性複雜——需分散式追蹤才能拼湊完整路徑。其五，維運成本上升——從管理一個部署單位變成數十個。建議從單體開始的原因：微服務的好處（獨立部署、獨立擴充、故障隔離）只有在「規模真的造成問題」時才顯現，而代價則是立即且全額承擔的；過早拆分往往只承擔代價而未獲好處（見案例四）。應等業務邊界清晰、團隊規模擴大、確有獨立擴充需求時再逐步拆分。

二、計算題

第 1 題

$f = 0.0005$ 元、 $S = 4,500$ 元。

- (1) 交叉點（式 31-2） $= 4,500 / (30 \times 0.0005) = 4,500 / 0.015 = 300,000$ 次／日。
- (2) 每日 5 萬次：月費 $= 50,000 \times 30 \times 0.0005 = 750$ 元；每日 50 萬次：月費 $= 500,000 \times 30 \times 0.0005 = 7,500$ 元。
- (3) 每日 5 萬次時應選無伺服器（750 元 vs 4,500 元，省約 83%）；每日 50 萬次時應選常駐服務（4,500 元 vs 7,500 元，省 40%）。

延伸思考：交叉點為每日 30 萬次，故在流量接近此數字時就應開始評估轉換，而非等到費用明顯異常才回頭檢視（見案例三）。

第 2 題

$T_w = 80$ 毫秒、 $T_c = 1,500$ 毫秒，依式 31-3 計算。

- (1) $p = 2\%$: $80 \times 0.98 + 1,500 \times 0.02 = 78.4 + 30 = 108.4$ 毫秒。
- (2) $p = 10\%$: $80 \times 0.90 + 1,500 \times 0.10 = 72 + 150 = 222$ 毫秒； $p = 30\%$: $80 \times 0.70 + 1,500 \times 0.30 = 56 + 450 = 506$ 毫秒。

- (3) 在 $p = 2\%$ 時，平均 108.4 毫秒看似健康；但那 2% 的使用者實際等待 1,500 毫秒，相當於熱啟動（80 毫秒）的 18.75 倍。

說明：平均值完全掩蓋了少數使用者所承受的極差體驗。若這 2% 恰好落在重度或重要客戶身上（第 3 章指出重度使用者往往落在尾端），問題將更嚴重。故評估無伺服器時必須檢視 p99 而非平均——這也是有些團隊寧可付費預留熱實例、以部分成本優勢換取延遲可預測性的原因。

三、思考與討論

第 1 題

重新評估的時機可包括：其一，規模里程碑——當流量、資料量或使用者數成長達某個倍數（如兩倍、五倍）時自動觸發檢視。其二，成本異常——當某項雲端支出佔比超過門檻或成長率異常時（呼應第 27 章的 FinOps 治理）。其三，定期節奏——每季或每半年做一次架構檢視，即使沒有明顯問題。其四，重大變更前——在新增重要功能或進入新市場前，先確認現有架構能否支撐。可用的指標包括：單位成本（每次請求、每 GB 資料的成本，成長時即為警訊）、與交叉點的距離（如本章式 31-2 算出的門檻，定期比對當前流量）、延遲的 p99 趨勢、維運工時佔比、以及部署頻率與失敗率。核心觀念：架構決策不是一次性的，而應被視為「有保存期限的假設」——當初的前提改變時，結論也應重新檢驗。

第 2 題

區分的判準可包括：其一，是否由已發生的痛點驅動——必要的複雜度來自「現在確實遇到的問題」（如某模組已成為擴充瓶頸），而非「未來可能需要」的想像。其二，能否指出具體的量化效益——若無法說明「這樣做能改善什麼、改善多少」，多半是為新穎而新穎。其三，代價是否被誠實評估——包含學習成本、除錯難度、維運負擔與人員異動的風險，而非只看優點。其四，能否漸進導入與回退——好的複雜度通常可以局部試行；若必須全面改造且難以回頭，風險極高。其五，團隊規模與能力是否匹配——五個人維護十二個微服務（案例四）顯然不成比例。實用的做法是採「最簡可行架構」：先用最簡單能運作的方案，並明確記錄「當某個指標達到某個值時，就重新評估」——把未來的複雜度變成有觸發條件的計畫，而非現在就付出的代價。這與第 22 章「能用固定流程就不要用代理」是同一個原則。

第捌篇 資料治理、安全與維運

Governance, Security & Operations

第 32 章

資料治理與資料品質

Data Governance and Data Quality

機器人叫獸（李世淵） 編著

叫獸開講

我要講一個發生在很多公司裡的真實場景，你聽了大概會覺得熟悉。

週一的主管會議上，業務部門報告本月營收是一億兩千萬。財務部門的簡報上寫的是一億一千八百萬。行銷部門引用的儀表板則顯示一億兩千三百萬。三個數字都號稱是「本月營收」，卻沒有一個對得上。

接下來會發生什麼？通常是主管要求「查一下為什麼不一樣」，然後三個部門各派一位分析師，花上一整週的時間比對——最後發現原來業務算的是含稅、財務扣掉了退貨、行銷的儀表板則多算了一個尚未確認的訂單。三個都「沒有錯」，只是定義不同。

這一整週的人力成本，遠超過建立一份「營收的標準定義」所需的時間。而更糟的是，下個月同樣的事情還會再發生一次——因為沒有人把這次的結論記錄下來、也沒有人被指定為「營收這個指標的負責人」。

這就是資料治理要解決的問題。它聽起來像是行政工作、像是文書作業、像是那種「重要但不緊急」而永遠被延後的事。但我要告訴你：第 30 章那個變成沼澤的資料湖、第 13 章那些對不上的數字、第 23 章那個跑不起來的專題，根源都是同一個——技術做好了，但沒有人管「誰負責、從哪來、能不能信」。這一章，我們就來談這件事。

學習目標

研讀完本章之後，你應該能夠：

1. 說明資料治理的目標，並區分它與資料管理的不同。
2. 說明資料目錄與中繼資料的作用，並指出其應涵蓋的內容。
3. 說明資料血緣的價值，並運用它進行影響分析與根因追溯。
4. 說明資料擁有權與管家制度的角色分工。
5. 運用六個維度量測資料品質，並計算綜合品質分數。
6. 設計資料品質的監控機制，並理解及早偵測的價值。

核心概念

核心概念	說明
資料治理	確立資料的責任歸屬、標準與規則，回答「誰決定、依什麼決定」。
中繼資料	描述資料的資料，包含技術性、業務性與營運性三類。
資料目錄	彙集全組織中繼資料、提供搜尋與瀏覽的系統。
資料血緣	記錄資料從來源到終點的流動與轉換路徑。
資料擁有者	對某項資料負最終責任、有權決定其定義與存取的人。
資料品質維度	完整性、正確性、一致性、唯一性、時效性、有效性六個可量測面向。
資料契約	資料提供方與使用方之間對格式、品質與變更的明確約定。

表 32-1 本章核心概念一覽

32.1 治理要解決什麼問題

32.1.1 治理與管理的區別

初學者常把「資料治理」與「資料管理」混為一談，但兩者的層次不同。

資料管理是「做事」——建置管線、維護資料庫、執行備份、優化查詢。這是本書前面各章談的內容。

資料治理則是「定規則」——決定誰有權定義某個指標、誰能存取哪些資料、資料該保留多久、品質不合格時該怎麼辦。它回答的不是「怎麼做」，而是「誰決定、依什麼決定、出事誰負責」。

用一個比喻：管理是駕駛，治理是交通規則與駕照制度。技術再好的駕駛，在沒有交通規則的路上也開不快——因為他永遠不知道別人下一步要幹什麼。

32.1.2 治理不足的四種症狀

治理是否到位，可以從幾個症狀來判斷。若你的組織出現以下情況，那不是技術問題，而是治理問題。

- **同一個指標多個數字：**如叫獸開講中的營收爭議——因為沒有唯一的定義與負責人。
- **找不到資料：**明明存在某處，但沒人知道在哪、欄位是什麼意思——第 30 章的資料沼澤。
- **不敢信任資料：**使用者寧可自己重新撈一份原始資料，因為不確定既有的加工資料是否可靠。
- **出事無人負責：**資料錯了，各方互指是對方的問題，因為從未明確定義過誰該負責。

這四個症狀的共同點是：它們都無法靠「買一套更好的工具」來解決。工具可以支援治理，但治理本身是關於「人與責任的安排」。

32.1.3 治理的三大目標

整理起來，資料治理要達成的目標可歸納為三項——這也是第 2 章 2.5.1 節提過的三大支柱，這裡我們深入展開。

目標	要回答的問題	主要手段
可發現與可理解	有什麼資料？在哪裡？什麼意思？	中繼資料與資料目錄（32.2）
可追溯與可問責	從哪來？誰負責？出事怎麼追？	資料血緣與擁有權（32.3、32.4）
可信任	品質如何？能不能用？	品質維度與監控（32.5、32.6）

表 32-2 資料治理的三大目標

叫獸提醒

我要先給你一個務實的提醒，因為這是治理專案最常失敗的地方：不要一開始就想做「全面治理」。我看過組織花一年時間制定了厚厚一本治理規範，結果沒有人遵守——因為規範太多、成本太高、而且看不到立即的好處。成功的做法通常是相反的：從「最痛的那一個問題」開始（例如那三個對不上的營收數字），只針對核心的幾個指標建立定義與負責人，做出成效後再逐步擴大。治理的推行方式本身，也要遵循「複雜度應該被需求逼出來」這個原則——這與第 22、31 章的判斷是一致的。

32.2 中繼資料與資料目錄

32.2.1 三類中繼資料

中繼資料就是「描述資料的資料」。實務上可分為三類，各自回答不同的問題。

類別	內容	回答的問題
技術性	欄位名稱、型別、大小、格式、儲存位置	這份資料長什麼樣、放在哪裡
業務性	欄位的業務意義、指標定義、使用限制	這個欄位到底代表什麼
營運性	更新頻率、最後更新時間、資料量、品質分數	它是否仍在更新、能不能信

表 32-3 三類中繼資料

多數組織只做到第一類——因為技術性中繼資料可以自動從系統中掃描取得。但真正能解決叫獸開講那個營收爭議的，是第二類「業務性中繼資料」：它必須由懂業務的人來撰寫，無法自動產生。

第三類「營運性中繼資料」則是判斷「能不能用」的關鍵。一份資料即使結構完整、定義清楚，若它其實已經三個月沒更新了，用它做決策就是災難。

32.2.2 資料目錄的價值

資料目錄是把全組織的中繼資料匯集起來、提供搜尋與瀏覽的系統。它的價值可以用一個簡單的問題來衡量：一位新進的分析師，要花多久才能找到並理解他需要的資料？

沒有目錄時，答案通常是「幾天，而且要問遍整個團隊」。有了好的目錄，答案可以是「幾分鐘」。這個差距乘上組織中所有人的所有次搜尋，就是目錄的實際價值。

一個有用的資料目錄至少應包含：可搜尋的資料清單、每份資料的三類中繼資料、擁有者與聯絡方式、血緣關係、以及品質狀態。

這裡有個實務上的關鍵：目錄的價值取決於「內容是否被維護」。一份三年沒更新的目錄，比沒有目錄更糟——因為它會誤導使用者。因此可行的做法是盡量自動化（技術性與營運性中繼資料自動蒐集），而把人力集中在無法自動化的業務性描述上。

32.3 資料血緣

32.3.1 資料從哪來、到哪去

資料血緣（data lineage）記錄的是資料的流動路徑：這張表是由哪些來源、經過哪些轉換而產生的？而它又被哪些下游使用？

血緣可以在不同的粒度上記錄。表層級的血緣說明「A 表與 B 表產生了 C 表」；欄位層級的血緣則更細——「C 表的營收欄位，是由 A 表的金額減去 B 表的退貨額計算而來」。後者更有價值，但記錄的成本也更高。

你應該注意到，血緣本質上是一張圖——節點是資料集、邊是轉換關係，且它是有向無環的。這正是第 18 章談過的 DAG，也是它在本書第三次出現（第 11 章的 Spark 執行計畫、第 13 章的工作流編排、以及這裡）。

32.3.2 兩個關鍵用途

血緣的價值集中在兩個方向相反的問題上。

- **影響分析（往下游看）：**「我要修改這個欄位，會影響到誰？」在變更前先查血緣，就能知道哪些報表、模型與下游系統會受影響，並事先通知——這正是第 2、13 章反覆提到的「上游無預警改結構」災難的解方。
- **根因追溯（往上游看）：**「這個報表的數字不對，問題出在哪一步？」沿著血緣往上追，可以快速定位是哪個來源或哪個轉換出了問題，而不必逐一猜測。

這兩個用途都有一個共同的效果：把「需要靠資深員工的記憶」的知識，變成「系統中可查詢的事實」。這在人員流動時特別重要——那位「什麼都知道」的資深工程師離職後，血緣圖是唯一還記得的東西。

32.3.3 如何取得血緣

血緣的取得方式有三種，各有優缺點。

自動解析是從 SQL 查詢、Spark 作業或編排器的定義中，自動推導出資料的流動關係。這是最理想的方式——不需額外人力，且不會過時。多數現代的資料平台都支援某種程度的自動血緣。

執行時追蹤則是在作業執行時記錄「讀了什麼、寫了什麼」。它比靜態解析更準確（能反映實際發生的事），但需要平台支援。

人工維護是最後的手段。它的問題與文件一樣——一開始很完整，但很快就會過時。因此人工血緣應該只用於「自動化涵蓋不到」的環節，例如透過手動匯出匯入的資料流。

32.4 擁有權與資料管家

32.4.1 沒有負責人就沒有治理

這一節談的是治理中最不技術、卻最關鍵的一環：責任的歸屬。

回想叫獸開講那個營收爭議。技術上，三個數字都算得沒錯；問題在於「沒有人有權決定哪一個才算數」。若組織中有一位明確的「營收指標擁有者」，他可以裁定標準定義，爭議就此結束。

因此資料治理的第一步，往往不是導入工具，而是為關鍵的資料資產指定負責人。而這個責任通常分為兩種角色。

32.4.2 兩種角色

角色	職責	通常由誰擔任
資料擁有者	對該資料負最終責任：裁定定義、核准存取、決定保留期限	該業務領域的主管
資料管家	日常維護：撰寫中繼資料、監控品質、處理問題、協調變更	熟悉該領域的資深分析師或工程師

表 32-4 資料治理的兩種角色

兩者的區別在於「決策權」與「執行責任」。擁有者不必懂技術，但必須有權做決定；管家不必有決策權，但必須熟悉資料的細節。這個分工避免了兩個常見的失敗：由技術人員決定業務定義（他不知道業務意圖），或由業務主管處理日常維護（他沒有時間也沒有能力）。

實務上還有一個關鍵：這些角色必須被「正式指派並公開」，而非默契上的認知。若資料目錄中每份資料都清楚標示擁有者與管家及其聯絡方式，使用者有疑問時就知道該找誰——這個小小的安排，能消除大量的溝通成本。

32.4.3 資料契約

當責任歸屬清楚後，就能建立「資料契約」——資料提供方與使用方之間的明確約定。

一份資料契約通常包含：綱要（有哪些欄位、什麼型別）、品質承諾（完整性與時效性的水準）、變更政策（要改結構時提前多久通知、如何處理相容性）、以及服務等級（多久更新一次、何時可用）。

這正是第 2、13 章反覆提到的「上游無預警改結構」問題的制度性解方。技術上你可以用綱要驗證來偵測變更（第 4 章的綱要演進、第 30 章的表格式），但真正防止它發生的，是一份雙方同意的契約與其背後的責任歸屬。

叫獸提醒

我要強調「契約」這個詞的分量。它不只是一份文件，而是意味著「違反是有後果的」。若上游可以任意改結構而不必承擔任何責任，那麼再詳盡的文件也只是一廂情願。所以推動資料契約時，真正的難點不在於寫文件，而在於讓組織接受「資料提供方對下游負有義務」這個觀念。這需要管理層的支持，也需要把它納入正式的流程（例如綱要變更必須經過核准與通知）。我看過很多團隊做了完整的技術準備，卻因為缺少這一層組織共識而功虧一簣——治理終究是關於人的問題。

32.5 資料品質的六個維度

32.5.1 從抱怨到量測

第 13 章我們列出了資料品質的六個維度，當時的重點是「如何在管線中檢核」。這裡我們談的是「如何量測與管理」——把品質從一句抱怨，變成一個可以追蹤、可以設定目標的數字。

維度	衡量什麼	量測方式範例
完整性	該有的資料是否都在	關鍵欄位的非空值比例
正確性	值是否符合現實	落在合理範圍內的比例
一致性	跨系統或欄位是否矛盾	明細加總與總額相符的比例
唯一性	是否有不該出現的重複	主鍵不重複的比例
時效性	資料是否夠新	在承諾時間內到達的比例
有效性	格式是否符合規範	符合格式與代碼表的比例

表 32-5 資料品質的六個維度與量測

關鍵在於「量測方式」那一欄——每個維度都必須被轉換成一個可自動計算的比例值。只有能被量測的東西，才能被管理、被追蹤趨勢、被設定改善目標。

32.5.2 綜合品質分數

六個維度各有一個分數，但管理層通常想看「一個數字」。因此實務上會計算加權的綜合分數——各維度依其對業務的重要性給予不同權重。

權重的設定必須反映業務的優先序。例如財務資料的「正確性」與「一致性」權重應該最高；而即時監控資料的「時效性」則遠比「完整性」重要——一份完整但遲到兩小時的告警資料，價值幾乎為零。

下一節的公式會示範這個計算。但我要先提醒：綜合分數的用途是「追蹤趨勢」與「跨資料集比較」，而非「取代各維度的細節」。當分數下降時，你仍然必須回頭看是哪一個維度出了問題。

32.5.3 設定合理的目標

最後一個實務問題：品質目標該訂多高？

答案不是「越高越好」，而是「符合用途所需」。追求 100% 的品質，成本會急遽上升（回想第 26 章水位線的報酬遞減），而多數用途其實不需要。

判斷的原則是回到「這份資料要拿來做什麼」。用於趨勢觀察的資料，98% 的完整性通常綽綽有餘；但用於財務結算或法規申報的資料，就必須接近 100%。同一份原始資料在不同用途下，可以有不同的品質要求——這也是第 30 章銅銀金分層的另一個理由：不同層對品質的承諾不同。

32.6 品質監控與持續改善

32.6.1 監控什麼、何時監控

第 13 章我們把品質檢核放進了管線，作為「攔截」的關卡。這裡要補充的是「監控」的面向——不只是攔截當下的錯誤，還要追蹤品質的趨勢。

- **絕對值監控**：品質分數是否低於設定的門檻（如完整性不得低於 95%）。這是最基本的告警。
- **趨勢監控**：品質分數是否持續下滑。即使還在門檻之上，下滑的趨勢往往預示著上游出了問題。
- **異常偵測**：資料量、分布、更新時間是否偏離歷史常態（第 13 章的安靜失敗偵測）。
- **血緣連動**：當某個上游資料集的品質下降時，自動標記所有下游資料集為「品質存疑」。

最後一項特別值得注意：它把 32.3 節的血緣與品質監控結合了起來。若沒有血緣，你只知道某張表有問題；有了血緣，你能立刻知道「哪些報表與模型正在使用有問題的資料」，並主動通知使用者——這比讓他們自己發現要好得多。

32.6.2 及早偵測的價值

品質管理有一個廣為引用的經驗法則，源自製造業：在源頭修正一個錯誤的成本若是 1，到了製程中間變成 10，到了成品階段變成 100，而到了客戶手上則是 1000。

資料領域完全適用同一個規律。一筆錯誤資料在擷取端被攔截，成本只是重送一次；流進管線後才發現，要重跑下游作業；進到報表後才發現，要通知所有看過的人並修正；而若已經據以做出了決策（調整產線參數、下了採購訂單），代價可能是真實的損失。

下一節的公式會把這個階梯量化。它的實務意涵很直接：品質投資應該盡量往上游集中——在擷取端多做一分檢查，勝過在下游做十分補救。這與第 13 章「把品質檢核內建為管線關卡」是同一個結論，只是這裡給了它成本上的論證。

32.6.3 持續改善的循環

最後，品質管理不是一次性的專案，而是持續的循環。

- 第 1 步 量測：** 依六個維度計算各資料集的品質分數，建立基準線。
- 第 2 步 發現：** 透過監控與告警找出問題，並依血緣評估影響範圍。
- 第 3 步 追溯：** 沿血緣往上定位根本原因——是來源系統、擷取邏輯、還是轉換錯誤。
- 第 4 步 修正：** 從根因處修正，而非只補救症狀；並在修正後加上防止再發的檢查。
- 第 5 步 回顧：** 定期檢視品質趨勢與問題類型，找出系統性的改善機會。

其中第四步「從根因處修正」最容易被忽略。趕時間時，人們傾向直接修改下游的錯誤數字（因為快），但這只是把問題掩蓋起來，下次還會再發生。真正的改善必須回到源頭。

32.6.4 承先啟後

本章我們談的是那件「重要但不緊急」的事。從治理與管理的區別出發（32.1），認識了三類中繼資料與資料目錄（32.2）、資料血緣的兩個關鍵用途（32.3）、擁有者與管家的角色分工及資料契約（32.4）、品質的六個維度與綜合分數（32.5），以及監控機制與及早偵測的價值（32.6）。

我最希望你記住的是 32.1.2 節那四個症狀——它們無法靠買工具解決，因為治理本質上是「人與責任的安排」。技術能支援治理，但不能取代它。

而治理的三大目標中，我們已經處理了「可發現」與「可信任」。還有一項尚未展開：可存取，或者說——誰「不」能存取。當資料變得容易發現、容易理解時，一個新的問題就浮現了：如何確保只有該看到的人才看得到？當資料涉及個人隱私、營業秘密與法規要求時，這已不只是技術問題，而是法律責任。這就是下一章的主題：資料安全、隱私與法規遵循。

公式與流程：綜合品質分數與偵測成本階梯

這一節用兩組計算，把「品質」這件常被當成主觀感受的事，變成可以量化與決策的依據。

加權綜合品質分數

設有 k 個品質維度，第 i 個維度的量測分數為 s_i 、其業務重要性權重為 w_i （各權重之和為 1），則綜合品質分數為：

$$Q = \sum w_i \times s_i \quad (\text{其中 } \sum w_i = 1)$$

式 32-1 加權綜合品質分數

以一份銷售資料為例，各維度的權重依其對業務的重要性設定，實測分數如下表。

維度	權重 w_i	分數 s_i	加權貢獻	判讀
完整性	0.25	0.980	0.2450	良好
正確性	0.30	0.950	0.2850	待改善
一致性	0.20	0.990	0.1980	良好
時效性	0.15	0.920	0.1380	最弱一環
唯一性	0.10	0.999	0.0999	優良
綜合分數	1.00	—	0.9659	約 96.6%

表 32-6 綜合品質分數的計算

綜合分數 96.6% 看起來相當健康，但這正是加權分數需要小心的地方——它掩蓋了「時效性只有 92%」這個最弱的一環。若這份資料的用途是即時監控，那麼 8% 的資料遲到，實際影響可能遠比分數所顯示的嚴重。

因此使用綜合分數的正確方式是：以它追蹤「趨勢」與做「跨資料集比較」，但診斷問題時必須回到各維度的細節。這與第 3 章「不要只看平均延遲」是同一個統計直覺。

偵測成本階梯

設在第 j 個階段才發現並修正一個錯誤的成本為 C_j 。依製造業廣為引用的經驗法則，成本大致呈十倍遞增：

$$C_j \approx C_1 \times 10^{(j-1)}$$

式 32-2 偵測階段與修正成本的關係

發現階段	相對成本	需要做什麼	影響範圍
擷取端攔截	1	拒收並請上游重送	無
管線中發現	10	重跑下游作業	內部作業
報表中發現	100	修正並通知所有使用者	跨部門
決策後發現	1,000	修正決策、承擔實際損失	業務損失

表 32-7 錯誤發現階段與修正成本

叫獸提醒

表 32-7 這個十倍階梯，給了品質投資一個非常清楚的方向：資源應該盡量往上游集中。在擷取端多寫十行檢查程式，可能省下的是下游數百倍的補救成本。但我要提醒一個常見的心理障礙——上游的檢查，成本是「立即可見」的（要寫程式、要維護、可能還會擋下一些其實沒問題的資料而引發抱怨），而它省下的成本卻是「看不見」的（因為那些災難根本沒發生）。這使得品質投資在組織中往往難以爭取資源。我的建議是：把「攔截到的問題」也納入監控指標並定期呈現——讓大家看到「這個月擋下了三百筆錯誤資料」，那些看不見的價值才會變得可見。這也是為什麼第 32.6.1 節要強調趨勢監控，而不只是告警。

案例研討

案例一 三個營收數字

回到叫獸開講。那場爭議最後花了一週才釐清，結論是三個部門各自採用了不同的計算口徑——但這個結論並沒有被記錄下來，也沒有人被指定為負責人。

三個月後，同樣的爭議再度發生。這次他們終於採取了行動：為「營收」這個指標指定了一位擁有者（財務主管）與一位管家（財務部的資深分析師），由擁有者裁定標準定義，管家則把定義寫入資料目錄的業務性中繼資料中，並在所有相關的報表上標註其定義來源。

此後爭議消失。這個案例說明了 32.4.1 節的核心論點——問題不在於「誰算錯了」，而在於「沒有人有權決定哪一個才算數」。治理的第一步，往往是指定負責人而非導入工具。

案例二 一次改欄位引發的連鎖反應

某上游團隊為了優化，把一個欄位的單位從「公克」改為「公斤」，並認為這只是內部調整。他們通知了直接的下游，但沒有更廣泛地公告。

兩週後，一份高層報表出現異常——某項成本指標暴增一千倍。追查花了三天，因為那份報表與該欄位之間隔了四層轉換，沒有人記得中間的關聯。

他們後來導入了自動血緣解析。此後任何欄位變更前，都能先查詢「影響分析」，列出所有受影響的下游資產與其負責人，一次通知到位。同時也建立了資料契約，明訂結構變更須提前兩週通知。

這個案例展示了 32.3.2 節兩個用途的實際價值——事前的影響分析可以預防，事後的根因追溯可以縮短診斷時間。而它也再次印證第 2、13 章反覆提到的「上游無預警改結構」，始終是資料系統最常見的災難來源。

案例三 九十六分的資料

某團隊為其核心資料集建立了品質儀表板，綜合分數長期維持在 96% 以上，大家都認為狀況良好。

直到某次產線異常，維運人員發現告警比實際發生時間晚了近半小時——追查後發現，時效性維度的分數其實只有 92%，且已持續下滑數月。但因為它在綜合分數中的權重只有 0.15，這個惡化被其他維度的良好表現掩蓋了。

這正是 32.5.2 節與表 32-6 所警告的情況。他們的修正有兩點：其一，調高時效性的權重（因為對即時監控而言它最關鍵）；其二，除了綜合分數外，同時監控各維度的個別趨勢，任一維度下滑即告警。這個案例的教訓與第 3 章的 p99 完全一致——彙總指標會掩蓋局部的惡化。

案例四 在源頭多寫的十行程式

某資料團隊統計了過去一年的資料問題處理紀錄，發現一個現象：在擷取端被攔截的問題平均耗時 15 分鐘處理，而流到報表後才被發現的問題，平均耗時超過 20 小時（含追查、修正、重跑、通知）。

這個比例大致符合式 32-2 的十倍階梯。他們據此重新分配了投入：在各資料來源的擷取端加上網要驗證與基本的範圍檢查，總共約增加了兩百行程式碼與一週的開發時間。

半年後回顧，流到下游的資料問題減少了約七成。更重要的是，他們開始在月報中呈現「本月攔截的問題數」，讓這些「沒有發生的災難」變得可見——這也讓後續爭取品質相關的資源變得容易許多。這正是叫獸提醒中所建議的做法。

實作活動

活動一 為一份資料建立中繼資料

請為你手上任一份資料集，建立完整的三類中繼資料。

- 步驟 1.** 技術性：列出所有欄位的名稱、型別、可否為空、以及資料的儲存位置與格式。
- 步驟 2.** 業務性：為每個欄位撰寫「業務意義」的說明，特別是那些名稱不直觀的欄位；並註明已知的使用限制。
- 步驟 3.** 營運性：記錄更新頻率、最後更新時間、資料量、以及已知的品質問題。
- 步驟 4.** 請一位不熟悉這份資料的同學，僅憑你的中繼資料判斷「這份資料能否用於某個特定分析」，並記錄他遇到的疑問。

活動二 計算並診斷品質分數

請實作品質量測並運用式 32-1。

- 步驟 1.** 為一份資料集撰寫程式，計算表 32-5 中六個維度的分數（可依實際情況取其中四至五個）。
- 步驟 2.** 依該資料的用途設定各維度的權重（並說明你的設定理由），計算綜合分數。
- 步驟 3.** 刻意在資料中注入一些問題（如空值、重複、格式錯誤），重新計算並觀察各維度分數的變化。
- 步驟 4.** 討論：若只看綜合分數，你注入的哪些問題會被掩蓋？這對監控設計有什麼啟示？

活動三 畫出你的血緣圖並做影響分析

請為一條你熟悉的資料流程繪製血緣圖。標明所有來源、中間的轉換步驟、以及最終的使用者（報表、模型或下游系統）。接著進行兩項分析：其一，影響分析——假設某個來源欄位要變更型別，列出所有會受影響的下游資產。其二，根因追溯——假設最終報表的某個數字有誤，寫出你會依什麼順序、檢查哪些環節來定位問題。最後說明：若這條流程涉及五個以上的轉換步驟，沒有血緣圖時，這兩項分析各要花多久？

延伸閱讀

- **《DMBOK：資料管理知識體系指南》**：資料管理協會（DAMA）出版，是資料治理與各項資料管理功能的權威參考，涵蓋本章所有主題。
- **《Data Quality Fundamentals》**：說明資料品質的量測、監控與可觀測性實務，是 32.5、32.6 節的完整延伸。
- **開源資料目錄工具的文件**：如 DataHub、Amundsen 等，說明中繼資料模型與血緣的自動蒐集機制，是 32.2、32.3 節的實作參考。
- **資料契約的實務文章**：近年興起的實務領域，說明如何以契約規範資料提供方與使用方的權責，是 32.4.3 節的深化。

本章小結

1. 資料管理是「做事」（建管線、維護系統），資料治理是「定規則」（誰決定、依什麼決定、出事誰負責）；治理不足的四個症狀——同一指標多個數字、找不到資料、不信任、出事無人負責——都無法靠買工具解決。
2. 中繼資料分技術性（自動可得）、業務性（需人工撰寫，卻最能解決定義爭議）與營運性（判斷資料能否使用的關鍵）三類；資料目錄的價值取決於內容是否被維護，故應盡量自動化並把人力集中在業務性描述。
3. 資料血緣本質上是一張 DAG，有兩個關鍵用途：往下游的影響分析（變更前預先評估與通知）與往上游的根因追溯（快速定位問題來源）；它把仰賴資深員工記憶的知識，變成系統中可查詢的事實。
4. 治理的第一步往往是指定負責人而非導入工具：資料擁有者握有決策權（裁定定義、核准存取），資料管家負責日常維護；兩者的角色必須被正式指派並公開。
5. 資料契約以明確的綱要、品質承諾與變更政策，制度性地解決「上游無預警改結構」的問題；其難點不在寫文件，而在讓組織接受「資料提供方對下游負有義務」的觀念。
6. 品質的六個維度都必須被轉換為可自動計算的比例值——能被量測，才能被管理；品質目標不是越高越好，而應「符合用途所需」，同一份資料在不同層級可有不同的品質承諾。

7. 品質監控應涵蓋絕對值、趨勢、異常偵測與血緣連動四個面向；其中血緣連動能在上游品質下降時，自動標記所有受影響的下游並主動通知使用者。
8. 綜合品質分數為 $Q = \sum w_i s_i$ (式 32-1)，適合追蹤趨勢與跨資料集比較，但會掩蓋最弱的維度，故診斷時仍須回到細節；偵測成本呈十倍階梯遞增 (式 32-2)，故品質投資應盡量往上游集中，並把「攔截到的問題數」納入監控以使其價值可見。

習題

一、觀念題

1. 請說明資料治理與資料管理的區別，並以叫獸開講的營收爭議為例，說明為何它是治理問題而非技術問題。
2. 三類中繼資料分別是什麼？為何說「業務性中繼資料」最難自動化卻最有價值？
3. 資料血緣的兩個關鍵用途為何？請各舉一個實際的使用情境。
4. 資料擁有者與資料管家的職責有何不同？為何這個分工能避免兩種常見的失敗？

二、計算題

1. 某資料集的品質量測結果為：完整性 0.96 (權重 0.20)、正確性 0.99 (0.35)、一致性 0.94 (0.20)、時效性 0.85 (0.15)、唯一性 1.00 (0.10)。(一)依式 32-1 計算綜合品質分數；(二)指出最弱的維度；(三)若把時效性的權重提高到 0.30、並相應把正確性降為 0.20，綜合分數變為多少？說明這個變化的意涵。
2. 某組織每月平均發生 50 起資料問題。目前有 20% 在擷取端被攔截、50% 在管線中發現、25% 在報表中發現、5% 在決策後才發現。(一)依式 32-2 以相對成本 1、10、100、1000 計算每月的總相對成本；(二)若透過強化擷取端檢查，使比例變為 60%、30%、9%、1%，新的總相對成本為何？(三)降低了幾成？

三、思考與討論

1. 本章指出品質投資的價值「看不見」——因為它省下的是沒有發生的災難。請討論：還有哪些工程實務具有這種特性？組織該如何衡量與呈現它們的價值？
2. 案例三中，綜合分數掩蓋了最弱維度的惡化。請討論：在你熟悉的領域中，還有哪些「彙總指標掩蓋局部問題」的例子？該如何設計監控來避免？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

區別：資料管理是「做事」——建置管線、維護資料庫、執行備份、優化查詢，屬執行層面；資料治理是「定規則」——決定誰有權定義指標、誰能存取、保留多久、不合格時如何處理，回答的是「誰決定、依什麼決定、出事誰負責」。以營收爭議為例：三個部門的計算在技術上都沒有錯（管理層面沒問題），問題在於「沒有人有權裁定哪一個定義才算數」，也沒有人被指定為該指標的負責人——這是責任與決策權的安排問題，故屬治理範疇。可補充：正因如此，它無法靠導入更好的工具解決，而必須先指定擁有者（見案例一）。

第 2 題

三類：技術性（欄位名稱、型別、格式、儲存位置——回答「長什麼樣、放哪裡」）、業務性（欄位的業務意義、指標定義、使用限制——回答「到底代表什麼」）、營運性（更新頻率、最後更新時間、資料量、品質分數——回答「是否仍在更新、能不能信」）。業務性最難自動化的原因：它需要理解業務脈絡與意圖，無法從系統中掃描取得，必須由懂業務的人撰寫。最有價值的原因：多數的資料爭議（如營收的三個數字）本質上是「定義不一致」，而唯有業務性中繼資料能記錄並統一這些定義——技術性中繼資料告訴你欄位叫什麼型別，卻無法告訴你它是否含稅、是否扣除退貨。

第 3 題

兩個用途方向相反。其一，影響分析（往下游看）——變更前先查詢「這個欄位被誰使用」，列出所有受影響的報表、模型與下游系統並事先通知；情境如案例二中要把單位從公克改為公斤時，先確認所有下游再行變更。其二，根因追溯（往上游看）——當結果有誤時沿血緣往上定位是哪個來源或轉換出問題；情境如報表數字異常時，快速鎖定是四層轉換中的哪一層，而非逐一猜測。兩者的共同效果是把「仰賴資深員工記憶」的知識，變成系統中可查詢的事實——這在人員流動時特別重要。

第 4 題

資料擁有者握有「決策權」：裁定定義、核准存取、決定保留期限，通常由該業務領域的主管擔任，不必懂技術但必須有權做決定。資料管家負責「執行」：撰寫中繼資料、監控品質、處理問題、協調變更，通常由熟悉該領域的資深分析師或工程師擔任，不必有決策權但須熟悉細節。避免的兩種失敗：其一，由技術人員決定業務定義——他不了解業務意圖，可能訂出技術上合理但業務上無意義的定義；其二，由業務主管處理日常維護——他既沒有時間也沒有能力，結果是中繼資料永遠不會被更新。此分工使決策與執行各得其所。

二、計算題

第 1 題

依式 32-1, $Q = \sum w_i s_{i0}$

- (1) 加權貢獻：完整性 $0.20 \times 0.96 = 0.1920$ ；正確性 $0.35 \times 0.99 = 0.3465$ ；一致性 $0.20 \times 0.94 = 0.1880$ ；時效性 $0.15 \times 0.85 = 0.1275$ ；唯一性 $0.10 \times 1.00 = 0.1000$ 。綜合分數 $Q = 0.9540$ ，即 95.4%。
- (2) 最弱維度為「時效性」（0.85），明顯低於其他各項。
- (3) 調整權重後（時效性 0.30、正確性 0.20，其餘不變）： $0.20 \times 0.96 + 0.20 \times 0.99 + 0.20 \times 0.94 + 0.30 \times 0.85 + 0.10 \times 1.00 = 0.1920 + 0.1980 + 0.1880 + 0.2550 + 0.1000 = 0.9330$ ，即 93.3%。

意涵：權重調整使綜合分數從 95.4% 降至 93.3%——資料本身完全沒變，只是「我們更在意什麼」變了。這說明綜合分數並非資料的客觀屬性，而是「資料品質相對於特定用途」的評估。若這份資料要用於即時監控，提高時效性權重才能讓分數真實反映其可用性；反之若用於財務結算，正確性的權重就該最高。故設定權重時必須先問「這份資料要拿來做什麼」。

第 2 題

每月 50 起，相對成本依式 32-2 為 1、10、100、1000。

- (1) 目前：擷取端 $50 \times 0.20 = 10$ 起 $\times 1 = 10$ ；管線 $50 \times 0.50 = 25$ 起 $\times 10 = 250$ ；報表 $50 \times 0.25 = 12.5$ 起 $\times 100 = 1,250$ ；決策後 $50 \times 0.05 = 2.5$ 起 $\times 1000 = 2,500$ 。總相對成本 $= 10 + 250 + 1,250 + 2,500 = 4,010$ 。
- (2) 改善後：擷取端 30 起 $\times 1 = 30$ ；管線 15 起 $\times 10 = 150$ ；報表 4.5 起 $\times 100 = 450$ ；決策後 0.5 起 $\times 1000 = 500$ 。總相對成本 $= 30 + 150 + 450 + 500 = 1,130$ 。

(3) 降低比例 = $(4,010 - 1,130) / 4,010 \approx 0.718$ ，即降低約 72%。

延伸思考：注意問題總數完全沒有減少（仍是每月 50 起），改變的只是「在哪個階段被發現」——但總成本卻降了七成以上。這正是十倍階梯的威力，也說明品質投資的重點不在於「消滅所有問題」（不可能），而在於「讓問題盡早被發現」。此外可觀察到，改善後最大的成本來源仍是那 0.5 起流到決策階段的問題（佔 500，約 44%）——這提示了下一步的改善方向。

三、思考與討論

第 1 題

具有此特性的實務包括：其一，資訊安全防護——沒有被入侵時，投資看起來像是浪費。其二，備份與災難復原演練——平時毫無產出，只在災難時才顯價值。其三，自動化測試與程式碼審查（第 23 章）——擋下的臭蟲不會被看見。其四，容量規劃與冗餘設計（第 3、29 章）——沒有發生的當機無人感謝。其五，技術債的償還——重構後「沒有變慢」很難被認可。共同特徵：它們的產出是「負面事件的不發生」，而不發生的事無法被直接觀察。衡量與呈現的方法：其一，量化攔截成果——統計並定期呈現「本月擋下的問題數、預估避免的損失」（如案例四）。其二，記錄事故成本作為對照——把過去發生過事故成本明確記錄，作為預防投資的參照基準。其三，以領先指標取代結果指標——監控「檢查覆蓋率、演練頻率、平均修復時間」等可觀察的過程指標。其四，說故事——以具體的事故案例（自身或同業）呈現代價，比抽象數字更有說服力。核心觀念：讓看不見的價值變得可見，本身就是一項需要刻意設計的工作。

第 2 題

彙總掩蓋局部的例子包括：其一，平均延遲掩蓋尾端（第 3 章的 p99、第 31 章的冷啟動）——多數請求很快而少數極慢，平均值看似健康。其二，整體良率掩蓋特定機台或批次的異常——總良率 98% 可能是九台機台 99.5% 加一台 85%。其三，模型的整體準確率掩蓋特定族群的低表現——這正是第 19、23 章談的偏誤問題。其四，叢集整體使用率掩蓋個別節點的瓶頸（第 20 章的落後者）。其五，總營收成長掩蓋某產品線的衰退。避免的監控設計：其一，同時監控彙總與分項，任一分項越界即告警（如案例三的做法）。其二，監控分布而非只看單一統計量——使用百分位數、直方圖或箱型圖。其三，設定「最差子群」指標——直接追蹤表現最差的那一組，而非平均。其四，以趨勢偵測補強——即使絕對值仍在門檻內，持續下滑就應告警。核心原則：任何彙總都是資訊的壓縮，而被壓縮掉的往往正是你最需要知道的異常。

第 33 章

資料安全、隱私與法規遵循

機器人叫獸（李世淵） 編著

叫獸開講

我要先問你一個問題：把姓名和身分證字號拿掉，這份資料就算「匿名」了嗎？

多數人的直覺是「應該算吧」。但我們來做一個簡單的計算。台灣約有兩千三百萬人。若一份資料保留了「性別、居住鄉鎮、出生年月日」這三個看起來無害的欄位——性別兩種、鄉鎮約三百六十八個、生日約三萬六千五百種組合——相乘起來超過兩千六百萬種組合，比總人口還多。

這意味著什麼？意味著平均而言，每一種組合對應不到一個人。換句話說，只要知道某人的性別、住哪個鄉鎮、以及完整生日，就有很大的機會能從這份「已匿名」的資料中把他指認出來——而这三項資訊，可能從他的社群貼文就能推得。

這個現象叫做「重識別」，而它是隱私工程中最重要也最反直覺的一課：匿名不是把幾個欄位塗黑就完事，而是一個需要計算的性質。

這一章我們要談安全、隱私與法規。我要先說明立場：我不會教你任何攻擊技巧——這既不必要也不恰當。我要教的是「防禦者的思維」：理解風險從何而來、有哪些機制可以降低它、以及當你手上握有他人的資料時，你承擔了什麼樣的責任。而最後這一點，才是這一章真正的重點。

學習目標

研讀完本章之後，你應該能夠：

1. 說明資料安全的三個核心目標，並辨識常見的風險來源。
2. 說明存取控制的模型，並運用最小權限原則設計授權。
3. 區分傳輸中與靜置時的加密，並說明金鑰管理的重要性。
4. 區分去識別化與匿名化，並運用 k-匿名評估重識別風險。
5. 說明個人資料保護的基本原則與組織應盡的義務。
6. 說明資料倫理的考量，並在技術決策中納入負責任的思維。

核心概念

核心概念	說明
機密性、完整性、可用性	資訊安全的三大目標，合稱 CIA 三要素。
最小權限原則	只授予完成工作所必需的最低權限。
靜置加密與傳輸加密	資料儲存於媒體上與在網路傳輸中的兩種保護。
準識別碼	本身不足以識別個人，但組合起來可能指認出特定個人的欄位。
k-匿名	確保每一組準識別碼組合至少對應 k 個人的去識別化標準。
去識別化與匿名化	前者可能仍有重識別風險，後者則應不可逆。
責任共擔	使用雲端服務時，供應商與使用者各自承擔不同的安全責任。

表 33-1 本章核心概念一覽

33.1 資料安全的目標與風險

33.1.1 三個核心目標

資訊安全的目標可歸納為三項，合稱 CIA 三要素。理解它們的區別很重要，因為不同的威脅對應不同的目標。

目標	要保護什麼	違反時的後果
機密性	確保只有授權者能存取	資料外洩、隱私侵害
完整性	確保資料未被未授權地竄改	決策依據錯誤、稽核失效
可用性	確保授權者需要時能取得	服務中斷、業務停擺

表 33-2 資訊安全的三個核心目標

多數人談資安時只想到「機密性」——不要被偷。但另外兩項同樣重要：一份被惡意竄改的品質數據，可能導致不良品流入市場；而一個因勒索軟體而無法存取的系統，即使資料沒外洩，業務照樣停擺。

你會注意到「可用性」其實在第 3 章就談過了——那時我們從系統可靠度的角度討論它。這裡的觀點是：可用性同時也是一個安全議題，因為它可能被惡意破壞。

33.1.2 大數據環境的特殊風險

大數據系統面對的風險，與傳統系統有幾個結構性的差異。

- **集中的高價值目標：**資料湖把全組織的資料集中在一處——這帶來分析的便利，但也意味著單一次入侵可能造成全面性的損害。
- **多方存取：**資料要被分析師、工程師、模型與外部夥伴使用，存取面遠比傳統系統廣，權限管理的複雜度也隨之上升。
- **組合帶來的敏感性：**個別看來無害的資料集，組合後可能產生高度敏感的資訊——這正是叫獸開講中重識別的原理。
- **長期保留：**第 30 章談的「先存下來再說」，意味著風險暴露的時間也拉長了。
- **跨境流動：**雲端服務可能把資料存放在其他國家，牽涉當地法規的管轄（第 27 章的資料主權）。

其中「組合帶來的敏感性」最容易被忽略。你可能對每一份資料集都做了合規審查，卻沒有評估「這些資料集被同一個人同時取得」的後果。這是大數據環境特有的風險，也是治理（第 32 章）與安全必須整合的原因。

33.2 存取控制與身分驗證

33.2.1 驗證與授權

兩個常被混用的詞，必須先區分清楚。

驗證（authentication）回答「你是誰」——確認來訪者的身分是否屬實，透過密碼、金鑰、多因素驗證等機制。

授權（authorization）回答「你能做什麼」——在確認身分之後，決定該身分能存取哪些資源、執行哪些操作。

兩者是先後關係，缺一不可。只有驗證而沒有授權，等於「確認你是本校學生，然後讓你進入所有教室包括校長室」；只有授權而沒有驗證，則是「宣稱自己是誰就是誰」。

33.2.2 常見的授權模型

模型	依什麼決定權限	特性
以角色為基礎	使用者被指派角色，角色具有權限	管理簡單、最為常見
以屬性為基礎	依使用者、資源與情境的屬性動態判斷	彈性高，可表達複雜規則
以標籤為基礎	依資料的分類標籤（如機密等級）授權	適合大量資料集的統一管理

表 33-3 三種授權模型

在資料平台中，以標籤為基礎的做法特別實用。你不必為每一張表逐一設定權限，而是為資料打上分類標籤（如「含個資」「營業機密」「公開」），再依標籤統一授權。這也把第 32 章的中繼資料與安全控制連結了起來——分類本身就是一種中繼資料。

33.2.3 最小權限原則

這是安全設計中最重要、也最常被違反的原則：只授予完成工作所必需的最低權限。

為什麼重要？因為它限制了「出事時的損害範圍」。第 22 章我們談代理的權限時說過同一句話：你無法保證不出錯，但你可以限制錯誤的後果。這個原則對人、對程式、對 AI 代理一體適用。

實務上違反這個原則的常見情境包括：為求方便給予管理員權限、專案結束後未收回權限、以及共用同一組高權限帳號（如此還會失去稽核軌跡——你不知道是誰做的）。

叫獸提醒

我要特別談「權限的清理」這件事，因為它是實務上最普遍的漏洞。授予權限時大家都很謹慎，但收回權限卻幾乎沒有人記得——某人換了部門、專案結束了、實習生離開了，他們的權限往往還留在系統裡好幾年。這種累積下來的「殭屍權限」，是資安稽核最常發現的問題。可行的做法有二：其一，權限設定到期日，需要就重新申請；其二，定期進行權限盤點，由各資料的擁有者（第 32 章）確認清單。這聽起來很行政，但它比任何高深的加密技術都更能實際降低風險——因為多數的資料外洩，不是被高手駭進來的，而是從一個早該被收回的帳號流出去的。

33.3 加密與金鑰管理

33.3.1 兩種狀態的加密

資料在不同狀態下需要不同的保護。

傳輸加密保護「在網路上移動中」的資料，防止傳輸過程被竊聽或竄改。現代實務上這幾乎是預設要求——任何跨網路的通訊都應加密，包括內部網路（因為「內部網路是安全的」正是第 3 章那八個誤解之一的變體）。

靜置加密保護「儲存在媒體上」的資料。它的主要價值在於：當儲存媒體遺失、被竊、或雲端的底層硬體被不當存取時，資料仍無法被讀取。多數雲端儲存服務都提供預設的靜置加密。

還有一種較新的方向是「使用中加密」——讓資料在運算過程中仍保持加密狀態。這在技術上相當困難，目前多用於特定的高敏感場景。

33.3.2 金鑰管理才是關鍵

這一節我要強調一個常被忽略的重點：加密的強度，最終取決於金鑰的保護。

道理很直觀——若金鑰與加密資料放在同一個地方，那就像把保險箱的鑰匙貼在保險箱上。而這種情況在實務中比想像的常見：金鑰被寫在程式碼裡、被存在設定檔中、被推送到版本控制系統（回想第 28 章案例三那個藏在映像檔第三層的密碼）。

正確的做法是使用專門的金鑰管理服務：金鑰集中保管、與資料分離、有存取控制與稽核紀錄、且支援定期輪替。應用程式在需要時才向該服務請求，且權限受最小權限原則約束。

這裡也要提醒：加密不是萬靈丹。它保護的是「資料在儲存與傳輸時不被未授權者讀取」，但對於「有權限的人濫用資料」完全無能為力。而後者，恰恰是實務上更常見的風險。

33.4 去識別化與重識別風險

33.4.1 三類欄位

回到叫獸開講的核心問題。要理解重識別，必須先區分三類欄位。

- **直接識別碼**：本身即可唯一指認個人，如姓名、身分證字號、手機號碼。去識別化的第一步就是移除或替換它們。
- **準識別碼**：本身不足以識別，但「組合起來」可能指認出特定個人，如性別、出生日期、郵遞區號、職稱。這是重識別風險的主要來源，也是最常被低估的一類。
- **敏感屬性**：本身不識別身分，但一旦與個人連結就構成隱私侵害，如病歷、薪資、宗教信仰。這是我們真正要保護的內容。

重識別攻擊的邏輯是：透過準識別碼把匿名紀錄與某個特定個人對應起來，進而得知該個人的敏感屬性。因此保護的重點不只在於移除直接識別碼，更在於「降低準識別碼組合的獨特性」。

33.4.2 k-匿名

k-匿名是量化這個風險最基本的工具。它的定義是：對於資料中任一筆紀錄，至少存在 $k-1$ 筆其他紀錄，具有完全相同的準識別碼組合。

換句話說，每一種準識別碼的組合，至少要對應 k 個人——如此攻擊者即使鎖定了某個組合，也無法確定是其中的哪一位。 k 值越大，保護越強。

達成 k-匿名的常見手法有兩種：「概化」（把精確值變粗糙，如把生日改為出生年、把鄉鎮改為縣市）與「抑制」（直接移除那些過於獨特的紀錄）。兩者都會損失資訊——這正是隱私保護的根本取捨：隱私與資料效用是此消彼長的。

下一節的公式會示範這個計算，你會看到概化的效果有多顯著。

33.4.3 k-匿名的不足

k-匿名雖然實用，但它有幾個已知的弱點，你必須認識它們才不會過度自信。

其一是「同質性問題」：即使某個組合有 10 個人 ($k = 10$)，若這 10 個人的敏感屬性全都相同（例如都罹患同一種疾病），那麼攻擊者不必知道是哪一位，也已經得知了敏感資訊。

其二是「背景知識問題」：攻擊者可能已經知道某些額外資訊，能藉此縮小範圍。

其三，也是最根本的： k -匿名只保護「單一份資料」，但若同一個人的資料出現在多個資料集中，交叉比對可能突破保護。這正是 33.1.2 節談的「組合帶來的敏感性」。

因此業界發展了更強的標準（如要求每組內的敏感屬性須具備多樣性），以及一個思路完全不同的方向——差分隱私。它不追求「讓個人藏在群體中」，而是「在結果中加入經過計算的雜訊，使得任一個人是否在資料集中，都不會顯著改變結果」。這提供了數學上更嚴謹的保證，但代價是結果的精確度會下降。

33.5 個資保護與法規遵循

33.5.1 共通的基本原則

各國的個人資料保護法規細節不同，但背後有一組共通的原則。理解這些原則，比背誦條文更有價值——因為原則相對穩定，而條文會變。

- **目的特定與限制**：蒐集資料時必須有明確的目的，且不得超出該目的使用。「先收集再想用途」在個資上是有問題的。
- **最小蒐集**：只蒐集達成目的所必需的資料。這與最小權限原則同源。
- **當事人權利**：個人對自己的資料享有查詢、更正、刪除等權利，組織必須有能力回應這些請求。
- **安全維護義務**：持有個資者有義務採取適當的保護措施，未盡義務即須負責。
- **可歸責性**：組織必須能夠證明自己遵守了這些原則——這意味著需要文件與紀錄。

第一項對大數據領域衝擊最大。第 30 章我們談資料湖的哲學是「先收下再說」——但這個哲學在涉及個資時必須修正。你不能因為儲存便宜就無限期保留個人資料，也不能把為 A 目的蒐集的資料拿去做 B 用途。這是技術思維與法律思維的一個重要落差。

33.5.2 技術如何支援合規

法規要求往往可以對應到具體的技術能力，而這些能力多數在前面章節已經談過。

法規要求	所需的技術能力	對應章節
知道自己持有哪個個資	資料目錄與分類標籤	第 32 章
能追溯資料的流向	資料血緣	第 32 章
能回應刪除請求	能定位並刪除特定個人的所有紀錄	第 30 章的表格式
能證明存取受控	存取控制與稽核紀錄	本章 33.2 節
能限制保留期限	生命週期管理與自動刪除	第 2 章

表 33-4 法規要求與技術能力的對應

其中「回應刪除請求」在資料湖架構中特別棘手——因為資料可能散落在多個檔案中，而物件儲存的檔案是不可修改的。這正是第 30 章開放表格式的另一個價值：它提供了刪除與更新的能力，使合規變得可行。

這裡有個值得注意的觀察：法規遵循並不是在既有系統上「加裝」的東西，而是必須從架構設計時就納入考量。若你的資料湖沒有記錄「哪些檔案含有哪個人的資料」，那麼收到刪除請求時，你可能得掃過整個資料湖——這在技術上與成本上都難以承受。

33.5.3 責任共擔

第 27 章我們提過「把系統交給供應商，不等於把責任也交出去」。這裡把它講得更明確。

在雲端環境中，安全責任是「共擔」的：供應商負責「雲本身的安全」（實體設施、底層硬體、虛擬化層），而使用者負責「雲中資料的安全」（存取控制設定、加密選項、資料分類、應用程式安全）。

這個分界隨服務模型而移動（第 27 章表 27-2），但有一件事永遠不變——資料的內容與其合規性，始終是使用者的責任。供應商可以提供加密功能，但「有沒有啟用」是你的決定；供應商可以提供存取控制，但「設定得對不對」是你的責任。

實務上最常見的雲端資料外洩，並非供應商被入侵，而是使用者把儲存桶設定成了公開存取。技術是中性的，責任在人。

33.6 資料倫理與負責任的實務

33.6.1 合法不等於正當

這一節要談的是法律之外的層次。有些做法在法律上站得住腳，在倫理上卻仍有問題——而技術人員往往是唯一能在事前看見這些問題的人。

幾個典型的例子：以使用者不會細讀的冗長條款取得「同意」；把為某目的蒐集的資料用於推論其未曾揭露的屬性；或以看似中性的特徵（如居住區域）間接產生具歧視性的效果。

這些都不必然違法，但它們都涉及一個共同的問題——資料的使用超出了當事人合理預期的範圍。

我在第 19 章談推薦系統時說過一句話，這裡要再說一次：當你優化一個指標時，你其實是在替一大群人做決定。而資料倫理的核心，就是意識到這件事並認真對待它。

33.6.2 實務上的自我檢核

倫理不能只停留在原則，必須落實為可操作的檢核。以下是我建議在專案中反覆自問的問題。

- **當事人會如何看待：**若資料當事人完整知道我們正在對他的資料做什麼，他會覺得合理嗎？
- **是否真的必要：**這個欄位、這項分析，對達成目的是必要的嗎？還是只是「有就順便收」？
- **誰會受到影響：**分析結果會被用來對誰做出什麼決定？若判斷錯誤，受影響最深的是誰？
- **是否存在系統性偏誤：**訓練資料是否充分涵蓋各族群？結果對某些群體是否明顯較差（第 23 章的模型卡）？
- **能否解釋與申訴：**受影響的人能否知道為什麼、並提出異議？

叫獸提醒

我要說一段比較嚴肅的話。你們畢業後，很可能會成為組織中「唯一真正理解這份資料能做什麼」的人。當有人提出一個技術上可行、但你隱約覺得不妥的需求時，往往只有你看得出問題所在——因為提出需求的人並不了解技術的能力邊界。這個時候，「我只是照著規格做」不是一個好的答案。我不是要你去對抗組織，而是希望你至少能把疑慮明確地說出來、記錄下來，

讓決策者在知情的狀況下做決定。專業能力帶來的不只是價值，也帶來相應的責任——這是我在這本書裡最想留給你們的一句話。

33.6.3 承先啟後

本章我們談的是責任。從安全的三個核心目標與大數據環境的特殊風險出發（33.1），認識了驗證與授權的區別及最小權限原則（33.2）、兩種狀態的加密與金鑰管理才是關鍵（33.3）、去識別化的三類欄位與 k-匿名的量化與不足（33.4）、個資保護的共通原則與技術如何支援合規（33.5），最後談了法律之外的倫理層次（33.6）。

我最希望你帶走的有兩點。第一，匿名不是把欄位塗黑，而是一個需要計算的性質——這個反直覺的認知，可能是本章最實用的一課。第二，多數的資料外洩不是被高手駭進來的，而是源於一個早該收回的權限、一個設錯的儲存桶、或一段寫在程式碼裡的密碼。安全的日常，比安全的高深技術更重要。

到這裡，我們談了治理（誰負責、能不能信）與安全（誰能存取、如何保護）。但還有最後一塊：這一切要如何在「每天實際運作」中維持下去？資料管線、模型與平台不是建好就結束，它們需要持續的部署、監控與改善——而這需要一套把開發與維運整合起來的方法。這就是第捌篇最後一章、也是本書技術篇章的收尾：DataOps 與 MLOps。

公式與流程：準識別碼的獨特性與 k-匿名

這一節要把叫獸開講那個「拿掉姓名就算匿名了嗎」的問題，變成一個可以計算的評估。

組合數與平均組大小

設某份資料涵蓋的母體人數為 N ，而準識別碼共有 m 個欄位，第 i 個欄位有 v_i 種可能取值。則準識別碼的組合總數為各欄位取值數的乘積：

$$\text{組合數 } C = v_1 \times v_2 \times \dots \times v_m$$

式 33-1 準識別碼的組合總數

在均勻分布的理想假設下，平均每一種組合所對應的人數為：

$$\text{平均組大小 } \bar{k} = N / C$$

式 33-2 平均每組的人數

這個 $\bar{k}$ 就是 k-匿名的粗略估計。當它小於 1 時，代表平均而言每種組合對應不到一個人——資料實質上已不具匿名性。以台灣約 2,300 萬人為母體：

準識別碼組合	組合數 C	平均組大小	風險判讀
性別 × 縣市	44	約 522,727 人	極低
性別 × 縣市 × 出生年	4,400	約 5,227 人	低
性別 × 鄉鎮 × 出生年月	883,200	約 26 人	中等
性別 × 鄉鎮 × 出生年月日	26,864,000	約 0.9 人	極高（已可識別）

表 33-5 準識別碼的獨特性（母體 2,300 萬人）

看最後一列：只用三個看似無害的欄位，平均每組就只剩不到一人——這正是叫獸開講的計算。而第三列則顯示，只要把「出生年月日」概化為「出生年月」、把「鄉鎮」保持不變，平均組大小就從 0.9 人躍升到 26 人，重識別風險大幅降低。

這就是「概化」的威力：損失一部分精確度（少了「日」這個資訊），卻換來數十倍的隱私保護。

叫獸提醒

我必須誠實指出式 33-2 的三個侷限，否則你會過度依賴它。第一，它假設均勻分布，但真實人口不是——某些鄉鎮人口稀少，那裡的組合可能只有個位數，而平均值完全看不出來。所以實務上必須「實際計算每一組的大小」，找出最小的那一組，而非只看平均（這與第 3 章看 p99 而非平均、第 32 章看最弱維度而非綜合分數，是同一個統計直覺，本書已經是第三次強調了）。第二，它只考慮這一份資料——若同一個人的資料也出現在別處，交叉比對可能突破保護。第三，母體 N 的認定會影響結果：若這份資料只涵蓋某公司的員工而非全台灣人，那麼實際的 N 小得多，風險也高得多。做隱私評估時，這三點都必須明確交代。

案例研討

案例一 拿掉姓名的健康資料

某研究團隊取得一份健康檢查資料用於分析，資料提供方已移除姓名與身分證字號，並認為這樣就完成了匿名化。

但該資料仍保留了性別、出生年月日與居住鄉鎮。依式 33-1 與式 33-2 計算，這三個欄位的組合數已超過母體人數，平均每組不到一人——換言之，任何知道某人這三項基本資訊的人，都可能從中找出該人的健康檢查結果。

團隊在評估後，將出生年月日概化為出生年、鄉鎮概化為縣市，平均組大小提升至數千人，才開始使用。這個案例是本章最核心的一課：移除直接識別碼只是第一步，真正的風險在準識別碼的組合。

案例二 離職三年還在的帳號

某企業進行資安稽核時，盤點了資料平台的所有存取權限，結果發現有超過三十個帳號屬於已離職或已轉調的人員，其中最久的一個已離職三年多，而該帳號仍具有多個敏感資料集的讀取權限。

這些帳號從未被使用過任何惡意行為——但風險在於：只要其中任何一組憑證外流，攻擊者就能合法地讀取大量資料，且不會觸發任何異常告警（因為那看起來就是正常的授權存取）。

他們後續導入了兩項機制：權限設定到期日、以及每季由資料擁有者（第 32 章）確認權限清單。這正是 33.2.3 節叫獸提醒所談的「殭屍權限」——它不需要高深的技術就能造成嚴重後果，也不需要高深的技術就能防範。

案例三 設定成公開的儲存桶

某團隊為了方便測試，把一個雲端儲存桶的權限暫時設為公開存取，打算測完就改回來。測試結束後，這件事被遺忘了。

三個月後，該儲存桶中的資料被外部發現。整個過程中，雲端供應商的系統完全正常運作——沒有任何漏洞被利用，加密功能也都可用（只是沒有啟用）。

這正是 33.5.3 節「責任共擔」的典型案列：供應商負責雲本身的安全，而「設定得對不對」是使用者的責任。業界統計中，這類「設定錯誤」造成的雲端資料外洩，遠多於供應商本身被入侵。他們後來導入了自動掃描機制，定期檢查是否有非預期的公開設定——把「靠人記得」改成「靠系統檢查」。

案例四 技術上可行的需求

某位資料工程師接到一項需求：分析員工的網路使用紀錄，找出「工作效率較低」的人員。技術上完全可行——資料都在、模型也不難。

但他提出了幾點疑慮：這些紀錄當初蒐集的目的是網路安全維護，用於績效評估已超出原始目的（33.5.1 節的目的限制原則）；員工並不知道這些資料會被如此使用（33.6.2 節的「當事人會如何看待」）；而「網路使用時間」與「工作效率」之間的關聯，在方法上也相當可疑（第 1 章的關聯不等於因果）。

他沒有直接拒絕，而是把這些疑慮以書面形式提交，請決策者在知情的情況下判斷。最終該需求被調整為「匯總層級的部門統計」，不再針對個人。

這個案例呼應 33.6.2 節的叫獸提醒——技術人員往往是唯一能事前看見問題的人，而「把疑慮明確說出來並記錄下來」，是專業能力所帶來的責任。

實作活動

活動一 評估一份資料的重識別風險

請針對一份含個人資料的資料集（可自行產生模擬資料），完成重識別風險評估。

- 步驟 1.** 列出所有欄位，並依 33.4.1 節分類為直接識別碼、準識別碼與敏感屬性三類。
- 步驟 2.** 依式 33-1 計算準識別碼的組合總數，並依式 33-2 估算平均組大小。
- 步驟 3.** 實際計算資料中每一組準識別碼組合的人數，找出「最小的那一組」有幾人——比較它與平均值的差距。
- 步驟 4.** 設計一套概化方案（例如把生日改為年、鄉鎮改為縣市），使最小組大小達到 $k = 10$ 以上，並說明損失了哪些資訊。

活動二 設計最小權限的授權方案

請為一個資料平台設計權限方案，實踐最小權限原則。

- 步驟 1.** 設定情境：一個資料平台含三類資料（公開統計、含個資的明細、營業機密），以及四種角色（分析師、資料工程師、外部合作夥伴、稽核人員）。
- 步驟 2.** 為每一種角色與每一類資料的組合，明確標示應給予的權限（無、唯讀、可寫），並說明理由。
- 步驟 3.** 指出你的方案中哪些權限「看似方便但應該拒絕」，並說明拒絕的理由。
- 步驟 4.** 設計權限的生命週期管理：如何申請、多久複審、離職或轉調時如何處理。

活動三 撰寫一份倫理檢核

請為一個你正在進行或設想的資料專案，撰寫一份倫理檢核文件。依 33.6.2 節的五個問題逐一評估：當事人若知情會如何看待？每個蒐集的欄位是否真的必要？分析結果會被用來對誰做出什麼決定？是否可能對特定群體產生系統性的不利影響？受影響者能否知道原因並提出異議？請針對每一項

寫出你的評估與（若有）改善措施。最後說明：在這五個問題中，哪一個最讓你猶豫？你會如何處理這個猶豫？

延伸閱讀

- **個人資料保護法及其施行細則**：我國個資法的條文與主管機關的解釋函令，是 33.5 節的法源依據，建議至少通讀一次基本原則的部分。
- **NIST 去識別化指引**：美國國家標準與技術研究院關於去識別化技術與風險評估的報告，是 33.4 節的權威參考。
- **《隱私設計》相關文獻**：說明如何在系統設計階段即納入隱私考量（Privacy by Design）的原則與實務。
- **雲端供應商的責任共擔模型文件**：各家供應商都會明確列出責任分界，是 33.5.3 節的實務參考，導入服務前務必查閱。

本章小結

1. 資訊安全的三個核心目標為機密性、完整性與可用性；多數人只關注機密性，但被竄改的資料會導致錯誤決策、而無法存取的系統同樣造成業務停擺。
2. 大數據環境的特殊風險包括集中的高價值目標、多方存取、組合帶來的敏感性、長期保留與跨境流動；其中「組合」的風險最易被忽略——個別合規的資料集，組合後可能產生高度敏感的資訊。
3. 驗證回答「你是誰」、授權回答「你能做什麼」，兩者缺一不可；授權模型中以標籤為基礎的做法最適合資料平台，因其把分類（中繼資料）與權限控制連結起來。
4. 最小權限原則限制的是「出事時的損害範圍」；實務上最普遍的漏洞不是加密不足，而是「殭屍權限」——早該收回卻仍留存的授權，應以到期日與定期盤點來防範。
5. 加密分傳輸中與靜置時兩種狀態，但其強度最終取決於金鑰的保護；金鑰絕不可與資料放在一起或寫入程式碼，且加密對「有權限者濫用資料」完全無能為力。
6. 去識別化須區分直接識別碼、準識別碼與敏感屬性三類；重識別風險主要來自準識別碼的「組合」，故保護的重點在於以概化或抑制降低組合的獨特性。
7. k -匿名要求每組準識別碼組合至少對應 k 人，但有同質性、背景知識與跨資料集比對三個弱點；更嚴謹的方向是差分隱私，以加入計算過的雜訊換取數學上的保證。
8. 組合數為各準識別碼取值數的乘積（式 33-1），平均組大小為 N/C （式 33-2）；但此估計假設均勻分布，實務上必須計算「最小的那一組」而非只看平均——這與第 3 章的 p99、第 32 章的最弱維度是同一個統計直覺。
9. 個資保護的共通原則為目的限制、最小蒐集、當事人權利、安全維護義務與可歸責性；其中「目的限制」與資料湖「先收下再說」的哲學直接衝突，須在架構設計時即納入考量而非事後加裝。
10. 雲端安全採責任共擔：供應商負責雲本身，使用者負責雲中資料的設定與內容；多數雲端外洩源於使用者的設定錯誤，而非供應商被入侵。合法不等於正當，技術人員往往是唯一能事前看見倫理問題的人。

習題

一、觀念題

1. 請說明資訊安全的三個核心目標，並各舉一個在資料平台中違反該目標的實際後果。

2. 請區分「直接識別碼」「準識別碼」與「敏感屬性」，並說明為何重識別風險主要來自準識別碼。
3. 何謂「殭屍權限」？為何說它比高深的技術漏洞更值得優先處理？應如何防範？
4. 請說明雲端的「責任共擔」模型，並解釋為何多數雲端資料外洩源於使用者而非供應商。

二、計算題

1. 某資料集涵蓋一個約 50,000 人的母體，準識別碼包含：性別（2 種）、年齡層（以 5 歲為一組，共 16 組）、部門（12 個）、職級（8 級）。（一）依式 33-1 計算組合總數；（二）依式 33-2 計算平均組大小；（三）判斷此資料的重識別風險，並提出一項概化建議。
2. 承上題，若把「職級」概化為三個大類（基層、中階、高階）、並把「年齡層」改為 10 歲一組（共 8 組）。（一）重新計算組合數與平均組大小；（二）平均組大小提升了幾倍？（三）說明這樣的概化損失了什麼、換得了什麼。

三、思考與討論

1. 本章的叫獸提醒指出，式 33-2 的平均值可能掩蓋「最小的那一組」。請說明這個統計直覺，並列舉本書前面章節中至少兩個相同的例子。
2. 案例四中，工程師以書面提出疑慮而非直接拒絕。請討論：當你認為某項需求在倫理上有疑慮時，有哪些可行的處理方式？各自的優缺點為何？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

機密性——確保只有授權者能存取；違反的後果如敏感資料集被未授權人員讀取，造成個資外洩與法律責任。完整性——確保資料未被未授權竄改；違反的後果如品質檢測數據被惡意或意外修改，導致不良品被判為良品而流入市場，或稽核軌跡失效而無法追溯。可用性——確保授權者需要時能取得；違反的後果如遭勒索軟體加密或阻斷服務攻擊，即使資料未外洩，產線監控與報表全數停擺，業務因而中斷。可補充：可用性在第 3 章是從系統可靠度角度討論，在此則是從「可能被惡意破壞」的安全角度看待同一件事。

第 2 題

直接識別碼：本身即可唯一指認個人，如姓名、身分證字號、手機號碼。準識別碼：本身不足以識別，但組合起來可能指認出特定個人，如性別、出生日期、郵遞區號、職稱。敏感屬性：本身不識別身分，但一旦與個人連結即構成隱私侵害，如病歷、薪資、宗教信仰——這是真正要保護的內容。風險主要來自準識別碼的原因：直接識別碼容易辨認且通常在第一步就被移除，人們因而誤以為已完成匿名；但準識別碼看似無害而常被保留，其「組合」的獨特性卻可能超過母體人數（如表 33-5 的最後一列），使攻擊者得以把匿名紀錄對應到特定個人，進而得知其敏感屬性。

第 3 題

殭屍權限指已無業務需要卻仍留存於系統中的授權——例如離職者、轉調者或已結束專案的成員仍保有的存取權。優先處理的理由：其一，它不需要任何技術漏洞就能造成嚴重後果，只要憑證外流，攻擊者即可「合法地」讀取大量資料。其二，此類存取不會觸發異常告警，因為在系統看來那是正常

的授權行為，故極難偵測。其三，多數實際發生的資料外洩正是循此途徑，而非透過高深的技術攻擊。防範方式：為權限設定到期日（需要則重新申請）、由資料擁有者定期（如每季）盤點並確認權限清單、以及把人員異動流程與權限回收流程正式串接，而非仰賴個人記得。

第 4 題

責任共擔模型：供應商負責「雲本身的安全」——實體設施、底層硬體、虛擬化層、以及服務本身的可用性；使用者負責「雲中資料的安全」——存取控制設定、是否啟用加密、資料分類、應用程式安全與合規。分界隨服務模型而移動（IaaS 使用者責任最重、SaaS 最輕），但資料的內容與其合規性始終屬於使用者。多數外洩源於使用者的原因：供應商的基礎設施有專業團隊與龐大資源維護，被直接攻破的機率低；而使用者端的設定卻仰賴個別人員的謹慎，如把儲存桶設為公開存取（見案例三）、未啟用加密、或權限設定過寬。技術是中性的——供應商提供了工具，但「有沒有正確使用」是使用者的責任。

二、計算題

第 1 題

$N = 50,000$ ，準識別碼取值數為 2、16、12、8。

- (1) 組合總數（式 33-1） $= 2 \times 16 \times 12 \times 8 = 3,072$ 。
- (2) 平均組大小（式 33-2） $= 50,000 / 3,072 \approx 16.3$ 人。
- (3) 風險判讀：平均約 16 人一組，粗看尚可（相當於 $k \approx 16$ ）。但須注意實際分布並不均勻——某些部門人數少、某些職級人數少，這些交叉組合可能只有個位數甚至唯一。故應實際計算最小組大小。概化建議：可把職級合併為較大的類別、或把年齡層改為較粗的區間，以提高最小組的人數。

第 2 題

職級由 8 級概化為 3 類、年齡層由 16 組概化為 8 組。

- (1) 新組合數 $= 2 \times 8 \times 12 \times 3 = 576$ ；新平均組大小 $= 50,000 / 576 \approx 86.8$ 人。
- (2) 提升倍數 $= 86.8 / 16.3 \approx 5.3$ 倍（亦可由組合數之比 $3,072 / 576 = 5.33$ 直接得出）。
- (3) 損失：無法分析「各職級之間的細緻差異」（只能看基層、中階、高階三類），也無法做 5 歲一組的細緻年齡分析。換得：平均組大小從約 16 人提升至約 87 人，重識別風險大幅降低，且最小組過小的機率也隨之下降。

延伸思考：這正是隱私與資料效用的根本取舍——每一次概化都同時提升隱私與降低分析精細度。實務上的做法是先確認「分析真正需要多細的粒度」，再在滿足該需求的前提下盡量概化，而非預設保留最細的資料。此與 33.5.1 節的「最小蒐集」原則一致。

三、思考與討論

第 1 題

統計直覺：平均值是一種彙總，而彙總必然壓縮資訊；被壓縮掉的往往正是最需要關注的極端值。當分布不均勻時，平均值看似健康而少數極端情況卻可能極為嚴重，故評估風險時應看「最差的那一個」而非平均。本書中的相同例子至少有：其一，第 3 章的延遲——應看 p99 而非平均，因為少數極慢的請求會被大量正常請求稀釋，而重度使用者往往正落在尾端。其二，第 32 章的綜合品質分數——96.6% 的綜合分數掩蓋了時效性只有 92% 的最弱維度（案例三）。其三，第 31 章的冷啟動——1% 的冷啟動比例使平均延遲僅微幅上升，但那 1% 的使用者實際等待了數十倍的時間。其四，

第 26 章的資料完整性——平均完整性良好不代表某個特定時段沒有大量遺漏。共同結論：任何以單一數字概括群體的指標，都應搭配「最差值」或「分布」一起檢視。

第 2 題

可行的處理方式與其優缺點：其一，書面提出疑慮並請決策者判斷（案例四的做法）——優點是既盡到專業告知義務、留下紀錄，又不逾越自身職權，且往往能促成需求調整；缺點是最終決定權仍不在自己手上。其二，提出替代方案——例如把個人層級改為匯總層級、以較粗的粒度達成類似目的；優點是建設性高、較易被接受，缺點是需要投入額外的設計時間。其三，尋求內部的專業支援——如法務、資安或個資保護窗口，借助制度性的力量；優點是判斷更專業且責任分擔，缺點是流程較長。其四，直接拒絕執行——優點是立場明確，缺點是可能損害合作關係且未必能改變結果，通常應留給明確違法或造成重大傷害的情形。其五，向外部揭露——這是最後手段，代價極高，應僅在涉及重大公共利益且內部管道完全失效時考慮。核心原則：多數情況下，「把問題明確化並讓決策在知情下做成」既是最務實也最專業的做法——因為許多不當需求並非出於惡意，而是提出者不了解技術的能力邊界與後果。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 33 章 資料安全、隱私與法規遵循

台灣自編大學教科書

@prof

大數據分析與雲端運算

第捌篇 資料治理、安全與維運

Governance, Security & Operations

第 34 章

DataOps 與 MLOps

DataOps and MLOps

機器人叫獸（李世淵） 編著

叫獸開講

我想從一句學生常說的話開始：「教授，模型做好了，準確率有九成二。」

每次聽到這句話，我都會問同一個問題：「然後呢？」

多數學生會愣住，因為在課堂與競賽的世界裡，模型做出來、準確率算出來，事情就結束了。但在真實世界裡，那才是開始——你要把它部署到某個地方、要有人真的去用它、要監控它的表現、要處理它出錯時的狀況、要在資料改變時更新它。

而這裡有一件事，是課堂上幾乎不會教、但業界每天都在面對的：模型會「壞掉」。不是程式當掉那種壞，而是它靜靜地變差——今天九成二，半年後可能只剩八成五，而系統不會發出任何錯誤訊息。因為程式運作得完全正常，只是這個世界已經跟訓練時不一樣了。

這一章要談的 DataOps 與 MLOps，就是在回答「然後呢」這個問題。它談的不是如何做出更好的模型或更快的管線，而是如何讓它們「持續地、可靠地、可改善地運作下去」。這也是本書技術篇章的最後一章——因為所有前面談過的技術，最終都要落在這件事上。

學習目標

研讀完本章之後，你應該能夠：

1. 說明 DevOps 的核心理念，及其延伸至資料與機器學習領域的動機。
2. 說明 DataOps 的實務要素，並設計資料管線的持續整合流程。
3. 說明機器學習系統相較於一般軟體的額外複雜性。
4. 描述 MLOps 的完整生命週期與各階段的關鍵活動。
5. 區分資料漂移與概念漂移，並設計模型的監控機制。
6. 運用模型衰退模型判斷重新訓練的時機。

核心概念

核心概念	說明
DevOps	把開發與維運整合、以自動化與快速回饋縮短交付週期的實務。
DataOps	把 DevOps 的理念應用於資料管線，強調品質、自動化與協作。
MLOps	把機器學習模型從實驗帶到生產並持續維持其效能的實務。
特徵儲存	集中管理特徵定義與計算的系統，確保訓練與推論的一致。
資料漂移	輸入資料的分布隨時間改變。
概念漂移	輸入與輸出之間的關係本身改變了。
模型登錄	管理模型版本、來源與部署狀態的系統。

表 34-1 本章核心概念一覽

34.1 從 DevOps 到 DataOps

34.1.1 DevOps 要解決什麼

傳統的軟體開發中，開發團隊與維運團隊常常是分開的，且目標相互衝突——開發要「快點推出新功能」，維運要「維持穩定不出事」。結果是開發把程式丟給維運、出事時互相指責，而交付週期以月甚至季為單位。

DevOps 的核心理念是把兩者整合：同一個團隊對「從開發到運作」的整段負責，並以高度自動化縮短回饋週期。你在第 23 章已經見過它的具體工具——版本控制、自動化測試、持續整合與持續部署。

它帶來的關鍵轉變是「小批量、高頻率」：與其累積三個月的變更一次上線（風險巨大、出事難查），不如每天上線數次小的變更（風險可控、問題容易定位）。這個思路，你會發現與第 13 章「及早發現問題」是同一個邏輯。

34.1.2 資料領域的特殊性

把 DevOps 的理念搬到資料領域時，會遇到一個根本的差異：軟體的行為由程式碼決定，而資料管線的行為由「程式碼加上流入的資料」共同決定。

這意味著什麼？意味著即使你的程式碼一行都沒改，管線的輸出仍可能出錯——因為上游的資料變了。這是一般軟體工程不會遇到的問題，也是為什麼資料領域需要一套額外的實務。

因此 DataOps 除了繼承 DevOps 的自動化與快速回饋，還額外強調兩件事：其一是「資料的測試」——不只測程式碼，還要測流過的資料（第 13、32 章的品質檢核）；其二是「可觀測性」——因為問題可能來自任何一個環節，必須能看見資料在管線中的流動狀態。

34.2 DataOps 的實務要素

34.2.1 五個要素

DataOps 沒有標準的定義，但實務上大致包含五個要素——你會發現它們幾乎都在前面章節出現過，這裡是把它們整合起來。

要素	內容	對應章節
版本控制一切	程式碼、設定、綱要定義、甚至管線的 DAG 定義	第 23 章
自動化測試	程式碼的單元測試，加上資料的品質檢核	第 13、23、32 章
持續整合與部署	變更自動測試、通過後自動部署	第 23 章
環境一致性	以容器確保開發、測試、正式環境相同	第 28 章
監控與可觀測性	管線狀態、資料量、品質指標、資料血緣	第 13、32 章

表 34-2 DataOps 的五個實務要素

我要強調第一項「版本控制一切」中的「一切」。多數團隊會把程式碼放進版本控制，卻把管線設定、綱要定義、甚至排程時間留在某個網頁介面上手動修改——結果是沒有人知道「這個設定為什

麼是這樣、誰改的、什麼時候改的」。把設定也納入版本控制（常稱為「以程式碼定義基礎設施」），能讓所有變更都有紀錄、可審查、可回退。

34.2.2 資料管線的測試

資料管線需要的測試，比一般軟體多一個層次。

- **單元測試**：測試個別的轉換函式：給定輸入，是否產生預期的輸出。這與一般軟體測試相同。
- **整合測試**：以小量的樣本資料跑過整條管線，確認各環節能正確銜接。
- **資料測試**：對實際流過的資料進行品質檢核（第 32 章的六個維度）。這是資料領域特有的。
- **回歸測試**：確認變更後，既有的輸出結果沒有意外改變——尤其在重構管線時至關重要。

其中「回歸測試」最容易被忽略。當你優化一段轉換邏輯時，很容易在不經意間改變了輸出（例如處理空值的方式變了），而這種改變往往要到報表數字對不上時才被發現。做法是保留一組固定的測試資料與其期望輸出，每次變更後自動比對。

34.3 機器學習系統的額外複雜性

34.3.1 三個維度的變動

如果說資料管線的行為由「程式碼加資料」決定，那麼機器學習系統又多了一層——它由「程式碼、資料、模型」三者共同決定。

這帶來一個實務上的困難：當系統的表現變差時，原因可能出在任何一個維度，甚至是它們的交互作用。程式碼有臭蟲？訓練資料有問題？模型本身不適合？還是這個世界變了？

而更麻煩的是，這三者的變動速度與週期都不同：程式碼由開發者主動修改、資料持續流入且不受控、模型則需要重新訓練才會改變。要讓三者保持協調，比一般軟體困難得多。

34.3.2 訓練與推論的落差

機器學習系統有一類特別隱蔽的錯誤，值得專門討論——「訓練與推論的不一致」。

典型的情境是：訓練時你用 Python 的某個函式庫做特徵計算，而線上推論時，工程師用另一種語言重新實作了同樣的邏輯。兩份實作看起來一樣，但在邊界情況上可能有微妙差異（如空值處理、四捨五入、時區）。結果是模型在線上的表現莫名地比測試時差。

這類問題極難察覺，因為兩邊都「看起來正確」。解方是「特徵儲存」——把特徵的定義與計算邏輯集中管理，訓練與推論都從同一處取得，從根本上消除不一致的可能。

你會發現這與第 26 章 Lambda 架構「同一邏輯寫兩次」的問題完全同構——只要同一份邏輯存在兩份實作，它們遲早會分歧。這是軟體工程的一條通則。

34.3.3 模型是會過期的資產

最後一個關鍵差異：一般軟體只要環境不變就能一直正確運作，但機器學習模型會隨時間「衰退」。

原因在於模型學到的是「訓練資料所反映的那個世界」，而真實世界會變——消費者偏好改變、產線導入新設備、原料供應商更換、季節轉換。當現實與訓練時的分布漸行漸遠，模型的預測就越來越不準。

這個衰退是「安靜的」——系統不會報錯，指標只是緩慢地下滑。這正是第 13 章談過的「安靜的失敗」在機器學習領域的形態，也是為什麼 MLOps 必須包含持續的監控與重新訓練。

34.4 MLOps 的生命週期

34.4.1 六個階段

MLOps 把機器學習視為一個持續的循環，而非一次性的專案。

階段	關鍵活動	常見的疏漏
資料準備	蒐集、清理、標註、版本化訓練資料	未記錄資料版本，日後無法重現
實驗與訓練	特徵工程、模型選擇、超參數調校	未記錄實驗參數與結果
評估與驗證	以測試集評估，並檢查各族群的表現	只看整體指標，忽略特定群體
部署	打包、上線、設定推論服務	訓練與推論的特徵計算不一致
監控	追蹤預測品質、資料漂移、系統效能	只監控系統指標，未監控模型品質
重新訓練	依監控結果決定何時、用什麼資料重訓	憑感覺而非依據指標

表 34-3 MLOps 的六個階段

表中「常見的疏漏」那一欄，是我從學生專題與業界案例中反覆看到的問題。它們的共同點是：在實驗階段都不會造成困擾，卻在系統實際運作後成為難以收拾的麻煩。

34.4.2 實驗追蹤與模型登錄

兩個支撐整個生命週期的基礎設施，值得特別說明。

實驗追蹤解決的是「我上週那個效果最好的設定是什麼」這個問題。它自動記錄每次訓練的資料版本、程式碼版本、超參數、評估指標與產出的模型，使實驗可比較、可重現——這正是第 23 章可重現性四條件在機器學習領域的落實。

模型登錄則管理模型的「生命週期狀態」：哪些版本存在、各自的來源與評估結果、目前哪一版在正式環境服務、以及需要回退時該用哪一版。它讓模型從「某個資料夾裡的檔案」，變成一個有版本、有狀態、有稽核軌跡的正式資產。

這兩者合起來，回答了一個關鍵問題：當線上模型出問題時，你能不能在十分鐘內查出「這個模型是用哪份資料、哪份程式碼、什麼參數訓練出來的」？若答案是否定的，那麼你的 MLOps 基礎還沒建立。

34.4.3 部署的策略

模型上線不應該是「換掉舊的、開始用新的」這麼粗暴。常見的漸進策略有幾種：

- **影子模式**：新模型與舊模型同時運行，但只有舊模型的結果被實際採用；新模型的預測僅記錄下來供比較。零風險地驗證新模型的實際表現。
- **金絲雀部署**：先把少量流量（如 5%）導向新模型，確認無異常後逐步提高比例。
- **A/B 測試**：把流量分為兩組分別使用新舊模型，比較實際的業務指標而非只看離線評估（呼應第 19 章離線指標的侷限）。

這些策略的共同精神，與第 29 章的滾動更新相同——不要一次全部替換，而是逐步驗證。而它們也都預設了一個能力：能夠快速回退到舊版本。這正是模型登錄存在的理由。

34.5 監控、漂移與重新訓練

34.5.1 兩種漂移

模型衰退的原因可分為兩類，它們的性質與應對方式不同。

類型	定義	範例
資料漂移	輸入資料的分布改變，但輸入與輸出的關係不變	產線換了新設備，感測訊號的數值範圍整體偏移
概念漂移	輸入與輸出之間的關係本身改變了	疫情改變了消費行為，原本的購買預測邏輯不再成立

表 34-4 資料漂移與概念漂移

兩者的關鍵差異在於「偵測的難易」。資料漂移可以在「不知道正確答案」的情況下偵測——你只需比較當前輸入的分布與訓練時的分布是否有顯著差異。而概念漂移則必須知道真實結果才能發現，因為問題不在輸入而在關係。

這帶來一個實務上的困難：許多場景中，真實結果要很久之後才知道（例如預測設備會不會在三個月後故障）。在這段等待期間，你只能靠資料漂移的偵測作為間接的預警訊號。

34.5.2 該監控什麼

- **系統指標**：延遲、吞吐量、錯誤率——這與一般服務的監控相同（第 3、29 章）。
- **資料品質**：輸入資料是否完整、格式是否正確、有無異常值（第 32 章的六個維度）。
- **資料漂移**：各特徵的分布是否偏離訓練時的基準。
- **預測分布**：模型輸出的分布是否改變——例如原本 5% 判為異常，突然變成 30%。
- **模型品質**：在有真實結果的樣本上，實際的準確率是否下滑。這是最直接的指標，但往往有延遲。

多數團隊只做到前兩項——因為它們與一般的系統監控相同，工具現成。但真正決定模型是否還堪用的是後三項，而它們需要刻意設計。這也呼應第 32 章那句話：能被量測的，才能被管理。

34.5.3 何時該重新訓練

重新訓練的觸發策略有三種，各有適用。

定期重訓是最簡單的：每週或每月固定重訓一次。優點是簡單可預期，缺點是可能過於頻繁（浪費資源）或太不頻繁（模型已經失效卻還沒到時間）。

依指標觸發則是當監控指標跌破門檻時才重訓。這在資源使用上最有效率，但需要有可靠的監控指標——而如前所述，模型品質指標往往有延遲。

依事件觸發則是當已知的重大變化發生時（如產線導入新設備、業務規則調整）主動重訓。這需要業務知識，無法自動化，但往往最為及時。

實務上通常混用：以定期重訓為基線，加上指標觸發作為安全網，並保留依事件手動觸發的能力。下一節的公式會示範如何估算合理的重訓週期。

34.6 組織、文化與成熟度

34.6.1 技術之外的部分

談了這麼多技術實務，我要用一節談談技術之外的東西——因為 DataOps 與 MLOps 的失敗，多數不是技術原因。

最常見的失敗模式是「交接的斷層」：資料科學家做出模型後交給工程團隊部署，但模型的假設、限制與注意事項並未完整傳遞；工程團隊也不清楚該監控什麼。結果是模型上線後無人真正負責——這正是第 32 章談的「出事無人負責」在機器學習領域的版本。

有效的做法是讓同一個團隊對「從實驗到運作」的整段負責，或至少建立明確的交接標準（模型卡、監控項目、重訓程序）。這與 34.1.1 節 DevOps 的核心理念完全相同——把分離的職能整合起來。

34.6.2 成熟度的階梯

組織導入 MLOps 通常會經歷幾個階段，認清自己在哪一階，有助於規劃下一步。

階段	特徵	主要瓶頸
手工階段	實驗在筆記本中進行，部署靠人工複製檔案	無法重現、上線緩慢
自動化訓練	訓練流程已自動化，但部署與監控仍靠人工	模型衰退無法及時發現
自動化部署	訓練、測試、部署已串成管線	重訓時機仍靠人判斷
持續學習	監控觸發自動重訓與部署，形成閉環	需要成熟的監控與嚴謹的驗證關卡

表 34-5 MLOps 的成熟度階梯

我要提醒的是：不必以最高階為目標。多數組織在「自動化部署」這一階就能得到絕大部分的效益，而「持續學習」的閉環需要極高的監控成熟度——否則自動重訓可能把一個有問題的模型自動推上線，反而更危險。

叫獸提醒

我想在本書技術篇的最後，說一件我認為最重要的事。這一章談的所有實務——版本控制、自動化測試、監控、可重現性——都有一個共同特徵：它們在專案初期看起來都是「浪費時間」。趕著做出模型時，誰想花時間寫測試？誰想記錄實驗參數？但我教書這些年看到的規律是：那些最後真正做出成果的學生與團隊，幾乎都是在這些「基本功」上紮實的人。原因不難理解——因為做研究與做產品，本質上都是一連串的嘗試與修正，而基本功決定了你「每一次嘗試的成本」。基本功差的人，每次修改都要花半天釐清狀況；基本功好的人，改一行程式碼、跑一次測試、五分鐘就知道結果。長期下來，兩者的產出會有數量級的差距。這不是關於勤奮，而是關於效率的複利。

34.6.3 承先啟後

本章我們回答了叫獸開講那個「然後呢」的問題。從 DevOps 的整合理念與資料領域的特殊性出發（34.1），整理了 DataOps 的五個實務要素與四層測試（34.2）、機器學習系統在程式碼與資料

之外多出的模型維度及其三個難題（34.3）、MLOps 的六階段生命週期與部署策略（34.4）、兩種漂移與監控重訓的判斷（34.5），最後談了組織文化與成熟度階梯（34.6）。

到這裡，第捌篇「資料治理、安全與維運」三章完整收束，而本書的技術篇章也告一段落。回顧這一篇：第 32 章談的是「誰負責、能不能信」，第 33 章談的是「誰能存取、如何保護」，而本章談的是「如何持續運作下去」。這三件事合起來，才使得前面七篇的技術真正成為一個「可信賴的系統」，而非一堆能跑的程式。

接下來的第玖篇，我們要把整本書學到的東西「用出來」。從資料分析的完整流程與視覺化開始，到邊緣運算與物聯網、機器人與智慧製造的整合應用，最後以一個完整的專案實作作結。理論到此為止，接下來是實踐。

公式與流程：模型衰退與重訓週期

這一節要把「什麼時候該重新訓練」這個常憑感覺決定的問題，變成一個可以計算的判斷。

線性衰退模型

設模型剛上線時的準確率為 A_0 ，每經過一個月因漂移而下降的幅度為 d （衰退率），則第 t 個月的準確率可估計為：

$$A(t) = A_0 - d \times t$$

式 34-1 模型的線性衰退估計

設業務可接受的最低準確率門檻為 $A_{\min}$ ，則模型跌破門檻所需的月數為：

$$T = (A_0 - A_{\min}) / d$$

式 34-2 跌破門檻的時間

以初始準確率 92%、每月衰退 0.8 個百分點、門檻 85% 為例： $T = (0.92 - 0.85) / 0.008 = 8.75$ ，即約第 9 個月會跌破門檻。

月數 t	估計準確率	距門檻	建議
0	92.0%	+7.0	剛上線
3	89.6%	+4.6	正常監控
6	87.2%	+2.2	開始準備重訓資料
8	85.6%	+0.6	應已完成重訓
9	84.8%	-0.2	已跌破門檻

表 34-6 模型衰退的推估（ $A_0 = 92\%$ 、 $d = 0.8\%/月$ 、門檻 85%）

這個推估的用途不是「等到第 9 個月才行動」，而是「反推出應該在何時開始準備」。因為重新訓練本身需要時間——蒐集新資料、標註、訓練、驗證、部署，可能耗時數週。

考慮重訓前置時間

設完成一次重新訓練並部署所需的時間為 L （前置時間），則啟動重訓的時機應為：

$$\text{啟動時機} = T - L$$

式 34-3 考慮前置時間的重訓啟動點

承上例，若重訓的完整流程需要 2 個月，則應在第 6.75 個月（約第 7 個月）就啟動，而非等到第 9 個月才開始。這也解釋了表 34-6 中「第 6 個月開始準備重訓資料」的建議。

叫獸提醒

我要對式 34-1 的線性假設提出兩點誠實的提醒。第一，真實的衰退很少是線性的——它可能長期平穩然後突然劇降（例如某個外部事件改變了整個市場），也可能有季節性的起伏。所以這個模型的價值不在於「精確預測第 9 個月」，而在於「建立一個可以持續修正的預期」：實際量測、與預期比對、發現偏離就檢討原因。第二，也更重要的——你必須真的有在量測。我看過不少團隊部署模型後就不再檢視其表現，直到使用者抱怨才發現問題。而諷刺的是，這類團隊往往在模型訓練階段投入了大量心力調整超參數以提升那零點幾個百分點，卻任由模型上線後靜靜地掉了七個百分點。優化的心力，應該分配在真正影響結果的地方。

案例研討

案例一 然後呢

某學生的專題做了一個設備異常預測模型，測試準確率達九成三，成果發表相當精彩。指導教授問：「如果工廠要真的用它，接下來要做什么？」

學生答不出來。追問之下發現：訓練資料沒有版本標記、實驗參數只記在筆記本裡、特徵計算的程式碼與訓練腳本混在一起、也沒有想過模型上線後要監控什麼。這個模型技術上有效，但實務上無法被交付。

他後來補上了四件事：把資料版本化、以實驗追蹤記錄所有訓練、把特徵計算獨立成可重用的模組、並定義了三項上線後的監控指標。這些工作沒有讓準確率提高哪怕零點一個百分點——但它讓這個模型從「一份報告」變成了「一個可交付的系統」。這正是叫獸開講那個「然後呢」的完整答案。

案例二 線上比測試差了五個百分點

某團隊的推薦模型在離線測試中準確率良好，但上線後的實際表現持續低於預期約五個百分點。他們反覆檢查模型與訓練資料，都找不出問題。

最後查出原因：訓練時的特徵計算以 Python 完成，其中某個時間差的特徵使用了「自然日」的定義；而線上推論的服務以另一種語言重新實作，該處使用的是「二十四小時」的定義。兩者在多數情況下相同，但在跨日的邊界會有差異——而這類樣本恰好佔了相當比例。

這正是 34.3.2 節的「訓練與推論落差」。他們導入特徵儲存後，兩邊改為從同一處取得特徵定義，問題消失。這個案例的教訓與第 26 章 Lambda 架構的「兩套邏輯分歧」完全相同——同一份邏輯只要存在兩份實作，遲早會分歧。

案例三 安靜衰退的品質預測

某工廠部署了一個產品品質預測模型，上線初期表現良好。一年多後，品管人員發現模型的預測「好像越來越不準」，但說不出具體從何時開始。

回頭分析歷史紀錄後發現，模型的準確率從上線時的 91% 緩慢下滑到 82%——而這段期間，產線更換過原料供應商、也導入了一台新的加工設備。系統從未報錯，因為程式運作完全正常，只是這個世界已經跟訓練時不同了。

這是典型的資料漂移加上概念漂移。他們後來建立了兩項機制：以式 34-1 建立衰退的預期曲線並每月比對實際表現、以及在「已知的重大變更」（換供應商、換設備）發生時主動觸發重訓。這個案例是 34.3.3 節「安靜衰退」最直接的體現，也呼應第 13 章的「安靜的失敗」。

案例四 自動重訓推上線的壞模型

某團隊建立了完整的自動化閉環：監控指標下滑時自動觸發重訓，訓練完成後自動部署。他們以為這代表了最高的成熟度。

某次上游資料源出現異常（一個欄位大量變成空值），觸發了指標下滑，系統自動以這批有問題的資料重新訓練，並自動把新模型推上線——結果模型品質不升反降，且因為是自動部署，數天後才被人發現。

問題不在於自動化本身，而在於自動化的路徑上缺少了驗證關卡：重訓前未檢查訓練資料的品質（第 32 章）、部署前未以獨立的測試集驗證新模型是否真的優於舊模型。

他們後來在流程中加入了兩道關卡：資料品質未通過則不啟動重訓、新模型未明顯優於現行模型則不自動部署。這呼應表 34-5 的提醒——「持續學習」的閉環需要極高的監控成熟度，否則自動化反而放大了風險。

實作活動

活動一 為模型建立實驗追蹤

請為一個機器學習專案補上完整的實驗追蹤。

- 步驟 1.** 選定一個你做過的模型專案，記錄目前有哪些資訊是「沒有被記錄下來」的（資料版本、參數、指標、程式碼版本）。
- 步驟 2.** 導入實驗追蹤工具（或自行以結構化的方式記錄），重跑至少五組不同參數的實驗。
- 步驟 3.** 驗證可重現性：隨機挑選其中一組，僅依紀錄嘗試完全重現該次結果。
- 步驟 4.** 說明你在重現過程中遇到的障礙，並據此補強你的紀錄項目。

活動二 偵測資料漂移

請實作一個簡易的漂移偵測機制。

- 步驟 1.** 取得或產生一份資料集，切分為「訓練期」與「後續各月」兩部分。
- 步驟 2.** 為訓練期的每個數值特徵計算基準統計量（平均、標準差、分位數）。
- 步驟 3.** 對後續各月的資料計算相同的統計量，並與基準比較，找出偏離最大的特徵。
- 步驟 4.** 設定一個合理的告警門檻（如平均值偏離超過兩個標準差），說明你的設定依據，並討論門檻過鬆或過緊各會有什麼問題。

活動三 規劃模型的重訓策略

請為一個實際或設想的模型，規劃完整的重訓策略。首先估計其初始準確率、可接受的門檻與每月的衰退率（可依領域特性合理假設並說明理由）。依式 34-2 計算跌破門檻的時間，再依式 34-3 考慮重訓的前置時間（蒐集資料、標註、訓練、驗證、部署各需多久），推算應在何時啟動重訓。接著設計三種觸發條件：定期、指標觸發、事件觸發，並各自說明門檻與判斷依據。最後回答：若要走向自動重訓的閉環，你會在流程中設置哪些驗證關卡以避免案例四的問題？

延伸閱讀

- 《Machine Learning Engineering》或《Designing Machine Learning Systems》：系統性介紹機器學習系統從設計到維運的完整實務，是本章最完整的延伸。
- Google 的〈機器學習系統中的隱藏技術債〉論文：闡述機器學習系統為何比一般軟體更容易累積技術債，是 34.3 節的思想源頭。
- MLOps 成熟度模型的相關文件：各大雲端供應商都有發布成熟度分級與導入建議，可對照表 34-5 使用。
- 《The DevOps Handbook》：說明 DevOps 的原則、實務與組織轉型，是 34.1、34.6 節的基礎讀物。

本章小結

1. DevOps 把開發與維運整合、以自動化與高頻小批量的交付縮短回饋週期；把它延伸到資料領域時遇到一個根本差異——資料管線的行為由「程式碼加上流入的資料」共同決定，故即使程式碼未改，輸出仍可能出錯。
2. DataOps 的五個實務要素為版本控制一切（含設定與綱要）、自動化測試、持續整合部署、環境一致性與監控可觀測性；其中「一切」應包含管線設定，使所有變更皆有紀錄可審查與回退。
3. 資料管線需要四層測試：單元、整合、資料品質與回歸測試；其中回歸測試最易被忽略，卻是防止重構時意外改變輸出的關鍵。
4. 機器學習系統由程式碼、資料、模型三者共同決定行為，且三者的變動速度不同，故問題定位比一般軟體困難得多。
5. 「訓練與推論的落差」是特別隱蔽的錯誤來源——同一份特徵邏輯的兩份實作遲早會分歧；解方是特徵儲存，使兩端從同一處取得定義。此與第 26 章 Lambda 架構的問題完全同構。
6. 模型是會過期的資產：它學到的是訓練當時的世界，而世界會變；這個衰退是安靜的，系統不會報錯，指標只是緩慢下滑。
7. MLOps 生命週期含資料準備、實驗訓練、評估驗證、部署、監控與重訓六階段；實驗追蹤與模型登錄是支撐整個循環的基礎設施，其檢驗標準是「能否在十分鐘內查出線上模型的來源」。
8. 部署應採漸進策略（影子模式、金絲雀、A/B 測試），其共同精神與第 29 章的滾動更新相同——逐步驗證並保留快速回退的能力。
9. 資料漂移是輸入分布改變（可在不知答案時偵測），概念漂移是輸入與輸出的關係改變（須知道真實結果才能發現）；監控應涵蓋系統指標、資料品質、資料漂移、預測分布與模型品質五個層次，而多數團隊只做到前兩項。
10. 模型衰退可估計為 $A(t) = A_0 - d \times t$ （式 34-1），跌破門檻的時間為 $(A_0 - A_{\min})/d$ （式 34-2），而啟動重訓的時機應扣除前置時間（式 34-3）；此估計的價值不在精確預測，而在建立可持續修正的預期——但前提是你真的有在量測。

習題

一、觀念題

1. 為什麼把 DevOps 的理念應用到資料領域時，需要額外的實務？請說明資料管線與一般軟體的根本差異。
2. 何謂「訓練與推論的落差」？它為何特別難以察覺？應如何從根本上避免？
3. 請區分資料漂移與概念漂移，並說明為何兩者的偵測難度不同。

4. 案例四中，自動重訓反而推上了一個更差的模型。請說明問題出在哪裡，以及自動化閉環應具備哪些前提。

二、計算題

1. 某模型上線時準確率為 88%，實測每月因漂移下降 1.2 個百分點，業務可接受的門檻為 80%。（一）依式 34-2 計算跌破門檻所需的月數；（二）計算第 4 個月與第 6 個月的估計準確率；（三）若完整的重訓流程需 1.5 個月，依式 34-3 應在第幾個月啟動重訓？
2. 承上題，若透過改善特徵設計，使每月衰退率降為 0.6 個百分點。（一）重新計算跌破門檻的月數；（二）在模型的一年壽命內，兩種情況各需重訓幾次？（三）說明降低衰退率的價值。

三、思考與討論

1. 本章的叫獸提醒指出，許多團隊在訓練階段拼命調參數提升零點幾個百分點，卻任由模型上線後掉了好幾個百分點。請討論：為什麼會出現這種資源錯置？組織該如何調整？
2. 本章是本書技術篇章的最後一章。請回顧全書，列出你認為「跨越最多章節、最值得記住」的三個核心觀念，並說明它們分別在哪些章節出現過。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

根本差異：一般軟體的行為完全由程式碼決定，只要程式碼與環境不變，輸出就穩定；而資料管線的行為由「程式碼加上流入的資料」共同決定，故即使程式碼一行未改，上游資料的改變（欄位變更、單位變更、品質下降）仍會導致輸出錯誤。因此需要的額外實務有二：其一，「資料的測試」——不只測程式碼邏輯，還要對實際流過的資料進行品質檢核（第 13、32 章）；其二，「可觀測性」——因為問題可能來自任何環節，必須能看見資料在管線中的流動狀態與各節點的品質指標，並輔以資料血緣以便追溯。

第 2 題

定義：訓練時的特徵計算邏輯與線上推論時的實作不一致，導致模型在線上的表現低於離線評估。難以察覺的原因：兩份實作「看起來都正確」，各自單獨測試也都能通過；差異往往只出現在邊界情況（空值處理、四捨五入、時區、跨日定義等），且不會產生任何錯誤訊息——系統運作正常，只是結果略差（如案例二的五個百分點）。從根本避免的方法：導入「特徵儲存」，把特徵的定義與計算邏輯集中管理，訓練與推論都從同一處取得，使不一致在結構上不可能發生。可補充：此問題與第 26 章 Lambda 架構「同一邏輯寫兩次」完全同構——只要同一份邏輯存在兩份實作，它們遲早會分歧，這是軟體工程的通則。

第 3 題

資料漂移：輸入資料的分布改變，但輸入與輸出之間的關係不變，例如產線換新設備使感測訊號的數值範圍整體偏移。概念漂移：輸入與輸出之間的「關係」本身改變了，例如消費行為改變使原本的購買預測邏輯不再成立。偵測難度不同的原因：資料漂移可在「不知道正確答案」的情況下偵測——只需比較當前輸入的統計分布與訓練時的基準是否有顯著差異，隨時可做；而概念漂移必須知道真實結果才能發現，因為問題不在輸入的樣貌而在其與結果的對應關係。實務困難在於，許多場景的

真實結果要很久之後才揭曉（如預測三個月後的故障），故在等待期間只能以資料漂移作為間接的預警訊號。

第 4 題

問題所在：自動化的路徑上缺少驗證關卡。上游資料異常導致指標下滑，系統據此自動以「有問題的資料」重新訓練，且未經比較就自動部署，使更差的模型上線且數天後才被發現。問題不在自動化本身，而在於自動化把一個原本會被人攔下的錯誤，快速且無阻礙地推到了正式環境。自動化閉環應具備的前提：其一，重訓前的資料品質關卡——訓練資料須通過品質檢核（第 32 章）才啟動重訓。其二，部署前的模型驗證關卡——新模型須在獨立的測試集上明顯優於現行模型才允許部署。其三，漸進部署與快速回退——採影子模式或金絲雀部署，並保有立即回退的能力。其四，充分的監控成熟度——如表 34-5 所提醒，「持續學習」需極高的監控水準，否則自動化反而放大風險。

二、計算題

第 1 題

$A_0 = 0.88$ 、 $d = 0.012$ / 月、 $A_{\min} = 0.80$ 。

- (1) 跌破門檻月數（式 34-2） $= (0.88 - 0.80) / 0.012 = 0.08 / 0.012 \approx 6.67$ 個月，即約第 7 個月跌破。
- (2) 第 4 個月（式 34-1）： $0.88 - 0.012 \times 4 = 0.832$ （83.2%）；第 6 個月： $0.88 - 0.012 \times 6 = 0.808$ （80.8%）。
- (3) 啟動重訓時機（式 34-3） $= 6.67 - 1.5 = 5.17$ ，即應在第 5 個月左右啟動重訓。

第 2 題

衰退率由 0.012 降為 0.006。

- (1) 新的跌破門檻月數 $= 0.08 / 0.006 \approx 13.3$ 個月。
- (2) 一年（12 個月）內：原本 $d = 0.012$ 時每 6.67 個月需重訓一次，故約需 2 次（第 6.67 與第 13.3 個月，其中第二次落在一年之後，故一年內為 1 至 2 次，視是否計入邊界）；改善後 13.3 個月才跌破，故一年內不需重訓。以「必須重訓的次數」計，前者約 1 至 2 次、後者 0 次。
- (3) 價值：重訓成本（資料蒐集、標註、訓練、驗證、部署的人力與運算）大幅降低；模型在門檻之上的時間更長，平均品質更高；且重訓頻率降低也減少了每次變更所帶來的風險。

延伸思考：這個結果凸顯了一個常被忽略的觀點——「降低衰退率」的投資報酬，可能高於「提升初始準確率」。若把初始準確率從 88% 提升到 90%（辛苦調參的成果），僅使跌破門檻的時間從 6.67 延長到 8.33 個月；但把衰退率減半，卻使它延長到 13.3 個月。降低衰退率的做法包括選用更穩定的特徵（避免容易隨時間變動者）、加入能反映變化的特徵、或改用對分布變化較不敏感的模式結構。

三、思考與討論

第 1 題

資源錯置的原因：其一，回饋週期不同——訓練階段的改善立即可見（跑一次就知道準確率上升了多少），而上線後的衰退緩慢且延遲，缺乏即時的成就感。其二，評量方式的引導——課堂、競賽與許多組織的績效評估都以「模型指標」為準，而非「系統長期表現」，故人們自然優化被評量的那一項（第 19 章談過的：當一個指標成為目標時，它就不再是好指標）。其三，技能與興趣的偏好——調參數屬於熟悉的技術工作，而監控與維運被視為「不有趣」的雜事。其四，可見性不對稱——

上線後沒出事時，維運的價值完全看不見（第 32 章談過的同一問題）。組織的調整方向：把「上線後的表現」納入評量（如模型在生產環境的平均品質、跌破門檻的次數）；讓同一團隊對從實驗到運作的整段負責，使長期後果與當事人相關；建立模型品質的常態儀表板使衰退可見；並在專案規劃時就把監控與重訓的工時明確編列，而非視為額外負擔。

第 2 題

可提出的核心觀念（舉三例並說明其跨章出現）：其一，「不可變的資料加上可切換的指標」——第 11 章 RDD 的不可變與 Lineage、第 25 章 Kafka 的追加式日誌與位移、第 28 章容器映像檔的唯讀層與寫時複製、第 30 章 Lakehouse 的中繼資料清單；它同時解決了容錯、並行安全與可回溯三個問題。其二，「冪等性與無狀態」——第 3 章的逾時重試、第 9 章 MapReduce 的可重跑任務、第 13 章管線的第一原則、第 25 章的至少一次語意、第 26 章的串流恢復、第 28、29、31 章的無狀態設計；它是所有重試、遷移與自動擴縮得以成立的前提。其三，「以可控的近似換取可行的計算」——第 14 章的 MinHash 與 LSH、第 15 章的剪枝、第 16 章的 BFR、第 17 章的隨機化分解、第 20 章的直方圖、第 24 章的整章串流演算法；核心是「聰明地不算」勝過「算得更快」。其四（亦可提出），「彙總會掩蓋極端」——第 3 章的 p99、第 31 章的冷啟動、第 32 章的最弱維度、第 33 章的最小組大小。周延的回答應說明「為何這些觀念會反覆出現」——因為它們回應的是分散式與大規模系統的結構性限制，而非特定技術的細節，故技術更迭而原理長存。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 34 章 DataOps 與 MLOps

台灣自編大學教科書

@prof

大數據分析與雲端運算

第玖篇 整合實務與應用

Integration & Applications

第 35 章

邊緣運算與物聯網

Edge Computing and the Internet of Things

機器人叫獸（李世淵） 編著

叫獸開講

我去參訪過一家工廠，他們剛裝了一百支高解析度攝影機做外觀檢測，計畫把影像全部傳到雲端做 AI 分析。

我問了一個問題：「你們算過頻寬嗎？」

現場算了一下：一百支攝影機、每支約 4 Mbps，合計 400 Mbps 持續上傳。他們的專線是 300 Mbps——連傳都傳不出去，更別說一個月要傳一百多 TB 的流量費用。

這不是他們規劃不周，而是一個很常見的認知落差：我們談了那麼多雲端的好處，卻很容易忘記一件事——資料要先「到得了」雲端，才談得上雲端運算。而在製造與機器人的場域中，資料的產生地離雲端往往很遠，中間隔著有限的頻寬、不穩的網路、以及毫秒級的時間要求。

邊緣運算的核心想法很簡單：既然搬資料這麼貴，那就把運算搬到資料旁邊。這句話你應該覺得眼熟——第 9 章談 MapReduce 的資料在地性時，我說過幾乎一模一樣的話。原理是相同的，只是尺度從「機房內的機架」放大到了「工廠到雲端」。

這一章開始，我們正式進入第玖篇的整合應用。前面八篇的技術，將在這裡與你們的專業領域交會。

學習目標

研讀完本章之後，你應該能夠：

1. 說明物聯網的架構層次，並辨識感測網路的特性與限制。
2. 比較常見邊緣裝置的定位，並依應用需求選擇合適的硬體。
3. 說明邊緣運算的動機，並判斷哪些運算應在邊緣、哪些應在雲端。
4. 說明邊緣推論的技術要點，包含模型壓縮與加速。
5. 描述邊雲協同的架構模式與資料分流策略。
6. 運用頻寬與延遲的估算，設計合理的邊緣架構。

核心概念

核心概念	說明
物聯網	以感測器與網路連結實體世界、蒐集並傳遞資料的系統。
邊緣運算	把運算配置在資料產生地附近，而非全部集中於雲端。
閘道器	匯集多個感測裝置、進行協定轉換與初步處理的中介設備。
邊緣推論	在邊緣裝置上執行模型推論，無須將原始資料上傳。
邊雲協同	邊緣與雲端依各自優勢分工的架構模式。
時間敏感	控制迴路對延遲有嚴格上限的特性。
資料分流	依價值與時效決定哪些資料上傳、哪些就地處理或丟棄。

表 35-1 本章核心概念一覽

35.1 物聯網與感測網路

35.1.1 四層架構

物聯網系統通常可分為四個層次，理解這個分層有助於定位各項技術的角色。

層次	內容	關鍵議題
感知層	感測器與致動器，實際與物理世界互動	精度、取樣率、校正、耗電
網路層	把資料從裝置傳到後端的通訊網路	頻寬、可靠度、功耗、覆蓋範圍
平台層	資料的接收、儲存、管理與裝置維護	第 25 章的訊息佇列、第 30 章的儲存
應用層	分析、告警、視覺化與決策支援	第肆篇的演算法、第 37 章的視覺化

表 35-2 物聯網的四層架構

你會發現，本書前面八篇談的內容，主要集中在平台層與應用層。而這一章要補上的，是前兩層——那是資料真正誕生的地方，也是多數大數據教材較少著墨的部分。

35.1.2 感測資料的特性

感測資料與一般的業務資料（如訂單、交易）有幾個結構性的差異，這些差異直接影響架構設計。

- **高頻且持續：**感測器可能以每秒數十到數千次的頻率產生資料，且永不停止——這正是第 2 章談的「無界資料」。
- **單筆價值低：**單獨一筆溫度讀數幾乎沒有意義，價值來自「趨勢」與「異常」，因此可以聚合、可以抽樣。
- **時序性強：**資料的意義高度依賴時間順序與間隔，這使它天然適合第 26 章的視窗與串流處理。
- **可能有雜訊與缺漏：**感測器會漂移、會故障、網路會斷；資料的完整性遠不如業務系統。
- **時間戳的難題：**裝置的時鐘可能不準或不同步，導致事件時間不可靠（第 26 章的事件時間問題在此更為棘手）。

其中「單筆價值低」這一點極為關鍵，它是邊緣運算得以成立的前提——正因為多數原始讀數不必保留，我們才能在邊緣先做聚合與篩選，只把有價值的部分上傳。

35.1.3 通訊協定的取捨

物聯網的通訊選擇眾多，但取捨的核心維度只有三個：頻寬、距離與功耗——而它們往往無法兼得。

近距離高頻寬的選擇（如有線網路、無線區域網路）適合傳輸影像等大量資料，但耗電且佈線受限；長距離低功耗的選擇（如低功耗廣域網路）能讓電池裝置運作數年，但頻寬極低，只夠傳送少量的數值。

在應用層的協定上，物聯網常用輕量的發布訂閱協定——它的設計思路與第 25 章的訊息佇列相同（發布者與訂閱者解耦），但針對頻寬受限與網路不穩的環境做了簡化，例如更小的封包標頭與可設定的傳遞保證等級。

你會注意到「可設定的傳遞保證等級」正對應第 25 章的三種語意（至多一次、至少一次、恰好一次）。在物聯網中，這個選擇更為重要——因為對電池裝置而言，重試的代價是電力，而電力往往比資料更珍貴。

35.2 邊緣裝置：樹莓派與 NVIDIA Jetson

35.2.1 三個層級的邊緣裝置

邊緣裝置涵蓋的範圍很廣，實務上大致可分為三個層級。

層級	代表	運算能力	典型用途
微控制器	ESP32 等單晶片	極低，僅能做簡單邏輯	感測、傳輸、簡易判斷
單板電腦	樹莓派	可執行完整作業系統	閘道器、資料匯集、輕量推論
邊緣運算平台	NVIDIA Jetson 系列	含 GPU，可執行深度學習	影像推論、機器人控制

表 35-3 三個層級的邊緣裝置

選擇的原則很直接：先確認「必須在本地完成的運算是什麼」，再選擇剛好足夠的硬體。過度配置會浪費成本與電力，配置不足則會使系統無法達成時間要求。

35.2.2 樹莓派的定位

樹莓派這類單板電腦，最適合的角色是「閘道器」與「輕量處理節點」。

作為閘道器，它負責匯集多個感測裝置的資料、進行協定轉換、做初步的過濾與聚合，再統一上傳。這個角色的價值在於：它讓數十個簡單的感測節點不必各自處理網路與安全問題，而由一個較有能力的節點統一處理。

它也能執行輕量的推論——例如簡單的異常判斷、或經過大幅壓縮的小型模型。但要注意其限制：沒有專用的加速硬體，執行較大的深度學習模型會非常吃力，且長時間高負載時的散熱與穩定性需要謹慎處理。

在教學與原型驗證上，樹莓派的價值極高——它便宜、生態成熟、且能執行完整的 Linux 環境，使你能直接套用本書前面談過的容器（第 28 章）、訊息佇列客戶端等技術。

35.2.3 Jetson 的定位

當應用需要在邊緣執行深度學習推論時——尤其是影像相關的任務——就需要 NVIDIA Jetson 這類具備 GPU 的邊緣運算平台。

它的核心價值在於「在有限的功耗下提供可觀的推論能力」。一般的桌上型顯示卡功耗可能達數百瓦，難以部署在產線或機器人上；而 Jetson 系列在數瓦到數十瓦的功耗範圍內，即可執行即時的影像辨識與物件偵測。

典型的應用包括：產線的即時外觀檢測（叫獸開講那個場景的正解）、自主移動機器人的環境感知、以及需要低延遲反應的視覺伺服控制。

實務上使用 Jetson 時，模型通常需要經過「推論最佳化」——把訓練框架產出的模型轉換為針對該硬體最佳化的格式，並搭配第 21 章談過的量化。這個轉換往往能帶來數倍的推論速度提升，是部署時不可省略的步驟。

35.3 為什麼需要邊緣運算

35.3.1 四個驅動力

承接叫獸開講，把運算放在邊緣的理由可歸納為四項。

- **頻寬：**原始資料量遠超過可負擔的傳輸容量。這在影像與高頻感測上尤其明顯——本章的公式會把它算清楚。
- **延遲：**控制迴路需要毫秒級的反應，而網路往返無法保證。第 22 章談具身代理時已指出這一點。
- **可用性：**產線不能因為對外網路中斷就停止運作——這正是第 3 章「網路是可靠的」那個誤解的實際後果。
- **隱私與法規：**某些資料不宜或不得離開現場（第 27 章的資料主權、第 33 章的個資保護）。

這四項中，「延遲」與「可用性」是硬約束——它們不會因為頻寬變便宜就消失。因此即使未來網路無限快速，邊緣運算在控制類的應用中仍有不可取代的地位。

35.3.2 哪些運算該放邊緣

判斷的原則可以用兩個問題來決定。

第一個問題是「時間要求」：這項運算必須在多久內完成？若是毫秒級的控制迴路，必須在邊緣；若是分鐘級的統計，放雲端無妨。

第二個問題是「資料縮減比」：這項運算能把資料量減少多少？若能從連續影像縮減為幾個判斷結果（縮減數千倍），放在邊緣的價值極高；若運算後的資料量與原始相當，那放哪裡差別不大。

依這兩個維度，可以得出一個實用的分工原則：邊緣做「即時的、能大幅縮減資料的」運算，雲端做「需要全局視野、需要大量歷史、或需要強大算力的」運算。

叫獸提醒

我要提醒一個常見的誤解：邊緣運算不是要取代雲端，而是要「分工」。我看過一些簡報把它講成「未來一切都在邊緣」——這是不對的。有些事情邊緣做不到：跨廠的比較分析需要全局資料、模型訓練需要大量算力與歷史資料、長期趨勢分析需要數年的累積。這些本來就該在雲端。真正的架構思維是問「這件事最適合在哪一層做」，而非「哪一層比較先進」。你在第 27 章的混合雲、第 21 章的雲端與地端模型取舍，遇到的都是同一個判斷——而答案永遠是「看需求」，不是「看趨勢」。

35.4 邊緣推論的技術要點

35.4.1 模型必須先瘦身

在雲端訓練好的模型，通常無法直接部署到邊緣——因為邊緣裝置的記憶體、算力與功耗都受到嚴格限制。因此需要一系列的壓縮技術，這些你在第 21 章已經見過。

- **量化：**把參數從 32 位元或 16 位元浮點數降為 8 位元整數，記憶體需求降為四分之一或更少，且推論速度提升（第 21 章式 21-1）。
- **剪枝：**移除對輸出影響甚微的連結或通道，減少計算量。
- **蒸餾：**用大模型的輸出訓練一個小模型，以較小的規模保留大部分能力。

- **架構選擇：**直接選用專為邊緣設計的輕量網路架構，而非把大模型硬塞進去。

實務上通常是組合使用：先選擇合適的輕量架構、訓練後量化、再依實測結果決定是否需要進一步剪枝。每一步都要驗證準確率的損失是否在可接受範圍——這是一個典型的取捨過程，沒有標準答案。

35.4.2 推論引擎與硬體加速

除了模型本身，執行環境也影響甚鉅。訓練用的框架通常為了彈性而犧牲效率，直接拿來做推論並非最佳選擇。

邊緣部署時通常會把模型轉換為專用的推論格式，並使用針對該硬體最佳化的推論引擎。這些引擎會做的最佳化包括：把多個運算合併為單一核心（減少記憶體往返）、選擇最適合該硬體的實作、以及善用專用的加速單元。

這個轉換帶來的加速往往相當可觀。所以我要提醒：若你測試邊緣推論的速度時，是直接用訓練框架跑的，那個數字並不代表實際部署後的效能——你可能會低估硬體的能力好幾倍。

35.4.3 容器化與遠端管理

當邊緣裝置的數量從個位數成長到數十、數百台時，「如何管理它們」就成了主要的挑戰。

這裡有一個很好的觀念連結：第 28 章的容器與第 29 章的編排，在邊緣環境同樣適用。把邊緣應用容器化，可以確保各裝置上的執行環境一致（避免「這台好好的、那台跑不起來」）、並使更新可以透過映像檔統一推送。

但邊緣環境也有其特殊性：網路可能不穩定或間歇連線、裝置可能位於難以實體接觸的位置、且資源遠比資料中心受限。因此邊緣的編排工具通常是簡化版的，強調「離線也能運作、恢復連線後自動同步」的能力。

遠端更新是另一個關鍵能力，而它必須極其謹慎——一次失敗的更新可能讓數百台裝置同時無法運作，且若它們位於偏遠處，實體修復的成本極高。因此邊緣更新通常採用第 29 章談過的漸進策略，並保留自動回退的機制。

35.5 邊雲協同架構

35.5.1 三層分工

成熟的架構通常呈現三層分工，各層承擔不同的職責。

層級	職責	時間尺度	資料量
裝置端	感測、致動、即時控制迴路	毫秒	原始資料，量最大
邊緣端	匯集、過濾、聚合、即時推論、本地告警	毫秒至秒	縮減後的事件與摘要
雲端	全局分析、模型訓練、長期儲存、跨廠比較	分鐘至天	彙總資料，量最小

表 35-4 邊雲協同的三層分工

這個表格有一個值得注意的模式：越往上層，時間尺度越長、資料量越小。這不是巧合，而是必然——因為每一層都在對下層的資料做縮減，只把有價值的部分往上送。

你也會發現，這與第 30 章的銅銀金分層有相通之處：都是以「逐層提煉」的方式，讓每一層專注於它最適合的職責。

35.5.2 資料分流策略

邊雲協同的核心決策是：哪些資料要上傳、哪些就地處理後丟棄？常見的策略有四種。

- **事件觸發上傳：**平時只傳摘要，偵測到異常時才上傳該時段的原始資料。這是最常用也最有效的策略。
- **聚合後上傳：**把高頻資料在邊緣聚合為統計量（如每分鐘的平均、最大、最小），大幅減少資料量。
- **抽樣上傳：**定期上傳一小部分原始資料，用於驗證邊緣處理的正確性與模型的重訓（呼應第 34 章的漂移偵測）。
- **分級保留：**原始資料在邊緣本地保留數天供追溯，過期即刪除；只有摘要與事件永久保存於雲端。

第三項「抽樣上傳」特別值得強調，因為它常被忽略。若邊緣只上傳「模型判斷的結果」而從不上傳原始資料，那麼當模型開始衰退時（第 34 章），你將沒有任何資料可以用來偵測與重訓。因此保留一定比例的原始樣本，是維持系統長期健康的必要投資。

35.5.3 邊緣的模型更新

邊雲協同還有一個重要的循環：模型在雲端訓練、部署到邊緣執行、邊緣回傳樣本與監控指標、據此在雲端重新訓練——這正是第 34 章 MLOps 生命週期在分散環境中的展開。

這個循環在邊緣環境中有幾個額外的困難：更新的頻寬成本（模型檔可能數十 MB，乘上數百台裝置）、更新期間的服務中斷、以及各裝置版本不一致所造成的管理複雜度。

實務上的做法包括：只傳送模型的差異部分而非完整檔案、在離峰時段分批更新、以及維持「版本清單」以掌握每台裝置目前的模型版本——這其實就是第 34 章模型登錄的邊緣版本。

35.6 智慧製造的應用場景

35.6.1 三類典型應用

把前面的技術落實到製造現場，常見的應用可分為三類。

- **設備健康監測：**以振動、電流、溫度等訊號判斷設備狀態，預測可能的故障。邊緣負責即時的特徵萃取與異常判斷，雲端負責跨設備的比較與模型訓練。
- **品質檢測：**以視覺或感測訊號判斷產品良窳。這是最需要邊緣推論的場景——因為影像量大且需即時判定（叫獸開講的情境）。
- **製程最佳化：**分析製程參數與結果的關係，找出最佳的操作條件。這偏向雲端的分析工作，但調整結果需下達到現場執行。

35.6.2 實務上的挑戰

我要誠實地談幾個在教科書上較少提及、但實務上一定會遇到的困難。

第一是「舊設備的資料取得」。工廠中往往有運轉數十年的設備，它們沒有網路介面、甚至沒有數位輸出。要取得資料可能需要外掛感測器，而這牽涉到安裝、校正與可能影響原有運作的風險。

第二是「標註資料的稀缺」。異常與不良品在良好的產線上本來就很少——這正是模型訓練最缺的資料。而且標註往往需要資深技師的專業判斷，成本高昂。

第三是「現場環境的嚴苛」。粉塵、震動、電磁干擾、溫濕度變化，都會影響設備的穩定性與感測資料的品質。實驗室裡運作良好的方案，到了現場可能完全不同。

第四，也是最容易被工程師低估的——「與現場人員的協作」。系統的實際使用者是操作員與技師，若介面不符合他們的工作習慣、或告警過於頻繁（第 13 章的告警疲勞），再準確的模型也會被忽略甚至關掉。

35.7 從感測到決策的完整鏈路

35.7.1 把全書串起來

這一節我想帶你走一遍完整的鏈路，看看前面各章的技術如何在一個實際系統中組合起來。

環節	所做的事	對應章節
感測與擷取	感測器產生訊號，經閘道器匯集	本章 35.1、35.2
邊緣處理	過濾、聚合、即時推論與告警	本章 35.4，第 21 章的量化
傳輸	以訊息佇列可靠地送至後端	第 25 章
串流處理	即時彙總、視窗計算、異常偵測	第 24、26 章
儲存	寫入 Lakehouse 的分層架構	第 5、30 章
批次分析	跨時段、跨設備的深度分析	第 11、12 章
模型訓練	以歷史資料訓練與更新模型	第 20、34 章
視覺化與決策	呈現給操作員與管理者	第 37 章
治理與安全	貫穿全程的品質、權限與稽核	第 32、33 章

表 35-5 從感測到決策的完整鏈路

我希望你看到這張表時，能感受到一件事：本書前面談的那些看似分散的技術，其實都是這條鏈路上的某一環。而真正的工程能力，不只是熟悉個別技術，更在於理解「它們如何組合起來」——以及在什麼情況下，哪一環會成為瓶頸。

35.7.2 承先啟後

本章我們補上了資料誕生的那一端。從物聯網的四層架構與感測資料的特性出發（35.1），認識了三個層級的邊緣裝置及樹莓派與 Jetson 的定位（35.2）、邊緣運算的四個驅動力與分工原則（35.3）、邊緣推論的模型壓縮與部署要點（35.4）、邊雲協同的三層分工與資料分流（35.5）、智慧製造的應用場景與實務挑戰（35.6），最後串起了從感測到決策的完整鏈路（35.7）。

我最希望你記住的是 35.3.2 節那個分工原則：邊緣做即時且能大幅縮減資料的運算，雲端做需要全局視野與強大算力的運算。而判斷的依據永遠是「這件事最適合在哪一層做」，不是「哪一層比較先進」。

這一章我們談的是「感測」與「訊號」。但在智慧製造與機器人的場域中，還有一類更複雜的整合——當機器不只是被動地感測，而要主動地移動、定位、抓取、與人協作時，需要的技術又是另一個層次。這就是下一章的主題：機器人與智慧製造的資料整合。

公式與流程：頻寬需求與邊緣的縮減效益

這一節要把叫獸開講那個「連傳都傳不出去」的問題算清楚，並量化邊緣處理能省下多少。

原始上傳的頻寬需求

設有 n 個資料來源，每個來源的持續資料率為 b (Mbps)，則所需的上行頻寬為：

$$\text{頻寬需求} = n \times b \text{ (Mbps)}$$

式 35-1 原始資料的上傳頻寬需求

而每月的傳輸總量（以 TB 計）為：

$$\text{月流量} = (n \times b / 8) \times 3,600 \times 24 \times 30 / 10^6 \text{ (TB)}$$

式 35-2 每月的傳輸總量

以叫獸開講的情境計算：100 支攝影機、每支 4 Mbps。

頻寬需求 = $100 \times 4 = 400$ Mbps；月流量 = $(400 / 8) \times 2,592,000 / 10^6 = 129.6$ TB。

這解釋了為什麼 300 Mbps 的專線根本不夠——而即使頻寬足夠，每月 129.6 TB 的雲端流量費用也相當可觀。

邊緣處理後的資料量

若改為在邊緣執行推論，只有偵測到異常時才上傳該事件的影像片段。設每個來源每日產生 e 個事件、每個事件的資料量為 s (MB)，則每月的上傳量為：

$$\text{邊緣月流量} = n \times e \times s \times 30 / 10^6 \text{ (TB)}$$

式 35-3 邊緣處理後的上傳量

設每支攝影機每日產生 50 個事件、每個事件約 200 KB (0.2 MB)：

邊緣月流量 = $100 \times 50 \times 0.2 \times 30 / 10^6 = 0.03$ TB (約 30 GB)。

方案	上行頻寬	月流量	相對比例
原始影像全上傳	400 Mbps	約 129.6 TB	100%
邊緣推論後上傳事件	極低 (尖峰時數 Mbps)	約 0.03 TB	約 0.023%
縮減倍數	—	約 4,300 倍	—

表 35-6 邊緣處理的頻寬縮減效益

叫獸提醒

四千多倍的縮減看起來很驚人，但我要你注意這個數字背後的取捨：你丟掉了 99.98% 的原始影像。若日後發現模型有問題想重新分析，那些資料已經不存在了。所以實務上的正確做法，是搭配 35.5.2 節的「抽樣上傳」——除了異常事件外，另外隨機保留一小部分正常樣本（例如 0.1%），用於驗證模型、偵測漂移與日後的重新訓練。這一小部分的成本微不足道，卻是系統能長期健康運作的保險。我看過不少專案只上傳「模型判斷有問題的資料」，結果一年後模型開始衰退時，手上完全沒有可用來重訓的資料——省下了頻寬，卻付出了更大的代價。

案例研討

案例一 傳不出去的一百支攝影機

回到叫獸開講。該工廠原本規劃把一百支攝影機的影像全部上傳雲端做 AI 檢測，依式 35-1 需要 400 Mbps 的持續上行頻寬，而他們的專線只有 300 Mbps——方案在物理上就不可行。

即使升級專線，依式 35-2 每月 129.6 TB 的流量費用也遠超過整個專案的預算。

改採邊緣架構後：每五支攝影機配置一台 Jetson 執行影像推論，僅在偵測到疑似缺陷時上傳該片段供人工複判。上傳量降至每月約 30 GB，現有頻寬綽綽有餘，且因為推論在本地完成，判定延遲也從數百毫秒降到數十毫秒——這對需要即時剔除不良品的產線至關重要。

這個案例完整展現了 35.3.1 節的四個驅動力中的前兩項：頻寬與延遲。而它們都是硬約束，不會因為預算增加就消失。

案例二 斷網兩小時的產線

某工廠的品質判定服務部署在雲端。某日對外網路因電信商施工而中斷兩小時，產線的自動判定功能隨之停擺，只能改為人工目檢，產能大幅下降。

事後檢討時，有人主張「加一條備援線路」。但更根本的問題是：一條產線的正常運作，不應該取決於「對外網路是否暢通」——這正是第 3 章「網路是可靠的」那個誤解的具體後果。

他們最終的改善是把即時判定移到邊緣，雲端只負責模型更新與長期分析。此後即使斷網，產線仍能正常運作，只是暫時無法上傳資料與接收模型更新——這些都是可以延後的工作。

這個案例說明了 35.3.1 節「可用性」這個驅動力：邊緣運算的價值不只是快與省，更在於「降低對外部依賴」。系統的可用性，取決於它最脆弱的那個依賴。

案例三 跑不動的模型

某團隊在工作站上訓練了一個影像分類模型，測試效果良好，直接部署到 Jetson 上——結果推論速度只有每秒兩幀，遠低於產線所需的每秒十五幀。

他們一度認為是硬體不夠力，考慮升級到更高階的型號。但在檢視後發現兩個問題：模型仍是訓練框架的原始格式、未經推論最佳化；且參數仍是 32 位元浮點數、未做量化。

經過模型轉換與 8 位元量化後，推論速度提升到每秒二十餘幀，準確率僅下降不到一個百分點——原有的硬體完全足夠。

這正是 35.4.2 節所提醒的：若你用訓練框架直接測量邊緣推論的速度，會嚴重低估硬體的能力。在考慮升級硬體之前，應先確認模型是否已經過適當的最佳化——這與第 20 章「先優化再分散」是同一個判斷原則。

案例四 沒有資料可以重訓

某產線的邊緣檢測系統運作一年後，品管人員發現誤判逐漸增加。團隊決定重新訓練模型，卻遇到一個尷尬的問題：系統只上傳「判定為不良」的影像，而正常品的影像從未被保留。

這意味著他們手上只有「模型認為有問題的樣本」——而這正是最有偏誤的一批資料（其中還包含了誤判）。用它來重訓，只會讓模型的偏誤更加固化。

他們最終不得不重新蒐集一批完整的標註資料，耗時數月。事後補上的機制是：除了異常事件外，每日隨機保留千分之一的正常樣本上傳。這批資料的量微不足道（每月不到 1 GB），卻是模型能持續健康的關鍵。

這個案例呼應了 35.5.2 節與本章叫獸提醒的警告，也串起了第 34 章的模型衰退與重訓——邊緣架構的資料分流策略，必須把「未來的重訓需求」一併納入考量。

實作活動

活動一 估算你的頻寬需求

請為一個具體的物聯網情境完成頻寬與成本評估。

- 步驟 1.** 設定情境：列出感測來源的種類、數量、以及各自的資料產生率（可查閱實際的感測器規格）。
- 步驟 2.** 依式 35-1 與式 35-2 計算原始上傳的頻寬需求與每月流量。
- 步驟 3.** 設計一個邊緣處理方案（過濾、聚合或推論），依式 35-3 估算縮減後的上傳量。
- 步驟 4.** 查詢雲端的資料傳輸與儲存費率，計算兩種方案的每月成本差異；並依 35.5.2 節加上抽樣上傳，說明它增加了多少成本。

活動二 在邊緣裝置上部署模型

請以樹莓派、Jetson 或任何可用的邊緣裝置（無實體裝置時可用個人電腦模擬資源受限的環境）完成部署。

- 步驟 1.** 選擇一個輕量的預訓練模型（可從第 23 章的 Hugging Face 取得），在裝置上直接以訓練框架執行推論，測量其速度。
- 步驟 2.** 把模型轉換為推論最佳化的格式，再次測量並比較。
- 步驟 3.** 進行 8 位元量化，第三次測量速度，並在測試集上比較準確率的變化。
- 步驟 4.** 整理三次測量的結果，說明速度提升了幾倍、準確率損失多少，並判斷這個取舍是否可接受。

活動三 設計一個邊雲協同架構

請為一個智慧製造情境設計完整的邊雲協同架構。首先明確定義應用需求（要偵測什麼、時間要求為何、資料量多大）。接著依表 35-4 的三層分工，說明每一層各負責什麼、使用什麼硬體、資料如何流動。然後設計資料分流策略：哪些即時處理後丟棄、哪些聚合上傳、哪些事件觸發上傳、以及如何保留供重訓的樣本。最後回答三個問題：若對外網路中斷，系統的哪些功能仍能運作？模型要更新時如何推送到各邊緣節點？以及若一年後要重新訓練模型，你手上會有什麼資料？

延伸閱讀

- **NVIDIA Jetson 開發者文件：**說明邊緣推論的模型轉換、最佳化與部署流程，是 35.2、35.4 節的實作參考。
- **物聯網通訊協定的比較文獻：**整理各類無線技術在頻寬、距離與功耗上的取舍，可深化 35.1.3 節。
- **《Edge Computing》相關教科書章節：**從系統架構角度探討邊緣運算的定位與挑戰，是 35.3、35.5 節的理論補充。

- **工業物聯網的標準與實務指引：**說明製造現場的通訊標準、資料模型與導入經驗，是 35.6 節的產業延伸。

本章小結

1. 物聯網分感知、網路、平台、應用四層；本書前面各章主要著墨於後兩層，而本章補上了資料誕生的前兩層。
2. 感測資料具有高頻持續、單筆價值低、時序性強、含雜訊缺漏、時間戳不可靠等特性；其中「單筆價值低」正是邊緣運算得以成立的前提——多數原始讀數不必保留。
3. 邊緣裝置分微控制器、單板電腦（如樹莓派，適合閘道器與輕量處理）與邊緣運算平台（如 Jetson，具 GPU 可執行深度學習推論）三個層級；選擇原則是先確認必須在本地完成的運算，再選剛好足夠的硬體。
4. 邊緣運算的四個驅動力為頻寬、延遲、可用性與隱私法規；其中延遲與可用性是硬約束，不會因頻寬變便宜而消失。
5. 判斷運算該放哪一層有兩個依據：時間要求與資料縮減比。原則是邊緣做「即時且能大幅縮減資料」的運算，雲端做「需全局視野與強大算力」的運算——判斷依據永遠是需求而非趨勢。
6. 邊緣推論須先對模型瘦身（量化、剪枝、蒸餾、輕量架構），並轉換為專用的推論格式；若直接用訓練框架測量速度，會嚴重低估硬體能力。邊緣裝置的管理同樣可運用容器與編排，但須因應間歇連線與遠端更新的風險。
7. 邊雲協同呈三層分工，越往上層時間尺度越長、資料量越小；資料分流策略包含事件觸發、聚合、抽樣與分級保留，其中「抽樣上傳」最常被忽略卻是模型長期健康的必要投資。
8. 原始上傳的頻寬需求為 $n \times b$ （式 35-1）、月流量依式 35-2 計算；邊緣處理後的上傳量（式 35-3）可能只有原始的萬分之二。但大幅縮減的代價是丟棄了絕大多數原始資料，故必須搭配抽樣保留，否則日後將無資料可供重訓。

習題

一、觀念題

1. 請說明感測資料的五項特性，並解釋其中哪一項是邊緣運算得以成立的前提。
2. 請比較樹莓派與 NVIDIA Jetson 的定位差異，並各舉一個最適合的應用場景。
3. 邊緣運算的四個驅動力為何？為什麼說其中兩項是「硬約束」？
4. 為什麼「只上傳模型判定有問題的資料」是危險的做法？請結合第 34 章的模型衰退說明。

二、計算題

1. 某廠區裝設 60 支攝影機，每支的串流資料率為 6 Mbps。（一）依式 35-1 計算所需的上行頻寬；（二）依式 35-2 計算每月的傳輸總量（TB）；（三）若專線為 500 Mbps，此方案是否可行？
2. 承上題，改採邊緣推論：每支攝影機每日產生 80 個事件，每個事件的片段約 0.5 MB。（一）依式 35-3 計算每月上傳量；（二）計算相對於原始方案的縮減倍數；（三）若另外每日隨機保留千分之一的正常影格（假設每支每日 20 萬影格、每影格 0.1 MB），這部分每月增加多少上傳量？占縮減後總量的比例為何？

三、思考與討論

1. 本章指出「邊緣運算不是要取代雲端，而是要分工」。請以你熟悉的一個應用，說明哪些運算你會放在邊緣、哪些放在雲端，以及你的判斷依據。

2. 案例四中，團隊因未保留正常樣本而無法重訓。請討論：在設計一個長期運作的系統時，還有哪些「當下看似不必要、但未來會需要」的資料或紀錄？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

五項特性：高頻且持續（每秒數十至數千次且永不停止，即無界資料）、單筆價值低（單獨一筆讀數幾無意義，價值來自趨勢與異常）、時序性強（意義依賴時間順序與間隔）、可能有雜訊與缺漏（感測器漂移故障、網路中斷）、時間戳的難題（裝置時鐘不準或不同步，使事件時間不可靠）。其中「單筆價值低」是邊緣運算的前提——正因為多數原始讀數不必保留，才能在邊緣先做聚合、篩選與推論，只把有價值的部分（異常事件、統計摘要）上傳。若每一筆原始資料都必須完整保存，邊緣的縮減就無從發揮。

第 2 題

樹莓派屬單板電腦，可執行完整作業系統但無專用加速硬體，最適合作為「閘道器」與輕量處理節點——匯集多個感測裝置的資料、協定轉換、初步過濾與聚合，或執行極輕量的判斷邏輯。適合場景如：匯集一整區溫濕度與振動感測器並上傳摘要。Jetson 屬邊緣運算平台，內含 GPU，能在數瓦至數十瓦的功耗下執行深度學習推論，適合影像與需要低延遲反應的任務。適合場景如：產線的即時外觀檢測（每秒需處理十餘幀影像）、自主移動機器人的環境感知。選擇原則：先確認必須在本地完成的運算是什麼，再選剛好足夠的硬體。

第 3 題

四個驅動力：頻寬（原始資料量遠超可負擔的傳輸容量）、延遲（控制迴路需毫秒級反應而網路往返無法保證）、可用性（產線不能因對外網路中斷而停止運作）、隱私與法規（某些資料不宜或不得離開現場）。其中「延遲」與「可用性」是硬約束的原因：它們源於物理與架構的本質，不會因為投入更多預算而消失。頻寬不足可以升級專線、流量費用可以編列預算，但光速造成的網路往返延遲無法縮短到毫秒以下，而「依賴外部網路」這件事本身就構成了可用性的風險（見案例二）。故即使未來網路無限快速且免費，控制類應用仍需在邊緣執行。

第 4 題

危險之處在於「樣本偏誤」與「無資料可重訓」兩個層面。其一，只上傳模型判定為異常的資料，等於只保留了「模型認為有問題」的樣本——這批資料本身就帶有模型的偏誤，且其中還混雜著誤判；用它重訓只會使既有偏誤更加固化。其二，正常樣本從未保留，故缺乏代表真實分布的訓練資料。結合第 34 章：模型會隨時間安靜地衰退（資料漂移與概念漂移），而偵測漂移需要能反映當前輸入分布的樣本、重新訓練更需要完整且無偏的標註資料。若這些都沒有保留，當衰退發生時將無從偵測、也無從修正，只能重新蒐集標註資料而耗時數月（見案例四）。故應採 35.5.2 節的抽樣上傳，定期隨機保留一小部分正常樣本。

二、計算題

第 1 題

$n = 60$ 、 $b = 6$ Mbps。

- (1) 上行頻寬 (式 35-1) $= 60 \times 6 = 360$ Mbps。
- (2) 月流量 (式 35-2) $= (360 / 8) \times 3,600 \times 24 \times 30 / 10^6 = 45 \times 2,592,000 / 10^6 = 116.64$ TB。
- (3) 可行性：360 Mbps 未超過 500 Mbps 的專線容量，理論上「傳得出去」；但須注意這是持續佔用約 72% 的頻寬，幾無餘裕供其他業務使用，且每月 116.64 TB 的雲端流量費用相當可觀。故技術上勉強可行，經濟上與穩定性上皆不理想。

第 2 題

$n = 60$ 、 $e = 80$ 事件／日、 $s = 0.5$ MB。

- (1) 邊緣月流量 (式 35-3) $= 60 \times 80 \times 0.5 \times 30 / 10^6 = 72,000 / 10^6 = 0.072$ TB (約 72 GB)。
- (2) 縮減倍數 $= 116.64 / 0.072 = 1,620$ 倍。
- (3) 抽樣保留：每支每日 200,000 影格 $\times 0.001 = 200$ 影格； 60 支 $\times 200 \times 0.1$ MB $= 1,200$ MB／日；每月 $= 1,200 \times 30 = 36,000$ MB $= 36$ GB $= 0.036$ TB。占縮減後總量的比例 $= 0.036 / (0.072 + 0.036) = 33.3\%$ 。

延伸思考：抽樣保留使總上傳量從 72 GB 增至 108 GB (增加五成)，但相對於原始方案的 116.64 TB 仍僅約千分之一。這 36 GB 是「維持模型長期健康」的保險費——相較於案例四中重新蒐集標註資料所耗費的數月人力，這個成本微不足道。設計時應把它視為必要開銷而非可選項。

三、思考與討論

第 1 題

答題應包含情境描述、分層配置與判斷依據。以「產線設備健康監測」為例：邊緣負責高頻振動訊號的即時特徵萃取 (如頻譜能量、峰值因子)、閾值告警、以及輕量的異常判斷——依據是時間要求 (異常需即時停機) 與資料縮減比 (原始波形每秒數千點，萃取後每分鐘數個特徵值，縮減數萬倍)。雲端負責跨設備的比較分析、長期趨勢與剩餘壽命預測、以及模型訓練——依據是需要全局視野 (單台設備看不出「這台比同型號其他設備衰退得快」)、需要大量歷史資料、且時間要求寬鬆。裝置端則負責最基本的安全連鎖 (如超過絕對上限立即停機)，因其時間要求最嚴且不容許任何外部依賴。判斷依據應明確歸結為 35.3.2 節的兩個問題：時間要求與資料縮減比。

第 2 題

可提出的項目包括：其一，訓練與驗證用的原始樣本 (本章案例四)——包含正常樣本而非只有異常。其二，資料的版本與血緣紀錄 (第 32 章)——當發現歷史分析有誤時，需要知道當時用的是哪一版資料。其三，實驗與模型的完整紀錄 (第 34 章)——參數、指標、程式碼版本，否則無法重現或比較。其四，設定與環境的變更歷程 (第 34 章的「版本控制一切」)——「這個參數為什麼是這個值」往往在數月後成為謎團。其五，人工判定與標註的紀錄——包含判定者與時間，這是日後校正標註品質的依據。其六，事件與異常的處理紀錄——包含當時的判斷與後續驗證結果，這是唯一能建立「模型判斷是否正確」之基準的資料。其七，稽核軌跡 (第 33 章)——誰在何時存取或修改了什麼。共同判準：凡是「事後無法重建」的資訊，都應在當下記錄；而判斷成本時，應以「日後重新取得的代價」而非「當下儲存的成本」為基準——多數情況下前者高出數個量級。

台灣自編大學教科書
@prof
大數據分析與雲端運算

第玖篇 整合實務與應用

Integration & Applications

第 36 章

機器人與智慧製造整合

Robotics and Smart Manufacturing Integration

機器人叫獸（李世淵） 編著

叫獸開講

我實驗室有一台機械手臂，去年有組學生做視覺抓取——用攝影機辨識零件位置，再控制手臂去抓。

模型訓練得很好，離線測試的定位誤差不到兩毫米。但實際跑起來時，手臂總是抓不準——不是抓偏，就是撞到零件。

學生查了很久，以為是模型不夠準、是相機校正有問題、是手臂的機械精度不足。最後我們一起算了一次「延遲」，答案就出來了：從攝影機拍到影像、到手臂真正動作，中間累積了超過兩百毫秒。而輸送帶上的零件在這兩百毫秒內，已經移動了將近五公分。

模型看到的是零件「兩百毫秒前」的位置，手臂卻依這個過時的位置去抓——當然抓不到。問題不在準確度，而在時間。

這件事說明了機器人系統與前面各章所談系統的一個根本差異：資料分析系統慢一點只是使用者等久一點，但機器人系統慢一點，物理世界不會等你。這一章我們要談的，就是當資料技術要與「會動的機器」整合時，多出來的那些考量。

學習目標

研讀完本章之後，你應該能夠：

1. 說明機器人系統的資料特性，並指出其與一般資料系統的差異。
2. 說明 ROS 的通訊模型，並理解其與訊息佇列的異同。
3. 說明感測融合的動機與基本方法。
4. 說明 SLAM 的問題定義與其在資料層面的挑戰。
5. 說明協作型機器人的安全要求與資料需求。
6. 運用延遲預算分析，判斷即時控制迴路的可行性。

核心概念

核心概念	說明
即時性	系統必須在確定的時間上限內完成反應的特性。
ROS	機器人作業系統，以節點與話題構成的分散式通訊框架。
感測融合	結合多種感測器的資料以取得比單一來源更可靠的估計。
SLAM	同步定位與地圖建構，機器人在未知環境中同時建圖與定位。
協作型機器人	可與人在同一空間工作、具備安全機制的機械手臂。
數位分身	以資料建構的實體系統虛擬對應，用於模擬與監控。
延遲預算	把端到端的時間上限分配給各處理環節的設計方法。

表 36-1 本章核心概念一覽

36.1 機器人系統的資料特性

36.1.1 四個根本差異

承接叫獸開講，機器人系統的資料處理與前面各章所談的系統，有四個結構性的差異。

面向	一般資料系統	機器人系統
時間要求	延遲高一點只是等久一點	超過上限即失效，甚至造成危險
資料型態	多為結構化紀錄與文字	感測訊號、影像、點雲，資料量極大
失敗後果	重跑一次即可	可能撞壞設備或傷及人員，不可逆
回饋迴路	分析結果供人判讀	分析結果直接驅動實體動作

表 36-2 一般資料系統與機器人系統的差異

其中「回饋迴路」這一項最為關鍵。在前面各章中，分析的輸出是給人看的報表或告警，人會用常識過濾明顯錯誤的結果；但在機器人系統中，模型的輸出直接變成馬達的指令——若判斷錯誤，機器會忠實地執行那個錯誤。

這也是為什麼第 22 章談具身代理時，我強調「人在迴路中」的設計原則。當系統的輸出會改變物理世界時，錯誤的代價與資訊世界完全不同。

36.1.2 軟即時與硬即時

「即時」這個詞在工程上有明確的定義，且分為兩種。

硬即時要求「絕對不能超過時間上限」——超過即視為系統失效。安全連鎖、伺服控制迴路屬於此類：一個必須在 1 毫秒內反應的緊急停止，慢了 2 毫秒就可能造成傷害。

軟即時則允許偶爾超時，只是品質下降。視覺辨識、路徑規劃多屬此類：偶爾慢個幾十毫秒，可能只是動作略微遲滯。

這個區分有一個重要的實務推論：硬即時的部分不應該由一般的作業系統與網路來承擔，而應交給專用的即時控制器；而資料分析與 AI 推論這類軟即時的工作，才適合放在本書談過的那些平台上。這正是第 22 章「語言模型高階規劃、傳統系統低階控制」分層架構的理由。

36.2 ROS 與機器人的資料流

36.2.1 節點與話題

機器人作業系統（ROS）並不是一個作業系統，而是一套讓機器人各功能模組彼此通訊的框架。它的核心模型你應該會覺得熟悉。

系統被拆分為多個「節點」，每個節點負責一項功能（讀取雷射掃描、進行定位、規劃路徑、驅動馬達）。節點之間不直接呼叫彼此，而是透過「話題」以發布訂閱的方式交換訊息——發布者不需要知道誰在訂閱，訂閱者也不需要知道誰在發布。

這正是第 25 章的發布訂閱模型，其價值也相同：解耦。你可以替換掉定位模組而不影響其他節點，只要它發布的訊息格式不變；也可以額外加入一個記錄節點來訂閱所有訊息，而發布者完全無感。

除了話題之外，ROS 還提供「服務」（同步的請求回應，適合偶爾呼叫的操作）與「動作」（有進度回報且可中途取消的長時間任務，如「移動到某個位置」）。三種機制對應不同的通訊模式。

36.2.2 與訊息佇列的異同

既然都是發布訂閱，ROS 的話題與第 25 章的 Kafka 有什麼不同？這個比較很有啟發性。

面向	ROS 話題	訊息佇列 (Kafka)
設計目標	低延遲的即時通訊	高吞吐與可靠的資料傳遞
訊息保留	通常不保留，收到即處理	持久化保存，可重播
典型延遲	毫秒級	數毫秒至數十毫秒
資料丟失	可接受（新資料很快就來）	不可接受（須確保送達）
適用範圍	單機或區域網路內	跨資料中心的大規模傳遞

表 36-3 ROS 話題與訊息佇列的比較

表中「資料丟失可接受」這一系列值得說明。對於以 30 赫茲發布的感測資料，丟掉一幀通常無關緊要——因為 33 毫秒後就有新的一幀。相對地，若堅持每一幀都要可靠送達，反而會因為重試而增加延遲，這在即時系統中是更糟的選擇。

這是一個很好的例子，說明「可靠性」與「即時性」有時是衝突的：第 25 章我們追求「至少一次」的送達保證，但在機器人的感測資料上，「盡快送達、丟了就算」反而是正確的設計。技術原則必須依情境調整，這正是理解原理而非死記做法的價值。

36.2.3 機器人資料的兩條路徑

實務上的機器人系統通常有兩條並行的資料路徑，它們的設計目標完全不同。

「控制路徑」處理即時的感測與控制迴路，資料在機器人本體或邊緣裝置內流動，追求最低延遲、不經過雲端，且必須在網路中斷時仍能運作。

「資料路徑」則把運轉紀錄、感測歷史、事件與影像上傳到後端，供分析、模型訓練與長期監控使用。它可以容忍延遲、可以批次傳輸、甚至可以在離線時暫存於本地、恢復連線後再補傳。

這兩條路徑的分離，正是第 35 章邊雲協同的具體展現。而它也解決了一個常見的困惑：機器人到底需不需要雲端？答案是——控制不需要，但學習與改善需要。

36.3 感測融合

36.3.1 為什麼需要融合

沒有任何一種感測器是完美的：相機在昏暗或逆光時失效、雷射雷達對玻璃與鏡面無能為力、超音波精度有限、慣性感測器會隨時間累積誤差。

感測融合的核心想法是：結合多種感測器，讓它們的優點互補、缺點互相掩蓋，得到比任何單一來源都更可靠的估計。

以移動機器人的定位為例：輪子的編碼器能提供高頻但會累積誤差的位移估計；雷射雷達能提供準確但較低頻的絕對位置；慣性感測器則能捕捉快速的姿態變化。把三者融合，就能得到既高頻、又不會漂移的定位。

36.3.2 融合的基本思路

融合演算法的細節超出本書範圍（那屬於機器人學的領域），但它的基本思路與本書談過的觀念高度相通，值得說明。

核心概念是「加權」：每個感測來源都有其不確定性，融合時依可信度給予不同的權重——越可靠的來源權重越高。這與第 19 章推薦系統的加權平均、第 32 章的加權品質分數，在數學形式上是相同的。

另一個關鍵是「預測與修正的循環」：先依運動模型預測「下一刻應該在哪裡」，再用實際的感測值來修正這個預測。這個循環不斷進行，使估計逐漸收斂。

而從資料工程的角度看，感測融合有兩個實務難題：其一是「時間對齊」——各感測器的取樣頻率不同、時間戳可能不同步，必須先對齊到共同的時間基準（這正是第 26 章事件時間問題在機器人上的形態）；其二是「座標轉換」——各感測器安裝在機器人的不同位置與角度，其量測必須轉換到統一的座標系。這兩件事看似瑣碎，卻是實務上最常出錯的環節。

36.4 SLAM：同步定位與地圖建構

36.4.1 先有雞還是先有蛋

SLAM 要解決的問題可以用一句話說明：機器人被放進一個完全陌生的環境，它必須一邊建立這個環境的地圖，一邊確定自己在地圖中的位置。

困難之處在於這是一個循環依賴：要知道自己在哪裡，需要在地圖作為參照；但要建立地圖，又需要知道自己在哪裡才能把觀測到的東西放對位置。這是典型的「先有雞還是先有蛋」問題。

你應該覺得這個結構很眼熟——第 18 章的 PageRank 也是循環定義（網頁的重要性由連入網頁的重要性決定），而它的解法是迭代逼近。SLAM 的解法在精神上相通：從一個初始估計出發，隨著新觀測不斷到來而逐步修正，使地圖與定位同時收斂。

這也再次印證了本書反覆的主張：不同領域的問題，骨子裡往往是同一類數學結構。

36.4.2 資料層面的挑戰

本書的角度不是講解 SLAM 的演算法，而是看它在資料層面帶來什麼挑戰。

- **資料量龐大：**雷射雷達每秒可產生數十萬個點，相機每秒數千萬像素。這些資料若全部保留，儲存與處理成本極高。
- **即時性要求：**定位必須跟上機器人的移動速度，否則估計出的位置已經過時——這正是叫獸開講那個延遲問題的另一種形態。
- **誤差累積：**每一步的估計都有微小誤差，長時間累積後可能顯著偏離。這需要「回環偵測」——當機器人回到曾經來過的地方時識別出來，並據此修正整條軌跡。
- **地圖的維護：**環境會變（貨架移動、隔間調整），地圖需要更新；而何時更新、如何判斷「這是暫時的障礙物還是永久的改變」，是實務上的難題。

其中「回環偵測」在資料處理上特別有意思：它本質上是在問「我現在看到的場景，與過去某個時刻看到的是否相同」——這正是第 14 章的相似性比對問題。而當歷史軌跡很長時，要與所有過去的觀測——比對就會遇到平方成長的困境，因此實務上也採用類似第 14 章的索引與近似方法來加速。

36.4.3 多機器人與地圖共享

當廠區內有多台自主移動機器人時，一個自然的問題是：它們能不能共享地圖？

這帶來明顯的好處：新加入的機器人不必要重新建圖、環境變化可以由發現的機器人更新給所有人、且多台機器人的觀測可以合併成更完整的地圖。

但它也帶來資料工程的挑戰：地圖需要一個集中的儲存與版本管理（誰的版本是最新的？如何合併不同機器人的更新？）、需要處理衝突（兩台機器人對同一區域的觀測不一致時怎麼辦？）、以及需要在網路不穩時仍能運作（本地保有一份副本）。

你會發現這些問題與第 7 章的資料複製與一致性完全同構——多個副本、更新衝突、離線可用性。同樣的原理，在機器人的地圖上再度出現。

36.5 協作型機器人與人機協作

36.5.1 從隔離到共處

傳統的工業機器人通常被安全柵欄圍起來——因為它們力量大、速度快、且完全不知道周圍有沒有人。人與機器在空間上是隔離的。

協作型機器人（Cobot）則設計為可以與人在同一空間工作。這需要一整套的安全機制：力量與速度受限、外殼設計避免夾傷、偵測到碰撞時立即停止、以及感知人員位置並主動減速或避讓。

這個轉變的價值在於彈性：許多任務需要人的判斷與靈巧、也需要機器的力量與穩定，協作型機器人讓兩者能在同一工位上互補，而非各自為政。

36.5.2 安全的資料需求

從資料的角度看，人機協作提出了幾項嚴格的要求。

- **感知的可靠性：**系統必須可靠地偵測人員的位置。這裡的失效模式極不對稱——把人誤判為障礙物只是效率降低，但沒偵測到人可能造成傷害。
- **延遲的上限：**從偵測到人員接近、到機器人減速或停止，必須在確定的時間內完成。這屬於硬即時的範疇。
- **失效安全：**當感測器故障或系統異常時，預設行為必須是「停止」而非「繼續」。這與一般資訊系統「盡量維持服務」的思維恰好相反。
- **可追溯性：**所有的安全相關事件都必須被記錄，以供事故調查與法規稽核（第 32、33 章）。

叫獸提醒

我要特別談「失效安全」這一項，因為它與你在本書前面學到的許多思維是相反的。第 3 章我們談可用性、談如何讓系統在故障時仍持續服務；但在人機協作的安全系統中，正確的設計是「一旦不確定就停下來」。一個因為感測器故障而「保守地停機」的系統，會造成產能損失；但一個「故障了還繼續動」的系統，可能造成無法挽回的傷害。這個取捨沒有商量的餘地。我要你記住的是：技術原則不是普世的教條，而是針對特定情境的權衡。當情境的代價結構改變時（從「服務中斷」變成「人員受傷」），原則也必須跟著改變。判斷「這個情境的代價結構是什麼」，比記住任何原則都更重要。

36.5.3 人機協作的資料價值

除了安全之外，人機協作也產生了有價值的資料。

例如：人在什麼情況下會介入修正機器人的動作？哪些工序人做得比較快、哪些機器人比較穩定？操作員的手動示教軌跡，能否用來改善機器人的動作規劃？

這些資料的分析，可以回饋到工序的重新分配與機器人的行為改善——而這正是本書談的資料技術在此場域的價值所在：不是取代人，而是讓人與機器的分工持續優化。

36.6 數位分身與智慧製造整合

36.6.1 什麼是數位分身

數位分身（Digital Twin）是實體系統在虛擬世界中的對應——它以資料驅動，持續反映實體系統的當前狀態，並可用於模擬、預測與最佳化。

要注意它與「模擬模型」的差別：一般的模擬模型是靜態的，建立後即獨立運作；而數位分身持續接收實體系統的資料，與實體保持同步。這個「持續同步」正是它需要本書所談之資料技術的原因。

它的價值主要在三個方面：模擬（在虛擬環境中測試新的參數或流程，不影響實際生產）、預測（依當前狀態預測未來的表現或故障）、以及最佳化（尋找更好的操作條件再套用到實體）。

36.6.2 資料需求與挑戰

建立數位分身的資料需求相當可觀，也是多數專案的困難所在。

- **模型的建立：**需要對實體系統的行為有足夠精確的模型——這往往需要領域知識，而非單靠資料驅動。
- **資料的同步：**虛擬與實體的狀態必須保持一致，這需要可靠的資料管線（第 25、26 章）與明確的更新頻率。
- **精度的驗證：**分身的預測與實體的實際表現是否相符？這需要持續比對，且會隨設備老化而偏離（呼應第 34 章的模型衰退）。
- **範圍的界定：**分身要模擬到多細？過於精細則成本高昂且難以維護，過於粗糙則失去價值。

最後一項最為關鍵，也最常被低估。數位分身容易淪為「為做而做」的展示品——建了一個很酷的三維視覺化，卻回答不了任何實際的問題。務實的做法是從一個具體的問題出發（例如「這個參數調整會如何影響良率」），建立剛好足以回答它的分身，再逐步擴充。

36.6.3 整合的層次

把機器人、設備與資料系統整合起來，實務上有幾個層次，而多數組織是逐層推進的。

層次	特徵	主要挑戰
連線	設備能上傳資料，可遠端查看狀態	舊設備的資料取得、通訊標準不一
可視	資料被整合並以儀表板呈現	資料定義的統一（第 32 章）
分析	能從資料中找出模式、預測異常	標註資料稀缺、模型維護（第 34 章）
優化	分析結果回饋到參數調整與排程	跨系統整合、變更管理
自主	系統能自行判斷並調整，人只需監督	可靠性要求極高、責任歸屬明確

表 36-4 智慧製造整合的五個層次

我要提醒的是：不必以最高層為目標。多數工廠在「分析」與「優化」這兩層就能得到絕大部分的效益，而「自主」需要極高的可靠度與明確的責任設計——這與第 34 章表 34-5 的成熟度階梯是同一個判斷：層次越高，前提條件越嚴苛。

36.7 整合實務的經驗

36.7.1 常見的失敗模式

我想在這一節誠實地談幾個實際專案中反覆出現的失敗模式。

- **從技術出發而非從問題出發：**「我們也來導入 AI」比「我們要解決什麼問題」更常成為專案的起點，結果是做出了技術上正確、業務上無用的系統。
- **低估資料的取得成本：**多數專案的時間，其實花在「把資料弄出來且弄乾淨」，而非演算法。低估這一點會使時程嚴重失準。
- **忽略現場的使用者：**系統的實際使用者是操作員與技師。若介面不符工作流程、或告警過多（第 13 章的告警疲勞），系統會被繞過甚至關閉。
- **只做示範不做維運：**在展示會上運作良好，上線半年後無人維護、模型衰退、資料管線斷裂——這正是第 34 章整章要處理的問題。

36.7.2 務實的推進方式

相對地，成功的專案往往有幾個共同特徵。

第一，從一個明確且夠小的問題開始。「降低某條產線某項缺陷的漏檢率」比「打造智慧工廠」更容易成功，且成功後能建立信任與資源。

第二，先做出能運作的最簡版本，再逐步改善。這與本書第 22、31 章反覆提到的「複雜度應該被需求逼出來」是同一個原則。

第三，把現場人員納入設計過程。他們知道實際的工作流程、知道哪些告警有意義、也知道系統在什麼情況下會被繞過。

第四，從一開始就規劃維運。誰負責監控？模型如何更新？資料品質如何檢核？這些若在專案結束後才想，通常就沒有人做了。

36.7.3 承先啟後

本章我們把資料技術帶進了「會動的機器」這個場域。從機器人系統的四個根本差異與軟硬即時的區分出發（36.1），認識了 ROS 的通訊模型及其與訊息佇列的異同（36.2）、感測融合的動機與資料難題（36.3）、SLAM 的循環依賴與資料挑戰（36.4）、協作型機器人的安全要求與失效安全原則（36.5）、數位分身與整合的五個層次（36.6），最後談了實務上的失敗模式與務實推進（36.7）。

我最希望你帶走的，是 36.5.2 節那個叫獸提醒的觀念：技術原則不是普世的教條。同樣是「系統故障了怎麼辦」，資料系統的答案是「盡量維持服務」，而安全系統的答案是「立刻停止」——因為兩者的代價結構完全不同。判斷情境的代價結構，比記住任何原則都重要。

我們已經走過了完整的技術鏈路：從資料的產生、傳輸、儲存、運算、分析、到與實體系統的整合。但還有最後一哩路——這些分析的結果，最終要被人看見、被理解、被用來做決定。如果一份精確的分析沒有人看得懂、或看懂了卻不知道該採取什麼行動，那麼前面所有的努力都白費了。這就是下一章的主題：資料視覺化與決策支援。

公式與流程：延遲預算與控制迴路的可行性

這一節要把叫獸開講那個「抓不準」的問題變成可計算的設計工具——這是機器人系統設計中最實用的一項分析。

延遲預算

設控制迴路的目標頻率為 f （赫茲），則每一個迴圈可用的時間預算為：

$$\text{時間預算} = 1,000 / f \text{（毫秒）}$$

式 36-1 控制迴路的時間預算

而端到端的實際延遲，是各個環節耗時的總和：

$$\text{端到端延遲} = \sum t_i \text{（各環節耗時之和）}$$

式 36-2 端到端的延遲

系統可行的條件是「端到端延遲小於時間預算」，且應保留餘裕以因應波動。以 30 赫茲的視覺伺服控制為例，時間預算為 $1,000 / 30 \approx 33.3$ 毫秒。

環節	耗時	佔預算	可優化方向
影像擷取與傳輸	6 ms	18.0%	降低解析度、改用更快的介面
前處理	3 ms	9.0%	簡化處理、移至硬體加速
模型推論	12 ms	36.0%	量化、模型精簡（第 21、35 章）
後處理與決策	2 ms	6.0%	簡化邏輯
通訊至控制器	3 ms	9.0%	改用即時通訊、縮短路徑
控制器執行	4 ms	12.0%	多為固定開銷
合計	30 ms	90.0%	餘裕僅 3.3 ms

表 36-5 30 赫茲視覺伺服的延遲預算分解

這張表的價值在於「找出瓶頸」：模型推論佔了 36%，是最值得優化的環節；而控制器執行的 4 毫秒多為固定開銷，優化空間有限。若要提升到 60 赫茲（預算 16.7 毫秒），現有的 30 毫秒完全不可行，必須大幅重新設計。

延遲造成的位置誤差

延遲的實際後果，可以換算為空間上的誤差。設目標物的移動速度為 v （公分／秒）、端到端延遲為 T （毫秒），則因延遲造成的位置偏差為：

$$\text{位置誤差} = v \times T / 1,000 \text{（公分）}$$

式 36-3 延遲造成的位置誤差

回到叫獸開講的情境：輸送帶速度 25 公分／秒、端到端延遲 200 毫秒，則位置誤差 = $25 \times 200 / 1,000 = 5$ 公分——正好對應學生觀察到的「總是抓偏」。

叫獸提醒

式 36-3 給了你一個非常實用的診斷工具。當機器人「抓不準」時，先別急著怪模型不準或機構精度不足——先量一下端到端延遲，再乘上目標的移動速度。若算出來的誤差與實際觀察到

的偏差相當，那問題就在延遲而非精度。我看過太多團隊花數週去調整模型與校正相機，卻從未量過延遲。順帶一提，這個公式也告訴你兩條解決路徑：降低延遲（優化各環節），或是預測補償（依速度預測目標在動作時刻的位置，主動提前量）。後者在延遲無法再壓縮時特別有用——你無法讓時間變快，但你可以預測它。

案例研討

案例一 抓偏五公分的手臂

回到叫獸開講。學生的視覺抓取系統定位誤差不到兩毫米，實際卻總是抓偏。

我們一起做了延遲預算分析，逐段量測後發現：影像從相機傳到電腦約 40 毫秒（使用了較慢的傳輸介面）、模型推論 85 毫秒（未經最佳化，直接以訓練框架執行）、經由一般網路傳到控制器約 45 毫秒、控制器規劃與執行約 30 毫秒，合計約 200 毫秒。

依式 36-3，輸送帶速度 25 公分／秒時，位置誤差達 5 公分——與觀察完全吻合。

改善分兩方面：技術上，以第 35 章的模型最佳化與量化把推論降到 15 毫秒、改用直連的傳輸介面與即時通訊，端到端降至約 50 毫秒；設計上，加入依輸送帶速度的預測補償，使剩餘的誤差降到毫米級。這個案例說明：在機器人系統中，「準確度」與「即時性」是兩個獨立的問題，而後者往往被忽略。

案例二 被玻璃騙倒的機器人

某物流倉庫導入自主移動機器人，運作大致良好，但偶爾會撞上某區域的玻璃隔間。

原因是該機器人主要依賴雷射雷達建圖與避障，而雷射光穿透玻璃，使機器人「看不到」那面牆——在它的感知中，那裡是暢通的。

解法是感測融合（36.3 節）：加裝超音波感測器（對玻璃有反應）與視覺辨識，並在地圖中把該區域標記為「已知的特殊區域」，使機器人經過時採取更保守的策略。

這個案例展現了 36.3.1 節的核心論點——沒有任何單一感測器是完美的，而融合的價值正在於「以彼此的優點掩蓋各自的盲點」。它也提醒我們：系統的失效往往不是因為技術不夠好，而是因為現實中存在設計時未曾預期的情況。

案例三 地圖過期的困擾

某工廠的自主移動機器人使用一年前建立的地圖。期間廠內經過數次產線調整、貨架移位，導致機器人頻繁地在「地圖上應該暢通、實際上有障礙」的地方停滯，需要人工介入。

團隊面臨一個判斷難題：機器人偵測到與地圖不符的障礙時，該視為「暫時的障礙物」（如停放的推車，繞過即可）還是「環境的永久改變」（應更新地圖）？

他們的解法是引入時間維度：同一位置的障礙若在多次經過中都被觀測到，且持續超過設定的時間，即更新地圖；反之則視為暫時障礙。同時建立了地圖的版本管理與多機共享機制，使任一台機器人發現的變化能同步給其他機器人。

這個案例呼應 36.4.2 節的地圖維護難題，而其解法在概念上與第 32 章的資料治理相通——地圖也是一份需要版本、需要負責人、需要更新流程的資料資產。

案例四 被關掉的安全告警

某產線導入協作型機器人，安全系統會在偵測到人員進入特定範圍時使機器人減速。上線初期一切正常，但數月後維護人員發現，該功能已被操作員設定為停用。

詢問後才知道原因：偵測範圍設定得過於保守，導致操作員在正常作業時頻繁觸發減速，嚴重影響效率。他們反映過但未被處理，最後選擇了自行關閉。

這是一個極危險的狀況，但責任不能全歸於操作員——問題的根源是「安全設定未考慮實際的工作流程」，使得遵守規則的代價高到讓人選擇繞過。

這正是 36.7.1 節「忽略現場使用者」的失敗模式，也與第 13 章的告警疲勞同源：當系統的干擾大於價值時，人會想辦法繞過它。他們後來與操作員共同重新設計了感測範圍與減速策略，在安全與效率之間取得平衡，同時把「安全功能被停用」納入監控告警——這是技術與人因必須並重的最好例證。

實作活動

活動一 做一次延遲預算分析

請為一個實際或設想的機器人控制迴路，完成完整的延遲預算分析。

- 步驟 1.** 設定目標控制頻率，依式 36-1 計算時間預算。
- 步驟 2.** 列出從感測到動作的所有環節，並逐一估計（或實測）其耗時。
- 步驟 3.** 依式 36-2 計算端到端延遲，判斷是否符合預算，並找出佔比最高的環節。
- 步驟 4.** 依式 36-3 計算：若目標物以某速度移動，此延遲會造成多少位置誤差？並提出兩種改善方案（降低延遲與預測補償）。

活動二 觀察 ROS 的資料流

請在 ROS 環境（可使用模擬器，無實體機器人亦可）中觀察其通訊模型。

- 步驟 1.** 啟動一個包含多個節點的範例系統，列出所有正在運作的節點與話題。
- 步驟 2.** 訂閱其中一個感測話題，觀察訊息的內容、格式與發布頻率。
- 步驟 3.** 額外撰寫一個節點訂閱該話題並做簡單處理（如計算移動平均），驗證「新增訂閱者不影響發布者」的解耦特性。
- 步驟 4.** 刻意讓你的節點處理得很慢，觀察會發生什麼——訊息是被排隊還是被丟棄？這與第 25 章的訊息佇列有何不同？

活動三 設計一個機器人的資料架構

請為一個自主移動機器人的車隊（假設十台，運作於一座工廠）設計完整的資料架構。首先區分 36.2.3 節的兩條路徑：哪些資料屬於控制路徑（必須本地即時處理）、哪些屬於資料路徑（可上傳後端分析）。接著針對資料路徑，說明要上傳什麼、頻率為何、如何處理網路中斷時的暫存與補傳（第 25 章）。然後設計地圖的共享機制：儲存在哪裡、如何更新、衝突如何處理（第 7 章）。最後說明：你會蒐集哪些資料用於改善機器人的行為？以及如何確保這些資料在一年後仍可用於重新訓練（第 34、35 章）？

延伸閱讀

- **ROS 官方教學與概念文件**：說明節點、話題、服務與動作的通訊模型與實作方式，是 36.2 節的實作參考。
- **《Probabilistic Robotics》**：Thrun 等人著，是感測融合與 SLAM 的權威教科書，可深化 36.3、36.4 節的數學基礎。
- **協作型機器人的安全標準文件**：說明人機協作的的安全要求、風險評估方法與驗證程序，是 36.5 節的規範依據。
- **工業數位分身的實務案例研究**：各產業導入數位分身的經驗與教訓，可作為 36.6 節的實務補充。

本章小結

1. 機器人系統與一般資料系統有四個根本差異：時間要求（超時即失效）、資料型態（訊號與影像量極大）、失敗後果（不可逆且可能危及人員）、以及回饋迴路（分析結果直接驅動實體動作）。
2. 即時分硬即時（絕不可超過上限，如安全連鎖與伺服控制）與軟即時（偶爾超時只是品質下降，如視覺辨識）；硬即時應交由專用控制器，軟即時才適合放在一般平台上——此即分層架構的理由。
3. ROS 以節點與話題構成發布訂閱模型，其解耦價值與第 25 章相同；但它追求低延遲而非可靠送達，故通常不保留訊息且允許丟失——這說明「可靠性」與「即時性」有時衝突，技術原則須依情境調整。
4. 機器人系統有兩條並行的資料路徑：控制路徑追求最低延遲且須離線可運作，資料路徑則可容忍延遲並支援批次補傳。故機器人的控制不需要雲端，但學習與改善需要。
5. 感測融合以加權結合多種感測器，使優點互補、盲點互掩；其資料工程難題主要在「時間對齊」與「座標轉換」，這兩項看似瑣碎卻最常出錯。
6. SLAM 是典型的循環依賴問題（定位需地圖、建圖需定位），其解法與第 18 章 PageRank 同樣採迭代逼近；資料層面的挑戰包含資料量龐大、即時性、誤差累積與地圖維護，而回環偵測本質上是第 14 章的相似性比對問題。
7. 協作型機器人的安全要求包含感知可靠性、延遲上限、失效安全與可追溯性；其中「失效安全」與一般資訊系統「盡量維持服務」的思維相反——因為代價結構不同（服務中斷 vs 人員受傷）。
8. 數位分身須持續與實體同步，其關鍵挑戰在於範圍的界定——應從具體問題出發建立剛好足夠的分身，而非追求全面精細；智慧製造的整合分連線、可視、分析、優化、自主五個層次，層次越高前提越嚴苛。
9. 控制迴路的時間預算為 $1,000/f$ 毫秒（式 36-1），端到端延遲為各環節之和（式 36-2），而延遲造成的位置誤差為 $v \times T/1,000$ （式 36-3）。當機器人「抓不準」時，應先量測延遲再乘上目標速度——若與觀察到的偏差相當，問題就在延遲而非精度。

習題

一、觀念題

1. 請說明機器人系統與一般資料系統的四個根本差異，並解釋為何「回饋迴路」這一項最為關鍵。
2. 為什麼 ROS 的話題「允許訊息丟失」反而是正確的設計？請與第 25 章的訊息佇列比較說明。
3. 請說明 SLAM 為何是「先有雞還是先有蛋」的問題，以及它與第 18 章 PageRank 在解法上的相通之處。
4. 何謂「失效安全」？為何它與一般資訊系統追求可用性的思維相反？

二、計算題

1. 某視覺檢測系統的目標控制頻率為 50 赫茲。各環節耗時為：影像擷取 4 ms、前處理 2 ms、模型推論 9 ms、後處理 1 ms、通訊 2 ms、執行 3 ms。（一）依式 36-1 計算時間預算；（二）依式 36-2 計算端到端延遲；（三）判斷是否可行，並計算餘裕與最大瓶頸環節的佔比。
2. 承上題，若產品在輸送帶上以 40 公分／秒移動。（一）依式 36-3 計算現有延遲造成的位置誤差；（二）若模型推論經量化後降為 3 ms，新的延遲與誤差各為何？（三）若要把誤差控制在 0.5 公分以內，端到端延遲需低於多少？

三、思考與討論

1. 案例四中，操作員因干擾過大而關閉了安全功能。請討論：在設計涉及安全的系統時，如何在「安全」與「不被繞過」之間取得平衡？
2. 本章指出「技術原則不是普世的教條」。請回顧全書，再舉出至少兩個「在不同情境下，正確答案相反」的技術判斷。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

四個差異：時間要求（一般系統延遲高只是等久，機器人超過上限即失效甚至危險）、資料型態（一般多為結構化紀錄，機器人為感測訊號、影像與點雲，量極大）、失敗後果（一般重跑即可，機器人可能撞壞設備或傷人，不可逆）、回饋迴路（一般是分析結果供人判讀，機器人是直接驅動實體動作）。「回饋迴路」最關鍵的原因：在一般系統中，人會用常識過濾明顯錯誤的分析結果，形成一道天然的防線；但在機器人系統中，模型輸出直接成為馬達指令，機器會忠實執行錯誤的判斷，中間沒有任何人為的檢查。這也是第 22 章強調「人在迴路中」的理由——當輸出會改變物理世界時，錯誤的代價與資訊世界完全不同。

第 2 題

因為兩者的設計目標不同。第 25 章的訊息佇列追求「可靠送達」，其資料（如交易事件）每一筆都有獨立價值，丟失即造成資料不完整，故採至少一次語意並以重試確保送達。而 ROS 話題傳遞的是高頻的感測資料（如 30 赫茲的影像或雷射掃描），單筆資料的價值低且很快會被新資料取代——丟掉一幀，33 毫秒後就有新的一幀。反之，若堅持每幀都要可靠送達，重試機制反而會增加延遲並造成資料堆積，使系統處理的永遠是「過時的資料」，這在即時控制中是更嚴重的問題。故此處「盡快送達、丟了就算」才是正確設計。這說明可靠性與即時性有時衝突，原則須依情境權衡。

第 3 題

循環依賴：要知道自己的位置，需要有地圖作為參照；但要建立地圖，又需要先知道自己在哪裡，才能把觀測到的特徵放在正確位置。兩者互為前提，無法先完成其中之一。與 PageRank 的相通之處：第 18 章的 PageRank 同樣是循環定義——網頁的重要性由連入它的網頁之重要性決定。兩者的解法在精神上一致：不試圖直接求解，而是從一個初始估計出發（PageRank 給所有網頁相同的初始值，SLAM 給一個初始位姿估計），再隨著新資訊不斷到來而反覆修正，使結果逐步收斂到穩定解。

這說明不同領域的問題，骨子裡可能是同一類數學結構——這也是本書反覆強調「原理會遷移」的實例。

第 4 題

失效安全指：當感測器故障或系統異常時，系統的預設行為必須是「停止」而非「繼續運作」。與一般思維相反的原因在於「代價結構不同」。一般資訊系統中，服務中斷的代價是使用者無法使用、業務停擺，故第 3 章談的所有容錯設計都在追求「即使部分故障也要維持服務」。但在人機協作的安全系統中，若感測器故障而系統仍繼續動作，可能造成人員受傷——這個代價不可逆且無法以任何效率補償。故正確的設計是寧可保守停機（造成產能損失），也不可在不確定的狀態下繼續運作。核心觀念：技術原則不是普世教條，而是針對特定代價結構的權衡；判斷「這個情境的代價結構是什麼」，比記住原則更重要。

二、計算題

第 1 題

目標 50 赫茲，各環節耗時 4、2、9、1、2、3 毫秒。

- (1) 時間預算（式 36-1） $= 1,000 / 50 = 20$ 毫秒。
- (2) 端到端延遲（式 36-2） $= 4 + 2 + 9 + 1 + 2 + 3 = 21$ 毫秒。
- (3) 判斷：21 毫秒已超過 20 毫秒的預算，故不可行（餘裕為 -1 毫秒）。最大瓶頸為模型推論的 9 毫秒，佔預算的 45%。

延伸：應優先優化模型推論（依第 21、35 章的量化與推論最佳化）。若能將其降至 5 毫秒，端到端變為 17 毫秒，即可在 20 毫秒的預算內留下 3 毫秒的餘裕。

第 2 題

$v = 40$ 公分／秒，依式 36-3 計算。

- (1) 現有延遲 21 毫秒：位置誤差 $= 40 \times 21 / 1,000 = 0.84$ 公分。
- (2) 推論降為 3 毫秒後：端到端 $= 4 + 2 + 3 + 1 + 2 + 3 = 15$ 毫秒；位置誤差 $= 40 \times 15 / 1,000 = 0.6$ 公分。此時亦符合 20 毫秒的預算，餘裕 5 毫秒。
- (3) 要求誤差 ≤ 0.5 公分：由 $40 \times T / 1,000 \leq 0.5$ 得 $T \leq 12.5$ 毫秒。

延伸思考：即使把推論降到 3 毫秒，端到端 15 毫秒仍略高於 12.5 毫秒的要求。此時有兩條路：繼續壓縮其他環節（如降低影像解析度以縮短擷取時間），或採用預測補償——依輸送帶的已知速度預測產品在動作時刻的位置，主動加上提前量。後者在延遲已難再壓縮時特別有效，因為它不需要讓時間變快，只需要預測它（如案例一的最終解法）。

三、思考與討論

第 1 題

平衡的做法可包括：其一，把現場人員納入設計——在設定感測範圍與觸發條件時，實地觀察正常作業流程，確保安全機制只在真正的風險情境觸發，而非在正常動作時頻繁誤觸。其二，分級反應而非一律停機——依距離分級（遠：正常運作；中：減速；近：停止），使多數情況下只是溫和的減速而非中斷。其三，降低遵守的代價——若安全機制造成的效率損失夠小，人就沒有繞過的動機；反之若代價過高，再嚴格的規定也會被規避。其四，建立回饋管道並確實處理——案例四中操作員曾反映而未被處理，這才是問題惡化的關鍵。其五，監控「安全功能是否被停用」並納入告警——把繞過行為變成可見的事件，而非默默發生。其六，教育與說明——讓使用者理解機制的目的與風險，而非

只是被規定。核心觀念：安全設計若與實際工作流程衝突，最終被犧牲的往往是安全；故人因考量本身就是安全設計的一部分，而非額外的軟性議題。

第 2 題

可舉的例子包括：其一，故障時的預設行為——一般系統應「盡量維持服務」（第 3 章的容錯），安全系統應「立即停止」（本章 36.5.2 節）；差異在於代價結構。其二，訊息傳遞的可靠度——業務事件應確保至少一次送達（第 25 章），高頻感測資料則寧可丟失也不重試（本章 36.2.2 節）；差異在於單筆資料的價值與時效。其三，資料的保留——資料湖主張「先收下再說」（第 30 章），個資保護則要求「最小蒐集、目的限制」（第 33 章）；差異在於是否涉及他人權利。其四，一致性的取捨——金融交易須強一致（第 3 章的 CP），社群動態可接受最終一致（AP）；差異在於錯誤的可回復性。其五，自動化的程度——一般部署追求全自動（第 29、34 章），而涉及不可逆動作時應保留人工確認（第 22 章）；差異在於錯誤是否可回復。共同結論：技術判斷的答案取決於情境的代價結構——包括錯誤的可逆性、影響的範圍、以及各種失效模式的相對代價。工程判斷的核心能力，正是辨識這個結構，而非套用通則。

台灣自編大學教科書 | 機器人叫獸 @prof | 第 36 章 機器人與智慧製造整合

台灣自編大學教科書

@prof

大數據分析與雲端運算

第玖篇 整合實務與應用

Integration & Applications

第 37 章

資料視覺化與決策支援

Data Visualization and Decision Support

機器人叫獸（李世淵） 編著

叫獸開講

我看過一個做得非常精美的產線儀表板：三十幾個指標、六種顏色、動態的儀表盤、還有會旋轉的三維長條圖。老闆看了很滿意，說「這才叫數位轉型」。

三個月後我再去，那個儀表板掛在牆上，沒有人在看。

我問領班為什麼，他的回答很實在：「上面東西太多了，我不知道該看哪個。而且看了也不知道要幹嘛——它告訴我良率掉了，但沒告訴我該調哪台機器。」

這句話點出了視覺化最常被誤解的地方。多數人以為視覺化的目的是「把資料呈現出來」，但真正的目的是「讓人做出更好的決定」。這兩者的差距，就是那面沒人看的牆與一個真正有用的工具之間的距離。

這一章是本書技術鏈路的最後一哩路。前面三十六章我們談了如何蒐集、傳輸、儲存、運算、分析資料——但如果分析的結果沒有人看得懂、或看懂了卻不知道該做什麼，那麼前面所有的努力都白費了。而這一哩路，靠的不只是技術，還有對人的理解。

學習目標

研讀完本章之後，你應該能夠：

1. 說明視覺化的目的，並區分探索式與解釋式視覺化。
2. 運用視覺編碼的原則，選擇合適的圖表類型。
3. 辨識常見的誤導性圖表，並避免在自己的作品中犯錯。
4. 說明儀表板的設計原則，並依使用者角色規劃內容。
5. 說明從指標到行動的設計，使分析結果能驅動決策。
6. 運用資料墨水比等原則，評估與改善視覺化的品質。

核心概念

核心概念	說明
探索式視覺化	分析者用來自己理解資料、發現模式的工具。
解釋式視覺化	用來向他人傳達已知結論的成果。
視覺編碼	把資料的數值映射為位置、長度、顏色等視覺屬性。
資料墨水比	圖表中用於呈現資料的部分佔全部視覺元素的比例。
儀表板	把關鍵指標集中呈現、供持續監看的介面。
可行動性	指標能否直接指向具體的應對措施。
決策支援	以資訊協助人做出更好判斷的系統與流程。

表 37-1 本章核心概念一覽

37.1 視覺化的目的

37.1.1 為什麼要視覺化

視覺化的存在，源於人類認知的一個基本特性：我們處理圖形的能力遠強於處理數字。

一張含有一千個數字的表格，人幾乎無法從中看出任何模式；但把同樣的資料畫成散佈圖，趨勢、群聚、異常值往往一眼可見。這不是因為資料變了，而是因為它被轉換成了人腦擅長處理的形式。

因此視覺化的本質是「一次翻譯」——把機器擅長的形式（數字），翻譯成人擅長的形式（圖形）。而翻譯的品質，決定了資訊能否被正確接收。

37.1.2 兩種截然不同的視覺化

這是本章最重要的區分之一，因為兩者的設計原則幾乎相反。

面向	探索式	解釋式
目的	自己理解資料、發現模式	向他人傳達已知的結論
受眾	分析者本人	決策者、同事、客戶
數量	大量、快速、可拋棄	少量、精緻、反覆打磨
重點	廣度——盡量多看幾個角度	深度——把一個訊息說清楚
美觀	不重要，能看懂就好	重要，影響訊息的接收

表 37-2 探索式與解釋式視覺化的比較

實務上的常見錯誤，是把兩者混為一談：有人在探索階段就花大量時間美化圖表（浪費時間，因為多數圖表看過就丟）；也有人把探索時的粗糙圖表直接放進簡報（受眾看不懂，因為缺少必要的說明與聚焦）。

正確的做法是分階段：探索時快速產出大量粗糙的圖，找到值得說的故事；然後才針對那一兩個發現，精心製作解釋式的圖表。

37.2 視覺編碼與圖表選擇

37.2.1 視覺屬性的精確度階梯

視覺化的核心是「編碼」——把資料的數值映射為某種視覺屬性。而不同的視覺屬性，人眼判讀的精確度差異極大。

精確度	視覺屬性	適用情境
最高	共同座標軸上的位置	需要精確比較的數值，如折線圖、散佈圖
高	長度	類別間的數量比較，如長條圖
中	角度、面積	整體中的比例，但比較不精確
低	顏色深淺、飽和度	呈現程度或密度的概略分布
最低	色相（不同顏色）	區分類別，不宜表達數量

表 37-3 視覺屬性的判讀精確度

這個階梯有一個直接的實務推論：需要精確比較的資料，應優先使用位置或長度來編碼。這解釋了為何長條圖通常優於圓餅圖——判斷長度的差異比判斷角度容易得多。

同樣地，用顏色深淺來表達數量（如熱力圖）雖然直觀，但人眼難以判斷「這個色塊比那個深多少」；它適合看整體的分布樣態，不適合精確比較。

37.2.2 依問題選圖表

選擇圖表的正確順序，是先確定「你想回答什麼問題」，而非「哪個圖表比較好看」。

你的問題	適合的圖表	注意事項
隨時間如何變化	折線圖	時間應在橫軸；避免過多線條
各類別如何比較	長條圖	考慮排序；避免立體效果
兩個變數的關係	散佈圖	點過多時可用透明度或抽樣
整體的組成比例	堆疊長條或圓餅圖	類別過多時難以判讀
數值的分布	直方圖、箱型圖	注意分組寬度的選擇
地理上的分布	地圖	注意面積大小造成的視覺偏誤

表 37-4 依問題類型選擇圖表

這裡我要特別提醒「數值的分布」這一系列。前面各章我們反覆強調過（第 3 章的 p99、第 32 章的最弱維度、第 33 章的最小組），平均值會掩蓋極端值。而在視覺化上，直方圖與箱型圖正是把「分布」而非「單一數字」呈現出來的工具——它們讓被平均掩蓋的東西重新可見。

37.3 常見的誤導與如何避免

37.3.1 六種常見的誤導

視覺化有一個危險的特性：它比文字更有說服力，因此錯誤或誤導也更容易被接受。以下是最常見的幾種。

- **座標軸不從零開始：**長條圖的縱軸若從非零處開始，會使微小的差異看起來很巨大。折線圖有時可接受（因為重點是趨勢），但長條圖幾乎不應如此。
- **雙軸的誤導：**把兩個不同單位的資料放在同一張圖的左右兩軸，可以透過調整比例讓它們「看起來高度相關」——而這個相關可能純屬人為製造。
- **刻意選擇的時間範圍：**只呈現對自己結論有利的那一段時間，隱藏了更長期的趨勢。
- **面積與體積的錯誤：**用圓形或立體圖形表達數量時，若把數值映射為半徑而非面積，視覺上的差距會被誇大數倍。
- **過度的裝飾：**三維效果、陰影、漸層會扭曲對長度與面積的判斷，且分散注意力。
- **不當的比較基準：**呈現絕對數量而非比率（如未考慮人口或產量的差異），使比較失去意義。

叫獸提醒

我要在這裡說一段話。上面這六種手法，你既可以把它們當成「要避免的錯誤」，也可以把它們當成「說服人的技巧」——而這正是我要特別提醒的地方。做視覺化的人握有相當大的權力：同一份資料，你可以讓它看起來危機四伏，也可以讓它看起來一切安好，而多數受眾沒有能力察覺其中的差別。這與第 19、33 章談的責任是同一件事：專業能力越強，就越有能力誤導。我的建議很簡單——把「如果對方完全懂視覺化，他會不會覺得我在誤導」當成自我檢核的標準。若答案是肯定的，那就是誤導，無論你是有意還是無意。

37.3.2 資料墨水比

一個實用的品質原則是「資料墨水比」：圖表中，真正用於呈現資料的視覺元素，佔全部視覺元素的比例應盡量高。

換言之，背景格線、外框、陰影、漸層、圖示裝飾——這些都不承載資料，卻佔用了讀者的注意力。移除它們，資訊反而更清楚。

這與其說是美學，不如說是認知效率的問題：讀者的注意力有限，每一分花在裝飾上的注意力，都是從理解資料上扣除的。下一節的公式會示範如何量化這件事。

37.4 儀表板的設計

37.4.1 為什麼儀表板容易失敗

回到叫獸開講那面沒人看的牆。儀表板失敗的原因，通常不是技術問題，而是設計問題。

- **指標過多**：三十個指標意味著每個平均只能分到 3% 的注意力——結果是沒有一個被真正看見。
- **沒有優先序**：所有指標以同樣的大小與顯著度呈現，讀者無從判斷哪個重要。
- **缺乏基準**：只呈現「目前是 87%」而沒有「目標是多少、昨天是多少、正常範圍在哪」，數字本身無法解讀。
- **不知道該做什麼**：指標顯示異常，但沒有指向任何具體的行動——這是最根本的失敗。
- **未考慮使用情境**：在辦公室電腦上設計、卻掛在產線的大螢幕上讓五公尺外的人看，字級與資訊密度完全不合。

37.4.2 依角色設計

一個關鍵的認知是：不同角色需要的資訊完全不同，不應該用同一個儀表板服務所有人。

角色	關心什麼	適合的呈現
現場操作員	現在有沒有問題、我該做什麼	極少數指標、明確的狀態與指示
班組長／工程師	問題出在哪、趨勢如何	可下鑽的細節、與基準的比較
部門主管	整體表現、資源該往哪配置	彙總指標、跨線或跨期的比較
高階管理	策略性的趨勢與異常	極少數關鍵指標、長期趨勢

表 37-5 不同角色的儀表板需求

注意「現場操作員」與「高階管理」都需要「極少數指標」，但原因不同：前者是因為要在忙碌中快速判斷，後者是因為要看見全局而非細節。中間的角色則需要較多的細節與下鑽能力。

這個分層設計，其實與第 35 章邊雲協同的三層分工在精神上相通——每一層只呈現該層需要的粒度，而非把所有資料塞給所有人。

37.4.3 好儀表板的檢核

我建議用以下幾個問題來檢核一個儀表板的設計。

- **五秒鐘測試**：使用者能否在五秒內判斷「現在是正常還是異常」？若不能，代表視覺層次不夠清楚。
- **行動測試**：當某個指標變紅時，使用者知道該做什麼嗎？若不知道，這個指標的價值有限。

- **基準測試：**每個數字旁邊是否有可比較的參照（目標、歷史、正常範圍）？
- **刪除測試：**若移除某個指標，會有人抱怨嗎？若不會，就該移除。

最後那個「刪除測試」是我最喜歡的一個，因為它直接對抗了儀表板最常見的病症——不斷累加而從不移除。每個人都覺得「多加一個指標又不會怎樣」，於是它慢慢變成了那面沒人看的牆。

37.5 從指標到行動

37.5.1 可行動性

承接領班那句「看了也不知道要幹嘛」。一個指標若無法指向具體的行動，它的價值就相當有限。

判斷可行動性的方法很簡單：問「當這個數字變差時，誰要做什麼？」若答案是「不知道」或「再研究看看」，那麼這個指標可能只是「知道了會安心」而非真正有用。

提升可行動性的做法有幾種：把彙總指標拆解到可操作的層級（不是「良率下降」而是「三號機的某個參數偏離」）、附上建議的處置（「檢查刀具磨耗」）、以及提供下鑽的路徑（讓使用者能從異常追到原因）。

37.5.2 告警的設計

第 13 章我們談過告警疲勞，這裡從視覺化的角度補充幾點。

告警必須分級：不是所有異常都需要立刻處理。實務上常分為「需立即處置」「需今日內處理」「僅供參考」三級，並以不同的呈現方式區隔——只有第一級才使用醒目的紅色與主動通知。

告警也必須可關閉與可調整：當使用者反映某個告警過於頻繁時，應該有管道調整門檻，而非讓他們自行想辦法忽略——第 36 章案例四那個被關掉的安全功能，正是這個問題的極端後果。

最後，告警應該包含「情境」：不只說「溫度過高」，而要說「溫度 85 度，超過警戒值 80 度，過去一小時持續上升，上次出現類似情況是三週前的軸承故障」。這些額外的資訊，決定了使用者能否快速判斷該如何反應。

37.5.3 決策支援而非決策取代

這一節我想談一個立場問題。

資料系統的角色，多數時候應該是「支援決策」而非「取代決策」——因為資料能提供的只是「已被量化的部分」，而決策往往還需要考慮那些未被量化的因素：客戶關係、員工士氣、法規風險、以及大量無法寫進資料庫的現場知識。

這也是為什麼好的視覺化不只呈現結論，還應呈現「不確定性」與「依據」——讓使用者知道這個數字有多可信、是基於什麼樣本、以及在什麼情況下可能失準。一個只給結論而不給依據的系統，會讓使用者要嘛盲目相信、要嘛完全不信，兩者都不理想。

這與第 22 章談 AI 代理時的「人在迴路中」、第 33 章談的倫理責任，是同一個立場：技術的角色是讓人做出更好的判斷，而非替人做判斷。

37.6 說故事與溝通

37.6.1 資料故事的結構

當你要向他人傳達分析結果時，單純呈現圖表往往不夠——你需要一個能被理解與記住的結構。

一個有效的結構通常包含四個部分：情境（我們原本面對什麼問題）、發現（資料告訴我們什麼）、意涵（這代表什麼、為什麼重要）、以及建議（因此我們該做什麼）。

多數技術背景的人容易犯的錯誤，是花 90% 的篇幅講「我怎麼做的」（方法、模型、參數），而只用 10% 講「所以呢」。但對受眾而言，比例應該恰好相反——他們關心的是結論與行動，方法只需要足以建立信任即可。

這不是要你隱瞞方法，而是要調整順序與比重：先講結論，再視需要提供方法的細節。

37.6.2 面對不同的受眾

同一份分析，對不同受眾要有不同的說法。

對技術同儕，可以直接討論方法的細節與限制；對業務單位，要用他們的語言（不是「模型的 F1 分數 0.87」，而是「大約每十件不良品能抓到八到九件」）；對高階管理，則要聚焦於決策相關的層面（影響多少、需要多少投入、風險為何）。

有一個實用的練習：把你的結論用一句話說完，且這句話裡不能有任何技術名詞。若做不到，通常代表你自己還沒想清楚結論是什麼。

37.6.3 誠實地呈現不確定性

最後這一點與本書的一貫立場一致。

分析結果幾乎總是帶有不確定性——樣本有限、模型有誤差、資料有品質問題、且未來未必與過去相同。誠實地呈現這些，不會削弱你的專業性，反而會建立長期的信任。

實務上的做法包括：呈現信賴區間或範圍而非單一數字、明確說明分析的假設與限制、以及在資料不足以支持某個結論時直說「這一點目前的資料無法回答」。

我要提醒的是相反的做法有多危險：若你為了讓簡報好看而隱藏了不確定性，一旦預測失準，失去的不只是那一次的信任，而是往後所有分析的可信度。

37.7 視覺化在整體架構中的位置

37.7.1 最後一哩路

讓我們回到全書的脈絡。前面各章建立了完整的技術鏈路：資料的產生（第 35 章）、傳輸（第 25 章）、儲存（第 5、30 章）、運算（第參篇）、分析（第肆篇）、模型（第伍篇）、治理與維運（第捌篇）。

而視覺化是這條鏈路的最後一哩——它決定了前面所有的投資能否轉化為實際的價值。一個技術上完美但沒有人使用的系統，其價值為零。

這也給了一個實務上的建議：在專案的規劃階段就應該想清楚「最終的使用者會看到什麼、據此做什麼決定」，並從這個終點反推需要什麼資料與分析。從終點反推，比從資料出發更容易做出有用的系統。

37.7.2 技術實作的考量

從系統架構的角度，視覺化層也有幾個技術考量值得注意。

- **查詢效能：**儀表板通常需要秒級的回應。若每次載入都掃描原始資料會太慢，故常預先計算彙總結果（第 30 章的金屬）。
- **更新頻率：**並非所有指標都需要即時更新。應依實際需求設定，避免不必要的運算成本（第 27 章）。
- **權限控制：**不同角色能看到的資料範圍不同，視覺化層必須落實第 33 章的存取控制，而非只在資料層把關。
- **行動裝置：**現場人員往往使用手機或平板查看，設計時須考慮螢幕尺寸與觸控操作。

37.7.3 承先啟後

本章我們走完了最後一哩路。從視覺化的目的與兩種類型的區分出發（37.1），認識了視覺編碼的精確度階梯與圖表選擇（37.2）、六種常見的誤導與資料墨水比（37.3）、儀表板為何失敗與依角色設計的原則（37.4）、從指標到行動的可行性（37.5）、說故事的結構與誠實呈現不確定性（37.6），最後把視覺化定位在整體架構中（37.7）。

我最希望你帶走的，是那位領班的話：「看了也不知道要幹嘛。」視覺化的目的不是呈現資料，而是讓人做出更好的決定。每次設計時，都應該回頭問一次：使用者看了這個，會做什麼？

到這裡，本書的知識架構已經完整。從第 1 章的大數據概論，到本章的視覺化與決策支援，你已經走過了完整的資料生命週期與技術堆疊。而最後一章，我們要把這一切整合起來——透過一個完整的專案實作，讓你親手把這些技術組裝成一個能運作的系統。因為真正的學會，是做出來的，不是讀出來的。

公式與流程：資料墨水比與注意力分配

這一節用兩組簡單的計算，把「圖表好不好」與「儀表板該放幾個指標」這兩件常憑感覺決定的事，變成可以評估的判斷。

資料墨水比

設圖表中所有視覺元素的量為 E （可以用面積、筆畫或元素個數粗略估計），其中真正用於呈現資料的部分為 D ，則資料墨水比定義為：

$$\text{資料墨水比} = D / E$$

式 37-1 資料墨水比

這個比例應盡量接近 1——意即圖表中的每一分視覺元素，都在傳達資訊。下表以一個長條圖的逐步簡化為例。

設計階段	資料墨水比	改善	移除了什麼
原始設計	0.38	—	含三維效果、背景圖、陰影
移除背景與陰影	0.55	+45%	背景圖案、立體陰影、外框
簡化座標軸與圖例	0.72	+31%	多餘刻度、重複的圖例
去除多餘裝飾	0.88	+22%	格線、邊框、色彩漸層

表 37-6 逐步簡化的資料墨水比改善

值得注意的是：整個過程中，圖表所傳達的「資訊」完全沒有減少——減少的只是干擾。這正是視覺化設計最反直覺的一點：多數情況下，改善的方法是「拿掉東西」而非「加上東西」。

注意力的分配

儀表板的設計則面對另一個限制：人的注意力是有限且會被均分的。設儀表板上有 n 個同等顯著的指標，則每個指標平均能分到的注意力為：

$$\text{單一指標的注意力} \approx 1 / n$$

式 37-2 注意力的平均分配

指標數 n	平均注意力	實務判讀
3	33.3%	每個都能被看見與記住
5	20.0%	接近人短期記憶的上限
8	12.5%	開始需要刻意搜尋
15	6.7%	多數指標會被忽略
30	3.3%	等同於沒有重點（如叫獸開講的案例）

表 37-7 指標數量與注意力分配

叫獸提醒

式 37-2 的假設是「所有指標同等顯著」——而這正是問題所在。解方不是把指標數硬性限制在五個以內（實務上常常做不到），而是「建立視覺層次」：讓真正重要的一兩個指標在大小、位置與色彩上明顯突出，其餘的則退為背景，需要時才去看。如此即使畫面上有二十個數字，注意力仍會先落在該落的地方。這也是為什麼 37.4.3 節的「五秒鐘測試」如此有效——它直接檢驗了你的視覺層次是否成功。順帶一提，這個「重要的東西要顯著、次要的要退讓」的原則，與第 13 章的告警分級是同一件事：當所有東西都很重要時，就等於沒有東西重要。

案例研討

案例一 沒人看的那面牆

回到叫獸開講。那個儀表板有三十幾個指標，依式 37-2 每個平均只能分到約 3.3% 的注意力——實質上等於沒有重點。加上大量的三維效果與動態元素，資料墨水比極低，讀者的注意力被裝飾大量佔用。

更根本的問題是領班那句話：「看了也不知道要幹嘛。」指標缺乏可行動性——它告訴你良率掉了，卻沒有指向任何具體的處置。

重新設計的版本只保留了三個主指標（當班良率、設備稼動、待處理異常），以大字級呈現於畫面上方，並各自附上目標值與趨勢；其餘指標收進下鑽的頁面。異常項目則直接顯示「哪一台機器、什麼問題、建議檢查什麼」。

此後現場人員開始真正使用它。這個案例同時體現了三個原則：控制指標數量、建立視覺層次、以及最重要的——讓每個指標都能指向行動。

案例二 被截斷的縱軸

某次會議中，一份簡報以長條圖呈現三個廠區的良率，圖上第三廠的長條明顯比其他兩廠矮了一大截，看起來問題嚴重。與會者據此討論了半小時該如何整頓第三廠。

直到有人問了縱軸的範圍——原來縱軸從 94% 開始，而三個廠的實際良率分別是 96.2%、96.0% 與 95.1%。差距只有一個百分點多，視覺上卻被放大成數倍的落差。

製作簡報的人並非有意誤導，只是使用了繪圖工具的預設設定（許多工具會自動縮放座標軸以「凸顯差異」）。但後果是實際的——半小時的討論建立在一個被扭曲的印象上。

這正是 37.3.1 節的第一種誤導。長條圖的縱軸應從零開始，因為讀者判讀的是「長度的比例」；若要凸顯細微差異，應改用折線圖或明確標註實際數值。這個案例也提醒我們：工具的預設值不必然正確的選擇。

案例三 每個人都想加一個指標

某團隊的儀表板上線時只有六個指標，設計簡潔清楚。但隨著各部門陸續提出需求——「能不能也加上我們部門關心的這個數字」——一年後累積到了二十七個。

每一次的新增看起來都很合理（多一個又不會怎樣），但累積的結果是原本清楚的畫面變得擁擠，連最初的六個核心指標都難以在第一眼被找到。

他們後來執行了 37.4.3 節的「刪除測試」：暫時移除每一個指標，觀察是否有人反映。結果有十一個指標整整一個月無人提及，於是被移出主畫面（收進次頁）。

這個案例呈現了一個組織現象：儀表板天生會膨脹，因為「加上」的阻力遠小於「移除」。因此必須有制度性的機制定期檢視與精簡——這與第 33 章的權限盤點是同一種紀律：定期清理比嚴格把關更實際。

案例四 一句話說不清的分析

某工程師花了三週完成一項設備健康的分析，做了完整的簡報：資料前處理的流程、特徵工程的細節、三種模型的比較、超參數的調校過程，最後三頁才是結論。

簡報進行到第十五分鐘時，主管打斷問：「所以我們現在該做什麼？」而他一時答不上來——因為他的重點放在「我怎麼做的」，而非「所以呢」。

他後來重做了一次簡報，第一頁就是：「三號機的軸承在未來六週內故障的機率偏高，建議在下次歲修時更換，預估可避免約兩天的非計畫停機。」方法的細節則放在附錄供追問。

同樣的分析、同樣的結論，但這次五分鐘就取得了決策。這正是 37.6.1 節的教訓——對受眾而言，結論與行動才是主體，方法只需足以建立信任。這也是許多技術背景的人最需要練習的一項能力。

實作活動

活動一 改造一張圖表

請找出一張品質不佳的圖表並改造它。

步驟 1. 從新聞、報告或你自己過去的作業中，找出一張設計不良的圖表（裝飾過多、編碼不當或有誤導）。

步驟 2. 指出它的問題：使用了什麼視覺編碼？依表 37-3 判斷是否恰當？是否含 37.3.1 節的任一種誤導？

步驟 3. 依式 37-1 粗略估計其資料墨水比，並逐步簡化，記錄每一步移除了什麼、比例如何改善。

步驟 4. 重新製作一張圖表，並請一位同學比較兩者——他能否更快、更正確地讀出資訊？

活動二 設計分角色的儀表板

請為一個情境（可用第 35、36 章的智慧製造場景）設計儀表板。

- 步驟 1.** 選定兩種角色（如現場操作員與部門主管），分別列出他們真正需要知道的資訊，並說明理由。
- 步驟 2.** 為每種角色設計一份儀表板草圖，控制指標數量並建立明確的視覺層次（哪個最大、最上方）。
- 步驟 3.** 為每個指標標註：目標值或正常範圍為何？異常時使用者該做什麼？
- 步驟 4.** 以 37.4.3 節的四項測試（五秒鐘、行動、基準、刪除）檢核你的設計，並記錄修正。

活動三 用一句話說完你的分析

請針對你做過的任一項分析（可用前面章節的實作活動成果），練習向不同受眾說明。首先，用一句話說完你的結論，且句中不得出現任何技術名詞——若做不到，代表你尚未把結論想清楚。接著依 37.6.1 節的四段結構（情境、發現、意涵、建議）寫出一份不超過一頁的說明。然後為三種受眾各調整一個版本：技術同儕、業務單位、高階管理，說明你各自調整了什麼。最後，在說明中明確加入「這項分析的限制與不確定性」，並反思：加上這一段之後，你的結論是否變得比較不吸引人？若是，你會如何取捨？

延伸閱讀

- 《The Visual Display of Quantitative Information》：Tufte 的經典著作，資料墨水比等原則的源頭，是 37.3 節的理論依據。
- 《Storytelling with Data》：Knaflitz 著，以大量實例說明如何從探索走向解釋、如何以資料說故事，與 37.6 節高度相關。
- 《Information Dashboard Design》：Few 著，專門討論儀表板的設計原則與常見錯誤，是 37.4 節的完整延伸。
- 視覺編碼的認知研究文獻：關於人眼判讀各種視覺屬性之精確度的實證研究，是表 37-3 的科學基礎。

本章小結

1. 視覺化的本質是把機器擅長的形式（數字）翻譯為人擅長的形式（圖形）；其目的不是「呈現資料」，而是「讓人做出更好的決定」。
2. 探索式視覺化求廣度與速度、可拋棄且不必美觀；解釋式視覺化求深度與精準、需反覆打磨。兩者的原則幾乎相反，混為一談是常見的錯誤。
3. 視覺屬性的判讀精確度有明確的階梯：位置最高、長度次之，而顏色與角度較低。需要精確比較的資料應以位置或長度編碼，這解釋了長條圖為何通常優於圓餅圖。
4. 六種常見的誤導為：座標軸不從零、雙軸的比例操作、刻意選擇時間範圍、面積與體積的錯誤映射、過度裝飾、以及不當的比較基準。做視覺化的人握有相當的權力，應以「若對方完全懂視覺化，會不會覺得我在誤導」自我檢核。
5. 資料墨水比 D/E（式 37-1）應盡量接近 1；改善視覺化多數時候靠「拿掉東西」而非「加上東西」，因為每一分花在裝飾上的注意力，都是從理解資料上扣除的。
6. 儀表板失敗的原因通常是設計而非技術：指標過多、缺乏優先序與基準、不知該做什麼、未考慮使用情境。應依角色分層設計，而非以同一份服務所有人。

7. 注意力會被均分（式 37-2），故指標越多每個越不被看見；解方不是硬性限制數量，而是建立視覺層次讓重要的突出——當所有東西都重要時，就等於沒有東西重要。
8. 指標必須具備可行動性：當它變差時，應能回答「誰要做什么」。告警應分級、可調整、並包含足以判斷的情境資訊。
9. 資料系統的角色是支援決策而非取代決策，因為決策還需考量大量未被量化的因素；故好的視覺化應同時呈現結論、依據與不確定性，讓使用者能判斷該多信任它。
10. 向他人傳達分析時應以結論與行動為主體、方法為輔；並依受眾調整語言。誠實呈現不確定性不會削弱專業性，隱藏它才會在預測失準時摧毀長期的信任。

習題

一、觀念題

1. 請區分探索式與解釋式視覺化，並各舉一個把兩者混淆所造成的實際問題。
2. 為什麼長條圖通常優於圓餅圖？請以視覺屬性的判讀精確度說明。
3. 請列舉至少四種常見的誤導性圖表手法，並各說明其造成的錯誤印象。
4. 何謂「可行動性」？請說明如何判斷一個指標是否具備它，以及如何提升。

二、計算題

1. 某圖表的全部視覺元素量估計為 100 單位，其中用於呈現資料的部分為 32 單位。（一）依式 37-1 計算其資料墨水比；（二）若移除背景圖案（12 單位）、立體陰影（10 單位）與多餘格線（8 單位），新的資料墨水比為何？（三）改善了幾個百分點？
2. 某儀表板原有 24 個同等顯著的指標。（一）依式 37-2 計算每個指標平均分到的注意力；（二）若經「刪除測試」後精簡為 6 個主指標，新的平均注意力為何？提升幾倍？（三）若不刪除而改以視覺層次讓 3 個指標特別突出，你認為效果與（二）相比如何？請說明理由。

三、思考與討論

1. 本章的叫獸提醒指出，誤導性手法既可以是「要避免的錯誤」也可以是「說服的技巧」。請討論：在實務中，你會如何拿捏「有效傳達」與「操弄印象」之間的界線？
2. 案例三中，儀表板天生會膨脹。請討論：組織中還有哪些事物具有「加上容易、移除困難」的特性？該如何設計制度來對抗這種累積？

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

探索式：分析者自己用來理解資料、發現模式，追求廣度與速度，大量產出且可拋棄，不必美觀。解釋式：向他人傳達已知結論，追求深度與精準，數量少但需反覆打磨，美觀影響訊息接收。混淆的實際問題：其一，在探索階段就花大量時間美化圖表——浪費時間，因為多數探索圖表看過即丟，且過早美化會減少嘗試的次數。其二，把探索時的粗糙圖表直接放進簡報——受眾看不懂，因為缺少標題、標註、聚焦與必要的說明脈絡。正確做法是分階段：探索時快速產出大量粗糙的圖以找到值得說的故事，再針對那一兩個發現精心製作解釋式圖表。

第 2 題

依表 37-3 的判讀精確度階梯：長條圖以「長度」編碼數值，而長度的判讀精確度僅次於位置，屬於高精確度；圓餅圖則以「角度」或「面積」編碼，兩者的判讀精確度明顯較低。人眼比較兩條長條的長度相當準確，但比較兩個扇形的角度則困難得多，尤其當數值接近或扇形不相鄰時。因此當目的是「精確比較各類別的數量」時，長條圖較佳。圓餅圖的適用情境相當有限——類別數少（如二至三個）、且目的僅是呈現「大致的比例關係」而非精確比較時，才是合理的選擇。

第 3 題

四種以上：其一，座標軸不從零開始——使微小差異看起來像巨大落差（如案例二中一個百分點被放大為數倍）。其二，雙軸的比例操作——把兩個不同單位的資料放在左右兩軸，透過調整比例使其「看起來高度相關」，而該相關可能是人為製造的。其三，刻意選擇時間範圍——只呈現對結論有利的區間，隱藏更長期的相反趨勢。其四，面積與體積的錯誤映射——把數值映射為圓形的半徑而非面積，使視覺差距被誇大數倍。其五，過度裝飾——三維效果與陰影會扭曲對長度的判斷。其六，不當的比較基準——呈現絕對數量而未考慮規模差異（如未除以人口或產量），使比較失去意義。

第 4 題

可行動性指「指標能否直接指向具體的應對措施」。判斷方法：問「當這個數字變差時，誰要做什么？」若答案是「不知道」「再研究看看」，則該指標可能只是「知道了會安心」而非真正有用（如叫獸開講中領班的「看了也不知道要幹嘛」）。提升方法：其一，把彙總指標拆解到可操作的層級——不是「良率下降」而是「三號機的某項參數偏離」。其二，附上建議的處置——如「檢查刀具磨耗」。其三，提供下鑽路徑——讓使用者能從異常追到原因。其四，明確標示負責人與處理時限。核心原則：指標的價值不在於它多精確，而在於它能否驅動正確的行動。

二、計算題

第 1 題

$E = 100$ 、 $D = 32$ 。

- (1) 原始資料墨水比（式 37-1） $= 32 / 100 = 0.32$ 。
- (2) 移除 $12 + 10 + 8 = 30$ 單位的非資料元素後： $E = 100 - 30 = 70$ ， D 不變仍為 32；新比例 $= 32 / 70 \approx 0.457$ 。
- (3) 改善 $= 0.457 - 0.32 = 0.137$ ，即提升約 13.7 個百分點（相對改善約 43%）。

延伸思考：注意分子 D 完全沒有改變——圖表所傳達的資訊一分未減，改善全部來自「拿掉不必要的東西」。這正是視覺化設計最反直覺之處：多數情況下，改善的方法是移除而非增加。若要進一步提升，可再檢視格線、外框與圖例是否仍有精簡空間。

第 2 題

依式 37-2 計算注意力的平均分配。

- (1) $n = 24$ ：平均注意力 $= 1 / 24 \approx 4.2\%$ 。
- (2) $n = 6$ ：平均注意力 $= 1 / 6 \approx 16.7\%$ ，提升為原本的 4 倍。
- (3) 以視覺層次突出 3 個指標的效果：理論上更佳。因為式 37-2 的前提是「所有指標同等顯著」；一旦建立視覺層次，注意力不再被均分，而會優先落在突出的 3 個上（可能各佔 25% 以上），同時保留了其餘 21 個指標供需要時查閱——兼得聚焦與完整。

理由與取舍：刪除法簡單直接但會損失資訊（被刪的指標仍有人偶爾需要）；視覺層次法保留完整資訊且聚焦效果更好，但需要更用心的設計，且若層次做得不夠明確就會失效。實務上最佳做法是

兩者並用——先以刪除測試移除真正無人使用的，再對保留下來的建立明確層次（重要的放大置頂、次要的收進下鑽頁面）。

三、思考與討論

第 1 題

可提出的判準包括：其一，意圖檢驗——你調整這張圖，是為了「讓對方更容易看懂真實情況」還是「讓對方得出你想要的結論」？前者是傳達，後者是操弄。其二，透明度檢驗——你所做的處理（截斷座標軸、選擇時間範圍、使用比率而非絕對值）是否明確標示？若處理本身被隱藏，即使動機良善也接近誤導。其三，本章提出的檢核——「若對方完全懂視覺化，他會不會覺得我在誤導？」這是最實用的一條。其四，可逆檢驗——若把同一份資料以最中性的方式呈現，結論是否會改變？若會，代表你的呈現方式正在承擔不該由它承擔的說服工作。其五，一致性檢驗——你是否對支持與不支持自己結論的資料採用了不同的呈現標準？合理的做法是：可以「強調」（把重要的放大、加註說明、去除干擾），但不可「扭曲」（改變比例關係、隱藏不利資料、製造虛假的相關）。強調是幫助理解，扭曲是替代理解。

第 2 題

具有「加上容易、移除困難」特性的事物包括：其一，系統的功能與設定選項——每個都有人要求，但沒人負責檢視是否還被使用。其二，權限（第 33 章的殭屍權限）——授予時謹慎，收回時無人記得。其三，監控告警（第 13 章的告警疲勞）——每次事故後都新增規則，卻很少移除失效的。其四，會議與報表——設立時有明確目的，目的消失後仍持續舉行。其五，程式碼中的特殊處理與設定參數——為某個特例加上，之後無人敢刪。其六，資料欄位與資料集（第 30 章的資料沼澤）。共同成因：新增的效益歸屬明確（提出者受益）而成本分散（所有使用者共同承擔），移除則相反——效益分散而風險集中於提出移除的人。制度性的對抗方式：其一，設定到期日，需要則主動續期（如第 33 章的權限做法）。其二，定期強制盤點與檢視（如案例三的刪除測試、第 33 章的權限盤點）。其三，明訂「新增必須伴隨移除」的配額制。其四，追蹤使用率並自動標示長期未被使用者。其五，指定明確的擁有者（第 32 章），使「檢視是否仍需要」有人負責。核心原則：對抗自然累積，必須靠制度而非自律。

第 38 章

專案實作：

從資料到決策的完整流程

Capstone Project: From Data to Decision

機器人叫獸（李世淵） 編著

叫獸開講

我們走到最後一章了。

前面三十七章，我帶你走過了一條很長的路：從第 1 章那個史丹佛的車庫，到分散式系統的原理、運算的框架、探勘的演算法、生成式 AI、串流、雲端、治理，一直到上一章的視覺化。

但我要誠實地說一件事：這些你都「讀過」了，卻不見得「會」。就像看完一百部游泳教學影片，不代表你下水不會沉。

我教書這些年，最確定的一件事就是——真正的學會，是做出來的，不是讀出來的。而且做的過程中你會發現，那些在書上讀起來理所當然的東西，實際動手時處處是坑：資料格式不對、欄位意義不明、模型跑得動但沒有用、系統做好了但沒人想用。

所以這最後一章，我不打算再介紹任何新技術。我要給你的是一套「怎麼把這些東西組裝起來」的方法，以及一個完整的專案架構，讓你能親手走一次從資料到決策的全程。這一章也是給那些即將做專題、實習或進入業界的你們，一份實務上的行動指南。

學習目標

研讀完本章之後，你應該能夠：

1. 以問題導向的方式界定一個資料專案的範圍與目標。
2. 評估資料的可得性與品質，並據以調整專案規劃。
3. 設計符合需求的技術架構，並說明各項選型的理由。
4. 規劃專案的階段與里程碑，並辨識關鍵風險。
5. 以價值評估判斷專案是否值得投入。
6. 完成一個從資料到決策的完整專案並適當呈現成果。

核心概念

核心概念	說明
問題導向	從業務問題出發規劃專案，而非從技術或資料出發。
可行性評估	在投入前確認資料、技術與組織條件是否具備。
最簡可行版本	先做出能端到端運作的最小系統，再逐步改善。
里程碑	可驗證的階段性成果，用於檢視進度與及早發現偏差。
價值評估	以量化的方式估算專案的效益與投入是否相稱。
交付與移交	使成果能被他人接手並持續運作的完整交付。
回顧	專案結束後檢討得失，把經驗轉為組織的知識。

表 38-1 本章核心概念一覽

38.1 從問題出發

38.1.1 最常見的起點錯誤

第 36 章我提過，專案最常見的失敗模式是「從技術出發而非從問題出發」。這裡我要把它講得更具體。

典型的錯誤起點包括：「我們也來導入 AI」「這批資料放著也是放著，來分析看看」「聽說某某技術很紅，我們試試」。這些起點的共同問題是——它們沒有定義「成功長什麼樣子」。

而正確的起點應該是一個具體的問題，且能回答三件事：誰有這個問題？現在他怎麼處理？若解決了，會有什麼具體的改變？

以智慧製造為例，「導入 AI 提升良率」不是好的起點；「三號產線的某類缺陷目前靠人工目檢，漏檢率約 8%，每月造成約二十萬元的客訴賠償」才是——因為它明確、可量測、且能判斷成功與否。

38.1.2 界定範圍

有了問題之後，下一步是把範圍縮到「小到能完成、大到有價值」。

初學者最常犯的錯誤是範圍過大。「打造智慧工廠」聽起來很有企圖心，但它無法在一個學期或一個專案週期內完成，也無法判斷是否成功。相對地，「把三號線某類缺陷的漏檢率從 8% 降到 3% 以下」是可完成、可驗證的。

界定範圍時，有幾個問題值得反覆確認：這個問題的邊界在哪（哪些包含、哪些明確排除）？成功的標準是什麼（用什麼數字判斷）？以及——若時間只剩一半，我會先做哪一部分？

最後那個問題特別有用，因為它逼你排出優先序。而在真實的專案中，時間永遠會不夠。

38.1.3 利害關係人

還有一個技術背景的人常忽略的步驟：確認誰會受這個專案影響。

至少要問清楚三種人：使用者（誰會實際操作這個系統）、決策者（誰決定要不要採用）、以及資料的提供者（誰擁有你需要的資料，他願意給嗎）。

第三項尤其關鍵，且經常是專案的最大阻礙。你需要的資料可能在另一個部門手上，而他們沒有義務配合你——這時候技術能力完全幫不上忙，靠的是溝通與說明價值。我看過不少專案卡在這裡好幾個月。

38.2 資料的可行性評估

38.2.1 四個必問的問題

在投入大量時間之前，務必先確認資料是否足以支撐這個專案。以下四個問題應該在專案初期就取得答案。

問題	要確認什麼	若答案不理想
資料存在嗎	所需的欄位是否真的被記錄下來	可能需要先加裝感測或改造系統
拿得到嗎	權限、格式、介面、以及對方是否配合	先解決取得問題，而非先寫程式
品質如何	完整性、正確性、時效性（第 32 章）	評估清理成本，可能佔專案大半時間

問題	要確認什麼	若答案不理想
量夠嗎	尤其是機器學習所需的標註樣本	考慮改用規則式方法或先蒐集資料

表 38-2 資料可行性的四個必問問題

我要強調第四項。學生專題最常見的挫敗，是想做異常偵測卻只有五筆異常樣本——而這在良好的產線上是常態（第 35 章提過的標註稀缺）。與其硬做，不如及早調整方向：改用不需大量標註的方法、或把專案的第一階段定位為「建立標註流程與累積資料」。

38.2.2 先看資料再承諾

我給學生的一個實務建議是：在承諾任何具體成果之前，先花時間實際看過資料。

「看過」不是指看欄位清單，而是真的把資料拉出來、畫幾張圖、算幾個統計量、檢查空值與異常值。這個過程通常只需一兩天，卻能避免數週後才發現「這份資料根本無法回答那個問題」。

典型會在這個階段被發現的問題包括：關鍵欄位大量空值、時間戳不可靠、不同來源的定義不一致（第 32 章的營收爭議）、或是資料的時間範圍不足以涵蓋要分析的週期。

這也是為什麼第 32 章的資料目錄與中繼資料如此重要——它們能讓這個評估從「兩天」縮短到「兩小時」。

38.3 架構設計與技術選型

38.3.1 從需求推導架構

架構設計的正确順序，是從需求推導技術，而非相反。以下是幾個關鍵的判斷點，它們各自對應本書的某一章。

判斷點	要問的問題	對應章節
批次或串流	結果需要多即時？分鐘級或秒級？	第參篇、第陸篇
邊緣或雲端	延遲要求？頻寬？資料能否外傳？	第 27、35 章
自建或託管	團隊的維運能力？用量多大？	第 27、31 章
儲存架構	資料型態？查詢模式？保留多久？	第 4、30 章
模型的複雜度	規則能解決嗎？有多少標註資料？	第 20、21 章

表 38-3 架構設計的關鍵判斷點

我要提醒的是：每一個判斷都應該有明確的理由，而且理由必須來自需求，不是來自「這個技術比較新」。當你日後要向他人說明架構時，「因為需求是 X，所以我選了 Y」是有說服力的；「因為大家都在用」則不是。

38.3.2 從最簡可行版本開始

這是我認為對學生最有用的一條建議：先做出一個能「端到端運作」的最簡版本，再逐步改善各環節。

具體而言，與其花三週把資料前處理做到完美、再花三週訓練最佳模型、最後才發現整合不起來，不如第一週就做出一條「雖然粗糙但能從頭跑到尾」的管線——即使模型只是簡單的規則、視覺化只是一張表格。

這樣做的好處有三：其一，你會及早發現整合的問題（往往比演算法本身更棘手）；其二，你隨時都有一個「能展示的東西」，這在專案匯報時極為重要；其三，你能看出哪一環才是真正的瓶頸，把後續的時間投在對的地方。

這個原則你在第 22、31、36 章都見過——複雜度應該被需求逼出來。而在專案執行上，它的形式就是「先跑通，再優化」。

38.3.3 技術選型的務實建議

- **選你熟悉的：**在有限的時間內，用熟悉的工具通常勝過學新的。除非新工具能帶來數量級的差異。
- **選生態成熟的：**遇到問題時能找到解答、能找到人問，這個價值往往被低估。
- **避免不必要的分散式：**若資料在單機上跑得動，就不要用叢集（第 20 章的「先優化再分散」）。
- **保留退路：**採用開放格式與標準介面，使日後更換元件的成本可控（第 27 章）。

38.4 專案的階段與里程碑

38.4.1 六個階段

一個資料專案大致可分為六個階段。這裡我也標出各階段常見的時間佔比——它可能會顛覆你的預期。

階段	主要工作	時間佔比	里程碑
問題界定	釐清問題、範圍與成功標準	5%	一頁的專案說明書
資料評估	確認可得性與品質	10%	可行性報告與資料樣本
資料工程	擷取、清理、整合、建立管線	40%	可重複執行的資料管線
分析建模	特徵工程、模型訓練與評估	20%	達到目標指標的模型
整合交付	系統整合、介面、部署	20%	可運作的端到端系統
驗證回顧	實際驗證、文件、移交	5%	驗收報告與交接文件

表 38-4 資料專案的六個階段與時間分配

請特別注意「資料工程」佔了 40%——這是實務上最常被低估的一項。多數人以為專案的重點是建模，但實際上，把資料弄出來、弄乾淨、弄成可用的形式，往往才是最耗時的部分。

這也解釋了為什麼本書用了那麼多篇幅談儲存、管線與治理——那些不是配角，而是主戲。

38.4.2 里程碑的設計

好的里程碑應該具備三個特性：可驗證（有明確的判準，而非「大致完成」）、有產出（有具體的成果物）、以及夠早（能及早發現偏差）。

我特別建議在專案的前四分之一，就要有一個「端到端跑通」的里程碑——即使每一環都很粗糙。這個里程碑的價值不在於成果的品質，而在於它會逼出所有的整合問題。

相對地，一個常見的錯誤設計是把整合放在最後：「先各自做好自己的部分，最後再串起來」。這幾乎必然導致專案末期的災難——因為串接的問題往往比預期的多。

38.4.3 風險與應變

專案規劃時應該預先辨識風險，並想好若發生了要怎麼辦。資料專案的常見風險包括：

- **資料拿不到或延遲：**應變：先以樣本或模擬資料進行開發，並及早升級溝通層級。
- **資料品質比預期差：**應變：把清理納入正式的工作項目而非「順便做」，必要時縮減範圍。
- **模型達不到目標：**應變：預先定義「最低可接受」的水準，並準備規則式的備案。
- **需求變動：**應變：以最簡可行版本及早展示，讓需求在早期而非後期浮現。
- **使用者不採用：**應變：從一開始就讓使用者參與設計（第 36 章的失敗模式）。

叫獸提醒

我想談談「時間估計」這件事，因為它是學生專題最常失控的部分。有一個相當可靠的經驗法則：把你憑直覺估出的時間乘以二到三倍，才接近實際所需。這不是因為你能力不足，而是因為直覺估計時，你只想到了「順利的情況」，而忽略了那些必然會發生的意外——資料格式與文件不符、某個套件在你的環境裝不起來、某位關鍵人物請假兩週、以及花了三天才發現是自己看錯了欄位。這些事情不是例外，它們就是專案的日常。所以做規劃時，請務必預留餘裕，並優先完成核心的部分——這樣即使時間不夠，你交出的仍是一個「完整但簡單」的系統，而非「精緻但殘缺」的半成品。

38.5 完整專案範例的架構

38.5.1 情境設定

為了讓前面的原則具體化，我們設想一個完整的專案，並看它如何串起本書各章。

情境：某工廠的三號產線有一項瓶頸——關鍵設備的非計畫停機。每年約發生四十小時，每小時的損失（停產加緊急維修）約十八萬元。目前僅靠定期保養與人員經驗判斷，缺乏預警能力。

專案目標：建立設備異常的預警系統，在故障發生前數小時發出告警，使維護能改為計畫性進行。
成功標準：能提前偵測出至少五成的非計畫停機事件，且誤報率控制在可接受範圍。

38.5.2 架構與各章的對應

環節	設計	對應章節
資料來源	設備既有的振動、電流、溫度感測，加裝必要的補充感測	第 35 章
邊緣處理	以邊緣裝置做特徵萃取與初步異常判斷	第 35 章
資料傳輸	以訊息佇列可靠傳送，斷線時本地暫存	第 25 章
即時分析	串流視窗計算指標，超出範圍即告警	第 24、26 章
資料儲存	以分層架構保存原始與整備資料	第 5、30 章
模型訓練	以歷史資料訓練異常偵測模型	第 20 章
部署與監控	模型部署至邊緣，並監控其表現與漂移	第 34 章

環節	設計	對應章節
視覺化	維護人員的告警介面與主管的趨勢儀表板	第 37 章
治理與安全	資料的品質檢核、權限控管與稽核	第 32、33 章

表 38-5 專案架構與本書各章的對應

我希望你看這張表時能感受到：本書並不是三十八個獨立的主題，而是一個完整系統的各個環節。真正的能力，是知道「在什麼情況下需要哪一環，以及它們如何銜接」。

38.5.3 分階段的推進

這個專案不應該一次做完，而應分階段推進——每一階段都能獨立產生價值。

- **第一階段：看得見：**先把資料匯集起來並做基本的視覺化，讓維護人員能看到設備的即時狀態與歷史趨勢。這一階段不需要任何模型，卻已經有實用價值。
- **第二階段：有規則：**與資深技師合作，把他們的經驗轉為明確的規則（如「振動超過某值且持續十分鐘」），建立第一版告警。這能立即產生效益，也累積了寶貴的標註資料。
- **第三階段：有模型：**以前兩階段累積的資料訓練模型，逐步取代或補強規則。此時已有基準可以比較，能明確判斷模型是否真的更好。
- **第四階段：能自我維持：**建立監控與重訓機制（第 34 章），使系統能長期運作而不衰退。

這個分階段的設計有一個重要的優點：即使專案在任何一階段停止，前面完成的部分仍有價值。相對地，「一次做完整套」的規劃若中途受阻，往往什麼都留不下。

38.6 交付、驗證與回顧

38.6.1 什麼叫做「做完了」

這是一個看似簡單卻常被誤解的問題。「模型訓練完成」不叫做完，「程式能跑」也不叫做完。

一個資料專案真正完成，至少應該包含五樣東西：能運作的系統、可重現的程式碼與環境（第 23、28 章）、說明文件（讓他人能接手）、驗證的證據（實際達成了什麼指標）、以及維運的安排（誰負責、如何監控）。

最後一項最常被遺漏。一個沒有指定維運負責人的系統，通常會在數個月內悄悄失效——第 34 章談的模型衰退、第 32 章談的管線斷裂，都是這樣發生的。

38.6.2 驗證的設計

驗證不應該只在最後做，而應該在專案初期就設計好。

關鍵是要在動手之前就確定「用什麼證明它有效」。是離線的測試集指標？還是實際運行一段時間的表現？由誰認定？達到什麼水準算成功？

第 19 章我們談過離線指標的侷限——RMSE 低不代表推薦好。同樣地，模型的準確率高不代表這個系統有用。真正的驗證應該回到最初的問題：漏檢率降低了嗎？停機時間減少了嗎？

實務上常見的做法是「並行運行」：新系統與既有做法同時運作一段時間，比較兩者的實際表現。這既能取得可信的驗證，也讓使用者有時間建立信任。

38.6.3 回顧與知識的累積

專案結束後的回顧，是最容易被跳過、卻價值最高的一步。

值得記錄的包括：哪些事情比預期困難（下次好估時間）、哪些決策事後看是對的或錯的（累積判斷力）、以及哪些是「早知道就好」的教訓。

我特別建議記錄「時間的實際分配」，並與表 38-4 的預期比較。多數人做完第一個專案後會發現：資料工程佔的時間遠比想像的多，而建模遠比想像的少。這個認知會改變你下一次的規劃方式。

而從組織的角度，回顧的價值在於「把個人的經驗轉為團隊的知識」。若每個專案的教訓都只留在當事人腦中，組織就會不斷重複同樣的錯誤——這與第 32 章談的中繼資料是同一個道理：沒有被記錄下來的知識，等於不存在。

38.7 結語：從這裡開始

38.7.1 全書的三條主線

走到最後，我想把三十八章收攏成三條主線，這是最希望你帶走的東西。

第一條是「規模改變一切」。當資料大到單機處理不了，幾乎所有事情都要重新思考：儲存要分散（第 5 章）、運算要平行（第參篇）、精確要讓位給近似（第 14、24 章）、而機器一定會壞（第 3 章）。這是本書最根本的前提。

第二條是「取捨無所不在」。CAP 的一致性與可用性、批次與串流、精確與即時、雲端與地端、彈性與可靠——本書幾乎每一章都在談某種取捨。而工程判斷的核心，正是辨識「這個情境的代價結構是什麼」，而非套用通則。

第三條是「原理會遷移，工具不會」。你在本書中一再看到同一組原理以不同面貌出現：分區、複製、冪等、不可變、以摘要換取可行。技術每幾年就換一輪，但這些原理不會。理解它們，你面對任何新技術都能快速上手。

38.7.2 給即將出發的你

最後，我想說幾句給正要進入這個領域的你們。

第一，基本功比新技術重要。第 34 章我說過，那些最後真正做出成果的人，幾乎都是在版本控制、測試、記錄這些「無聊」的事情上紮實的人——因為基本功決定了你每一次嘗試的成本，而成本決定了你能嘗試幾次。

第二，誠實比漂亮重要。無論是呈現不確定性（第 37 章）、承認模型的限制（第 21、23 章）還是把疑慮說出來（第 33 章），誠實在短期看來會削弱你的說服力，長期卻是唯一能累積信任的方式。

第三，也是我最想說的——你們將擁有的能力，會影響到許多看不見的人。推薦系統會塑造他們看到的世界（第 19 章）、模型會影響對他們的判斷（第 23 章）、資料會揭露他們未曾同意揭露的事（第 33 章）。技術能力帶來的不只是價值，也帶來責任。

叫獸提醒

這是本書最後一段話了，我想說得直白一點。你們可能會覺得，這本書講的東西很多、很雜、也很難全部記住——這是正常的，我也不指望你們記住。真正重要的是：當你日後遇到某個問題時，你能想起「這件事書上好像談過」，然後知道該回頭翻哪一章、該用什麼角度思考。這

就夠了。知識的價值不在於背下來，而在於它改變了你看問題的方式。而最後我想請你們記得的，是這本書從第一章到現在反覆出現的那句話——技術會過時，原理會留下。當你們十年後回頭看，這本書提到的工具可能一個都不在了，但那些關於取捨、關於責任、關於「先想清楚問題再動手」的思考方式，會一直跟著你們。祝各位一路順風。

38.7.3 全書終

本章我們把全書串成了一個可執行的專案方法：從問題出發而非從技術出發（38.1）、在投入前評估資料的可行性（38.2）、從需求推導架構並從最簡可行版本開始（38.3）、規劃六個階段與可驗證的里程碑（38.4）、以一個完整範例對應本書各章（38.5）、以及交付、驗證與回顧的完整定義（38.6）。

到這裡，《大數據分析與雲端運算》三十八章全部結束。從第 1 章的 4V 與雲端概念，到第 38 章的專案實作，我們走過了資料的完整生命週期與現代資料系統的技术堆疊。

但真正的開始，是你闔上這本書之後。去找一個你真正在意的問題，把手弄髒，做出一個雖然簡陋但能運作的東西——那一刻你學到的，會比讀完這三十八章更多。

公式與流程：專案的價值評估

這一節要回答一個在專案開始前就該問的問題：這件事值得做嗎？

效益的估算

設某問題目前每年造成的損失為 L （元），而系統能改善的比例為 r （如異常偵測率），則每年可節省的金額為：

$$\text{年效益} = L \times r$$

式 38-1 專案的年效益估算

以 38.5.1 節的情境為例：每年四十小時的非計畫停機、每小時損失十八萬元，故 $L = 40 \times 180,000 = 7,200,000$ 元。

偵測率 r	年效益	說明	達成難度
30%	216 萬元	只抓到最明顯的異常	低
50%	360 萬元	本專案的目標水準	中
70%	504 萬元	需要更完整的感測與模型	高

表 38-6 不同偵測率的年效益（ $L = 720$ 萬元）

這張表的價值在於：它讓「目標訂多高」變成一個可以討論的決策，而非憑感覺。從 50% 提升到 70% 能多帶來 144 萬元的效益，但若為此需要投入 300 萬元的額外感測設備，就不值得。

投入與回收期

設專案的一次性建置成本為 C 、每年的維運成本為 M ，則第一年的總投入為 $C + M$ 。回收期（以月計）可估算為：

$$\text{回收期} = (C + M) / (L \times r) \times 12 \text{ (月)}$$

式 38-2 專案的回收期

以建置成本 120 萬元、年維運 30 萬元、偵測率 50% 為例：

回收期 = $(1,200,000 + 300,000) \div 3,600,000 \times 12 = 1,500,000 \div 3,600,000 \times 12 = 5$ 個月。

五個月的回收期在製造業屬於相當有吸引力的投資，這個數字比任何技術上的說明都更能說服決策者。

叫獸提醒

我要對這兩條公式提出三點誠實的提醒。第一，這些數字必然是估計，且通常偏樂觀——偵測率在實驗室與在現場往往有相當落差（第 34 章的模型衰退、第 36 章的現場環境）。做評估時應給出範圍而非單一數字，並明確說明假設。第二，維運成本 M 極常被低估。它不只是伺服器費用，還包括監控、重訓、故障處理的人力——第 34 章整章談的就是這件事。第三，也是最重要的：不是所有價值都能被量化。有些專案的效益在於「累積了資料與能力」「建立了團隊的經驗」「讓組織開始相信資料」，這些無法寫進公式，卻可能比帳面數字更有價值。所以請把這兩條公式當成「溝通的工具」與「思考的框架」，而非決策的唯一依據——這也呼應了第 37 章那句話：資料系統是支援決策，而非取代決策。

案例研討

案例一 範圍太大的專題

某組學生的專題題目是「智慧工廠整合平台」，計畫涵蓋設備監控、品質預測、排程最佳化與能耗分析四大模組。

學期過了一半，他們才發現連第一個模組的資料都還沒拿到——因為需要協調三個不同的資料來源，而其中一個單位遲遲未回應。最後交出的是四個都只做了三成的半成品。

檢討時我問他們：若一開始只做「設備監控」一項，會如何？他們的答案是——那應該可以完整做完，而且還有時間做得不錯。

這正是 38.1.2 節的範圍界定問題。「小到能完成、大到有價值」是一條難以拿捏但必須拿捏的線。而那個「若時間只剩一半，我會先做哪一部分」的提問，若在期初就認真回答，結果會完全不同。

案例二 第三週才發現的欄位問題

某團隊接到一項分析需求，看過欄位清單後認為資料充足，隨即投入開發。兩週後管線建置完成，開始實際處理資料時才發現：關鍵的那個欄位有超過六成是空值——因為該欄位是選填的，多數操作員不會填。

此時已投入兩週的工作，而整個分析的前提不成立。他們不得不重新界定問題，改用其他欄位間接推估。

若在第一天就依 38.2.2 節的建議「先把資料拉出來看一眼」，這個問題兩小時內就會被發現。兩小時對兩週——這是我認為投資報酬率最高的一項專案習慣。

這也再次凸顯第 32 章資料目錄的價值：若中繼資料中記錄了「此欄位為選填，填寫率約 38%」，這個評估連兩小時都不用。

案例三 最後一週才整合

某專案分三組進行：資料組負責管線、模型組負責演算法、前端組負責介面。三組各自進度良好，約定在最後兩週整合。

整合時問題全面爆發：資料組輸出的欄位名稱與模型組預期的不同、模型組假設輸入已標準化而資料組沒做、前端組需要的即時查詢在現有架構下要跑十幾秒。三個問題都不難解，但發現得太晚，最後兩週在慌亂中度過，交出的系統勉強能動。

這正是 38.4.2 節所警告的——把整合放在最後幾乎必然導致災難。若在第三週就強制做一次「端到端跑通」（即使模型只是隨機輸出、介面只有一個按鈕），這三個問題都會提早暴露，且當時修改的成本遠低於後期。

這個原則的價值在於：整合問題不會因為你晚點面對就消失，只會變得更貴——這與第 32 章的十倍成本階梯是同一個道理。

案例四 沒有人接手的系統

某位工程師花了半年建立了一套設備監控系統，運作良好、主管也很滿意。一年後他轉調其他部門。

再過三個月，系統開始出現問題：某個資料來源的格式變更導致管線靜默失敗、模型因未重訓而準確率下滑、而告警設定中的門檻早已不符現況。但沒有人知道該如何處理——程式碼沒有說明文件、環境設定只存在於他的筆記型電腦、也從未指定接手的負責人。

最終系統被停用，半年的投入化為烏有。

這正是 38.6.1 節「什麼叫做完了」的反面教材。系統能運作只是最低標準；沒有可重現的環境（第 23、28 章）、沒有文件、沒有維運負責人（第 32 章）、沒有監控與重訓機制（第 34 章），這個專案其實從來沒有真正完成。

實作活動

活動一 寫一頁專案說明書

請為你打算執行的專案（專題、實習或工作上的任務）撰寫一頁的說明書。

- 步驟 1.** 以一句話描述問題，並回答：誰有這個問題？現在他怎麼處理？解決了會有什麼具體改變？
- 步驟 2.** 明確界定範圍：包含什麼、明確排除什麼、成功的判準是什麼（要有數字）。
- 步驟 3.** 列出三種利害關係人（使用者、決策者、資料提供者）並註明具體是誰。
- 步驟 4.** 回答那個關鍵問題：若時間只剩一半，你會先做哪一部分？

活動二 做一次資料可行性評估

請在投入開發之前，先花兩小時完成資料的可行性評估。

- 步驟 1.** 依表 38-2 的四個問題逐一確認，並誠實記錄每一項的答案。
- 步驟 2.** 實際把資料拉出來：檢視前一百筆、計算各欄位的空值比例、畫出關鍵欄位的分布圖。
- 步驟 3.** 記錄你在這個過程中發現的意外（欄位意義不如預期、空值過多、時間範圍不足等）。
- 步驟 4.** 依評估結果調整你在活動一的專案範圍，並說明調整的理由。

活動三 完成一個端到端的專案

這是本書的總結性實作，建議以數週的時間完成。請執行你在活動一界定的專案，並遵守以下要求：第一，在專案的前四分之一必須完成一次「端到端跑通」，即使每一環都很粗糙。第二，依表 38-4 記錄你在各階段實際花費的時間，並在結束後與預期比較。第三，依式 38-1 與式 38-2 估算專案的效益與回收期（若情境不涉及金額，可改以節省的人時估算）。第四，交付時必須包含 38.6.1 節的五樣東西：能運作的系統、可重現的程式碼與環境、說明文件、驗證證據、以及維運安排。最後，撰寫一份不超過兩頁的回顧：哪些比預期困難？哪些決策事後看是錯的？下次你會怎麼做？

延伸閱讀

- **《Fundamentals of Data Engineering》**：Reis 與 Housley 著。以資料工程生命週期貫穿全書，與本章的專案方法高度呼應，是本書最佳的整體延伸閱讀物。
- **CRISP-DM 等資料探勘流程方法論**：說明從業務理解到部署的標準流程，可與表 38-4 的六階段對照參考。
- **《The Mythical Man-Month》**：Brooks 的軟體工程經典，關於時間估計、複雜度與溝通成本的洞見，至今仍完全適用。
- **各領域的實際案例研究**：閱讀他人的專案經驗（包含失敗的），是累積判斷力最快的方式之一。

本章小結

1. 專案應從問題出發而非從技術出發；好的起點必須能回答「誰有這個問題、現在怎麼處理、解決後有何具體改變」，並以可量測的方式定義成功。
2. 範圍應「小到能完成、大到有價值」；「若時間只剩一半，我會先做哪一部分」是界定範圍最有用的提問。並須及早確認三種利害關係人，其中資料提供者常是最大的阻礙。
3. 投入前必須評估資料的四個問題：存在嗎、拿得到嗎、品質如何、量夠嗎；而「先把資料拉出來看一眼」是投資報酬率最高的專案習慣——兩小時可能省下兩週。
4. 架構應從需求推導，每個選型都要有來自需求的理由；並應從「端到端跑通」的最簡可行版本開始，因為它能及早暴露整合問題、隨時有可展示的成果、並看出真正的瓶頸。
5. 專案分六階段，其中資料工程約佔 40% 的時間——這是最常被低估的一項，也解釋了為何本書用大量篇幅談儲存、管線與治理。
6. 里程碑應可驗證、有產出且夠早；特別應在專案前四分之一安排一次端到端整合，因為整合問題不會因為晚點面對而消失，只會變得更貴。
7. 時間估計應把直覺值乘以二到三倍，因為直覺只涵蓋了順利的情況；規劃時應預留餘裕並優先完成核心，寧可交出「完整但簡單」也不要「精緻但殘缺」。
8. 專案應分階段推進且每階段都能獨立產生價值（看得見、有規則、有模型、能自我維持），如此即使中途停止，前面的成果仍然有用。
9. 「做完了」至少包含五樣：能運作的系統、可重現的程式碼與環境、說明文件、驗證證據、維運安排；其中維運負責人最常被遺漏，而缺少它的系統通常在數個月內悄悄失效。
10. 年效益為 $L \times r$ （式 38-1）、回收期為 $(C+M)/(L \times r) \times 12$ （式 38-2）；但這些估計必然樂觀、維運成本常被低估、且並非所有價值都能量化——公式應作為溝通與思考的工具，而非決策的唯一依據。

習題

一、觀念題

1. 為什麼「我們也來導入 AI」不是一個好的專案起點？一個好的起點應該能回答哪三件事？
2. 請說明資料可行性評估的四個問題，並解釋為何「先把資料拉出來看一眼」是投資報酬率極高的習慣。
3. 為什麼應該從「端到端跑通」的最簡可行版本開始？請說明其三項好處。
4. 一個資料專案要具備哪五樣東西才算真正完成？請說明其中最常被遺漏的是哪一項及其後果。

二、計算題

1. 某產線每年因某類品質問題造成的損失為 480 萬元。（一）若系統能偵測出其中 45%，依式 38-1 計算年效益；（二）若建置成本 150 萬元、年維運 40 萬元，依式 38-2 計算回收期；（三）若偵測率只達 25%，回收期變為多少？此專案是否仍值得投入？
2. 某專案預估各階段的工作量為：問題界定 3 天、資料評估 6 天、資料工程 24 天、分析建模 12 天、整合交付 12 天、驗證回顧 3 天。（一）計算總天數與資料工程的佔比；（二）依 38.4.3 節的建議把估計乘以 2.5 倍，實際應預留多少天？（三）若可用時間只有 60 天，你會如何調整？

三、思考與討論

1. 本章指出「並非所有價值都能被量化」。請討論：一個帳面回收期很長、但你認為仍值得投入的專案，你會如何向決策者說明其價值？
2. 本書即將結束。請回顧全書，寫下你認為對自己最有啟發的三個觀念，並說明你會如何把它們應用在接下來的學習或工作上。

習題解答提示

以下提供解題方向與關鍵步驟，供你自我檢核。建議先自行作答，再對照本節內容；思考題並無標準答案，提示僅供參考。

一、觀念題

第 1 題

不好的原因：它沒有定義「成功長什麼樣子」——沒有具體問題、沒有可量測的目標、也無法判斷做完之後是否有價值。以技術為起點的專案，容易做出技術上正確但業務上無用的系統。好的起點應能回答三件事：誰有這個問題（明確的使用者或受影響者）、現在他怎麼處理（既有做法與其痛點，這也是比較的基準）、以及若解決了會有什麼具體改變（可量測的效益）。範例對照：「導入 AI 提升良率」不合格；「三號產線某類缺陷目前靠人工目檢，漏檢率約 8%，每月造成約二十萬元客訴賠償」才合格，因為它明確、可量測、且能判斷成功與否。

第 2 題

四個問題：資料存在嗎（所需欄位是否真的被記錄）、拿得到嗎（權限、格式、介面、對方是否配合）、品質如何（完整性、正確性、時效性）、量夠嗎（尤其機器學習所需的標註樣本）。「先看一眼」報酬率高的原因：這個動作通常只需一兩小時，卻能發現那些從欄位清單看不出來的致命問題——關鍵欄位大量空值、時間戳不可靠、不同來源定義不一致、時間範圍不足。若跳過這一步，這些問題往往要到投入數週、管線都建好之後才會浮現（如案例二的六成空值），屆時大量工作必須重做。兩小時對兩週，這是最划算的一項投資。可補充：若組織有完善的資料目錄（第 32 章），這個評估甚至不需兩小時。

第 3 題

三項好處：其一，及早發現整合問題——整合的困難往往比演算法本身更棘手（如案例三的欄位名稱不符、假設不一致、查詢過慢），而這些問題早期修改的成本遠低於後期。其二，隨時有可展示的成果——這在專案匯報、爭取資源與建立信任上極為重要，也讓需求方能及早給出回饋而非等到最後才發現方向錯誤。其三，能看出真正的瓶頸——跑通一次之後，你會清楚知道哪一環最慢、最不準或最脆弱，從而把後續有限的時間投在對的地方，而非憑想像優化。此原則與第 22、31、36 章的「複雜度應該被需求逼出來」一脈相承，在專案執行上的形式即「先跑通，再優化」。

第 4 題

五樣東西：能運作的系統、可重現的程式碼與環境（第 23、28 章）、說明文件（讓他人能接手）、驗證的證據（實際達成了什麼指標）、以及維運的安排（誰負責、如何監控、如何重訓）。最常被遺漏的是「維運安排」。後果：系統會在數個月內悄悄失效——上游資料格式變更導致管線靜默失敗（第 13 章的安靜失敗）、模型因未重訓而衰退（第 34 章）、告警門檻不符現況而被忽略。而因為沒有指定負責人、沒有文件、環境無法重現，接手者也無從修復，最終整個投入化為烏有（如案例四）。故「系統能運作」只是最低標準，而非完成的定義。

二、計算題

第 1 題

$L = 4,800,000$ 元、 $C = 1,500,000$ 元、 $M = 400,000$ 元。

- (1) $r = 45\%$ 時的年效益（式 38-1） $= 4,800,000 \times 0.45 = 2,160,000$ 元。
- (2) 回收期（式 38-2） $= (1,500,000 + 400,000) / 2,160,000 \times 12 = 1,900,000 / 2,160,000 \times 12 \approx 10.6$ 個月。
- (3) $r = 25\%$ 時：年效益 $= 4,800,000 \times 0.25 = 1,200,000$ 元；回收期 $= 1,900,000 / 1,200,000 \times 12 \approx 19$ 個月。

是否值得投入的判斷：19 個月的回收期雖較長，但仍在多數製造業可接受的投資期限內（通常以二至三年為界），故單就財務而言仍屬可行。但更周延的評估應納入幾點：其一，25% 是保守估計，實際上線後可透過持續改善提升。其二，第二年起不再有建置成本，僅需負擔 40 萬元維運，故第二年的淨效益達 80 萬元。其三，專案還會累積資料與團隊能力，這些無法計入公式。反之若偵測率的估計本身不可靠，則應先做小規模試點驗證再決定。

第 2 題

各階段：3、6、24、12、12、3 天。

- (1) 總天數 $= 3 + 6 + 24 + 12 + 12 + 3 = 60$ 天；資料工程佔比 $= 24 / 60 = 40\%$ （與表 38-4 的經驗值完全吻合）。
- (2) 乘以 2.5 倍後 $= 60 \times 2.5 = 150$ 天。
- (3) 若只有 60 天：直接執行原估計等於毫無餘裕，幾乎必然延誤。應縮減範圍至原規劃的四成左右（ $60 \div 150 = 0.4$ ），例如：減少資料來源的數量、以規則式方法取代模型（省下分析建模的多數時間）、簡化介面、或把驗證範圍縮小為單一產線。

調整的原則：應「縮減範圍」而非「壓縮各階段的時間」。後者的結果通常是每一環都做得粗糙、且整合階段被犧牲，最終交出精緻但殘缺的半成品；前者則能交出完整但簡單、可實際運作的系統。這正是 38.4.3 節叫獸提醒的核心建議，也呼應 38.5.3 節「分階段推進、每階段都能獨立產生價值」的設計。

三、思考與討論

第 1 題

可提出的論述方向：其一，把未量化的效益明確列出並盡量間接量化——如「累積了未來三個專案都能使用的資料基礎」可估算為省下的重複建置成本；「建立了團隊的能力」可估算為外包同等工作的費用。其二，呈現選擇權的價值——這個專案雖然本身回收期長，但它使組織具備了日後快速開展其他應用的能力，這在技術快速變動時特別有價值（呼應第 27 章「彈性的價值」）。其三，指出不做的代價——競爭對手已在進行、人才留不住、或問題會隨時間惡化。其四，以分階段降低風險——提議先投入小規模的第一階段（如 38.5.3 節的「看得見」），以較低的成本驗證可行性，再決定是否繼續，把一次大賭注拆成數次小決策。其五，誠實呈現不確定性（第 37 章）——給出範圍而非單一數字，並說明關鍵假設，這反而更能建立信任。核心立場：不應為了通過審核而美化數字，而應把「難以量化的價值」明確攤開，讓決策者在知情的情況下判斷。

第 2 題

此題無標準答案，重點在於能具體說明「觀念」與「如何應用」的連結。可能被提出的觀念包括：其一，「規模改變一切」——理解為何單機的直覺在大規模下會失效，日後遇到效能問題時會先問「這是規模問題還是實作問題」。其二，「取捨無所不在」——不再尋找「最好的技術」，而是問「這個情境的代價結構是什麼」（第 36 章）。其三，「原理會遷移，工具不會」——學習新技術時先找它對應的原理（分區、複製、冪等、不可變），上手速度會快得多。其四，「以可控的近似換取可行的計算」——面對算不完的問題時，先問「需要多精確」而非「如何算得更快」。其五，「彙總會掩蓋極端」——看任何指標時都追問分布與最差值。其六，「基本功決定每次嘗試的成本」——把版本控制、測試與記錄當成習慣而非負擔。其七，「技術能力帶來責任」——在設計會影響他人的系統時主動提出疑慮。評分時應著重於：是否舉出了具體的應用情境，而非只是複述觀念。

索引

依詞條首字排列；英文詞條在前，中文詞條依筆畫序在後。頁碼為該詞主要出現之處。

英文詞條

A-Priori.....	174、177、179、180、182
Avro.....	45
B-tree.....	42、46、47、49、51
BFR.....	184、188、189、193、195
Bloom Filter.....	291、292、295、297、298
CAP 定理.....	34、38
Catalyst.....	135、138、139、140、143
CURE.....	184、189、193、196
DataFrame.....	135、136、137、142、143
Delta Lake.....	376
DGIM.....	289、293
Docker.....	115、119

ESP32.....	436
etcd.....	87、91、356、363
Flink.....	9、317、323、324
GraphX.....	214、218
HDFS.....	54、55、56、60、62
HITS.....	209、212、213、215、218
Hudi.....	59、367、371
HyperLogLog.....	293、296、297
Hypervisor.....	341、342、350、351
Iceberg.....	59、367、371
Jetson.....	436、442、443、444
k-匿名.....	408、411、414
Kafka.....	9、302、303、304、311
Kubernetes.....	115、354、355、356、362
Lakehouse.....	9、367、371、372、376
LoRA.....	287
LSH.....	159、163、164、165、166
LSM-tree.....	46、47、49、50、51
MapReduce.....	99、100、103、106、107
MinHash.....	159、162、166、167、300
PageRank.....	211、212、213、217、218
Parquet.....	45、50、138、141、142
Paxos.....	87、90、91
PCA.....	197、198、199、204、205
PCY.....	174、175、180、292
Pregel.....	209、213、214、215、218
Raft.....	87、90、91、92、96
RAG.....	249、254、255、258、263
RDD.....	123、125、127、130、131
ReAct.....	262、263、264、270、271
ROS.....	448、449、450、457、458
SLAM.....	448、451、458
SON.....	43、44、61、137、176
Spark.....	124、126、128、131、143
SVD.....	197、199、200、201、205
Transformer.....	238、249、250、258、279
Tungsten.....	139
YARN.....	111、112、115、119、355
ZooKeeper.....	87、91、94

中文詞條

一致性雜湊.....	81、82、83、84、85
分區裁剪.....	374、377

水位線.....	318、321、322、323、326
水塘抽樣.....	289、291、297、298
失效安全.....	452、458
共識演算法.....	87、89、90、91、94
冷啟動.....	226、383、386、387、390
冷啟動問題.....	222
利特爾法則.....	24、27
告警疲勞.....	152、154、155
事件時間.....	315、318、323、324、325
事件溯源.....	302、306、307、311
兩階段提交.....	87、88、89、95
協同過濾.....	222、224、225、231、233
協作型機器人.....	448、452、458
法定人數.....	65、70、71、72、93
注意力機制.....	249、250、259
物件儲存.....	8、58、59、61、62
物聯網.....	434、435、443、444
阿姆達爾定律.....	12
宣告式.....	136、143、354、362、364
述詞下推.....	138、140、141、143、144
容器編排.....	354
差分隱私.....	412、417
特徵儲存.....	421、423、428
矩陣分解.....	197、205、222、226、231
剪枝.....	174、177、178、179、181
參數伺服器.....	236、239、240、245
處理時間.....	24、315、321、324、325
責任共擔.....	417
最小權限原則.....	408
單調性原理.....	171、177、178、180、181
提示工程.....	249、253
湧現能力.....	258
無伺服器.....	381、382、386、387、388
虛擬化.....	341、342、350
詞元.....	250、254、259、260、279
量化.....	143、249、255、256、489
微批次.....	315、316、317、324
微服務.....	381、384、388、390
感測融合.....	448、450、458
概念漂移.....	421、425、430、431
滑動視窗.....	289、293
資料平行.....	236、240、245、246
資料目錄.....	23、375、394、396、403

資料血緣.....	394、396
資料沼澤.....	367、369
資料契約.....	398、403
資料倉儲.....	25、359、367、368、369
資料湖.....	58、59、61、367、370
資料漂移.....	421、425、429、430、431
資料墨水比.....	462、465、468、472
蒸餾.....	255、437、444
維度災難.....	185、190、192、198
綱要演進.....	45、50
寫時複製.....	344
數位分身.....	453、458
模型卡.....	275、279、284、285、287
模型平行.....	236、240、241、245
調和迴圈.....	354、356、362、364
冪等性.....	35、37、150、155、156
樹莓派.....	436、444
邊緣運算.....	434、436、437、444、445
欄式儲存.....	42、45、46、48、50